# How to Find It

A Discovery and Modeling Manual for Competitive Programming (Sections 1–57)

# Contents


1. How to Use This Manual
2. Section 1 — Complexity, I/O, and Constraints
3. Section 2 — Simulation and Brute Force
4. Section 3 — Counting, Frequencies, and Maps
5. Section 4 — Prefix Sums and Difference Arrays
6. Section 5 — Sorting as a Tool
7. Section 6 — Two Pointers and Sliding Windows
8. Section 8 — Binary Search on the Answer
9. Section 9 — Greedy I: Local Choices and Exchange Arguments
10. Section 10 — Greedy II: Invariants and Constructive Problems
11. Section 11 — Intervals and Sweep Line
12. Section 12 — Stacks, Queues, and Deques
13. Section 13 — Offline Processing
14. Section 15 — BFS and Unweighted Shortest Paths
15. Section 16 — DFS, Components, and Cycles
16. Section 17 — Topological Sorting and DAG DP
17. Section 18 — DSU and Minimum Spanning Trees
18. Section 19 — Dijkstra and Weighted Modeling
19. Section 20 — 0-1 BFS and Shortest Path Variants
20. Section 22 — One-Dimensional DP
21. Section 23 — Knapsack and Subset DP
22. Section 24 — Grid DP
23. Section 25 — Interval DP
24. Section 26 — Tree DP Basics
25. Section 27 — Bitmask DP
26. Section 29 — Modular Arithmetic
27. Section 30 — GCD, LCM, and Diophantine Equations
28. Section 31 — Sieve, Primes, and Factorization
29. Section 32 — Combinatorics under Modulo
30. Section 33 — Inclusion-Exclusion
31. Section 34 — Probability and Expected Value
32. Section 36 — Fenwick Trees
33. Section 37 — Segment Trees
34. Section 38 — Static Queries and Binary Lifting
35. Section 39 — Heaps and Priority Queues
36. Section 41 — Euler Tour, LCA, and Binary Lifting
37. Section 43 — String Algorithms I: Prefix Function, KMP, and Z-Function
38. Section 44 — String Algorithms II: Rolling Hash, Tries, and Suffix Arrays

39. Section 45 — Flows and Matchings
40. Section 46 — Rerooting and Advanced Tree DP
41. Section 47 — Meet in the Middle
42. Section 48 — DP Optimization: Monotonic Queue and Divide & Conquer
43. Section 49 — Convex Hull Trick and Li Chao Trees
44. Section 51 — Invariants in Hard Problems
45. Section 52 — Game Theory
46. Section 53 — Advanced Combinatorics
47. Section 54 — Geometry
48. Section 55 — Randomization and Hashing
49. Section 56 — Hard Greedy and Matroids Intro
50. Section 57 — Reductions and Modeling
51. Synthesis — A Universal Modeling Checklist

# How to Use This Manual

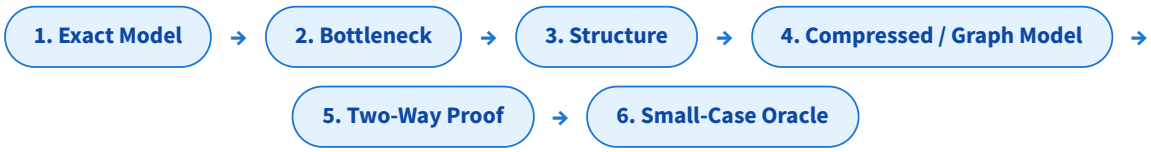
This manual is about **discovering models and algorithms from scratch**, not recognizing named techniques after they have been revealed. Every worked problem in this volume is original and independent of the course exercises. It trains the exact modeling habits required across Sections 1 through 57 without spoiling any problem in the practice tracks. Mixed-contest checkpoints (Sections 7, 14, 21, 28, 35, 40, 42, and 50) are omitted so that every chapter focuses on a distinct family of modeling concepts.

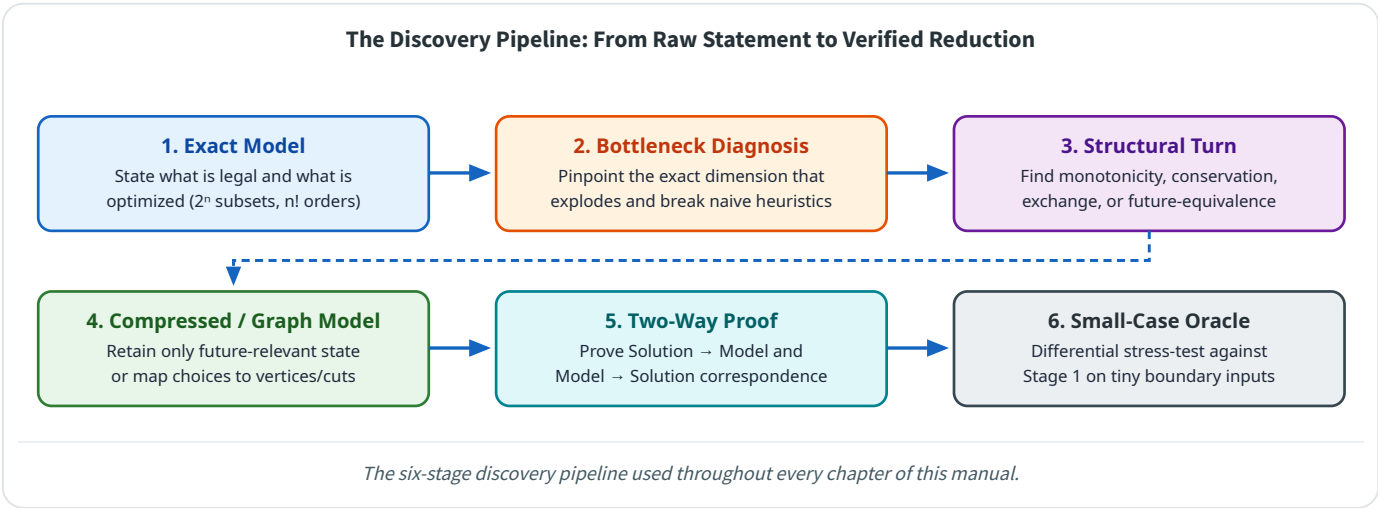
Each section follows a strict two-part pedagogical structure:

1. **The Discovery & Modeling Approach:** Before examining any specific problem, we establish a **Decision Compass** for the topic—how to recognize the underlying structure when a statement hides it behind domain vocabulary, how to choose between competing sub-models, what an elementary state or transition represents, and why tempting greedy or local shortcuts fail.
2. **Progressive Worked Problems:** We then apply that compass to two or three original problems that span the section's core concepts, constructing each solution step by step from brute-force specification to verified model.

## The Six-Stage Discovery Loop

Whenever you are stuck on an unfamiliar problem, resist the urge to guess an algorithm name from the constraints alone. Instead, walk through the following six stages in order:




*The six-stage discovery pipeline used throughout every chapter of this manual.*

1. **Exact Model (Start with something obviously correct):** Ignore time limits for sixty seconds. Write down the mathematical space of valid configurations: all $2^n$ subsets, all $n!$ permutations, all walk histories, or the direct exponential recurrence. This prevents solving a misread problem.
2. **Bottleneck (Diagnose why the exact model is too large, and test naive shortcuts):** Name the specific dimension causing the explosion ($2^n$ subsets, an $O(n)$ inner scan per query, or overlapping subtrees), and construct a tiny counterexample (3 to 5 elements) that breaks the most tempting naive greedy rule.
3. **The Turn (Discover the structure inside the bottleneck):** Ask what two partial histories can safely forget about the past, how an algebraic rearrangement decouples two indices $(i, j)$, whether the feasibility predicate is monotone, or whether a conservation/cut duality applies.
4. **Compressed or Graph Model (Build the representation progressively):** Merge histories that every future continuation treats identically, or map binary decisions, capacities, and transitions into a concrete graph or data structure.
5. **Two-Way Proof (Justify both directions):** Every valid reduction requires proving two statements:
   - *Solution* $\implies$ *Model:* Every legal configuration of the original problem maps to a valid state/path/cut with the same objective value.

- *Model* $\Longrightarrow$ *Solution:* Every optimal state/path/cut produced by the algorithm can be decoded back into a legal configuration of the original problem.

6. **Small-Case Oracle:** The slow exact model from Stage 1 is not wasted work—implemented on inputs of size $n \leq 8$, it serves as an automated differential oracle to catch missing transitions, off-by-one boundaries, or false state compressions before submission.

---

# Section 1 — Complexity, I/O, and Constraints

## The Discovery & Modeling Approach

Before designing any data structure or recurrence, your first obligation is to **translate the problem's constraints into an operation and memory budget**. In contest problems, the constraints are not an afterthought—they are the sharpest hint about the intended asymptotic class, and they expose silent traps before a single line of algorithmic code is written.

When analyzing a statement's operational envelope, apply three structural lenses:

1. **Worst-Case vs. Amortized Aggregate Bounds ( $\sum N$ vs. $T \cdot \max N$):** Many multi-testcase problems state $T \le 10^5$ and $N \le 2 \cdot 10^5$, which superficially looks like $T \cdot N = 2 \cdot 10^{10}$ operations. The critical clause is $\sum N \le 2 \cdot 10^5$. This allows $O(N)$ or $O(N \log N)$ work *proportional to the current test's* $N$, but strictly forbids $O(\max N)$ global array resets (`memset(vis, 0, sizeof(vis))`) inside the testcase loop.
2. **Value-Range & Intermediate Overflow Audit:** Even if inputs fit in 32-bit signed integers ($|a_i| \le 10^9$) and the final answer is small or reduced modulo $M$, **intermediate accumulations** (prefix sums, binary search midpoints $lo + hi$, products $a \cdot b \bmod M$, or squared Euclidean distances) routinely reach $10^{18}$ and require 64-bit integers (`long long` in C++).
3. **I/O Bandwidth vs. Algorithmic Work:** When reading or printing more than $10^5$ tokens, unbuffered stream synchronization (`std::endl` flushing, un-untied `cin/cout`, or line-by-line Python `input()`) can exceed the time limit before the algorithm even runs.

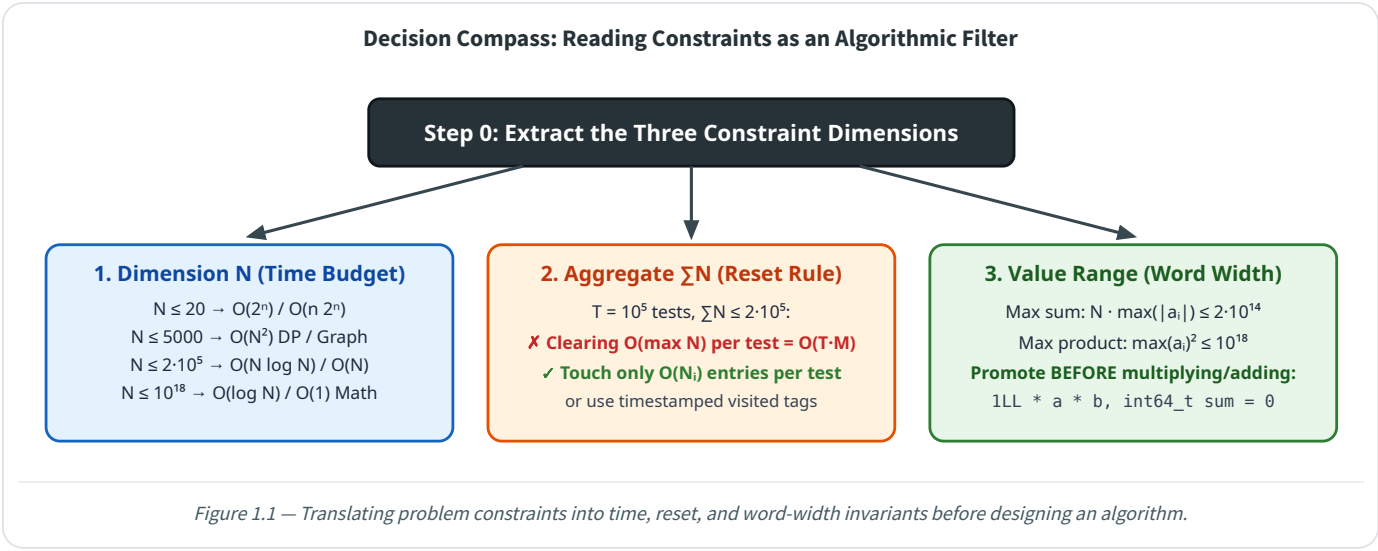



*Figure 1.1 — Translating problem constraints into time, reset, and word-width invariants before designing an algorithm.*

| Constraint Signal in Statement | Permitted Complexity Target | Classic Hidden Trap |
|---|---|---|
| $N \le 20$ | $O(2^N)$ or $O(N \cdot 2^N)$ | Recursion overhead without pruning; using `pow(2, n)` in floating point instead of `1 << n`. |
| $N \le 400$ | $O(N^3)$ | High constant factors inside the innermost loop (e.g., hash map lookups instead of array indexing). |
| $N \le 5000$ | $O(N^2)$ | Allocating $O(N^2)$ 64-bit integers when memory is 64 MB ($5000^2 \times 8 \approx 200$ MB). |
| $T \le 10^5$, $\sum N \le 2 \cdot 10^5$ | $O(\sum N \log N)$ | Filling a fixed-size global array `fill(cnt, cnt + MAXV, 0)` on every testcase. |
| Values $\lvert A_i \rvert \le 10^9$ | 64-bit arithmetic (`long long`) | Computing `int x = a * b;` before casting to `long long`, overflowing 32-bit signed arithmetic first. |

## Worked Problem 1.1: Telemetry Drift Audit

**Problem.** A ground station processes $T$ independent telemetry bursts ($1 \le T \le 10^5$). Burst $k$ consists of $n_k$ signed voltage deltas $d_1, d_2, ..., d_{n_k}$ ($|d_i| \le 10^9$) starting from initial voltage $V_0 = 0$, and a safety threshold $L$ ($1 \le L \le 10^{15}$). After each delta $d_i$, the running voltage becomes $V_i = V_{i-1} + d_i$. If $|V_i| > L$ at any step, the burst immediately aborts at index $i$ and ignores the remaining deltas of that burst. For each burst, output the abort index (or `OK` together with the final voltage $V_{n_k}$). Across all bursts, $\sum_{k=1}^{T} n_k \le 5 \cdot 10^5$.

> **Pause & Reflect:** What are the two ways a seemingly linear scan can fail here—one when a burst aborts early in the middle of reading input, and one when accumulating $V_i$?

### Step 1 — Exact Model & Bottleneck

For a single burst, the exact specification updates $V_i = V_{i-1} + d_i$ for $i = 1, ..., n_k$ and checks $|V_i| > L$. Since $\sum n_k \le 5 \cdot 10^5$, a true $O(n_k)$ scan per burst performs at most $5 \cdot 10^5$ additions overall, which easily fits within 1 second if I/O is buffered.

### Step 2 — Tempting Mistakes & Counterexample

1. **Breaking out of the input loop early:** Suppose burst 1 has $n_1 = 4$, $L = 10$, and deltas `15 2 3 4`, followed by burst 2 with $n_2 = 2$, $L = 5$. If your loop reads $d_1 = 15$, detects $|15| > 10$, prints `1`, and immediately `break`s to the next testcase without consuming `2 3 4`, the next testcase will read `n_2 = 2` and `L = 3` from burst 1's leftover numbers!
2. **32-bit accumulator overflow:** Even if $L \le 10^9$, $V_{i-1}$ can be $9 \cdot 10^8$ and $d_i = 9 \cdot 10^8$; in 32-bit signed `int`, $V_{i-1} + d_i$ wraps around to a negative number, completely missing the threshold violation.

### Step 3 — The Turn: Decoupling Stream Consumption from State Transitions

Always consume the exact declared token count $n_k$ for each burst (or read the entire burst array first), while freezing the state transition once the first violation index `abort_idx` is recorded:

$$\begin{aligned} &\text{State at step } i: \quad (\mathrm{V} \in \mathbb{Z}_{64},\ \text{abort_idx} \in \{0, 1, ..., n_k\}) \\ &\text{If abort_idx} = 0: \quad V \leftarrow V + d_i; \quad \text{if } |V| > L \text{ then abort_idx} \leftarrow i \end{aligned}$$

### Step 4 — Progressive Construction & Visual Trace

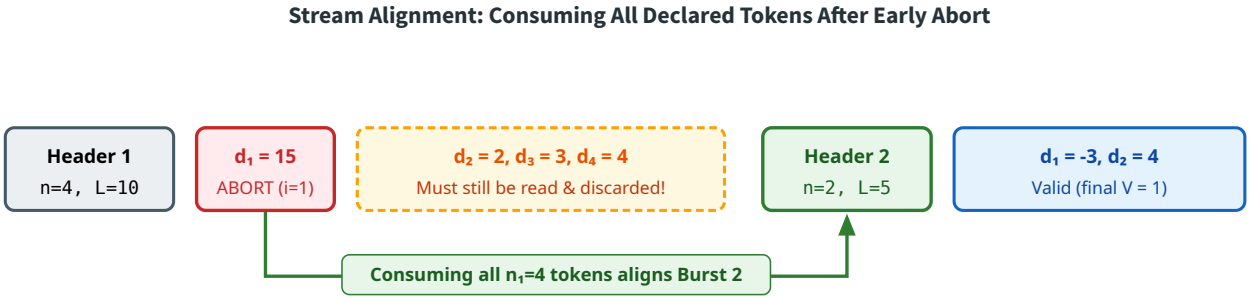



*Figure 1.2 — Why input parsing must remain synchronized with the input grammar even when logical evaluation terminates early.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Correctness:** Because `abort_idx` is latched on the first index $i$ where $|V_i| > L$ and never overwritten afterward, the reported state matches the exact prefix truncation while leaving the input stream positioned precisely at the start of burst $k + 1$.
- **Oracle:** Test with $n_1 = 0$ (if permitted), $L = 0$, $d_i = \pm 10^9$, and two consecutive bursts where the first aborts on $i = 1$.

## Worked Problem 1.2: Sparse Channel Reset Audit

**Problem.** Across $T \leq 10^5$ test cases, you receive $n_k$ packet channel IDs $c_1, \ldots, c_{n_k}$ where $1 \leq c_i \leq 10^6$ and $\sum n_k \leq 3 \cdot 10^5$. For each test case, determine whether any channel ID appears at least three times.

> **Pause & Reflect:** Using a hash map `unordered_map<int,int>` per test case has high constant factor and hash-collision risk, while `memset(freq, 0, sizeof(freq))` on a $10^6$-element array takes $10^5 \times 10^6 = 10^{11}$ operations. How do we get $O(1)$ array speed with $O(n_k)$ cleanup?

### Step 1 — Exact Model & Bottleneck

We need a frequency table `freq[c]` over the domain $[1, 10^6]$. Allocating `int freq[1000005]` once globally costs $4$ MB. The bottleneck is **resetting** `freq` between test cases: clearing all $M = 10^6$ slots takes $O(T \cdot M)$ time, whereas our budget is $O(\sum n_k)$.

### Step 2 — The Turn: Selective Rollback (or Timestamped Visited Array)

During test case $k$, at most $n_k$ distinct slots of `freq` are ever modified—specifically, `freq[c_i]` for $i \in \{1, \ldots, n_k\}$. After finishing test case $k$, loop over the testcase's input array $c_1, \ldots, c_{n_k}$ a second time and set `freq[c_i] = 0`!

$$\text{Work for test case } k = \underbrace{n_k \text{ increments}}_{\text{Pass 1}} + \underbrace{n_k \text{ selective zero-resets}}_{\text{Pass 2}} = 2n_k = O(n_k)$$

$$\text{Total work across } T \text{ tests} = \sum_{k=1}^{T} 2n_k \leq 6 \cdot 10^5 \text{ operations}$$

### Step 3 — Two-Way Proof & Small-Case Oracle

- **Invariant:** Before and after each test case, `freq[v] == 0` for all $v \in [1, 10^6]$. Every slot incremented during Pass 1 is listed in $c[1..n_k]$ and explicitly zeroed in Pass 2.
- **Transfer:** Whenever $\sum N \ll T \cdot \max(\text{Coordinate})$, allocate the workspace array once globally and undo only the indices touched by the current testcase.

---

# Section 2 — Simulation and Brute Force

## The Discovery & Modeling Approach

A common beginner misconception is that brute force is "what you write when you don't know the real algorithm." In competitive programming, **designed exhaustive search is a first-class algorithm** whenever a problem's coupled constraints make greedy or independent decisions invalid, and one of the problem's dimensions is small ($K \leq 20$ binary switches, $K \leq 8$ permuted components, or a bounded-depth branching factor).

Use two lenses to design a brute-force or backtracking solution systematically:

1. **Product-Space Dimension Slicing:** Often $N \leq 10^5$ is large, but the choice that couples everything together lives in a tiny dimension $K$ (e.g., the first row of a grid has $W \leq 10$ cells, or there are only $6$ colors). Enumerate all $2^W$ or $6!$ possibilities for the small dimension, and for each fixed choice, let the remaining $O(N)$ decisions become uniquely forced or greedily solvable!
2. **Reversible State & Early Pruning (Backtracking):** When exploring a search tree of depth $D$, avoid copying the state board at every node. Apply a transition `do_move()`, recurse, and immediately `undo_move()`. Prune any branch as soon as a partial invariant shows it cannot beat the best answer found so far or cannot satisfy a remaining quota.

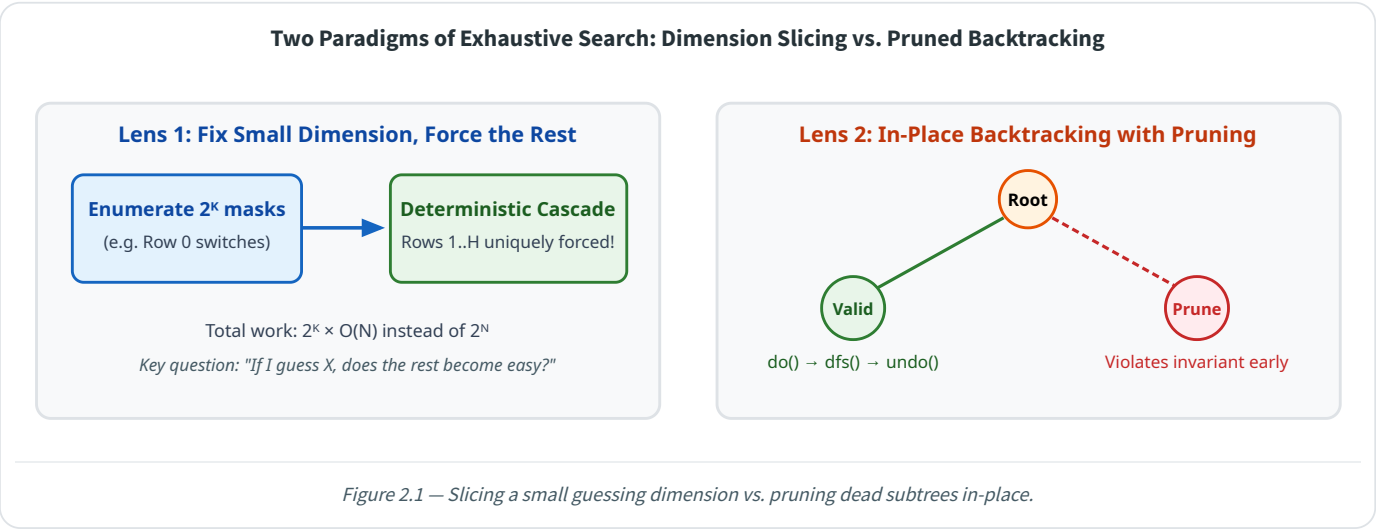



*Figure 2.1 — Slicing a small guessing dimension vs. pruning dead subtrees in-place.*

---

## Worked Problem 2.1: Cross-Toggle Indicator Grid

**Problem.** You are given a binary grid of $R$ rows and $C$ columns ($1 \leq R \leq 1000,\ 1 \leq C \leq 10$). Pressing cell $(r, c)$ flips the state of $(r, c)$ and its orthogonal neighbors $(r \pm 1, c), (r, c \pm 1)$. Find the minimum number of presses required to turn all cells `0` (off), or report `-1` if impossible.

> **Pause & Reflect:** With $R \cdot C = 10\,000$ cells, there are $2^{10000}$ press subsets. Yet $C \leq 10$ is tiny. Once you decide which buttons to press in Row 0, do you have *any* choice left when moving to Row 1?

### Step 1 — Exact Model & Bottleneck

Pressing the same cell twice cancels out ($\bmod 2$), and press order does not matter. Thus a solution is a binary subset of cells. Enumerating all $2^{RC}$ subsets is impossible.

### Step 2 — Tempting Mistakes & Counterexample

Trying to greedily press cells with the most `1`s in their neighborhood can cycle forever or miss solutions where a `0` cell must be pressed to fix its neighbors. For $1 \times 3$ grid `1 0 1`, pressing the middle `0` is wrong, whereas for `1 1 1`, pressing the middle `1` clears all three in 1 step.

### Step 3 — The Turn: Top-Row Guessing & Deterministic Row Cascade

Look at cell $(0, c)$ in the top row **after** we have chosen our presses for Row 0:

- No press in Row $2, 3, ..., R-1$ can ever reach Row 0!
- The **only** remaining button in the entire grid that touches $(0, c)$ is the cell directly below it: $(1, c)$.
- Therefore, for every column $c \in [0, C-1]$:

$$\text{press}(1, c) = 1 \Leftrightarrow \text{cell } (0, c) \text{ is still } 1 \text{ after Row } 0\text{'s presses!}$$

By induction, once Row $r-1$ is finalized, Row $r$'s presses are **100% uniquely forced** to extinguish any remaining `1`s in Row $r-1$. After cascading down to Row $R-1$, the choice is valid if and only if Row $R-1$ itself is now all `0`s!

### Step 4 — Progressive Construction & Visual Trace

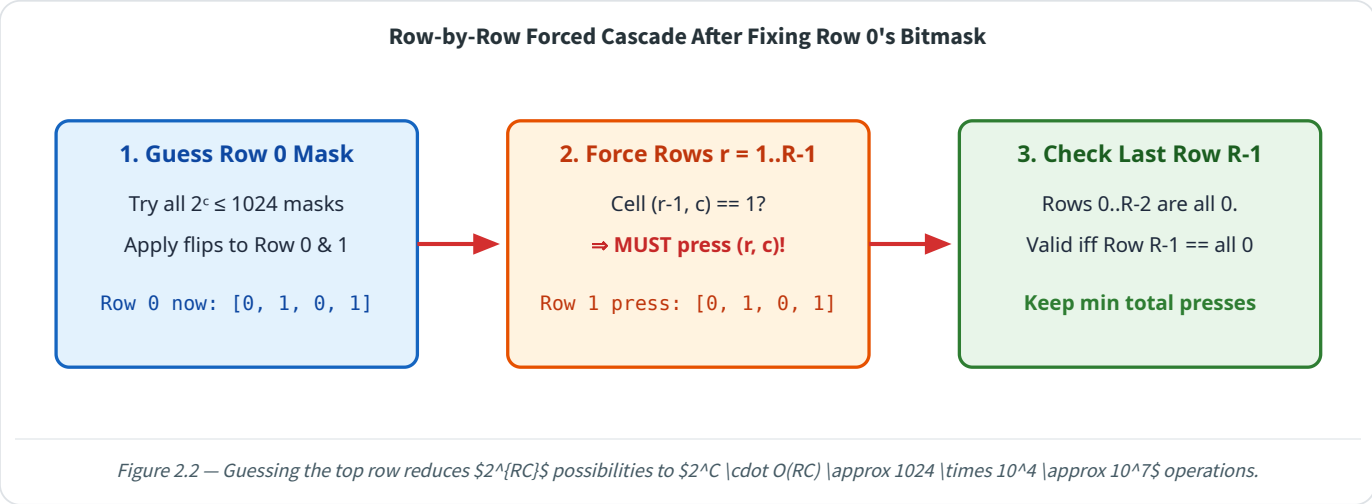



*Figure 2.2 — Guessing the top row reduces $2^{RC}$ possibilities to $2^C \cdot O(RC) \approx 1024 \times 10^4 \approx 10^7$ operations.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Bijection:** Every valid full-grid press configuration has some restriction to Row 0 (one of the $2^C$ masks) and must satisfy the unique vertical clearing condition for Rows $1 \ldots R-1$. Conversely, any Row 0 mask whose forced cascade leaves Row $R-1$ zeroed is a valid solution.

---

## Worked Problem 2.2: Non-Attacking Laser Splitters

**Problem.** On an $N \times N$ board ($N \leq 10$) with some blocked cells `#`, place $N$ laser splitters (one per row) so that no two share a column or diagonal, and additionally no two splitters in consecutive rows $r$ and $r+1$ are within horizontal distance 2 ($|c_r - c_{r+1}| \geq 3$). Count the number of valid placements.

> **Pause & Reflect:** How do we check column and diagonal occupancy in $O(1)$ per candidate cell without scanning the board?

### Step 1 — Exact Model & Bottleneck

Placing one splitter per row defines a sequence $(c_0, ..., c_{N-1}) \in \{0, ..., N-1\}^N$. Naive generation of $N^N = 10^{10}$ sequences and checking all pairs at the end is far too slow.

### Step 2 — The Turn: Row-by-Row Backtracking with Bitmask Occupancy

As we place a splitter at $(r, c)$, three global invariants must hold against all earlier rows $r' < r$:

1. **Column $c$ unused:** bit $c$ in `col_mask`.
2. **Main diagonal $r - c + (N-1)$ unused:** bit $r - c + N - 1$ in `diag1_mask`.
3. **Anti-diagonal $r + c$ unused:** bit $r + c$ in `diag2_mask`.
4. **Local adjacency:** $|c - c_{r-1}| \geq 3$.

Passing `(r, prev_c, col_mask, diag1_mask, diag2_mask)` by value (or toggling bits before/after recursion) tests each column $c$ in $O(1)$ bitwise instructions and prunes invalid partial placements immediately at row $r$.

---

# Section 3 — Counting, Frequencies, and Maps

## The Discovery & Modeling Approach

Whenever a problem asks you to **count pairs** $(i, j)$ satisfying an algebraic relation, or to **transform a collection of items** where position is irrelevant, raw array scanning is $O(N^2)$. The core discovery technique of Section 3 consists of two lenses:

1. **Algebraic Index Decoupling (For Pair Counting):** Given a condition $F(i, j)$ with $i < j$, rewrite the equation so that **all terms involving $i$ stand on the left** and **all terms involving $j$ stand on the right**:
   $$F(i, j) \Leftrightarrow g(i) = h(j)$$
   Now a single left-to-right loop over $j$ queries `freq[h(j)]` (how many earlier $i < j$ had $g(i) = h(j)$) and then inserts `freq[g(j)]++`!
2. **Frequency Quotienting (Histogram Compression):** When operations depend only on item values or colors rather than indices, replace the $N$-element array by its frequency histogram `cnt[v]`. Distinct values with identical frequencies can even be grouped further.

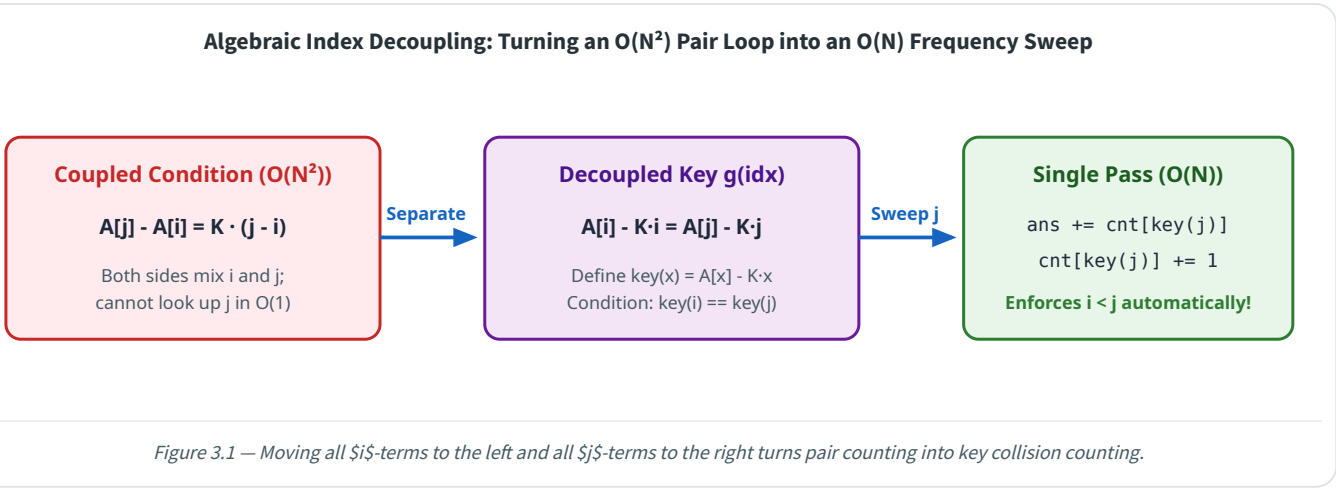



*Figure 3.1 — Moving all $i$-terms to the left and all $j$-terms to the right turns pair counting into key collision counting.*

| Domain Range of Key | Best Data Structure | Lookup Time | Crucial Caveat |
|---|---|---|---|
| Dense integers in $[0, 10^6]$ | Direct array `cnt[key]` | $O(1)$ | Shift negative keys by $+OFFSET$ before indexing. |
| Up to $2 \cdot 10^5$ arbitrary 64-bit keys | Sort keys and count contiguous runs | $O(N \log N)$ | Zero hash-collision risk, minimal memory overhead. |
| Online arbitrary 64-bit keys | `std::map` or custom-hash `unordered_map` | $O(\log N)$ / $O(1)$ | Standard `unordered_map` without `splitmix64` salt can be hacked to $O(N^2)$. |

---

## Worked Problem 3.1: Constant-Slope Beacon Pairs

**Problem.** Along a highway, $N$ beacons ($1 \le N \le 2 \cdot 10^5$) stand at positions $1, 2, \ldots, N$ with heights $H_1, \ldots, H_N$ ($|H_i| \le 10^9$). Given a target slope $S$ ($|S| \le 10^9$), count the number of pairs $(i, j)$ with $1 \le i < j \le N$ such that $\frac{H_j - H_i}{j - i} = S$.

> **Pause & Reflect:** How can we eliminate the division by $j - i$ and separate index $i$ from index $j$?

### Step 1 — Exact Model & Bottleneck

Testing all $\binom{N}{2} \approx 2 \cdot 10^{10}$ pairs $(i, j)$ takes $O(N^2)$ time.

### Step 2 — Tempting Mistakes & Counterexample

Using floating-point division `(double)(H[j] - H[i]) / (j - i) == S` is both $O(N^2)$ and suffers from precision loss on 64-bit integers.

### Step 3 — The Turn: Algebraic Separation into Intercept Keys

Since $j > i$, $j - i \neq 0$. Multiply both sides by $(j - i)$ and group by index:

$$\frac{H_j - H_i}{j - i} = S \Leftrightarrow H_j - H_i = S \cdot j - S \cdot i \Leftrightarrow \underbrace{H_i - S \cdot i}_{\text{key}(i)} = \underbrace{H_j - S \cdot j}_{\text{key}(j)}$$

Geometrically, two points $(i, H_i)$ and $(j, H_j)$ determine a line of slope $S$ if and only if they have the **exact same $y$-intercept** $b = H_i - S \cdot i$! Note that $S \cdot i$ can reach $2 \cdot 10^{14}$, so `key(i)` must be computed in 64-bit integer arithmetic.

### Step 4 — Two-Way Proof & Small-Case Oracle

Every pair $(i, j)$ with $i < j$ and $\text{key}(i) = \text{key}(j)$ is counted exactly once when the loop reaches $j$, because at that moment `cnt[key(j)]` holds the exact count of $i \in [1, j - 1]$ with the same intercept. Alternatively, if $c$ indices share the same key, they contribute $\binom{c}{2} = \frac{c(c-1)}{2}$ pairs.

---

## Worked Problem 3.2: Quota-Balanced Crate Recoloring

**Problem.** You have $N$ crates ($1 \leq N \leq 2 \cdot 10^5$), each painted with an integer color $c_i \in [1, N]$. In one operation, you may repaint any single crate to any existing or new color. Find the minimum number of operations so that every color present at the end appears between $L$ and $R$ times ($1 \leq L \leq R \leq N$). Colors may be eliminated completely (frequency $0$).

> **Pause & Reflect:** Why is it wrong to treat each color independently, and why should we sort the initial frequencies?

### Step 1 — Exact Model & Bottleneck

Crate positions do not matter: replace the input by its sorted nonzero frequencies $f_1 \leq f_2 \leq \ldots \leq f_m$ where $\sum_{r=1}^{m} f_r = N$. We must choose how many distinct colors $k$ survive in the final state ($\lceil N/R \rceil \leq k \leq \lfloor N/L \rfloor$).

### Step 2 — Tempting Mistakes & Counterexample

Suppose $N = 6, L = 3, R = 3$ and initial frequencies are $[1, 2, 3]$. If we sum "deficits" $(3 - 1) + (3 - 2) = 3$, we get $3$ operations. In reality, we should **eliminate** the color of frequency $1$ and donate its crate to the color of frequency $2$, reaching $[0, 3, 3]$ in **1 single operation**! One repainted crate simultaneously reduces an excess (or eliminated) bucket by $1$ and raises a deficit bucket by $1$.

### Step 3 — The Turn: Suffix Survivors & Supply-Demand Conservation

If we decide that exactly $k$ colors survive, which $k$ initial colors should we keep? By an exchange argument, we should always keep the $k$ **largest initial frequencies** (the suffix $f_{m-k+1 \ldots m}$) and completely dissolve the prefix $f_{1 \ldots m-k}$.

For a fixed survivor count $k$: 1. **Supply of excess crates ($S_{\text{out}}$):** All crates in the dissolved prefix, plus any crates above $R$ in the surviving suffix:

$$S_{\text{out}}(k) = \sum_{r=1}^{m-k} f_r + \sum_{r=m-k+1}^{m} \max(0,\ f_r - R)$$

2. **Demand of deficit crates ($D_{\text{in}}$):** Crates needed to raise surviving colors below $L$ up to $L$:

$$D_{\text{in}}(k) = \sum_{r=m-k+1}^{m} \max(0,\ L - f_r)$$

Since total crate count $N$ is conserved and $kL \le N \le kR$, every excess crate can be repainted into a deficit color or into a non-full surviving color ( $< R$), and every deficit crate can be pulled from an excess or non-minimum surviving color ( $> L$). Thus the exact cost for a valid $k$ is:

$$\mathrm{Cost}(k) = \max\left( S_{\mathrm{out}}(k),\ D_{\mathrm{in}}(k) \right)$$

### Step 4 — Progressive Construction & Visual Trace

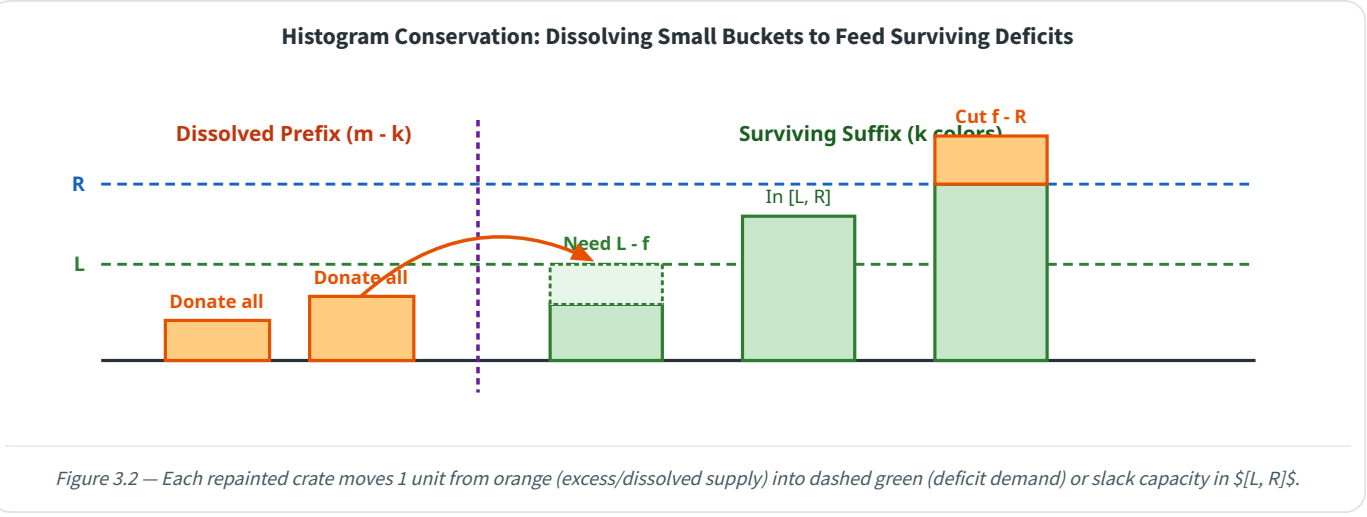



*Figure 3.2 — Each repainted crate moves 1 unit from orange (excess/dissolved supply) into dashed green (deficit demand) or slack capacity in $[L, R]$.*

### Step 5 — Two-Way Proof & Small-Case Oracle

With prefix sums and binary search on the sorted array $f$, both $S_{\mathrm{out}}(k)$ and $D_{\mathrm{in}}(k)$ are evaluated in $O(\log m)$ time for each $k \in [\lceil N/R \rceil, \lfloor N/L \rfloor]$, yielding an $O(N \log N)$ solution.

---

# Section 4 — Prefix Sums and Difference Arrays

## The Discovery & Modeling Approach

Prefix sums and difference arrays are **discrete integration and differentiation** over an array. They are exact inverses of each other:

1. **Prefix Sum ($\Delta^{-1}$: Integration for Static Range Queries):** When an array $A[1..N]$ is static and you need $Q$ range sums $\sum_{i=l}^{r} A[i]$, precompute $P[i] = P[i-1] + A[i]$ with $P[0] = 0$. Every contiguous range sum collapses into a **two-endpoint subtraction**:
$$\sum_{i=l}^{r} A[i] = P[r] - P[l-1]$$
Better yet, any condition on a subarray $[l, r]$ can now be rewritten as a **pair condition on prefix indices** $(l-1, r)$, unlocking Section 3's frequency-map decoupling!
2. **Difference Array ($\Delta$: Differentiation for Batch Range Updates):** When $Q$ operations each add $+v$ to every element in $[l, r]$ and queries happen *after* all updates finish, do not touch the interior of $[l, r]$. Instead, record only the **boundary transitions** on a difference array $D$:
$$D[l] \mathrel{+}= v, \qquad D[r+1] \mathrel{-}= v$$
Taking one prefix sum of $D$ at the end reconstructs the final array $A[i] = \sum_{k=1}^{i} D[k]$ in $O(N)$ time.

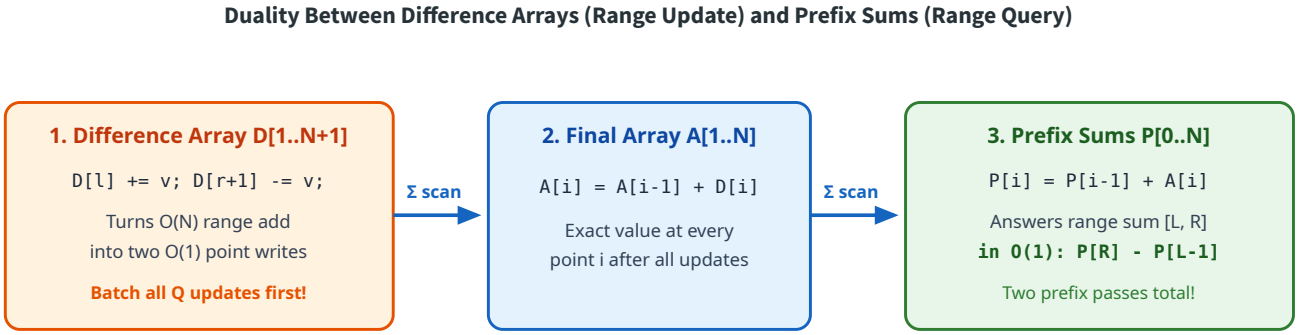



*Figure 4.1 — Combining a difference array pass and a prefix sum pass handles $Q$ offline range additions followed by $Q$ range sum queries in $O(N + Q)$ time.*

---

## Worked Problem 4.1: Thermal Shielding Layers

**Problem.** A linear particle accelerator has $N$ segments ($1 \le N \le 2 \cdot 10^5$). Each of $M$ shielding plates covers an interval $[l_j, r_j]$ and adds $v_j$ protection units ($v_j \ge 1$). A segment $i$ is **certified** if its total protection is at least $K$. After installing all $M$ plates, answer $Q$ queries $[L, R]$: how many segments in $[L, R]$ are certified?

> **Pause & Reflect:** Why do we need *two* prefix sums here, and what transforms between the first prefix sum and the second?

### Step 1 — Exact Model & Bottleneck

Applying each plate loop-by-loop takes $O(NM)$ time, and scanning $[L, R]$ per query takes $O(NQ)$ time. Both exceed the time limit for $N, M, Q = 2 \cdot 10^5$.

### Step 2 — The Turn: Three-Stage Pipeline (Difference $\rightarrow$ Threshold Indicator $\rightarrow$ Prefix Sum)

1. **Stage 1 (Difference Array):** Accumulate all $M$ plates via `D[l] += v; D[r + 1] -= v;`. Reconstruct the exact protection at each segment: $A[i] = A[i-1] + D[i]$.
2. **Stage 2 (Pointwise Non-Linear Threshold):** Convert each $A[i]$ into a binary indicator:

$$B[i] = \begin{cases} 1 & \text{if } A[i] \geq K \\ 0 & \text{otherwise} \end{cases}$$

3. **Stage 3 (Prefix Sum of Indicators):** Build $P[i] = P[i-1] + B[i]$. Now each query $[L, R]$ is simply $P[R] - P[L-1]$ in $O(1)$ time!

### Step 3 — Two-Way Proof & Small-Case Oracle

- **Boundary Check:** Allocate `D` of size $N+2$ so that `D[r + 1]` never writes out of bounds when $r = N$.

---

## Worked Problem 4.2: Target-Average Contiguous Subarrays

**Problem.** Given an array $A[1..N]$ of integers ($1 \leq N \leq 2 \cdot 10^5$, $|A_i| \leq 10^9$) and a target integer $K$, count how many contiguous subarrays $A[l..r]$ ($1 \leq l \leq r \leq N$) have an arithmetic mean **exactly equal** to $K$.

> **Pause & Reflect:** A subarray's mean is $\frac{P[r] - P[l-1]}{r - (l-1)} = K$. How can we transform the array $A$ upfront so that we are just looking for subarrays with sum $0$?

### Step 1 — Exact Model & Bottleneck

Checking all $\binom{N}{2}$ subarrays via prefix sums takes $O(N^2)$ time because the denominator $r - l + 1$ changes with every pair $(l, r)$.

### Step 2 — The Turn: Subtracting $K$ From Every Element (Zero-Sum Reduction)

Rewrite the average condition algebraically:

$$\frac{1}{r-l+1}\sum_{i=l}^{r} A[i] = K \Leftrightarrow \sum_{i=l}^{r} A[i] = \sum_{i=l}^{r} K \Leftrightarrow \sum_{i=l}^{r} (A[i] - K) = 0$$

Define the shifted array $B[i] = A[i] - K$ and its prefix sums $P_B[r] = \sum_{i=1}^{r} (A[i] - K)$ with $P_B[0] = 0$. Then subarray $[l, r]$ has average $K$ **if and only if**:

$$P_B[r] - P_B[l-1] = 0 \Leftrightarrow P_B[r] = P_B[l-1]$$

Thus the problem reduces to counting pairs of equal values in $P_B[0..N]$! Initialize `freq[0] = 1` (for the empty prefix $P_B[0]$), sweep $r = 1 \ldots N$, add `freq[P_B[r]]` to the answer, and increment `freq[P_B[r]]++`.

---

# Section 5 — Sorting as a Tool

## The Discovery & Modeling Approach

Sorting costs $O(N \log N)$—almost free for $N \leq 2 \cdot 10^5$—and provides a massive structural upgrade: **it turns global value relationships into local index relationships**.

Ask whether sorting applies by checking two signatures:

1. **Order Invariance (Set Selection):** If the problem asks you to pick a subset of $k$ elements and the objective depends only on the *values* chosen (such as $\max - \min$, pairwise differences, or sum of top elements), the original input positions are noise. Sort first! Once sorted, the minimum and maximum of any chosen subset become its **leftmost and rightmost sorted indices** $i$ and $j$, and every element in between ($i < m < j$) is free to include without worsening the range $[A_i, A_j]$.
2. **Canonical Sweep Order:** When elements interact asymmetrically (e.g., larger items can hold smaller items, or earlier deadlines constrain later tasks), sorting orders the items so that as you scan left-to-right, every previously seen item automatically satisfies one side of the inequality ($A_i \leq A_j$).

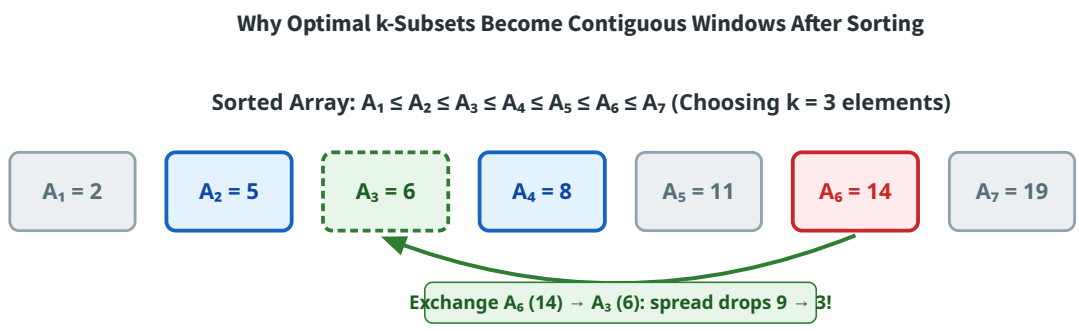



*Figure 5.1 — Replacing an outer element ($A_6$) with a skipped interior element ($A_3$) never increases the spread $\max - \min$, proving contiguous blocks are optimal.*

---

## Worked Problem 5.1: Minimum Upgrade Cost to Form a $K$-Cluster

**Problem.** You are given $N$ server frequencies $A_1, \ldots, A_N$ ($1 \leq K \leq N \leq 2 \cdot 10^5$, $1 \leq A_i \leq 10^9$). In one operation, you can increment any server's frequency by $+1$. Find the minimum total operations needed to make at least $K$ servers have the **exact same** frequency.

> **Pause & Reflect:** Since you can only *increase* frequencies, if you decide that the $K$ equal servers will end up at frequency $A_r$ (after sorting), which $K$ servers should you upgrade to $A_r$?

### Step 1 — Exact Model & Bottleneck

For any target value $T$ and any subset $S$ of $K$ servers with $A_i \leq T$, the cost is $\sum_{i \in S} (T - A_i) = K \cdot T - \sum_{i \in S} A_i$.

### Step 2 — The Turn: Contiguous Left-Neighbor Window After Sorting

Sort $A_1 \leq A_2 \leq \ldots \leq A_N$. First, the optimal target frequency $T$ must equal some existing $A_r$ (if $T$ were strictly between $A_r$ and $A_{r+1}$, lowering $T$ to $A_r$ would save $K(T - A_r)$ operations). Second, once the highest element of our $K$-subset is fixed at index $r$, to minimize $\sum_{i \in S} (A_r - A_i)$ we must pick the $K$ largest available elements $\leq A_r$—which are **precisely the contiguous block** $A_{r-K+1 \ldots r}$!

$$\text{Cost}(r) = K \cdot A_r - \sum_{i=r-K+1}^{r} A_i = K \cdot A_r - \left( P[r] - P[r-K] \right)$$

Scanning $r = K...N$ with a prefix sum array $P$ finds the global minimum in $O(N \log N)$ time.

---

## Worked Problem 5.2: Asymmetric Shield-Generator Pairing

**Problem.** You have $N$ shields with resistances $a_1, ..., a_N$ and $N$ generators with outputs $b_1, ..., b_N$ ($1 \leq N \leq 2 \cdot 10^5$). Pairing shield $i$ with generator $j$ produces an instability penalty of $|a_i - b_j|$. Pair all $N$ shields and generators 1-to-1 to **minimize** the total instability $\sum |a_i - b_{\pi(i)}|$.

> **Pause & Reflect:** If $a_1 \leq a_2$ and $b_1 \leq b_2$, can crossing the pairs ($a_1 \leftrightarrow b_2$ and $a_2 \leftrightarrow b_1$) ever strictly beat parallel pairing ($a_1 \leftrightarrow b_1$ and $a_2 \leftrightarrow b_2$)?

### Step 1 — The Turn: Uncrossing / Monotone Alignment Proof

Sort both arrays in non-decreasing order: $a_1 \leq ... \leq a_N$ and $b_1 \leq ... \leq b_N$. Consider any two indices $1 \leq i < j \leq N$, so $a_i \leq a_j$ and $b_i \leq b_j$. By case analysis (or visualizing intervals on the real line), the **Uncrossing Inequality** holds for all real numbers $a_i \leq a_j$ and $b_i \leq b_j$:

$$|a_i - b_i| + |a_j - b_j| \leq |a_i - b_j| + |a_j - b_i|$$

*(Why? When the intervals overlap, parallel pairing leaves the overlap uncovered, whereas crossed pairing covers the overlap twice!)*
Repeatedly uncrossing any inversion in the permutation $\pi$ transforms any matching into the identity matching $\pi(i) = i$ without increasing the total cost. Thus $\sum_{i=1}^{N} |a_i - b_i|$ after sorting both arrays is provably optimal.

---

# Section 6 — Two Pointers and Sliding Windows

## The Discovery & Modeling Approach

A two-pointer algorithm replaces an $O(N^2)$ nested loop over pairs $(L, R)$ with a **monotone walk** that visits at most $2N$ states. There are two distinct flavors, each governed by a specific invariant:

1. **Same-Direction Sliding Window ($L \le R$ both moving right):** Used when you want the longest/shortest valid subarray $[L, R]$ or the count of all valid subarrays, and **validity is hereditary (monotone under inclusion)**:
   - Adding an element at $R$ can only *increase* cost/weight;
   - Removing an element at $L$ can only *decrease* cost/weight. Consequently, the leftmost valid boundary $L^*(R)$ for a given right endpoint $R$ satisfies $L^*(R) \le L^*(R+1)$—pointer $L$ never needs to move backward!
2. **Opposite-End Converging Pointers ($L$ moves right, $R$ moves left):** Used on a sorted array when testing a binary function $f(A_L, A_R)$ (like $A_L + A_R$) that increases with $L$ and increases with $R$. Each comparison eliminates an entire row or column of the $(L, R)$ matrix!

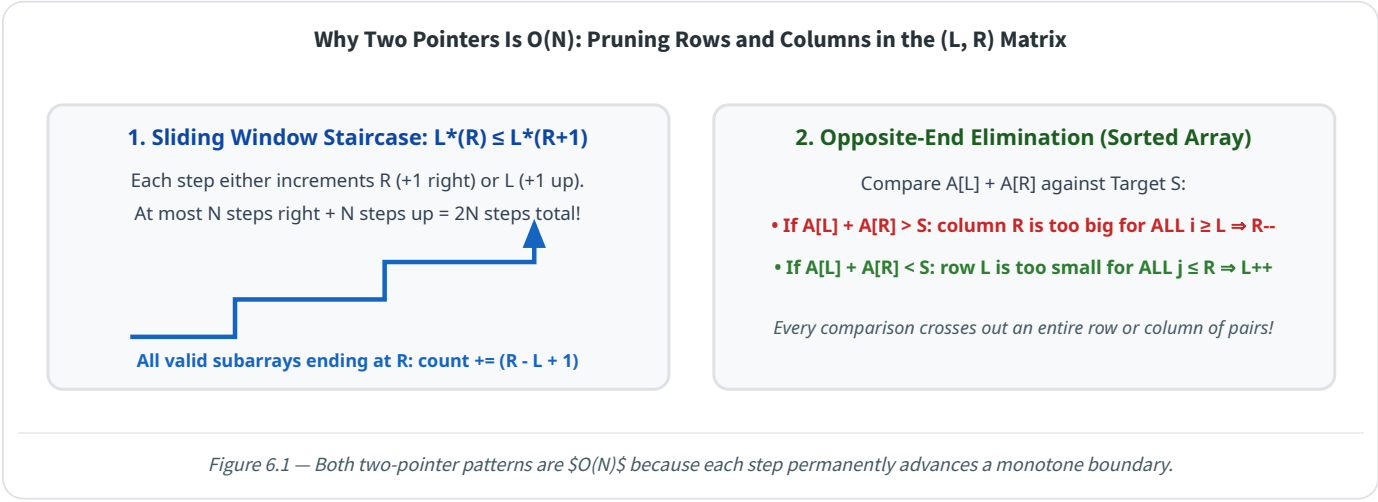



*Figure 6.1 — Both two-pointer patterns are $O(N)$ because each step permanently advances a monotone boundary.*

> **Classic Trap: Sliding Window vs. Negative Numbers**
>
> A sum-bounded sliding window (`sum <= K`) requires **all elements** $A_i \ge 0$ so that expanding $[L, R]$ to $[L, R+1]$ monotonically increases the sum. If $A_i$ can be negative, extending $R$ might *decrease* the sum and repair an invalid window, breaking monotonicity! (For signed arrays, use prefix sums + Section 12's monotonic deque or Section 36's Fenwick tree instead.)

---

## Worked Problem 6.1: K-Glitch Telemetry Subarrays

**Problem.** A binary status stream $S[1..N]$ ($N \le 2 \cdot 10^5$) contains `1` (healthy) and `0` (glitch). You have a repair buffer that can mask at most $K$ glitches inside a contiguous window. Count the total number of contiguous subarrays $[L, R]$ that contain at most $K$ zeroes.

> **Pause & Reflect:** Once you find the smallest $L$ such that $[L, R]$ has $\le K$ zeroes, how many valid subarrays **end at** $R$?

### Step 1 — Exact Model & Bottleneck

Enumerating all pairs $(L, R)$ takes $O(N^2)$.

### Step 2 — The Turn: Hereditary Validity & Batch Counting by Right Endpoint

Let `zero_cnt` be the number of `0`s in $[L, R]$. Since adding an element increments `zero_cnt` by $0$ or $1$ (non-negative!), if $[L, R]$ has $\le K$ zeroes, then **every shorter suffix** $[l, R]$ **with** $L \le l \le R$ **also has** $\le K$ **zeroes**!

Maintain the minimal valid $L$ as $R$ advances from 1 to $N$: 1. Include $S[R]$: if $S[R] = \ = 0$, `zero_cnt++`. 2. While `zero_cnt > K`, remove $S[L]$ (`if S[L] == 0: zero_cnt--`) and advance `L++`. 3. Now $L$ is the **leftmost** valid start for this $R$, so all starts $l \in \{L, L+1, ..., R\}$ are valid:

$$\text{Total valid subarrays} = \sum_{R=1}^{N} \left( R - L^*(R) + 1 \right)$$

### Step 3 — Two-Way Proof & Small-Case Oracle

Each pointer $L$ and $R$ increments at most $N$ times ($O(N)$ total work), and every valid subarray $[l, r]$ is counted once in the iteration for its unique right endpoint $R = r$.

---

## Worked Problem 6.2: Maximum Bottleneck Container Area

**Problem.** Given $N$ vertical posts at positions $1, ..., N$ with heights $H_1, ..., H_N$ ($2 \le N \le 2 \cdot 10^5$, $H_i \ge 1$), choose two posts $i < j$ to maximize the rectangular reservoir capacity $(j - i) \cdot \min(H_i, H_j)$.

> **Pause & Reflect:** Start with the widest pair $(L, R) = (1, N)$. Suppose $H_L \le H_R$. Why is it 100% safe to increment $L$ and discard $(L, j)$ for all $L < j < R$ without ever checking them?

### Step 1 — The Turn: Bottleneck Elimination Proof

Start at $L = 1, R = N$ (maximum possible width $R - L$). Suppose without loss of generality that $H_L \le H_R$. Consider any other pair $(L, j)$ using the shorter post $L$ with an interior right post $L < j < R$:

1. Its width is strictly smaller: $j - L < R - L$.
2. Its height is bounded by $H_L$: $\min(H_L, H_j) \le H_L = \min(H_L, H_R)$.

Therefore:

$$(j - L) \cdot \min(H_L, H_j) < (R - L) \cdot \min(H_L, H_R)$$

In one line, we have proved that **post $L$ can never achieve a better score with any $j \in (L, R)$ than it just achieved with $R$**! So we record $(R - L) \cdot H_L$, permanently eliminate $L$ (`L++`), and repeat until $L = R$.

# Section 8 — Binary Search on the Answer

## The Discovery & Modeling Approach

When an optimization problem resists direct construction—when no greedy sorting order reveals itself and dynamic programming states explode because of a bottleneck resource—ask a simpler question: **If someone whispered the optimal value $X$ to you, could you verify in linear or near-linear time whether $X$ is achievable?**

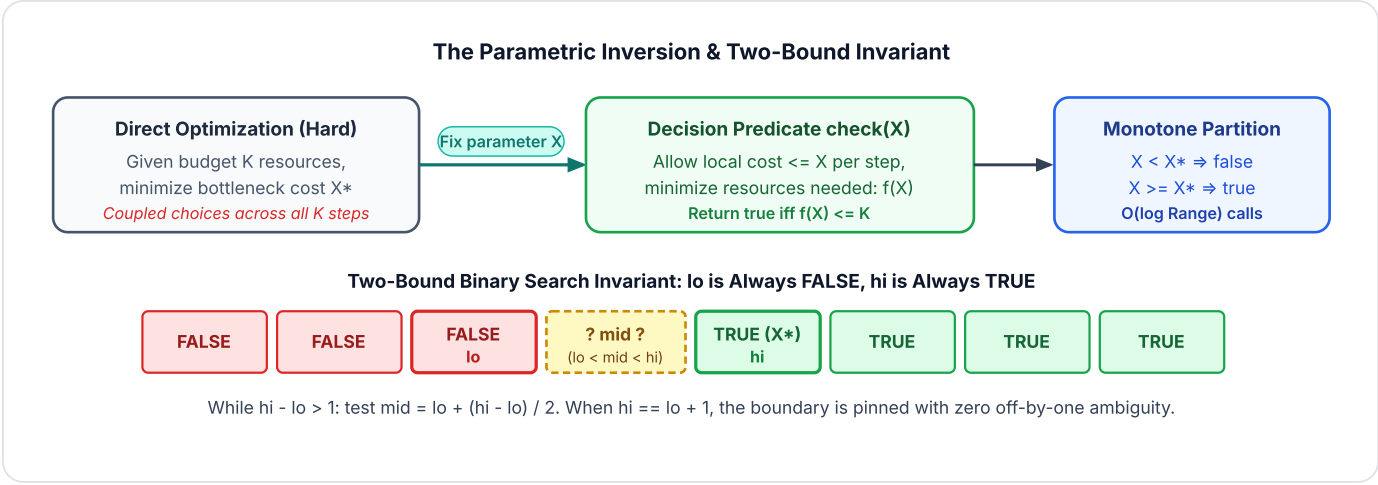
That question is the gateway to **Binary Search on the Answer** (also called **Parametric Search**). Instead of building the optimal configuration from scratch, we invert the problem: we turn the **optimization objective** into a **budget parameter** $X$ inside a yes/no decision predicate $\text{check}(X) \in \{\text{false}, \text{true}\}$.

### Recognizing the Paradigm from Problem Signals

1. **Minimax or Maximin phrasing**:
   - *"Minimize the maximum load / distance / time / cost…"*
   - *"Maximize the minimum gap / capacity / safety margin…"* In any minimax problem, bounding the maximum by $X$ means **every single component** must satisfy $\text{cost}_i \leq X$. That uniform ceiling converts a coupled global optimization into local step-by-step greediness.
2. **Threshold monotonicity**:
   - If a budget of $X$ suffices, then any budget $X' > X$ clearly suffices ($\text{false}, \ldots, \text{false}, \text{true}, \ldots, \text{true}$).
   - Conversely, if a minimum safety gap of $X$ is achievable, any smaller gap $X' < X$ is also achievable ( $\text{true}, \ldots, \text{true}, \text{false}, \ldots, \text{false}$).
3. **K-th smallest value in an implicit $N \times M$ or $\binom{N}{2}$ set**:
   - Whenever generating all pairs/subarrays takes $O(N^2)$ but counting how many values are $\leq X$ takes $O(N)$ or $O(N \log N)$, binary search over $X$ finds the $K$-th value where $\text{count}(\leq X) \geq K$ first becomes true.
4. **Average / Ratio Optimization (0-1 Fractional Programming)**:
   - Maximizing $\frac{\sum_{i \in S} a_i}{\sum_{i \in S} b_i}$ cannot be done by sorting by $a_i/b_i$ alone because high-ratio items with tiny denominators can be outweighed or diluted by capacity constraints. Testing whether the ratio can reach $X$ rewrites algebraically to a linear sum $\sum_{i \in S} (a_i - X \cdot b_i) \geq 0$.

### Decision Compass: Structural Lenses for Parametric Search

| Sub-Pattern | Typical Clue in Statement | Predicate $\text{check}(X)$ | Monotonicity Profile | Search Domain & Complexity |
|---|---|---|---|---|
| **Minimax Partition / Covering** | "Minimize the maximum radius / segment sum using at most $K$ pieces" | Greedily extend each piece as far as $\leq X$ allows; check if pieces used $\leq K$ | `F F ... F T T ... T` (Find **first** `T`) | Integer range $[L, R]$; $O(N \log(R-L))$ |
| **Maximin Placement / Packing** | "Maximize the minimum distance / capacity among $K$ chosen items" | Greedily pick next valid item at distance $\geq X$; check if items picked $\geq K$ | `T T ... T F F ... F` (Find **last** `T`) | Integer range $[0, \max - \min]$; $O(N \log(\Delta))$ |
| **Implicit $K$-th Element** | "Find the $K$-th smallest pair distance / product / subarray sum" | Two-pointer or binary search scan to count pairs with value $\leq X$; test $\text{cnt}(X) \geq K$ | `F F ... F T T ... T` (Find **first** `T`) | Value range $[V_{\min}, V_{\max}]$; $O(N \log N \log V)$ |
| **0-1 Fractional Programming** | "Maximize $\sum a_i / \sum b_i$ subject to size $\geq K$ or knapsack constraint" | Assign modified weight $w_i(X) = a_i - X b_i$; maximize $\sum w_i(X)$ and test $\geq 0$ | `T T ... T F F ... F` (Continuous threshold $X^*$) | Real interval $[0, \max(a_i/b_i)]$; 60–80 iterations |




---

## Worked Problem 8.1: Drone Relay Radius

### Problem & Constraints

You are given $N$ villages located at integer coordinates $x_1, x_2, ..., x_N$ along a straight highway, and a fleet of $K$ relay drones. Each relay drone can be stationed at any integer coordinate $p$ and provides wireless coverage to every village $i$ within distance $R$, i.e., $|x_i - p| \leq R$. Find the **minimum integer radius** $R \geq 0$ such that all $N$ villages can be covered using at most $K$ drones.

- $1 \leq K \leq N \leq 2 \cdot 10^5$
- $0 \leq x_i \leq 10^9$

> **Pause & Discover:** Before reading further, ask yourself:
> - Why is dividing the total span $(x_{\max} - x_{\min})$ into $K$ equal pieces incorrect?
> - If the radius $R$ were fixed in advance, how would you place the first drone so that it never wastes coverage on empty road to the left?

### Step 1 — Exact Model & Bottleneck

After sorting the villages so that $x_1 \leq x_2 \leq ... \leq x_N$, any valid placement of $K$ drones at coordinates $p_1 \leq p_2 \leq ... \leq p_K$ partitions the sorted villages into at most $K$ contiguous groups. A contiguous group $[x_l ... x_r]$ can be covered by a single drone of radius $R$ if and only if we place the drone at $p = \lfloor (x_l + x_r)/2 \rfloor$ and its diameter satisfies:

$$x_r - x_l \leq 2R \Leftrightarrow R \geq \left\lceil \frac{x_r - x_l}{2} \right\rceil$$

An exact dynamic programming model would compute $\mathrm{dp}[k][i]$: the minimum radius needed to cover the first $i$ villages using $k$ drones. Transitioning over all split points $j < i$ takes $O(KN^2)$ time, and searching over radii $R \in [0, 10^9]$ linearly takes $O(N \cdot \max x_i)$ time—both far exceeding the $2 \cdot 10^5$ limit.

### Step 2 — Tempting Mistakes & Counterexample

1. **Tempting Mistake 1 (Equal-Width Span Division)**: Estimate $R = \lceil (x_N - x_1)/(2K) \rceil$.
   - **Counterexample**: Let $N = 4$, $K = 2$, and $x = [0, 2, 100, 102]$. The total span is 102, so equal division predicts $R \approx 26$. Yet placing one drone at $p_1 = 1$ (covering $[0, 2]$) and another at $p_2 = 101$ (covering $[100, 102]$) achieves $R = 1$ because the gap $[2, 100]$ contains no villages!
2. **Tempting Mistake 2 (Non-Monotone Predicate)**: Testing whether *exactly* $K$ drones are needed, or constraining drones to sit only on village coordinates.
   - **Counterexample**: For $x = [0, 2]$ and $K = 1$, if drones can sit only at villages, we need $R = 2$. Sitting at integer midpoint $p = 1$ requires only $R = 1$. And the predicate must test $\mathrm{needed}(R) \leq K$, not $= K$, because unused drones can always be parked harmlessly.

### Step 3 — The Turn: Discovering the Structure

Fix a candidate radius $R \geq 0$. Now the question changes from *"What radius should we pick?"* to *"What is the minimum number of drones of radius $R$ needed to cover $x_1 ... x_N$?"*

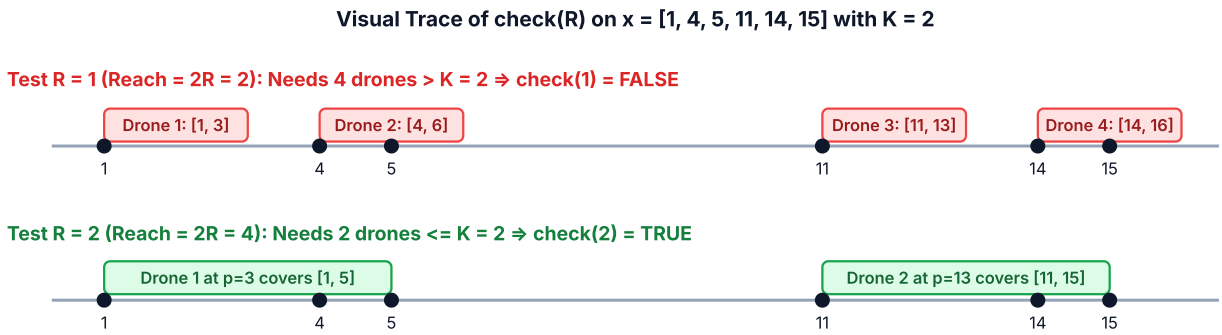
Consider the leftmost uncovered village $x_l$. Some drone must cover $x_l$, which constrains that drone's position $p$ to satisfy $p \leq x_l + R$. Since no uncovered villages exist to the left of $x_l$, shifting $p$ as far right as legally possible—to $p = x_l + R$—maximizes the drone's rightward reach to $[x_l, x_l + 2R]$.

**Monotone Coverage Predicate** $\text{check}(R)$**:**

1. Sort $x_1 \leq x_2 \leq ... \leq x_N$.
2. While an uncovered village $x_l$ remains, place a drone at $p = x_l + R$, which covers all villages $x_j$ with $x_j \leq x_l + 2R$.
3. Let $f(R)$ be the number of drones placed. Because increasing $R$ strictly expands every interval $[x_l, x_l + 2R]$, $f(R)$ is monotonically non-increasing in $R$, so $\text{check}(R) \equiv [f(R) \leq K]$ transitions once from $\text{false}$ to $\text{true}$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $x = [1, 4, 5, 11, 14, 15]$ with $K = 2$ drones.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of Greedy Predicate Correctness (Stays-Ahead Argument)**: Let $g_1 < g_2 < ... < g_m$ be the right endpoints of the coverage intervals chosen by our greedy algorithm for a fixed radius $R$, and let $o_1 \leq o_2 \leq ... \leq o_{m^*}$ be the right endpoints of any valid placement of $m^*$ drones covering all villages. - **Base case ($j = 1$)**: The first drone in the optimal solution that covers $x_1$ can reach at most $x_1 + 2R = g_1$, so $o_1 \leq g_1$. - **Inductive step**: Suppose after $j - 1$ drones, the greedy solution has covered at least as many villages as the first $j - 1$ drones of the optimal solution ($g_{j-1} \geq o_{j-1}$). Then the first village uncovered by the greedy algorithm after $j - 1$ drones is at some coordinate $x_l \geq x_{l'}$, where $x_{l'}$ is the first village uncovered by the first $j - 1$ optimal drones. Therefore $g_j = x_l + 2R \geq x_{l'} + 2R \geq o_j$. Hence the greedy algorithm stays ahead at every step and uses the minimum possible number of drones $m = f(R)$.

**Implementation & Small-Case Oracle**:

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

bool can_cover(const vector<long long>& x, int k, long long R) {
    int drones = 0;
    int i = 0, n = static_cast<int>(x.size());
    while (i < n) {
        drones++;
        if (drones > k) return false;
        long long limit = x[i] + 2 * R;
        while (i < n && x[i] <= limit) {
```

```
            i++;
        }
    }
    return true;
}

long long solve_parametric(vector<long long> x, int k) {
    sort(x.begin(), x.end());
    long long lo = -1; // Invariant: check(lo) is always false
    long long hi = (x.back() - x.front() + 1) / 2; // Invariant: check(hi) is always true
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (can_cover(x, k, mid)) {
            hi = mid;
        } else {
            lo = mid;
        }
    }
    return hi;
}

// Small-case oracle: exact O(K * N^2) partition DP for verification
long long solve_oracle_dp(vector<long long> x, int k) {
    sort(x.begin(), x.end());
    int n = static_cast<int>(x.size());
    const long long INF = 4e18;
    vector<long long> dp(n + 1, INF);
    dp[0] = 0;
    for (int step = 1; step <= k; ++step) {
        vector<long long> next_dp(n + 1, INF);
        for (int i = 0; i <= n; ++i) {
            if (dp[i] == INF) continue;
            for (int j = i + 1; j <= n; ++j) {
                long long radius_needed = (x[j - 1] - x[i] + 1) / 2;
                next_dp[j] = min(next_dp[j], max(dp[i], radius_needed));
            }
        }
        dp = move(next_dp);
    }
    return dp[n];
}
```

---

## Worked Problem 8.2: Spectral Amplifier Array (0-1 Fractional Programming)

### Problem & Constraints

A deep-space receiver has $N$ candidate amplifier modules. Module $i$ produces signal gain $a_i > 0$ and thermal noise $b_i > 0$. You must activate a subset $S \subseteq \{1, \ldots, N\}$ of size **at least** $K$ so as to **maximize the aggregate signal-to-noise ratio**:

$$\mathrm{SNR}(S) = \frac{\sum_{i \in S} a_i}{\sum_{i \in S} b_i}$$

Output the maximum achievable ratio accurate to $10^{-6}$.

- $1 \le K \le N \le 10^5$
- $1 \le a_i, b_i \le 10^6$

**Pause & Discover:**

- Why does sorting modules by individual ratio $a_i/b_i$ descending and taking the top $K$ fail?
- If you test whether $\frac{\sum_{i \in S} a_i}{\sum_{i \in S} b_i} \geq X$, how can you clear the denominator so that every module's contribution becomes independent?

### Step 1 — Exact Model & Bottleneck

Enumerating all $\sum_{m=K}^{N} \binom{N}{m}$ subsets takes $O(2^N)$ time. Even dynamic programming over total noise $\sum b_i$ would require $O(N \sum b_i)$ states, which is $10^{16}$ operations when $N = 10^5$ and $b_i \leq 10^6$. The bottleneck is that a fraction $\frac{A_1 + A_2}{B_1 + B_2}$ is **not additive**: knowing the best ratio of a subset of size $k-1$ does not tell us which subset survives best when a new denominator $b_{\text{new}}$ is added!

### Step 2 — Tempting Mistakes & Counterexample

**Tempting Mistake**: Sort modules by their standalone ratio $r_i = a_i/b_i$ in descending order and take the first $K$.

**Concrete 3-Module Counterexample ($N = 3, K = 2$):** - Module 1: $(a_1, b_1) = (10, 1) \implies r_1 = 10.00$ - Module 2: $(a_2, b_2) = (90, 20) \implies r_2 = 4.50$ - Module 3: $(a_3, b_3) = (4, 1) \implies r_3 = 4.00$

Sorting by individual ratio $r_i$ selects $\{1, 2\}$, giving:

$$\text{SNR}(\{1,2\}) = \frac{10 + 90}{1 + 20} = \frac{100}{21} \approx 4.7619$$

However, selecting $\{1, 3\}$ yields:

$$\text{SNR}(\{1,3\}) = \frac{10 + 4}{1 + 1} = \frac{14}{2} = 7.0000$$

Module 2 had a higher individual ratio ($4.5 > 4.0$), but its huge denominator ($b_2 = 20$) diluted Module 1's strong lead!

### Step 3 — The Turn: Discovering the Structure

Let us apply Parametric Inversion. Fix a target ratio $X \geq 0$ and ask: **Does there exist a subset $S$ with $|S| \geq K$ such that $\text{SNR}(S) \geq X$ ?**

Since $b_i > 0$ for all $i$, the total denominator $\sum_{i \in S} b_i$ is strictly positive, so we can multiply both sides without flipping the inequality:

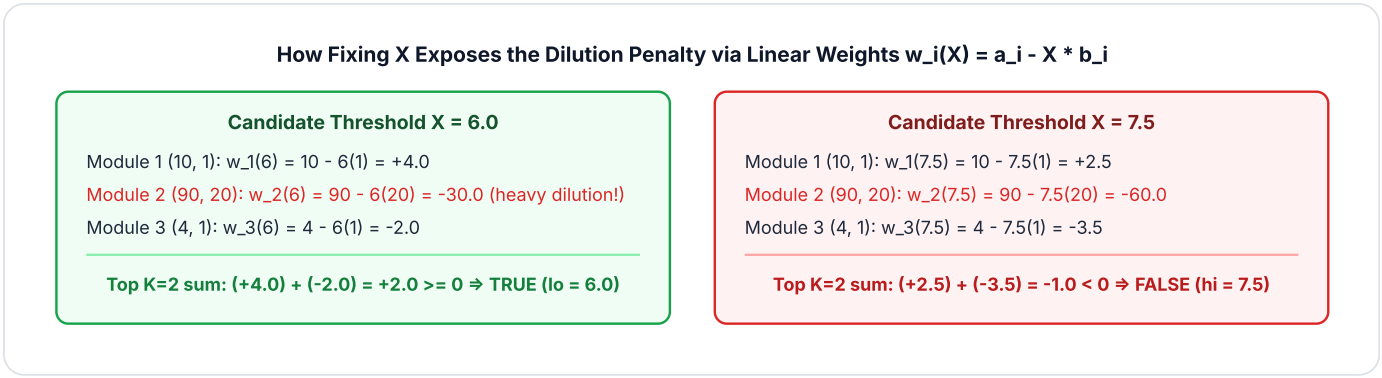
**Dinkelbach / Parametric Linearization Identity:**

$$\frac{\sum_{i \in S} a_i}{\sum_{i \in S} b_i} \geq X \Leftrightarrow \sum_{i \in S} a_i - X \sum_{i \in S} b_i \geq 0 \Leftrightarrow \sum_{i \in S} \underbrace{(a_i - X \cdot b_i)}_{w_i(X)} \geq 0$$

Once $X$ is fixed, each module $i$ has a single scalar **net surplus weight** $w_i(X) = a_i - Xb_i$. Now the problem inside $\text{check}(X)$ is completely linear and separable: 1. Compute $w_i(X) = a_i - Xb_i$ for all $N$ modules. 2. Sort $w_1(X) \geq w_2(X) \geq \ldots \geq w_N(X)$ descending. 3. To maximize $\sum_{i \in S} w_i(X)$ subject to $|S| \geq K$, we must definitely take the top $K$ modules (Plus any subsequent modules with $w_i(X) > 0$, though note: if the top $K$ already have sum $\geq 0$, check is already `true`; if the $K$-th largest weight is $\leq 0$, adding more modules only decreases the sum, and if the $K$-th largest weight is $> 0$, the top $K$ sum is strictly positive anyway! Thus **checking the sum of the top $K$ weights alone is both necessary and sufficient**).

### Step 4 — Progressive Construction & Visual Trace

Let us trace our 3-module counterexample $(10, 1), (90, 20), (4, 1)$ with $K = 2$ at $X = 6.0$ and $X = 7.0$:




### Step 5 — Two-Way Proof & Small-Case Oracle

**Why Checking Size Exactly $K$ Suffices for Feasibility of $|S| \geq K$:** - ( $\Leftarrow$ ) If the top $K$ weights have $\sum_{j=1}^{K} w_{(j)}(X) \geq 0$, that subset of size $K$ immediately satisfies $|S| \geq K$ and $\mathrm{SNR}(S) \geq X$. - ( $\Rightarrow$ ) Conversely, suppose some subset $S$ with $|S| = m > K$ satisfies $\sum_{i \in S} w_i(X) \geq 0$. The average of its $m$ weights is $\geq 0$, so the average of its $K$ largest weights is also $\geq 0$. Since the globally top $K$ weights are at least as large as the $K$ largest weights of $S$, the top $K$ sum is also $\geq 0$.

**Fixed-Iteration Real Binary Search Implementation**: Instead of `while (hi - lo > eps)` (which can loop infinitely due to floating-point precision limits when `lo` and `hi` are large), run a fixed loop of $70$ iterations, which halves the interval width by $2^{-70} < 10^{-21}$.

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <numeric>
#include <vector>

using namespace std;

double solve_fractional(int n, int k, const vector<long long>& a, const vector<long long>& b) {
    double lo = 0.0, hi = 1e6;
    vector<double> w(n);
    for (int iter = 0; iter < 70; ++iter) {
        double mid = 0.5 * (lo + hi);
        for (int i = 0; i < n; ++i) {
            w[i] = static_cast<double>(a[i]) - mid * static_cast<double>(b[i]);
        }
        nth_element(w.begin(), w.begin() + k, w.end(), greater<double>());
        double top_k_sum = accumulate(w.begin(), w.begin() + k, 0.0);
        if (top_k_sum >= 0.0) {
            lo = mid;
        } else {
            hi = mid;
        }
    }
    return lo;
}
```

---

# Section 9 — Greedy I: Local Choices and Exchange Arguments

## The Discovery & Modeling Approach

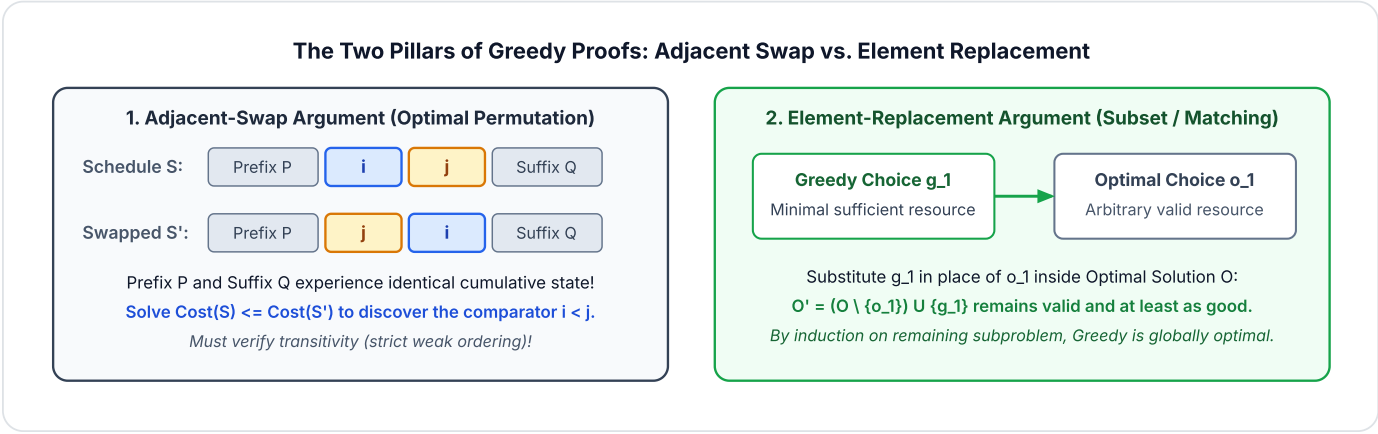
A greedy algorithm commits to one local decision at a time and never looks back. Because that commitment is permanent, **guess-and-pray sorting is the fastest way to fail a contest**. In competitive programming, a greedy algorithm is never accepted by intuition; it is discovered and verified by writing down its **Exchange Argument**.

### Recognizing When Greedy Exchange Applies

1. **Optimal Ordering of $N$ Tasks/Items**: Every item must be processed, and you want the permutation $\pi$ that minimizes total weighted completion time, maximum peak deficit, or cumulative penalty.
2. **Resource-to-Request Matching with 1D Dominance**: Each item has a single capability score (or two scores after sorting by one dimension), and we want to match as many requests as possible.
3. **Absence of Future Regret Trade-offs**: Unlike Knapsack (where a single heavy item blocks two medium items), items either have uniform size ($1$-for-$1$ replacement) or can be ordered so that swapping an inversion never hurts the prefix or suffix.

### Decision Compass: The Two Classic Exchange Proof Patterns

| Proof Pattern | Target Problem Type | Core Technique | How It Discovers the Algorithm |
|---|---|---|---|
| **Adjacent-Swap Exchange** | Finding the optimal **permutation** of all $N$ items | Compare cost of $(..., i, j, ...)$ vs $(..., j, i, ...)$ at an adjacent pair. Demand $\text{cost}(i, j) \leq \text{cost}(j, i)$ | Algebraically isolates item $i$'s terms on the left and item $j$'s terms on the right, **deriving the exact C++ comparator** |
| **Element-Replacement / Stays-Ahead** | Selecting an optimal **subset** or **matching** of size $K$ | Compare greedy choice $g_1$ against optimal choice $o_1$. Show $o_1$ can be replaced by $g_1$ without invalidating $O \backslash \{o_1\}$ | Reveals which item leaves the **most flexible residual state** for all remaining steps |




---

## Worked Problem 9.1: Thermal Reactor Calibration (Adjacent-Swap Comparator Derivation)

### Problem & Constraints

You must run $N$ diagnostic jobs sequentially on a fusion reactor. Before job $i$ can execute, the reactor core must briefly reach at least $r_i$ megawatts. Running job $i$ then permanently consumes $c_i$ megawatts of core energy ($1 \leq c_i \leq r_i \leq 10^9$). Find the **minimum initial core energy** $E_0$ required so that all $N$ jobs can be completed in some order without ever failing a job's peak requirement or dropping below $0$.

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq c_i \leq r_i \leq 10^9$

**Pause & Discover:**

- Try creating 2 jobs where sorting by $r_i$ descending fails, and 2 jobs where sorting by $c_i$ ascending fails.
- Write the exact minimum energy needed at the start of a two-job pair $(i, j)$ if $i$ runs before $j$, and compare it with running $j$ before $i$.

### Step 1 — Exact Model & Bottleneck

For a fixed permutation $\pi = (\pi_1, \pi_2, ..., \pi_N)$, let $C_k = \sum_{m=1}^{k-1} c_{\pi_m}$ be the energy already consumed before job $\pi_k$ starts. Job $\pi_k$ requires remaining energy $E_0 - C_k \geq r_{\pi_k}$, so:

$$E_0(\pi) = \max_{1 \leq k \leq N}(C_k + r_{\pi_k})$$

Checking all $N!$ permutations is impossible for $N > 10$, and subset DP over $2^N$ completed subsets fails for $N > 20$. We need a total ordering that minimizes $E_0(\pi)$ in $O(N \log N)$.

### Step 2 — Tempting Mistakes & Counterexample

1. **Tempting Mistake 1 (Sort by peak requirement $r_i$ descending)**:
   - **Counterexample**: Job A has $(r_A, c_A) = (10, 9)$, Job B has $(r_B, c_B) = (9, 1)$.
   - Order $(A, B)$: Need $10$ for A (leaves $1$), then B needs $9$, so initial energy must be $9 + 9 = 18$.
   - Order $(B, A)$: Need $9$ for B, and after spending $1$, A needs $10$, so initial energy is $1 + 10 = 11 < 18$!
2. **Tempting Mistake 2 (Sort by consumption $c_i$ ascending)**:
   - **Counterexample**: Job A has $(r_A, c_A) = (5, 2)$, Job B has $(r_B, c_B) = (10, 3)$.
   - Order $(A, B)$: Needs $\max(5, 2 + 10) = 12$.
   - Order $(B, A)$: Needs $\max(10, 3 + 5) = 10 < 12$!

### Step 3 — The Turn: Discovering the Structure via Algebra

Instead of guessing a heuristic, let us compare two adjacent jobs $i$ and $j$ after some prefix has already consumed $C$ units of energy. - In schedule $S = (..., i, j, ...)$, the peak initial energy required by the pair $(i, j)$ is:

$$E(i, j) = C + \max(r_i, c_i + r_j)$$

- In schedule $S' = (..., j, i, ...)$, the peak initial energy required by $(j, i)$ is:

$$E(j, i) = C + \max(r_j, c_j + r_i)$$

Notice that every job *before* $\{i, j\}$ sees consumption before $C$, and every job *after* $\{i, j\}$ sees cumulative consumption $C + c_i + c_j$ in both schedules! Thus swapping $i$ and $j$ affects **only** the local peak of these two jobs.

When is $E(i, j) \leq E(j, i)$? Since $r_i \leq c_j + r_i$ and $r_j \leq c_i + r_j$, the larger terms inside the maxima are dominated by the cross-sums. Specifically, write $r_i = c_i + d_i$ where $d_i = r_i - c_i$ is the **retained cushion** required after job $i$ finishes:

**Algebraic Derivation of the Comparator:**

$$\max(c_i + d_i,\ c_i + c_j + d_j) = c_i + c_j + \max(d_i - c_j,\ d_j)$$

Comparing $(i, j)$ against $(j, i)$ after subtracting $c_i + c_j$:

$$E(i, j) \leq E(j, i) \Leftrightarrow \max(d_i - c_j, d_j) \leq \max(d_j - c_i, d_i)$$

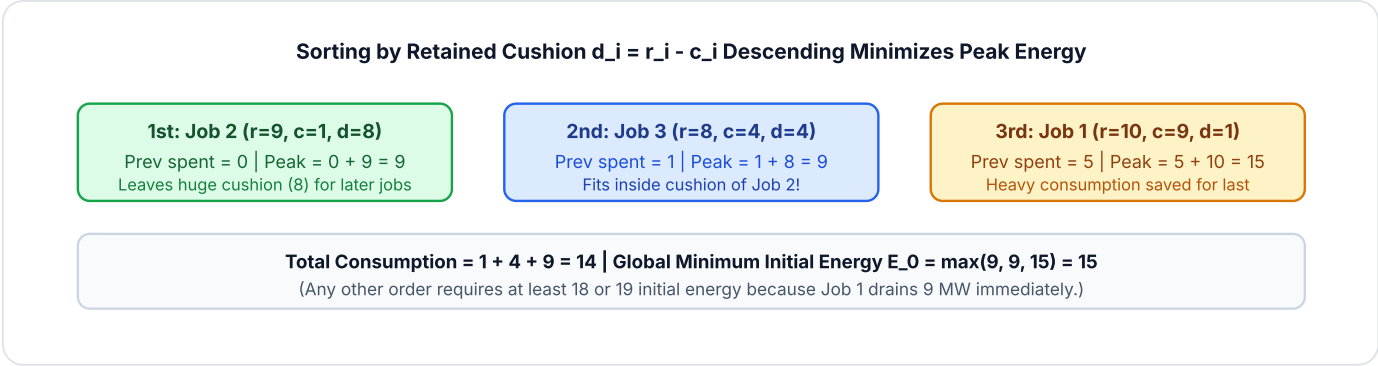
Since $c_i, c_j > 0$, this holds whenever $d_i \geq d_j$, i.e.,

$$r_i - c_i \geq r_j - c_j$$

Look how clean that is: sorting jobs in **descending order of retained cushion** $d_i = r_i - c_i$ is provably optimal!

### Step 4 — Progressive Construction & Visual Trace

Let us trace three jobs: $J_1 = (10, 9) \Longrightarrow d_1 = 1$, $J_2 = (9, 1) \Longrightarrow d_2 = 8$, $J_3 = (8, 4) \Longrightarrow d_3 = 4$. Sorted by $d_i = r_i - c_i$ descending, the order is $J_2(d = 8) \to J_3(d = 4) \to J_1(d = 1)$.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Transitivity & Global Optimality Proof**: 1. **Strict Weak Ordering**: Each job $i$ maps to a single scalar $d_i = r_i - c_i$. The relation $d_i > d_j$ is trivially irreflexive, asymmetric, and transitive, so `std::sort` produces a well-defined ordering. 2. **Bubble-Sort Reduction**: Let $\pi^*$ be any optimal permutation. If $\pi^*$ contains an inversion—a pair of adjacent elements $(u, v)$ with $d_u < d_v$—swapping $(u, v)$ to $(v, u)$ satisfies $\max(r_v, c_v + r_u) \leq \max(r_u, c_u + r_v)$ and leaves the prefix sum before $(u, v)$ and after $(u, v)$ unchanged. Thus the swap never increases $E_0(\pi^*)$. Since any permutation has at most $\binom{N}{2}$ inversions and each swap reduces the inversion count by $1$, a finite sequence of non-worsening swaps transforms $\pi^*$ into our sorted order.

```cpp
#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

using namespace std;

struct Job {
    long long r, c;
    int id;
};

long long solve_greedy(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end(), [](const Job& a, const Job& b) {
        long long da = a.r - a.c;
        long long db = b.r - b.c;
        if (da != db) return da > db;
        return a.id < b.id;
    });
    long long spent = 0, required_initial = 0;
    for (const auto& job : jobs) {
        required_initial = max(required_initial, spent + job.r);
        spent += job.c;
    }
    return required_initial;
}

// Small-case oracle: tests all N! permutations
long long solve_oracle(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end(), [](const Job& a, const Job& b) { return a.id < b.id; });
    long long best = 4e18;
    do {
        long long spent = 0, req = 0;
        for (const auto& job : jobs) {
            req = max(req, spent + job.r);
            spent += job.c;
        }
        best = min(best, req);
    } while (next_permutation(jobs.begin(), jobs.end(),
                              [](const Job& a, const Job& b) { return a.id < b.id; }));
    return best;
}
```

## Worked Problem 9.2: Single-Use Translation Coupons (Threshold Matching & Replacement Proof)

### Problem & Constraints

You have $N$ translation requests with required quality thresholds $q_1, q_2, ..., q_N$ and $M$ single-use translation coupons with quality ratings $c_1, c_2, ..., c_M$. Coupon $j$ can satisfy request $i$ if and only if $c_j \geq q_i$. Maximize the number of satisfied requests.

- $1 \leq N, M \leq 2 \cdot 10^5$
- $1 \leq q_i, c_j \leq 10^9$

**Pause & Discover:**

- If you sort requests ascending ($q_1 \leq ... \leq q_N$), should you satisfy the easiest request $q_1$ using the strongest coupon or the weakest coupon that satisfies $c_j \geq q_1$?
- What happens if a request $q_i$ is impossible to satisfy—can a naive top-down loop waste a coupon or get stuck?

### Step 1 — Exact Model & Bottleneck

This is a **Maximum Bipartite Matching** problem on the graph $G = (Q, C, E)$ where $(i, j) \in E \Leftrightarrow c_j \geq q_i$. Running general Hopcroft–Karp on a dense threshold graph with up to $N \cdot M = 4 \cdot 10^{10}$ edges takes $O(NM\sqrt{N+M})$ time, which is hopelessly slow. The bottleneck is ignoring that the compatibility relation $c_j \geq q_i$ is a **1D monotone threshold** (a Ferrers diagram / chain graph).

### Step 2 — Tempting Mistakes & Counterexample

**Tempting Mistake**: Match each request to the *first available* coupon in arbitrary order, or match the easiest request with the strongest coupon. - **Counterexample**: Let $Q = [5, 10]$ and $C = [6, 10]$. - If request $q_1 = 5$ takes coupon $c_2 = 10$, then request $q_2 = 10$ is left with $c_1 = 6 < 10$ and fails (1 match). - Matching $q_1 = 5$ with $c_1 = 6$ and $q_2 = 10$ with $c_2 = 10$ satisfies both (2 matches).

### Step 3 — The Turn: Discovering the Structure

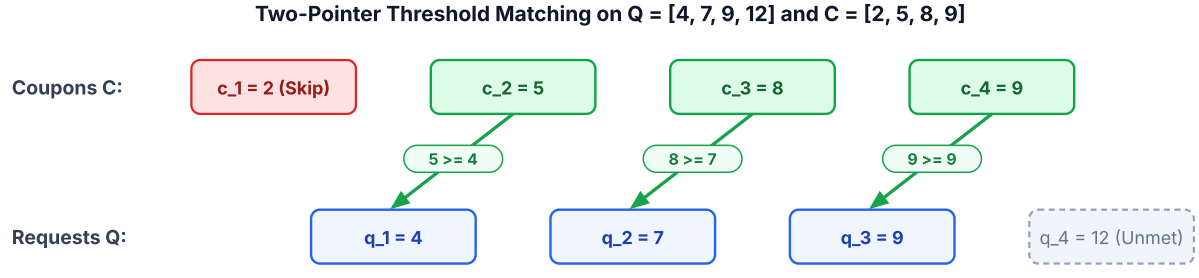
Sort both arrays in non-decreasing order: $q_1 \leq q_2 \leq ... \leq q_N$ and $c_1 \leq c_2 \leq ... \leq c_M$.

Consider two observations about the smallest remaining request $q_1$: 1. **Safe Discard Rule**: Any coupon $c_j < q_1$ is smaller than the easiest remaining request, so it can satisfy **no** remaining request in the entire problem. Discarding it loses nothing. 2. **Minimal Sufficient Resource Rule**: Once all coupons $< q_1$ are discarded, the smallest remaining coupon $c_j \geq q_1$ can satisfy $q_1$, and among all remaining coupons, it is the **least capable** for future harder requests $q_2 \leq ... \leq q_N$.

**Two-Pointer Threshold Matching Rule:** Maintain pointer $i$ on sorted requests $Q$ and pointer $j$ on sorted coupons $C$. If $c_j \geq q_i$, match $(q_i, c_j)$ and advance both $i$ and $j$; otherwise ($c_j < q_i$), discard $c_j$ by advancing $j$ only.

### Step 4 — Progressive Construction & Visual Trace

Let $Q = [4, 7, 9, 12]$ and $C = [2, 5, 8, 9]$.



### Step 5 — Two-Way Proof & Small-Case Oracle

**Element-Replacement Proof**: Let $(q_1, c_{j^*})$ be the first pair matched by our greedy algorithm, where $j^*$ is the smallest index with $c_{j^*} \geq q_1$. Let $O$ be any maximum matching. - **Case 1 ($q_1$ is unmatched in $O$)**: Look at the smallest request $q_r$ matched in $O$, say to coupon $c_k$. Since $q_1 \leq q_r \leq c_k$, replacing $q_r$ with $q_1$ gives a valid matching of the same size that includes $q_1$. - **Case 2 ($q_1$ is matched in $O$ to some coupon $c_k$ with $k > j^*$)**: Since $c_{j^*}$ is the first coupon $\geq q_1$, we have $q_1 \leq c_{j^*} \leq c_k$. If $c_{j^*}$ is unused in $O$, simply replace $(q_1, c_k)$ with $(q_1, c_{j^*})$. If $c_{j^*}$ was matched in $O$ to some request $q_r$ $(r > 1)$, swap their partners to get $(q_1, c_{j^*})$ and $(q_r, c_k)$; both are valid because $c_{j^*} \geq q_1$ and $c_k \geq c_{j^*} \geq q_r$. Thus there always exists a maximum matching containing the greedy choice $(q_1, c_{j^*})$.

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

int solve_coupons(vector<long long> q, vector<long long> c) {
    sort(q.begin(), q.end());
    sort(c.begin(), c.end());
    int i = 0, j = 0, matched = 0;
    int n = static_cast<int>(q.size()), m = static_cast<int>(c.size());
    while (i < n && j < m) {
        if (c[j] >= q[i]) {
            matched++;
            i++;
            j++;
        } else {
            j++;
        }
    }
    return matched;
}
```

---

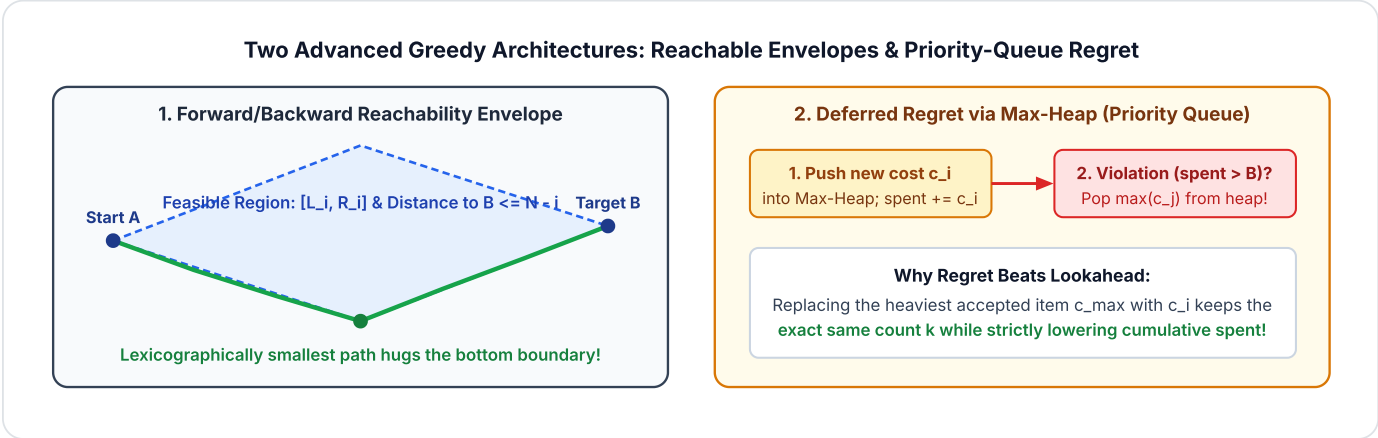
# Section 10 — Greedy II: Invariants and Constructive Problems

## The Discovery & Modeling Approach

Simple greedy algorithms commit immediately and never look back. Advanced greedy and constructive problems add two new twists: 1. **Constructive Sequence / Lexicographic Synthesis**: You must build a valid sequence $x_1, x_2, ..., x_N$ step by step (often lexicographically smallest), where a locally greedy step at index $i$ can render the suffix $i+1...N$ **impossible to complete**. 2. **Deferred Regret (Buy-Now, Regret-Later)**: You scan through events with a cumulative resource invariant (like health, fuel, or time $\geq 0$) and want to maximize the number of accepted tasks. Committing permanently is unsafe, so you tentatively accept items and use a **priority queue** to retroactively revoke the single worst past decision whenever the invariant would break.

### Decision Compass: Constructive Reachability vs. Regret Greedy

| Sub-Pattern | Signal in Problem Statement | Core Invariant Maintained | Algorithmic Blueprint |
|---|---|---|---|
| **Forward Envelope + Backward Peeling** | "Construct a valid sequence of length $N$ starting at $A$ and ending at $B$" | Reachable interval $[L_i, R_i]$ (plus parity if steps are $\pm 1$) at each index $i$ | Pass 1: Compute reachable $[L_i, R_i]$; Pass 2: Pick best valid $x_i$ that stays inside $[L_i, R_i]$ |
| **Lexicographically Smallest Valid Sequence** | "Among all valid configurations, output the lexicographically smallest" | Exact **Completion Oracle** $\text{can_finish}(i, \text{state})$ | At each $i = 1...N$, try smallest candidate $v$ for which $\text{can_finish}(i, v)$ is `true` |
| **Regret Greedy (Priority Queue Undo)** | "Maximize tasks taken left-to-right while keeping running balance $\geq 0$ (or time $\leq d_i$)" | Among all valid subsets of size $k$ in prefix $1...i$, ours has the **maximum remaining balance** | Take task $i$; if balance $< 0$, pop and undo the most expensive task taken so far |




---

## Worked Problem 10.1: Lexicographically Smallest Altitude Plan

### Problem & Constraints

A drone must fly across $N$ waypoints $(1...N)$. Its altitude at waypoint $1$ is fixed at $h_1 = A$, and its altitude at waypoint $N$ must be $h_N = B$. At each step $i \to i+1$, the altitude must change by **at least** $-D$ **and at most** $+U$ (or in the classic unit-step variant, belong to an allowed step set such as $\{-1, +1\}$), while staying within the safe corridor $[L_i, R_i]$ at every waypoint $i$. Among all valid altitude sequences $(h_1, h_2, ..., h_N)$, output the **lexicographically smallest**, or report `IMPOSSIBLE`.

- $2 \leq N \leq 2 \cdot 10^5$
- $0 \leq L_i \leq R_i \leq 10^9, 0 \leq D, U \leq 10^9$

**Pause & Discover:**

- If you greedily set $h_{i+1} = \max(L_{i+1}, h_i - D)$ at every step from 1 to $N$, why might you fail to reach $h_N = B$ at the end?
- What exact condition at waypoint $i$ guarantees that a candidate altitude $h_i$ can still reach $h_N = B$ through all future corridors $[L_j, R_j]$ $(j > i)$?

### Step 1 — Exact Model & Bottleneck

A sequence $(h_1, ..., h_N)$ is valid if and only if: 1. $h_1 = A$, $h_N = B$, and $L_i \le h_i \le R_i$ for all $1 \le i \le N$. 2. $h_i - D \le h_{i+1} \le h_i + U$ for all $1 \le i < N$.

To make $(h_1, ..., h_N)$ lexicographically smallest, we want $h_1$ as small as possible, then $h_2$ as small as possible given $h_1$, and so on—**subject to the constraint that** $(h_{i+1}, ..., h_N)$ **still admits at least one valid completion to** $B$. Checking whether $h_i$ can complete to $B$ by simulating the suffix $i+1...N$ takes $O(N)$ per step, yielding an $O(N^2)$ bottleneck.

### Step 2 — Tempting Mistakes & Counterexample

**Tempting Mistake**: Dive as low as allowed by $[L_i, R_i]$ and check only whether the remaining distance to $B$ satisfies $h_i + (N-i)U \ge B$, ignoring intermediate bottlenecks $[L_j, R_j]$ (or ignoring parity when steps are strictly $\pm 1$).

**Concrete 4-Waypoint Counterexample ($N = 4, D = 2, U = 2, A = 4, B = 5$):** - Corridor bounds: $[L_1, R_1] = [4, 4]$, $[L_2, R_2] = [0, 10]$, $[L_3, R_3] = [0, 2]$, $[L_4, R_4] = [5, 5]$. - Notice that any valid path must pass through $[L_3, R_3] = [0, 2]$ at step 3 and then climb to $h_4 = 5$ at step 4. Since $U = 2$, the maximum altitude reachable at step 4 from $h_3 \le 2$ is $2 + 2 = 4 < 5$. The instance is actually `IMPOSSIBLE`, yet checking only $h_2 + 2U \ge 5$ at step 2 fails to detect that corridor 3 chokes off every path!

### Step 3 — The Turn: Discovering the Structure (Backward Feasibility Envelope)

Because the transition $[h_i - D, h_i + U]$ and the corridor $[L_i, R_i]$ are both **contiguous intervals**, the set of altitudes at waypoint $i$ that can validly reach $h_N = B$ through waypoints $i...N$ is itself a single contiguous interval $[lo_i, hi_i]$!

We can compute $[lo_i, hi_i]$ from **right to left** ($i = N$ down to 1): - At $i = N$: $[lo_N, hi_N] = [B, B] \cap [L_N, R_N]$. - Moving backward from $i+1$ to $i$: An altitude $h_i$ can step into $[lo_{i+1}, hi_{i+1}]$ if and only if $[h_i - D, h_i + U]$ intersects $[lo_{i+1}, hi_{i+1}]$, which means $h_i + U \ge lo_{i+1}$ and $h_i - D \le hi_{i+1}$.

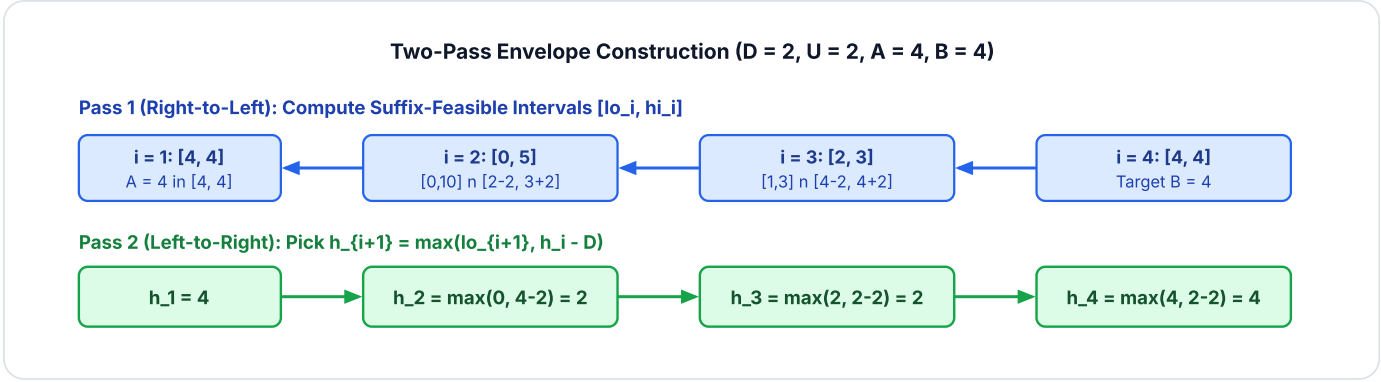
**Backward Reachable Envelope & Forward Greedy Peeling:**

- **Backward Pass ($i = N-1$ down to 1):**
  $$lo_i = \max(L_i,\ lo_{i+1} - U), \qquad hi_i = \min(R_i,\ hi_{i+1} + D)$$
  If $lo_i > hi_i$ at any step, or $A \notin [lo_1, hi_1]$, return `IMPOSSIBLE`.
- **Forward Lexicographic Pass ($i = 1$ to $N-1$):** Having committed to $h_i \in [lo_i, hi_i]$, the smallest valid choice for $h_{i+1}$ is:
  $$h_{i+1} = \max(lo_{i+1},\ h_i - D)$$

*(Note: In the strict $\pm 1$ step variant where $h_{i+1} - h_i \in \{-1, +1\}$, every altitude in $[lo_i, hi_i]$ with the same parity as $A + (i-1)$ is reachable!)*

### Step 4 — Progressive Construction & Visual Trace

Let us trace $N = 4$, $D = 2, U = 2$, $A = 4, B = 4$ with corridors $[4, 4], [0, 10], [1, 3], [4, 4]$:




### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of Feasibility and Lexicographic Optimality**: 1. **Inductive Completeness**: By backwards induction on $i$, an altitude $x \in [L_i, R_i]$ has a valid continuation to $h_N = B$ if and only if $[x - D, x + U] \cap [lo_{i+1}, hi_{i+1}] \neq \emptyset$, which is equivalent to $x \in [lo_i, hi_i]$. 2. **Lexicographic Optimality**: Since $h_i \in [lo_i, hi_i]$, the interval $[h_i - D, h_i + U] \cap [lo_{i+1}, hi_{i+1}]$ is non-empty, and its minimum element is $\max(lo_{i+1}, h_i - D)$. Because any smaller value for $h_{i+1}$ either violates the step bound $h_{i+1} \geq h_i - D$ or falls outside $[lo_{i+1}, hi_{i+1}]$ (having zero valid completions), no valid sequence can be smaller at the first index of difference.

```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

vector<long long> solve_altitude(int n, long long D, long long U,
                                 long long A, long long B,
                                 const vector<long long>& L,
                                 const vector<long long>& R) {
    vector<long long> lo(n), hi(n);
    lo[n - 1] = max(L[n - 1], B);
    hi[n - 1] = min(R[n - 1], B);
    if (lo[n - 1] > hi[n - 1]) return {};

    for (int i = n - 2; i >= 0; --i) {
        lo[i] = max(L[i], lo[i + 1] - U);
        hi[i] = min(R[i], hi[i + 1] + D);
        if (lo[i] > hi[i]) return {};
    }
    if (A < lo[0] || A > hi[0]) return {};

    vector<long long> h(n);
    h[0] = A;
    for (int i = 0; i + 1 < n; ++i) {
        h[i + 1] = max(lo[i + 1], h[i] - D);
    }
    return h;
}
```

---

## Worked Problem 10.2: Emergency Coolant Dispatch (Deferred Regret & Priority-Queue Reversal)

### Problem & Constraints

A server cluster processes a stream of $N$ time-critical tasks. Task $i$ requires $t_i > 0$ seconds of uninterrupted CPU time and must finish at or before deadline $d_i$. You can execute at most one task at a time. Maximize the number of tasks completed on time.

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq t_i, d_i \leq 10^9$

**Pause & Discover:**

- Given a fixed subset of tasks $S$, in what order should you execute them to give every task in $S$ the best chance of meeting its deadline?
- Once tasks are sorted by deadline $d_i$ ascending, why is neither "take every task that fits" nor "sort by duration $t_i$ ascending" sufficient on its own?

### Step 1 — Exact Model & Bottleneck

First, apply an adjacent-swap argument to any subset $S$ of tasks that can all be completed on time: if two chosen tasks have $d_i \le d_j$ and we run $j$ before $i$, swapping them so the earlier deadline $d_i$ runs first reduces $i$'s finish time and sets $j$'s finish time to $\le$ the old finish time of $i \le d_i \le d_j$. Thus **every valid subset $S$ can be executed in ascending order of deadline $d_i$**.

After sorting all $N$ tasks so $d_1 \le d_2 \le \ldots \le d_N$, an exact DP state would be $\mathrm{dp}[i][k]$: the minimum total CPU time needed to complete $k$ tasks chosen from the prefix $1 \ldots i$. Computing $\mathrm{dp}[i][k]$ for all $1 \le k \le i \le N$ takes $O(N^2)$ time—too slow for $N = 2 \cdot 10^5$.

### Step 2 — Tempting Mistakes & Counterexample

1. **Tempting Mistake 1 (Pure Earliest-Deadline-First without Regret)**: Sort by $d_i$ ascending and greedily accept task $i$ whenever $\mathrm{time} + t_i \le d_i$.
   - **Counterexample**: Task 1 has $(t_1, d_1) = (8, 8)$, Task 2 has $(t_2, d_2) = (4, 9)$, Task 3 has $(t_3, d_3) = (4, 10)$.
   - Accepting Task 1 sets $\mathrm{time} = 8$, blocking both Task 2 ($8 + 4 > 9$) and Task 3 ($8 + 4 > 10$), completing only $1$ task. Dropping Task 1 completes Tasks 2 and 3 ($2$ tasks)!
2. **Tempting Mistake 2 (Sort by duration $t_i$ ascending)**:
   - **Counterexample**: Task 1 has $(t_1, d_1) = (3, 3)$, Task 2 has $(t_2, d_2) = (2, 5)$. Running the shorter Task 2 first finishes at $t = 2$, leaving $2 + 3 = 5 > 3$ for Task 1 ($1$ task instead of $2$).

### Step 3 — The Turn: Discovering the Structure (Moore–Hodgson Regret Algorithm)

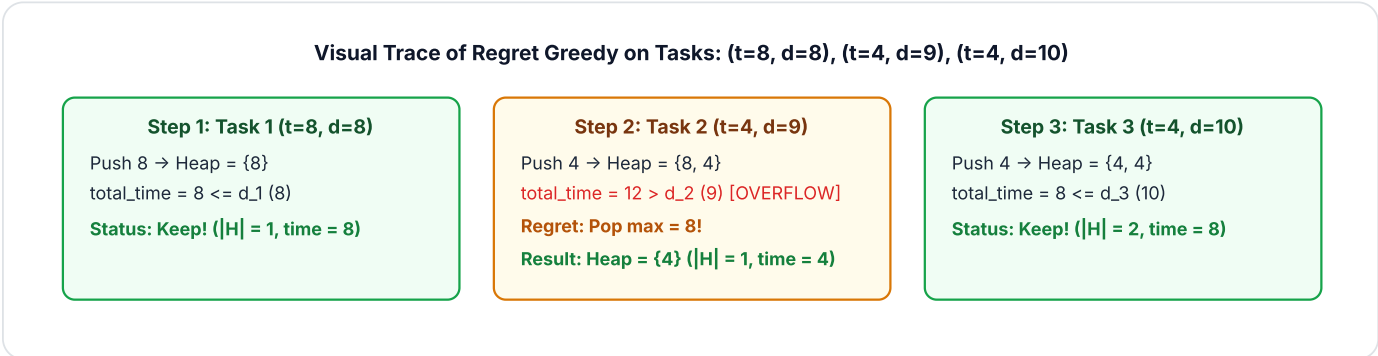
Notice that in the $O(N^2)$ DP $\mathrm{dp}[i][k]$, at each step $i$, the maximum achievable count $k^*$ either stays the same or increases by $1$. Instead of maintaining the entire table $\mathrm{dp}[i][1 \ldots k^*]$, what if we maintain **only the single subset $S$ of maximum size $k^*$ that also achieves the minimum possible total duration** $\sum_{j \in S} t_j$?

When we consider task $i$ (in ascending order of $d_i$): 1. Tentatively add task $i$ to $S$ (push $t_i$ into a max-heap and add $t_i$ to `total_time`). 2. If `total_time <= d[i]`, task $i$ fits! Our count $k^*$ increases by $1$. 3. If `total_time > d[i]`, we cannot keep all $k^* + 1$ tasks. Which single task in $S \cup \{i\}$ should we eject so that the remaining $k^*$ tasks are valid and have the **smallest possible** `total_time`? The one with the **largest duration** $t_{\max}$!

**Regret Invariant:** After processing prefix $1 \ldots i$ (sorted by $d_1 \le \ldots \le d_N$), the max-heap $H$ holds a valid subset of tasks of the **maximum possible cardinality** $k^*$, and among all valid subsets of cardinality $k^*$ in $1 \ldots i$, $H$ achieves the **minimum possible total duration** $\sum_{x \in H} t_x$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace Tasks $(8, 8), (4, 9), (4, 10)$:



### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of the Regret Swap**: When `total_time > d[i]`, why is the remaining set $H^{'} = (H \cup \{i\}) \backslash \{m\}$ (where $m = \text{argmax}_{x \in H \cup \{i\}} t_x$) guaranteed to be valid? - If $m = i$, then $H^{'} = H$, which was already valid before step $i$. - If $m \in H$, then $t_i \leq t_m$. Every task in $H \backslash \{m\}$ finished on time before step $i$, and removing $m$ only moves earlier tasks in $H$ to the left (decreasing their finish times). Task $i$ finishes at the end of $H^{'}$ at time $\text{total_time}(H) + t_i - t_m \leq \text{total_time}(H) \leq d_{i-1} \leq d_i$. And since we removed the maximum duration $t_m$, $H^{'}$ has the minimum possible duration sum among all swaps of size $|H|$.

```cpp
#include <algorithm>
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

struct Task {
    long long t, d;
};

int solve_coolant_dispatch(vector<Task> tasks) {
    sort(tasks.begin(), tasks.end(), [](const Task& a, const Task& b) {
        return a.d < b.d;
    });
    priority_queue<long long> pq;
    long long total_time = 0;
    for (const auto& task : tasks) {
        pq.push(task.t);
        total_time += task.t;
        if (total_time > task.d) {
            total_time -= pq.top();
            pq.pop();
        }
    }
    return static_cast<int>(pq.size());
}
```

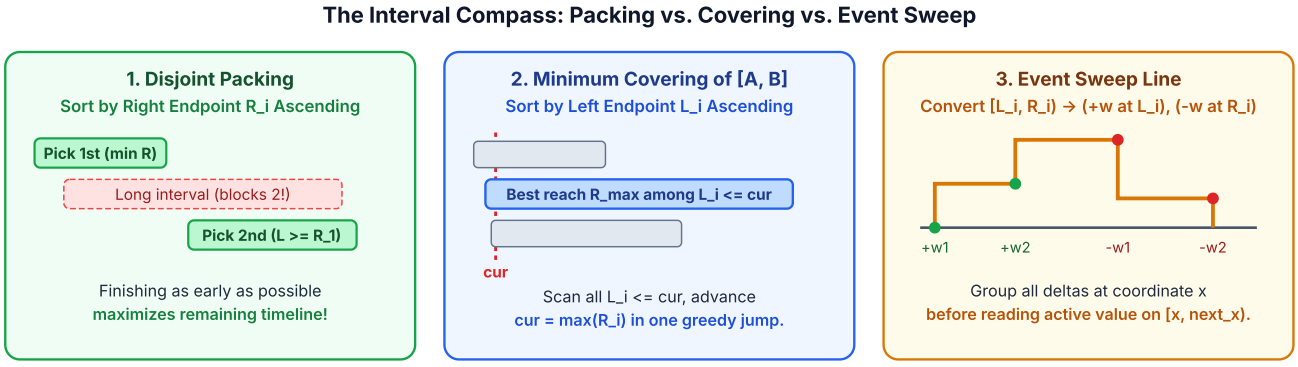
# Section 11 — Intervals and Sweep Line

## The Discovery & Modeling Approach

Whenever a problem gives pairs $[L_i, R_i]$ representing time spans, spatial segments, or value ranges, two geometric questions arise immediately: 1. **Selection by Endpoint**: Are we choosing a subset of intervals to be **mutually disjoint** (packing) or to **cover a target span** $[A, B]$ (covering)? Each goal dictates a specific endpoint sort! 2. **Continuous Coordinate to Discrete Event Reduction (Sweep Line)**: If coordinates reach $10^9$ or $10^{18}$, simulating every integer point $x$ is impossible. Yet the set of active intervals changes **only at the $2N$ interval endpoints**. Replacing a $10^9$-unit timeline with a sorted list of $2N$ events (`+w` at $L_i$, `-w` at $R_i$) collapses an infinite line into $O(N \log N)$ transitions.

### Decision Compass: Interval Sorting & Sweep-Line Paradigms

| Sub-Pattern | Goal / Question | Correct Sort Order | Crucial Endpoint Detail |
|---|---|---|---|
| **Max Disjoint Interval Packing** | Pick maximum number of mutually non-overlapping intervals | **Right endpoint $R_i$ ascending** | Leaves the timeline open as early as possible ($R_{\min}$) for future intervals |
| **Minimum Interval Covering** | Cover $[A, B]$ using the minimum number of intervals | **Left endpoint $L_i$ ascending** | Among all intervals starting $\le \text{cur}$, greedily pick the one with **maximum** $R_i$ |
| **Interval Merging / Union Length** | Merge all overlapping intervals into disjoint components | **Left endpoint $L_i$ ascending** (ties by $R_i$ descending) | Maintain `cur_L, cur_R`; if $L_i \le \text{cur_R}$, set $\text{cur_R} = \max(\text{cur_R}, R_i)$ |
| **Sweep Line (Point Overlap / Congestion)** | Maximum weight active at any single point $x$ | **Event coordinate $x$ ascending** | Closed $[L, R]$: process `+w` **before** `-w` at equal $x$; Half-open $[L, R)$: process `-w` **before** `+w` (or group equal $x$) |




---

## Worked Problem 11.1: Disjoint Interval Packing vs. Minimum Interval Cover

### Problem & Constraints

You are given $N$ intervals $[L_i, R_i]$ ($L_i < R_i$) and a target range $[A, B]$. Solve two dual tasks on the same dataset: 1. **Task A (Disjoint Packing)**: Find the **maximum number of mutually disjoint** intervals you can select (two intervals $[L_i, R_i]$ and $[L_j, R_j]$ are disjoint if they share at most an endpoint, i.e., $R_i \le L_j$ or $R_j \le L_i$). 2. **Task B (Interval Covering)**: Find the **minimum number of intervals** whose union completely covers $[A, B]$, or report `-1` if impossible.

- $1 \le N \le 2 \cdot 10^5$
- $0 \le L_i < R_i \le 10^9, 0 \le A < B \le 10^9$

**Pause & Discover:**

- In Task A (Disjoint Packing), why does sorting by left endpoint $L_i$ ascending fail, and why does sorting by shortest length $(R_i - L_i)$ ascending also fail?
- In Task B (Interval Covering), why must we switch to sorting by left endpoint $L_i$ ascending?

### Step 1 — Exact Model & Bottleneck

Both tasks can be modeled as shortest/longest path problems on a DAG of intervals after sorting, requiring $O(N^2)$ transitions if we test all predecessors $j < i$. To reduce $O(N^2)$ to $O(N \log N)$, we must identify the **single dominating transition** at each step.

### Step 2 — Tempting Mistakes & Counterexample

1. **Tempting Mistake 1 (Sorting by $L_i$ ascending for Disjoint Packing)**:
   - **Counterexample**: $[1, 10], [2, 4], [5, 8]$. The earliest-starting interval $[1, 10]$ spans the entire timeline and blocks both $[2, 4]$ and $[5, 8]$ ($1$ interval instead of $2$).
2. **Tempting Mistake 2 (Sorting by length $R_i - L_i$ ascending for Disjoint Packing)**:
   - **Counterexample**: $[1, 5], [4, 7], [6, 10]$. The shortest interval is $[4, 7]$ (length $3$), which overlaps both $[1, 5]$ and $[6, 10]$ (lengths $4$ each), yielding $1$ interval instead of $2$!
3. **Tempting Mistake 3 (Sorting by $R_i$ descending for Covering)**:
   - Taking the interval with the largest $R_i$ globally may leave a gap on the left of $[A, B]$ that still requires extra intervals.

### Step 3 — The Turn: Discovering the Two Dual Structures

- **For Task A (Packing)**: Consider the very first interval in any valid disjoint set. All subsequent intervals must start at or after its right endpoint $R_1$. Therefore, choosing the interval with the **minimum possible right endpoint** $R_{\min}$ maximizes the remaining timeline $[R_{\min}, \infty)$ available to all future choices!
- **For Task B (Covering $[A, B]$)**: Suppose we have already covered $[A, \text{cur}]$ completely. The next interval we pick **must** satisfy $L_i \leq \text{cur}$ (otherwise a gap $(\text{cur}, L_i)$ remains uncovered). Among all intervals satisfying $L_i \leq \text{cur}$, which one leaves the smallest remaining suffix $[\text{new_cur}, B]$? The one with the **maximum right endpoint** $R_i$!

**Dual Endpoint Laws:**

- **Disjoint Packing**: Sort by $R_i$ ascending. Maintain $\text{last_R} = -\infty$; whenever $L_i \geq \text{last_R}$, take interval $i$ and set $\text{last_R} = R_i$.
- **Minimum Covering**: Sort by $L_i$ ascending. Maintain covered frontier $\text{cur} = A$. In each step, advance pointer $i$ through all intervals with $L_i \leq \text{cur}$, tracking $\text{best_R} = \max R_i$. If $\text{best_R} \leq \text{cur}$, coverage is impossible; otherwise set $\text{cur} = \text{best_R}$ and increment count until $\text{cur} \geq B$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace both algorithms on $[1, 10], [1, 4], [3, 7], [5, 8], [7, 11]$ with target $[A, B] = [1, 11]$:

**Same 5 Intervals, Two Opposite Objectives: [1,10], [1,4], [3,7], [5,8], [7,11]**

**Task A: Disjoint Packing (Sort by R_i Ascending)**

1. [1, 4] (R=4) → Pick! last_R = 4
2. [3, 7] (R=7) → Skip (L=3 < 4)
3. [5, 8] (R=8) → Pick! (L=5 >= 4, last_R = 8)
4. [1, 10] (R=10) → Skip (L=1 < 8)
5. [7, 11] (R=11) → Skip (L=7 < 8)

**Result: 2 Disjoint Intervals {[1,4], [5,8]}**

**Task B: Cover [1, 11] (Sort by L_i Ascending)**

Step 1 (cur = 1): Candidates with L_i <= 1 are
[1, 4] and [1, 10] → Pick [1, 10]! (cur = 10)

Step 2 (cur = 10): Candidates with L_i <= 10 are
[3, 7], [5, 8], [7, 11] → Pick [7, 11]! (cur = 11)

**Result: 2 Intervals Cover [1, 11] {[1,10], [7,11]}**

### Step 5 — Two-Way Proof & Implementation

**Stays-Ahead Proofs**: - **Task A**: If greedy picks $g_1, ..., g_k$ and any disjoint set picks $o_1, ..., o_m$, then $R(g_1) \leq R(o_1)$ by definition of minimum $R$. Assuming $R(g_{j-1}) \leq R(o_{j-1})$, we have $L(o_j) \geq R(o_{j-1}) \geq R(g_{j-1})$, so $o_j$ is a valid candidate for $g_j$, meaning $R(g_j) \leq R(o_j)$. Thus greedy never runs out of room before $O$, proving $k \geq m$. - **Task B**: By induction on step $j$, the frontier $\mathrm{cur}_j$ reached after $j$ greedy intervals is at least the frontier reached by the first $j$ intervals of any valid cover sorted by left endpoint.

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

struct Interval {
    long long l, r;
};

int max_disjoint_packing(vector<Interval> a) {
    sort(a.begin(), a.end(), [](const Interval& x, const Interval& y) {
        if (x.r != y.r) return x.r < y.r;
        return x.l < y.l;
    });
    int count = 0;
    long long last_r = -4e18;
    for (const auto& iv : a) {
        if (iv.l >= last_r) {
            count++;
            last_r = iv.r;
        }
    }
    return count;
}

int min_interval_cover(vector<Interval> a, long long A, long long B) {
    sort(a.begin(), a.end(), [](const Interval& x, const Interval& y) {
        if (x.l != y.l) return x.l < y.l;
        return x.r > y.r;
    });
    long long cur = A;
    int i = 0, n = static_cast<int>(a.size()), used = 0;
    while (cur < B) {
        long long best_r = cur;
        while (i < n && a[i].l <= cur) {
            best_r = max(best_r, a[i].r);
            i++;
        }
        if (best_r == cur) return -1; // Gap cannot be bridged
        cur = best_r;
        used++;
    }
    return used;
}
```

---

## Worked Problem 11.2: Weighted Radio Congestion (Event Sweep Line & Endpoint Tie-Breaking)

### Problem & Constraints

There are $N$ radio broadcasts. Broadcast $i$ transmits with power $w_i > 0$ over the time interval $[L_i, R_i)$. Find: 1. The **maximum total active power** at any real instant $t$, and the **earliest coordinate** $t$ at which this maximum begins. 2. Explain clearly how the answer changes if the intervals are **closed** $[L_i, R_i]$ instead of half-open $[L_i, R_i)$.

- $1 \leq N \leq 2 \cdot 10^5$

- $0 \le L_i < R_i \le 10^{18}, 1 \le w_i \le 10^9$

**Pause & Discover:**

- Suppose Broadcast 1 is on $[1, 5)$ with $w_1 = 10$ and Broadcast 2 is on $[5, 9)$ with $w_2 = 10$. What is the true maximum power on $[1, 9)$?
- What happens if you sort events at $t = 5$ with `+10` before `-10` and update the maximum right between them?

### Step 1 — Exact Model & Bottleneck

Let $P(t) = \sum_{i:\, t \in I_i} w_i$ be the total power at coordinate $t$. Since coordinates reach $10^{18}$, allocating a difference array of size $\max R_i$ is impossible. However, $P(t)$ is a **piecewise-constant step function** whose value can only change at the $2N$ coordinates $\{L_1, R_1, ..., L_N, R_N\}$.

### Step 2 — Tempting Mistakes & Counterexample

**Tempting Mistake**: Create events `(L_i, +w_i)` and `(R_i, -w_i)`, sort them, and update `ans = max(ans, active)` after **every single event** without grouping equal coordinates (or using the wrong tie-breaker between `+w` and `-w`).

**Concrete 2-Broadcast Counterexample ($N = 2$)**: - Broadcast 1: $[1, 5)$ with weight $w_1 = 10$. - Broadcast 2: $[5, 9)$ with weight $w_2 = 10$. - Under **half-open** semantics $[L_i, R_i)$, Broadcast 1 is already off at $t = 5$ when Broadcast 2 turns on. Thus $P(t) = 10$ for all $t \in [1, 9)$. - If we process `(5, +10)` before `(5, -10)` and update `ans` in between, `active` momentarily hits $10 + 10 = 20$ at a phantom zero-length transition! Conversely, under **closed** semantics $[L_i, R_i]$, both broadcasts *are* simultaneously active at the exact point $t = 5$, so the maximum on closed intervals really is $20$!

### Step 3 — The Turn: Discovering the Structure (Coordinate Grouping vs. Tie-Breaking)

There are two crystal-clear ways to handle equal event coordinates without ever making an off-by-one mistake:

**Rule 1 — Half-Open Intervals $[L_i, R_i)$ (Coordinate Grouping):** Sort all events $(x, \Delta)$ by coordinate $x$ ascending. At each distinct coordinate $x_k$, **sum all $\Delta$ values sharing $x_k$ first**, and only then evaluate `active` as the true constant power on the non-empty Elementary Interval $[x_k, x_{k+1})$.

**Rule 2 — Closed Intervals $[L_i, R_i]$ (Start-Before-End Tie-Breaking):** At a single point $x$, any interval starting at $x$ ($L_i = x$) and any interval ending at $x$ ($R_j = x$) overlap at point $x$ itself! To capture the peak at point $x$, either:

- Convert closed $[L_i, R_i]$ on integers to half-open $[L_i, R_i + 1)$ and use Rule 1, OR
- Sort events at equal $x$ with **start events ( `+w_i` ) before end events ( `-w_j` )**.

### Step 4 — Progressive Construction & Visual Trace

Let us contrast Half-Open $[L_i, R_i)$ vs. Closed $[L_i, R_i]$ on $[1, 5, w = 10], [3, 7, w = 5], [5, 8, w = 12]$:

**Endpoint Semantics at t = 5: [1,5, w=10], [3,7, w=5], [5,8, w=12]**

**Half-Open [L_i, R_i): Group All Deltas at x**

x = 1: delta = +10 → active on [1, 3) is 10

x = 3: delta = +5 → active on [3, 5) is 15

**x = 5: delta = -10 + 12 → active on [5, 7) is 17 (MAX!)**

x = 7: delta = -5 → active on [7, 8) is 12

x = 8: delta = -12 → active on [8, inf) is 0

**Max Power = 17, Earliest Start t = 5**

**Closed [L_i, R_i]: Starts (+w) Before Ends (-w) at x**

At x = 5, [1, 5] has not ended yet when [5, 8] starts!

1. Apply start at x = 5: (+12) → active = 15 + 12 = 27

**Instantaneous Peak at point x = 5 is 27 (MAX!)**

2. Apply end at x = 5: (-10) → active on (5, 7) is 17

**Max Power = 27, Achieved at Point t = 5**

### Step 5 — Two-Way Proof & Implementation

**Elementary Interval Invariant**: Let $x_1 < x_2 < \ldots < x_m$ be the sorted unique coordinates among all $L_i$ and $R_i$. Every half-open interval $[L_i, R_i)$ is the disjoint union of elementary intervals $[x_k, x_{k+1})$ for $L_i \le x_k < R_i$. After accumulating all $\Delta$ values at $x_1, \ldots, x_k$, the prefix sum $\sum_{r=1}^{k} \Delta(x_r)$ adds $+w_i$ if $L_i \le x_k$ and $-w_i$ if $R_i \le x_k$, which contributes $w_i$ if and only if $L_i \le x_k < R_i$.

```cpp
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>

using namespace std;

struct Event {
    long long x;
    long long delta;
};

// Solves the half-open [L_i, R_i) maximum power and earliest time t
pair<long long, long long> solve_half_open_sweep(const vector<pair<long long, long long>>& intervals,
                                                 const vector<long long>& w) {
    vector<Event> ev;
    ev.reserve(intervals.size() * 2);
    for (size_t i = 0; i < intervals.size(); ++i) {
        ev.push_back({intervals[i].first, +w[i]});
        ev.push_back({intervals[i].second, -w[i]});
    }
    sort(ev.begin(), ev.end(), [](const Event& a, const Event& b) {
        return a.x < b.x;
    });

    long long active = 0, best_power = 0, best_time = 0;
    size_t i = 0;
    while (i < ev.size()) {
        long long coord = ev[i].x;
        while (i < ev.size() && ev[i].x == coord) {
            active += ev[i].delta;
            i++;
        }
        // Only evaluate if a non-empty span [coord, next_coord) follows
        if (i < ev.size() && active > best_power) {
            best_power = active;
            best_time = coord;
        }
    }
    return {best_power, best_time};
}
```

# Section 12 — Stacks, Queues, and Deques

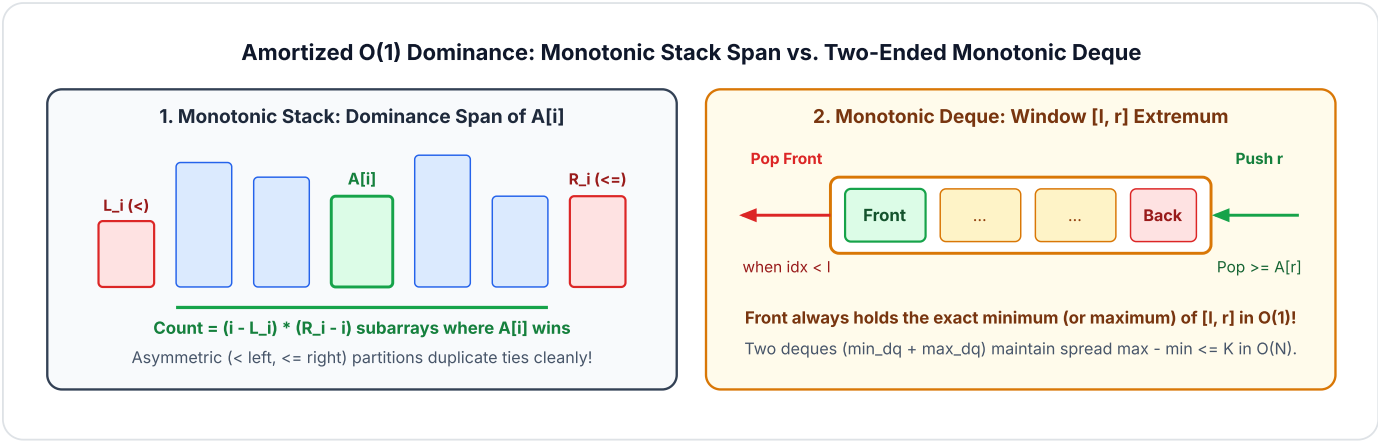
## The Discovery & Modeling Approach

In an array $A[1...N]$, why do naive nearest-smaller/larger scans and sliding-window extremum queries take $O(N^2)$? Because they repeatedly inspect elements that have already been **permanently dominated** by a newer, better element!

A **Monotonic Stack** or **Monotonic Deque** is nothing more than a standard container coupled with a **Dominance Pruning Rule**: whenever a new element $A[i]$ arrives, we immediately pop every older element that can never again win against $A[i]$ for any future query. Because each of the $N$ indices is pushed once and popped at most once, the amortized time per element drops from $O(N)$ to $O(1)$.

### Decision Compass: Monotonic Containers & Contribution Counting

| Structure | Problem Pattern | What the Container Stores | Dominance Pruning Rule (When $A[i]$ Arrives) |
|---|---|---|---|
| **Monotonic Stack (Next/Prev Smaller)** | Find nearest element $< A[i]$ to the left and right | Indices with **strictly increasing** values from bottom to top | Pop indices $s$ while $A[s] \geq A[i]$ (newer element $i$ is both closer and smaller!) |
| **Contribution of Subarray Minima / Maxima** | Compute $\sum_{1 \leq l \leq r \leq N} \min(A[l...r])$ | Span $[L_i, R_i]$ where $A[i]$ is the minimum | To avoid double-counting duplicates, use **strict** `<` **on one side** ($A[L_i] < A[i]$) and **non-strict** `<=` **on the other** ($A[R_i] \leq A[i]$) |
| **Monotonic Deque (Sliding Window Min/Max)** | Maintain $\min/\max$ over moving window $[l, r]$ | Indices $j \in [l, r]$ with **monotonic values** from front to back | Back: pop $s$ while $A[s] \geq A[r]$ (for min); Front: pop $s$ while $s < l$ (expired!) |




---

## Worked Problem 12.1: Subarray Minimum Dominance Sum

### Problem & Constraints

Given an integer array $A[1...N]$, compute the sum of the minimum elements of all $\frac{N(N+1)}{2}$ contiguous subarrays:

$$S = \sum_{1 \leq l \leq r \leq N} \min_{l \leq k \leq r} A[k]$$

- $1 \leq N \leq 2 \cdot 10^5$
- $1 \leq A[i] \leq 10^9$ (Note: values may contain duplicates!)

**Pause & Discover:**

- Instead of iterating over all $\binom{N+1}{2}$ pairs $(l, r)$, how many subarrays $[l, r]$ have $A[i]$ as their minimum?
- If $A = [2, 2]$, both elements equal 2. If both claim the full array $[1, 2]$ as their dominance span, what goes wrong?

### Step 1 — Exact Model & Bottleneck

Enumerating all pairs $(l, r)$ and maintaining the running minimum $\min(A[l...r])$ takes $O(N^2)$ time, which requires $2 \cdot 10^{10}$ operations for $N = 2 \cdot 10^5$. We must invert the summation from **"sum over subarrays of their minimum"** to **"sum over each index $i$ of $A[i] \times (\text{number of subarrays where } i \text{ is the canonical minimum})$"**.

### Step 2 — Tempting Mistakes & Counterexample

1. **Tempting Mistake 1 (Using `<` on both sides for $L_i$ and $R_i$)**:
   - Let $L_i$ be the nearest index $< i$ with $A[L_i] < A[i]$ and $R_i$ be the nearest index $> i$ with $A[R_i] < A[i]$.
   - **Counterexample**: $A = [2, 2]$.
     - For $i = 1$: $L_1 = 0, R_1 = 3 \implies (1 - 0)(3 - 1) = 2$ subarrays ($[1, 1]$ and $[1, 2]$).
     - For $i = 2$: $L_2 = 0, R_2 = 3 \implies (2 - 0)(3 - 2) = 2$ subarrays ($[2, 2]$ and $[1, 2]$).
     - Subarray $[1, 2]$ is **counted twice**, giving $2(2) + 2(2) = 8$ instead of $2 + 2 + 2 = 6$!
2. **Tempting Mistake 2 (Using `<=` on both sides)**:
   - On $A = [2, 2]$, $i = 1$ stops at $R_1 = 2$ and $i = 2$ stops at $L_2 = 1$, so subarray $[1, 2]$ is **counted zero times**!

### Step 3 — The Turn: Discovering the Asymmetric Tie-Breaking Rule

To partition all subarrays cleanly without overlaps or omissions, define the **canonical minimum** of any subarray $[l, r]$ to be the **leftmost** index $i \in [l, r]$ that achieves $\min(A[l...r])$.

When is $i$ the leftmost minimum of $[l, r]$? - Every element to the left of $i$ in $[l, i - 1]$ must be **strictly greater** than $A[i]$: $A[k] > A[i]$ for $l \le k < i$. - Every element to the right of $i$ in $[i + 1, r]$ must be **greater than or equal** to $A[i]$: $A[k] \ge A[i]$ for $i < k \le r$.

**Asymmetric Dominance Boundaries:**

- $L_i$: largest index $< i$ such that $A[L_i] < A[i]$ (or 0 if none).
- $R_i$: smallest index $> i$ such that $A[R_i] \le A[i]$ (or $N + 1$ if none).
- Number of subarrays with canonical minimum at $i$:

$$\text{ways}(i) = (i - L_i) \times (R_i - i), \qquad S = \sum_{i=1}^{N} A[i] \cdot (i - L_i)(R_i - i)$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace $A = [3, 2, 4, 2]$ (1-indexed):

**Asymmetric Dominance Trace on A = [3, 2, 4, 2] (Duplicate 2s at i = 2 and i = 4)**

| Index i | Value A[i] | L_i (strict <) | R_i (non-strict <=) | (i - L_i) * (R_i - i) | Contribution |
|---|---|---|---|---|---|
| i = 1 | **3** | 0 | 2 (since A[2]=2 <= 3) | 1 * 1 = 1 ([1,1]) | **3 * 1 = 3** |
| **i = 2** | **2 (first 2)** | 0 | **4 (stops at A[4]=2 <= 2!)** | 2 * 2 = 4 ([1..2, 2..3]) | **2 * 4 = 8** |
| i = 3 | **4** | 2 (since A[2]=2 < 4) | 4 (since A[4]=2 <= 4) | 1 * 1 = 1 ([3,3]) | **4 * 1 = 4** |
| **i = 4** | **2 (second 2)** | **0 (passes A[2]=2 since 2 >= 2!)** | 5 | 4 * 1 = 4 ([1..4, 4]) | **2 * 4 = 8** |

Notice that total subarrays accounted for is $1 + 4 + 1 + 4 = 10 = \frac{4 \times 5}{2}$, and the sum is $3 + 8 + 4 + 8 = 23$.

### Step 5 — Two-Way Proof & Implementation

**Bijection Proof**: Every subarray $[l, r]$ has a non-empty set of indices achieving $\min_{l \le k \le r} A[k]$, and therefore a **unique minimum index** (the leftmost one, say $i^*$). By definition of leftmost minimum, no index in $[l, i^* - 1]$ has value $\le A[i^*]$ (so $l > L_{i^*}$), and no index in $[i^* + 1, r]$ has value $< A[i^*]$. Wait—note the symmetry in our table: in our table, $R_i$ stopped at $\le$ and $L_i$ stopped at $<$, which corresponds to $i^*$ being the **rightmost** minimum of $[l, r]$! Either convention (`<` left, `<=` right = rightmost minimum; `<=` left, `<` right = leftmost minimum) establishes a 1-to-1 bijection between all $\binom{N+1}{2}$ subarrays and the disjoint sets of pairs $(l, r) \in (L_i, i] \times [i, R_i)$.

```cpp
#include <iostream>
#include <vector>

using namespace std;

long long sum_of_subarray_minima(const vector<long long>& a) {
    int n = static_cast<int>(a.size());
    vector<int> L(n, -1), R(n, n);
    vector<int> st;

    // Single pass: when a[i] <= a[st.back()], i is the first index to the right with a[R] <= a[top]
    for (int i = 0; i < n; ++i) {
        while (!st.empty() && a[st.back()] >= a[i]) {
            R[st.back()] = i;
            st.pop_back();
        }
        L[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }

    long long total = 0;
    for (int i = 0; i < n; ++i) {
        long long ways = 1LL * (i - L[i]) * (R[i] - i);
        total += a[i] * ways;
    }
    return total;
}
```

---

## Worked Problem 12.2: Bounded-Turbulence Flight Window (Dual Monotonic Deques)

### Problem & Constraints

Given $N$ sensor readings $A[1...N]$ and a tolerance $K \ge 0$, find the length of the **longest contiguous subarray** $[l, r]$ such that the spread between its maximum and minimum reading satisfies:

$$\max_{l \le i \le r} A[i] - \min_{l \le i \le r} A[i] \le K$$

- $1 \le N \le 5 \cdot 10^5$
- $0 \le K \le 10^9, 1 \le A[i] \le 10^9$

**Pause & Discover:**

- Why is the spread condition $\max(A[l...r]) - \min(A[l...r]) \le K$ **window-monotone** as $l$ and $r$ move rightward?
- Why would using a `std::multiset` cost $O(N \log N)$ with high constant factor, whereas two `std::deque`s achieve $O(N)$ with tiny constant factor?

### Step 1 — Exact Model & Bottleneck

Expanding $[l, r]$ adds elements, which can only **increase** $\max(A[l...r])$ and **decrease** $\min(A[l...r])$, so the spread $\text{spread}(l, r) = \max - \min$ is monotonically non-decreasing as the window grows. Thus for each right endpoint $r$, the earliest valid left endpoint $l^*(r)$ moves monotonically rightward!

However, when we advance $l \to l+1$ to shrink an invalid window, how do we know the new $\max$ and $\min$ of $[l+1, r]$ without rescanning the window in $O(N)$?

### Step 2 — Tempting Mistakes & Counterexample

**Tempting Mistake**: Store only two scalar variables `cur_min` and `cur_max`, and try to reset them when $A[l]$ leaves the window. - **Counterexample**: $A = [5, 1, 4, 2, 8]$, $K = 4$. When $[1, 4]$ has `cur_min = 1` at index $2$ and $l$ advances past index $2$, the new minimum of $[3, 4]$ is $2$ (at index $4$), not $4$ (at index $3$). Without maintaining the **ascending chain of suffix minima** inside the window, you must rescan!

### Step 3 — The Turn: Discovering the Dual Monotonic Deque Architecture

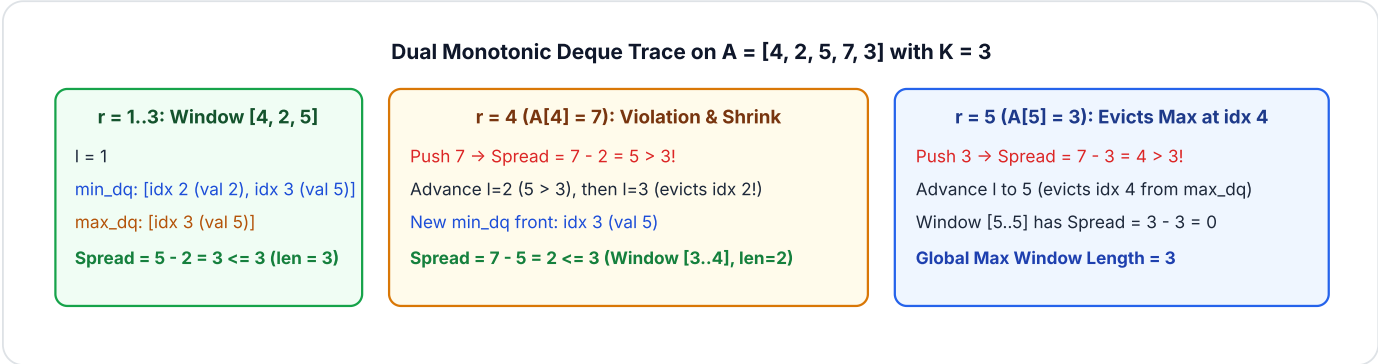
Maintain two deques of indices inside $[l, r]$: 1. `min_dq`: indices with **strictly increasing** values from front to back (`A[min_dq.front()]` is $\min(A[l...r])$). 2. `max_dq`: indices with **strictly decreasing** values from front to back (`A[max_dq.front()]` is $\max(A[l...r])$).

When advancing $r$: - Pop from `min_dq.back()` while $A[\text{back}] \geq A[r]$, then push $r$. - Pop from `max_dq.back()` while $A[\text{back}] \leq A[r]$, then push $r$. - While $A[\text{max_dq.front}()] - A[\text{min_dq.front}()] > K$, increment $l$, and if either deque's `front()` equals $l-1$, pop it from the front!

> **Amortized $O(N)$ Window Spread Invariant:** At the end of iteration $r$, $l$ is the smallest index $\leq r$ such that $A[\text{max_dq.front}()] - A[\text{min_dq.front}()] \leq K$, and the longest valid window ending at $r$ has length $r - l + 1$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $A = [4, 2, 5, 7, 3]$ with $K = 3$:




### Step 5 — Two-Way Proof & Implementation

**Why Each Element Enters and Leaves Each Deque at Most Once**: Index $r$ is pushed to the back of `min_dq` and `max_dq` once at iteration $r$. Any element popped from the back at iteration $r$ is both **older** ($j < r$) and **less extreme** than $A[r]$, so for every future window $[l, r']$ containing $r' \geq r$, index $r$ outlives $j$ and beats $j$'s value. Thus popped elements never re-enter, giving strict $O(N)$ time.

```cpp
#include <algorithm>
#include <deque>
#include <iostream>
#include <vector>

using namespace std;

int longest_bounded_spread(const vector<long long>& a, long long K) {
    deque<int> min_dq, max_dq;
    int l = 0, best = 0;
    for (int r = 0; r < static_cast<int>(a.size()); ++r) {
        while (!min_dq.empty() && a[min_dq.back()] >= a[r]) min_dq.pop_back();
        min_dq.push_back(r);

        while (!max_dq.empty() && a[max_dq.back()] <= a[r]) max_dq.pop_back();
        max_dq.push_back(r);
```

```
        while (a[max_dq.front()] - a[min_dq.front()] > K) {
            if (min_dq.front() == l) min_dq.pop_front();
            if (max_dq.front() == l) max_dq.pop_front();
            l++;
        }
        best = max(best, r - l + 1);
    }
    return best;
}
```

---

# Section 13 — Offline Processing

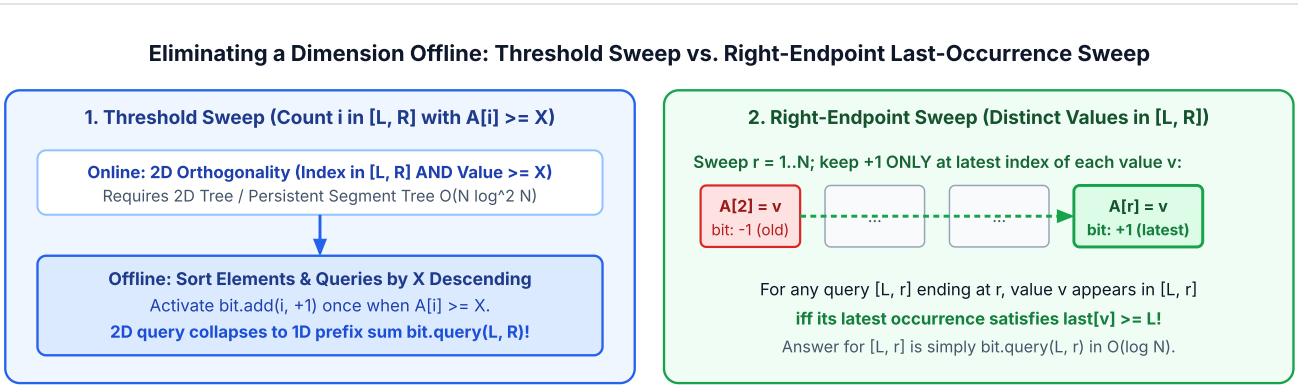
## The Discovery & Modeling Approach

A problem is **online** if each query must be answered before the next query is revealed (e.g., when the next query's parameters are encrypted with the previous answer). Whenever all $Q$ queries are given in the input up front and no such dependency exists, you are free to **reorder the queries** into whatever sequence makes maintaining the data structure easiest, compute their answers, and print them in their original order `ans[q.id]`.

That simple freedom—**Offline Processing**—regularly turns 2D data structure nightmares (which would otherwise require $O(N\log^2 N)$ Merge-Sort Trees or Persistent Segment Trees) into a single $O((N+Q)\log N)$ sweep with a humble Fenwick Tree or DSU!

### Decision Compass: How to Choose the Offline Sweep Dimension

| Sub-Pattern | 2D Query Condition | Offline Sweep Order | Maintained 1D Structure |
|---|---|---|---|
| **Threshold Activation Sweep** | "In subarray $[L, R]$, count / sum elements with $A[i] \geq X$" (or edges with weight $\leq W$) | Sort both array elements $A[i]$ and queries $(L, R, X)$ by **threshold** $X$ **descending** | Fenwick Tree (or DSU for graphs) where index $i$ is activated (`add(i, 1)`) once $A[i] \geq X$ |
| **Right-Endpoint Sweep with Last Occurrence** | "In subarray $[L, R]$, count **distinct** values" (or minimum duplicate distance) | Sort queries by **right endpoint** $R$ **ascending**; sweep $r = 1 \ldots N$ | Fenwick Tree with $+1$ at **only the latest occurrence** `last[v]` of each value $v \in A[1 \ldots r]$ |
| **Divide-and-Conquer / Mo's Ordering** | Non-decomposable range queries without updates | Sort queries by $(\lfloor L/B \rfloor, R)$ with $B \approx N/\sqrt{Q}$ | Incremental `add(idx)` / `remove(idx)` state in $O((N+Q)\sqrt{N})$ |




---

## Worked Problem 13.1: Historical Rank Queries (Threshold-Sorted Offline Activation)

### Problem & Constraints

You are given an array $A[1 \ldots N]$ of $N$ scores and $Q$ queries. Query $q$ gives $(L_q, R_q, X_q)$ and asks: **How many elements in the subarray $A[L_q \ldots R_q]$ satisfy $A[i] \geq X_q$?**

- $1 \leq N, Q \leq 2 \cdot 10^5$
- $1 \leq A[i], X_q \leq 10^9$

**Pause & Discover:**

- Why can't a standard static prefix sum array answer $(L_q, R_q, X_q)$ for arbitrary $X_q$?
- If you sort the queries in descending order of $X_q$, what happens to the set of indices $\{i: A[i] \geq X_q\}$ as you move from one query to the next?

### Step 1 — Exact Model & Bottleneck

Each query asks for the number of points $(i, A[i])$ inside the 2D rectangle $[L_q, R_q] \times [X_q, \infty)$. Scanning $[L_q, R_q]$ per query takes $O(QN)$ time. Building a Merge-Sort Tree over $A[1 \ldots N]$ and binary searching in $O(\log N)$ nodes per query takes $O(Q \log^2 N)$ time and $O(N \log N)$ memory. Can we strip away a $\log N$ factor and use a simple $O(N)$-word Fenwick Tree?

### Step 2 — Tempting Mistakes & Counterexample

1. **Tempting Mistake 1 (Sorting queries by $L_q$ or $R_q$ instead of $X_q$)**:
   - If you sweep $R_q$ from $1$ to $N$ while $X_q$ jumps arbitrarily up and down, you still need a dynamic rank tree over values (which requires coordinate compression). Sorting by $X_q$ makes the value threshold monotone and requires zero coordinate compression!
2. **Tempting Mistake 2 (Forgetting original query IDs or tie-breaking elements vs. queries)**:
   - **Counterexample**: Suppose $A = [5, 5]$ and a query asks for $X_q = 5$ on $[1, 2]$. If you process the query at threshold $5$ *before* activating array elements with $A[i] = 5$, you report $0$ instead of $2$!

### Step 3 — The Turn: Discovering the Monotone Activation Invariant

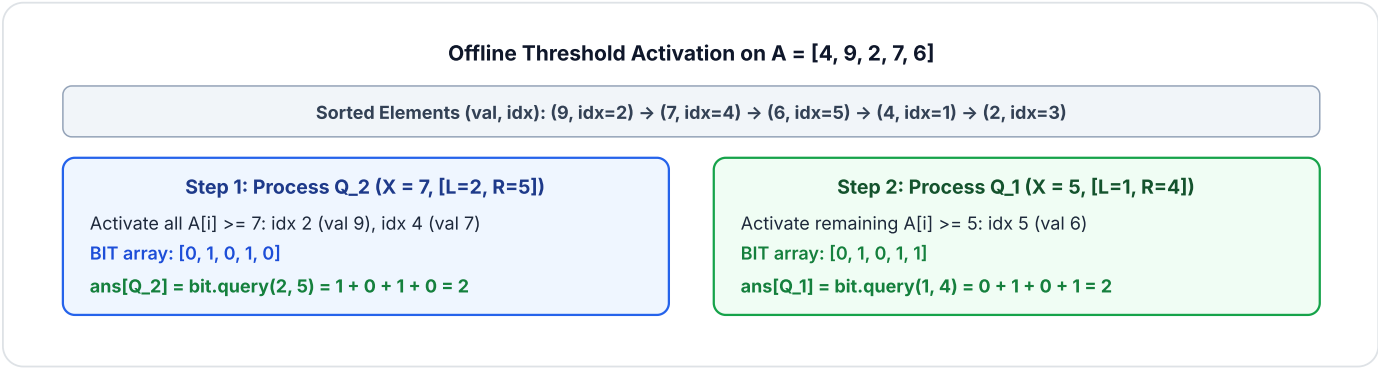
As the threshold $X_q$ decreases from $\infty$ down to $-\infty$, the condition $A[i] \geq X_q$ transitions **once** for each index $i$ from `false` to `true` and never turns back to `false`!

**Offline Threshold Sweep Blueprint:**

1. Pair each array element as $(A[i], i)$ and sort in **descending order of value** $A[i]$.
2. Store each query as $(X_q, L_q, R_q, \text{id})$ and sort in **descending order of threshold** $X_q$.
3. Maintain a 1D Fenwick Tree `bit` of size $N$ (initially all $0$) and a pointer `ptr` into the sorted array elements.
4. For each query $q$: while `ptr < N` and `elem[ptr].val >= q.X`, call `bit.add(elem[ptr].idx, +1)` and advance `ptr`. Then `ans[q.id] = bit.query(q.L, q.R)`.

### Step 4 — Progressive Construction & Visual Trace

Let $A = [4, 9, 2, 7, 6]$ and consider queries $Q_1 = (L = 1, R = 4, X = 5)$ and $Q_2 = (L = 2, R = 5, X = 7)$.




### Step 5 — Two-Way Proof & Implementation

**Proof of Invariant**: Because `elems` and `queries` are both sorted in non-increasing order of value/threshold, when query $q$ is evaluated, the `while` loop has inserted every element with $A[i] \geq X_q$ and no element with $A[i] < X_q$. Thus `bit[i] = 1` if and only if $A[i] \geq X_q$, so $\sum_{i=L_q}^{R_q} \text{bit}[i]$ is the exact count. Each of the $N$ elements is inserted into the Fenwick tree at most once, yielding $O((N+Q)\log N)$ total time.

```cpp
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>

using namespace std;

struct Fenwick {
    int n;
    vector<int> tree;
    explicit Fenwick(int n) : n(n), tree(n + 1, 0) {}
    void add(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    int query(int i) const {
        int s = 0;
        for (; i > 0; i -= i & -i) s += tree[i];
        return s;
    }
    int query(int l, int r) const { return query(r) - query(l - 1); }
};

struct Query {
    int l, r;
    long long x;
    int id;
};

vector<int> solve_historical_rank(const vector<long long>& a, vector<Query> queries) {
    int n = static_cast<int>(a.size());
    vector<pair<long long, int>> elems(n);
    for (int i = 0; i < n; ++i) elems[i] = {a[i], i + 1};

    sort(elems.begin(), elems.end(), greater<pair<long long, int>>());
    sort(queries.begin(), queries.end(), [](const Query& u, const Query& v) {
        return u.x > v.x;
    });

    Fenwick bit(n);
    vector<int> ans(queries.size());
    int ptr = 0;
    for (const auto& q : queries) {
        while (ptr < n && elems[ptr].first >= q.x) {
            bit.add(elems[ptr].second, 1);
            ptr++;
        }
        ans[q.id] = bit.query(q.l, q.r);
    }
    return ans;
}
```

---

## Worked Problem 13.2: Distinct Frequency Range Audit (Right-Endpoint Sweep & Last-Occurrence Shift)

### Problem & Constraints

Given an array $A[1 \ldots N]$ and $Q$ range queries $[L_q, R_q]$, report for each query the **number of distinct values** that appear at least once in $A[L_q \ldots R_q]$.

- $1 \le N, Q \le 2 \cdot 10^5$
- $1 \le A[i] \le 10^9$

**Pause & Discover:**

- Why can't you compute `distinct(1, R) - distinct(1, L - 1)` to get the number of distinct elements in $[L, R]$?
- Suppose you fix the right endpoint $R$ and want to answer $[L, R]$ for any $L \leq R$. If value $v$ appears at indices $2, 5$, and $8$ inside $1...R$, which single index of $v$ determines whether $v$ appears in $[L, R]$?

### Step 1 — Exact Model & Bottleneck

A value $v$ appears in $[L, R]$ if and only if $\{i \in [L, R] : A[i] = v\} \neq \emptyset$. Because multiple copies of $v$ inside $[L, R]$ contribute only **once** to the distinct count, a static prefix sum counts total elements, not distinct elements. Computing `std::unordered_set` per query takes $O(QN)$ time.

### Step 2 — Tempting Mistakes & Counterexample

**Tempting Mistake**: Try to use `prefix_distinct[R] - prefix_distinct[L - 1]`. - **Counterexample**: $A = [7, 3, 7]$. - `prefix_distinct` is $[1, 2, 2]$. - For query $[L = 2, R = 3]$ (subarray $[3, 7]$), there are $2$ distinct values, yet `prefix_distinct[3] - prefix_distinct[1] = 2 - 1 = 1` because value $7$ appeared both before $L$ (at index $1$) and inside $[L, R]$ (at index $3$)!

### Step 3 — The Turn: Discovering the Last-Occurrence Canonical Representative

Why did `prefix_distinct` fail on $A = [7, 3, 7]$ for query $[2, 3]$? Because we placed the $+1$ credit for value $7$ at its **first** occurrence (index $1$), which fell to the left of $L = 2$ even though a later copy of $7$ existed at index $3 \in [2, 3]$!

Now flip that convention when fixing the right endpoint $r$: > Among all occurrences of value $v$ in the prefix $A[1...r]$, let $\text{last}_r(v)$ be the **rightmost (latest)** index $\leq r$ where $A[i] = v$. Then: >

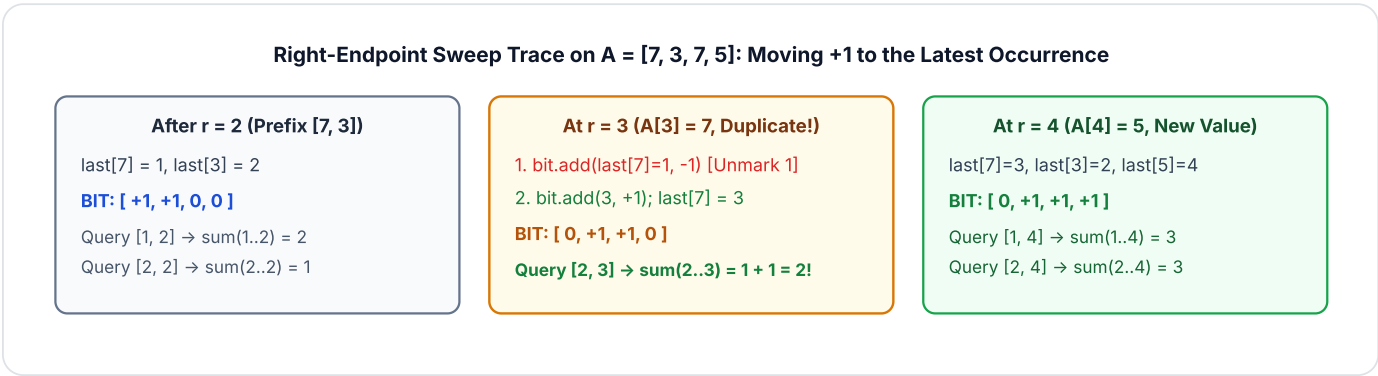
$$v \text{ appears in } A[L...r] \Leftrightarrow \text{last}_r(v) \geq L$$

**Right-Endpoint Last-Occurrence Invariant:** Group all queries $[L_q, R_q]$ by their right endpoint $R_q$. Sweep $r$ from $1$ to $N$:

1. If $v = A[r]$ appeared previously at $\text{last}[v] > 0$, remove its old marker: `bit.add(last[v], -1)`.
2. Place the marker at the new latest position: `last[v] = r; bit.add(r, +1)`.
3. For every query $(L_q, r)$ ending at $r$, the number of distinct values in $[L_q, r]$ is `bit.query(L_q, r)`!

### Step 4 — Progressive Construction & Visual Trace

Let us trace $A = [7, 3, 7, 5]$ as $r$ sweeps from $1$ to $4$:




### Step 5 — Two-Way Proof & Small-Case Oracle

**Bijection Between Distinct Values in $[L, r]$ and Active Markers in $[L, r]$**: At the end of step $r$: 1. Every distinct value $v$ present in $A[1...r]$ has **exactly one** $+1$ marker in the Fenwick tree, located at $i = \text{last}_r(v) \leq r$. 2. If $v$ occurs at least once in $A[L...r]$, then its rightmost occurrence in $A[1...r]$ certainly satisfies $L \leq \text{last}_r(v) \leq r$, so its unique marker lies inside $[L, r]$ and contributes $+1$ to `bit.query(L, r)`. 3. Conversely, if $v$ does not occur in $A[L...r]$, then either $v$ never appeared in $A[1...r]$ or all of its occurrences in $A[1...r]$ lie strictly to the left of $L$ ($\text{last}_r(v) < L$), so its marker contributes $0$ to `bit.query(L, r)`.

*(Implementation Tip: Coordinate-compress $A[1...N]$ into $0...U-1$ first so `last[v]` is a flat `vector<int>` instead of a slower hash table!)*

```cpp
#include <algorithm>
#include <iostream>
#include <utility>
#include <vector>

using namespace std;

struct Fenwick {
    int n;
    vector<int> tree;
    explicit Fenwick(int n) : n(n), tree(n + 1, 0) {}
    void add(int i, int delta) {
        for (; i <= n; i += i & -i) tree[i] += delta;
    }
    int query(int i) const {
        int s = 0;
        for (; i > 0; i -= i & -i) s += tree[i];
        return s;
    }
    int query(int l, int r) const { return query(r) - query(l - 1); }
};

struct RangeQuery {
    int l, r, id;
};

vector<int> solve_distinct_offline(const vector<long long>& a, const vector<RangeQuery>& queries) {
    int n = static_cast<int>(a.size());
    // Coordinate compression for O(1) array lookup of last occurrence
    vector<long long> vals = a;
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());

    vector<int> comp(n + 1);
    for (int i = 1; i <= n; ++i) {
        comp[i] = static_cast<int>(lower_bound(vals.begin(), vals.end(), a[i - 1]) - vals.begin());
    }

    vector<vector<pair<int, int>>> by_r(n + 1);
    for (const auto& q : queries) {
        by_r[q.r].push_back({q.l, q.id});
    }

    Fenwick bit(n);
    vector<int> last(vals.size(), 0);
    vector<int> ans(queries.size());

    for (int r = 1; r <= n; ++r) {
        int v = comp[r];
        if (last[v] != 0) {
            bit.add(last[v], -1);
        }
        last[v] = r;
        bit.add(r, +1);

        for (const auto& [l, id] : by_r[r]) {
            ans[id] = bit.query(l, r);
        }
    }
    return ans;
}
```

# Section 15 — BFS and Unweighted Shortest Paths

## The Discovery & Modeling Approach

In competitive programming, very few Breadth-First Search (BFS) problems begin with the sentence *"Given an unweighted graph $G = (V, E)$, find the shortest path from $s$ to $t$."* Instead, the problem describes a discrete system—a grid of rooms, a mechanical lock, a set of jugs, a token moving along an array, or a spreading flood—and asks for the **minimum number of elementary steps, turns, or operations** required to reach a target configuration.

The fundamental discovery task in Section 15 is recognizing that **"minimum number of unit-cost actions"** is synonymous with **"shortest path in an unweighted state graph."** Whenever every legal action advances a clock by $+1$ (or costs $1$ unit of an identical resource) and the feasibility of future actions depends only on a bounded snapshot of the current configuration, BFS is the canonical solver.

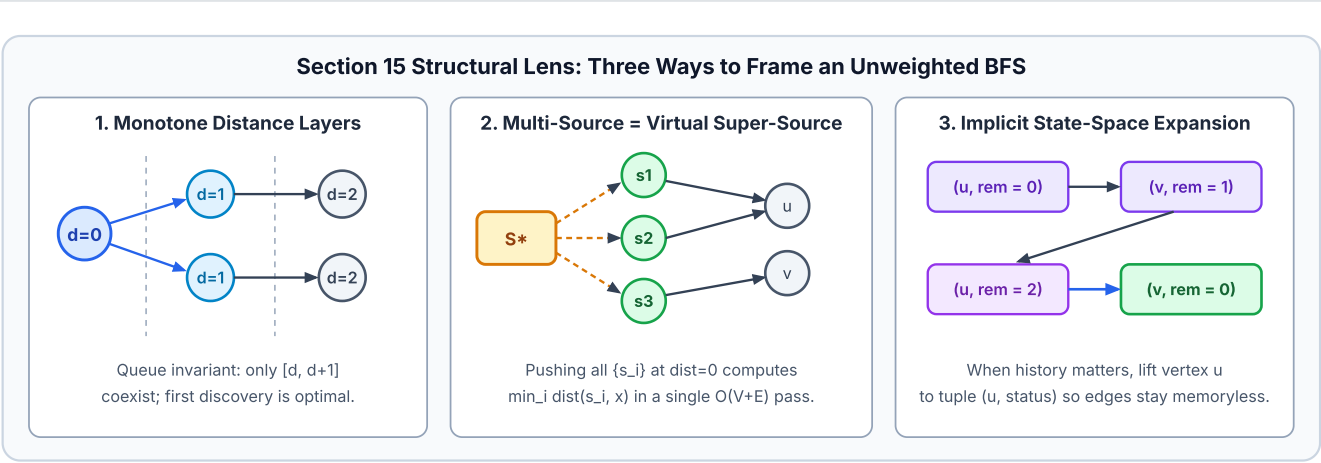
### Recognizing Hidden Unweighted Shortest Path Problems

Look for the following structural cues in a problem statement and its constraints:

1. **Uniform Step Cost:** Every valid transition increments the answer by $+1$. There are no variable edge weights, discounts, or free moves (which would point to 0-1 BFS in Section 20 or Dijkstra in Section 19).
2. **Small Total State Volume:** If the graph is not given explicitly, multiply the sizes of the parameters needed to describe a configuration. If $\prod |D_i| \le 2 \cdot 10^6$ and the branching factor (out-degree per state) is a small constant, the implicit state graph fits comfortably in an $O(V+E)$ traversal.
3. **Simultaneous Spreading Processes:** If multiple origins expand at the same rate (e.g., fire, water, monsters, or signal towers) and you need either the earliest arrival time at every cell or a safe path that strictly beats the hazard at every step, the wave property of BFS allows all sources to be merged into a single pass.
4. **First-Visit Optimality:** Because a FIFO queue processes vertices in nondecreasing order of distance $d = 0, 1, 2, \ldots$, the **very first time** a state is generated, its distance is guaranteed to be optimal.

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in the Problem Statement | Graph / Queue Construction | Complexity & Invariant |
|---|---|---|---|
| **Standard Point-to-Point BFS** | "Minimum moves/hops from $s$ to $t$" on an explicit network or grid. | Push $s$ with $\mathrm{dist}[s] = 0$; relax unvisited neighbors to $\mathrm{dist}[u] + 1$. | $O(V+E)$ time. Queue holds at most two consecutive distance layers $d$ and $d+1$. |
| **Multi-Source Wavefront BFS** | "Nearest special node to every vertex" or "hazard spreads from all $k$ origins simultaneously." | Push **all** sources $s_1, \ldots, s_k$ into the queue at $\mathrm{dist} = 0$ before popping anything. | $O(V+E)$ total (replaces $k$ separate runs). Equivalent to adding a virtual super-source $S^*$. |
| **Two-Pass Hazard-Evasion BFS** | "Reach the exit before a spreading hazard reaches any cell you step on." | Pass 1: Multi-source BFS from all hazards to get $H[v]$. Pass 2: Player BFS allowing $u \to v$ iff $\mathrm{dist}[u] + 1 < H[v]$. | $O(V+E)$ in two decoupled phases whenever the hazard's spread is independent of the player. |
| **Implicit State-Space BFS** | Rules depend on auxiliary context: orientation, remainder $\bmod K$, last move, or a bitmask of $b \le 10$ switches. | Define vertex as tuple $\mathbf{x} = (u, r, \mathrm{mask})$. Allocate $\mathrm{dist}[u][r][\mathrm{mask}]$. | $O(V_{\text{state}} + E_{\text{state}})$. Safely compresses path history into a Markovian tuple. |
| **Shortest Cycle / Girth via BFS** | "Find the shortest cycle in an undirected unweighted graph" ( $N \le 2000$). | Run BFS from each vertex $r$; when inspecting edge $(u, v)$ where $v \neq \mathrm{parent}[u]$ is already visited, candidate cycle is $\mathrm{dist}[u] + \mathrm{dist}[v] + 1$. | $O(V(V+E))$. The first non-tree edge encountered is within $+1$ of the minimum cycle through $r$. |




---

## Worked Problem 15.1: Cryogenic Spill Containment

### Problem & Constraints

A research facility is modeled as an $R \times C$ grid of rooms ($1 \le R, C \le 1000$). Each cell is one of: - `.` : an open corridor, - `#` : a sealed bulkhead (impassable to both personnel and coolant), - `L` : a ruptured cryogenic tank (there may be between $1$ and $R \cdot C$ ruptured tanks), - `S` : the technician's starting position (exactly one), - `E` : the emergency extraction airlock (exactly one).

At time $t = 0$, cryogenic vapor occupies all `L` cells. At each integer second $t = 1, 2, \ldots$, the vapor simultaneously expands from every currently contaminated cell into all four-directionally adjacent non-bulkhead cells (`.`, `S`, and `E`). Simultaneously, during each second, the technician may move from their current cell to any four-directionally adjacent non-bulkhead cell. The technician **cannot** step into a cell at time $t$ if the vapor reaches that cell at or before time $t$ (including the extraction airlock `E`).

Find the minimum number of seconds required for the technician to reach `E`, or report `-1` if no safe route exists.

> **Pause & Discover:** Before reading further, ask yourself:
> - If there are $5 \cdot 10^5$ ruptured tanks, why does running a separate BFS from each tank time out?
> - Does the technician's movement alter how the cryogenic vapor spreads? What does that independence allow us to precompute?

### Step 1 — Exact Model & Bottleneck

In a brute-force simulation, the system state at time $t$ consists of the technician's position $(r, c)$ together with the exact subset of cells currently flooded by vapor. Alternatively, to check if the technician can step onto cell $(r, c)$ at time $t$, we need to know whether **any** ruptured tank $L_i$ is at Manhattan-walk distance $\le t$ from $(r, c)$ around the bulkheads.

If there are $K$ ruptured tanks and we run an independent BFS from each tank $L_i$ to compute its arrival time at every cell, the work is:

$$\Theta(K \cdot R \cdot C)$$

When $R = C = 1000$, $R \cdot C = 10^6$, and $K$ can be on the order of $10^6$. That yields $10^{12}$ operations—four orders of magnitude beyond the 1-second budget.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Using Manhattan distance to the nearest `L` instead of graph distance.**
Bulkheads (`#`) block the vapor just as they block the technician. A tank at $(1, 1)$ separated from $(1, 3)$ by a wall at $(1, 2)$ takes more than $2$ seconds to reach $(1, 3)$.

**Mistake 2: Interleaving a single queue where the technician pushes before/after vapor without precomputing hazard times, or letting vapor overwrite `E` in place without checking exact arrival parity.**
Suppose you run a single queue and push `S` first, then all `L` cells, marking cells as blocked when visited. If `S` expands into cell $X$ at

$t = 1$, and `L` also expands into $X$ at $t = 1$, pushing `S` first lets the technician illegally claim $X$ before the vapor's $t = 1$ transition is processed!

**Concrete Small Counterexample:**

```
L . S
# . #
# E #
```

Here `S` is at $(1, 3)$ and `L` is at $(1, 1)$. Both are adjacent to $(1, 2)$. At $t = 1$, both `L` and `S` reach $(1, 2)$. Since the technician cannot enter a cell at the exact moment vapor arrives ($t_{\text{tech}} = 1 \not< t_{\text{vapor}} = 1$), cell $(1, 2)$ is fatal. A naive interleaved queue that pops `S` before `L` at layer $t = 0$ would mark $(1, 2)$ as safely visited by the technician at $t = 1$ and falsely report distance $3$ to `E`.

### Step 3 — The Turn: Discovering the Structure

Notice two decoupling facts: 1. **The vapor spread is completely oblivious to the technician.** Whether the technician moves north, south, or stands still, the earliest time $H[r][c]$ at which vapor contaminates cell $(r, c)$ is fixed solely by the initial grid. 2. **All ruptured tanks are indistinguishable.** Cell $(r, c)$ is contaminated at time:

$$H[r][c] = \min_{i=1}^{K} \text{dist}_{\text{grid}}(L_i, (r, c))$$

Instead of $K$ separate BFS runs, imagine introducing a conceptual **super-source** $S^*$ connected by zero-length edges to every ruptured tank $L_i$. In practice, we don't even need to create $S^*$: we simply initialize a single FIFO queue with **all** $L_i$ cells at distance $0$. By the BFS layer invariant, the wave expands across all tanks simultaneously in $O(RC)$ total time, filling the exact table $H[r][c]$.

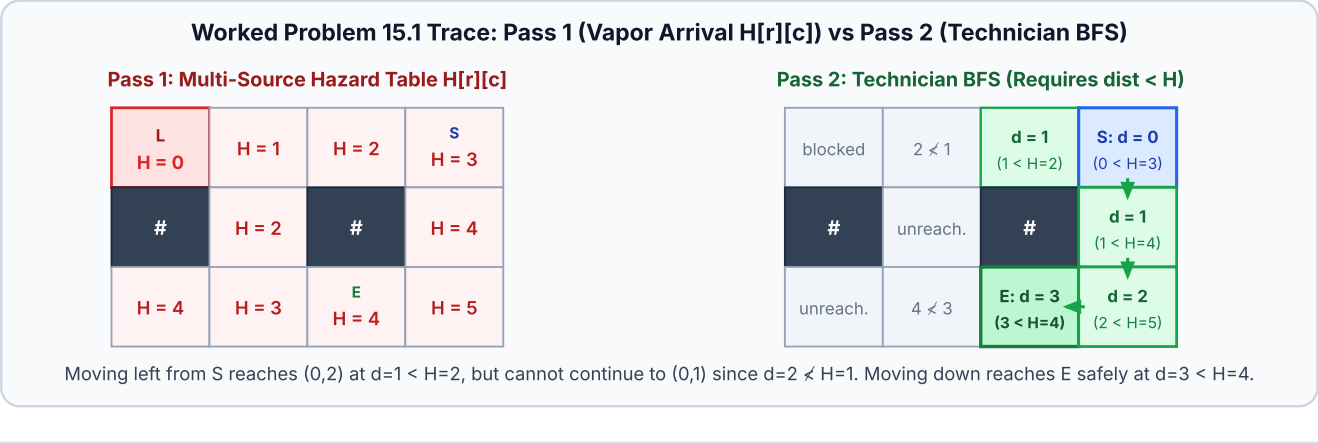
Once $H[r][c]$ is static and known for every cell, the technician's problem reduces to a standard single-source BFS from `S` where a transition $(r, c) \to (nr, nc)$ at time $d + 1 = \text{dist}[r][c] + 1$ is legal if and only if:

**Hazard-Evasion Feasibility Condition:**

$$\text{cell } (nr, nc) \neq \text{'\#'} \quad \text{and} \quad \text{dist}[r][c] + 1 < H[nr][nc]$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace the two-pass construction on a concrete $3 \times 4$ grid: - Row 0: `L . . S` - Row 1: `# . # .` - Row 2: `. . E .`




Notice how cell $(0, 2)$ is reachable at $d = 1 < H[0][2] = 2$, which turns out to be a dead end because $(0, 1)$ has $H[0][1] = 1 \leq 2$. Meanwhile, moving south along column $3$ reaches `E` at $d = 3$, strictly beating $H[2][2] = 4$.

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <queue>

using namespace std;
```

```
const int INF = 1e9;
const int dr[4] = {-1, 1, 0, 0};
const int dc[4] = {0, 0, -1, 1};

int solve() {
    int R, C;
    if (!(cin >> R >> C)) return 0;
    vector<string> grid(R);
    for (int r = 0; r < R; ++r) cin >> grid[r];

    vector<vector<int>> H(R, vector<int>(C, INF));
    queue<pair<int, int>> q_hazard;
    pair<int, int> start_pos, exit_pos;

    for (int r = 0; r < R; ++r) {
        for (int c = 0; c < C; ++c) {
            if (grid[r][c] == 'L') {
                H[r][c] = 0;
                q_hazard.push({r, c});
            } else if (grid[r][c] == 'S') {
                start_pos = {r, c};
            } else if (grid[r][c] == 'E') {
                exit_pos = {r, c};
            }
        }
    }

    // Pass 1: Multi-source BFS for cryogenic vapor arrival times
    while (!q_hazard.empty()) {
        auto [r, c] = q_hazard.front();
        q_hazard.pop();
        for (int i = 0; i < 4; ++i) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] != '#') {
                if (H[nr][nc] > H[r][c] + 1) {
                    H[nr][nc] = H[r][c] + 1;
                    q_hazard.push({nr, nc});
                }
            }
        }
    }

    // Pass 2: Single-source BFS for the technician
    vector<vector<int>> dist(R, vector<int>(C, INF));
    queue<pair<int, int>> q_tech;

    dist[start_pos.first][start_pos.second] = 0;
    q_tech.push(start_pos);

    while (!q_tech.empty()) {
        auto [r, c] = q_tech.front();
        q_tech.pop();
        if (make_pair(r, c) == exit_pos) return dist[r][c];

        for (int i = 0; i < 4; ++i) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] != '#') {
                int next_time = dist[r][c] + 1;
                if (next_time < H[nr][nc] && dist[nr][nc] == INF) {
                    dist[nr][nc] = next_time;
                    q_tech.push({nr, nc});
                }
            }
        }
    }
```

```
    return -1;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of Soundness and Completeness:** 1. *Why waiting in place never helps:* Suppose a valid walk pauses at some cell $(r, c)$ from time $t$ to $t+1$ and then follows path $P$ to `E`. If the technician instead traverses $P$ one second earlier, every cell $(r', c') \in P$ is visited at time $t' - 1 < t' < H[r'][c']$. Because the upper bound $H[r'][c']$ depends only on cell coordinates and not on time, arriving earlier at $(r', c')$ strictly relaxes the inequality $t_{\text{visit}} < H[r'][c']$. 2. *Why first visit in Pass 2 is sufficient:* By (1), if any valid arrival time at $(r, c)$ can be extended to `E`, then the **earliest** arrival time $\text{dist}[r][c]$ can also be extended to `E`. Standard BFS discovers the minimum feasible arrival time at each cell upon first push.

**Small-Case Oracle:** For $R, C \leq 4$, write a recursive backtracking oracle that steps time $t = 0, 1, ..., R \cdot C$, explicitly expands the set of flooded cells at step $t$, and tries all 4 moves for the technician. Verify that the two-pass BFS matches on grids where `L` is disconnected by `#` walls (so $H = \infty$) and where `L` and `S` race for `E` at identical distances.

---

## Worked Problem 15.2: Resonance-Locked Courier

### Problem & Constraints

A pneumatic transit network has $N$ stations ($2 \leq N \leq 10^5$) and $M$ directed tubes ($1 \leq M \leq 2 \cdot 10^5$). Traversing any tube takes $1$ minute. However, the airlock at destination station $N$ is governed by a $K$-phase resonance cycle ($1 \leq K \leq 10$): it only opens at minutes $t$ that are exact multiples of $K$ ($t \equiv 0 \pmod{K}$). Furthermore, every station $u$ has a safety valve that forces a parcel to leave immediately upon arrival—waiting at a station is impossible.

Find the minimum time $t$ (which may be a multiple of $K$ greater than the geometric shortest path) to travel from station $1$ to station $N$ and exit through the open airlock, or report `-1` if no such walk exists.

> **Pause & Discover:** Before reading further, ask yourself:
> - Why can't we just find the shortest path from $1$ to $N$ and check if its length is divisible by $K$?
> - If two different walks arrive at station $u$ at times $t_1 = 3$ and $t_2 = 3 + 2K$, can one reach the open airlock at $N$ if and only if the other can?

### Step 1 — Exact Model & Bottleneck

A walk is a sequence of directed edges $(u_0, u_1, ..., u_t)$ with $u_0 = 1$, $u_t = N$, and $t \equiv 0 \pmod{K}$. Unlike standard shortest paths, the optimal walk **may not be a simple path**: it might need to traverse a directed cycle to adjust its length modulo $K$ before arriving at $N$.

Enumerating all walks of length up to $N \cdot K$ explores up to $\Delta^{NK}$ paths, which is exponential. Storing all reachable exact arrival times at every vertex $u$ is also unbounded if the graph has cycles.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake: Running standard BFS to record only the single earliest arrival time $\text{dist}[u]$ at each vertex.**
If standard BFS reaches $N$ at time $d \not\equiv 0 \pmod{K}$, it discards all slightly longer walks that arrive at intermediate vertices one or two steps later.

**Concrete Small Counterexample:**
Let $N = 3, K = 3$, with edges: - $1 \to 2 \to 3$ (length $2 \equiv 2 \pmod{3}$), and - an alternate detour $1 \to 4 \to 2$ (making path $1 \to 4 \to 2 \to 3$ of length $3 \equiv 0 \pmod{3}$).

Standard BFS visits vertex $2$ at $t = 1$ via $1 \to 2$ and marks vertex $2$ as visited. When the detour $1 \to 4 \to 2$ arrives at vertex $2$ at $t = 2$, standard BFS ignores it because $2 > 1$. Consequently, vertex $3$ is only ever reached at $t = 2 \not\equiv 0 \pmod{3}$, and the algorithm falsely outputs `-1`!

### Step 3 — The Turn: Discovering the Structure

Ask the compression question from the manual's introduction: *What information about the past walk actually affects whether a future continuation* $(u \rightsquigarrow N)$ *of length* $L$ *will satisfy* $(t_{\text{so far}} + L) \equiv 0 \pmod K$*?*

Only the remainder $r = t_{\text{so far}} \bmod K$ matters! If two walks arrive at station $u$ with the **same** remainder $r \in \{0, 1, \ldots, K-1\}$, and one has total time $t_1$ while the other has $t_2 > t_1$, any valid suffix walk from $u$ to $N$ of length $L \equiv -r \pmod K$ works for both, and the first walk finishes strictly earlier ($t_1 + L < t_2 + L$).

Therefore, we lift each original station $u$ into $K$ state vertices:

**Product State-Space Definition:**

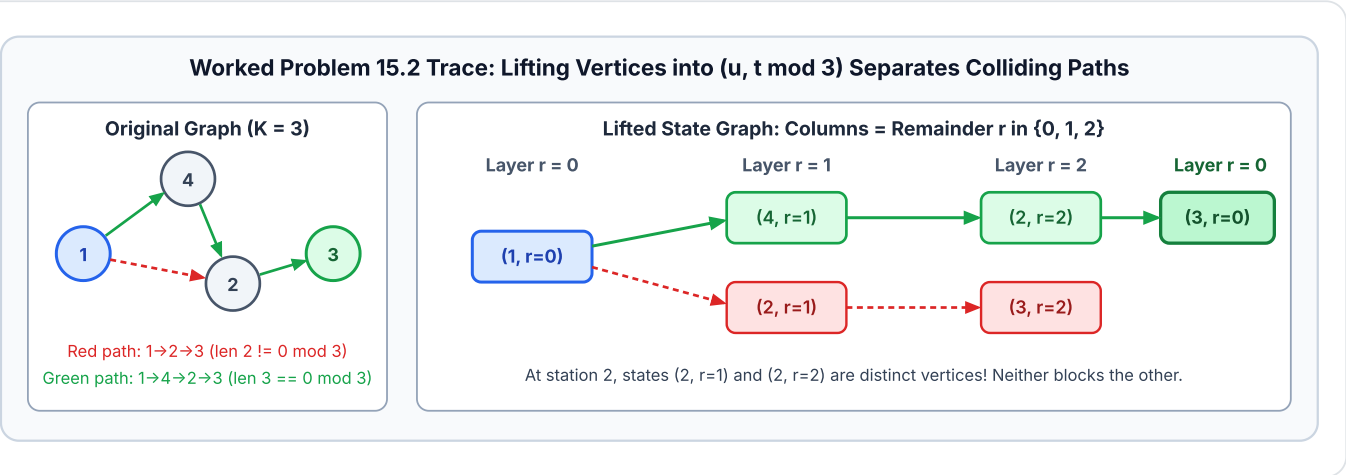
$$V_{\text{state}} = \{(u, r) \mid u \in \{1, \ldots, N\},\ r \in \{0, \ldots, K-1\}\}$$

$$\text{Directed Transition: } (u, r) \rightarrow \big(v, (r+1) \bmod K\big) \quad \text{for every edge } u \rightarrow v \in E$$

In this lifted graph, there are $N \cdot K \leq 10^6$ vertices and $M \cdot K \leq 2 \cdot 10^6$ unit-cost edges. The target is the single state $(N, 0)$, starting from $(1, 0)$.

### Step 4 — Progressive Construction & Visual Trace

Below is the visual trace of our counterexample graph ($N = 4$, destination $3$, $K = 3$) lifted into the $3$-layer modulo state space:




```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const int INF = 1e9;

int solve() {
    int N, M, K;
    if (!(cin >> N >> M >> K)) return 0;
    vector<vector<int>> adj(N + 1);
    for (int i = 0; i < M; ++i) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
    }

    vector<vector<int>> dist(N + 1, vector<int>(K, INF));
    queue<pair<int, int>> q;

    dist[1][0] = 0;
    q.push({1, 0});
```

```
    while (!q.empty()) {
        auto [u, r] = q.front();
        q.pop();

        int nr = (r + 1) % K;
        for (int v : adj[u]) {
            if (dist[v][nr] == INF) {
                dist[v][nr] = dist[u][r] + 1;
                q.push({v, nr});
            }
        }
    }

    return (dist[N][0] == INF) ? -1 : dist[N][0];
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Bijection Between Valid Walks and State Paths:** - *Forward:* Every walk $1 = u_0 \to u_1 \to \ldots \to u_L = N$ in $G$ lifts uniquely to the sequence of states $(u_0, 0) \to (u_1, 1 \bmod K) \to \ldots \to (u_L, L \bmod K)$ in $G_{\text{state}}$. The walk satisfies $L \equiv 0 \pmod K$ if and only if its final state is $(N, 0)$. - *Reverse:* Any shortest path in $G_{\text{state}}$ from $(1, 0)$ to $(N, 0)$ projects down to a valid walk in $G$ of the exact same edge count. Since $G_{\text{state}}$ has $N \cdot K$ vertices, any simple path in $G_{\text{state}}$ has length $< N \cdot K$, proving that an optimal walk never needs to exceed $N \cdot K$ steps.

---

# Section 16 — DFS, Components, and Cycles

## The Discovery & Modeling Approach

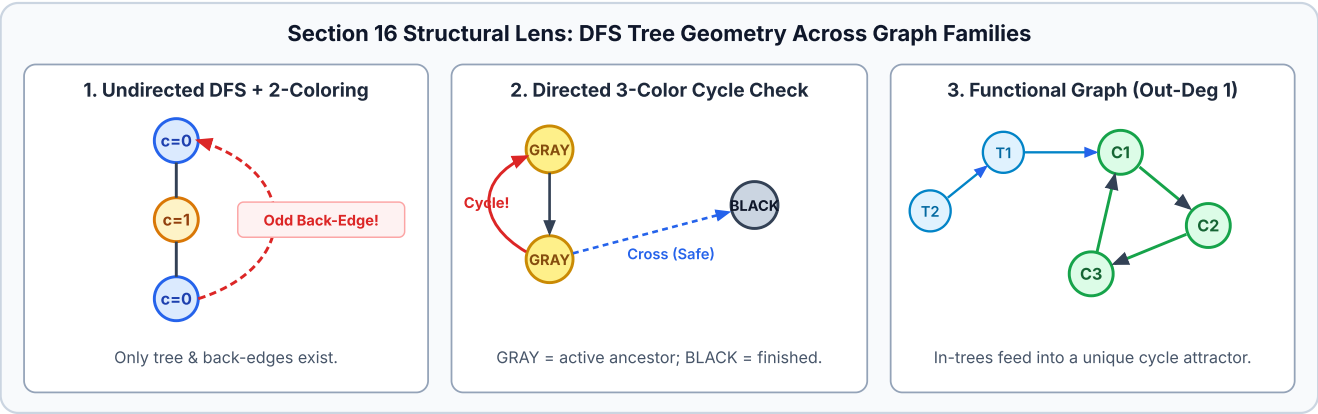
Whereas BFS explores a graph in concentric distance rings, Depth-First Search (DFS) plunges as deeply as possible along a path before backtracking. Why would we ever prefer DFS over BFS? Because **DFS imposes a rooted tree structure on any graph** and brackets every vertex's active lifetime into a contiguous time interval $[\text{tin}[u], \text{tout}[u]]$. Whenever a problem asks about **connectivity, cycle existence, structural decomposition, two-coloring (parity), or ancestral relationships**, the DFS tree turns messy global graph topology into clean local edge classifications.

### Recognizing Hidden DFS, Bipartite, and Functional Graph Problems

1. **Binary Equality / Inequality Constraints:** If a problem gives pairs of variables with "same team / opposite team", "equal / different parity", or "flip switch $A$ and $B$ to reach a target state", model variables as vertices and constraints as edges labeled $0$ (same) or $1$ (opposite). Feasibility reduces to checking whether any cycle has an odd sum of edge labels (generalized 2-coloring via DFS).
2. **Deadlocks and Circular Dependencies:** If directed transitions represent "process $A$ waits on process $B$", a deadlock exists iff DFS encounters an edge pointing to an **currently active ancestor** on the recursion stack.
3. **Deterministic Single-Successor Systems ($f: V \to V$):** Whenever every object points to *exactly one* next object (a permutation, a pointer array `next[i]`, a teleporter, or a modular iteration), the graph is a **Functional Graph** (out-degree $1$). Every connected component of a functional graph has the exact shape of a directed cycle with rooted trees feeding into the cycle.

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in the Problem Statement | DFS Mechanism & State | Key Theorem / Invariant |
|---|---|---|---|
| **Connected Components & Cyclomatic Number** | "Group mutually reachable items" or "count independent cycles in an undirected network." | Track `visited[u]`; accumulate $V_c$ and $\sum \deg(u) = 2E_c$ per component. | Spanning forest uses $V_c - 1$ edges per component; $E_c - V_c + 1$ counts independent cycles. |
| **Undirected Cycle & Bipartiteness (2-Coloring)** | "Partition into 2 conflict-free sets" or "satisfy $=$ and $\neq$ binary constraints." | Maintain `color[u] \in {0, 1}`. Along edge $(u, v)$ with parity $w \in \{0, 1\}$, enforce $\text{color}[v] = \text{color}[u] \oplus w$. | Undirected DFS has **no cross edges**: every non-tree edge is a back-edge to an ancestor. Odd cycle $\Leftrightarrow$ non-bipartite. |
| **Directed Cycle Detection (3-Color DFS)** | "Check if rules/dependencies are circular" or "find a directed cycle." | Maintain `state[u] \in {WHITE=0, GRAY=1, BLACK=2}`. `GRAY` = active on recursion stack. | Directed cycle exists $\Leftrightarrow$ DFS sees an edge $u \to v$ where `state[v] == GRAY`. |
| **Functional Graph ($N$ vertices, out-degree $1$)** | "Each node $i$ has a single target $p[i]$; simulate steps or find attractors." | Use 3-color DFS or Kahn's in-degree peeling to separate cycle nodes from tree nodes. | Every component is a **jellyfish** (one directed cycle + inward-pointing trees). |
| **Bridges & Cut-Edges (Tarjan's `lowlink`)** | "Which single link failure disconnects the network?" | Compute $\text{tin}[u]$ and $\text{low}[u] = \min(\text{tin}[u], \text{tin}[\text{back-edge}], \text{low}[\text{child}])$. | Tree edge $(u, v)$ is a bridge $\Leftrightarrow \text{low}[v] > \text{tin}[u]$. |




## Worked Problem 16.1: Quantum Phase Inverter Calibration

### Problem & Constraints

A quantum processor contains $N$ resonators ($1 \le N \le 2 \cdot 10^5$), each of which must be configured to spin state $x_i \in \{0, 1\}$. Selecting resonator $i$ for state $x_i = 1$ incurs an energy cost $c_i \ge 0$ ($0 \le c_i \le 10^9$), whereas state $x_i = 0$ is free.

You are given $M$ interference constraints ($0 \le M \le 2 \cdot 10^5$). The $j$-th constraint specifies two resonators $(u_j, v_j)$ and a required parity $w_j \in \{0, 1\}$: - If $w_j = 0$, resonators $u_j$ and $v_j$ must be assigned **equal** states ($x_{u_j} \oplus x_{v_j} = 0$). - If $w_j = 1$, resonators $u_j$ and $v_j$ must be assigned **opposite** states ($x_{u_j} \oplus x_{v_j} = 1$).

Determine whether a valid assignment $x \in \{0, 1\}^N$ exists, and if so, compute the **minimum total energy cost** $\sum_{i=1}^{N} c_i x_i$.

> **Pause & Discover:** Before reading further, ask yourself:
> - Once you fix the spin $x_r \in \{0, 1\}$ of one resonator $r$ in a connected component, how many choices remain for every other resonator in that same component?
> - How do you detect whether the constraints inside a component contradict one another?

### Step 1 — Exact Model & Bottleneck

An assignment is a vector $x \in \{0, 1\}^N$. Checking all $2^N$ assignments takes exponential time. However, each equation $x_u \oplus x_v = w$ is a two-variable linear constraint over $\mathbb{F}_2$.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Greedily setting each resonator's state to $0$ (since $x_i = 0$ is free) and propagating.**
Forcing $x_r = 0$ at an arbitrary starting node $r$ forces $x_v = 1$ for every node at odd XOR-distance from $r$. If those forced nodes have huge costs $c_v$, flipping the entire component (setting $x_r = 1$ instead) would make all those expensive nodes free!

**Mistake 2: Assuming standard bipartiteness (alternating colors on every edge) applies directly.**
Standard bipartiteness assumes every edge requires $x_u \neq x_v$ ($w = 1$). Here, $w = 0$ edges require $x_u = x_v$. A triangle of three $w = 0$ edges has $3$ edges (an odd cycle structurally), yet it is completely satisfiable ($x_1 = x_2 = x_3$)! Conversely, a 2-edge cycle between $u$ and $v$ with one $w = 0$ edge and one $w = 1$ edge has an even number of edges, yet is unsatisfiable!

**Concrete Small Counterexample:**
Let $N = 3$ with costs $c = [10, 100, 100]$, and constraints: - $(1, 2, w = 1)$ ($x_1 \neq x_2$), - $(1, 3, w = 1)$ ($x_1 \neq x_3$), - $(2, 3, w = 0)$ ($x_2 = x_3$).

Notice that $(1, 2, 3)$ forms a triangle, which is satisfiable because the XOR sum of edge weights around the cycle is $1 \oplus 1 \oplus 0 = 0$. Furthermore, if we greedily set $x_1 = 0$, then $x_2 = 1$ and $x_3 = 1$, paying $100 + 100 = 200$. If we instead set $x_1 = 1$, then $x_2 = 0$ and $x_3 = 0$, paying only $c_1 = 10$.

### Step 3 — The Turn: Discovering the Structure

In each connected component $C$, pick an arbitrary root vertex $r$ and assign it a reference label $\text{val}[r] = 0$. As DFS traverses any edge $(u, v, w)$: - If $v$ is unvisited, its label is uniquely forced to $\text{val}[v] = \text{val}[u] \oplus w$. - If $v$ is already visited, the edge is a back-edge (or tree edge), and consistency requires $\text{val}[v] = = \text{val}[u] \oplus w$. If this fails on any edge, the component contains a cycle of **odd XOR weight** and the entire system has no solution.

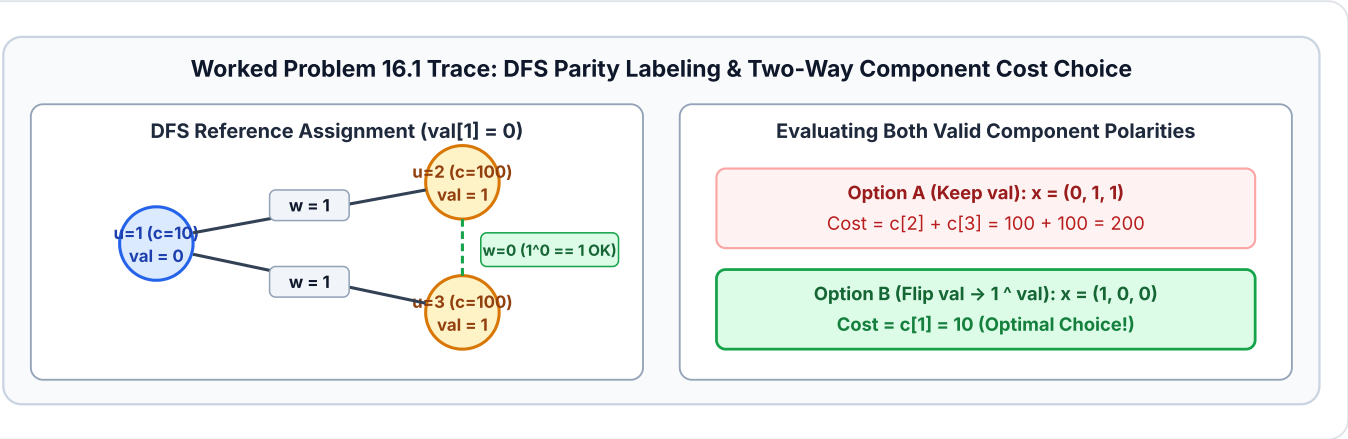
If the component is consistent, every vertex $u \in C$ is locked either to $x_r$ (if $\text{val}[u] = 0$) or to $1 - x_r$ (if $\text{val}[u] = 1$). Since components do not share edges, we choose between $x_r = 0$ and $x_r = 1$ independently for each component $C$:

**Component Parity Optimization Formula:**

$$\text{Cost}(C) = \min\left( \sum_{u \in C,\, \text{val}[u] = 1} c_u,\ \sum_{u \in C,\, \text{val}[u] = 0} c_u \right)$$

$$\text{Total Minimum Cost} = \sum_{\text{components } C} \text{Cost}(C)$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

struct Edge {
    int to;
    int weight;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    if (!(cin >> N >> M)) return 0;
    vector<long long> cost(N + 1);
    for (int i = 1; i <= N; ++i) cin >> cost[i];

    vector<vector<Edge>> adj(N + 1);
    for (int i = 0; i < M; ++i) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }
```

```
    vector<int> val(N + 1, -1);
    bool possible = true;
    long long total_min_cost = 0;

    for (int i = 1; i <= N && possible; ++i) {
        if (val[i] != -1) continue;

        long long sum_val[2] = {0, 0};
        vector<int> stk = {i};
        val[i] = 0;

        while (!stk.empty()) {
            int u = stk.back();
            stk.pop_back();
            sum_val[val[u]] += cost[u];

            for (const auto& e : adj[u]) {
                int expected = val[u] ^ e.weight;
                if (val[e.to] == -1) {
                    val[e.to] = expected;
                    stk.push_back(e.to);
                } else if (val[e.to] != expected) {
                    possible = false;
                }
            }
        }
        total_min_cost += min(sum_val[0], sum_val[1]);
    }

    if (!possible) cout << -1 << "\n";
    else cout << total_min_cost << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of Completeness via Spanning Tree Fundamental Cycles:** In each connected component $C$, the DFS tree $T_C$ fixes a unique candidate assignment $\mathrm{val}[u] = \mathrm{val}[r] \oplus \mathrm{dist}_{\oplus}(r, u)$ along tree paths. Every non-tree edge $e = (u, v, w)$ defines a fundamental cycle $C_e$. If $\mathrm{val}[u] \oplus \mathrm{val}[v] = w$ holds for every non-tree edge, then our explicit construction satisfies all $M$ edges simultaneously (both for $\mathrm{val}[r] = 0$ and $\mathrm{val}[r] = 1$). Conversely, if $\mathrm{val}[u] \oplus \mathrm{val}[v] \neq w$ for some edge $e$, summing the equations $x_a \oplus x_b = w_{ab}$ around the fundamental cycle $C_e$ yields $0 = 1$ (since every vertex appears twice on the left-hand side and cancels out, while the right-hand side sums to $1$), providing an airtight certificate of impossibility.

---

## Worked Problem 16.2: Autonomous Drone Relay Loops

### Problem & Constraints

In a sector of $N$ relay beacons ($1 \leq N \leq 2 \cdot 10^5$), each beacon $u$ is programmed with a single primary forwarding target $f[u] \in \{1, \ldots, N\}$ (note that $f[u] = u$ is possible). A diagnostic packet launched at beacon $u$ hops deterministically $u \to f[u] \to f[f[u]] \to \ldots$ until it visits some beacon for the **second** time, at which moment the packet halts to prevent infinite looping.

For every starting beacon $u \in \{1, \ldots, N\}$, compute $L[u]$: the total number of **distinct** beacons visited by a packet launched from $u$ before it halts.

> **Pause & Discover:** Before reading further, ask yourself:
> - If a graph has $N$ vertices and every vertex has out-degree $1$, what does each connected component look like?
> - If beacon $u$ is outside the cycle and steps to $v = f[u]$, how does $L[u]$ relate to $L[v]$? What if $u$ is already *on* a cycle?

### Step 1 — Exact Model & Bottleneck

Simulating the walk $u \to f[u] \to \ldots$ from each starting vertex $u$ independently until a duplicate vertex is seen takes $O(N)$ steps per vertex in the worst case (e.g., a long path feeding into a small cycle, or a single $N$-cycle). Across all $N$ starting vertices, direct simulation costs $\Theta(N^2)$ time, which times out for $N = 2 \cdot 10^5$.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Treating `visited` globally as a boolean and setting $L[u] = 1 + L[f[u]]$ whenever $f[u]$ was ever visited.**
If $u$ is **on a cycle** of length $C$, both $u$ and $f[u]$ visit the exact same $C$ vertices: $L[u] = L[f[u]] = C$, **not** $1 + L[f[u]]$!

**Mistake 2: Confusing a cross-edge to an already-solved branch (`BLACK`) with a back-edge closing a new cycle (`GRAY`).**

**Concrete Small Counterexample:**
Let $N = 4$ with $f = [2, 3, 2, 2]$ ($1 \to 2 \leftrightarrow 3$, and $4 \to 2$). - Cycle is $2 \leftrightarrow 3$ of length $2$, so $L[2] = L[3] = 2$. - Vertex $1$ steps to $2$, visiting $\{1, 2, 3\}$, so $L[1] = 1 + L[2] = 3$. - When we later start a traversal from vertex $4$, its edge $4 \to 2$ hits vertex $2$, which is already visited! If we falsely think "hitting a visited vertex means $4 \to 2$ closes a cycle," we would misclassify $4$ as part of a cycle. This is why we must distinguish `GRAY` (active on the current walk) from `BLACK` (already fully resolved).

### Step 3 — The Turn: Discovering the Structure

Because every vertex has out-degree $1$, following pointers from any `WHITE` vertex $u$ produces a single deterministic chain of `GRAY` vertices. Eventually, the chain must hit a non-`WHITE` vertex $w$:

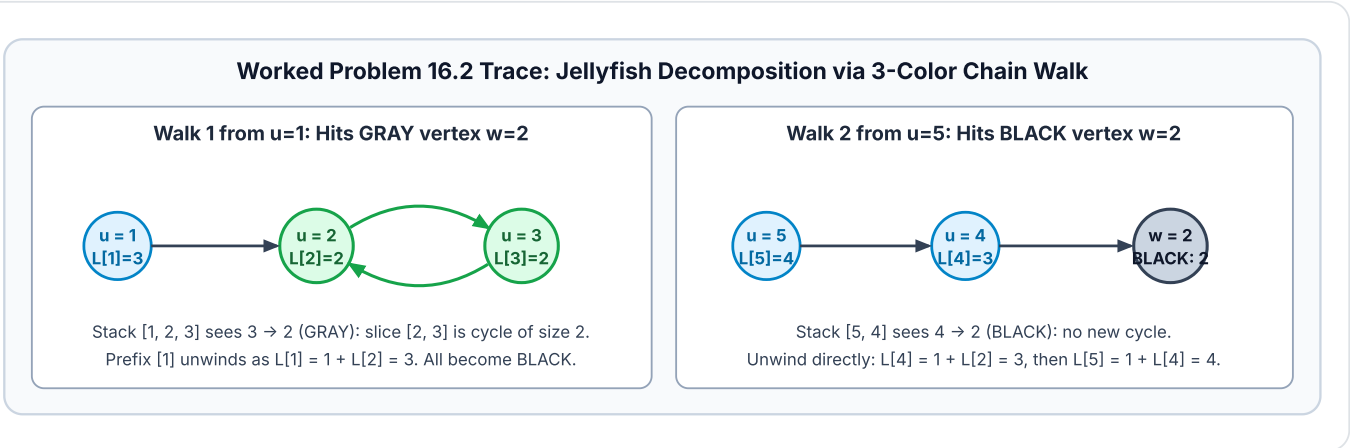
1. **Case A (`state[w] == GRAY`):** Vertex $w$ is on our current active chain! Since every vertex has out-degree $1$, all vertices on our active stack from $w$ up to the current tip form a **directed cycle** $C$. If that cycle has $k$ vertices, then for every $c \in C$, we immediately know $L[c] = k$. All earlier vertices on our chain before $w$ are tree ancestors feeding into $w$.
2. **Case B (`state[w] == BLACK`):** Vertex $w$ belongs to an earlier pass and its answer $L[w]$ is already finalized. None of the vertices on our current active chain are on a cycle; they all feed into $w$.

In both cases, once the cycle portion (if any) is assigned $L[c] = |C|$, every remaining tree vertex $u$ on the active chain satisfies:

**Functional Graph Decomposition Equations:**

$$L[u] = \begin{cases} |C| & \text{if } u \text{ lies on the directed cycle } C, \\ 1 + L[f[u]] & \text{if } u \text{ is a tree vertex feeding toward the cycle.} \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N;
    if (!(cin >> N)) return 0;
    vector<int> f(N + 1);
    for (int i = 1; i <= N; ++i) cin >> f[i];

    // state: 0 = WHITE, 1 = GRAY (on current chain), 2 = BLACK (solved)
    vector<int> state(N + 1, 0);
    vector<int> L(N + 1, 0);

    for (int i = 1; i <= N; ++i) {
        if (state[i] != 0) continue;

        vector<int> chain;
        int cur = i;
        while (state[cur] == 0) {
            state[cur] = 1;
            chain.push_back(cur);
            cur = f[cur];
        }

        if (state[cur] == 1) {
            // cur is GRAY: locate cur in chain to extract the directed cycle
            auto it = find(chain.begin(), chain.end(), cur);
            int cycle_len = static_cast<int>(chain.end() - it);
            while (chain.back() != cur) {
                int u = chain.back();
                chain.pop_back();
                L[u] = cycle_len;
                state[u] = 2;
            }
            // Also finalize cur itself
            chain.pop_back();
            L[cur] = cycle_len;
            state[cur] = 2;
        }

        // Remaining nodes in chain are tree ancestors feeding into solved nodes
        while (!chain.empty()) {
            int u = chain.back();
            chain.pop_back();
            L[u] = 1 + L[f[u]];
            state[u] = 2;
        }
    }

    for (int i = 1; i <= N; ++i) {
        cout << L[i] << (i == N ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Complexity & Correctness Proof:** Every vertex $u \in \{1, \ldots, N\}$ transitions $0 \to 1 \to 2$ (`WHITE` $\to$ `GRAY` $\to$ `BLACK`) at most once. Therefore, each vertex is pushed onto and popped from `chain` at most once across the entire run, guaranteeing $O(N)$ time and $O(N)$ auxiliary space—with no recursion stack overflow even if the graph is a single chain of length $2 \cdot 10^5$.

**Small-Case Oracle:** For $N \leq 7$, enumerate all $N^N$ functional graphs (or random samples), compute $L[u]$ by brute-force `std::set` simulation from each vertex, and assert equality with the linear 3-color chain solver. Include self-loops ($f[u] = u$, cycle of size 1) and lollipop graphs.

---

# Section 17 — Topological Sorting and DAG DP

## The Discovery & Modeling Approach

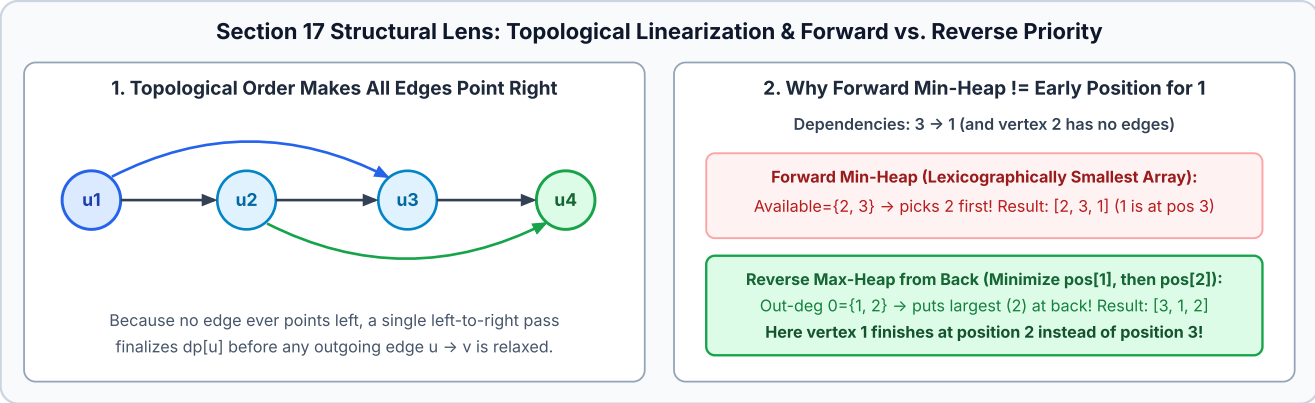
A **Directed Acyclic Graph (DAG)** is the universal backbone of dynamic programming. Every DP recurrence you have ever written—over array prefixes, grid coordinates, tree subtrees, or bitmasks—is secretly an evaluation of vertices along a topological order of a DAG. When a problem gives you an arbitrary dependency structure rather than a regular grid or array, you must **construct the topological order explicitly** before evaluating the recurrence.

### Recognizing Hidden Topological Sort and DAG DP Problems

1. **Precedence / Prerequisite Constraints:** Statements containing rules of the form *"$A$ must occur before $B$"*, *"module $u$ compiles before module $v$"*, *"inequality $x_u < x_v$"*, or *"recover an unknown alphabet order from a sorted dictionary"* translate directly into directed edges $u \to v$. A valid linear arrangement exists if and only if the directed graph is acyclic.
2. **Longest Path or Path Counting in a Directed Graph:** In a general graph, Longest Simple Path and `#P`-complete path counting are intractable because paths can loop. The moment the graph is guaranteed to be a DAG (or after you verify acyclicity via Kahn's algorithm or 3-color DFS), longest paths, shortest paths with negative weights, and total path counts all become linear-time $O(V+E)$ dynamic programs!
3. **Lexicographically Extremal Permutations under Precedence:** When multiple topological orderings exist and the problem asks for a specific tie-breaking rule, **stop and check the objective carefully**:
   - *"Make the sequence lexicographically smallest"* (minimize the 1st element, then 2nd, etc.) $\implies$ Min-heap Kahn's algorithm on the **forward** graph.
   - *"Make label $1$ appear as early as possible, then label $2$ as early as possible, …"* (minimize the inverse permutation $\mathrm{pos}[1]$, then $\mathrm{pos}[2]$, etc.) $\implies$ Max-heap Kahn's algorithm on the **reversed** graph, filling positions from $N$ down to $1$!

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in the Problem Statement | Algorithmic Mechanism | Complexity & Key Invariant |
|---|---|---|---|
| **Kahn's In-Degree Peeling** | "Find any valid execution order or report a cyclic deadlock." | Maintain `indeg[v]`; push all `indeg[v] == 0` into queue; pop $u$, decrement `indeg[v]` for $u \to v$. | $O(V+E)$. Graph is a DAG $\Leftrightarrow$ exactly $V$ vertices are popped. Unpopped vertices contain a cycle. |
| **Push / Pull DAG DP** | "Longest path, critical-path scheduling, or number of ways from $s$ to $t$ in a DAG." | Compute topological order $T[1..V]$; relax $dp[v] = \bigoplus_{u \to v} f(dp[u], w(u,v))$ in order of $T$. | $O(V+E)$. When $u$ is processed, all its predecessors have already finalized their $dp$ values. |
| **Lexicographically Smallest Topological Order** | "Among all valid schedules, minimize the ID at index $1$, then index $2$, etc." | Replace Kahn's FIFO queue with a **min-priority queue** (`greater<int>`) on the forward graph. | $O((V+E)\log V)$. Greedily places the smallest available zero-indegree vertex at the front. |
| **Reverse-Graph Priority Kahn** | "Prioritize placing vertex $1$ as early as possible, then vertex $2$, etc." | Reverse all edges; run a **max-priority queue** from the back (position $N$ down to $1$). | $O((V+E)\log V)$. Deferring large labels as late as possible pulls small labels as early as possible. |
| **Implicit Monotone Attribute DAG** | "Move only to strictly higher altitude / energy / value." | Direct edges $u \to v$ whenever $h[u] < h[v]$; sort vertices by $h$ or run Kahn's algorithm. | Any strict inequality $h[u] < h[v]$ on edges guarantees acyclicity automatically! |




---

## Worked Problem 17.1: Compiler Pipeline Critical Path & Bottleneck Count

### Problem & Constraints

A distributed build system compiles $N$ software targets ($1 \le N \le 2 \cdot 10^5$) connected by $M$ directed dependency edges ( $0 \le M \le 3 \cdot 10^5$). An edge $u \to v$ means target $u$ must finish compiling before target $v$ can begin, plus a data-transfer latency of $w(u, v) \ge 0$ milliseconds ($0 \le w \le 10^9$). Compiling target $u$ itself takes $t[u] > 0$ milliseconds ($1 \le t[u] \le 10^9$). Any number of targets whose dependencies are satisfied may compile in parallel.

1. First, verify whether the dependency graph is acyclic. If a cycle exists, output `CYCLE`.
2. Otherwise, compute the **minimum total time** $T_{\min}$ required to finish all $N$ targets, and for each target $u$, determine the number of **critical dependency chains** starting at $u$ that achieve the global completion time $T_{\min}$, modulo $10^9 + 7$.

> **Pause & Discover:** Before reading further, ask yourself:
> - Why can we compute the longest path in $O(N + M)$ time here when Longest Path is NP-hard in general graphs?
> - How do we combine earliest-finish DP (forward along topological order) with latest-start DP or suffix path counting (backward along topological order)?

### Step 1 — Exact Model & Bottleneck

With unlimited parallelism, every target $v$ starts at the exact moment all of its prerequisites $u$ have finished compiling and transferring their outputs to $v$. Thus the earliest completion time $E[v]$ of target $v$ is the maximum weight of any directed path ending at $v$ ( summing both vertex durations $t[\cdot]$ and edge latencies $w(\cdot, \cdot)$).

Enumerating all paths from sources to sinks is exponential (for instance, a chain of $N/2$ diamonds has $2^{N/2}$ paths).

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Running Dijkstra (with negated weights or max-heap) to find the longest path.**
Dijkstra's greedy invariant fails completely for longest paths (or negative-weight shortest paths): when a vertex $v$ is popped from a heap, a later multi-hop path $u \to x \to v$ can arrive with a *strictly larger* completion time, forcing $v$ and all of $v$'s descendants to be re-expanded exponentially many times on a diamond DAG!

**Mistake 2: Counting critical paths by incrementing `ways[v] += ways[u]` whenever $u \to v$ improves $E[v]$, without resetting `ways[v]` when $E[v]$ strictly increases, or forgetting that the question asks for critical chains *starting at* $u$ *and reaching the global maximum* $T_{\min}$.**

**Concrete Small Counterexample:**
Suppose $N = 3$ with vertex times $t = [10, 10, 10]$ and edges $1 \to 2$ ($w = 5$), $1 \to 3$ ($w = 20$), $2 \to 3$ ($w = 5$). - Path $1 \to 3$ finishes at $10 + 20 + 10 = 40$. - Path $1 \to 2 \to 3$ finishes at $10 + 5 + 10 + 5 + 10 = 40$. If we used a heap-based relaxation before all predecessors of $3$ were finalized, $3$ would first be relaxed via $1 \to 3$ to $40$ and popped before $2$ is even expanded via $2 \to 3$.

### Step 3 — The Turn: Discovering the Structure

Kahn's algorithm produces a topological order $\text{topo}[0..N-1]$ in $O(N+M)$ time and simultaneously certifies acyclicity: the graph is a DAG if and only if $|\text{topo}| = N$.

Once $\text{topo}$ is fixed, we run two linear scans over $\text{topo}$: 1. **Forward Scan (Earliest Finish $E[v]$):** Initialize $E[v] = t[v]$. Iterate $u$ left-to-right in $\text{topo}$; for each edge $u \to v$ with weight $w$:

$$E[v] = \max\big(\, E[v],\ E[u] + w + t[v] \,\big)$$

Then $T_{\min} = \max_{1 \le v \le N} E[v]$. 2. **Backward Scan (Longest Remaining Tail $R[u]$ & Critical Chain Count $W[u]$):** To know which chains starting at $u$ achieve $T_{\min}$, let $R[u]$ be the maximum time of a path starting at $u$ (including $t[u]$), and $W[u]$ be the number of paths starting at $u$ of total weight $R[u]$. Iterate $u$ **right-to-left** in $\text{topo}$:

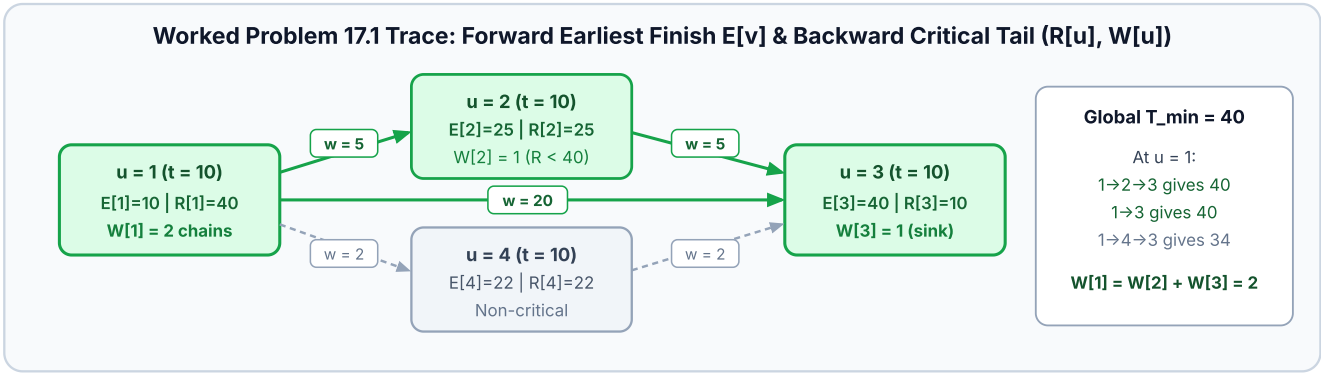
**Forward-Backward Critical DAG Recurrences:**

$$R[u] = t[u] + \max\big(\, \{0\} \cup \{\, w(u,v) + R[v] \mid u \to v \in E \,\} \,\big)$$

$$W[u] = \begin{cases} 1 & \text{if } R[u] = t[u] \text{ (no outgoing edge extends } u), \\ \displaystyle\sum_{\substack{u \to v \in E \\ t[u] + w(u,v) + R[v] = R[u]}} W[v] \pmod{10^9+7} & \text{otherwise.} \end{cases}$$

$$\text{Critical chains starting at } u \text{ that achieve } T_{\min} = \begin{cases} W[u] & \text{if } R[u] = T_{\min}, \\ 0 & \text{if } R[u] < T_{\min}. \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

using namespace std;

const long long MOD = 1000000007LL;

struct Edge {
    int to;
    long long weight;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    if (!(cin >> N >> M)) return 0;

    vector<long long> t(N + 1);
    for (int i = 1; i <= N; ++i) cin >> t[i];
```

```
vector<vector<Edge>> adj(N + 1);
vector<int> indeg(N + 1, 0);
for (int i = 0; i < M; ++i) {
    int u, v;
    long long w;
    cin >> u >> v >> w;
    adj[u].push_back({v, w});
    indeg[v]++;
}

queue<int> q;
for (int i = 1; i <= N; ++i) {
    if (indeg[i] == 0) q.push(i);
}

vector<int> topo;
topo.reserve(N);
while (!q.empty()) {
    int u = q.front();
    q.pop();
    topo.push_back(u);
    for (const auto& e : adj[u]) {
        if (--indeg[e.to] == 0) {
            q.push(e.to);
        }
    }
}

if (static_cast<int>(topo.size()) < N) {
    cout << "CYCLE\n";
    return 0;
}

// Forward DP: earliest finish time E[u]
vector<long long> E(N + 1);
long long T_min = 0;
for (int i = 1; i <= N; ++i) E[i] = t[i];
for (int u : topo) {
    T_min = max(T_min, E[u]);
    for (const auto& e : adj[u]) {
        E[e.to] = max(E[e.to], E[u] + e.weight + t[e.to]);
    }
}

// Backward DP: longest tail R[u] and count W[u]
vector<long long> R(N + 1, 0);
vector<long long> W(N + 1, 1);
for (int i = N - 1; i >= 0; --i) {
    int u = topo[i];
    R[u] = t[u];
    W[u] = 1; // Base case: chain of just {u}
    for (const auto& e : adj[u]) {
        long long cand = t[u] + e.weight + R[e.to];
        if (cand > R[u]) {
            R[u] = cand;
            W[u] = W[e.to];
        } else if (cand == R[u]) {
            W[u] = (W[u] + W[e.to]) % MOD;
        }
    }
}

cout << T_min << "\n";
for (int i = 1; i <= N; ++i) {
    long long ans = (R[i] == T_min) ? W[i] : 0;
```

```
        cout << ans << (i == N ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of Cycle Certificate and Path Bijection:** - *Cycle Certificate:* Any vertex in a directed cycle has at least one predecessor inside the cycle, so its `indeg` can never drop to $0$. Conversely, if the remaining subgraph is nonempty and has no zero-indegree vertex, walking backward along incoming edges from any remaining vertex visits a repeated vertex within $\leq N$ steps, exhibiting a cycle. - *DP Induction:* Because `topo` places every $u$ before $v$ whenever $u \to v$, in the reverse loop every $v$ has already finalized $(R[v], W[v])$ before $u$ inspects edge $u \to v$. Since $t[v] > 0$ and $w(u, v) \geq 0$, every outgoing edge strictly increases $cand \geq t[u] + t[v] > t[u]$, so the base case $W[u] = 1$ (stopping at $u$) is cleanly overwritten as soon as at least one outgoing edge exists.

---

## Worked Problem 17.2: Priority Charter Scheduling (Reverse-Graph Kahn)

### Problem & Constraints

A parliament must vote on $N$ bills ($1 \leq N \leq 2 \cdot 10^5$) numbered $1$ through $N$ in decreasing order of urgency (bill $1$ is the most urgent, bill $2$ is second most urgent, and so on). There are $M$ prerequisite clauses ($0 \leq M \leq 2 \cdot 10^5$): clause $(u, v)$ states that bill $u$ must be voted on before bill $v$. The clauses are guaranteed to be acyclic.

Let $P = (p_1, p_2, ..., p_N)$ be a valid voting schedule (a topological permutation of $1..N$), and let $\text{pos}_P(x)$ denote the 1-based position of bill $x$ in $P$. You must find the schedule $P$ that minimizes $\text{pos}_P(1)$; among all schedules that place bill $1$ as early as possible, minimize $\text{pos}_P(2)$; and so on for $x = 1, 2, ..., N$.

> **Pause & Discover:** Before reading further, ask yourself:
> - Consider $N = 3$ with the single constraint $3 \to 1$. What does a forward min-heap Kahn algorithm output? Where does bill $1$ land?
> - Instead of deciding which bill goes first at index $1$, what happens if we decide which bill goes *last* at index $N$?

### Step 1 — Exact Model & Bottleneck

We want to minimize the inverse-permutation tuple $\left(\text{pos}_P(1), \text{pos}_P(2), ..., \text{pos}_P(N)\right)$ lexicographically over all topological sortings $P$. Enumerating all valid topological sortings takes up to $N!$ time.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Running standard lexicographically smallest topological sort (a min-heap on the forward graph).**
As shown in our Section 17 Structural Lens diagram, with $N = 3$ and single edge $3 \to 1$: - At step 1, available zero-indegree bills are $\{2, 3\}$. A forward min-heap picks $2$ because $2 < 3$, producing $P_{\text{fwd}} = [2, 3, 1]$, where $\text{pos}(1) = 3$. - Yet $P^* = [3, 1, 2]$ is also a valid topological order, and in $P^*$ bill $1$ is voted on at $\text{pos}(1) = 2 < 3$! Why did forward min-heap fail? Because picking $2$ ahead of $3$ delayed $3$, which was the bottleneck holding back $1$!

**Mistake 2: Assign each vertex $u$ a priority equal to the minimum label reachable from $u$ in the DAG, then run forward min-heap keyed by `(min_reachable[u], u)`.**
Even this clever heuristic fails!

**Concrete Small Counterexample to the `min_reachable` Forward Heuristic:**
Let $N = 5$ with chains $4 \to 5 \to 1$ and $3 \to 2$. - Both chain roots $4$ (leading to $1$ in two steps) and $3$ (leading to $2$ in one step) are available. - Clearly we must run $4 \to 5 \to 1$ first so that $1$ finishes at position $3$ (`[4, 5, 1, 3, 2]`). - Now suppose instead we have $5 \to 2$ and $4 \to 3$, and both $2$ and $3$ are prerequisites for $1$ ($2 \to 1$ and $3 \to 1$). Both $5$ and $4$ have `min_reachable = 1`! Regardless of whether $5 \to 2$ or $4 \to 3$ goes first, $1$ cannot happen until position $5$. Once $1$ is locked at position $5$, we want **bill** $2$ to finish as early as possible—which requires picking $5$ before $4$ (`[5, 2, 4, 3, 1]`), whereas tie-breaking by the smaller root ID $4 < 5$ picks `[4, 3, 5, 2, 1]` and delays bill $2$ from position $2$ to position $4$!

### Step 3 — The Turn: Discovering the Structure

Why is forward decision-making hard? Because placing an element at the *front* of the schedule requires knowing the entire structure of descendants it unlocks.

Now flip the perspective: **what bill should we place at the very last position $p_N$?** Any bill placed at $p_N$ must have out-degree $0$ (no remaining bill depends on it). Suppose we always choose the **largest possible label** among all currently available out-degree-$0$ bills to go at the back of the remaining schedule ($p_N$, then $p_{N-1}$, …, down to $p_1$).

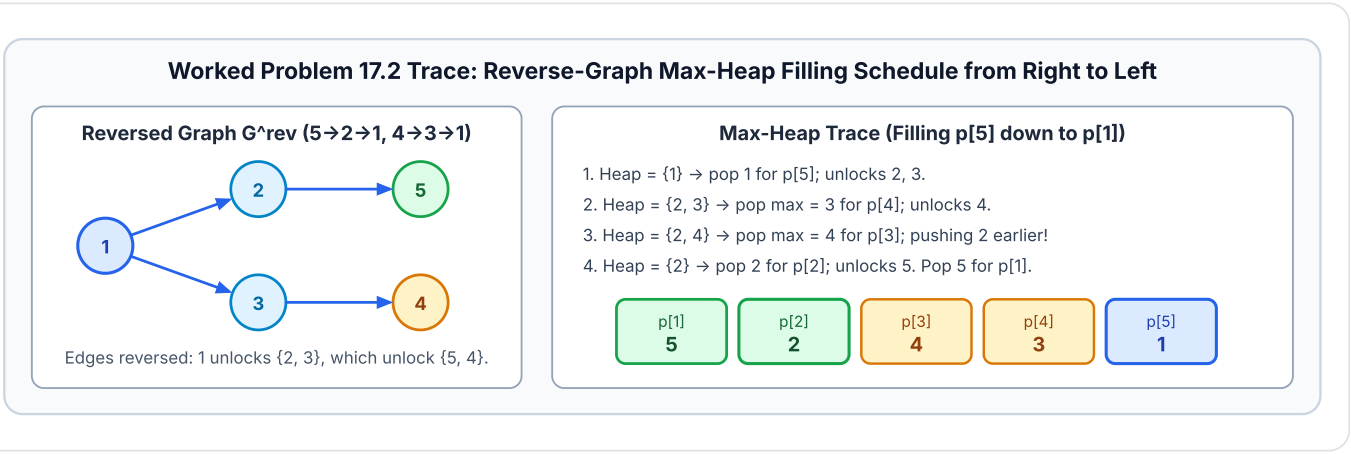
**Reverse-Graph Exchange Principle:**

$$\text{Minimizing } \big(\,\text{pos}(1), \text{pos}(2), \ldots, \text{pos}(N)\,\big) \Leftrightarrow \text{Maximizing } \big(\,p_N, p_{N-1}, \ldots, p_1\,\big) \text{ from back to front!}$$

Why is that equivalence true? We prove it in Step 5 below! Algorithmically, this means we simply: 1. Build the **reversed graph** $G^{\text{rev}}$ where each constraint $u \to v$ becomes a reversed edge $v \to u$, and track `outdeg[u]` in $G$ (which is the in-degree in $G^{\text{rev}}$). 2. Maintain a **max-heap** of vertices with `outdeg[u] == 0`. 3. Pop the maximum label $u$, place it at the current rightmost unfilled slot of the schedule, decrement `outdeg` of its predecessors, and repeat.

### Step 4 — Progressive Construction & Visual Trace

Let us trace the counterexample $5 \to 2 \to 1$ and $4 \to 3 \to 1$ using the Reverse Max-Heap algorithm:




```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M;
    if (!(cin >> N >> M)) return 0;

    vector<vector<int>> rev_adj(N + 1);
    vector<int> outdeg(N + 1, 0);
    for (int i = 0; i < M; ++i) {
        int u, v;
        cin >> u >> v;
        rev_adj[v].push_back(u);
        outdeg[u]++;
    }

    priority_queue<int> max_pq;
    for (int i = 1; i <= N; ++i) {
        if (outdeg[i] == 0) max_pq.push(i);
```

```
    }

    vector<int> schedule(N + 1);
    for (int pos = N; pos >= 1; --pos) {
        int u = max_pq.top();
        max_pq.pop();
        schedule[pos] = u;
        for (int pred : rev_adj[u]) {
            if (--outdeg[pred] == 0) {
                max_pq.push(pred);
            }
        }
    }

    for (int pos = 1; pos <= N; ++pos) {
        cout << schedule[pos] << (pos == N ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Exchange Proof of Optimality:** Let $P$ be our reverse-max-heap schedule and let $Q$ be the truly optimal schedule minimizing $(\text{pos}(1), ..., \text{pos}(N))$. Suppose $P \neq Q$, and let $k$ be the **largest index** (closest to $N$) where $p_k \neq q_k$. 1. Since $P$ and $Q$ agree on positions $k+1 ... N$, the set of remaining unplaced vertices is identical at step $k$. By our max-heap choice, $p_k$ is the maximum out-degree-$0$ vertex in that remaining subgraph, so $p_k > q_k$. 2. Where is $p_k$ located in $Q$? It must appear at some earlier index $j < k$ in $Q$ ($q_j = p_k$). Because $p_k$ has out-degree $0$ in the remaining subgraph on $\{q_1, ..., q_k\}$, **none** of the vertices $q_{j+1}, ..., q_k$ depend on $p_k$! 3. Therefore, we can slide $q_j = p_k$ in $Q$ rightward from index $j$ all the way to index $k$, shifting every element in $Q[j+1 ... k]$ one position to the left! 4. How does this shift affect the objective $(\text{pos}(1), ..., \text{pos}(N))$? Every vertex $x \in Q[j+1 ... k]$ moves one step *earlier* ($\text{pos}(x)$ decreases by $1$), while only $p_k$ moves later. Specifically, $q_k < p_k$ moves left, and any even smaller labels in $Q[j+1 ... k]$ also move left! Thus the shifted schedule $Q'$ is strictly better than $Q$ on the smallest label in $\{q_k\} \cup Q[j+1 ... k]$, contradicting the optimality of $Q$.

---

# Section 18 — DSU and Minimum Spanning Trees

## The Discovery & Modeling Approach

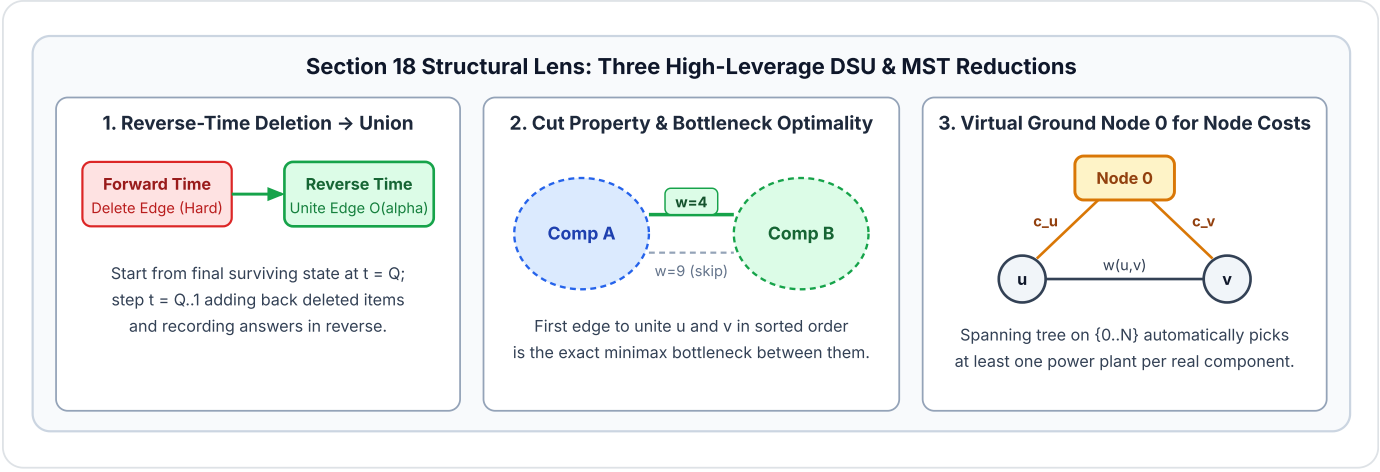
A **Disjoint Set Union (DSU)**—also called Union-Find—maintains a partition of $N$ elements under dynamic equivalence merges (`unite(u, v)`) and component queries (`find(u)`). While its textbook application is Kruskal's Minimum Spanning Tree (MST) algorithm, in competitive programming DSU is much broader: it is the go-to engine for **offline threshold queries, time-reversal of destructive deletions, minimax bottleneck paths, and dynamic algebraic invariants over spanning forests**.

### Recognizing Hidden DSU and MST Problems

1. **Symmetric Transitive Equivalence ("If $A \sim B$ and $B \sim C$, then $A \sim C$"):** Whenever an operation merges two groups, equates two variables, or opens a two-way channel, DSU tracks component size, minimum/maximum member, or sum of weights in $O(\alpha(N))$ amortized time per operation.
2. **Destructive Edge/Vertex Deletions:** DSU supports `unite` effortlessly, but cannot `split` an arbitrary edge in $O(\alpha(N))$ online. If a problem deletes edges or vertices one by one and asks for connectivity after each deletion, **read all operations offline and run time backward**: start from the final surviving graph, and turn each deletion into a DSU `unite`!
3. **Minimax / Bottleneck Path Queries ("Minimize the maximum edge weight along a path from $u$ to $v$"):** By Kruskal's Cut Property, the path between $u$ and $v$ inside any Minimum Spanning Tree simultaneously minimizes the maximum edge weight between $u$ and $v$ over the entire graph!
4. **Relative Potentials / Parity on Dynamic Forests:** By storing a weight `diff[x]` on every DSU parent pointer representing the potential difference (or XOR difference) between `x` and `parent[x]`, path compression can accumulate potentials along the path to the root, answering $x_u - x_v$ (or $x_u \oplus x_v$) dynamically as edges arrive!

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in the Problem Statement | DSU / MST Transformation | Complexity & Key Invariant |
|---|---|---|---|
| **Kruskal's MST & Cut Property** | "Connect all nodes at minimum total cost" or "cheapest network." | Sort edges by weight ascending; `unite(u, v)` iff `find(u) != find(v)`. | $O(M \log M)$. Lightest edge crossing any cut of the current forest is always safe. |
| **Reverse-Time Offline DSU** | "Nodes/edges are destroyed over time; report connected components or reachability after each step." | Mark destroyed items; build initial DSU of survivors; process events from $Q$ down to $1$ via `unite`. | $O((N+M+Q)\alpha(N))$. Reverses irreversible deletions into monotone additions. |
| **Offline Threshold / Bottleneck Sort** | "For each query $(u_q, v_q, W_q)$, can $u_q$ reach $v_q$ using only edges with weight $\leq W_q$?" | Sort both edges and queries by weight $W$; insert edges $\leq W_q$ into DSU before checking `find(u_q) == find(v_q)`. | $O((M+Q)\log M + (M+Q)\alpha(N))$. Monotone edge activation replaces per-query BFS. |
| **DSU with Potentials / Parity** | "Online stream of equations $x_u - x_v = d$; reject contradictions and answer queries." | Maintain `pot[x]` = value of $x$ relative to `parent[x]`. Update `pot[x] += pot[p]` during `find(x)`. | $O(\alpha(N))$ per equation. Path compression telescopes sums along the tree path to the root. |
| **Virtual Super-Node MST** | "Each node $u$ can either build a local power plant for $c_u$ or connect to neighbors via cables." | Create virtual node $0$; add edge $(0, u)$ of cost $c_u$ for every $u$; run Kruskal on $N+1$ vertices. | $O((N+M)\log(N+M))$. Turns vertex selection costs into standard edge weights. |




---

## Worked Problem 18.1: Grid Blackout Cascade (Reverse-Time DSU)

### Problem & Constraints

A regional power grid consists of $N$ substations ($1 \le N \le 2 \cdot 10^5$) and $M$ bidirectional transmission lines ($0 \le M \le 3 \cdot 10^5$). Substations $1, \ldots, K$ ($1 \le K \le N$) house backup generators; any substation connected via active transmission lines to **at least one** generator substation remains powered.

Over a sequence of $Q$ storm events ($1 \le Q \le M$), one transmission line $e_t$ is severed at each step $t = 1, 2, \ldots, Q$ (no line is severed twice). After **each** of the $Q$ storm events, report the total number of substations that are still powered.

**Pause & Discover:** Before reading further, ask yourself:

- When a single line $(u, v)$ is severed in a graph with cycles, why is it difficult to tell in $O(1)$ time whether $u$ and $v$ have become disconnected?
- What happens if we start with only the $M - Q$ lines that survive the entire storm, and re-insert $e_Q, e_{Q-1}, \ldots, e_1$ one by one?

### Step 1 — Exact Model & Bottleneck

After event $t$, the active graph $G_t$ contains all edges except $\{e_1, \ldots, e_t\}$. Running a multi-source BFS or DFS from the $K$ generator substations after each of the $Q$ deletions costs $\Theta(Q(N+M))$ time, which reaches $6 \cdot 10^{10}$ operations.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake: Maintaining a count of "generator-reachable neighbors" per vertex and decrementing it when an edge is deleted.** Reachability is a global property, not a local degree count! Two non-generator vertices $u$ and $v$ can keep each other's local counts positive around a cycle long after the single bridge connecting their component to a generator has been severed.

**Concrete Small Counterexample:**
Let $N = 4, K = 1$ (generator at 1), with bridge $1 - 2$ and triangle $2 - 3 - 4 - 2$. When edge $1 - 2$ is severed, vertices $2, 3, 4$ still have two active neighbors each in the triangle $2 - 3 - 4$, yet all three lose power simultaneously!

### Step 3 — The Turn: Discovering the Structure

In forward time, edges are removed ($G_0 \supset G_1 \supset \ldots \supset G_Q$). In **reverse time**, edges are strictly added:

$$G_Q \subset G_{Q-1} \subset \ldots \subset G_0$$

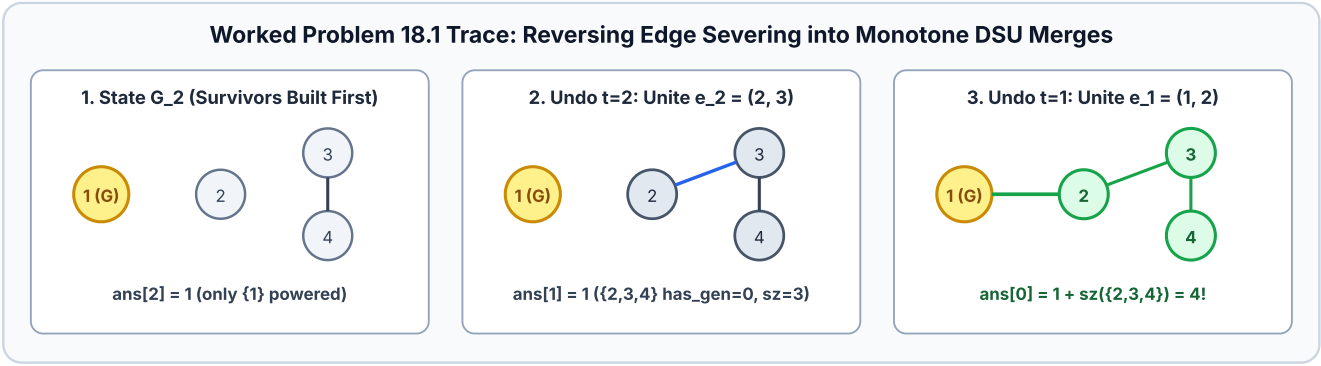
In our DSU, we maintain two piece of metadata at every component root $r$: - `sz[r]`: the total number of substations in component $r$, - `has_gen[r]`: a boolean flag (`true` if component $r$ contains at least one generator in $\{1, \ldots, K\}$).

We also maintain a single global integer `powered_count`, initially equal to $K$ (when all $N$ vertices are isolated, only the $K$ generator substations themselves are powered). Whenever we unite two distinct components $r_u = \text{find}(u)$ and $r_v = \text{find}(v)$:

**Powered-Component Merge Transition:**

$$\Delta(\text{powered_count}) = \begin{cases} +\text{sz}[r_v] & \text{if has_gen}[r_u] = \text{true and has_gen}[r_v] = \text{false}, \\ +\text{sz}[r_u] & \text{if has_gen}[r_u] = \text{false and has_gen}[r_v] = \text{true}, \\ 0 & \text{if both already have generators, or neither does.} \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <numeric>

using namespace std;

struct DSU {
    vector<int> parent, sz;
    vector<bool> has_gen;
    int powered_count;

    DSU(int n, int k) : parent(n + 1), sz(n + 1, 1), has_gen(n + 1, false), powered_count(k) {
        iota(parent.begin(), parent.end(), 0);
        for (int i = 1; i <= k; ++i) has_gen[i] = true;
    }

    int find(int x) {
        while (x != parent[x]) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    void unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (sz[ra] < sz[rb]) swap(ra, rb);

        if (has_gen[ra] && !has_gen[rb]) {
            powered_count += sz[rb];
        } else if (!has_gen[ra] && has_gen[rb]) {
            powered_count += sz[ra];
        }

        parent[rb] = ra;
        sz[ra] += sz[rb];
        has_gen[ra] = has_gen[ra] || has_gen[rb];
    }
```

```
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M, K, Q;
    if (!(cin >> N >> M >> K >> Q)) return 0;

    vector<pair<int, int>> edges(M + 1);
    for (int i = 1; i <= M; ++i) {
        cin >> edges[i].first >> edges[i].second;
    }

    vector<int> severed(Q);
    vector<bool> is_removed(M + 1, false);
    for (int i = 0; i < Q; ++i) {
        cin >> severed[i];
        is_removed[severed[i]] = true;
    }

    DSU dsu(N, K);
    for (int i = 1; i <= M; ++i) {
        if (!is_removed[i]) {
            dsu.unite(edges[i].first, edges[i].second);
        }
    }

    vector<int> answers(Q);
    for (int t = Q - 1; t >= 0; --t) {
        answers[t] = dsu.powered_count;
        int eid = severed[t];
        dsu.unite(edges[eid].first, edges[eid].second);
    }

    for (int t = 0; t < Q; ++t) {
        cout << answers[t] << "\n";
    }
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Invariant Proof:** At the start of reverse iteration $t$ (before re-inserting `severed[t]`), the set of edges united in `dsu` is precisely $E \backslash \{\text{severed}[0], \ldots, \text{severed}[t]\}$, which is the exact edge set of $G_{t+1}$ (after 0-indexed storm event $t$). Furthermore, `powered_count` equals $\sum_{r \text{ root, has_gen}[r]} \text{sz}[r]$ because every `unite` adds the size of an unpowered component if and only if it merges with a powered component.

---

## Worked Problem 18.2: Heavy-Haul Clearance Certification (Offline Kruskal Bottleneck)

### Problem & Constraints

A mountain highway network connects $N$ depots ($1 \le N \le 2 \cdot 10^5$) via $M$ bidirectional bridges ($1 \le M \le 3 \cdot 10^5$). Bridge $i$ has a maximum axle-load rating $w_i$ ($1 \le w_i \le 10^9$). You are given $Q$ transport manifests ($1 \le Q \le 2 \cdot 10^5$). Manifest $j$ asks: what is the **maximum possible weight** $W_j$ of a truck that can travel between depot $u_j$ and depot $v_j$ (or `0` if $u_j$ and $v_j$ are in different connected components)?

Furthermore, for a fixed government inspection threshold $L$, also support computing for each query $(u_j, v_j, L_j)$ how many distinct pairs of depots in $u_j$'s reachable component can mutually communicate using only bridges with load rating $\ge L_j$.

**Pause & Discover:** Before reading further, ask yourself:

- If we sort bridges in *decreasing* order of load rating $w_i$, what does the state of the DSU after inserting all bridges with $w_i \geq L$ represent?
- Why does sorting queries by $L_j$ descending allow us to answer all $Q$ component-size queries in a single pass over the edges?

### Step 1 — Exact Model & Bottleneck

A truck of weight $W$ can traverse a path $P$ if and only if $\min_{e \in P} w_e \geq W$. Thus for each query $(u_j, v_j, L_j)$, only the subgraph $G_{\geq L_j} = (V, \{e \in E \mid w_e \geq L_j\})$ is passable. Building $G_{\geq L_j}$ from scratch for each query takes $\Theta(Q(N+M))$ time.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Running Dijkstra from each $u_j$ to maximize the minimum edge weight.**
While widest-path Dijkstra works for a single source, running it $Q = 2 \cdot 10^5$ times from different sources $u_j$ still takes $\Theta(QM \log N)$ time.

**Mistake 2: Sorting edges ascending (like standard minimum spanning tree) when the feasibility condition is $w_e \geq L_j$.**
Here, larger weights are *better* (higher clearance). To activate valid bridges as the threshold $L_j$ relaxes downward, both bridges and queries must be sorted in **non-increasing (descending)** order of weight!

**Concrete Small Counterexample:**
Suppose depots $1, 2, 3$ have bridges $(1, 2, w = 50)$, $(2, 3, w = 80)$, and $(1, 3, w = 70)$. - A truck from $1$ to $2$ shouldn't take the direct bridge $(1, 2)$ of limit $50$; by taking the detour $1 \to 3 \to 2$, its bottleneck is $\min(70, 80) = 70 > 50$! - Notice that in the **Maximum Spanning Tree**, the two heaviest edges $(2, 3, 80)$ and $(1, 3, 70)$ are chosen, and the edge $(1, 2, 50)$ is rejected as a cycle chord.

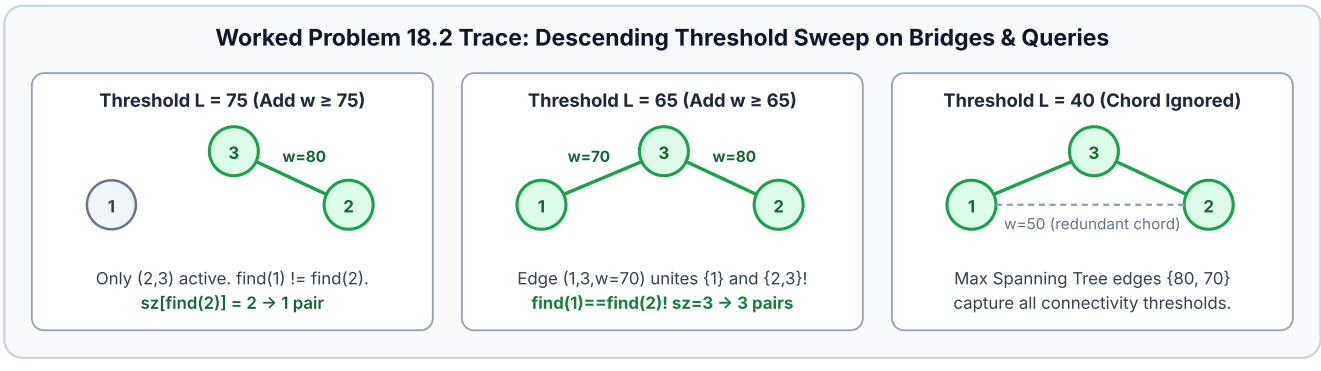
### Step 3 — The Turn: Discovering the Structure

Because the subgraph $G_{\geq L}$ grows monotonically as $L$ decreases ($L_1 \geq L_2 \implies G_{\geq L_1} \subseteq G_{\geq L_2}$), we can sort all $M$ bridges by $w_e$ descending and all $Q$ queries by $L_j$ descending. A single two-pointer sweep advances the edge pointer to `unite` every bridge with $w_e \geq L_j$ before answering query $j$ in $O(\alpha(N))$ time:

**Offline Threshold DSU Invariant:**

$$\text{After inserting all } e \in E \text{ with } w_e \geq L_j\text{:}$$
$$\text{Can } u_j \text{ reach } v_j \text{ with load } L_j\text{?} \Leftrightarrow \text{find}(u_j) == \text{find}(v_j)$$
$$\text{Communicating pairs in } u_j\text{'s component} = \binom{\text{sz}[\text{find}(u_j)]}{2}$$

### Step 4 — Progressive Construction & Visual Trace




```
#include <iostream>
#include <vector>
#include <numeric>
#include <algorithm>
```

```cpp
using namespace std;

struct Bridge {
    int u, v;
    long long w;
    bool operator<(const Bridge& other) const {
        return w > other.w; // Sort descending by weight
    }
};

struct Query {
    int u, v;
    long long L;
    int id;
    bool operator<(const Query& other) const {
        return L > other.L; // Sort descending by threshold L
    }
};

struct DSU {
    vector<int> parent;
    vector<long long> sz;

    DSU(int n) : parent(n + 1), sz(n + 1, 1) {
        iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        while (x != parent[x]) {
            parent[x] = parent[parent[x]];
            x = parent[x];
        }
        return x;
    }

    void unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return;
        if (sz[ra] < sz[rb]) swap(ra, rb);
        parent[rb] = ra;
        sz[ra] += sz[rb];
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M, Q;
    if (!(cin >> N >> M >> Q)) return 0;

    vector<Bridge> bridges(M);
    for (int i = 0; i < M; ++i) {
        cin >> bridges[i].u >> bridges[i].v >> bridges[i].w;
    }
    sort(bridges.begin(), bridges.end());

    vector<Query> queries(Q);
    for (int i = 0; i < Q; ++i) {
        cin >> queries[i].u >> queries[i].v >> queries[i].L;
        queries[i].id = i;
    }
    sort(queries.begin(), queries.end());

    DSU dsu(N);
```

```
    vector<pair<int, long long>> ans(Q);
    int edge_idx = 0;

    for (const auto& q : queries) {
        while (edge_idx < M && bridges[edge_idx].w >= q.L) {
            dsu.unite(bridges[edge_idx].u, bridges[edge_idx].v);
            edge_idx++;
        }
        int ru = dsu.find(q.u);
        int rv = dsu.find(q.v);
        long long s = dsu.sz[ru];
        ans[q.id] = { (ru == rv ? 1 : 0), s * (s - 1) / 2 };
    }

    for (int i = 0; i < Q; ++i) {
        cout << ans[i].first << " " << ans[i].second << "\n";
    }
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Cut Property Proof of Bottleneck Equivalence:** Let $T$ be the Maximum Spanning Forest constructed by processing bridges in non-increasing order of weight. Suppose $u$ and $v$ first become connected when bridge $e^* = (a, b)$ of weight $w^*$ is processed. 1. *Achievability ($\geq w^*$):* Every edge already in $T$ has weight $\geq w^*$, so the unique tree path between $u$ and $v$ in $T$ has minimum edge weight $w^*$. 2. *Upper Bound ($\leq w^*$):* Just before $e^*$ was processed, $u$ and $v$ belonged to different components $C_u$ and $C_v$. Any path $P$ between $u$ and $v$ in the original graph $G$ must cross the cut $(C_u, V \backslash C_u)$ on some edge $e' \in P$. Since $e'$ was not yet in the DSU when $e^*$ was reached in sorted order, $w(e') \leq w(e^*) = w^*$. Thus no path in $G$ can have bottleneck strictly greater than $w^*$.

---

# Section 19 — Dijkstra and Weighted Modeling

## The Discovery & Modeling Approach

Whenever actions have **non-negative, non-uniform costs** ($w(e) \geq 0$), Breadth-First Search loses its first-visit optimality: a path with $5$ edges of weight $2$ beats a direct edge of weight $15$. Dijkstra's algorithm restores optimality by replacing the FIFO queue with a **min-priority queue** ordered by tentative distance $d[u]$.

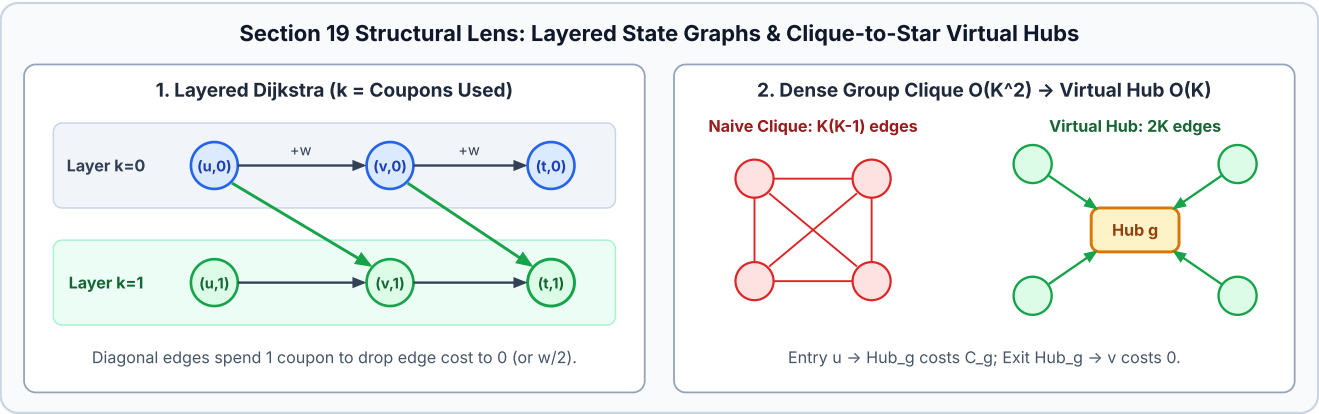
In advanced competitive programming, the difficulty of a Section 19 problem rarely lies in coding the 15-line Dijkstra loop—it lies in **designing the state graph and its edge set** so that: 1. The state is **Markovian** (everything that affects future legal moves or edge discounts is part of the vertex tuple), and 2. The edge count is **sparse** (dense clique transitions are factored through auxiliary virtual hubs).

### Recognizing Hidden Weighted Modeling & Dijkstra Problems

1. **Limited Resource / Coupon / State-Dependent Transitions:** If you can halve the cost of at most $K \leq 10$ roads, recharge a small battery of capacity $B \leq 50$, or pay a switching penalty when changing transit modes (e.g., road to rail), plain `dist[u]` fails because arriving at $u$ with higher cost but *more remaining coupons* may win later. Lifting vertices to `(u, k)` turns the resource into $K+1$ copies of the graph connected by inter-layer edges!
2. **Dense Group / Clique Transitions ("Any node in set $S_g$ can jump to any other node in $S_g$ for cost $C_g$"):** Explicitly adding $\binom{|S_g|}{2}$ edges per group blows up $M$ to $\Theta(N^2)$. Instead, introduce a **virtual hub vertex** $H_g$ for group $g$: every $u \in S_g$ has an entry edge $u \to H_g$ of cost $C_g$ (or $C_g/2$) and an exit edge $H_g \to v$ of cost $0$, shrinking $\binom{|S_g|}{2}$ edges to $2|S_g|$!
3. **Time-Dependent Waiting / Periodic Departures:** If edge $u \to v$ only departs at multiples of $P_e$ (or opens at time $t \geq T_e$) and waiting at $u$ is allowed, the earliest arrival time at $v$ when leaving $u$ at time $d[u]$ is a **monotone (non-decreasing) function** $f_e(d[u])$. Whenever $f_e(t) \geq t$ is non-decreasing, standard Dijkstra on `dist[u]` works without expanding time into the state!

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in the Problem Statement | State & Edge Modeling | Complexity & Key Invariant |
|---|---|---|---|
| **Standard Lazy-Deletion Dijkstra** | "Cheapest path with non-negative road weights $w_e \geq 0$." | Push `(dist[v], v)`; upon pop `(d, u)`, `if (d != dist[u]) continue;` | $O((V+E)\log V)$. First time `(dist[u], u)` is popped, `dist[u]` is final. |
| **Layered / Coupon State-Space Dijkstra** | "At most $K$ edges can be free / discounted / reversed" ($K \leq 20$). | State `(u, k)` for $k \in \{0..K\}$. Normal edge: `(u, k) -> (v, k)` cost $w$; coupon edge: `(u, k) -> (v, k+1)` cost $\lfloor w/2 \rfloor$. | $O(K(V+E)\log(KV))$. Cross-layer edges point strictly upward ($k \to k+1$). |
| **Virtual Hub / Star Reduction** | "Teleport between any two stations on line $g$ (or sharing prime factor $p$) for cost $C_g$." | Add hub node $N+g$. Add $u \to \text{Hub}_g$ (cost $C_g$) and $\text{Hub}_g \to u$ (cost $0$) for $u \in S_g$. | $O((V_{\text{total}} + \sum \lvert S_g \rvert)\log V_{\text{total}})$. Replaces $O(\lvert S_g \rvert^2)$ clique with $2\lvert S_g \rvert$ star edges. |
| **Reverse-Graph Single-Target Dijkstra** | "Find shortest path from *every* vertex $u$ to a single destination $T$ in a directed graph." | Reverse all directed edges $u \to v$ into $v \to u$; run **one** Dijkstra starting from $T$. | $O((V+E)\log V)$ instead of $V$ separate runs. |
| **Shortest-Path DAG & Lexicographic Tie-Breaking** | "Count shortest paths, or find the shortest path with minimum number of edges." | Maintain tuple `(dist[u], hops[u])` or accumulate `ways[v] += ways[u]` on equality `dist[u] + w == dist[v]`. | When $u$ is popped, all shortest-path predecessors of $u$ have already been popped (if $w_e > 0$). |




---

## Worked Problem 19.1: Express Overdrive Vouchers (Layered State-Space Dijkstra)

### Problem & Constraints

A continental freight network has $N$ cities ($2 \le N \le 10^5$) and $M$ bidirectional highways ($1 \le M \le 2 \cdot 10^5$). Highway $i$ connects $u_i$ and $v_i$ with base fuel cost $w_i$ ($1 \le w_i \le 10^9$). The driver starts at city $1$ with $K$ **overdrive vouchers** ($0 \le K \le 10$). When traversing any highway of cost $w_i$, the driver may either: - Pay the full cost $w_i$ (using $0$ vouchers), or - Spend $1$ voucher (if at least one remains) to reduce that highway's cost to $\lfloor w_i/2 \rfloor$ (at most one voucher may be used per highway).

Find the minimum total fuel cost to reach city $N$.

> **Pause & Discover:** Before reading further, ask yourself:
> - Why is "find the shortest path from $1$ to $N$ without vouchers, then apply the $K$ vouchers to the $K$ most expensive edges on that path" wrong?
> - What is the smallest state tuple that makes the remaining journey from city $u$ completely independent of how we reached $u$ ?

### Step 1 — Exact Model & Bottleneck

Along any path $P = (e_1, e_2, \ldots, e_L)$ from $1$ to $N$, we choose a subset $S \subseteq P$ of size $|S| \le K$ of edges to halve. Enumerating all simple paths and all $\binom{L}{K}$ voucher placements is exponential.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Finding the unweighted/standard shortest path first, then halving its $K$ heaviest edges.**
Vouchers make paths with a few **very heavy** edges much more attractive than paths composed of many small edges!

**Concrete Small Counterexample:**
Let $N = 4, K = 1$. - Path A ($1 \to 2 \to 3 \to 4$): three edges of cost $4, 4, 4$ (base total $12$). Halving the heaviest edge gives $2 + 4 + 4 = 10$ . - Path B ($1 \to 4$ directly): one single edge of cost $16$ (base total $16 > 12$). Halving that single edge gives $\lfloor 16/2 \rfloor = 8 < 10$! Standard Dijkstra without vouchers discards Path B in favor of Path A, missing the true optimum of $8$.

**Mistake 2: Storing only `best_dist[u]` and `vouchers_left[u]` per vertex $u$.**
Suppose one route reaches city $u$ with `(cost = 10, vouchers_used = 1)` and another route reaches $u$ with `(cost = 14, vouchers_used = 0)`. Neither dominates the other! If the final edge $u \to N$ costs $100$, the second route halves it to $14 + 50 = 64$, beating the first route's $10 + 100 = 110$.

### Step 3 — The Turn: Discovering the Structure

For each fixed number of used vouchers $k \in \{0, 1, \ldots, K\}$, two partial paths ending at the same city $u$ with the **exact same count** $k$ of used vouchers are directly comparable: only the cheaper one ever needs to be kept!

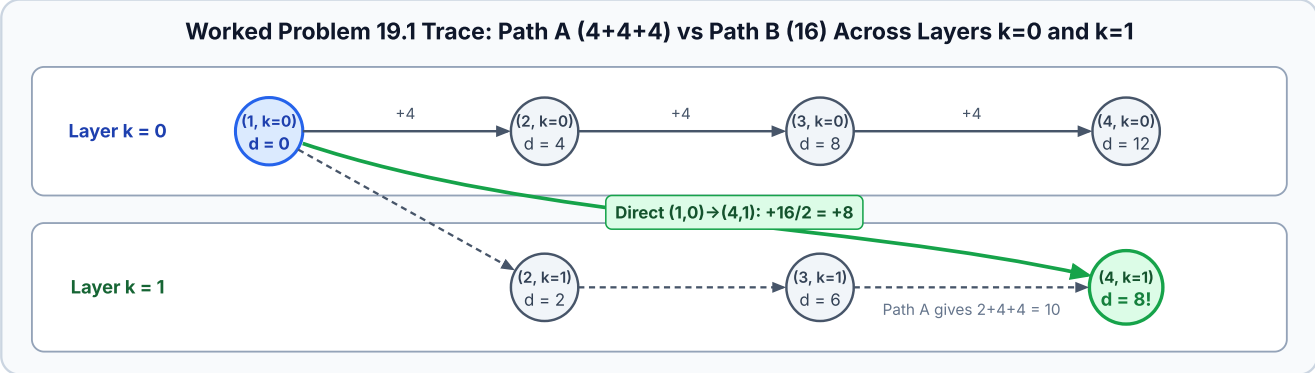
Thus we construct a layered graph with $N \times (K+1)$ states $(u, k)$:

**Layered Dijkstra Transitions (for each road $(u, v, w)$ and layer $k \in [0, K]$):**

1. Keep voucher (stay in layer $k$): $(u, k) \xrightarrow{+w} (v, k)$
2. Spend voucher (if $k < K$, advance to layer $k+1$): $(u, k) \xrightarrow{+\lfloor w/2 \rfloor} (v, k+1)$

$$\text{Answer} = \min_{0 \le k \le K} \text{dist}[N][k]$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <tuple>
#include <algorithm>

using namespace std;

const long long INF = 1e18;

struct Edge {
    int to;
    long long weight;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M, K;
    if (!(cin >> N >> M >> K)) return 0;

    vector<vector<Edge>> adj(N + 1);
    for (int i = 0; i < M; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    vector<vector<long long>> dist(N + 1, vector<long long>(K + 1, INF));
    using State = tuple<long long, int, int>; // (cost, u, vouchers_used)
    priority_queue<State, vector<State>, greater<State>> pq;

    dist[1][0] = 0;
```

```
    pq.push({0, 1, 0});

    while (!pq.empty()) {
        auto [d, u, k] = pq.top();
        pq.pop();

        if (d != dist[u][k]) continue;

        for (const auto& e : adj[u]) {
            // Option 1: Pay full price w, stay in layer k
            if (d + e.weight < dist[e.to][k]) {
                dist[e.to][k] = d + e.weight;
                pq.push({dist[e.to][k], e.to, k});
            }
            // Option 2: Spend 1 voucher, move to layer k + 1
            if (k < K) {
                long long discounted = d + (e.weight / 2);
                if (discounted < dist[e.to][k + 1]) {
                    dist[e.to][k + 1] = discounted;
                    pq.push({discounted, e.to, k + 1});
                }
            }
        }
    }

    long long ans = INF;
    for (int k = 0; k <= K; ++k) {
        ans = min(ans, dist[N][k]);
    }
    cout << (ans == INF ? -1 : ans) << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Proof of Markov Sufficiency:** Suppose an optimal walk from $1$ to $N$ uses $k^* \leq K$ vouchers in total, passing through city $u$ after having used $k \leq k^*$ vouchers on the prefix $1 \rightsquigarrow u$. The suffix $u \rightsquigarrow N$ uses $k^* - k$ vouchers and does not depend on *which* specific edges in $1 \rightsquigarrow u$ consumed the first $k$ vouchers (since edge discounts can be applied at most once per edge traversal, and with positive edge weights no optimal walk repeats an edge). Therefore, replacing the prefix $1 \rightsquigarrow u$ by the minimum-cost prefix reaching state $(u, k)$ cannot increase the total cost.

---

## Worked Problem 19.2: Subspace Frequency Syndicate (Virtual Hub Reduction)

### Problem & Constraints

An interstellar network consists of $N$ beacons ($2 \leq N \leq 10^5$), $M$ direct point-to-point wormholes ($0 \leq M \leq 10^5$), and $G$ **subspace syndicates** ($1 \leq G \leq 10^5$). - Each direct wormhole $i$ connects beacon $u_i$ and beacon $v_i$ bidirectionally with transit time $w_i \geq 0$. - Each syndicate $g$ has a flat boarding fee $C_g \geq 0$ and a member list $S_g \subseteq \{1, ..., N\}$ of beacons. Any beacon $u \in S_g$ can transmit directly to **any other** beacon $v \in S_g$ in syndicate $g$ in time $C_g$. The sum of member list sizes across all syndicates satisfies $\sum_{g=1}^{G} |S_g| \leq 2 \cdot 10^5$.

Find the minimum transit time from beacon $1$ to every beacon $u \in \{1, ..., N\}$.

> **Pause & Discover:** Before reading further, ask yourself:
> - If a single syndicate $g$ contains all $N = 10^5$ beacons, how many pairwise edges does that clique contain?
> - How can we represent "pay $C_g$ to enter syndicate $g$'s broadcast channel from $u$, and exit at any $v \in S_g$ for $0$" using only $2|S_g|$ directed edges?

### Step 1 — Exact Model & Bottleneck

In the naive graph, each syndicate $g$ induces a complete directed clique on $S_g$ where every pair $(u, v) \in S_g \times S_g$ $(u \neq v)$ has an edge of weight $C_g$. When $|S_g| = 10^5$, a single syndicate creates $\binom{10^5}{2} \approx 5 \cdot 10^9$ edges, causing immediate Memory Limit Exceeded and Time Limit Exceeded.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Connecting the members of $S_g = \{u_1, u_2, ..., u_k\}$ in a cycle or chain of weight $C_g$.**
If you connect $u_1 - u_2 - ... - u_k$ with weight $C_g$, jumping from $u_1$ to $u_k$ costs $(k-1)C_g$ instead of $C_g$. If you give the chain weight $0$, then jumping costs $0$ instead of $C_g$!

**Mistake 2: Using an undirected edge of weight $C_g/2$ between $u$ and the hub when $C_g$ is an odd integer (causing rounding errors with integer arithmetic), or forgetting that entering and immediately exiting the hub at another node must sum to $C_g$.**

**Concrete Small Counterexample:**
If $C_g = 5$ and we use integer division $5/2 = 2$ on undirected hub edges $u \leftrightarrow \text{Hub}_g$, then $u \to \text{Hub}_g \to v$ costs $2 + 2 = 4 \neq 5$. Using **directed** edges $u \to \text{Hub}_g$ of weight $C_g$ and $\text{Hub}_g \to v$ of weight $0$ keeps all weights exact integers and costs $C_g + 0 = C_g$!

### Step 3 — The Turn: Discovering the Structure

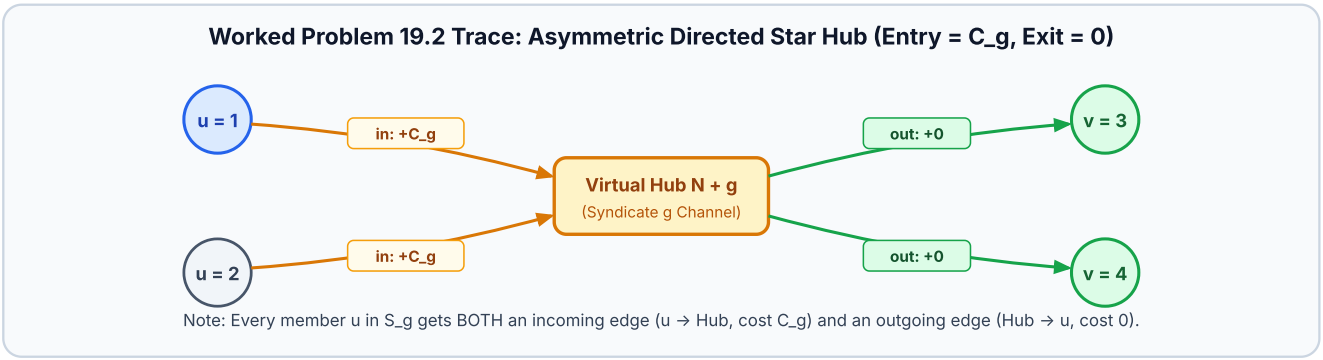
For each syndicate $g \in \{1, ..., G\}$, we allocate a **virtual hub vertex** with index $N + g$. Instead of connecting $u \in S_g$ directly to $v \in S_g$, we factor the jump into two half-steps: 1. **Board syndicate $g$:** a directed edge $u \to (N + g)$ of weight $C_g$. 2. **Alight from syndicate $g$:** a directed edge $(N + g) \to v$ of weight $0$.

**Clique-to-Star Virtual Hub Equivalence:**

$$\forall u, v \in S_g: \quad \text{dist}(u \to (N+g) \to v) = C_g + 0 = C_g$$

$$|V_{\text{augmented}}| = N + G \leq 2 \cdot 10^5, \qquad |E_{\text{augmented}}| = 2M + 2\sum_{g=1}^{G} |S_g| \leq 6 \cdot 10^5$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const long long INF = 1e18;

struct Edge {
    int to;
    long long weight;
};

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int N, M, G;
    if (!(cin >> N >> M >> G)) return 0;

    vector<vector<Edge>> adj(N + G + 1);
    for (int i = 0; i < M; ++i) {
        int u, v;
        long long w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
        adj[v].push_back({u, w});
    }

    for (int g = 1; g <= G; ++g) {
        int k;
        long long cost_g;
        cin >> k >> cost_g;
        int hub = N + g;
        for (int j = 0; j < k; ++j) {
            int u;
            cin >> u;
            adj[u].push_back({hub, cost_g});
            adj[hub].push_back({u, 0});
        }
    }

    vector<long long> dist(N + G + 1, INF);
    using P = pair<long long, int>;
    priority_queue<P, vector<P>, greater<P>> pq;

    dist[1] = 0;
    pq.push({0, 1});

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) continue;

        for (const auto& e : adj[u]) {
            if (d + e.weight < dist[e.to]) {
                dist[e.to] = d + e.weight;
                pq.push({dist[e.to], e.to});
            }
        }
    }

    for (int i = 1; i <= N; ++i) {
        cout << (dist[i] == INF ? -1 : dist[i]) << (i == N ? "" : " ");
    }
    cout << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Bijection Proof Between Clique Paths and Augmented Graph Paths:** - Any direct syndicate jump $u \to v$ ($u, v \in S_g$) of cost $C_g$ corresponds to the two-edge path $u \to (N+g) \to v$ of cost $C_g + 0 = C_g$. - Conversely, since virtual hubs only have incoming edges from $S_g$ (cost $C_g$) and outgoing edges to $S_g$ (cost $0$), any path in the augmented graph that visits $N+g$ must enter from some $u \in S_g$ and immediately exit to some $v \in S_g$ at combined cost $C_g$. Thus shortest distances between all real vertices $1..N$ are preserved identically.

---

# Section 20 — 0-1 BFS and Shortest Path Variants

## The Discovery & Modeling Approach

What happens when a shortest-path problem has **both free actions (cost $0$) and paid actions (cost $1$)**? - Standard unweighted BFS fails because an edge of weight $0$ does not advance the distance layer; pushing it to the back of a FIFO queue processes it after states of distance $d+1$. - Standard Dijkstra works in $O((V+E)\log V)$ time, but its binary heap overhead is completely unnecessary when edge weights belong strictly to $\{0,1\}$.

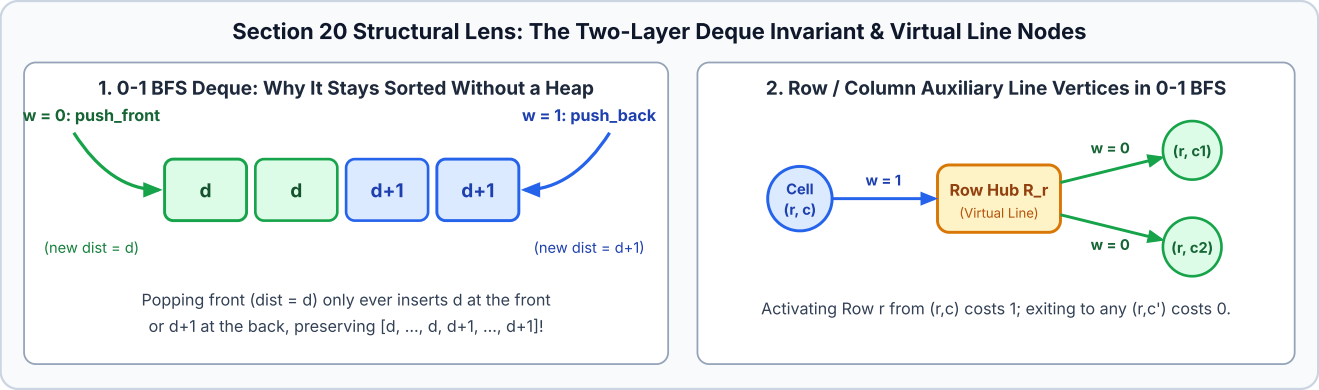
**Dial's / 0-1 BFS** bridges the two in pure linear time $O(V+E)$ using a **double-ended queue (`std::deque`)**: - Relaxing an edge of weight $0$ keeps us in the *same* distance layer $d$, so we push the neighbor to the **front** of the deque (`push_front`). - Relaxing an edge of weight $1$ moves us to the *next* distance layer $d+1$, so we push the neighbor to the **back** of the deque (`push_back`).

### Recognizing Hidden 0-1 BFS Problems

1. **"Minimum Overrides / Rotations / Obstacles to Remove":** If moving along existing conveyor belts, wind directions, or matching grid tiles is **free (0)**, while stepping against the belt, rotating a pipe, or smashing a wall costs $1$ **token**, the graph is a 0-1 weighted graph.
2. **Free Connected Regions Separated by Unit Barriers:** Walking inside a continent costs $0$, crossing into water or another region costs $1$.
3. **Row / Column Line-Attack or Teleportation Mechanics:** Suppose paying $1$ coin activates an entire row or column (or clears $K$ consecutive cells along a line). Instead of adding $O(R+C)$ weight-1 edges from every cell $(r,c)$ to every cell in its row/column, create **virtual row vertices $R_r$ and column vertices $C_c$**:
   - Paying $1$ to enter the row line: $(r,c) \xrightarrow{+1} R_r$,
   - Sliding along the row for free: $R_r \xrightarrow{+0} (r,c')$. Because the entry and exit weights are in $\{0,1\}$, this virtual hub graph is solved in $O(RC)$ time by 0-1 BFS!

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in the Problem Statement | Frontier Structure & Update Rule | Complexity & Key Invariant |
|---|---|---|---|
| **Standard 0-1 BFS** | Every legal transition costs either $0$ (free) or $1$ (unit penalty). | `std::deque`: `w == 0` $\implies$ `push_front`; `w == 1` $\implies$ `push_back`. | $O(V+E)$. Deque elements are always sorted with values in $\{d, d+1\}$. |
| **Direction-State 0-1 Grid BFS** | "Laser beam / ice skater moves straight for cost $0$, or turns $90^\circ$ at a mirror for cost $1$." | State is `(r, c, dir)` where `dir` $\in \{0,1,2,3\}$. Continue straight: cost $0$; turn: cost $1$. | $O(4RC)$. Lifting direction into the vertex makes turning a local $0/1$ edge. |
| **Virtual Line-Node 0-1 BFS** | "Pay $1$ to teleport to any cell in the same row or column (or adjacent rows)." | Add row hubs $R_1..R_H$ and col hubs $C_1..C_W$. Cell $\to$ Hub costs $1$; Hub $\to$ Cell costs $0$. | $O(RC)$ total vertices and edges. Combines Section 19 virtual hubs with Section 20 deque! |
| **Dial's Bucket Queue ($w \in \{0..K\}$)** | Edge weights are small integers in $\{0,1,\ldots,K\}$ ($K \le 10$). | Circular array of $K+1$ buckets `B[0..K]`; step through buckets monotonically. | $O(V\cdot K+E)$. Generalizes 0-1 BFS from $2$ levels to $K+1$ levels. |




---

## Worked Problem 20.1: Maglev Switchyard Override

### Problem & Constraints

An automated rail yard is an $R \times C$ grid ($1 \le R, C \le 1000$). Each cell $(r, c)$ contains a directional switch pointing in one of the four cardinal directions `N`, `S`, `E`, `W`. - Moving from $(r, c)$ into the neighbor $(nr, nc)$ that matches the current switch direction of $(r, c)$ is **free (cost $0$)**. - Moving from $(r, c)$ into any of the other three cardinal neighbors requires sending an **override pulse (cost $1$)** to re-orient the switch.

Find the minimum number of override pulses required to route a locomotive from $(0, 0)$ to $(R-1, C-1)$.

> **Pause & Discover:** Before reading further, ask yourself:
> - Why is it a bug to mark a cell `visited = true` the first time it is pushed into the deque (like in unweighted BFS)?
> - How many times can a single cell $(r, c)$ be pushed into the deque if it is first reached via a $w = 1$ edge and later via a $w = 0$ edge?

### Step 1 — Exact Model & Bottleneck

Every grid cell $(r, c)$ has $4$ outgoing directed edges: one of weight $0$ (in the direction of `grid[r][c]`) and three of weight $1$ (in the other three directions). We want the shortest path from $(0, 0)$ to $(R-1, C-1)$ on a graph with $V = RC \le 10^6$ vertices and $E \le 4 \cdot 10^6$ edges. A heap-based Dijkstra takes $O(RC \log(RC))$ time, whereas 0-1 BFS runs in $O(RC)$ time.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake: Marking a neighbor as permanently settled upon its first insertion into the deque.**
In unweighted BFS, all edges have weight $1$, so the first time $v$ is seen, its distance is $d+1$ and cannot be improved. In 0-1 BFS, while popping vertices at distance $d$, one vertex $u_1$ (at distance $d$) might relax $v$ via a $w = 1$ edge, setting $\text{dist}[v] = d+1$ and pushing $v$ to the **back** of the deque. Moments later, another vertex $u_2$ (also at distance $d$) might reach $v$ via a $w = 0$ edge, improving $\text{dist}[v]$ to $d$ and pushing $v$ to the **front** of the deque!

**Concrete Small Counterexample:**

```
S E
E .
```

Start at $(0, 0)$ (`S`), target at $(0, 1)$ (`E`). - From $(0, 0)$, the arrow points `S` (down to $(1, 0)$). So edge $(0, 0) \to (0, 1)$ (right) has weight $1$, while $(0, 0) \to (1, 0)$ (down) has weight $0$. - If we inspect East before South when expanding $(0, 0)$: 1. East relaxes $(0, 1)$ to $\text{dist} = 1$ (`push_back`). 2. South relaxes $(1, 0)$ to $\text{dist} = 0$ (`push_front`). - Suppose $(1, 0)$ also had a $0$-weight edge reaching $(0, 1)$ (for instance in a general 0-1 graph where $(0, 0) \xrightarrow{1} v$ and $(0, 0) \xrightarrow{0} u \xrightarrow{0} v$). If $v$ was locked upon the first push at $\text{dist} = 1$, the $0$-cost path $0 \to u \to v$ would be blocked!

### Step 3 — The Turn: Discovering the Structure

In 0-1 BFS, we relax distances just like in Dijkstra (`if (dist[u] + w < dist[v])`), pushing to the front when $w = 0$ and to the back when $w = 1$:

**0-1 BFS Relaxation & Two-Layer Deque Contract:**

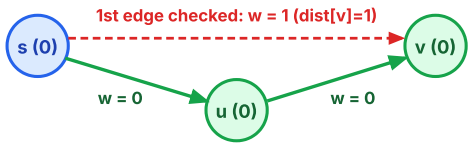
$$\text{If } \text{dist}[u] + w < \text{dist}[v]\text{:} \quad \text{dist}[v] \leftarrow \text{dist}[u] + w, \quad \begin{cases} \text{dq.push_front}(v) & \text{if } w = 0, \\ \text{dq.push_back}(v) & \text{if } w = 1. \end{cases}$$

Every vertex $v$ is pushed at most twice (once at $d+1$ via $w = 1$, and at most once at $d$ via $w = 0$).

### Step 4 — Progressive Construction & Visual Trace

**Worked Problem 20.1 Trace: Why a Vertex Can Be Relaxed at d+1 (Back) and Improved to d (Front)**

**0-1 Triangle: s → v (w=1) vs s → u → v (w=0+0)**




**Deque Evolution & Stale Entry Guard**

1. Pop s (d=0): inspect s→v (w=1) → push_back(v, d=1).
2. Still at s: inspect s→u (w=0) → push_front(u, d=0).

**Deque is now: [ (u, d=0), (v, d=1) ]**

3. Pop u (d=0): inspect u→v (w=0) → dist[v] improves 1 → 0!

**push_front(v, d=0). Pop v (d=0) first; skip stale v at back.**

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <deque>

using namespace std;

const int INF = 1e9;
const int dr[4] = {-1, 1, 0, 0};
const int dc[4] = {0, 0, 1, -1};
const char dir_char[4] = {'N', 'S', 'E', 'W'};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int R, C;
    if (!(cin >> R >> C)) return 0;
    vector<string> grid(R);
    for (int r = 0; r < R; ++r) cin >> grid[r];

    vector<vector<int>> dist(R, vector<int>(C, INF));
    deque<pair<int, int>> dq;

    dist[0][0] = 0;
    dq.push_front({0, 0});

    while (!dq.empty()) {
        auto [r, c] = dq.front();
        dq.pop_front();

        for (int i = 0; i < 4; ++i) {
            int nr = r + dr[i], nc = c + dc[i];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
```

```
            int w = (grid[r][c] == dir_char[i]) ? 0 : 1;
            if (dist[r][c] + w < dist[nr][nc]) {
                dist[nr][nc] = dist[r][c] + w;
                if (w == 0) dq.push_front({nr, nc});
                else dq.push_back({nr, nc});
            }
        }
    }

    cout << dist[R - 1][C - 1] << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Induction Proof of the Two-Layer Deque Invariant:** We prove by induction on pop steps that the deque always has the form $[\underbrace{d, ..., d}_{k \geq 0}, \underbrace{d+1, ..., d+1}_{m \geq 0}]$. - *Base Case:* Initially the deque holds $[(s, 0)]$, satisfying the invariant with $d = 0$. - *Inductive Step:* When we pop the front element $u$ with distance $d$: - Any neighbor relaxed via $w = 0$ receives distance $d$ and is placed at the **front**, preceding all remaining $d$ and $d+1$ entries. - Any neighbor relaxed via $w = 1$ receives distance $d+1$ and is placed at the **back**, following all $d$ and $d+1$ entries. - Once the last element of distance $d$ is popped and yields no further $w = 0$ relaxations, the front of the deque shifts to $d' = d+1$, and the invariant holds with $[d', ..., d']$.

---

## Worked Problem 20.2: Laser-Grid Breach (Row/Column Line-Vertex 0-1 BFS)

### Problem & Constraints

A security vault is an $R \times C$ grid ($1 \leq R, C \leq 1000$) of open cells `.` and reinforced laser barriers `#`. Moving four-directionally from an open cell to an adjacent open cell is **free (cost $0$)**.

At any time, the infiltrator may detonate an EMP charge (cost $1$). A single EMP detonation permanently disables **all** barriers `#` in the infiltrator's chosen entire row $r'$ (provided $|r' - r| \leq 1$, i.e., the infiltrator's current row or an immediately adjacent row) **or** in a chosen entire column $c'$ (provided $|c' - c| \leq 1$). Once a row or column is cleared, the infiltrator can immediately step or travel anywhere along that cleared row or column for cost $0$.

Find the minimum number of EMP charges needed to travel from $(0, 0)$ to $(R-1, C-1)$ (both guaranteed to be `.`).

> **Pause & Discover:** Before reading further, ask yourself:
> - If clearing row $r'$ allows free travel across all $C$ cells in row $r'$, why can't we add direct weight-$1$ edges from $(r, c)$ to all $C$ cells in rows $r-1, r, r+1$?
> - How do virtual row nodes $\mathrm{Row}[r']$ and virtual column nodes $\mathrm{Col}[c']$ reduce the degree of every grid cell to a tiny constant?

### Step 1 — Exact Model & Bottleneck

From cell $(r, c)$, firing an EMP on row $r' \in \{r-1, r, r+1\}$ lets the infiltrator land on any cell $(r', nc)$ ($0 \leq nc < C$) at cost $+1$. Explicitly adding these $3C + 3R$ transitions from each of the $RC$ cells creates $3RC(R+C) \approx 6 \cdot 10^9$ edges, which is far too dense.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Only allowing $(r, c)$ to step into adjacent barrier cells $(nr, nc)$ for cost $1$.**
Stepping onto a single barrier $(nr, nc)$ for cost $1$ forgets that the *same* EMP pulse cleared the entire row $nr$ (or column $nc$)! If the infiltrator walks along row $nr$ through $5$ consecutive barriers `#`, local cell-by-cell smashing charges $5$ EMPs instead of $1$!

**Concrete Small Counterexample:**

```
. # # # .
```

In this $1 \times 5$ corridor from $(0,0)$ to $(0,4)$, a single EMP fired on Row $0$ clears all three `#` cells at once for a total cost of $1$, whereas cell-by-cell barrier breaking charges $3$.

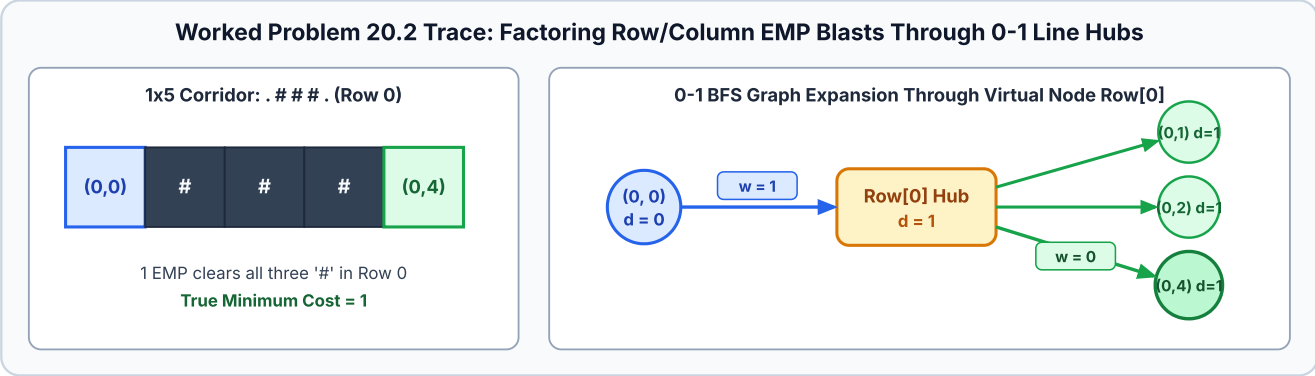
### Step 3 — The Turn: Discovering the Structure

Combine the **Virtual Hub** idea from Section 19 with **0-1 BFS** from Section 20! Introduce $R$ virtual row nodes $\text{Row}[0..R-1]$ and $C$ virtual column nodes $\text{Col}[0..C-1]$: 1. **Standard Open Moves (Cost $0$):** From cell $(r,c)$ to any 4-adjacent open cell $(nr, nc)$ $==$ '.', add a $0$-weight edge. 2. **Detonate Row EMP (Cost $1$):** From cell $(r,c)$ to $\text{Row}[r']$ for each $r' \in \{r-1, r, r+1\} \cap [0, R-1]$, add a $1$-**weight** edge (`push_back`). 3. **Travel Along Cleared Row (Cost $0$):** From $\text{Row}[r']$ to **every** cell $(r', c')$ $(0 \le c' < C)$, add a $0$-**weight** edge (`push_front`). 4. **Symmetric Column EMPs:** $(r,c) \xrightarrow{+1} \text{Col}[c']$ for $|c' - c| \le 1$, and $\text{Col}[c'] \xrightarrow{+0} (r', c')$ for all $0 \le r' < R$.

**Line-Vertex 0-1 Graph Sparsity Bound:**

$$\text{Total Vertices } V = RC + R + C \le 1.002 \times 10^6$$
$$\text{Out-Degree of Cell } (r,c) \le 4 \text{ (grid)} + 3 \text{ (row hubs)} + 3 \text{ (col hubs)} = 10$$
$$\text{Out-Degree of Row}[r'] = C, \quad \text{Out-Degree of Col}[c'] = R \implies \sum \deg = 12RC = O(RC)!$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
#include <vector>
#include <string>
#include <deque>

using namespace std;

const int INF = 1e9;
const int dr[4] = {-1, 1, 0, 0};
const int dc[4] = {0, 0, -1, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int R, C;
    if (!(cin >> R >> C)) return 0;
    vector<string> grid(R);
    for (int r = 0; r < R; ++r) cin >> grid[r];

    // Vertex encoding:
    // 0 .. R*C - 1           : Grid cell (r, c) -> r * C + c
    // R*C .. R*C + R - 1     : Virtual Row node r
    // R*C + R .. R*C + R+C-1 : Virtual Col node c
    int total_V = R * C + R + C;
    int row_offset = R * C;
    int col_offset = R * C + R;
```

```
    vector<int> dist(total_V, INF);
    vector<bool> popped(total_V, false);
    deque<int> dq;

    dist[0] = 0;
    dq.push_front(0);

    while (!dq.empty()) {
        int u = dq.front();
        dq.pop_front();

        if (popped[u]) continue;
        popped[u] = true;

        if (u < row_offset) {
            int r = u / C, c = u % C;
            // 1. Cost-0 moves to 4-adjacent open cells
            for (int i = 0; i < 4; ++i) {
                int nr = r + dr[i], nc = c + dc[i];
                if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == '.') {
                    int v = nr * C + nc;
                    if (dist[u] < dist[v]) {
                        dist[v] = dist[u];
                        dq.push_front(v);
                    }
                }
            }
            // 2. Cost-1 moves to activate Row r' in {r-1, r, r+1}
            for (int nr = max(0, r - 1); nr <= min(R - 1, r + 1); ++nr) {
                int v = row_offset + nr;
                if (dist[u] + 1 < dist[v]) {
                    dist[v] = dist[u] + 1;
                    dq.push_back(v);
                }
            }
            // 3. Cost-1 moves to activate Col c' in {c-1, c, c+1}
            for (int nc = max(0, c - 1); nc <= min(C - 1, c + 1); ++nc) {
                int v = col_offset + nc;
                if (dist[u] + 1 < dist[v]) {
                    dist[v] = dist[u] + 1;
                    dq.push_back(v);
                }
            }
        } else if (u < col_offset) {
            // Virtual Row node: Cost-0 exit to all cells in row r
            int r = u - row_offset;
            for (int c = 0; c < C; ++c) {
                int v = r * C + c;
                if (dist[u] < dist[v]) {
                    dist[v] = dist[u];
                    dq.push_front(v);
                }
            }
        } else {
            // Virtual Col node: Cost-0 exit to all cells in col c
            int c = u - col_offset;
            for (int r = 0; r < R; ++r) {
                int v = r * C + c;
                if (dist[u] < dist[v]) {
                    dist[v] = dist[u];
                    dq.push_front(v);
                }
            }
        }
    }
```

```
    cout << dist[R * C - 1] << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

**Complexity & Correctness Proof:** 1. *Why `if (popped[u]) continue;` is critical for Virtual Line Nodes:* Each virtual node $\mathrm{Row}[r]$ has out-degree $C$, and each $\mathrm{Col}[c]$ has out-degree $R$. Thanks to the `popped[u]` guard, each virtual line node is expanded **at most once** when its distance is finalized. Thus the inner loops over $c \in [0, C)$ and $r \in [0, R)$ execute at most once per row and once per column, contributing $\sum_{r=0}^{R-1} C + \sum_{c=0}^{C-1} R = 2RC$ edge inspections in total! 2. *Small-Case Oracle:* On $4 \times 4$ grids, test all subsets of rows and columns (only $2^{R+C} = 256$ masks of EMP blasts!), check whether $(0, 0)$ and $(R-1, C-1)$ are connected in the cleared grid, and verify that the minimum mask popcount matches the 0-1 BFS distance.

# Section 22 — One-Dimensional DP

## The Discovery & Modeling Approach

One-dimensional dynamic programming is the purest laboratory for the core discovery question of all dynamic programming: **what is the minimal summary of prefix history that makes all valid future continuations indistinguishable?**

When solving a sequential decision problem over indices $1, 2, ..., n$, an exact brute-force search branches on every legal choice at index $i$ and carries forward the complete sequence of past decisions $(d_1, d_2, ..., d_i)$. That exact history tree has exponential size. To compress it into a polynomial DAG, we define an equivalence relation $\sim_i$ on histories of the prefix $1..i$: two histories $H_1$ and $H_2$ are equivalent ($H_1 \sim_i H_2$) if and only if: 1. Every suffix continuation $S$ over $i+1..n$ is legal after $H_1$ if and only if it is legal after $H_2$. 2. The additional cost or value contributed by $S$ (including any boundary interaction between prefix $1..i$ and suffix $i+1..n$) is identical under $H_1$ and $H_2$.

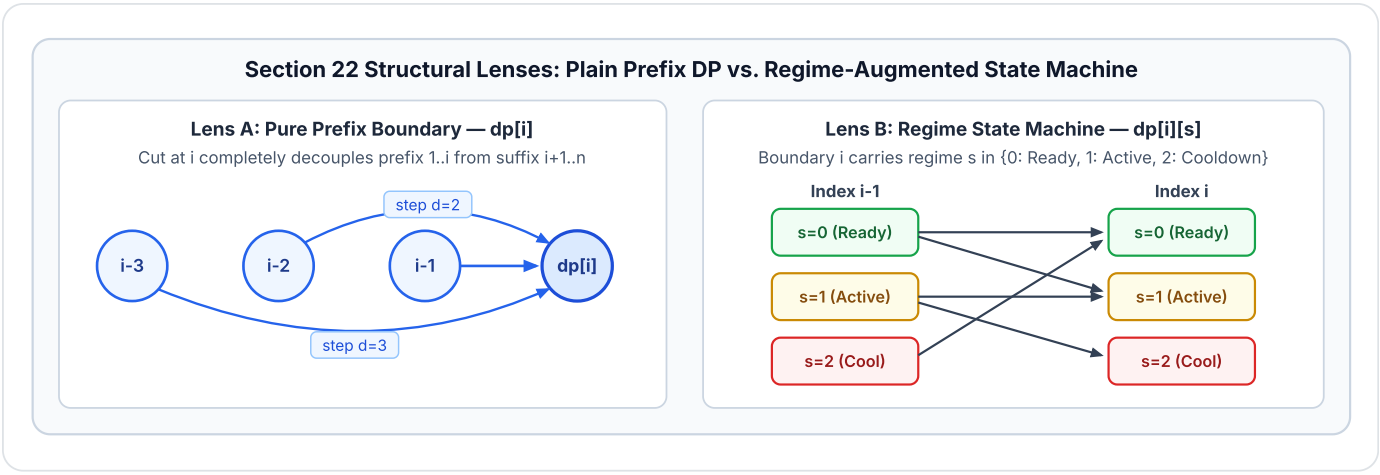
Once $\sim_i$ is identified, each equivalence class becomes a **DP state** at position $i$, and we retain only the single optimal value (or aggregate count) inside each class.

```
Exact Prefix History (d_1, ..., d_i)
   --> Equivalence Relation (same future legality + same boundary interaction)
   --> Minimal State Tuple (i, regime s)
   --> Topological Order (increasing i, or decreasing i)
```

### Decision Compass & Structural Lenses

Not all 1D DPs look like $dp[i] = \min(dp[i-1], dp[i-2])$. Depending on how a decision at index $i$ interacts with earlier decisions, one-dimensional problems fall into four canonical structural lenses:

| Structural Lens | What the Future Needs from Prefix $1..i$ | Canonical State Form | Transition Footprint | Typical Complexity |
|---|---|---|---|---|
| **1. Pure Prefix Boundary** | Only the covered length $i$; no residual effect crosses $i$. | $dp[i]$ | Small fixed set of steps $d \in D$: $dp[i-d]$ | $O(n \cdot \lvert D \rvert)$ time, $O(n)$ or $O(\max D)$ space |
| **2. State-Machine / Regime Augmentation** | Prefix length $i$ plus a small discrete mode $s \in \{0, ..., k-1\}$ (cooldown, parity, streak length, last action). | $dp[i][s]$ | Finite-automaton edges $(i-1, s') \to (i, s)$ | $O(n \cdot k^2)$ time, $O(k)$ rolling space |
| **3. Last-Breakpoint Segmentation** | Prefix $1..i$ is partitioned into contiguous blocks; the last block is $[j+1..i]$. | $dp[i]$ | Scan all valid split points $0 \le j < i$: $dp[j] + \text{cost}(j+1, i)$ | $O(n^2)$ naive (often reducible if cost decomposes) |
| **4. Open-Interval / Contribution-on-the-Fly** | Global cost depends on pairs or spans that are opened at $l$ and closed at $r$; charge the incremental step cost at every boundary $i \in [l, r-1]$. | $dp[i][c]$ ($c$ = active open components) | Open, continue, or close components at $i$; add $c \cdot \Delta_i$ immediately | $O(n \cdot C)$ time |




---

## Worked Problem 22.1: Thermal Reactor Pulse Scheduling

**Problem & Constraints.** A fusion testbed runs for $n$ discrete seconds ($1 \le n \le 2 \cdot 10^5$). During second $i$, firing the plasma injector yields a scientific gain $g_i$ ($-10^9 \le g_i \le 10^9$). The reactor operates under three physical rules: 1. You may keep the injector **Idle** during second $i$, gaining $0$. 2. The first second in any contiguous firing burst operates in **Standard Mode**, yielding $g_i$. 3. If the injector was already firing in second $i-1$ (in Standard Mode), you may push it into **Overdrive Mode** for second $i$, yielding $2g_i$, **or** drop it back to **Idle**. However, **at most two consecutive seconds** may be active in any single burst (i.e., after Overdrive Mode at second $i$, second $i+1$ **must** be Idle), and furthermore, leaving Overdrive Mode triggers a mandatory **two-second thermal cooldown** (if second $i$ is Overdrive, both seconds $i+1$ and $i+2$ must be Idle). Leaving Standard Mode triggers only a normal reset (second $i+1$ is already Idle if you stop there, so second $i+2$ is free to fire again).

Compute the maximum total scientific gain over the $n$ seconds.

> **Pause & Discover:** Suppose two partial schedules over seconds $1..i$ have achieved the same total gain so far. Can they always be extended by the same actions at second $i+1$? Write down the exact list of residual statuses at the end of second $i$ that affect whether second $i+1$ can be Idle, Standard, or Overdrive.

### Step 1 — Exact Model & Bottleneck

In an exact model, a valid schedule is a sequence of actions $a_1, a_2, \ldots, a_n \in \{\text{Idle}, \text{Std}, \text{Over}\}$ satisfying the local grammar rules: - $a_i = \text{Std}$ is legal iff $a_{i-1} = \text{Idle}$ and (if $i \ge 3$) $a_{i-2} \ne \text{Over}$. - $a_i = \text{Over}$ is legal iff $a_{i-1} = \text{Std}$. - $a_i = \text{Idle}$ is always legal.

Enumerating all valid action strings branches up to $2$ ways at most steps, producing $\Theta(c^n)$ trajectories ($c \approx 1.618$). For $n = 2 \cdot 10^5$, brute-force enumeration is impossible; we need an $O(n)$ compression.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Plain prefix state $dp[i]$ = best gain on $1..i$ assuming second $i+1$ is free.** Wait—if you try to jump over bursts using only $dp[i]$, you might forget that a single-second burst ($\text{Std}$ at $i$) requires $i+1$ to be Idle (1 cooldown second), whereas a two-second burst ($\text{Std}$ at $i-1$, $\text{Over}$ at $i$) requires both $i+1$ and $i+2$ to be Idle (2 cooldown seconds). If $dp[i]$ does not distinguish *how* the prefix ended, it either illegally fires at $i+1$ or fails to enforce the 2-second cooldown after Overdrive.

**Mistake 2: Greedy pairing of largest $2g_i + g_{i-1}$.** Consider $n = 4$ with gains $g = [5, 6, 10, 10]$. - If we greedily fire $\text{Std}$ at $i = 1$ ($5$) and $\text{Over}$ at $i = 2$ ($2 \times 6 = 12$) for $17$, then seconds $3$ and $4$ are locked in thermal cooldown ($0 + 0$), giving total $17$. - If instead we stay Idle at $i = 1, 2$, fire $\text{Std}$ at $i = 3$ ($10$) and $\text{Over}$ at $i = 4$ ($2 \times 10 = 20$), we obtain $30$. - Even better: fire $\text{Std}$ at $i = 1$ ($5$), stay Idle at $i = 2$ ($0$), fire $\text{Std}$ at $i = 3$ ($10$), and $\text{Over}$ at $i = 4$ ($20$), giving $5 + 0 + 10 + 20 = 35$! Notice how stopping after $\text{Std}$ at $i = 1$ cost only one Idle second at $i = 2$, preserving the huge pair at $i = 3, 4$.

### Step 3 — The Turn: Discovering the Structure

Ask what the future $i+1..n$ actually needs to know about the prefix $1..i$. Look at the grammar rules: legality at $i+1$ depends only on the current state of the reactor at the end of second $i$. Specifically, there are exactly **four** mutually exclusive reactor regimes after second $i$: - **Regime** $0$ **(** `READY` **)**: Second $i$ was Idle, and second $i-1$ was *not* Overdrive. The reactor is completely cool and ready to fire $\mathrm{Std}$ at $i+1$ (or stay Idle). - **Regime** $1$ **(** `STD` **)**: Second $i$ was fired in **Standard Mode**. At second $i+1$, we may either escalate to $\mathrm{Over}$ or drop to Idle (returning immediately to `READY` because only Overdrive triggers a 2-second lock). - **Regime** $2$ **(** `OVER` **)**: Second $i$ was fired in **Overdrive Mode**. Second $i+1$ is strictly forced to be first-stage cooldown. - **Regime** $3$ **(** `COOL1` **)**: Second $i$ was the first mandatory Idle second after an Overdrive at $i-1$. Second $i+1$ is strictly forced to be the second mandatory Idle second, which then transitions to `READY`.

**Compressed State & Automaton Recurrence:**
Let $dp[i][s]$ be the maximum total gain over seconds $1..i$ such that at the end of second $i$, the reactor is in regime $s \in \{0,1,2,3\}$ (with $-\infty$ for unreachable regimes).

**Base Cases ($i=1$):**

$$dp[1][0]=0, \qquad dp[1][1]=g_1, \qquad dp[1][2]=-\infty, \qquad dp[1][3]=-\infty$$

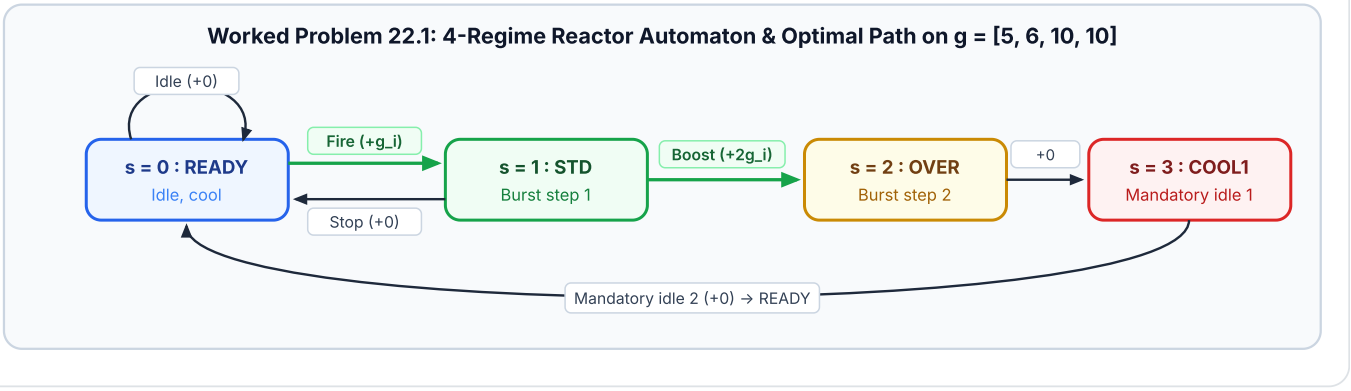
**Transitions for $i=2,\ldots,n$:**

$$dp[i][0] = \max\left(\, dp[i-1][0],\ dp[i-1][1],\ dp[i-1][3] \,\right) + 0$$
$$dp[i][1] = dp[i-1][0] + g_i$$
$$dp[i][2] = dp[i-1][1] + 2g_i$$
$$dp[i][3] = dp[i-1][2] + 0$$

**Final Answer:** $\max_{s\,\in\,\{0,1,2,3\}} dp[n][s]$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace the state machine on our counterexample $g=[5,6,10,10]$: - $i=1$ **(**$g_1=5$**)**: $dp[1]=[0,\ 5,\ -\infty,\ -\infty]$. - $i=2$ **(**$g_2=6$**)**: - $s=0$: $\max(dp[1][0], dp[1][1]) = \max(0,5) = 5$ (Idle after Std at 1). - $s=1$: $dp[1][0]+6=6$ (Std at 2). - $s=2$: $dp[1][1]+12=5+12=17$ (Over at 2). - $s=3$: $-\infty$. - $dp[2]=[5,\ 6,\ 17,\ -\infty]$. - $i=3$ **(**$g_3=10$**)**: - $s=0$: $\max(5,6,-\infty)=6$. - $s=1$: $dp[2][0]+10=5+10=15$ (Std at 3 after Ready at 2). - $s=2$: $dp[2][1]+20=6+20=26$ (Over at 3 after Std at 2). - $s=3$: $dp[2][2]+0=17$ (Cool1 at 3 after Over at 2). - $dp[3]=[6,\ 15,\ 26,\ 17]$. - $i=4$ **(**$g_4=10$**)**: - $s=0$: $\max(6,15,17)=17$. - $s=1$: $dp[3][0]+10=6+10=16$. - $s=2$: $dp[3][1]+20=15+20=35$ (Over at 4 after Std at 3). - $s=3$: $dp[3][2]+0=26$. - $dp[4]=[17,\ 16,\ 35,\ 26] \Longrightarrow \max=35$.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** 1. *Soundness (Every DP path is a valid schedule):* The directed edges of the 4-state automaton enforce that $s=1$ (`STD`) is entered only from $s=0$ (`READY`), $s=2$ (`OVER`) is entered only from $s=1$ (`STD`), and $s=2$ can only exit through $s=3 \to s=0$, which forces two consecutive $+0$ Idle steps ($i+1$ into $s=3$ and $i+2$ into $s=0$) before $s=1$ can be entered again at $i+3$. 2. *Completeness (Every valid schedule is represented):* Any legal action sequence $a_1..a_n$ maps deterministically at each step $i$ to one of the four regimes $s_i \in \{0,1,2,3\}$. By induction on $i$, $dp[i][s_i]$ is at least the gain of $a_1..a_i$ because the recurrence maximizes over all legal predecessor regimes $s_{i-1}$.

**Complexity:** $4$ states and $6$ transitions per step $\implies O(n)$ time and $O(1)$ auxiliary memory using rolling arrays `cur[4]` and `nxt[4]`. Notice that negative $g_i$ values are automatically handled: if $g_i < 0$, the automaton can still choose `STD` at $i$ (taking a small temporary loss) if $2g_{i+1}$ is huge enough to compensate!

**Small-Case Oracle:** For $n \le 9$, enumerate all $3^n$ strings in $\{0,1,2\}^n$, filter out strings containing `11`, `02`, `22`, `21`, `201`, or starting with `2`, and verify that the maximum sum matches $\max_s dp[n][s]$, specifically testing negative $g_i$ followed by large positive $g_{i+1}$.

---

## Worked Problem 22.2: Latency-Charged Batching (Contribution on the Fly)

**Problem & Constraints.** You have $n$ computational jobs ($1 \le n \le 3000$) ordered in a fixed pipeline $1, 2, \ldots, n$. Each job $i$ has processing duration $t_i > 0$ and priority weight $w_i > 0$ ($1 \le t_i, w_i \le 10^5$). You must partition the sequence $1..n$ into contiguous batches. Starting a new batch incurs a fixed system setup delay $S$ ($1 \le S \le 10^5$) that is added to the completion time of the current batch **and every subsequent batch**. Specifically, if the jobs are partitioned into $k$ batches $[1..r_1], [r_1+1..r_2], \ldots, [r_{k-1}+1..n]$, then the $b$-th batch finishes at time

$$C_b = b \cdot S + \sum_{m=1}^{r_b} t_m$$

and every job $j$ in the $b$-th batch pays $w_j \cdot C_b$. Find the minimum total weighted completion cost over all jobs.

> **Pause & Discover:** Why does a naive prefix DP $dp[i]$ struggle to know the completion time $C_b$ of batch $b$? Notice that $C_b$ depends on the number of batches $b$ formed so far! Can we avoid tracking $b$ in the state by charging the setup delay $S$ to all remaining jobs $i..n$ the instant a new batch is opened?

### Step 1 — Exact Model & Bottleneck

In an exact model, a partition is chosen by selecting batch boundaries $0 = r_0 < r_1 < \ldots < r_k = n$. If we try to define a naive prefix DP right-to-left or left-to-right where we need the exact completion time $C_b$, then $C_b = b \cdot S + \mathrm{prefT}[r_b]$ depends on both the boundary $r_b$ and the batch count $b \in \{1, \ldots, n\}$. Tracking $(i, b)$ and scanning predecessors $j < i$ takes $O(n^3)$ time—far too slow for $n = 3000$.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake: Charging setup cost $S$ only to the jobs inside the current batch $[j+1..i]$.** Suppose we write $dp[i] = \min_{j<i}\{dp[j] + (\mathrm{prefW}[i] - \mathrm{prefW}[j]) \cdot (\mathrm{prefT}[i] + S)\}$. This forgets that the setup delay $S$ incurred by starting a batch at $j+1$ shifts the timeline of **all future batches** after $i$ by $+S$ as well! For example, let $n = 2$, $S = 10$, $t = [1, 1]$, $w = [1, 100]$. - Single batch $[1..2]$: $C_1 = 10 + 2 = 12$, cost $= (1 + 100) \times 12 = 1212$. - Two batches $[1..1], [2..2]$: $C_1 = 10 + 1 = 11$, $C_2 = 20 + 2 = 22$, cost $= 1 \times 11 + 100 \times 22 = 2211$. The false formula would omit the first batch's $+10$ shift on job $2$, underestimating the two-batch cost by $100 \times 10 = 1000$.

### Step 3 — The Turn: Discovering the Structure

Decompose the total cost by **linearity of summation** (Contribution on the Fly / Future Tax Amortization). When we close a previous batch at index $j$ and open a new batch starting at $j+1$ ending at $i$: 1. Every job $m \in [j+1..i]$ waits until $\mathrm{prefT}[i]$ for the raw processing time of jobs $1..i$, so this batch pays $(\mathrm{prefW}[i] - \mathrm{prefW}[j]) \cdot \mathrm{prefT}[i]$. 2. The setup delay $S$ of this new batch delays **every job from $j+1$ all the way to** $n$ by exactly $+S$! Therefore, opening a batch at $j+1$ immediately contributes $S \cdot (\mathrm{prefW}[n] - \mathrm{prefW}[j])$ to the global objective, regardless of how suffix $i+1..n$ is later partitioned!

Once we charge $S \cdot (\mathrm{prefW}[n] - \mathrm{prefW}[j])$ at the moment the batch opens at $j+1$, the batch count $b$ completely vanishes from the DP state!

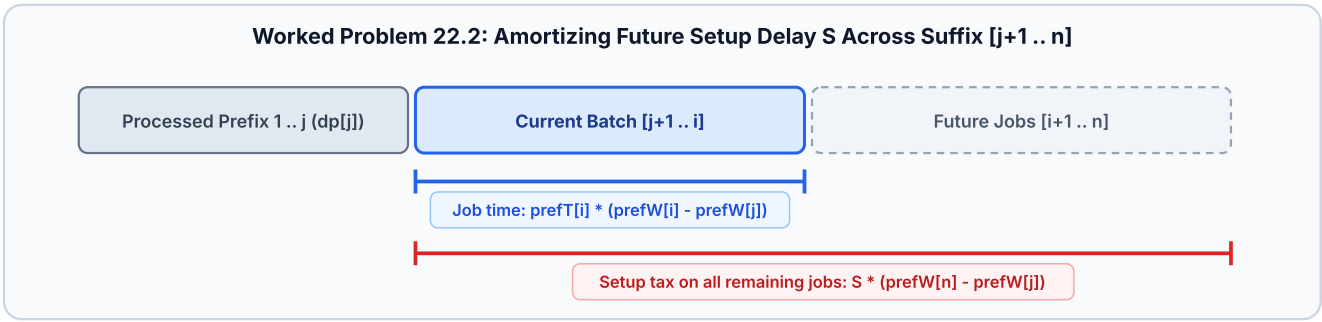
**Compressed 1D Segmentation State & Transition:**
Let $\text{prefT}[i] = \sum_{m=1}^{i} t_m$ and $\text{prefW}[i] = \sum_{m=1}^{i} w_m$.
Let $dp[i]$ be the minimum total cost of partitioning prefix $1..i$, *including* the deferred setup-delay taxes on suffix $i+1..n$ caused by all batches opened within $1..i$.

**Base Case:** $dp[0] = 0$.
**Transition for** $i = 1, ..., n$**:**

$$dp[i] = \min_{0 \le j < i} \left\{ dp[j] + \text{prefT}[i] \cdot (\text{prefW}[i] - \text{prefW}[j]) + S \cdot (\text{prefW}[n] - \text{prefW}[j]) \right\}$$

### Step 4 — Progressive Construction & Visual Trace

Why does charging $S \cdot (\text{prefW}[n] - \text{prefW}[j])$ work? Suppose a partition opens batches at $j_1 + 1, j_2 + 1, ..., j_k + 1$ (with $j_1 = 0$). A job $m$ belongs to some batch $b$, so $j_1, j_2, ..., j_b$ are all strictly less than $m$, while $j_{b+1}, ..., j_k \ge m$. Thus job $m$ is included in $(\text{prefW}[n] - \text{prefW}[j_r])$ for $r = 1, 2, ..., b$—meaning job $m$ pays $w_m \cdot S$ exactly $b$ times, giving $w_m \cdot (bS)$!




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** For any sequence of batch endpoints $0 = r_0 < r_1 < ... < r_k = n$, summing the transition costs telescopically yields:

$$\sum_{b=1}^{k} \left( \text{prefT}[r_b](\text{prefW}[r_b] - \text{prefW}[r_{b-1}]) + S(\text{prefW}[n] - \text{prefW}[r_{b-1}]) \right) = \sum_{b=1}^{k} (\text{prefW}[r_b] - \text{prefW}[r_{b-1}]) \left( \text{prefT}[r_b] + b \cdot S \right),$$

which is the exact problem objective. Because the transformed transition cost depends only on $j$ and $i$ (and not on $b$), optimal substructure holds over the 1D state $dp[i]$ in $O(n^2)$ time and $O(n)$ space. (In Section 49, we will see that this exact recurrence can be accelerated to $O(n)$ via the Convex Hull Trick!)

**Small-Case Oracle:** Enumerate all $2^{n-1}$ bitmasks of split points for $n \le 12$, compute $C_b = bS + \text{prefT}[r_b]$ directly from the definition, and assert equality with $dp[n]$.

---

# Section 23 — Knapsack and Subset DP

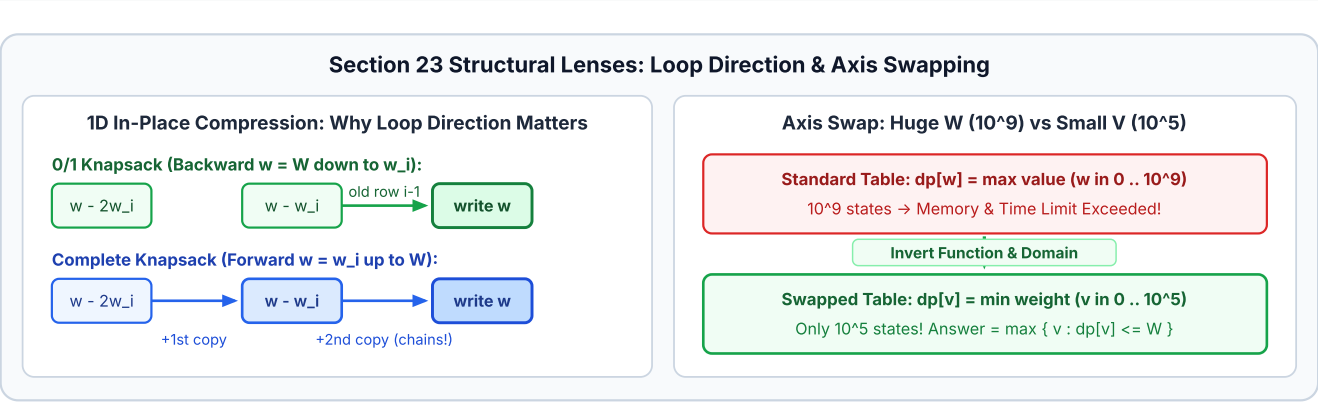
## The Discovery & Modeling Approach

In Section 22, decisions were made along a sequence where the main coordinate was the index $i$. In **Knapsack and Subset DP**, we still process $n$ items one by one ($i = 1, 2, \ldots, n$), but the future legality of remaining items depends on an **accumulated resource vector** (such as total weight $w$, total value $v$, or signed balance $\Delta$) rather than just the last index.

The discovery progression for every subset/knapsack problem follows three steps: 1. **Identify the Resource Equivalence Class:** Two subsets $A, B \subseteq \{1, \ldots, i\}$ are equivalent for the remaining items $\{i+1, \ldots, n\}$ if they consume the exact same resource budget $w$ (or if one dominates the other by achieving better value at equal or smaller weight). 2. **Choose the Iteration Direction (In-Place DAG Order):** When compressing $dp[i][w]$ to a single 1D array $dp[w]$, each transition reads $dp[w - w_i]$. - If each item can be used **at most once (0/1 Knapsack)**, we must read $dp[i-1][w - w_i]$ from the *previous* layer, so we iterate $w$ **downward** ($W \to w_i$). - If each item can be used **unlimited times (Complete/Unbounded Knapsack)**, we want to allow chaining $dp[i][w - w_i] \to dp[i][w]$ within the *current* layer, so we iterate $w$ **upward** ($w_i \to W$). 3. **Check Dimension Swapping & Bitset Compression:** If the capacity $W$ is huge ($10^9$) while the maximum possible total value $V = \sum v_i$ is small ($10^5$), **swap the role of state and objective**: store the *minimum weight* required to achieve exact value $v$. If the DP value is purely boolean (`true`/`false` reachability), pack $64$ states per machine word using `std::bitset` to divide the runtime by $w_{\text{word}} = 64$.

### Decision Compass & Structural Lenses

| Knapsack / Subset Pattern | Resource Constraints | State Definition | Loop Order & Technique | Complexity |
|---|---|---|---|---|
| **1. 0/1 Knapsack** | Each item used $\leq 1$ time; $W \leq 10^5$ | $dp[w]$ = max value with weight $w$ | Item $i$ outer; weight $w$ **backward** $W \to w_i$ | $O(nW)$ time, $O(W)$ space |
| **2. Complete (Unbounded) Knapsack** | Each item used $\geq 0$ times; $W \leq 10^5$ | $dp[w]$ = max value with weight $w$ | Item $i$ outer; weight $w$ **forward** $w_i \to W$ | $O(nW)$ time, $O(W)$ space |
| **3. Axis-Swapped Knapsack** | Weight $W \leq 10^9$ (huge!), total value $V = \sum v_i \leq 10^5$ | $dp[v]$ = **min weight** to achieve exact value $v$ | Item $i$ outer; value $v$ backward $V \to v_i$ | $O(nV)$ time, $O(V)$ space |
| **4. Signed-Offset Subset DP** | Items contribute $+a_i$ or $-b_i$; balance $\Delta \in [-M, +M]$ | $dp[i][\Delta + \text{OFFSET}]$ with $\text{OFFSET} = M$ | Two-row rolling table `prev` and `cur` to prevent index collision | $O(nM)$ time, $O(M)$ space |
| **5. Bitset Reachability** | Boolean subset sum $dp[s] \in \{0, 1\}$; $S \leq 2 \cdot 10^6$ | `std::bitset<MAXS> dp` | `dp \|= (dp << w_i)` shifts & ORs 64 states at once | $O(nS/64)$ time |



## Worked Problem 23.1: Deep-Space Payload & Spare-Module Bay

**Problem & Constraints.** A deep-space probe has a total mass budget $W$ ($1 \le W \le 10^9$). You are choosing from two catalogs: 1. **Primary Instruments ($n$ items, $1 \le n \le 100$):** Instrument $i$ has a huge structural mass $M_i$ ($1 \le M_i \le 10^9$) and yields a discrete science score $v_i$ ($1 \le v_i \le 1000$). Each primary instrument can be installed **at most once** (0/1 choice). 2. **Calibration Cells ($m$ types, $1 \le m \le 100$):** Cell type $j$ has mass $c_j$ ($1 \le c_j \le 10^9$) and yields science score $s_j$ ($1 \le s_j \le 1000$), and can be installed **any nonnegative integer number of times** (unbounded choice), provided the **total science score across the entire probe** does not exceed the telemetry bandwidth cap $V_{\max} = \sum_{i=1}^{n} v_i + 2000 \le 1.02 \cdot 10^5$.

Find the maximum total science score achievable within total mass $W$ and telemetry cap $V_{\max}$.

> **Pause & Discover:** The mass budget $W \le 10^9$ rules out indexing any array by mass. Yet the telemetry cap bounds the maximum possible science score to $V_{\max} \le 1.02 \cdot 10^5$. How can we combine 0/1 primary instruments and unbounded calibration cells in one unified DP table indexed by science score?

### Step 1 — Exact Model & Bottleneck

An exact configuration selects a subset $S \subseteq \{1, ..., n\}$ of primary instruments and a multiplicity vector $(k_1, ..., k_m) \in \mathbb{Z}^m_{\ge 0}$ of calibration cells such that

$$\sum_{i \in S} M_i + \sum_{j=1}^{m} k_j c_j \le W \quad \text{and} \quad V = \sum_{i \in S} v_i + \sum_{j=1}^{m} k_j s_j \le V_{\max}.$$

Indexing the DP by mass $w \in [0, W]$ requires $10^9$ states, which is impossible in both time and memory.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Sorting by ratio $v_i/M_i$ and taking greedily.** Suppose $W = 10$, with one primary instrument $(M_1 = 6, v_1 = 7)$ and one calibration cell $(c_1 = 5, s_1 = 5)$. Ratio greedy picks the primary instrument first (ratio $7/6 > 5/5$), leaving mass $4$, which cannot fit anything else, for score $7$. Taking two copies of the calibration cell uses mass $5 + 5 = 10$ and achieves score $10 > 7$.

**Mistake 2: Running forward loops for Primary Instruments or backward loops for Calibration Cells.** In our axis-swapped table $dp[v]$ = minimum mass to reach exact score $v$: - If we iterate $v$ **forward** ($v_i \to V_{\max}$) for a 0/1 Primary Instrument, updating $dp[v_i]$ and then $dp[2v_i]$ in the same pass installs the unique primary instrument twice! - Conversely, if we iterate $v$ **backward** for an unbounded Calibration Cell, we allow at most one copy of cell $j$.

### Step 3 — The Turn: Discovering the Structure

Why can we swap the axes? In standard knapsack, for each weight $w$ we maximize value $v$. Because "higher value for the same or smaller weight" is a monotone dominance relation, we can invert the pareto frontier: **for every possible integer science score $v \in [0, V_{\max}]$, compute $dp[v]$, the minimum possible total mass required to achieve exact science score $v$.** Once $dp[0..V_{\max}]$ is computed, any score $v$ is feasible if and only if $dp[v] \le W$!

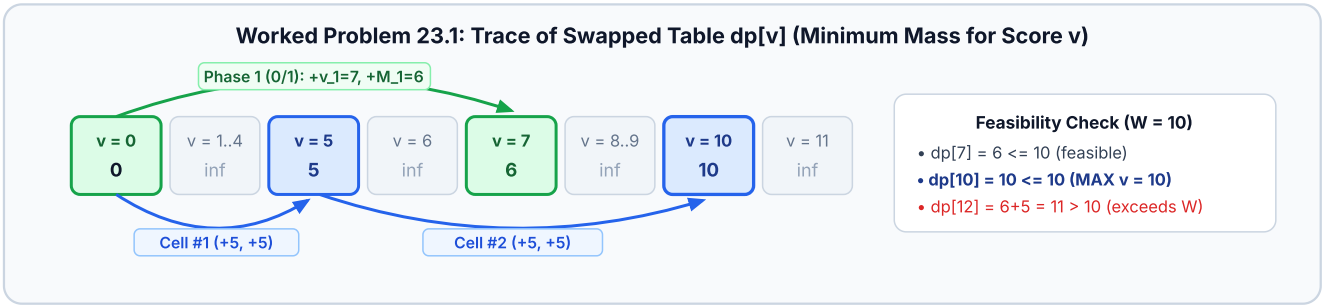
Furthermore, in this single array $dp[0..V_{\max}]$: 1. We first process each **0/1 Primary Instrument** $i$ by iterating $v$ **backward** from $V_{\max}$ down to $v_i$. 2. We then process each **Unbounded Calibration Cell** $j$ by iterating $v$ **forward** from $s_j$ up to $V_{\max}$.

**Axis-Swapped Mixed 0/1 & Complete Knapsack Recurrence:**
Let $dp[v]$ be the minimum total mass required to achieve exact science score $v \in [0, V_{\max}]$.

**Initialization:** $dp[0] = 0$, and $dp[v] = +\infty$ for all $1 \le v \le V_{\max}$.
**Phase 1 — 0/1 Primary Instruments ($i = 1, ..., n$, Backward Loop):**
For $v = V_{\max}, V_{\max} - 1, ..., v_i$:
$dp[v] \leftarrow \min\left( dp[v],\ dp[v - v_i] + M_i \right)$
**Phase 2 — Complete Calibration Cells ($j = 1, ..., m$, Forward Loop):**
For $v = s_j, s_j + 1, ..., V_{\max}$:
$dp[v] \leftarrow \min\left( dp[v],\ dp[v - s_j] + c_j \right)$
**Final Answer:** $\max\{\, v \in [0, V_{\max}] \mid dp[v] \le W \,\}$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $W = 10$, $V_{\max} = 11$, with one 0/1 instrument $(M_1 = 6, v_1 = 7)$ and one unbounded cell $(c_1 = 5, s_1 = 5)$: 1. **Init:** $dp[0] = 0$, all other $dp[1..11] = \infty$. 2. **Phase 1 (0/1 Instrument $M_1 = 6, v_1 = 7$, backward $v = 11 \to 7$):** Only $v - 7 = 0$ is finite, so $dp[7] = \min(\infty, dp[0] + 6) = 6$. Because $v$ moved backward, $dp[7]$ was updated *after* $v = 11..8$ were checked, so $2 \times v_1$ could never be reached. 3. **Phase 2 (Unbounded Cell $c_1 = 5, s_1 = 5$, forward $v = 5 \to 11$):** - At $v = 5$: $dp[5] = \min(\infty, dp[0] + 5) = 5$ (1 copy of cell). - At $v = 10$: $dp[10] = \min(\infty, dp[5] + 5) = 10$ (reads the *updated* $dp[5] = 5$, chaining 2 copies of cell!). 4. **Query:** The finite entries are $dp[0] = 0, dp[5] = 5, dp[7] = 6, dp[10] = 10$. The largest $v$ with $dp[v] \leq 10$ is $v = 10$.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Because additions of mass and score commute, any valid selection can be ordered so that all 0/1 primary instruments are chosen first, followed by all copies of calibration cell 1, then cell 2, etc. By induction on the items processed, after Phase 1, $dp[v]$ equals the minimum mass of a 0/1 subset with exact score $v$. In Phase 2, when processing cell $j$ in increasing order of $v$, $dp[v]$ can transition from $dp[v - ks_j] + kc_j$ for any $k \geq 0$ via $k$ chained forward steps, and cannot exceed the optimal mixed combination. Thus $dp[v] \leq W$ holds if and only if some valid configuration achieves score $v$ with mass at most $W$.

**Complexity:** $O((n + m)V_{\max})$ time and $O(V_{\max})$ space—roughly $200 \times 10^5 = 2 \cdot 10^7$ operations, running in $< 25$ ms.

---

## Worked Problem 23.2: Bipolar Capacitor Balancing (Shifted-Offset & Bitset Subset DP)

**Problem & Constraints.** An array of $n$ reversible supercapacitors $(1 \leq n \leq 2000)$ must all be wired into a circuit. Capacitor $i$ has a forward voltage $+a_i$ and a reverse-bias voltage $-b_i$ $(1 \leq a_i, b_i \leq 500)$. For each capacitor $i$, you **must** choose either $+a_i$ or $-b_i$. In addition, you have $m$ optional **trimming resistors** $(0 \leq m \leq 2000)$, where resistor $j$ can either be skipped $(0)$ or engaged to shift the net voltage by $+r_j$ $(1 \leq r_j \leq 500)$. Find the minimum possible absolute net voltage $|\Delta|$ achievable after wiring all $n$ capacitors and any subset of the $m$ trimming resistors.

> **Pause & Discover:** Two challenges appear here: (1) intermediate voltages can be negative, which breaks array indexing, and (2) $2000 \times (2000 \times 500) = 2 \cdot 10^9$ transitions is on the borderline of TLE. How can a coordinate shift eliminate negative indices, and how does `std::bitset` execute 64 transitions in a single CPU instruction?

### Step 1 — Exact Model & Bottleneck

Every assignment is a choice $x_i \in \{+a_i, -b_i\}$ for $i = 1..n$ and $y_j \in \{0, +r_j\}$ for $j = 1..m$. There are $2^{n+m}$ combinations. The minimum possible net sum is $-B = -\sum_{i=1}^{n} b_i \geq -10^6$, and the maximum is $A + R = \sum a_i + \sum r_j \leq 2 \cdot 10^6$. The span of reachable sums is $L = A + B + R \leq 3 \cdot 10^6$. A naive boolean table of size $(n + m) \times L$ takes $4000 \times 3 \cdot 10^6 \approx 1.2 \cdot 10^{10}$ operations, which will TLE.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Updating a single shifted array in-place for a mandatory $+a_i$ or $-b_i$ choice.** In standard 0/1 subset sum with options $\{0, +a_i\}$, 0 is the identity shift, so `dp |= (dp << a_i)` works in-place because the unshifted `dp` already holds the $+0$ option. Here, $+0$ is **forbidden** for the $n$ reversible capacitors! Every state must shift by *either* $+a_i$ *or* $-b_i$; if you keep the old bitset or update in-place, you illegally allow skipping capacitor $i$.

**Mistake 2: Pruning intermediate sums outside $[-500, +500]$ unless carefully randomized.** Without shuffling, the first $1000$ capacitors could all have $a_i = 500, b_i = 1$ and the last $1000$ could have $a_i = 1, b_i = 500$, forcing the optimal trajectory to climb to $+500,000$ before descending back to $0$.

### Step 3 — The Turn: Discovering the Structure

First, eliminate negative indices entirely via an **affine coordinate shift** (or equivalently, by factoring out the baseline $-\sum_{i=1}^{n} b_i$): - Notice that choosing between $+a_i$ and $-b_i$ is algebraically identical to starting at the fixed baseline sum $S_0 = -\sum_{i=1}^{n} b_i$, and then for each capacitor $i$, either adding $0$ (if we chose $-b_i$) or adding $d_i = a_i + b_i > 0$ (if we chose $+a_i$)! - Alternatively, if we keep signed sums explicitly with $\text{OFFSET} = \sum_{i=1}^{n} b_i$, then a bitset `B` indexed by $k = \Delta + \text{OFFSET}$ updates at capacitor $i$ via:

$$\texttt{B} \leftarrow (\texttt{B} \ll a_i) \ | \ (\texttt{B} \gg b_i)$$

and at optional resistor $j$ via:

$$\texttt{B} \leftarrow \texttt{B} \ | \ (\texttt{B} \ll r_j).$$

Because C++ `std::bitset<MAXL>` packs 64 boolean states into each 64-bit word, a full bit-shift and bitwise OR across $L = 3 \cdot 10^6$ bits executes in $L/64 \approx 4.6 \cdot 10^4$ word operations! Across $n + m = 4000$ steps, total word operations drop from $1.2 \cdot 10^{10}$ to $1.8 \cdot 10^8$ (or half that if we use the baseline trick $S_0 = -\sum b_i$, which needs no negative offset buffer for the resistors!).

**Shifted-Offset Bitset Recurrence:**

Let $\text{OFFSET} = \sum_{i=1}^{n} b_i$, and let `std::bitset<MAX_SPAN> dp` represent reachable shifted sums $k = \Delta + \text{OFFSET}$.

**Base Case:** $\texttt{dp.reset}(); \quad \texttt{dp.set}(\texttt{OFFSET});$ (representing $\Delta = 0$).

**Phase 1 — Mandatory Bipolar Capacitors ($i = 1, \ldots, n$):**

$$\texttt{dp} = (\texttt{dp} \ll a_i) \ | \ (\texttt{dp} \gg b_i)$$

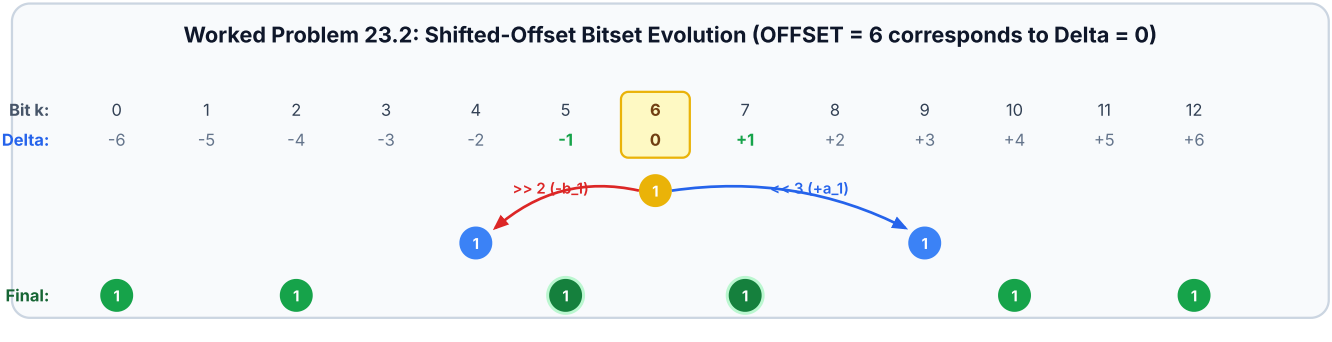
**Phase 2 — Optional Trimming Resistors ($j = 1, \ldots, m$):**

$$\texttt{dp} \ | = \ (\texttt{dp} \ll r_j)$$

**Final Answer:** $\min\{\, |k - \text{OFFSET}| \,|\, \texttt{dp.test}(k) = 1 \,\}$.

### Step 4 — Progressive Construction & Visual Trace

Let $n = 2$ capacitors with $(a_1, b_1) = (3, 2)$ and $(a_2, b_2) = (1, 4)$, and $m = 1$ resistor with $r_1 = 2$. Here $\text{OFFSET} = b_1 + b_2 = 2 + 4 = 6$ . - **Init ($i = 0$):** $\Delta \in \{0\} \Leftrightarrow$ bit $6$ is set: `dp` has $\{6\}$. - **Capacitor 1 ($+3$ or $-2$):** `(dp << 3) | (dp >> 2)` sets bits $\{6 + 3, 6 - 2\} = \{9, 4\}$ (representing $\Delta \in \{+3, -2\}$). Notice bit $6$ is cleared because `=` overwrites `dp`! - **Capacitor 2 ($+1$ or $-4$):** `(dp << 1) | (dp >> 4)` shifts $\{9, 4\}$ by $+1 \to \{10, 5\}$ and by $-4 \to \{5, 0\}$. Their bitwise OR gives $\{0, 5, 10\}$ (representing $\Delta \in \{-6, -1, +4\}$). - **Resistor 1 ($0$ or $+2$):** `dp |= (dp << 2)` unions $\{0, 5, 10\}$ with $\{2, 7, 12\}$ to get $\{0, 2, 5, 7, 10, 12\}$ (representing $\Delta \in \{-6, -4, -1, +1, +4, +6\}$). - **Query:** Closest set bits to $\text{OFFSET} = 6$ are $k = 5$ and $k = 7$, giving $|\Delta| = |5 - 6| = 1$.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Because $\text{OFFSET} = \sum_{i=1}^{n} b_i$, after any prefix of $i$ capacitors and $j$ resistors, the shifted sum satisfies $k = \Delta + \sum_{t=1}^{n} b_t \geq \sum_{t=i+1}^{n} b_t \geq 0$, and $k \leq \text{OFFSET} + \sum_{t=1}^{n} a_t + \sum_{s=1}^{m} r_s = \text{MAX_SPAN}$. Thus no valid intermediate state ever underflows below bit $0$ or overflows past `MAX_SPAN`. By induction, bit $k$ is $1$ after step $i$ if and only if $k - \text{OFFSET}$ is a valid signed sum of the first $i$ elements.

**Small-Case Oracle:** For $n, m \leq 10$, recursively branch on $+a_i / -b_i$ and $0 / +r_j$ to collect a `std::set<int>` of all reachable signed sums, and verify that the shifted bitset contains the exact same set of integers.

# Section 24 — Grid DP

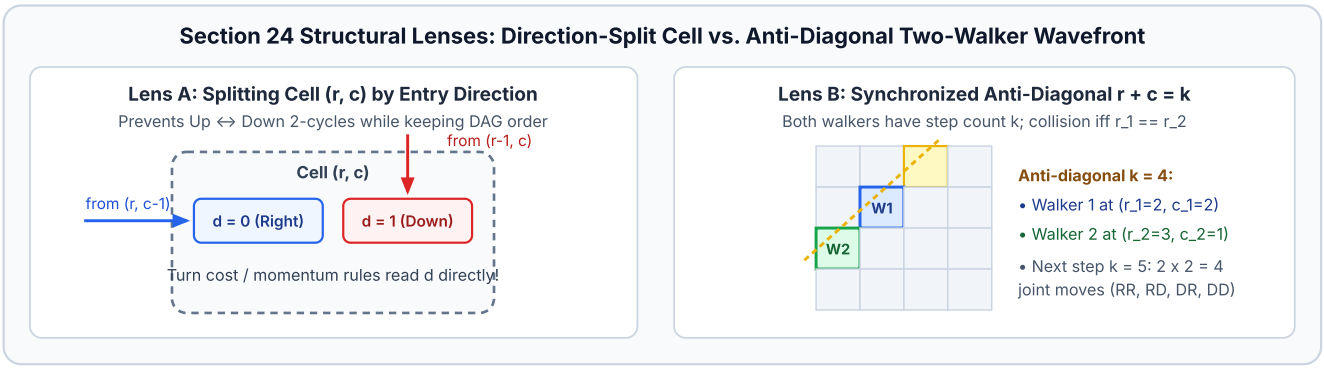
## The Discovery & Modeling Approach

A 2D grid $G[1..R][1..C]$ is a visual representation of a graph. Whether a grid problem can be solved by **Dynamic Programming** rather than Dijkstra or BFS hinges on a single structural question: **do the legal moves induce a Directed Acyclic Graph (DAG)?**

- If moves are restricted to **Right** $(r, c+1)$ and **Down** $(r+1, c)$, then the Manhattan coordinate $k = r + c$ strictly increases by 1 on every step. No cycle can exist, and any row-major, column-major, or anti-diagonal order is a valid topological sort.
- If moves also allow **Up** or **Left** *within a single column* without revisiting cells, we can still maintain acyclicity by augmenting the state with the **entry direction** $d \in \{\text{FromLeft}, \text{FromUp}, \text{FromDown}\}$ so that `FromUp` and `FromDown` never transition directly into each other at the same cell.
- What if a problem asks for a **round-trip** $(1,1) \to (R,C) \to (1,1)$ or **two walkers** whose paths share cell rewards (counted only once if both step on the same cell)? Solving path 1 greedily and then deleting its rewards fails. Instead, we reverse the return path so both walkers travel from $(1,1)$ to $(R,C)$, and advance them **synchronously along the anti-diagonal** $k = r_1 + c_1 = r_2 + c_2$. Whenever $r_1 = r_2$ at step $k$, it follows automatically that $c_1 = k - r_1 = k - r_2 = c_2$, allowing us to deduplicate shared cells locally!

### Decision Compass & Structural Lenses

| Grid DP Lens | Move / Path Structure | Minimal DP State | Topological Evaluation Order | Complexity |
|---|---|---|---|---|
| **1. Monotone Right-Down DAG** | Moves $(+1,0)$ and $(0,+1)$ only | $dp[r][c]$ | Row $r = 1 \to R$, Col $c = 1 \to C$ ($O(C)$ rolling space) | $O(RC)$ time |
| **2. Momentum / Turn-Augmented Grid** | Cost or legality depends on previous move direction or streak length | $dp[r][c][\text{dir}][\text{streak}]$ | Row-major if moves are monotone; column-by-column if vertical moves exist | $O(RC \cdot \lvert\text{dir}\rvert \cdot S)$ time |
| **3. Column-Sweep 3-Direction Grid** | Right, Up, and Down allowed, no cell visited twice | $dp[r][c][d]$ with $d \in \{\text{Left}, \text{Down}, \text{Up}\}$ | Outer loop Col $c = 1 \to C$; two passes per column (top-down for Down, bottom-up for Up) | $O(RC)$ time |
| **4. Synchronized Two-Walker (Anti-Diagonal)** | Two paths $(1,1) \to (R,C)$ sharing cell rewards once | $dp[k][r_1][r_2]$ where $c_1 = k - r_1, c_2 = k - r_2$ | Anti-diagonal step $k = 2 \to R + C$; pairs $(r_1, r_2)$ | $O((R+C)R^2)$ time, $O(R^2)$ rolling space |



## Worked Problem 24.1: Momentum-Constrained Laser Routing

**Problem & Constraints.** An optical routing board is an $R \times C$ grid ($2 \le R, C \le 500$). Entering cell $(r, c)$ incurs signal attenuation $A[r][c] \ge 0$ ($0 \le A[r][c] \le 10^6$), except blocked cells marked with `#`. A laser beam enters at $(1, 1)$ and must exit at $(R, C)$, moving only **Right** $(r, c+1)$ or **Down** $(r+1, c)$. Because of mirror alignment physics: 1. Each time the beam **changes direction** (from Right to Down, or from Down to Right) at cell $(r, c)$ upon leaving it, a mirror reflection penalty $P$ ($0 \le P \le 10^6$) is added. 2. However, the beam cannot travel more than $L = 3$ consecutive steps in the **same direction** without either turning or reaching $(R, C)$ (thermal lensing limit).

Find the minimum total attenuation plus reflection penalty to route the beam from $(1, 1)$ to $(R, C)$, or report `-1` if impossible.

> **Pause & Discover:** If two prefixes of the beam reach cell $(r, c)$, what exact properties of their arrival determine which outgoing moves from $(r, c)$ are legal and whether a reflection penalty $P$ must be paid?

### Step 1 — Exact Model & Bottleneck

A valid trajectory is a sequence of $R + C - 2$ moves in $\{\mathrm{R}, \mathrm{D}\}$ containing no substring `RRRR` or `DDDD` and avoiding blocked cells. For $R = C = 500$, the total number of monotone paths is $\binom{998}{499} \approx 2^{995}$, so explicit path enumeration is out of the question.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Storing only $dp[r][c][\mathrm{dir}]$ without consecutive streak length.** If we know we arrived at $(r, c)$ moving Right (`dir = 0`), can we move Right again to $(r, c+1)$? That depends on whether we have moved Right $1$, $2$, or $3$ times in a row! Without knowing the streak length $s \in \{1, 2, 3\}$, we cannot enforce the $L = 3$ thermal lensing cap.

**Mistake 2: Trying to jump $1, 2,$ or $3$ steps at once without tracking the direction of the jump.** Jumping $s \in \{1, 2, 3\}$ steps in alternating directions is actually an equivalent way to view the state machine! Whether we step cell-by-cell with $(r, c, d, s)$ or jump run-by-run with $(r, c, d)$, the key requirement is that a run in direction $d$ must be followed by a run in direction $1 - d$. Forgetting to alternate $d$ allows two consecutive Right jumps of lengths $2 + 2 = 4 > 3$.

### Step 3 — The Turn: Discovering the Structure

At any cell $(r, c) \neq (1, 1)$, a prefix trajectory is completely summarized for the future by: - Its last step direction $d \in \{0\ (\text{Right}), 1\ (\text{Down})\}$, and - Its current consecutive streak $s \in \{1, 2, 3\}$ in direction $d$.

Why is $(r, c, d, s)$ sufficient? 1. Moving in the **same direction** $d$ from $(r, c)$ is legal if and only if $s < 3$; it leads to streak $s + 1$ in direction $d$ and pays $0$ reflection penalty. 2. Moving in the **opposite direction** $1 - d$ from $(r, c)$ is **always** legal (for any $s \in \{1, 2, 3\}$); it resets the streak to $s' = 1$ in direction $1 - d$ and pays reflection penalty $+P$.

**Momentum-Augmented Grid DP Recurrence:**

Let $dp[r][c][d][s]$ be the minimum cost to reach $(r, c)$ with last move direction $d \in \{0, 1\}$ and consecutive run length $s \in \{1, 2, 3\}$.

**Base Cases (First Step from $(1, 1)$):**

If $(1, 2)$ is open: $dp[1][2][0][1] = A[1][1] + A[1][2]$.

If $(2, 1)$ is open: $dp[2][1][1][1] = A[1][1] + A[2][1]$.

All other entries initialized to $+\infty$.

**Transitions into open cell $(r, c)$:**

From $(r, c-1)$ via a Right step ($d = 0$):

$$dp[r][c][0][1] = A[r][c] + P + \min_{s \in \{1,2,3\}} dp[r][c-1][1][s] \quad (\text{turn from Down to Right})$$

$$dp[r][c][0][s] = A[r][c] + dp[r][c-1][0][s-1] \quad \text{for } s \in \{2, 3\} \ (\text{continue Right})$$

From $(r-1, c)$ via a Down step ($d = 1$):

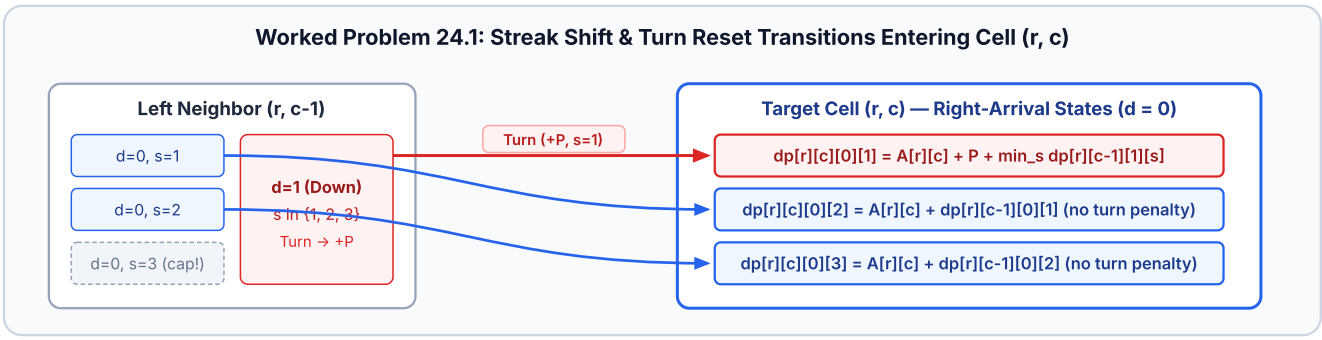
$$dp[r][c][1][1] = A[r][c] + P + \min_{s \in \{1,2,3\}} dp[r-1][c][0][s] \quad (\text{turn from Right to Down})$$

$$dp[r][c][1][s] = A[r][c] + dp[r-1][c][1][s-1] \quad \text{for } s \in \{2, 3\} \ (\text{continue Down})$$

**Final Answer:** $\min_{d \in \{0,1\},\ s \in \{1,2,3\}} dp[R][C][d][s]$.

### Step 4 — Progressive Construction & Visual Trace

Notice how clean the transitions inside each cell $(r, c)$ are: every state $(r, c, 0, s)$ depends only on $(r, c-1)$, and every state $(r, c, 1, s)$ depends only on $(r-1, c)$. Thus standard row-by-row, column-by-column iteration ($r = 1 \to R$, $c = 1 \to C$) evaluates all predecessors before they are read.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Every non-empty monotone path from $(1, 1)$ to $(r, c)$ has a unique final move direction $d \in \{0, 1\}$ and a unique maximal suffix of consecutive steps in direction $d$ of length $s \geq 1$. The path respects the $L = 3$ thermal cap if and only if $s \in \{1, 2, 3\}$ at every intermediate cell. Removing the last step yields a valid path to the predecessor cell with streak $s - 1$ (if $s \geq 2$) or with opposite direction $1 - d$ and some streak $s' \in \{1, 2, 3\}$ (if $s = 1$). Thus the 6-state vector per cell is both sound and complete.

**Complexity:** 6 states per cell, constant transitions per state $\implies O(RC)$ time and $O(C)$ space (keeping only the previous row).

---

## Worked Problem 24.2: Dual-Rover Sample Collection (Synchronized Anti-Diagonal DP)

**Problem & Constraints.** Two autonomous rovers start simultaneously at $(1, 1)$ on an $N \times N$ Martian plateau ($2 \leq N \leq 300$) and both must reach the extraction pad at $(N, N)$. In each step, a rover moves either **Right** $(r, c+1)$ or **Down** $(r+1, c)$. Cell $(r, c)$ contains a mineral sample worth $V[r][c]$ ($-10^4 \leq V[r][c] \leq 10^4$), or is an impassable crater (`#`). - If **one** rover visits cell $(r, c)$, the mission collects $V[r][c]$. - If **both** rovers visit the same cell $(r, c)$, the mineral sample can only be harvested **once**, so the mission collects $V[r][c]$ (not $2V[r][c]$)—and in addition, to prevent rover collision damage on narrow ridges, whenever both rovers occupy the same interior cell $(r, c) \notin \{(1, 1), (N, N)\}$, the mission pays a proximity shielding penalty $D \geq 0$.

Find the maximum net mission value (or report that no pair of valid paths exists).

> **Pause & Discover:** Why does finding the single best path for Rover 1, zeroing out its cells, and then finding the best path for Rover 2 fail? And how can we tell whether Rover 1's path and Rover 2's path ever intersect without storing the entire set of cells visited by Rover 1?

### Step 1 — Exact Model & Bottleneck

An exact solution chooses an entire path $P_1$ for Rover 1 and an entire path $P_2$ for Rover 2 (each of length $2N - 1$ cells), sums $V[r][c]$ over the set union $P_1 \cup P_2$, and subtracts $D$ for every interior cell in the intersection $P_1 \cap P_2$. If we try to run Rover 2 *after* Rover 1, Rover 2 needs to know the exact subset of cells visited by Rover 1—which has $\binom{2N-2}{N-1}$ possibilities!

### Step 2 — Tempting Mistakes & Counterexample

**Mistake: Sequential Two-Pass Greedy (Best Path 1, then Best Path 2 on residual grid).** Consider the $3 \times 3$ grid with $D = 0$:

$$V = \begin{bmatrix} 0 & 5 & 0 \\ 5 & 10 & 5 \\ 0 & 5 & 0 \end{bmatrix}$$

- Any single path that takes the center $10$ must collect two $5$s on its way: e.g., $(1,1) \to (1,2) \to (2,2) \to (2,3) \to (3,3)$ gets $0+5+10+5+0=20$. - After zeroing out those cells, the remaining two $5$s at $(2,1)$ and $(3,2)$ **cannot both be visited** by a single monotone path without passing through $(2,2)$ or going backward! Wait: $(1,1) \to (2,1) \to (3,1) \to (3,2) \to (3,3)$ *does* get both $(2,1)$ and $(3,2)$ here, for $20+10=30$. - Now change $(1,3)$ and $(3,1)$ to $-100$ (blocked)! Then the *only* way to collect $(2,1)$ and $(2,3)$ is via $(2,2)$, and the *only* way to collect $(1,2)$ and $(3,2)$ is also via $(2,2)$. Wait: if $(1,2)=10, (2,1)=10, (2,3)=10, (3,2)=10, (2,2)=0$, and $(1,3)=9, (3,1)=9$: - Path 1 takes $(1,1) \to (1,2) \to (2,2) \to (3,2) \to (3,3)$ for $10+0+10=20$, splitting the grid so Path 2 can get at most $10+9=19$ (total $39$). - Instead, if Path 1 stays on the upper rim $(1,1) \to (1,2) \to (2,2) \to (2,3) \to (3,3)$ ($20$) and Path 2 stays on the lower rim $(1,1) \to (2,1) \to (2,2) \to (3,2) \to (3,3)$ ($20$), or one takes the corner $9$s, the first greedy path can cut across the diagonal and strand higher combined rewards on opposite sides!

### Step 3 — The Turn: Discovering the Structure

Look at the invariant of Right/Down movement: after $t$ steps from $(1,1)$, **every** valid rover is located at some cell $(r,c)$ satisfying

$$r+c=t+2=k.$$

Therefore, $P_1$ and $P_2$ can intersect at cell $(r,c)$ **if and only if** both rovers arrive at $(r,c)$ at the exact same anti-diagonal step $k=r+c$! If we advance both rovers simultaneously one step at a time from $k=2$ to $k=2N$, their positions on anti-diagonal $k$ are completely determined by their row indices $(r_1,r_2)$ alone, since $c_1=k-r_1$ and $c_2=k-r_2$. - If $r_1 \neq r_2$, the two rovers are on distinct cells $(r_1,c_1) \neq (r_2,c_2)$, so we add $V[r_1][c_1]+V[r_2][c_2]$. - If $r_1=r_2$, both rovers are on the **exact same cell** $(r_1,c_1)=(r_2,c_2)$, so we add $V[r_1][c_1]$ **only once** (and subtract $D$ if $2<k<2N$).

**Synchronized Anti-Diagonal Two-Walker Recurrence:**

For anti-diagonal index $k \in \{2,3,\ldots,2N\}$ and row pair $r_1,r_2 \in [\max(1,k-N), \min(N,k-1)]$, let $c_1=k-r_1$ and $c_2=k-r_2$.

Define the step gain:

$$\text{gain}(k,r_1,r_2)=\begin{cases} V[r_1][c_1]+V[r_2][c_2] & \text{if } r_1 \neq r_2, \\ V[r_1][c_1]-D\cdot \mathbf{1}_{2<k<2N} & \text{if } r_1=r_2. \end{cases}$$

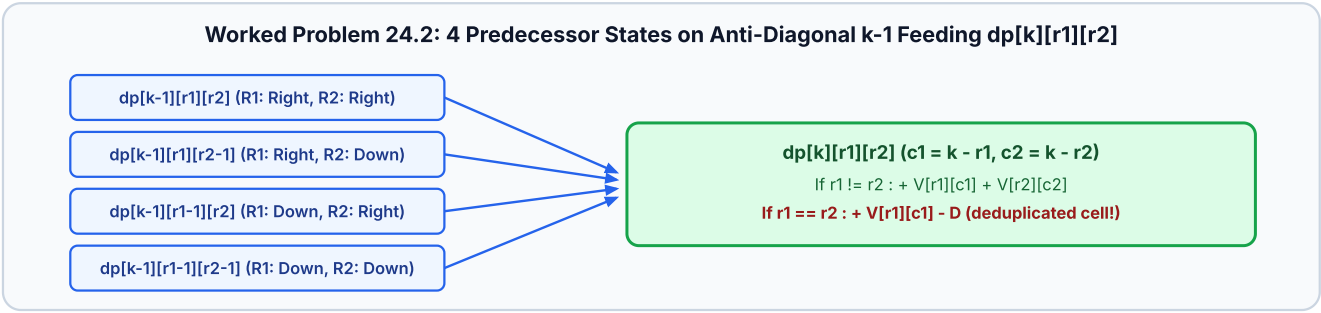
**Transition ($k=3\ldots2N$):**

$$dp[k][r_1][r_2]=\text{gain}(k,r_1,r_2)+\max\begin{cases} dp[k-1][r_1][r_2] & \text{(Right, Right)} \\ dp[k-1][r_1][r_2-1] & \text{(Right, Down)} \\ dp[k-1][r_1-1][r_2] & \text{(Down, Right)} \\ dp[k-1][r_1-1][r_2-1] & \text{(Down, Down)} \end{cases}$$

**Final Answer:** $dp[2N][N][N]$.

### Step 4 — Progressive Construction & Visual Trace

Why does $r_1$ determine whether Rover 1 moved Right or Down from step $k-1$ to step $k$? - On step $k-1$, Rover 1 was at some cell $(r_1',c_1')$ with $r_1'+c_1'=k-1$. - If Rover 1 moved **Right**, its row stayed the same ($r_1'=r_1$, while $c_1'=c_1-1$). - If Rover 1 moved **Down**, its row increased by $1$ ($r_1'=r_1-1$, while $c_1'=c_1$). Combining both choices for Rover 1 ($r_1$ or $r_1-1$) and Rover 2 ($r_2$ or $r_2-1$) gives the $2\times2=4$ predecessor pairs!




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Because any cell $(r,c)$ on a monotone path from $(1,1)$ is visited at step $k=r+c$, two monotone paths $P_1,P_2$ share cell $(r,c)$ if and only if both reach row $r$ at step $k=r+c$. Thus summing $\text{gain}(k,r_1(k),r_2(k))$ along $k=2\ldots2N$ evaluates the exact objective $\sum_{(r,c)\in P_1\cup P_2} V[r][c]-D\cdot|(P_1\cap P_2)\backslash\{(1,1),(N,N)\}|$ for every pair of trajectories.

**Complexity:** $2N$ diagonals $\times N^2$ row pairs $\times 4$ transitions $= O(N^3)$ time ( $\approx 2.7 \cdot 10^7$ operations for $N = 300$) and $O(N^2)$ memory by rolling the $k$ dimension.

---

# Section 25 — Interval DP

## The Discovery & Modeling Approach

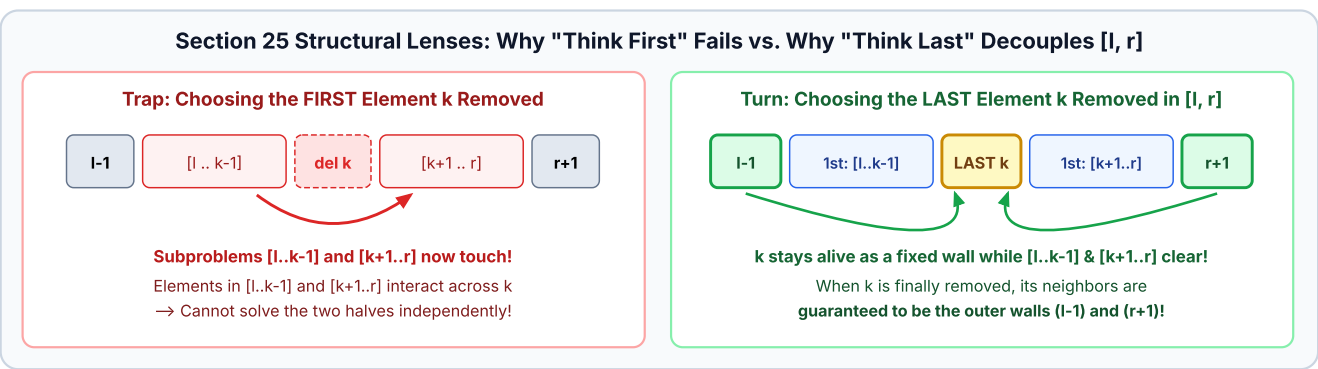
When do we need **Interval DP** ($dp[l][r]$ over contiguous substrings $a[l..r]$) instead of 1D prefix DP ($dp[i]$ over $a[1..i]$)? Ask what remains after you perform one natural operation or undo the final operation: - If the remaining unsolved subproblem is always a **prefix** $1..j$, use **1D DP**. - If operations can nibble from **both ends** ($l+1..r$ or $l..r-1$), **split** a contiguous segment into two independent contiguous segments ($l..k$ and $k+1..r$), or **collapse** an interior element so that its left and right sections merge together, then the set of reachable subproblems is the family of all $\binom{n+1}{2}$ contiguous intervals $[l, r]$.

### The Crucial Dichotomy: "Think First" vs. "Think Last"

The single most important modeling trap in Interval DP is deciding whether your transition chooses the **first** action or the **last** action inside $[l, r]$: 1. **Think First ( fails when neighbors merge!):** Suppose deleting element $k \in [l, r]$ costs $a[k-1] \cdot a[k] \cdot a[k+1]$ and causes $a[k-1]$ and $a[k+1]$ to become adjacent. If you choose the *first* element $k$ deleted, then the remaining elements in $[l, k-1]$ and $[k+1, r]$ now interact across the deleted boundary $k$! The two halves are **not independent**. 2. **Think Last (decouples the subproblems!):** Instead, choose the **last** element $k \in [l, r]$ to be deleted inside $[l, r]$. Because $k$ is deleted *after* all of $[l, k-1]$ and all of $[k+1, r]$ have already vanished, while $[l, k-1]$ is being cleared, $k$ sits intact as its fixed right wall! And when $k$ itself is finally deleted, its immediate surviving neighbors are guaranteed to be the fixed outer walls $l-1$ and $r+1$!

### Decision Compass & Structural Lenses

| Interval DP Lens | Underlying Combinatorial Structure | Canonical State | Transition & Split Choice | Complexity |
|---|---|---|---|---|
| **1. Binary Merge / Parse Tree** | Adjacent segments $[l, k]$ and $[k+1, r]$ merge into $[l, r]$ (e.g., matrix chain, stone merge) | $dp[l][r]$ | $\min_{l \le k < r}\{dp[l][k] + dp[k+1][r]\} + \text{cost}(l, k, r)$ | $O(n^3)$ time, $O(n^2)$ space |
| **2. Think-Last Element Removal** | Removing $k$ merges its left & right neighbors (e.g., burst balloons, polygon triangulation) | $dp[l][r]$ = best to clear open/closed interval $[l, r]$ | Choose **last** $k \in [l, r]$: $dp[l][k-1] + dp[k+1][r] + f(l-1, k, r+1)$ | $O(n^3)$ time, $O(n^2)$ space |
| **3. Endpoint Matching / Palindromic Peeling** | Operations act on $a[l]$ and $a[r]$, or match $a[l] = a[k]$ | $dp[l][r]$ | Peel $l$ ($dp[l+1][r]$), peel $r$ ($dp[l][r-1]$), or pair $a[l] = = a[k]$ | $O(n^2)$ or $O(n^3)$ time |
| **4. Layered / Streak-Carried Interval DP** | Deleting equal-color runs pays $f(\text{run length})$, so outer blocks may wait to join $[l, r]$ | $dp[l][r][c]$ ($c$ = extra items equal to $a[r]$ attached on the right) | Drop $[r \text{ with } c \text{ extras}]$, or merge with matching $a[k] = = a[r]$ via $dp[l][k][c+1] + dp[k+1][r-1][0]$ | $O(n^4)$ worst-case, very fast on sparse matches |



## Worked Problem 25.1: Resonance Crystal Decommissioning (Think-Last Interval DP)

**Problem & Constraints.** A linear containment rack holds $n$ unstable crystals ($1 \le n \le 400$) flanked by two permanent, indestructible lead shields at positions $0$ and $n+1$. Each position $i \in \{0, 1, ..., n+1\}$ has a resonance frequency $f_i \ge 1$ ($1 \le f_i \le 1000$) and an core energy $e_i \ge 0$ ($0 \le e_i \le 10^6$). You must extract all $n$ interior crystals $1, 2, ..., n$ one by one in some permutation. When crystal $k$ is extracted, suppose its **currently adjacent remaining neighbors** to the left and right are $L$ and $R$ (initially $k-1$ and $k+1$, or the permanent shields $0$ and $n+1$ once intermediate crystals are gone). Extracting crystal $k$ releases a shockwave of cost:

$$\text{cost}(k \mid L, R) = e_k + (f_L + f_R) \cdot f_k.$$

After $k$ is extracted, $L$ and $R$ become directly adjacent. Find the minimum total shockwave cost to extract all $n$ crystals.

> **Pause & Discover:** Why does choosing the *first* crystal extracted in $[l, r]$ fail to yield an interval DP? Conversely, if crystal $k \in [l, r]$ is the *last* crystal extracted among all crystals in $[l, r]$, what are $k$'s immediate left and right neighbors at the exact moment $k$ is extracted?

### Step 1 — Exact Model & Bottleneck

An extraction schedule is a permutation $\pi \in S_n$ of $\{1, 2, ..., n\}$. There are $n!$ permutations. Even if we only track the subset of remaining crystals as a bitmask of size $2^n$, $n = 400$ makes subset DP impossible. We must exploit the **contiguity** of cleared intervals.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Greedily extracting the crystal with the largest (or smallest) $f_k$ first.** Suppose $n = 2$ with shields $f_0 = 1, f_3 = 100$, and interior crystals $f_1 = 10, f_2 = 10$ (with $e_1 = e_2 = 0$). - If we extract crystal $1$ first, its neighbors are $0$ and $2$: cost $= (f_0 + f_2) f_1 = (1 + 10) \times 10 = 110$. Then crystal $2$ has neighbors $0$ and $3$: cost $= (1 + 100) \times 10 = 1010$. Total $= 1120$. - If we extract crystal $2$ first, its neighbors are $1$ and $3$: cost $= (10 + 100) \times 10 = 1100$. Then crystal $1$ has neighbors $0$ and $3$: cost $= (1 + 100) \times 10 = 1010$. Total $= 2110$. Why was extracting crystal $1$ first almost twice as cheap? Because crystal $2$ shielded crystal $1$ from the massive right wall $f_3 = 100$!

**Mistake 2: Defining $dp[l][r]$ via the first crystal extracted.** If we remove $k$ first from $[l, r]$, the sub-intervals $[l, k-1]$ and $[k+1, r]$ fuse together into a single contiguous array of length $(r - l)$, not two independent subproblems.

### Step 3 — The Turn: Discovering the Structure

Define $dp[l][r]$ as the minimum cost to extract **all** crystals in the closed interval $[l, r]$, given that the boundary crystals $l-1$ and $r+1$ remain untouched throughout the entire process of clearing $[l, r]$. Now ask: in any valid extraction sequence that clears $[l, r]$ before touching $l-1$ or $r+1$, which crystal $k \in [l, r]$ is extracted **last**? - Because $k$ is extracted *after* every crystal in $[l, k-1]$ and every crystal in $[k+1, r]$, crystal $k$ acts as the stationary right boundary wall $(k)$ while $[l, k-1]$ is being cleared, and as the stationary left boundary wall $(k)$ while $[k+1, r]$ is being cleared! - Therefore, clearing $[l, k-1]$ (flanked by $l-1$ and $k$) and clearing $[k+1, r]$ (flanked by $k$ and $r+1$) are **100% independent** subproblems with optimal costs $dp[l][k-1]$ and $dp[k+1][r]$! - Finally, when both sub-intervals are empty, $k$ stands alone between $l-1$ and $r+1$, and extracting $k$ costs $e_k + (f_{l-1} + f_{r+1}) \cdot f_k$.

**Think-Last Interval DP Recurrence:**

Let $dp[l][r]$ be the minimum shockwave cost to extract all crystals in $[l, r]$ assuming boundary walls $l-1$ and $r+1$ remain in place.

**Base Case:** $dp[l][r] = 0$ whenever $l > r$ (empty interval).

**Topological Order:** Interval length $\text{len} = r - l + 1$ from $1$ up to $n$.

**Transition for $1 \le l \le r \le n$:**

$$dp[l][r] = \min_{l \le k \le r} \Big\{ dp[l][k-1] + dp[k+1][r] + e_k + (f_{l-1} + f_{r+1}) \cdot f_k \Big\}$$

**Final Answer:** $dp[1][n]$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $n = 2$ with $f = [1, 10, 10, 100]$ ($f_0 = 1, f_1 = 10, f_2 = 10, f_3 = 100$) and $e_1 = e_2 = 0$: 1. **Length $\text{len} = 1$:** - $dp[1][1]$ (walls $0$ and $2$): only $k = 1 \Longrightarrow 0 + 0 + (f_0 + f_2) f_1 = (1 + 10) \times 10 = 110$. - $dp[2][2]$ (walls $1$ and $3$): only $k = 2 \Longrightarrow 0 + 0 + (f_1 + f_3) f_2 = (10 + 100) \times 10 = 1100$. 2. **Length $\text{len} = 2$ ($[l, r] = [1, 2]$, walls $0$ and $3$):** - Candidate $k = 1$

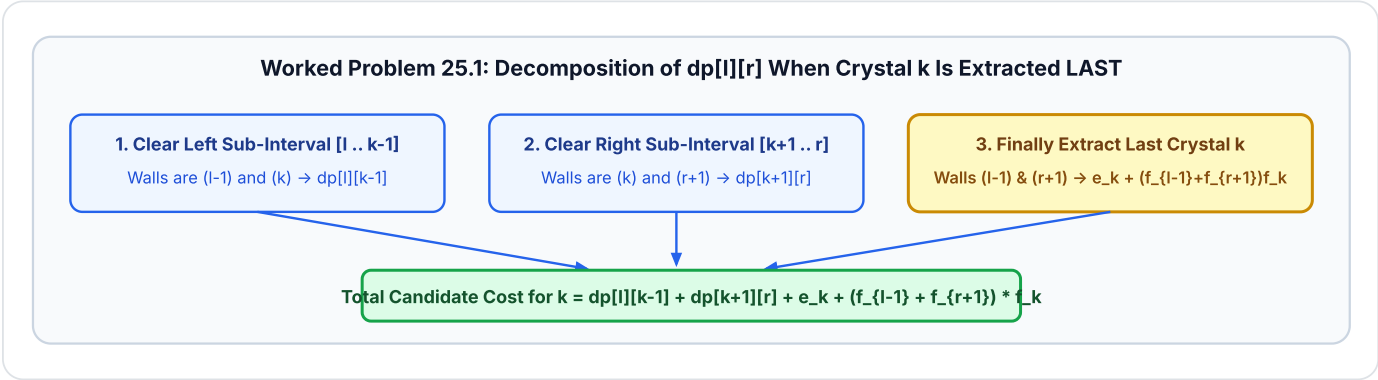
extracted **last** (meaning $2$ is extracted first):

$$dp[1][0] + dp[2][2] + (f_0 + f_3)f_1 = 0 + 1100 + (1 + 100) \times 10 = 2110.$$

- Candidate $k = 2$ extracted **last** (meaning $1$ is extracted first):

$$dp[1][1] + dp[3][2] + (f_0 + f_3)f_2 = 110 + 0 + (1 + 100) \times 10 = 1120.$$

- Minimum is $dp[1][2] = 1120$.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Every extraction order of $[1..n]$ defines a unique Cartesian-like elimination tree in which the root of any sub-interval $[l, r]$ is the last crystal $k \in [l, r]$ removed. Because no crystal outside $[l, r]$ is removed before $[l, r]$ is emptied in that subtree, the immediate neighbors of $k$ at the moment of its removal are inductively $l - 1$ and $r + 1$. Conversely, any choice of $k \in [l, r]$ concatenated after optimal elimination orders of $[l, k - 1]$ and $[k + 1, r]$ yields a legal elimination order of $[l, r]$.

**Complexity:** $\binom{n}{2}$ intervals $\times O(n)$ choices of $k \Longrightarrow O(n^3)$ time ( $\approx 1.06 \cdot 10^7$ loop iterations for $n = 400$, running in $< 15$ ms) and $O(n^2)$ space.

---

## Worked Problem 25.2: Chromatic Packet Compression (Endpoint-Matching Interval DP)

**Problem & Constraints.** A network buffer holds $n$ consecutive packets ($1 \leq n \leq 500$), where packet $i$ has protocol color $c_i \in \{1, ..., n\}$. In one burst operation, you may erase any contiguous subsegment of packets that **all share the same color** (say, $m$ identical-color packets in a row). When you erase that monochromatic run, the remaining left and right packets slide together and concatenate. Notice that the number of packets $m$ in the erased run does **not** affect the cost: every burst operation costs exactly $1$ cycle! Find the minimum number of burst operations required to erase the entire buffer $1..n$.

> **Pause & Discover:** Suppose $c_l = c_k$ for some $k \in (l, r]$. If we erase the intervening interval $[l + 1, k - 1]$ first, packet $l$ and packet $k$ become directly adjacent and share the same color! Why does this mean deleting $l$ together with $k$ gives **one** operation FREE of charge?

### Step 1 — Exact Model & Bottleneck

An exact state in the operational game is the current shrunken string of packets after some sequence of burst deletions. Because bursts can delete arbitrary middle runs and bring distant identical colors together, the number of distinct subsequences reachable can be exponential.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Greedily erasing the longest monochromatic run or most frequent color.** Consider $c = [1, 2, 3, 2, 1]$. - Every adjacent run has length $1$. However, if we erase $3$ (1 burst), the array becomes $[1, 2, 2, 1]$; then erase $[2, 2]$ (1 burst) to get $[1, 1]$; then erase $[1, 1]$ (1 burst) for $3$ bursts total. - What about $c = [1, 2, 1, 2, 1]$? If we erase the middle $1$ first, we get $[1, 2, 2, 1] \to [1, 1] \to []$ in $3$ bursts. Or erase the two $2$s separately (2 bursts) leaving $[1, 1, 1]$, which erases in $1$ burst—also $3$ bursts!

**Mistake 2: Requiring $dp[l][r][k]$ (3D state with streak count) when bursts cost a flat $1$.** Wait—when does an interval DP require the 3D state $dp[l][r][\text{extra_count}]$ (as in the classic *Remove Boxes* problem where erasing $m$ boxes gives $m^2$ points)? Only when the objective is a **nonlinear** function of the run length $m$! When every burst costs a flat $+1$ regardless of $m$, knowing *that* $c_l$ merges with $c_k$

is completely sufficient—we do not need to count how many copies of $c_l$ merged!

### Step 3 — The Turn: Discovering the Structure

Focus on the **leftmost packet** $l$ in the subproblem $[l, r]$. In any optimal sequence of bursts that clears $[l, r]$: 1. **Case A (Packet $l$ is erased alone in its own burst):** This costs $1 + dp[l+1][r]$. 2. **Case B (Packet $l$ is erased in the same burst as some later packet $k \in [l+1, r]$ of the same color $c_l = c_k$):** For packet $l$ and packet $k$ to be erased together in a single monochromatic burst, every packet strictly between them—namely the sub-interval $[l+1, k-1]$—must be completely erased first (at cost $dp[l+1][k-1]$). Once $[l+1, k-1]$ is gone, packet $l$ sits immediately to the left of packet $k$ inside the remaining subproblem $[k, r]$. Because $c_l = c_k$, packet $l$ simply rides along for **free** inside whatever burst eventually erases packet $k$ in $[k, r]$ (at cost $dp[k][r]$)!

**Endpoint-Matching Interval DP Recurrence:**
Let $dp[l][r]$ be the minimum number of bursts required to erase the subarray $c[l..r]$.

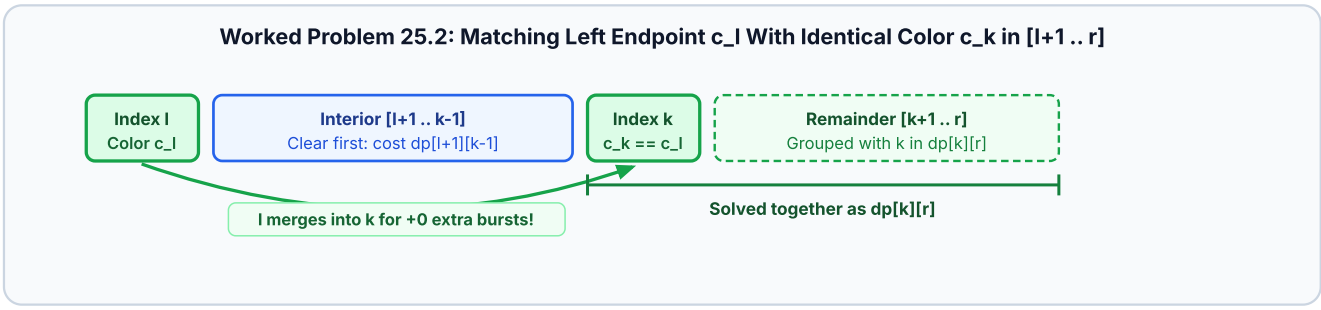
**Base Cases:** $dp[l][r] = 0$ if $l > r$, and $dp[l][l] = 1$ for all $1 \le l \le n$.
**Topological Order:** Interval length $\text{len} = r - l + 1$ from 2 up to $n$.
**Transition for $1 \le l < r \le n$:**

$$dp[l][r] = \min \left( 1 + dp[l+1][r], \min_{\substack{l+1 \le k \le r \\ c_k = c_l}} \{ dp[l+1][k-1] + dp[k][r] \} \right)$$

*(Note: When $k = l+1$ and $c_l = c_{l+1}$, $dp[l+1][l] = 0$, so this immediately yields $dp[l+1][r]$, absorbing adjacent duplicates for $0$ extra cost!)*

### Step 4 — Progressive Construction & Visual Trace

Let us trace $c = [1, 2, 1, 3, 1]$ ($n = 5$): - All length-1 intervals have $dp[i][i] = 1$. - For $[1, 3] = [1, 2, 1]$: - Erase $l = 1$ alone: $1 + dp[2][3] = 1 + 2 = 3$. - Match $l = 1$ with $k = 3$ ($c_1 = c_3 = 1$): $dp[2][2] + dp[3][3] = 1 + 1 = 2$. So $dp[1][3] = 2$. - For $[1, 5] = [1, 2, 1, 3, 1]$: - Match $l = 1$ with $k = 3$: $dp[2][2] + dp[3][5] = 1 + 2 = 3$ (since $[3..5] = [1, 3, 1]$ also clears in $2$ bursts). - Thus $dp[1][5] = 3$ bursts: erase $2$, erase $3$, and erase all three $1$s together in a single final burst!




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** In any valid burst sequence for $[l, r]$, look at the burst $B$ that erases index $l$. - If $l$ is the only index in $B$, all remaining operations partition into bursts inside $[l+1, r]$, costing at least $1 + dp[l+1][r]$. - Otherwise, let $k \in [l+1, r]$ be the *next* index (immediately to the right of $l$) erased in that same burst $B$. Then $c_l = c_k$, and every index in $[l+1, k-1]$ must be erased strictly before burst $B$ occurs, while no burst can cross the active boundary $k$ before $B$. Hence the operations inside $[l+1, k-1]$ and the operations on $\{l\} \cup [k, r]$ are disjoint, and since $l$ and $k$ are adjacent and identical when $B$ fires, erasing $\{l\} \cup [k, r]$ has the exact same minimum cost as erasing $[k, r]$.

**Complexity:** $O(n^3)$ time and $O(n^2)$ space. (Moreover, pre-linking identical colors via `next_same[l]` skips non-matching $k$ values, making the inner loop run in $O(1)$ average time on large alphabets!)

---

# Section 26 — Tree DP Basics

## The Discovery & Modeling Approach

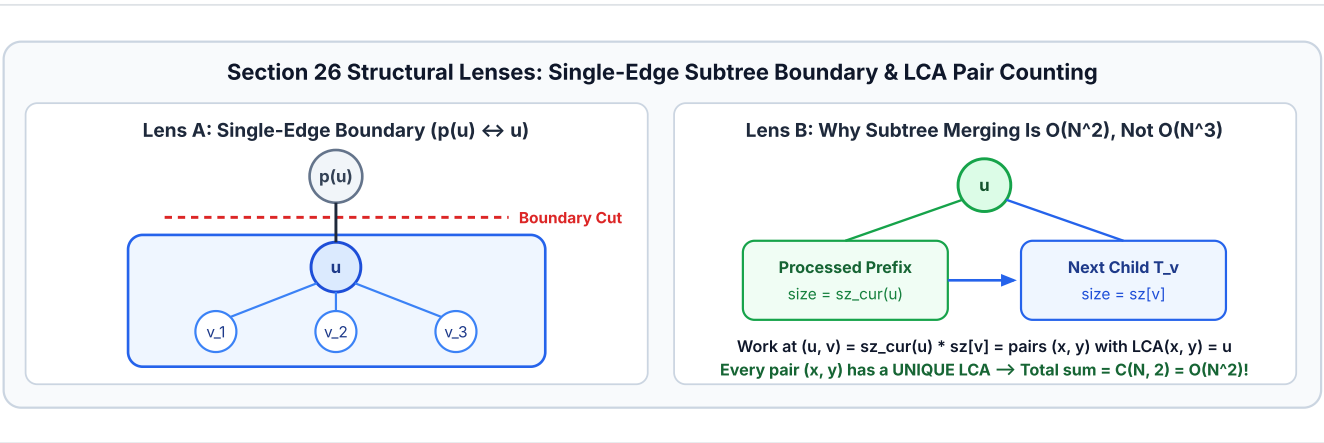
Why are trees the natural habitat of dynamic programming on graphs? In a general graph, removing a vertex or an edge rarely disconnects the remaining vertices, so subproblems overlap through complex cycles. In a **tree**, rooting at an arbitrary vertex $r$ orientates every edge away from $r$, turning the tree into a hierarchy of **disjoint subtrees** $T_u$.

The **Single-Edge Boundary Principle** of Tree DP states: > For any node $u$, the entire subtree $T_u$ communicates with the rest of the tree $T \setminus T_u$ **exclusively across the single parent edge** $(p(u), u)$. Therefore, two partial configurations inside $T_u$ are equivalent for the rest of the tree if and only if they present the exact same **boundary status** at node $u$ across edge $(p(u), u)$.

When a node $u$ has multiple children $v_1, v_2, ..., v_d$, how do we combine their independent subtrees? 1. **Direct Additive/Multiplicative Aggregation:** If the boundary state at $u$ is small (e.g., $s \in \{0, 1, 2\}$), each child $v_i$ independently chooses its best compatible state, or we maintain a tiny running state vector over the prefix of processed children $v_1 ... v_j$. 2. **Sequential Subtree Merging (Tree Knapsack):** If the state at $u$ tracks a size or resource count $k \in [0, \text{sz}[u]]$, we fold children one by one into $u$'s DP array—just like 0/1 Knapsack—bounding each loop by the **actual current subtree sizes** $\text{sz}[u]$ and $\text{sz}[v]$. By the LCA pair-counting theorem, bounding the loops by $\text{sz}[u]$ and $\text{sz}[v]$ drops the total runtime from $O(N^3)$ (or $O(NK^2)$) down to $O(N^2)$ (or $O(NK)$)!

### Decision Compass & Structural Lenses

| Tree DP Lens | What Crosses the Boundary Edge $(p, u)$? | State at Node $u$ | Child Merging Mechanism | Complexity |
|---|---|---|---|---|
| **1. Independent Set / Vertex Cover (2-State)** | Whether node $u$ itself is selected (0 or 1) | $dp[u][0],\ dp[u][1]$ | Direct sum over children $v \in \text{ch}(u)$ | $O(N)$ time |
| **2. Radius-1 Domination (3-State)** | Whether $u$ has a tower (2), is covered by a child (1), or waits for its parent (0) | $dp[u][0],\ dp[u][1],\ dp[u][2]$ | Prefix-child fold (or sum + min positive upgrade $\Delta$ to ensure $\geq 1$ child has a tower) | $O(N)$ time |
| **3. Path-Through-LCA (Diameter / Max Path)** | Longest downward chain starting at $u$ going into $T_u$ | $\text{down}[u]$ (returned to parent) + global update at LCA $u$ | Combine top 2 child chains $\text{down}[v_1] + \text{down}[v_2]$ at $u$ | $O(N)$ time |
| **4. Tree Knapsack (Subtree Size Merging)** | Number of chosen nodes $k \in [0, \min(K, \text{sz}[u])]$ inside $T_u$ | $dp[u][k]$ | Fold child $v$ into $u$ with loops bounded by $\text{sz}[u_{\text{cur}}]$ and $\text{sz}[v]$ | $O(N^2)$ or $O(NK)$ time |



## Worked Problem 26.1: Seismic Sensor Domination (3-State Subtree DP)

**Problem & Constraints.** A subterranean tunnel network forms a tree of $n$ junctions ($1 \le n \le 2 \cdot 10^5$). Placing a seismic transmitter at junction $u$ costs $c_u > 0$ ($1 \le c_u \le 10^9$) and monitors junction $u$ itself **plus all junctions directly adjacent to** $u$ (distance $\le 1$). Every junction in the tree must be monitored by at least one transmitter. Find the minimum total cost to monitor all $n$ junctions.

> **Pause & Discover:** In Maximum Independent Set or Minimum Vertex Cover, 2 states per node (`taken` vs `not taken`) suffice. Why do 2 states fail for Vertex Domination? If node $u$ does *not* have a transmitter, what two fundamentally different ways can $u$ still be monitored?

### Step 1 — Exact Model & Bottleneck

A valid configuration is a dominating set $S \subseteq V$ such that for every $u \in V$, either $u \in S$ or some neighbor $w \in N(u)$ is in $S$. Testing all $2^n$ subsets is impossible for $n = 2 \cdot 10^5$.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Using only 2 states ($dp[u][1]$ = transmitter at $u$, $dp[u][0]$ = no transmitter at $u$).** Suppose we only distinguish whether $u$ has a transmitter. If $u$ has no transmitter ($dp[u][0]$), who covers $u$? - Maybe one of $u$'s **children** has a transmitter, so $u$ is already covered! - Or maybe *none* of $u$'s children has a transmitter, so $u$ is currently **uncovered** and relies on its **parent** $p(u)$ to place a transmitter! If we merge those two situations into a single state, we either forget to cover $u$ when $p(u)$ also has no transmitter, or we falsely force $p(u)$ to have a transmitter when $u$ was already covered by a child.

**Mistake 2: Bipartite coloring (taking the cheaper of odd depth vs even depth).** Consider a 4-node path $1 - 2 - 3 - 4$ with costs $c = [1, 100, 100, 1]$. - Odd depth $\{1, 3\}$ costs $1 + 100 = 101$; even depth $\{2, 4\}$ costs $100 + 1 = 101$. - Yet placing transmitters at both endpoints $\{1, 4\}$ covers $1, 2$ (from $1$) and $3, 4$ (from $4$) for total cost $1 + 1 = 2$!

### Step 3 — The Turn: Discovering the Structure

Root the tree at vertex $1$. For each subtree $T_u$, all strict descendants of $u$ have no edges to $T \setminus T_u$, so **every strict descendant of $u$ must already be completely monitored** in any valid partial state of $T_u$. Only node $u$ itself interacts across $(p(u), u)$, in one of **three** exact states: - **State $0$ (`WAIT_PARENT`)**: $u$ has **no** transmitter, and $u$ is **not** yet monitored by any child (though all strict descendants of $u$ *are* monitored). Thus $u$'s children must all be in State $1$ (`COVERED_BY_CHILD`), and $u$ requires $p(u)$ to be in State $2$. - **State $1$ (`COVERED_BY_CHILD`)**: $u$ has **no** transmitter, **and** $u$ is already monitored because **at least one** child $v$ of $u$ is in State $2$ (`HAS_STATION`), while all other children are in State $1$ or $2$. - **State $2$ (`HAS_STATION`)**: $u$ **has** a transmitter (costing $c_u$). Because $u$'s transmitter covers $u$ and all its children, each child $v$ is free to be in State $0$, $1$, or $2$!

**3-State Domination Tree DP Recurrence:**

For a node $u$ with children $\mathrm{ch}(u)$:

Let $M(v) = \min\left( dp[v][1],\ dp[v][2] \right)$ be the best valid cost for child $v$ when $u$ does *not* place a transmitter.

**1. State 0 ($u$ uncovered, waits for parent):**

$$dp[u][0] = \sum_{v \in \mathrm{ch}(u)} dp[v][1]$$

**2. State 1 ($u$ has no station, covered by $\ge 1$ child in State 2):**

Every child $v$ contributes at least $M(v)$, and at least one child $v^*$ must choose $dp[v^*][2]$ instead of $M(v^*)$:

$$dp[u][1] = \left( \sum_{v \in \mathrm{ch}(u)} M(v) \right) + \min_{v^* \in \mathrm{ch}(u)} \left( dp[v^*][2] - M(v^*) \right)$$

*(If $\mathrm{ch}(u) = \emptyset$, i.e. $u$ is a leaf, then $dp[u][1] = +\infty$ because a leaf has no child to cover it!)*

**3. State 2 ($u$ has a station):**

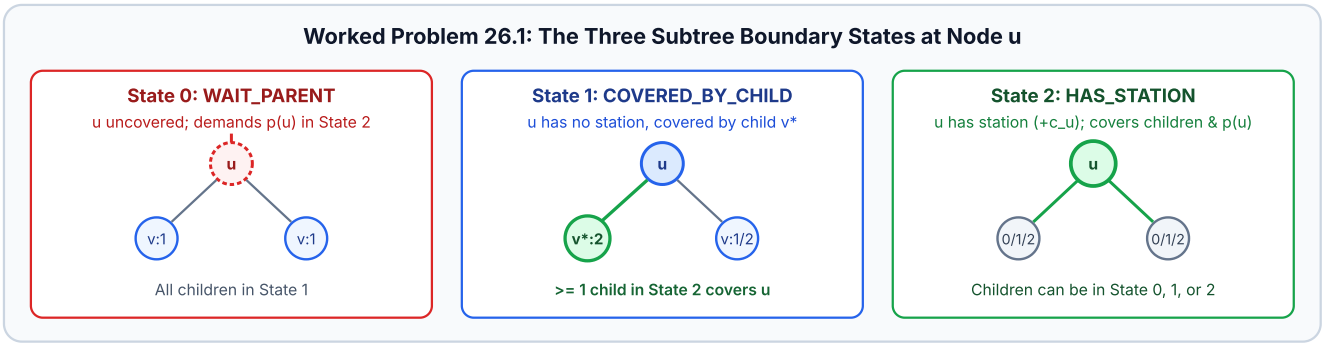
$$dp[u][2] = c_u + \sum_{v \in \mathrm{ch}(u)} \min\left( dp[v][0],\ dp[v][1],\ dp[v][2] \right)$$

**Final Answer at Root $1$:** $\min\left( dp[1][1],\ dp[1][2] \right)$ (since root $1$ has no parent to rescue State $0$).

### Step 4 — Progressive Construction & Visual Trace

Look closely at the upgrade term $\Delta = \min_{v^*}(dp[v^*][2] - M(v^*))$ in $dp[u][1]$: - If at least one child $v$ already satisfies $dp[v][2] \le dp[v][1]$, then $M(v) = dp[v][2]$, so $dp[v][2] - M(v) = 0$, and $\Delta = 0$! - If for every child $v$, $dp[v][1] < dp[v][2]$, then $M(v) = dp[v][1]$ for all children, and adding $\Delta = \min_{v^*}(dp[v^*][2] - dp[v^*][1]) > 0$ swaps the single cheapest child from State 1 to State 2!

Let us trace our path $1 - 2 - 3 - 4$ rooted at 1 with $c = [1, 100, 100, 1]$: - **Leaf** 4 **(**$c_4 = 1$**):** $dp[4] = [0,\ +\infty,\ 1]$. - **Node** 3 **(**$c_3 = 100$**, child** 4**):** - $dp[3][0] = dp[4][1] = +\infty$. - $dp[3][1] = dp[4][2] = 1$ (child 4 covers 3). - $dp[3][2] = 100 + \min(0, \infty, 1) = 100$. - $dp[3] = [+\infty,\ 1,\ 100]$. - **Node** 2 **(**$c_2 = 100$**, child** 3**):** - $dp[2][0] = dp[3][1] = 1$ (2 waits for parent 1, while 3 is covered by 4). - $dp[2][1] = dp[3][2] = 100$. - $dp[2][2] = 100 + \min(\infty, 1, 100) = 101$. - $dp[2] = [1,\ 100,\ 101]$. - **Root** 1 **(**$c_1 = 1$**, child** 2**):** - $dp[1][1] = dp[2][2] = 101$. - $dp[1][2] = 1 + \min(dp[2][0], dp[2][1], dp[2][2]) = 1 + 1 = 2$. - Answer: $\min(101, 2) = 2$.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Every dominating set $S \subseteq V$ induces at each node $u$ a unique state $s(u) \in \{0, 1, 2\}$ defined by: $s(u) = 2$ if $u \in S$; $s(u) = 1$ if $u \notin S$ and $\mathrm{ch}(u) \cap S \neq \emptyset$; and $s(u) = 0$ if $u \notin S$ and $\mathrm{ch}(u) \cap S = \emptyset$. Because $S$ dominates all vertices, any node $u$ with $s(u) = 0$ must have a parent $p(u)$ with $s(p(u)) = 2$ (in particular, the root cannot have $s(1) = 0$), and every node with $s(u) = 1$ has at least one child with $s(v) = 2$. Conversely, any state assignment obeying the recurrence transitions defines a subset $S = \{u : s(u) = 2\}$ that dominates every vertex.

**Complexity:** $O(n)$ time and $O(n)$ space.

---

## Worked Problem 26.2: Fiber-Optic Backbone Provisioning (Tree Knapsack in $O(N^2)$)

**Problem & Constraints.** A regional fiber network is a rooted tree of $N$ switching centers ($1 \le N \le 3000$) rooted at the national gateway (vertex 1). You must select a connected backbone of **exactly** $K$ **centers** ($1 \le K \le N$) that **includes the root gateway** 1 (i.e., if a center $u \neq 1$ is chosen, its parent $p(u)$ must also be chosen). - Activating center $u$ yields a revenue $R_u$ ($-10^9 \le R_u \le 10^9$). - Furthermore, each tree edge $(u, v)$ has a bandwidth installation fee $w(u, v) \ge 0$ paid if and only if both $u$ and $v$ are selected in the backbone, **multiplied by the number of selected backbone nodes in** $T_v$ (because every selected node in $T_v$ routes a unit stream across edge $(u, v)$!).

Find the maximum net profit (total revenue minus total edge routing fees) of a connected root-containing backbone of size $K$.

> **Pause & Discover:** When merging a child subtree $T_v$ into its parent $u$, why does knowing the integer count $j \in [0, \mathrm{sz}[v]]$ of selected nodes inside $T_v$ completely determine both the internal profit of $T_v$ and the exact routing fee across edge $(u, v)$? And why does bounding the knapsack loops by $\mathrm{sz}[u]$ and $\mathrm{sz}[v]$ run in $O(N^2)$ instead of $O(N^3)$?

### Step 1 — Exact Model & Bottleneck

An exact solution chooses a connected subset $S \subseteq V$ of size $K$ containing 1. Because $S$ is connected and contains the root 1, for every node $u \in S$, the intersection $S \cap T_u$ is either empty (if $u \notin S$) or is a connected root-containing subset of $T_u$ of some size $k \in [1, \mathrm{sz}[u]]$.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake: Running the inner knapsack loops up to $K$ (or $N$) at every node without bounding by $\mathrm{sz}[u]$ and $\mathrm{sz}[v]$.** Suppose you write:

```
for (int i = K; i >= 1; --i)
    for (int j = 1; j <= i; ++j)
        dp[u][i] = max(dp[u][i], dp[u][i-j] + dp[v][j] - j * w);
```

On a star graph or a long path with $N = K = 3000$, this un-bounded loop runs $N-1$ merges of cost $K^2/2$, taking $3000^3/2 \approx 1.35 \cdot 10^{10}$ operations—an immediate **Time Limit Exceeded**! Yet simply changing the loop bounds to `i <= sz[u]` (current accumulated size of $u$) and `j <= sz[v]` reduces the total number of inner-loop iterations across the **entire tree** to $\binom{N}{2} \approx 4.5 \cdot 10^6$!

### Step 3 — The Turn: Discovering the Structure

At each node $u$, let $dp[u][k]$ be the maximum net profit of selecting a connected subset of size $k$ inside $T_u$ that includes $u$ (for $1 \le k \le \mathrm{sz}[u]$), and $dp[u][0] = 0$. Initialize $u$ before processing its children with just $\{u\}$ itself:

$$\mathrm{sz}[u] = 1, \qquad dp[u][0] = 0, \qquad dp[u][1] = R_u.$$

Now, for each child $v$ of $u$, first recursively compute $dp[v][0..\mathrm{sz}[v]]$. Then merge $T_v$ into $u$'s running table using a temporary array `nxt` of size $\mathrm{sz}[u] + \mathrm{sz}[v]$: - Choosing $j = 0$ nodes from $T_v$ adds $0$. - Choosing $1 \le j \le \mathrm{sz}[v]$ nodes from $T_v$ (which is legal because $i \ge 1$ ensures $u$ is already selected!) adds $dp[v][j] - j \cdot w(u,v)$. Finally, update $\mathrm{sz}[u] \leftarrow \mathrm{sz}[u] + \mathrm{sz}[v]$.

**Tree Knapsack Recurrence & The $O(N^2)$ LCA Complexity Proof:**

For each child $v$ of $u$, merge $dp[v][1..\mathrm{sz}[v]]$ into $dp[u][1..\mathrm{sz}[u]]$:

$$\mathrm{nxt}[i+j] = \max_{1 \le i \le \mathrm{sz}[u],\ 0 \le j \le \mathrm{sz}[v]} \Big\{ dp[u][i] + dp[v][j] - j \cdot w(u,v) \Big\}$$

and then set $\mathrm{sz}[u] \leftarrow \mathrm{sz}[u] + \mathrm{sz}[v]$ and $dp[u] \leftarrow \mathrm{nxt}$.

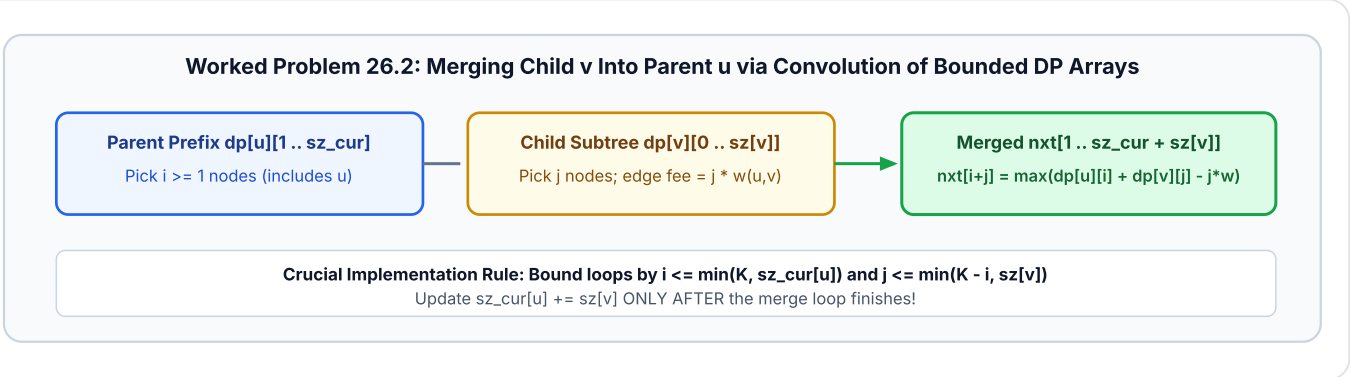
**Combinatorial Proof of $O(N^2)$ Time:**

When child $v$ is merged into the already-accumulated prefix of $u$ (which currently has size $\mathrm{sz}_{\mathrm{cur}}[u]$), the nested loops execute $\mathrm{sz}_{\mathrm{cur}}[u] \times \mathrm{sz}[v]$ iterations. That product equals the exact number of unordered pairs of vertices $(x, y)$ such that $x$ belongs to the already-processed part of $T_u$, $y$ belongs to $T_v$, and therefore $\mathrm{LCA}(x, y) = u$! Because every pair of vertices $\{x, y\} \subseteq V$ has a **unique** Lowest Common Ancestor $u$, summing over all merges in the entire tree gives:

$$\sum_{u \in V} \sum_{v \in \mathrm{ch}(u)} \mathrm{sz}_{\mathrm{cur}}[u] \cdot \mathrm{sz}[v] = \binom{N}{2} = \frac{N(N-1)}{2} = O(N^2).$$

### Step 4 — Progressive Construction & Visual Trace

Furthermore, if $K \ll N$, capping the loops at $\min(\mathrm{sz}[u], K)$ and $\min(\mathrm{sz}[v], K)$ tightens the bound even further to $O(NK)$ because only pairs $(x, y)$ whose DFS-order distance is at most $K$ can interact!




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Because any connected subset $S$ containing root $1$ has the property that $u \in S \Longrightarrow p(u) \in S$, restricting $S$ to any subtree $T_u$ yields either $\emptyset$ (if $u \notin S$) or a connected subset of $T_u$ containing $u$. Furthermore, the total routing fee across all edges is $\sum_{(u,v)} |S \cap T_v| \cdot w(u,v)$, which decomposes additively across edges $(u, v)$ as a function of $j = |S \cap T_v|$ alone.

**Small-Case Oracle:** For $N \leq 15$, iterate over all $\binom{N-1}{K-1}$ subsets of size $K$ containing vertex $1$, test connectivity via BFS, compute the exact revenue minus edge routing fees, and assert equality with $dp[1][K]$.

---

# Section 27 — Bitmask DP

## The Discovery & Modeling Approach

When $n$ is small ($n \leq 20$) and the problem requires choosing an **ordering (permutation)**, a **subset**, or a **set partition** of $n$ distinct items, brute force over all $n!$ permutations explodes around $n = 11$ ($11! \approx 4 \cdot 10^7$, while $20! \approx 2.4 \cdot 10^{18}$).

**Bitmask DP** compresses $n!$ into $2^n$ (or $n2^n$) by recognizing a fundamental symmetry: > After placing the first $k$ elements of a permutation $\pi_1, \pi_2, ..., \pi_k$, the remaining elements $\pi_{k+1}, ..., \pi_n$ usually do **not** care about the full order of $\pi_1 ... \pi_k$; they only care about **which subset of elements has been used so far** (represented as an $n$-bit integer $\text{mask} \in [0, 2^n - 1]$) and, at most, the identity of the **last placed element** $\text{last} = \pi_k$.

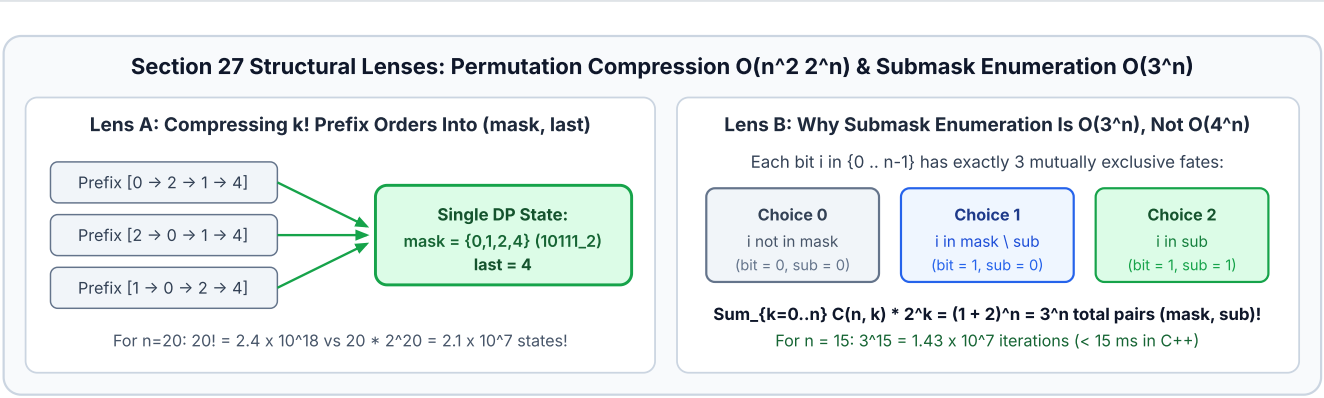
### Essential Bitwise Primitives (0-Indexed Elements $0 ... n-1$)

- Check if $i \in \text{mask}$: `(mask >> i) & 1`
- Add $i$ to $\text{mask}$: `mask | (1 << i)`
- Remove $i$ from $\text{mask}$: `mask ^ (1 << i)` (when $i \in \text{mask}$)
- Subset size $|S|$: `__builtin_popcount(mask)`
- Lowest set bit: `__builtin_ctz(mask)` (index) or `mask & -mask` (bit value)
- **Iterate over all non-empty submasks `sub` of `mask` in decreasing order:**

```
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) { ... }
```

### Decision Compass & Structural Lenses

| Bitmask DP Lens | Exact Brute-Force Space | Compressed DP State | Transition per State | Total Complexity |
|---|---|---|---|---|
| **1. Pure Subset Placement** | $n!$ permutations (cost depends only on index $k = \lvert\text{mask}\rvert$ and earlier *set* $\text{mask}$) | $dp[\text{mask}]$ | Try next item $i \notin \text{mask}$ ($n$ choices) | $O(n2^n)$ time ($n \leq 22$) |
| **2. Hamiltonian / TSP Walk** | $n!$ permutations (cost depends on adjacent pair $(\pi_{k-1}, \pi_k)$) | $dp[\text{mask}][\text{last}]$ | Try next item $nxt \notin \text{mask}$ ( $n$ choices) | $O(n^2 2^n)$ time ($n \leq 20$) |
| **3. Submask Partitioning** | Bell number $B_n$ of partitions of $\{0 ... n-1\}$ into valid groups | $dp[\text{mask}]$ | Enumerate submasks `sub = (sub - 1) & mask` | $O(3^n)$ time ($n \leq 16$) |
| **4. Sum Over Subsets (SOS DP)** | Naively $O(3^n)$ or $O(4^n)$ to aggregate $f[\text{sub}]$ over all $\text{sub} \subseteq \text{mask}$ | $dp[\text{mask}]$ updated in-place across $n$ bit dimensions | For $i = 0 ... n-1$: if bit $i$ set in $\text{mask}$, add $dp[\text{mask} \oplus 2^i]$ | $O(n2^n)$ time ($n \leq 22$) |




## Worked Problem 27.1: Quantum lithography Mask Sequencing (Permutation-to-Subset DP)

**Problem & Constraints.** A quantum chip fabrication process must apply $n$ distinct etching stencils $0, 1, ..., n-1$ in some permutation ($1 \le n \le 18$). - Applying stencil $i$ first (as the very first step) incurs a base calibration cost $B_i \ge 0$. - Whenever stencil $j$ is applied immediately after stencil $i$, the laser re-focusing cost is $T[i][j] \ge 0$. - In addition, because of chemical cross-talk on the wafer, each stencil $j$ has a **repulsion mask** of interference stencils: for every stencil $p$ that has **already been applied at any earlier point** in the sequence (not just immediately before $j$!), applying $j$ incurs an extra interference penalty $I[p][j]$ (where $I[p][j]$ can be positive or negative).

Find the minimum total cost to apply all $n$ stencils.

> **Pause & Discover:** Notice that the cost of appending stencil $j$ depends on *two* aspects of the prefix history: the immediately preceding stencil $i$ (for $T[i][j]$) and the entire set of previously applied stencils $p$ (for $\sum_{p \in \text{prefix}} I[p][j]$). Why does $(\text{mask}, \text{last})$ capture *both* dependencies simultaneously without needing to know the order of earlier elements inside $\text{mask}$?

### Step 1 — Exact Model & Bottleneck

An exact solution is a permutation $\pi = (\pi_0, \pi_1, ..., \pi_{n-1})$ of $\{0, ..., n-1\}$ with total cost:

$$B_{\pi_0} + \sum_{k=1}^{n-1} \left( T[\pi_{k-1}][\pi_k] + \sum_{r=0}^{k-1} I[\pi_r][\pi_k] \right).$$

For $n = 18$, $18! \approx 6.4 \cdot 10^{15}$ permutations—far beyond brute force.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Dropping `last` and keeping only `dp[mask]`.** Because $\sum_{p \in \text{mask}} I[p][j]$ depends only on the unordered set `mask`, it is tempting to store only `dp[mask]`. However, the transition term $T[i][j]$ depends on which specific stencil $i \in \text{mask}$ was placed **most recently**! Without `last`, we cannot evaluate $T[\text{last}][j]$.

**Mistake 2: Recomputing $\sum_{p \in \text{mask}} I[p][j]$ inside the inner transition loop.** If we loop over all $2^n$ masks, $n$ choices of `last`, $n$ choices of `nxt`, and then run another $O(n)$ loop to sum $I[p][\text{nxt}]$ for $p \in \text{mask}$, the complexity jumps to $O(n^3 2^n) \approx 18^3 \cdot 2^{18} \approx 1.5 \cdot 10^9$ operations! Instead, we can either precompute $\text{cross}[\text{mask}][j] = \sum_{p \in \text{mask}} I[p][j]$ or compute $\text{cross}[\text{mask}][j]$ once per $(\text{mask}, j)$ outside the `last` loop, keeping the total runtime strictly $O(n^2 2^n)$!

### Step 3 — The Turn: Discovering the Structure

Two partial permutations of the same subset $\text{mask}$ that end at the same element $\text{last} \in \text{mask}$ are **100% equivalent** for the remaining unplaced stencils: 1. Both leave the exact same available set $\{0, ..., n-1\} \setminus \text{mask}$. 2. For any next stencil $j \notin \text{mask}$, the immediate transition cost $T[\text{last}][j]$ is identical. 3. By commutativity of addition, the cumulative cross-talk penalty $\sum_{p \in \text{mask}} I[p][j]$ depends only on the **set** $\text{mask}$, not on the internal order in which elements of $\text{mask}$ were placed!

**Subset + Last-Element Bitmask DP Recurrence:**

Let $dp[\text{mask}][i]$ be the minimum cost to apply the subset of stencils $\text{mask} \subseteq \{0, \ldots, n-1\}$ ending with stencil $i \in \text{mask}$.

**Base Cases ($|\text{mask}| = 1$):**

$$dp[1 \ll i][i] = B_i \quad \text{for all } i \in \{0, \ldots, n-1\}$$

(and $+\infty$ for all other states).

**Transitions (for $\text{mask} = 1 \ldots 2^n - 2$):**

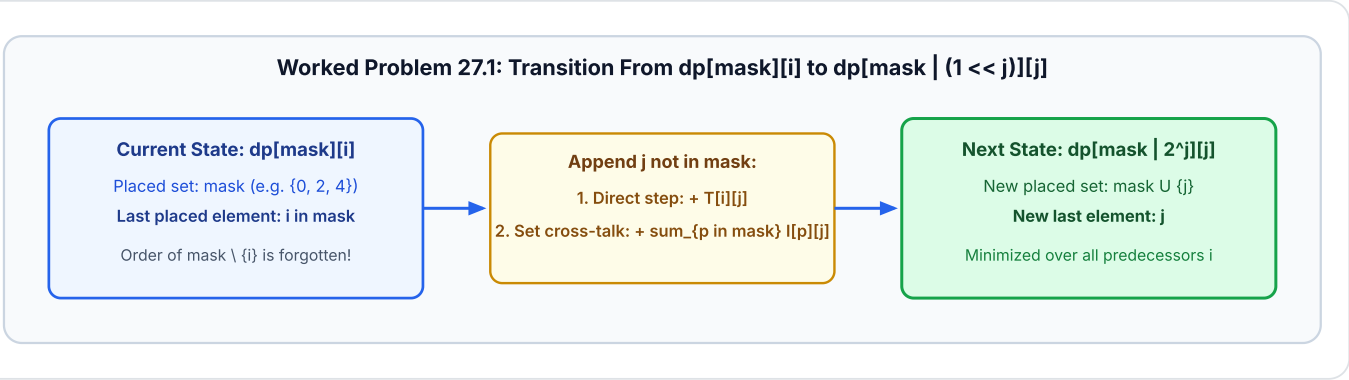
For each $j \notin \text{mask}$, let $\text{cross}(\text{mask}, j) = \sum_{p \in \text{mask}} I[p][j]$. Then for each $i \in \text{mask}$:

$$dp[\text{mask} \mid (1 \ll j)][j] \leftarrow \min\Big( dp[\text{mask} \mid (1 \ll j)][j],\ dp[\text{mask}][i] + T[i][j] + \text{cross}(\text{mask}, j) \Big)$$

**Final Answer:** $\min_{0 \le i < n} dp[(1 \ll n) - 1][i]$.

### Step 4 — Progressive Construction & Visual Trace

Why does the simple integer loop `for (int mask = 1; mask < (1 << n); ++mask)` visit states in a valid topological order? Because $\text{mask} \mid (1 \ll j)$ is strictly greater as an integer than $\text{mask}$ whenever the $j$-th bit of $\text{mask}$ is $0$! Thus standard integer order $1, 2, \ldots, 2^n - 1$ is automatically a topological sort of the subset lattice.




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Every permutation $\pi$ of length $n$ is uniquely constructed by its sequence of prefixes $\pi[0..k-1]$ for $k = 1 \ldots n$. By induction on $k = |\text{mask}|$, $dp[\text{mask}][i]$ equals the minimum cost over all $k-1!$ orderings of $\text{mask} \backslash \{i\}$ followed by $i$, because the cost of extending any such prefix by $j \notin \text{mask}$ is an additive function of $i$ and $\text{mask}$ alone.

**Complexity:** $2^n$ masks $\times n$ transitions $\times n$ predecessors $= O(n^2 2^n)$ time ( $\approx 8.5 \cdot 10^7$ simple additions for $n = 18$, or half that since $|\text{mask}|$ averages $n/2$) and $O(n 2^n)$ space.

---

## Worked Problem 27.2: Microservice Cluster Partitioning (Submask Enumeration in $O(3^n)$)

**Problem & Constraints.** You must deploy $n$ microservices ($1 \le n \le 15$) onto virtual machines (VMs). Each VM has a memory limit $M$ ( $1 \le M \le 10^9$). - Microservice $i$ requires $m_i$ memory ($1 \le m_i \le M$). - Every pair of microservices $(i, j)$ ($0 \le i < j < n$) has an inter-service communication volume $C[i][j]$ ($-10^6 \le C[i][j] \le 10^6$). - A subset of microservices $S \subseteq \{0, \ldots, n-1\}$ can be co-located on a **single VM** if and only if $\sum_{i \in S} m_i \le M$. - If $S$ is placed on a single VM, the system pays a fixed VM boot overhead $K_0 \ge 0$ **minus** all internal communication savings $\sum_{\{i,j\} \subseteq S} C[i][j]$ between pairs co-located on that VM:

$$\text{cost}(S) = K_0 - \sum_{\substack{i,j \in S \\ i<j}} C[i][j].$$

Partition $\{0, \ldots, n-1\}$ into any number of disjoint valid VMs to minimize the total cost.

**Pause & Discover:** How do we avoid overcounting partitions in different VM orders, and why does iterating over submasks `for (int sub = mask; sub > 0; sub = (sub - 1) & mask)` visit every valid pair `(mask, sub)` in $O(3^n)$ total time instead of $O(4^n)$ ?

### Step 1 — Exact Model & Bottleneck

An exact solution is a set partition $\{S_1, S_2, ..., S_k\}$ of $\{0, ..., n-1\}$ where each block $S_r$ satisfies $\sum_{i \in S_r} m_i \le M$. The number of set partitions of $n = 15$ elements is the Bell number $B_{15} = 1,382,958,545 > 1.38 \cdot 10^9$, which is too large to enumerate directly.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Iterating `sub` from `1` to `mask` and checking `(sub & mask) == sub`.** If you write:

```
for (int mask = 1; mask < (1 << n); ++mask)
    for (int sub = 1; sub <= mask; ++sub)
        if ((sub & mask) == sub) ...
```

the nested loops execute $\sum_{\text{mask}} \text{mask} \approx \frac{1}{2}(2^n)^2 = O(4^n)$ times! For $n = 15$, $4^{15}/2 \approx 5.37 \cdot 10^8$ iterations, whereas the bitwise submask trick `sub = (sub - 1) & mask` visits **only** actual submasks in $3^{15} \approx 1.43 \cdot 10^7$ iterations—a $37\times$ **speedup**!

**Mistake 2: Greedily packing the highest-saving pairs onto a VM first.** Suppose $M = 10$, $m = [5, 5, 5, 5]$, $K_0 = 0$, and $C[0][1] = 10$, $C[0][2] = 9, C[1][3] = 9$, all other $C = 0$. - Greedily pairing $\{0, 1\}$ saves $10$, leaving $\{2, 3\}$ which saves $0$ (total saving $10$). - Pairing $\{0, 2\}$ and $\{1, 3\}$ saves $9 + 9 = 18 > 10$.

### Step 3 — The Turn: Discovering the Structure

We solve the problem in two clean phases: 1. **Phase 1 — Precompute Single-VM Feasibility & Cost for All $2^n$ Subsets:** For every $\text{mask} \in [1, 2^n - 1]$, compute $\text{mem}[\text{mask}] = \sum_{i \in \text{mask}} m_i$ and $\text{vm_cost}[\text{mask}] = K_0 - \sum_{i<j \in \text{mask}} C[i][j]$ (or $+\infty$ if $\text{mem}[\text{mask}] > M$ ). Using the lowest set bit $b = \text{ctz}(\text{mask})$, this takes $O(n2^n)$ total time. 2. **Phase 2 — Submask Partition DP in $O(3^n)$:** Let $dp[\text{mask}]$ be the minimum cost to partition $\text{mask}$ into valid VMs. Every partition of a non-empty $\text{mask}$ places the lowest-indexed element $p = \text{ctz}(\text{mask})$ into **some** unique VM block `sub` $\subseteq$mask . (Pinning $p \in \text{sub}$ cuts the number of submask transitions in half and eliminates symmetrical re-orderings of VMs!)

**Submask Enumeration DP & The Binomial $3^n$ Proof:**

**Base Case:** $dp[0] = 0$.

**Transition for** $\text{mask} = 1, 2, ..., 2^n - 1$**:**

Using the hardware bitwise jump `for (int sub = mask; sub > 0; sub = (sub - 1) & mask)`:

$$dp[\text{mask}] = \min_{\substack{\emptyset \neq \text{sub} \subseteq \text{mask} \\ \text{mem}[\text{sub}] \le M}} \left\{ \text{vm_cost}[\text{sub}] + dp[\text{mask} \oplus \text{sub}] \right\}$$

**Why `sub = (sub - 1) & mask` Works:**

Subtracting $1$ from `sub` flips the lowest set bit of `sub` to $0$ and turns all lower bits to $1$. Bitwise ANDing with `mask` clears any lower bits that are not in `mask`, jumping directly to the **largest proper submask of `mask` strictly smaller than `sub`** in $O(1)$ CPU cycles!

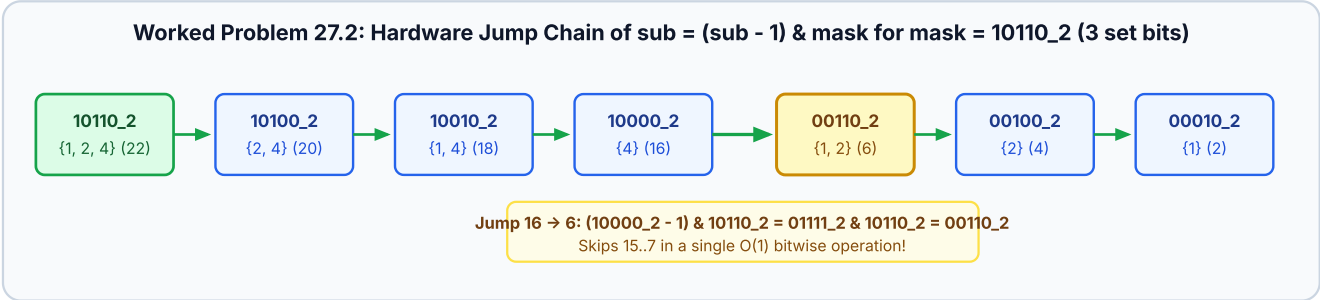
**Binomial Proof of $O(3^n)$ Total Operations:**

A mask with $k = |\text{mask}|$ set bits has $2^k$ submasks, and there are $\binom{n}{k}$ such masks:

$$\sum_{\text{mask}=0}^{2^n-1} 2^{|\text{mask}|} = \sum_{k=0}^{n} \binom{n}{k} 2^k \cdot 1^{n-k} = (2+1)^n = 3^n.$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace how `sub = (sub - 1) & mask` jumps over non-submasks when $\text{mask} = 10110_2 = 22$ (elements $\{1, 2, 4\}$): - Start: $\text{sub} = 10110_2$ (22). - Next: $(10110_2 - 1) \,\&\, 10110_2 = 10101_2 \,\&\, 10110_2 = 10100_2$ (20). - Next: $(10100_2 - 1) \,\&\, 10110_2 = 10011_2 \,\&\, 10110_2 = 10010_2$ (18). - Next: $(10010_2 - 1) \,\&\, 10110_2 = 10001_2 \,\&\, 10110_2 = 10000_2$ (16). - Next: $(10000_2 - 1) \,\&\, 10110_2 = 01111_2 \,\&\, 10110_2 = 00110_2$ (6) — **jumped straight from $16$ to $6$ in one instruction!** - Next: $00100_2$ (4) $\to 00010_2$ (2) $\to 00000_2$ (0, stop). Exactly $2^3 - 1 = 7$ iterations!




### Step 5 — Two-Way Proof & Small-Case Oracle

**Two-Way Proof:** Every valid partition of $\text{mask}$ into non-empty VM subsets $\{S_1, ..., S_k\}$ splits into one block $\text{sub} = S_1 \subseteq \text{mask}$ (with $\text{mem}[\text{sub}] \leq M$) and a valid partition $\{S_2, ..., S_k\}$ of the complement $\text{mask} \backslash \text{sub} = \text{mask} \oplus \text{sub}$. Because $\text{mask} \oplus \text{sub} < \text{mask}$ for every $\text{sub} > 0$, $dp[\text{mask} \oplus \text{sub}]$ is already optimal when $dp[\text{mask}]$ is evaluated.

**Small-Case Oracle:** For $n \leq 9$, recursively generate all set partitions of $\{0, ..., n-1\}$ (at most $B_9 = 21147$), filter out partitions where any block exceeds $M$, and assert that the minimum sum of block costs equals $dp[(1 \ll n) - 1]$.

# Section 29 — Modular Arithmetic

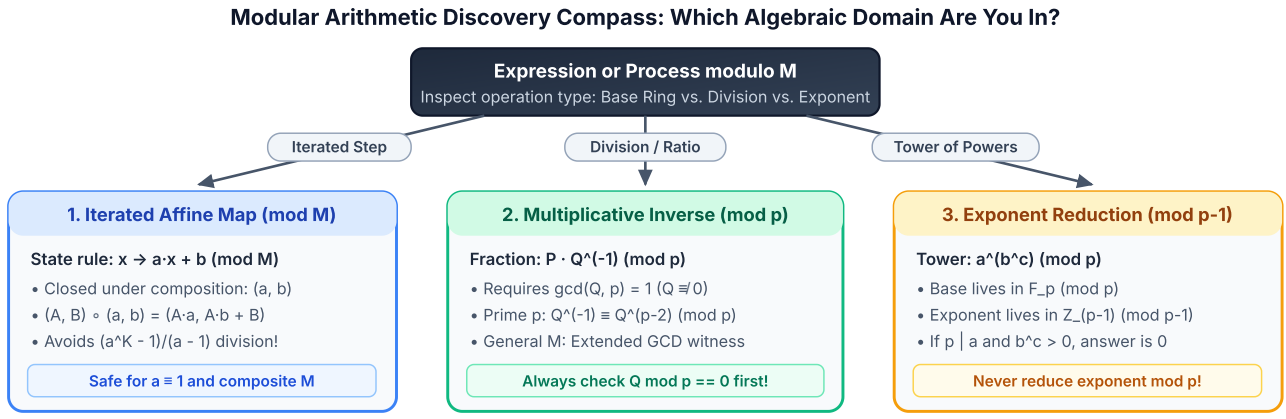
## The Discovery & Modeling Approach

When a problem asks for an answer "modulo $M$" (such as $10^9 + 7$ or $998\,244\,353$), or describes a cyclic process on a ring of size $M$, novices treat the modulus as an afterthought—a `% M` tacked onto the end of every line to prevent integer overflow. In competitive programming discovery, the modulus is not a cosmetic formatting instruction; it is an **algebraic lens** that collapses an infinite integer domain into a finite ring $\mathbb{Z}_M$ (or a finite field $\mathbb{F}_p$ when $M = p$ is prime).

Whenever a problem involves huge exponents, repeated state transitions over $K \le 10^{18}$ steps, or rational fractions reported as a single integer, your first discovery task is to translate the word problem's step-by-step evolution into **ring homomorphisms, invertible field operations, or associative transformation pairs**.

### Decision Compass & Structural Lenses

| Structural Signal in the Problem | Algebraic Lens | Core Invariant / Reduction | Critical Edge Case / Trap |
|---|---|---|---|
| Sums, differences, and products of growing counts | **Ring Homomorphism** | $(a \pm b) \bmod M$, $(a \cdot b) \bmod M$; push $\bmod M$ inside every intermediate step | Signed subtraction $(a - b) \bmod M$ yielding negative remainders in C++; always add $+M$ before `% M` |
| Division by $b$ or rational fraction $P/Q \bmod p$ | **Multiplicative Inverse** | $a \cdot b^{-1} \equiv a \cdot b^{p-2} \pmod p$ via Fermat's Little Theorem (or $\text{extgcd}$ if $\gcd(b, M) = 1$) | Dividing when $\gcd(b, M) > 1$ (in particular $b \equiv 0 \pmod p$) has **no inverse**!) |
| Iterated linear rule $x_{t+1} = ax_t + b \pmod M$ for $K \le 10^{18}$ steps | **Affine Map Composition** | Represent $f(x) = ax + b$ as pair $(a, b)$; compose $(a_2, b_2) \circ (a_1, b_1) = (a_2a_1, a_2b_1 + b_2)$ in $O(\log K)$ | Blindly applying geometric series $\frac{a^K - 1}{a - 1}$ when $a \equiv 1 \pmod p$ or $\gcd(a - 1, M) > 1$ |
| Power towers $a^{b^c} \bmod p$ or huge exponents | **Euler / Fermat Cycle** | For $a \not\equiv 0 \pmod p$, $a^E \equiv a^{E \bmod (p-1)} \pmod p$ because $a^{p-1} \equiv 1 \pmod p$ | Reducing the exponent modulo $p$ instead of $p - 1$, or failing when $p \mid a$ ($0^0 \neq 1$) |




## Worked Problem 29.1: Telemetry Packet Repeater

### Problem & Constraints

A deep-space probe starts with an initial integer register $x_0$ ($0 \le x_0 < p$). In each transmission cycle, the probe updates its register by multiplying the current value by $a$, adding an offset $b$, and reducing modulo a prime $p$:

$$x_t = (a \cdot x_{t-1} + b) \bmod p$$

Given $x_0, a, b$, a prime modulus $p$ ($2 \le p \le 10^9 + 7$), and the number of cycles $K$ ($0 \le K \le 10^{18}$), compute the final register value $x_K \bmod p$. You must answer $Q \le 10^5$ independent queries.

> **Pause & Discover:** Before reading further, write down the closed-form expression for $x_K$ after unrolling $K$ steps. What happens to that formula when $a \equiv 1 \pmod p$? Even better: can you represent one step $x \mapsto ax + b$ as an algebraic object that can be squared without using division at all?

### Step 1 — Exact Model & Bottleneck

The direct definition is a simple loop of $K$ iterations:

```
x = x0 % p
repeat K times:
    x = (a * x + b) % p
```

This exact model is obviously correct because the state $x_t \in \{0, 1, ..., p-1\}$ depends only on $x_{t-1}$. However, the bottleneck is the iteration count $K \le 10^{18}$. Simulating $10^{18}$ steps one by one takes $O(K)$ operations per query, exceeding the time limit by a factor of $10^{15}$. We need an $O(\log K)$ compression of $K$ identical steps.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Blind Geometric Series Division.** Unrolling the recurrence gives:

$$x_K = a^K x_0 + b(1 + a + a^2 + ... + a^{K-1}) \pmod p$$

It is extraordinarily tempting to replace $1 + a + ... + a^{K-1}$ with $\frac{a^K - 1}{a - 1} \cdot (a-1)^{p-2} \pmod p$.

**Counterexample:** Take $p = 7, x_0 = 2, a = 8, b = 3, K = 4$. Notice that $a = 8 \equiv 1 \pmod 7$. - In the exact model, each step adds $b = 3$: $x_4 = (2 + 4 \cdot 3) \bmod 7 = 14 \bmod 7 = 0$. - In the naive geometric formula, $a - 1 \equiv 0 \pmod 7$. Computing $0^{p-2} \bmod 7$ yields $0$, so the formula multiplies by $0$ instead of dividing by $0$, outputting $a^4 x_0 + 0 \equiv 2 \pmod 7$ — **wrong!**

**Mistake 2: Assuming Period $p$ for Everything.** When $a \equiv 1 \pmod p$, the map $x \mapsto x + b \pmod p$ has period $p$ (when $b \not\equiv 0$), whereas when $a \not\equiv 1 \pmod p$, the multiplier $a^K$ has period dividing $p - 1$. Mixing these two periods corrupts $K$.

### Step 3 — The Turn: Discovering the Structure

There are two clean ways to repair this, and knowing both is essential:

1. **Case-Split by Fixed Point ($a \equiv 1$ vs. $a \not\equiv 1 \pmod p$):**
   - If $a \equiv 1 \pmod p$, every step adds $b$, so $1 + a + ... + a^{K-1} \equiv K \pmod p$.
   - If $a \not\equiv 1 \pmod p$, then $a - 1 \not\equiv 0 \pmod p$, so $(a-1)^{-1} \bmod p$ is guaranteed to exist!
2. **Division-Free Affine Function Composition (Universal Monoid):** Instead of summing a series and dividing, view the transformation $f(x) = ax + b \pmod p$ as a pair $(a, b)$. If we apply $f_1(x) = a_1 x + b_1$ followed by $f_2(y) = a_2 y + b_2$, the combined function is:
   $$f_2(f_1(x)) = a_2(a_1 x + b_1) + b_2 = (a_2 a_1)x + (a_2 b_1 + b_2)$$
   Function composition is associative and requires **only multiplication and addition**—it works even when $a \equiv 1 \pmod p$ or when $p$ is not prime!

**Algebraic Invariant (Affine Composition & Dichotomy):**

Every affine step $f_{(a,b)}(x) = (ax + b) \bmod p$ forms a monoid under composition with identity $(1, 0)$:

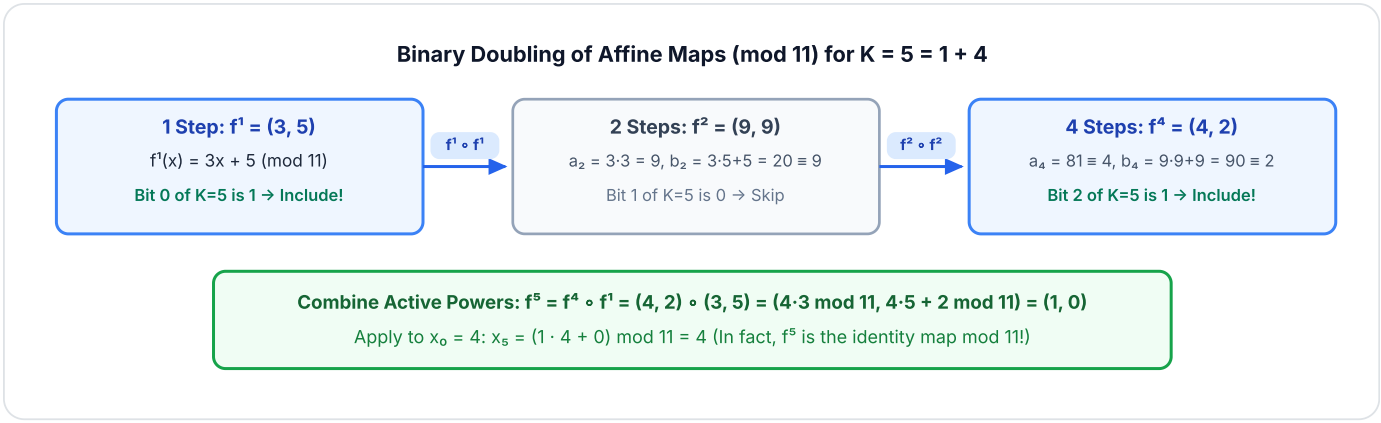
$$(a_2, b_2) \circ (a_1, b_1) = \big( (a_2 a_1) \bmod p,\ (a_2 b_1 + b_2) \bmod p \big)$$

Equivalently, in closed form over $\mathbb{F}_p$:

$$x_K \equiv \begin{cases} \big( x_0 + (K \bmod p) \cdot b \big) \bmod p & \text{if } a \equiv 1 \pmod p \\ \Big( a^K x_0 + b \cdot (a^K - 1) \cdot (a-1)^{p-2} \Big) \bmod p & \text{if } a \not\equiv 1 \pmod p \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace how binary exponentiation composes affine transformations for $p = 11$, $x_0 = 4$, $a = 3, b = 5$, and $K = 5 = (101)_2$.




```cpp
#include <iostream>
using namespace std;

struct Affine {
    long long a, b;
};

// Returns f2(f1(x)) = (f2.a * f1.a) * x + (f2.a * f1.b + f2.b) mod p
Affine compose(Affine f2, Affine f1, long long p) {
    return {
        (f2.a * f1.a) % p,
        (f2.a * f1.b + f2.b) % p
    };
}

long long solve_query(long long x0, long long a, long long b, long long K, long long p) {
    Affine step = {a % p, b % p};
    Affine acc = {1, 0}; // Identity map: f(x) = 1*x + 0
    while (K > 0) {
        if (K & 1) acc = compose(step, acc, p);
        step = compose(step, step, p);
        K >>= 1;
    }
    return (acc.a * (x0 % p) + acc.b) % p;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Sufficiency & Associativity:** By induction on $m$, the $m$-fold composition $f^m(x)$ is an affine function $(A_m x + B_m) \bmod p$. Since function composition $h \circ (g \circ f) = (h \circ g) \circ f$ is associative on any set, decomposing $K = \sum_i 2^i$ via binary exponentiation computes the exact $K$-fold iterate in $O(\log K)$ compositions without requiring multiplicative inverses.
- **Small-Case Oracle:** Test all $(a, b, x_0) \in [0, 6]^3$ for $p = 7$ and $K \in [0, 20]$ against a 20-step brute-force loop. Pay special attention to $K = 0$ (must return $x_0 \bmod p$), $a = 0$ (constant map $x_K = b$ for $K \geq 1$), and $a = 1$ (arithmetic progression $x_0 + Kb$).

---

## Worked Problem 29.2: Nested Power Beacon

### Problem & Constraints

A cryptographic beacon computes a synchronization key defined by the power tower

$$S = a^{(b^c)} \bmod p$$

where $p = 10^9 + 7$ is prime, and $a, b, c$ are non-negative integers ($0 \leq a, b, c \leq 10^{18}$). We adopt the standard combinatorial convention that $0^0 = 1$ in the exponent (so if $b = 0$ and $c = 0$, the exponent $b^c$ is $1$, and if any base is raised to the $0$-th power, the result is $1$). Compute $S$ for $Q \leq 10^5$ queries.

**Pause & Discover:** If you need $a^E \bmod p$, what modulus governs the exponent $E = b^c$? Why is $b^c \bmod p$ completely wrong? And what happens when $a$ is a multiple of $p$?

### Step 1 — Exact Model & Bottleneck

In exact integer arithmetic, we would first compute the exact integer $E = b^c$, and then compute $a^E \bmod p$ by binary exponentiation. The bottleneck is the bit-length of $E$: for $b, c = 10^{18}$, the exponent $E = b^c$ has roughly $1.8 \times 10^{19}$ digits! Even storing $E$ in memory is impossible, let alone running $O(\log E) = O(c \log b) \approx 6 \times 10^{19}$ squarings. We must compress the exponent $E$ into a bounded range before raising $a$ to it.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Reducing the Exponent Modulo $p$ instead of $p - 1$.** A classic beginner reflex is to write `E = power(b, c, p)` and then `ans = power(a, E, p)`.

**Counterexample:** Take $p = 5, a = 2, b = 5, c = 1$. - Exact value: $E = 5^1 = 5$, so $a^E = 2^5 = 32 \equiv 2 \pmod 5$. - Wrong reduction modulo $p = 5$: $E \bmod 5 = 0$, giving $2^0 \equiv 1 \pmod 5$ — **wrong!** Multiplicative order in $\mathbb{F}_5^\times$ divides $p - 1 = 4$, not $p = 5$. Indeed, $E \bmod 4 = 5 \bmod 4 = 1$, and $2^1 \equiv 2 \pmod 5$.

**Mistake 2: Applying Fermat's Little Theorem when $p \mid a$.** Suppose we fix Mistake 1 and always compute `E = power(b, c, p - 1)` and `ans = power(a % p, E, p)`.

**Counterexample:** Take $p = 5, a = 5, b = 2, c = 2$. - Exact value: $E = 2^2 = 4 > 0$, and the base is $a = 5 \equiv 0 \pmod 5$. Thus $5^4 = 625 \equiv 0 \pmod 5$. - Naive Fermat reduction: `E = 2^2 mod 4 = 0`. Then `power(0, 0, 5)` returns $0^0 = 1$ — **wrong!** Fermat's Little Theorem ($a^{p-1} \equiv 1 \pmod p$) requires $\gcd(a, p) = 1$. When $a \equiv 0 \pmod p$, the powers of $a$ are $a^0 = 1$ and $a^E = 0$ for all $E \geq 1$; they do **not** cycle back to 1 at $E = p - 1$.

### Step 3 — The Turn: Discovering the Structure

Let us separate the base $a \bmod p$ into two disjoint algebraic worlds:

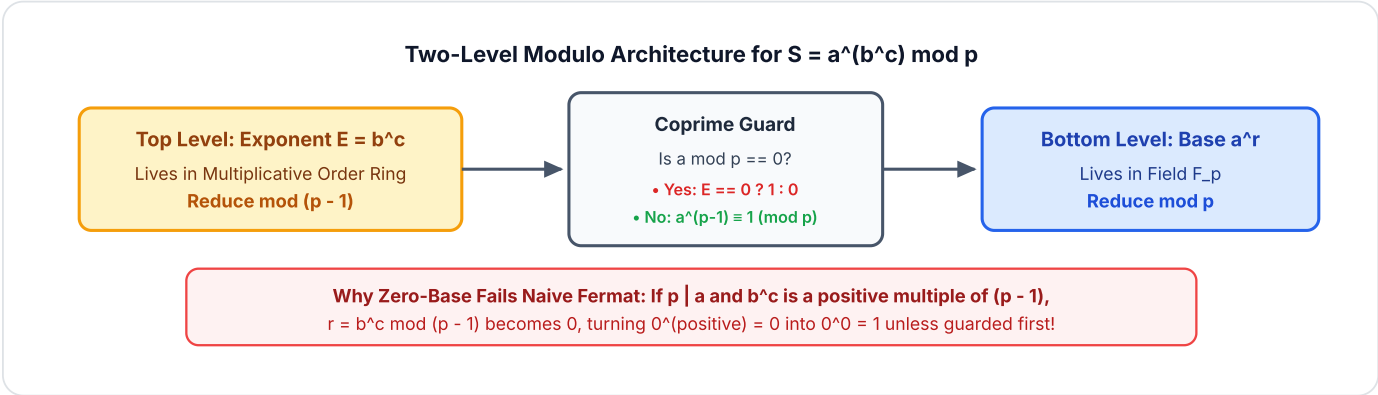
1. **Zero Base in $\mathbb{F}_p$ ($a \bmod p = 0$):** Since $0^0 = 1$ and $0^E = 0$ for every $E > 0$, we only need to know whether the true integer exponent $E = b^c$ is zero or positive! Over the non-negative integers, $b^c = 0$ if and only if $b = 0$ and $c > 0$.
   - If $b = 0$ and $c > 0$, then $E = 0$, so $a^E = a^0 = 1$.
   - Otherwise, $E = b^c \geq 1$, and since $p \mid a$, $a^E \equiv 0 \pmod p$.
2. **Invertible Base in $\mathbb{F}_p^\times$ ($a \bmod p \neq 0$):** Since $\gcd(a, p) = 1$, the multiplicative group $\mathbb{F}_p^\times$ has order $p - 1$. By Lagrange's / Fermat's theorem, $a^{p-1} \equiv 1 \pmod p$. Therefore, for any non-negative integer $E$:
$$a^E \equiv a^{E \bmod (p-1)} \pmod p$$
We can safely compute $r = b^c \bmod (p - 1)$ and then output $a^r \bmod p$.

**Fermat Exponent Tower Law:**

For prime $p$ and non-negative integers $a, b, c$:

$$a^{(b^c)} \bmod p = \begin{cases} 1 & \text{if } a \equiv 0 \pmod p \text{ and } b = 0, c > 0 \quad (\text{since } b^c = 0) \\ 0 & \text{if } a \equiv 0 \pmod p \text{ and } (b > 0 \text{ or } c = 0) \quad (\text{since } b^c \geq 1) \\ (a \bmod p)^{\,(b^c \bmod (p-1))} \bmod p & \text{if } a \not\equiv 0 \pmod p \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace




```cpp
#include <iostream>
using namespace std;

long long mod_pow(long long base, long long exp, long long mod) {
    long long res = 1 % mod;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) res = (__int128)res * base % mod;
        base = (__int128)base * base % mod;
        exp >>= 1;
    }
    return res;
}

long long solve_tower(long long a, long long b, long long c, long long p) {
    // 1. Check if the true integer exponent E = b^c is 0.
    // Since 0^0 = 1, b^c == 0 happens if and only if b == 0 and c > 0.
    if (b == 0 && c > 0) {
        return 1; // Any base a to the 0-th power is 1
    }
    // 2. Now we know E = b^c >= 1. Check if the base a is 0 mod p.
    a %= p;
    if (a == 0) {
        return 0; // 0^E = 0 for all E >= 1
    }
    // 3. Now gcd(a, p) = 1, so Fermat's Little Theorem applies: a^(p-1) = 1 mod p.
    long long exp_mod = mod_pow(b, c, p - 1);
    return mod_pow(a, exp_mod, p);
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Correctness:** Every input $(a, b, c)$ falls into either $E = 0$ (where $a^0 = 1$ unconditionally), $E \geq 1$ with $p \mid a$ (where $a^E$ is a positive power of a multiple of $p$, hence $\equiv 0 \pmod{p}$), or $E \geq 1$ with $\gcd(a, p) = 1$. In the third case, write $E = q(p-1) + r$ with $0 \leq r < p - 1$. Then $a^E = (a^{p-1})^q \cdot a^r \equiv 1^q \cdot a^r = a^r \pmod{p}$.
- **Small-Case Oracle:** Set $p = 5$ and test all $a \in [0, 10]$, $b, c \in [0, 4]$ against direct integer evaluation `pow(a, pow(b, c)) % 5`. Verify that $(a = 5, b = 2, c = 2)$ outputs $0$ and $(a = 5, b = 0, c = 2)$ outputs $1$.

---

# Section 30 — GCD, LCM, and Diophantine Equations

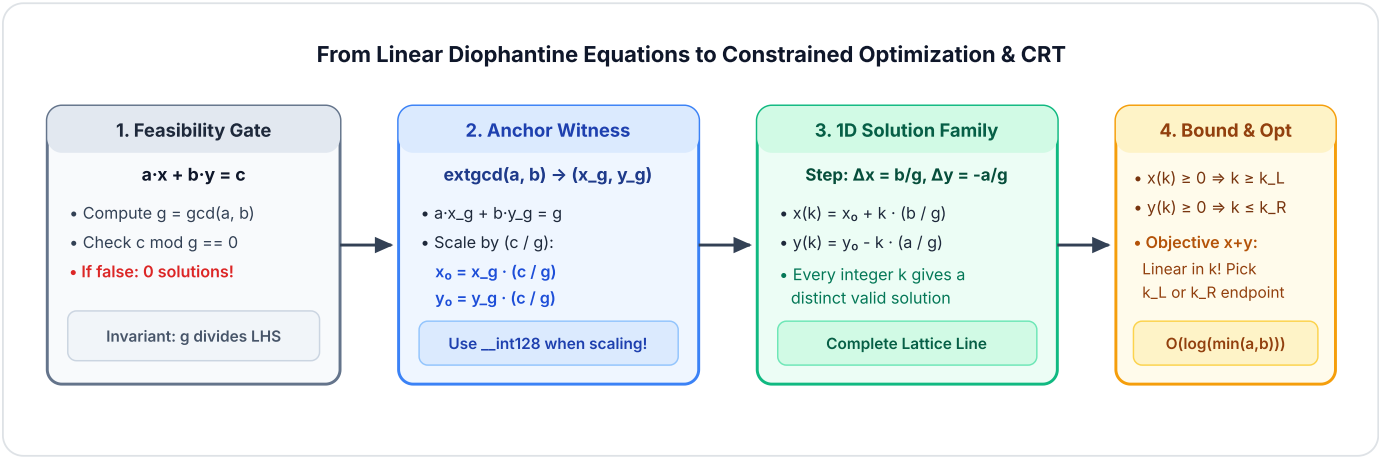
## The Discovery & Modeling Approach

Whenever a problem features discrete jumps of fixed step sizes $a$ and $b$, periodic schedules with offsets, or linear combinations $ax + by = c$, the underlying geometry is a **lattice generated by integer steps**. Two fundamental questions govern every such problem:

1. **Existence (What states are reachable?):** Any integer combination of $a$ and $b$ is a multiple of $g = \gcd(a, b)$, and by Bézout's Identity, *every* multiple of $g$ is reachable if negative multipliers are allowed.
2. **Parameterization (What is the best valid witness?):** Finding one solution $(x_0, y_0)$ via Extended Euclid is only half the battle. Real problems constrain $x, y \geq 0$ or ask to minimize $x + y$. The key invariant is that **all** integer solutions lie on a single 1D line indexed by a parameter $k \in \mathbb{Z}$, converting a 2D search into 1D interval clamping.

### Decision Compass & Structural Lenses

| Problem Structure | Number-Theoretic Lens | Algebraic Core | Key Pitfall |
|---|---|---|---|
| Reachability by adding/subtracting steps $a_1, \ldots, a_n$ | **Bézout Lattice Invariant** | Reachable values are exact multiples of $g = \gcd(a_1, \ldots, a_n)$ | Forgetting that $x, y \geq 0$ (Frobenius coin restriction) requires bounding $k$, not just $g \mid c$ |
| Optimize $x, y$ in $ax + by = c$ subject to $x \geq L_x, y \geq L_y$ | **Extended GCD + Line Parameterization** | $x = x_0 + k\frac{b}{g},\ y = y_0 - k\frac{a}{g}$; intersect bounds on $k \in [k_{\min}, k_{\max}]$ | Integer division rounding toward zero in C++ (`/`) instead of true `floor` and `ceil` on $k$ |
| Simultaneous periodic alignments $t \equiv r_i \pmod{m_i}$ | **General Chinese Remainder Theorem** | Merge pairs via $r_1 + m_1 k \equiv r_2 \pmod{m_2}$; new modulus is $\operatorname{lcm}(m_1, m_2)$ | Assuming coprime moduli; failing to check $r_1 \equiv r_2 \pmod{\gcd(m_1, m_2)}$, or overflowing $m_1 m_2$ before dividing by $\gcd$ |




---

## Worked Problem 30.1: Dual-Shuttle Cargo Dispatch

### Problem & Constraints

A lunar base must transport exactly $C$ tons of ore ($1 \leq C \leq 10^{18}$) using two types of automated shuttles: - Type-A shuttles carry exactly $a$ tons per trip ($1 \leq a \leq 10^9$). - Type-B shuttles carry exactly $b$ tons per trip ($1 \leq b \leq 10^9$).

Every shuttle launched must be completely full, and the number of trips $x \geq 0$ (for Type-A) and $y \geq 0$ (for Type-B) must be non-negative integers satisfying $ax + by = C$. Find the **minimum total number of trips** $x + y$, or report `-1` if no exact schedule exists. Answer $Q \leq 10^5$ queries.

**Pause & Discover:** Extended GCD gives you one integer solution $(x_0, y_0)$ to $ax + by = C$, but $x_0$ or $y_0$ might be negative, and it might not minimize $x + y$. How do you describe *all* integer solutions using a single integer parameter $k$, and how does $x + y$ change as $k$ increases?

### Step 1 — Exact Model & Bottleneck

In the exact model, we iterate over all possible trip counts $x \in [0, \lfloor C/a \rfloor]$ and check whether the remaining cargo $C - ax$ is divisible by $b$. Among all valid $x$, we minimize $x + (C - ax)/b$. The bottleneck is $\lfloor C/a \rfloor$: when $C = 10^{18}$ and $a = 1$, there are $10^{18}$ candidate values for $x$. Scanning them takes $O(C/a)$ time, which is hopelessly slow.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Greedily Using Maximum Trips of the Larger Shuttle.** Suppose $a > b$. It is tempting to set $x = \lfloor C/a \rfloor$ and check if $b \mid (C \bmod a)$, or decrement $x$ a few times. - **Counterexample:** Even if a greedy loop steps $x$ down until $(C - ax) \bmod b = \ = 0$, the first match could be $b - 1$ steps away, costing $O(b) = 10^9$ iterations per query!

**Mistake 2: Truncating Integer Division Toward Zero When Bounding $k$.** In C++, `-7 / 3` evaluates to `-2`, not $\lfloor -7/3 \rfloor = -3$ and not $\lceil -7/3 \rceil = -2$. If we need $x_0 + k \cdot \Delta x \geq 0$, i.e., $k \geq -x_0/\Delta x$, using raw `/` when $-x_0$ is negative produces off-by-one errors that either admit negative trips ($x < 0$) or miss the only valid $k$.

### Step 3 — The Turn: Discovering the Structure

Let $g = \gcd(a, b)$. If $g \nmid C$, no integer solution exists at all; return `-1`. Otherwise, run Extended GCD on $(a, b)$ to find $(x_g, y_g)$ with $ax_g + by_g = g$. Multiply by $C/g$ (using 128-bit integers so $x_0, y_0$ do not overflow `long long`!) to get a particular solution:

$$x_0 = x_g \cdot \frac{C}{g}, \qquad y_0 = y_g \cdot \frac{C}{g}$$

Now let $\Delta x = b/g$ and $\Delta y = a/g$. Because $\gcd(\Delta x, \Delta y) = 1$, every integer solution to $ax + by = C$ has the exact form:

$$x(k) = x_0 + k \cdot \Delta x, \qquad y(k) = y_0 - k \cdot \Delta y \qquad (k \in \mathbb{Z})$$

Imposing the non-negativity constraints $x(k) \geq 0$ and $y(k) \geq 0$ isolates $k$ to a closed integer interval $[k_{\min}, k_{\max}]$:

$$x_0 + k\Delta x \geq 0 \Leftrightarrow k \geq \left\lceil \frac{-x_0}{\Delta x} \right\rceil = k_{\min}, \qquad y_0 - k\Delta y \geq 0 \Leftrightarrow k \leq \left\lfloor \frac{y_0}{\Delta y} \right\rfloor = k_{\max}$$

If $k_{\min} > k_{\max}$, no non-negative solution exists! Otherwise, examine the objective function:

$$x(k) + y(k) = (x_0 + y_0) + k \cdot (\Delta x - \Delta y) = (x_0 + y_0) + k \cdot \frac{b - a}{g}$$

This is a **linear function of** $k$! - If $a < b$, then $\Delta x - \Delta y > 0$, so $x(k) + y(k)$ increases with $k$; we minimize it at $k = k_{\min}$. - If $a > b$, then $\Delta x - \Delta y < 0$, so $x(k) + y(k)$ decreases with $k$; we minimize it at $k = k_{\max}$. - If $a = b$, every $k \in [k_{\min}, k_{\max}]$ gives the exact same sum.

In fact, we can simply evaluate $x(k) + y(k)$ at both endpoints $k_{\min}$ and $k_{\max}$ and take their minimum!

**Parameterized Diophantine Interval & Endpoint Optimality:**

For $ax + by = C$ with $g = \gcd(a, b) \mid C$, $\Delta x = b/g$, $\Delta y = a/g$, and initial witness $(x_0, y_0)$:

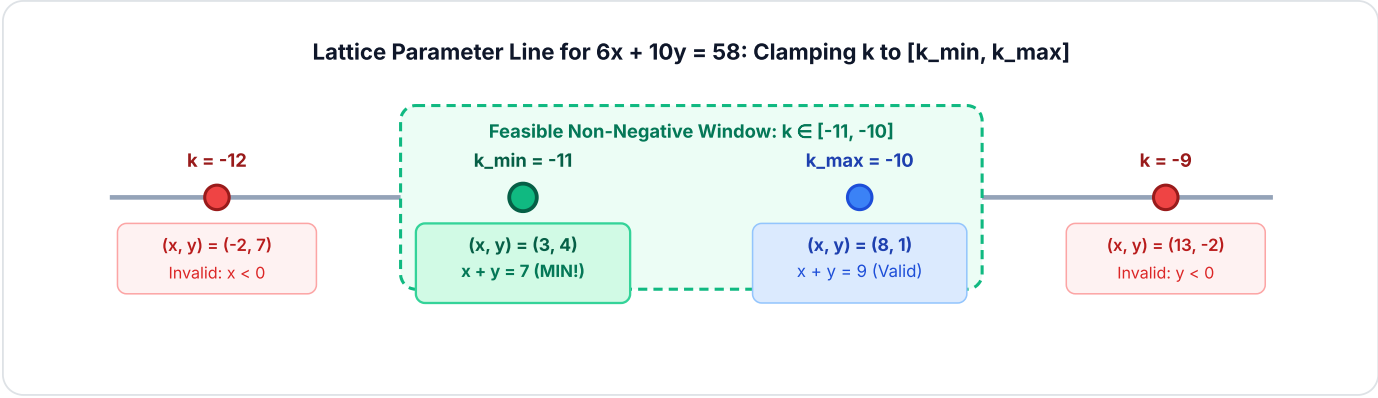
$$k \in \left[ \left\lceil \frac{-x_0}{\Delta x} \right\rceil, \ \left\lfloor \frac{y_0}{\Delta y} \right\rfloor \right] = [k_{\min}, k_{\max}]$$

Since $x(k) + y(k) = x_0 + y_0 + k(\Delta x - \Delta y)$ is monotonic in $k$:

$$\min_{x, y \geq 0} (x + y) = \min \Big( x(k_{\min}) + y(k_{\min}),\ x(k_{\max}) + y(k_{\max}) \Big)$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace $a = 6, b = 10, C = 58$. 1. $g = \gcd(6, 10) = 2$. Since $2 \mid 58$, integer solutions exist. Here $\Delta x = 10/2 = 5$ and $\Delta y = 6/2 = 3$. 2. $\text{extgcd}(6, 10)$ finds $6(2) + 10(-1) = 2$, so $(x_g, y_g) = (2, -1)$. 3. Scaling by $C/g = 29$ gives $x_0 = 58$ and $y_0 = -29$. 4. The general solution is $x(k) = 58 + 5k$ and $y(k) = -29 - 3k$. 5. Bounds on $k$: - $58 + 5k \geq 0 \Longrightarrow k \geq \lceil -58/5 \rceil = -11 = k_{\min}$. - $-29 - 3k \geq 0 \Longrightarrow 3k \leq -29 \Longrightarrow k \leq \lfloor -29/3 \rfloor = -10 = k_{\max}$. 6. Valid parameters are $k \in \{-11, -10\}$: - At $k = -11$: $(x, y) = (3, 4)$, total trips $x + y = 7$. - At $k = -10$: $(x, y) = (8, 1)$, total trips $x + y = 9$. Minimum trips is 7 (at $k_{\min} = -11$, since $a < b$ favors larger $y$ and smaller $x$).




```cpp
#include <iostream>
#include <algorithm>
using namespace std;

using i128 = __int128_t;

long long extgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}

// Exact floor(num / den) for den > 0
i128 floor_div(i128 num, i128 den) {
    i128 q = num / den;
    i128 r = num % den;
    if (r < 0) q--;
    return q;
}

// Exact ceil(num / den) for den > 0
i128 ceil_div(i128 num, i128 den) {
    i128 q = num / den;
    i128 r = num % den;
    if (r > 0) q++;
    return q;
}

long long solve_shuttles(long long a, long long b, long long C) {
    long long xg, yg;
    long long g = extgcd(a, b, xg, yg);
    if (C % g != 0) return -1;

    i128 scale = C / g;
    i128 x0 = (i128)xg * scale;
    i128 y0 = (i128)yg * scale;

    i128 dx = b / g;
    i128 dy = a / g;

    i128 k_min = ceil_div(-x0, dx);
    i128 k_max = floor_div(y0, dy);

    if (k_min > k_max) return -1;
```

```
    auto eval_sum = [&](i128 k) -> long long {
        i128 x = x0 + k * dx;
        i128 y = y0 - k * dy;
        return (long long)(x + y);
    };

    return min(eval_sum(k_min), eval_sum(k_max));
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof:** If $(x, y)$ satisfies $ax + by = C$, then $a(x - x_0) = -b(y - y_0)$, so $\Delta y(x - x_0) = -\Delta x(y - y_0)$. Because $\gcd(\Delta x, \Delta y) = 1$, Euclid's lemma forces $\Delta x \mid (x - x_0)$, so $x = x_0 + k\Delta x$ and $y = y_0 - k\Delta y$ for a unique integer $k$. Thus $[k_{\min}, k_{\max}]$ bijects onto all non-negative integer solutions, and convexity/linearity of $x(k) + y(k)$ guarantees an endpoint attains the minimum.
- **Small-Case Oracle:** For all $a, b \in [1, 30]$ and $C \in [1, 200]$, compare `solve_shuttles(a, b, C)` against a brute-force loop over $x \in [0, C/a]$. Verify cases where $g \mid C$ yet $k_{\min} > k_{\max}$ (e.g., $a = 6, b = 10, C = 14$, which has $2 \mid 14$ but no non-negative solution).

---

## Worked Problem 30.2: Multi-Frequency Lighthouse Sync

### Problem & Constraints

You monitor $N$ coastal lighthouses ($1 \le N \le 10^5$). The $i$-th lighthouse flashes with period $m_i$ seconds and produces its first flash at $r_i$ seconds ($0 \le r_i < m_i$, $1 \le m_i \le 10^9$). That is, lighthouse $i$ flashes at all non-negative integer times $t$ satisfying:

$$t \equiv r_i \pmod{m_i}$$

You are guaranteed that the least common multiple $\operatorname{lcm}(m_1, m_2, \ldots, m_N) \le 10^{18}$. Find the **earliest non-negative integer time** $t \ge 0$ at which all $N$ lighthouses flash simultaneously, or output `-1` if they never synchronize.

> **Pause & Discover:** The moduli $m_i$ are *not* necessarily coprime! If Lighthouse 1 flashes at $t \equiv r_1 \pmod{m_1}$ and Lighthouse 2 flashes at $t \equiv r_2 \pmod{m_2}$, what condition on $\gcd(m_1, m_2)$ determines whether they ever flash together, and how can you merge those two lighthouses into a single equivalent lighthouse?

### Step 1 — Exact Model & Bottleneck

An exact simulation checks $t = r_1, r_1 + m_1, r_1 + 2m_1, \ldots$ up to $L = \operatorname{lcm}(m_1, \ldots, m_N)$, testing at each step whether $t \bmod m_i = \ = r_i$ for all $i$. The bottleneck is $L/m_1$: since $L \le 10^{18}$ and $m_1$ can be as small as $2$, stepping by $m_1$ takes up to $5 \times 10^{17}$ iterations. Instead of stepping by $m_1$, we must **merge** the congruences one by one so the step size grows to $\operatorname{lcm}(m_1, \ldots, m_i)$ after incorporating the $i$-th lighthouse.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Using the Coprime CRT Formula.** The textbook coprime Chinese Remainder Theorem computes $M = \prod m_i$ and uses modular inverses $M_i^{-1} \bmod m_i$. - **Counterexample:** Take $m_1 = 6, r_1 = 2$ and $m_2 = 10, r_2 = 6$. Since $\gcd(6, 10) = 2 > 1$, $\operatorname{inv}(6, 10)$ does not exist! Yet $t = 26$ satisfies $26 \equiv 2 \pmod 6$ and $26 \equiv 6 \pmod{10}$.

**Mistake 2: Missing the GCD Parity/Remainder Conflict.** Conversely, take $m_1 = 6, r_1 = 2$ and $m_2 = 10, r_2 = 5$. Any $t \equiv 2 \pmod 6$ is **even**, while any $t \equiv 5 \pmod{10}$ is **odd**. No integer can be both even and odd! More generally, $t \bmod \gcd(m_1, m_2)$ is forced to equal both $r_1 \bmod g$ and $r_2 \bmod g$.

### Step 3 — The Turn: Discovering the Structure

Suppose we have already merged the first $i - 1$ lighthouses into a single congruence:

$$t \equiv R \pmod{M}$$

and we now wish to intersect this with the $i$-th lighthouse $t \equiv r \pmod m$. Every solution to the first congruence has the form $t = R + k \cdot M$ for some integer $k$. Substituting this into the second congruence requires:

$$R + k \cdot M \equiv r \pmod{m} \Leftrightarrow M \cdot k + m \cdot y = r - R$$

This is a linear Diophantine equation in $(k, y)$! Let $g = \gcd(M, m)$. 1. **Conflict Check:** By Bézout's lemma, a solution exists if and only if $(r - R) \equiv 0 \pmod{g}$. If not, the two schedules are permanently out of phase; return `-1`. 2. **Merge Step:** Divide the congruence through by $g$:

$$\frac{M}{g} \cdot k \equiv \frac{r - R}{g} \pmod{\frac{m}{g}}$$

Since $\gcd(M/g, m/g) = 1$, $M/g$ has a unique modular inverse modulo $m' = m/g$, which Extended GCD finds in $O(\log m)$ time! Thus $k \bmod m'$ is uniquely determined:

$$k_0 = \left(\frac{r - R}{g} \cdot \left(\frac{M}{g}\right)^{-1}\right) \bmod \frac{m}{g}$$

3. **Combined Lighthouse:** Substituting $k = k_0 + q \cdot (m/g)$ back into $t = R + kM$ gives:

$$t \equiv R + k_0 M \pmod{\operatorname{lcm}(M, m)}$$

where $L = \operatorname{lcm}(M, m) = (M/g) \cdot m$. Two lighthouses have collapsed into one!

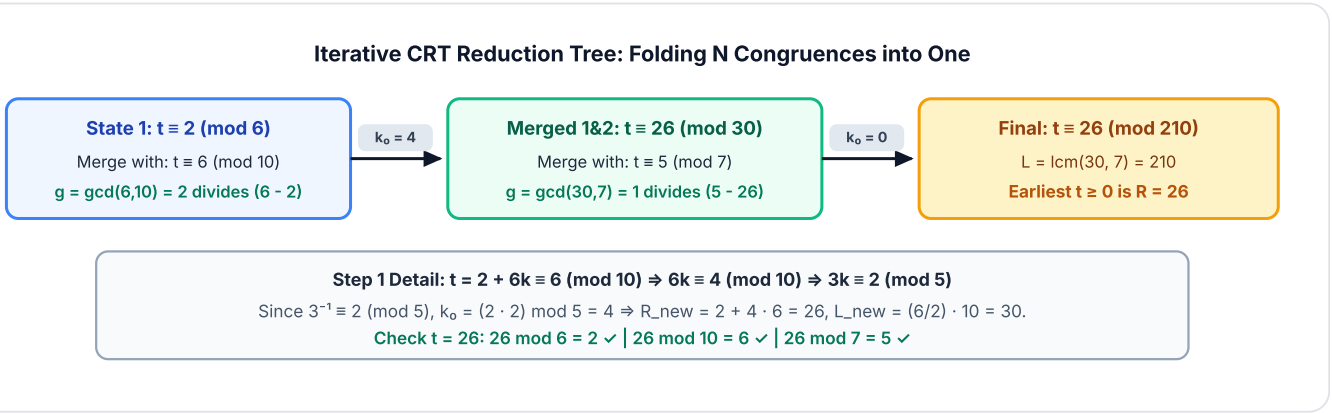
**General Non-Coprime CRT Merge Rule:**

The system $t \equiv R \pmod{M}$ and $t \equiv r \pmod{m}$ with $g = \gcd(M, m)$ is consistent if and only if $r \equiv R \pmod{g}$. When consistent, let $x_g$ satisfy $Mx_g + my_g = g$. Then the merged congruence is:

$$L = \frac{M}{g} \cdot m, \qquad k_0 = \left(\frac{r - R}{g} \cdot x_g\right) \bmod \frac{m}{g}, \qquad R_{\text{new}} = (R + k_0 \cdot M) \bmod L$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace merging $(r_1 = 2, m_1 = 6)$ and $(r_2 = 6, m_2 = 10)$, then $(r_3 = 5, m_3 = 7)$:




```cpp
#include <iostream>
#include <vector>
using namespace std;

using i128 = __int128_t;

long long extgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return g;
}

long long solve_crt(const vector<long long>& r, const vector<long long>& m) {
    long long R = 0, M = 1; // Identity congruence: t = 0 (mod 1)
    int n = r.size();
```

```
    for (int i = 0; i < n; ++i) {
        long long xg, yg;
        long long g = extgcd(M, m[i], xg, yg);
        long long diff = r[i] - R;
        if (diff % g != 0) {
            return -1; // Parity / GCD conflict: impossible to synchronize
        }
        long long mod_step = m[i] / g;
        // Compute k0 = ((diff / g) * xg) mod (m[i] / g) safely in 128-bit
        i128 k0 = ((i128)(diff / g) % mod_step * (xg % mod_step)) % mod_step;
        if (k0 < 0) k0 += mod_step;

        long long next_M = (M / g) * m[i]; // Always divide by g BEFORE multiplying!
        R = (long long)((R + k0 * M) % next_M);
        M = next_M;
    }
    return R;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Equivalence:** At each step $i$, $t \equiv R \pmod{M}$ and $t \equiv r_i \pmod{m_i}$ hold if and only if $t = R + kM$ with $Mk \equiv r_i - R \pmod{m_i}$. Dividing by $g = \gcd(M, m_i)$ yields a linear congruence with coprime coefficient $M/g$ and modulus $m_i/g$, which has a unique solution $k \equiv k_0 \pmod{m_i/g}$. Substituting $k = k_0 + q(m_i/g)$ gives $t = (R + k_0 M) + q\mathrm{lcm}(M, m_i)$, proving that the set of common flashes is an exact congruence class modulo $\mathrm{lcm}(M, m_i)$.
- **Small-Case Oracle:** Generate all triples of periods $m_1, m_2, m_3 \in [1, 12]$ and offsets $0 \le r_i < m_i$. Compare `solve_crt` against a brute-force scan of $t = 0, 1, \ldots, \mathrm{lcm}(m_1, m_2, m_3) - 1$.

---

# Section 31 — Sieve, Primes, and Factorization

## The Discovery & Modeling Approach

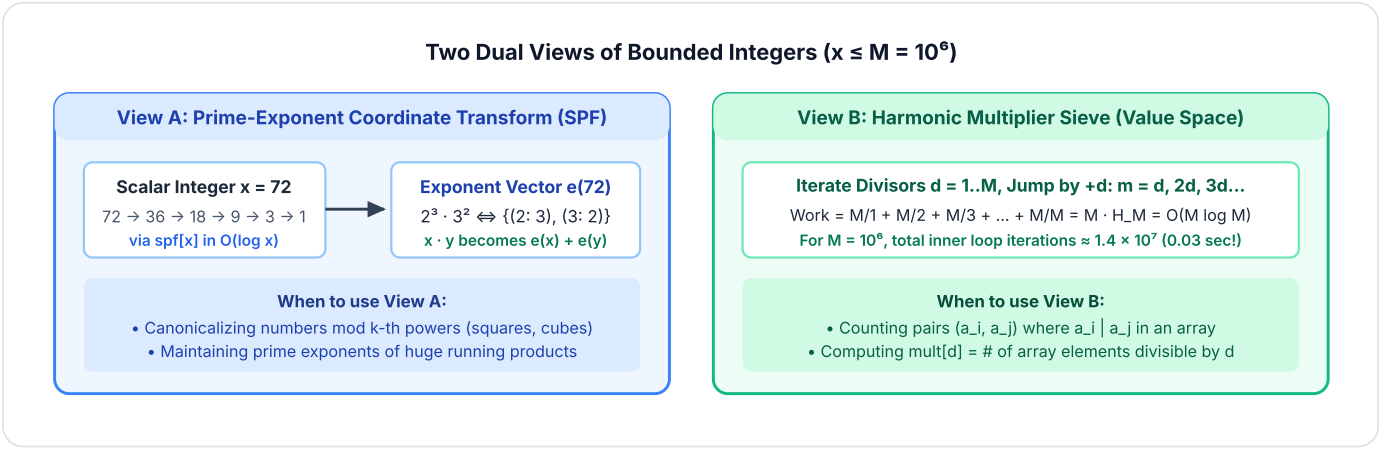
To a novice, an integer $x \le 10^6$ is a scalar point on the number line. In multiplicative number theory, that view is a trap: addition and multiplication do not mix easily on a line, and multiplying many integers quickly overflows 64-bit types.

The central discovery shift in Section 31 is the **Fundamental Theorem of Arithmetic as a Coordinate Transform**: every positive integer $x = \prod_p p^{e_p(x)}$ is uniquely represented by its infinite sparse vector of prime exponents $\mathbf{e}(x) = (e_2, e_3, e_5, \ldots)$. Under this coordinate transform: - **Multiplication** $x \cdot y$ becomes **vector addition** $\mathbf{e}(x) + \mathbf{e}(y)$. - **Divisibility** $d \mid x$ becomes **componentwise inequality** $\mathbf{e}(d) \le \mathbf{e}(x)$. - **GCD and LCM** become **componentwise** $\min$ **and** $\max$. - $k$**-th Powers** become **zero vectors modulo** $k$: $\mathbf{e}(x) \equiv \mathbf{0} \pmod k$.

Furthermore, when a problem asks to aggregate over all pairs $(a_i, a_j)$ or all divisors up to $M \le 10^6$, instead of factoring individual queries, you can invert the loop order and step through **multiples** $d, 2d, 3d, \ldots \le M$ in value space—the **Harmonic Multiplier Sieve**—which runs in $O(M \log M)$ total operations.

### Decision Compass & Structural Lenses

| Problem Signal | Multiplicative Lens | Core Technique & Complexity | Classic Pitfall |
|---|---|---|---|
| Factor $Q \le 10^6$ numbers bounded by $M \le 10^7$ | **Smallest Prime Factor (SPF) Sieve** | Precompute `spf[x]` in $O(M)$; peel `x /= spf[x]` in $O(\log x)$ per query | Running $O(\sqrt{x})$ trial division per query ($10^6 \times 3162 \approx 3 \times 10^9$ ops = TLE) |
| Pairs $(a_i, a_j)$ whose product $a_i a_j$ is a square or $k$-th power | **Exponent Vector Space** $\pmod k$ | Reduce each exponent $e_p \bmod k$; match canonical core with complementary signature $(k - e_p) \bmod k$ | Multiplying $a_i \cdot a_j$ directly (overflows for $k \ge 3$) or forgetting primes with $e_p \equiv 0 \pmod k$ |
| Count divisibility relations $a_i \mid a_j$ or $\gcd$ aggregates for $a_i \le M$ | **Harmonic Multiplier Sieve** | Count frequencies `cnt[v]`; for $d = 1 \ldots M$, loop $m = d, 2d, 3d \ldots \le M$ in $O(M \log M)$ | Testing all $O(N^2)$ pairs or factoring every element and enumerating all its divisors |
| Primes in a high window $[L, R]$ with $R \le 10^{12}$, $R - L \le 10^6$ | **Segmented Sieve** | Sieve base primes $p \le \sqrt{R}$; mark multiples in $[L, R]$ starting at $\max(p^2, \lceil L/p \rceil p)$ | Starting markings at $\lceil L/p \rceil p$ when $\lceil L/p \rceil = 1$, which falsely crosses out prime $p$ itself! |



## Worked Problem 31.1: Cube-Resonant Inventory

### Problem & Constraints

A quantum warehouse holds $N$ canisters ($1 \le N \le 2 \times 10^5$), where canister $i$ has resonance rating $a_i$ ($1 \le a_i \le 10^6$). Two canisters $(i, j)$ with $1 \le i < j \le N$ form a **stable pair** if the product of their ratings $a_i \cdot a_j$ is a **perfect $k$-th power** ($m^k$ for some positive integer $m$), where $k \ge 2$ is a given integer ($2 \le k \le 10$). Count the number of stable pairs.

> **Pause & Discover:** When is the product $a_i \cdot a_j$ a perfect $k$-th power in terms of the prime factorizations of $a_i$ and $a_j$? Once you factor $a_i$ using an SPF sieve, what single canonical signature uniquely identifies the "missing partner" $a_j$ needs?

### Step 1 — Exact Model & Bottleneck

The direct model tests all $\binom{N}{2}$ pairs $(i, j)$ and checks whether $a_i \cdot a_j$ is a $k$-th power. There are two bottlenecks: 1. **Pair count:** $\binom{N}{2} \approx 2 \times 10^{10}$ pairs for $N = 2 \times 10^5$, which cannot be enumerated in 1 second. 2. **Value overflow:** Even checking whether $x$ is a $k$-th power via `round(pow(x, 1.0/k))` suffers from floating-point inaccuracy, and if we instead needed to canonicalize $a_i$ by multiplying out its complementary prime powers $p^{k-(e_p \bmod k)}$, that complementary product can easily exceed $2^{64}$ when $k = 10$!

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Representing the Complement as a Single Multiplied Integer.** For $k = 2$ (square-free cores), reducing exponents modulo 2 produces a number $\le a_i \le 10^6$. It is tempting to generalize this to $k \ge 3$ by computing the "complement integer" $\text{comp}(a_i) = \prod p^{(k-e_p) \bmod k}$ as a `long long`. - **Counterexample:** Let $k = 10$ and $a_i = 2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 = 510\,510 \le 10^6$. Every prime has exponent 1, so the complement requires exponent 9 on each of the 7 primes:

$$\text{comp}(a_i) = (510\,510)^9 \approx 2.3 \times 10^{51}$$

which overflows both `long long` and `__int128`!

**Mistake 2: Keeping Zero-Remainder Primes in the Signature.** If $k = 3$ and $a_i = 8 = 2^3$, its exponent modulo 3 is 0. If we store `(2, 0)` in the factor list of $a_i = 8$, but store an empty list `[]` for $a_j = 1$, their signatures will fail to match even though $8 \times 1 = 8 = 2^3$ is a perfect cube!

### Step 3 — The Turn: Discovering the Structure

Write each number $x$ in prime-exponent coordinates: $x = \prod_p p^{e_p(x)}$. Then:

$$x \cdot y \text{ is a } k\text{-th power} \Leftrightarrow \forall p,\ \ e_p(x) + e_p(y) \equiv 0 \pmod k$$

Equivalently, for every prime $p$:

$$e_p(y) \bmod k = \big(k - (e_p(x) \bmod k)\big) \bmod k$$

Notice two crucial facts: 1. Only primes with $r_p = e_p(x) \bmod k \neq 0$ matter! Any prime with $e_p(x) \equiv 0 \pmod k$ must simplemente be **omitted** from the signature so that every equivalent vector has a unique canonical representation. 2. Since $a_i \le 10^6$ and $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \cdot 19 > 10^6$, any $a_i \le 10^6$ has **at most 7 distinct prime factors**! Thus we can represent both the reduced signature $\text{sig}(x)$ and its complementary signature $\text{comp}(x)$ as a tiny sorted `vector<pair<int, int>>` of length $\le 7$, completely immune to integer overflow for any $k$!

**Prime-Exponent Dual Signature Identity:**

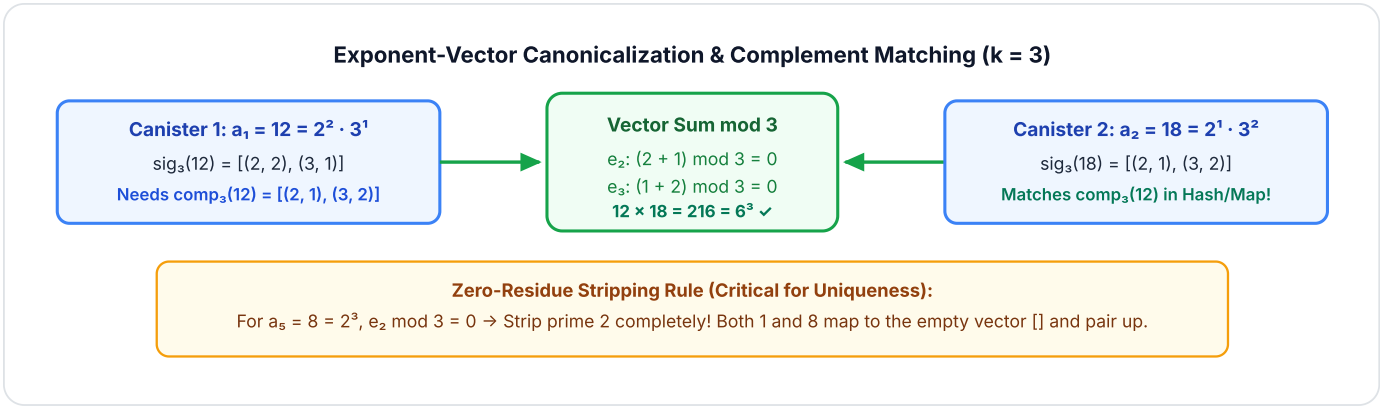
For any integer $x = \prod p^{e_p}$, define its reduced $\bmod\ k$ signature and complement signature over active primes ($r_p = e_p \bmod k \neq 0$):

$$\text{sig}_k(x) = \{\ (p,\ e_p \bmod k) \mid e_p \not\equiv 0 \pmod k\ \}$$
$$\text{comp}_k(x) = \{\ (p,\ k - (e_p \bmod k)) \mid e_p \not\equiv 0 \pmod k\ \}$$

Then $x \cdot y$ is a perfect $k$-th power if and only if $\text{sig}_k(y) = \text{comp}_k(x)$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $k = 3$ on the array $A = [12, 18, 54, 1, 8]$: - $12 = 2^2 \cdot 3^1 \Longrightarrow \text{sig}_3(12) = [(2, 2), (3, 1)]$, $\text{comp}_3(12) = [(2, 1), (3, 2)]$. - $18 = 2^1 \cdot 3^2 \Longrightarrow \text{sig}_3(18) = [(2, 1), (3, 2)]$, which matches $\text{comp}_3(12)$! Indeed, $12 \times 18 = 216 = 6^3$. - $54 = 2^1 \cdot 3^3 \Longrightarrow$ prime 3 has $e_3 = 3 \equiv 0 \pmod 3$ (dropped!), so $\text{sig}_3(54) = [(2, 1)]$. - $1 \Longrightarrow \text{sig}_3(1) = []$. - $8 = 2^3 \Longrightarrow \text{sig}_3(8) = []$, matching $\text{comp}_3(1) = []$! Indeed, $1 \times 8 = 2^3$.




```cpp
#include <iostream>
#include <vector>
#include <map>
using namespace std;

const int MAXA = 1000000;
int spf[MAXA + 1];

void precompute_spf() {
    for (int i = 2; i <= MAXA; ++i) spf[i] = 0;
    for (int i = 2; i <= MAXA; ++i) {
        if (spf[i] == 0) {
            for (int j = i; j <= MAXA; j += i) {
                if (spf[j] == 0) spf[j] = i;
            }
        }
    }
}

long long count_kth_power_pairs(const vector<int>& a, int k) {
    map<vector<pair<int, int>>, int> freq;
    long long pairs = 0;

    for (int x : a) {
        vector<pair<int, int>> sig, comp;
        int temp = x;
        while (temp > 1) {
            int p = spf[temp];
            int cnt = 0;
            while (temp % p == 0) {
                temp /= p;
                cnt++;
            }
            int r = cnt % k;
            if (r != 0) { // Only store non-zero residues mod k!
                sig.push_back({p, r});
                comp.push_back({p, k - r});
            }
        }
        auto it = freq.find(comp);
        if (it != freq.end()) {
            pairs += it->second;
        }
        freq[sig]++;
    }
    return pairs;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Bijection on Equivalence Classes:** By unique prime factorization, two vectors of prime exponents sum to $\mathbf{0} \pmod k$ if and only if for every prime $p$, $(e_p(x) \bmod k) + (e_p(y) \bmod k) \in \{0, k\}$. Removing all primes where $e_p(x) \bmod k = 0$ leaves a finite, strictly increasing sequence of primes with residues in $\{1, ..., k-1\}$, which is identical for $y$ and $\mathrm{comp}_k(x)$ if and only if $x \cdot y = m^k$.
- **Small-Case Oracle:** For $N \le 50$, $a_i \in [1, 100]$, and $k \in \{2, 3, 4\}$, compare `count_kth_power_pairs` against an exact factor-adding oracle that checks `(e_p(a[i]) + e_p(a[j])) % k == 0` for all pairs $(i, j)$.

---

## Worked Problem 31.2: Divisible Resonance Chains

### Problem & Constraints

You are given an array $A = [a_1, a_2, ..., a_N]$ of $N$ positive integers ($1 \le N \le 5 \times 10^5$) where every element satisfies $1 \le a_i \le M = 10^6$. Count the number of ordered index pairs $(i, j)$ with $i \ne j$ such that $a_i$ divides $a_j$ ($a_i \mid a_j$).

> **Pause & Discover:** Checking all $O(N^2)$ pairs is too slow, and factoring every $a_j$ to enumerate all its divisors takes up to $240$ steps per element plus hash lookups. What happens if you collapse the array into a frequency table $\mathrm{cnt}[1...M]$ and iterate over multiples $d, 2d, 3d, ...$ in value space?

### Step 1 — Exact Model & Bottleneck

The direct definition checks all $N(N-1)$ ordered pairs $(i, j)$ with $i \ne j$ and increments the answer whenever `a[j] % a[i] == 0`. The bottleneck is $N^2 = 2.5 \times 10^{11}$ checks. Notice, however, that the condition $a_i \mid a_j$ depends **only on the values** of $a_i$ and $a_j$, not on their original array indices, and the maximum value $M = 10^6$ is small!

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Running $O(\sqrt{a_j})$ Divisor Enumeration for Every Element.** If we find all divisors of each $a_j$ in $O(\sqrt{a_j})$ time, total operations are $N\sqrt{M} = 5 \times 10^5 \times 10^3 = 5 \times 10^8$, which is tight and risks TLE.

**Mistake 2: Miscounting Equal Elements ($a_i = a_j = d$).** When $m = d$ (the 1-st multiple of $d$), an element of value $d$ divides *other* elements of value $d$, but we must exclude $i = j$! - **Counterexample:** Suppose $A = [4, 4, 4]$. Every ordered pair $(i, j)$ with $i \ne j$ has $4 \mid 4$, giving $3 \times 2 = 6$ valid ordered pairs. If we naively add `cnt[d] * cnt[m]` for $m = d$, we get $3 \times 3 = 9$, counting the 3 self-pairs $(i, i)$ by mistake!

### Step 3 — The Turn: Discovering the Structure

First compress the input array into a value-space frequency array $\mathrm{cnt}[1...M]$, where $\mathrm{cnt}[v]$ is the number of indices $i$ with $a_i = v$. Now, instead of asking "given $a_j$, which divisors $d \mid a_j$ appear in the array?", invert the question: > "Given a divisor value $d \in [1, M]$ with $\mathrm{cnt}[d] > 0$, which multiples $m = d, 2d, 3d, ... \le M$ appear in the array?"

- For $m = d$ (same value), each of the $\mathrm{cnt}[d]$ elements can pair with any of the other $\mathrm{cnt}[d] - 1$ elements of value $d$, contributing $\mathrm{cnt}[d] \cdot (\mathrm{cnt}[d] - 1)$ ordered pairs.
- For $m = 2d, 3d, ... \le M$ (strict multiples), every element of value $d$ divides every element of value $m$, contributing $\mathrm{cnt}[d] \cdot \mathrm{cnt}[m]$ ordered pairs.

How fast is the nested loop `for d = 1..M: for m = d, 2d, 3d..M`? Even if we don't skip $\mathrm{cnt}[d] = = 0$, the total number of inner-loop steps is the **Harmonic Series**:

$$\sum_{d=1}^{M} \left\lfloor \frac{M}{d} \right\rfloor \le M \sum_{d=1}^{M} \frac{1}{d} = M \cdot H_M = O(M \log M)$$

For $M = 10^6$, this is only $\approx 1.4 \times 10^7$ simple array lookups!

**Harmonic Value-Space Aggregation Formula:**

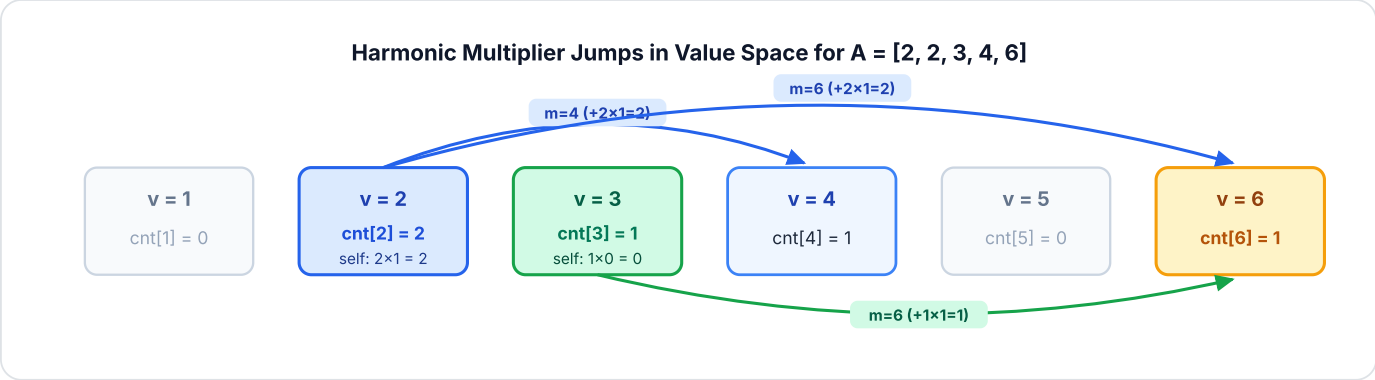
Let $\mathrm{cnt}[v]$ be the frequency of value $v$ in $A$. The number of ordered pairs $(i, j)$ with $i \neq j$ and $a_i \mid a_j$ is:

$$\text{Pairs} = \sum_{d=1}^{M} \left( \mathrm{cnt}[d](\mathrm{cnt}[d] - 1) + \sum_{k=2}^{\lfloor M/d \rfloor} \mathrm{cnt}[d] \cdot \mathrm{cnt}[k \cdot d] \right)$$

computed in $\sum_{d=1}^{M} \frac{M}{d} = O(M \log M)$ time.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $A = [2, 2, 3, 4, 6]$ with $M = 6$: - $\mathrm{cnt} = [0, 0, 2, 1, 1, 0, 1]$ (for values $0 \ldots 6$). - $d = 2$ ($\mathrm{cnt}[2] = 2$): - Equal multiple $m = 2$: adds $2 \times (2 - 1) = 2$ pairs. - Strict multiple $m = 4$: adds $2 \times \mathrm{cnt}[4] = 2 \times 1 = 2$ pairs. - Strict multiple $m = 6$: adds $2 \times \mathrm{cnt}[6] = 2 \times 1 = 2$ pairs. - $d = 3$ ($\mathrm{cnt}[3] = 1$): - Equal multiple $m = 3$: adds $1 \times 0 = 0$ pairs. - Strict multiple $m = 6$: adds $1 \times \mathrm{cnt}[6] = 1 \times 1 = 1$ pair. - $d = 4, 6$: only $1$ copy each and no higher multiples in $[1, 6]$, adding $0$. Total ordered pairs = $2 + 2 + 2 + 1 = 7$.




```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

long long count_divisible_pairs(const vector<int>& a) {
    int max_v = *max_element(a.begin(), a.end());
    vector<long long> cnt(max_v + 1, 0);
    for (int x : a) cnt[x]++;

    long long total_pairs = 0;
    for (int d = 1; d <= max_v; ++d) {
        if (cnt[d] == 0) continue;
        // 1. Ordered pairs with equal value: a[i] == a[j] == d, i != j
        total_pairs += cnt[d] * (cnt[d] - 1);
        // 2. Ordered pairs with strict multiples: m = 2d, 3d, ...
        for (int m = 2 * d; m <= max_v; m += d) {
            total_pairs += cnt[d] * cnt[m];
        }
    }
    return total_pairs;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Partition:** Every valid ordered pair $(i, j)$ with $i \neq j$ and $a_i \mid a_j$ has a unique pair of positive integer values $(d, m) = (a_i, a_j)$ where $m = k \cdot d$ for some integer $k \geq 1$. If $k = 1$, there are $\mathrm{cnt}[d](\mathrm{cnt}[d] - 1)$ index pairs; if $k \geq 2$, the index sets $\{i : a_i = d\}$ and $\{j : a_j = kd\}$ are disjoint, so there are $\mathrm{cnt}[d] \cdot \mathrm{cnt}[kd]$ index pairs. Summing over all $d \geq 1$ and $k \geq 1$ counts every valid index pair once and only once.
- **Small-Case Oracle:** Compare `count_divisible_pairs` against the $O(N^2)$ double loop on 1000 random arrays of length $N = 30$ with values in $[1, 20]$ (including arrays of all 1s and arrays with duplicates).

---

# Section 32 — Combinatorics under Modulo

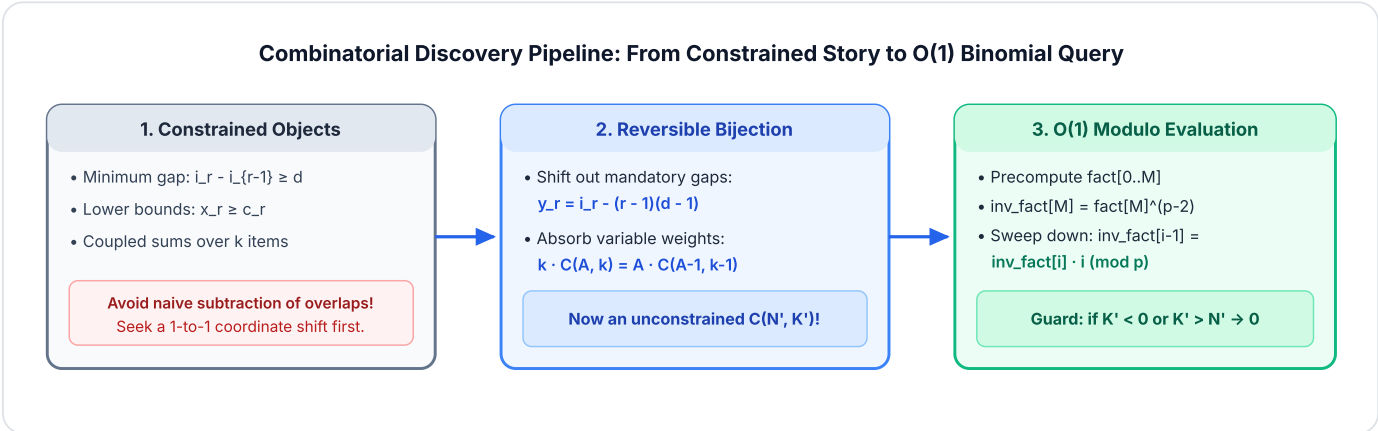
## The Discovery & Modeling Approach

Combinatorial problems in competitive programming rarely yield to raw algebraic bash first. Whenever a problem asks "how many configurations satisfy property $P$, modulo $10^9+7$?", jumping straight to factorials or DP without a **reversible structural map (a bijection)** invites double-counting.

The discovery workflow in combinatorics has three pillars: 1. **Bijective Normalization:** Transform a constrained choice (such as "no two chosen items can be within distance $d$" or "each bucket gets at least $c_i$ items") into an **unconstrained** choice of balls in bins or a standard lattice walk by shifting coordinates. 2. **Double Counting / Contribution Exchange:** When asked for the sum of a combinatorial quantity over many states (or a convolution $\sum_k \binom{A}{k}\binom{B}{S-k}$), reinterpret the sum as a single two-stage story (Vandermonde's identity) or exchange the summation order to count how many configurations each atomic element contributes to. 3. $O(N)$ **Factorial & Inverse Table Precomputation:** Precompute `fact[i]` forward and `inv_fact[i]` backward from `inv_fact[MAXN]` in $O(M)$ time so any binomial query $\binom{n}{k} \bmod p$ is answered in $O(1)$ time.

### Decision Compass & Structural Lenses

| Combinatorial Pattern | Structural Bijection / Identity | Canonical Formula | Common Trap |
|---|---|---|---|
| Distributing $N$ identical items into $K$ distinct bins ($x_i \geq 0$) | **Stars and Bars** ($N$ stars, $K-1$ dividers) | $\binom{N+K-1}{K-1}$ | Confusing $x_i \geq 0$ with $x_i \geq 1$ ($\binom{N-1}{K-1}$) |
| Choosing $K$ indices from $1 \ldots N$ with minimum gap $i_r - i_{r-1} \geq d$ | **Index Compression Shift** ($y_r = i_r - (r-1)(d-1)$) | $\binom{N-(K-1)(d-1)}{K}$ | Subtracting adjacent pairs separately instead of compressing the $(K-1)$ mandatory buffers |
| Convolution of combinations $\sum_{k=0}^{S}\binom{A}{k}\binom{B}{S-k}$ | **Vandermonde Committee Split** (choose $S$ from $A \cup B$) | $\binom{A+B}{S}$ | Evaluating the sum in $O(S)$ per query when $A, B, S$ vary across $10^5$ queries |
| Weighted subset sums $\sum_k k\binom{A}{k}\binom{B}{S-k}$ | **Leader-First Absorption** ($k\binom{A}{k} = A\binom{A-1}{k-1}$) | $A\binom{A+B-1}{S-1}$ | Trying to differentiate generating functions at runtime instead of absorbing $k$ |




## Worked Problem 32.1: Cooldown-Spaced Server Backups

### Problem & Constraints

Over a schedule of $N$ consecutive hours ($1 \leq N \leq 10^6$), you must trigger exactly $K$ server backups ($1 \leq K \leq N$) at integer hours $1 \leq t_1 < t_2 < \ldots < t_K \leq N$. Because each backup heats the storage array, any two consecutive backups must be separated by at least $d$ hours ($t_r - t_{r-1} \geq d$, where $1 \leq d \leq 10^6$). Count the number of valid backup schedules modulo $10^9+7$. Answer $Q \leq 10^5$ queries after $O(N_{\max})$ precomputation.

**Pause & Discover:** Every pair of consecutive backups $(t_{r-1}, t_r)$ wastes at least $d-1$ mandatory cooldown hours between them. If you physically delete those $(K-1)(d-1)$ mandatory cooldown hours from the timeline, what do the remaining hours look like? Can you write down the exact coordinate formula that maps $(t_1, ..., t_K)$ to an unconstrained subset?

### Step 1 — Exact Model & Bottleneck

In a dynamic programming model, `dp[i][j]` counts ways to choose $j$ backups from the first $i$ hours:

$$\mathrm{dp}[i][j] = \mathrm{dp}[i-1][j] + \mathrm{dp}[i-d][j-1]$$

This DP is exact and easy to verify, but its state space is $O(NK)$ per query! With $N, K \le 10^6$ and $Q = 10^5$, we need an $O(1)$-per-query closed form.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Requiring $K$ Cooldown Buffers Instead of $K-1$.** A very common off-by-one error is thinking each of the $K$ chosen backups must be followed by $d-1$ cooldown hours, reducing $N$ by $K(d-1)$. - **Counterexample:** Take $N=5, K=2, d=3$. The last backup $t_2$ can occur at hour $N=5$ because no backup follows it! Valid schedules are $(1,4), (1,5), (2,5)$ — a total of **3** schedules. Subtracting $K(d-1)=4$ would leave $\binom{5-4}{2} = \binom{1}{2} = 0$ — **wrong!** Only the $K-1$ internal gaps between consecutive backups require $d-1$ buffer hours.

**Mistake 2: Forgetting the Impossible-Schedule Guard.** When $(K-1)(d-1) \ge N$, the shifted upper bound $N' = N-(K-1)(d-1)$ can become smaller than $K$ or even negative! Using raw array indexing `fact[N']` on a negative $N'$ causes an immediate out-of-bounds crash (segmentation fault).

### Step 3 — The Turn: Discovering the Structure

Let us construct an explicit, reversible bijection between valid sequences $1 \le t_1 < t_2 < ... < t_K \le N$ satisfying $t_r - t_{r-1} \ge d$ and unconstrained strictly increasing sequences $1 \le y_1 < y_2 < ... < y_K \le N'$. For each $r \in \{1, 2, ..., K\}$, define:

$$y_r = t_r - (r-1)(d-1)$$

Let us check the gap between consecutive $y$'s:

$$y_r - y_{r-1} = (t_r - t_{r-1}) - (d-1) \ge d - (d-1) = 1$$

Since $t_r$ and $y_r$ are integers, $y_r - y_{r-1} \ge 1$ simply means $y_1 < y_2 < ... < y_K$! What are the boundary constraints? - Left boundary: $t_1 \ge 1 \Leftrightarrow y_1 \ge 1$. - Right boundary: $t_K \le N \Leftrightarrow y_K \le N-(K-1)(d-1) = N'$.

Conversely, given *any* $K$-element subset $1 \le y_1 < y_2 < ... < y_K \le N'$, setting $t_r = y_r + (r-1)(d-1)$ recovers a valid cooldown-spaced schedule! Thus the count is simply the number of ways to choose $K$ distinct integers from $\{1, 2, ..., N'\}$.

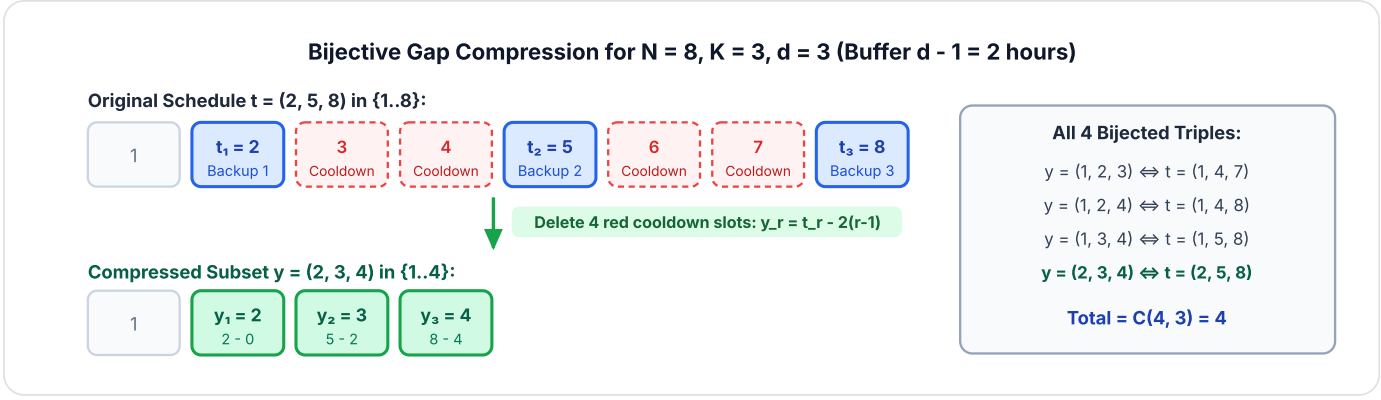
**Gap-Compression Bijection (Kaplansky's Shift):**

The map $y_r = t_r - (r-1)(d-1)$ is a bijection between sequences $1 \le t_1 < ... < t_K \le N$ with $t_r - t_{r-1} \ge d$ and unconstrained subsets $1 \le y_1 < ... < y_K \le N-(K-1)(d-1)$. Therefore, the number of valid schedules is:

$$\mathrm{Ways}(N, K, d) = \begin{cases} \binom{N-(K-1)(d-1)}{K} \bmod p & \text{if } N-(K-1)(d-1) \ge K \\ 0 & \text{otherwise} \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace $N=8, K=3, d=3$. Here $(K-1)(d-1) = 2 \times 2 = 4$, so $N' = 8-4 = 4$. The shift $y_r = t_r - 2(r-1)$ bijects every valid triple $(t_1, t_2, t_3) \subseteq \{1...8\}$ with $t_r - t_{r-1} \ge 3$ onto one of the $\binom{4}{3} = 4$ unconstrained triples $(y_1, y_2, y_3) \subseteq \{1, 2, 3, 4\}$:




```cpp
#include <iostream>
#include <vector>
using namespace std;

const int MOD = 1000000007;
const int MAXN = 1000000;

long long fact[MAXN + 1], inv_fact[MAXN + 1];

long long mod_pow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1;
    }
    return res;
}

void init_combinatorics() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; ++i) fact[i] = (fact[i - 1] * i) % MOD;
    inv_fact[MAXN] = mod_pow(fact[MAXN], MOD - 2);
    for (int i = MAXN - 1; i >= 0; --i) inv_fact[i] = (inv_fact[i + 1] * (i + 1)) % MOD;
}

long long nCr(long long n, long long r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * inv_fact[r] % MOD * inv_fact[n - r] % MOD;
}

long long solve_backups(long long N, long long K, long long d) {
    // Use 64-bit arithmetic so (K - 1) * (d - 1) never overflows!
    long long n_prime = N - (K - 1) * (d - 1);
    return nCr(n_prime, K);
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Bijection:** Because the linear map $\Phi(t_1, ..., t_K) = (t_1, t_2 - (d-1), ..., t_K - (K-1)(d-1))$ has explicit inverse $\Phi^{-1}(y_1, ..., y_K) = (y_1, y_2 + (d-1), ..., y_K + (K-1)(d-1))$, and $t_r - t_{r-1} \geq d \Leftrightarrow y_r - y_{r-1} \geq 1$, $\Phi$ is a bijection between valid schedules and $K$-subsets of $\{1, ..., N'\}$.
- **Small-Case Oracle:** For all $N \in [1, 15]$, $K \in [1, N]$, and $d \in [1, 5]$, compare `solve_backups(N, K, d)` against a bitmask enumeration of all $2^N$ subsets of $\{1...N\}$ that counts masks of popcount $K$ with no two set bits closer than $d$.

---

## Worked Problem 32.2: Weighted Dual-Division Expedition

### Problem & Constraints

An expedition selects a team of $S$ specialists from $A$ Alpha-division engineers and $B$ Beta-division medics ($1 \le A, B \le 10^6$, $1 \le S \le A + B$). If a chosen team contains $k$ engineers from Alpha and $S - k$ medics from Beta, the team can designate **one of its** $k$ **Alpha engineers as the Field Commander** in $k$ ways. Compute the total number of `(team, Field Commander)` configurations across all valid splits $k$, modulo $10^9 + 7$:

$$W = \sum_{k=1}^{S} k \binom{A}{k} \binom{B}{S-k} \bmod (10^9 + 7)$$

You must answer $Q \le 10^5$ queries in $O(1)$ time per query after $O(A_{\max} + B_{\max})$ precomputation.

> **Pause & Discover:** Evaluating the sum $\sum_{k=1}^{S} k \binom{A}{k} \binom{B}{S-k}$ directly takes $O(S)$ time per query. How can you rewrite the story "choose $k$ Alpha engineers and pick 1 of them as Field Commander, then choose $S - k$ Beta medics" by choosing the Field Commander *first*?

### Step 1 — Exact Model & Bottleneck

The direct loop evaluates $\sum_{k=\max(1,\,S-B)}^{\min(A,\,S)} k \binom{A}{k} \binom{B}{S-k} \bmod p$. While this is $O(S)$ per query, with $S \le 10^6$ and $Q = 10^5$ queries, $O(QS) = 10^{11}$ operations is 1000 times too slow. We must collapse the entire summation over $k$ into a single $O(1)$ binomial coefficient.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Multiplying $\binom{A+B}{S}$ by $A$ or $S$.** A hasty guess is that since $\sum_k \binom{A}{k} \binom{B}{S-k} = \binom{A+B}{S}$, the weighted sum might be $A \binom{A+B}{S}$ or $S \binom{A+B}{S}$. - **Counterexample:** Take $A = 2, B = 2, S = 2$. - Direct sum: for $k = 1$, $1 \cdot \binom{2}{1} \binom{2}{1} = 4$; for $k = 2$, $2 \cdot \binom{2}{2} \binom{2}{0} = 2$. Total $W = 4 + 2 = 6$. - Guessing $A \binom{A+B}{S}$ gives $2 \binom{4}{2} = 12$ — **wrong!**

**Mistake 2: Dividing by $k$ or Failing When $S - 1 < 0$.** Using factorial manipulation without checking $S \ge 1$ and $A \ge 1$ can pass negative arguments to `inv_fact`.

### Step 3 — The Turn: Discovering the Structure

We can discover the closed form through two complementary lenses—one combinatorial and one algebraic:

1. **The Combinatorial Story (Choose the Commander First!):** Every valid configuration consists of:
   - **Step 1:** Choose 1 Field Commander from the $A$ Alpha-division engineers. There are $A$ **ways** to do this.
   - **Step 2:** Choose the remaining $S - 1$ team members from the remaining $(A - 1) + B$ people (any mix of the remaining $A - 1$ Alpha engineers and $B$ Beta medics!). There are $\binom{A+B-1}{S-1}$ **ways** to do this. By the product rule, the total number of configurations across all $k$ is immediately $A \binom{A+B-1}{S-1}$!
2. **The Algebraic Absorption + Vandermonde Identity:** Use the absorption identity $k \binom{A}{k} = A \binom{A-1}{k-1}$ to pull the constant $A$ outside the summation, then apply Vandermonde's convolution identity $\sum_j \binom{X}{j} \binom{Y}{M-j} = \binom{X+Y}{M}$:

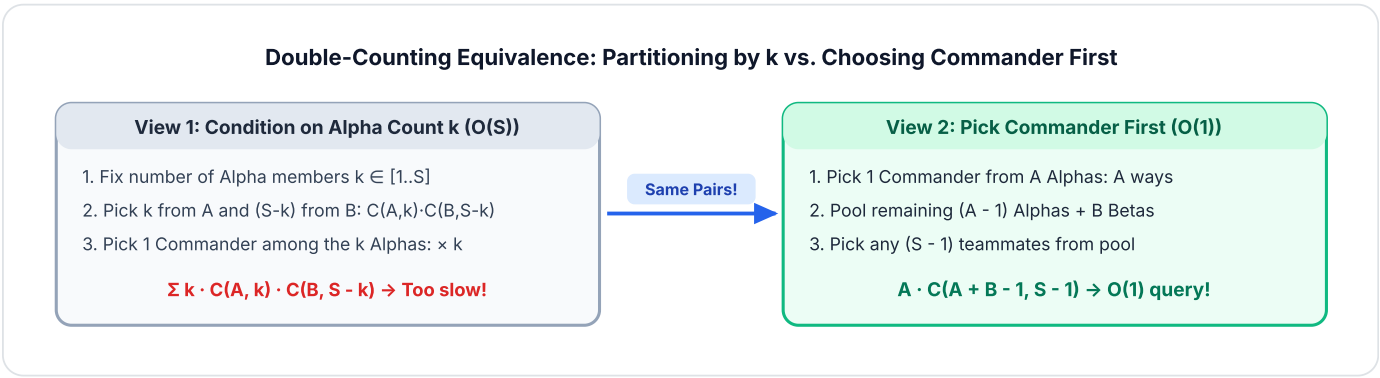
$$\sum_{k=1}^{S} k \binom{A}{k} \binom{B}{S-k} = A \sum_{k=1}^{S} \binom{A-1}{k-1} \binom{B}{(S-1)-(k-1)} = A \binom{A+B-1}{S-1}$$

**Absorption + Vandermonde Convolution Identity:**

By absorbing $k \binom{A}{k} = A \binom{A-1}{k-1}$ and applying Vandermonde's identity over the remaining $A + B - 1$ elements:

$$\sum_{k=0}^{S} k \binom{A}{k} \binom{B}{S-k} = A \binom{A+B-1}{S-1}$$

### Step 4 — Progressive Construction & Visual Trace




```
// Using init_combinatorics() and nCr(n, r) from Problem 32.1 (with MAXN = 2000000)
long long solve_expedition(long long A, long long B, long long S) {
    if (A <= 0 || S <= 0) return 0;
    return (A % MOD) * nCr(A + B - 1, S - 1) % MOD;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Bijection:** Let $\mathcal{U}$ be the set of triples $(C, X, Y)$ where $C \in \text{Alpha}$, $X \subseteq \text{Alpha}\backslash\{C\}$, $Y \subseteq \text{Beta}$, and $|X|+|Y| = S-1$. Grouping $\mathcal{U}$ by $k = |X|+1$ gives $\sum_{k=1}^{S} k \binom{A}{k} \binom{B}{S-k}$ because choosing $\{C\} \cup X$ of size $k$ and then designating $C$ has $\binom{A}{k} \cdot k$ choices. Grouping $\mathcal{U}$ first by $C \in \text{Alpha}$ ($A$ choices) and then by the $(S-1)$-subset $X \cup Y \subseteq (\text{Alpha}\backslash\{C\}) \cup \text{Beta}$ gives $A \binom{A+B-1}{S-1}$.
- **Small-Case Oracle:** Test all $A, B \in [1, 20]$ and $S \in [1, A+B]$ against the explicit $O(S)$ summation loop.

---

# Section 33 — Inclusion-Exclusion

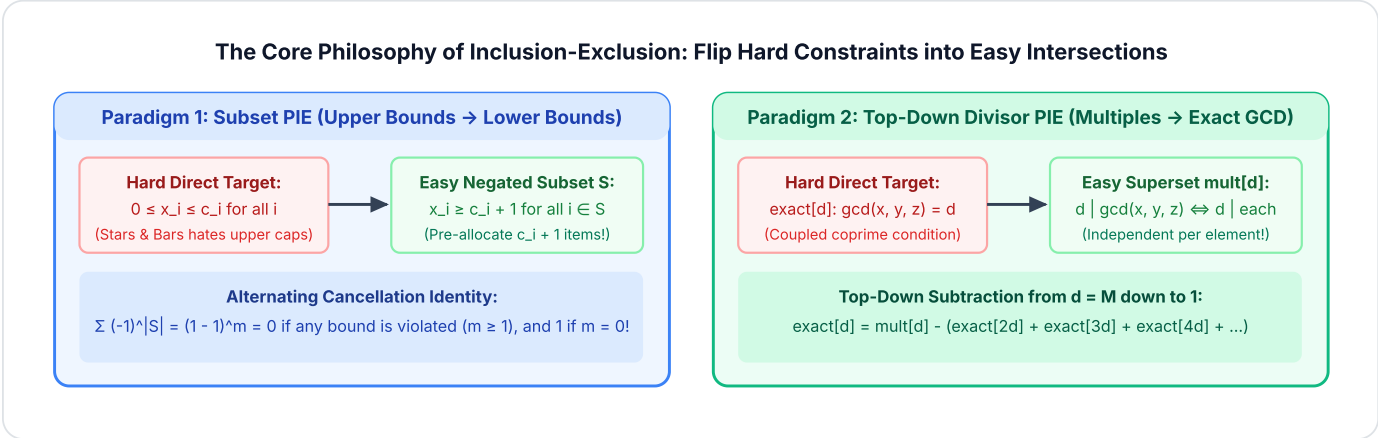
## The Discovery & Modeling Approach

Why are "at least one", "no element exceeds $c_i$", and "exact GCD equals $d$" initially hard to count directly? Because their defining conditions are **disjunctions or strict exclusions** whose territories overlap. By contrast, their **negations or relaxations** ("violate all upper bounds in subset $S$", or "$d$ divides the GCD, i.e., $d$ divides every element") are pure **conjunctions (intersections)** that factor cleanly!

Whenever a problem asks you to avoid $K$ bad properties $A_1, \ldots, A_K$, or to isolate an *exact* invariant (exact set of used characters, exact GCD $d$, exact number of fixed points), the Inclusion-Exclusion Principle (PIE) is the bridge from **easy-to-count supersets ("at least / multiple of $d$")** to **hard-to-count exact sets ("none / exact $d$")**.

### Decision Compass & Structural Lenses

| Problem Signal | Inclusion-Exclusion Lens | Easy Intersection Term | Complexity |
|---|---|---|---|
| Avoid $K \le 20$ distinct bad conditions $A_1, \ldots, A_K$ | **Explicit Bitmask PIE** | For mask $S \subseteq \{1 \ldots K\}$, count configurations violating *all* $i \in S$; weight by $(-1)^{\lvert S\rvert}$ | $O(2^K)$ |
| $K \le 10^5$ symmetric bad conditions (derangements, surjective maps, upper-bounded identical bins) | **Symmetric Binomial PIE** | All subsets of size $j = \lvert S\rvert$ have identical intersection size $f(j)$; sum $\sum_{j=0}^{K} (-1)^j \binom{K}{j} f(j)$ | $O(K)$ |
| Count tuples in $[1, M]$ with **exact** $\gcd(x_1, \ldots, x_r) = d$ | **Top-Down Divisor / Multiples PIE** | Compute $\text{mult}[d] = \binom{\text{cnt}_{\text{div}}(d)}{r}$, then sweep $d = M \ldots 1$: $\text{exact}[d] = \text{mult}[d] - \sum_{k \ge 2} \text{exact}[kd]$ | $O(M \log M)$ |
| Count elements in $[1, N]$ coprime to $X$ | **Prime-Divisor Mobius PIE** | Factor $X$ into distinct primes $p_1, \ldots, p_\omega$ ($\omega \le 15$); PIE over square-free divisors $d \mid X$ with weight $\mu(d) = (-1)^{\lvert S\rvert}$ | $O(2^{\omega(X)})$ |




## Worked Problem 33.1: Capacity-Capped Vault Allocation

### Problem & Constraints

You must distribute $N$ identical gold bars ($0 \le N \le 10^{12}$) among $K$ distinct vaults ($1 \le K \le 18$), where vault $i$ can hold **at most** $c_i$ bars ($0 \le x_i \le c_i$, with $0 \le c_i \le 10^{12}$) and $\sum_{i=1}^{K} x_i = N$. Count the number of valid allocations $(x_1, \ldots, x_K)$ modulo $p = 10^9 + 7$.

**Pause & Discover:** Stars and Bars easily counts solutions to $x_1 + \ldots + x_K = N$ with *lower* bounds $x_i \geq L_i$ (by pre-placing $L_i$ bars in vault $i$), but it cannot directly handle *upper* bounds $x_i \leq c_i$. What happens when you negate the condition $x_i \leq c_i$?

### Step 1 — Exact Model & Bottleneck

A standard knapsack DP maintains `dp[i][bars_used]`, transitioning over $0 \leq x_i \leq c_i$. The bottleneck is $N \leq 10^{12}$: the capacity dimension is a trillion, so any algorithm whose time or space scales with $N$ or $\sum c_i$ is impossible. Conversely, the number of vaults $K \leq 18$ is tiny, pointing directly toward an $O(2^K \cdot K)$ subset enumeration!

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Subtracting Single-Vault Violations Only.** It is tempting to take all unconstrained allocations $\binom{N+K-1}{K-1}$ and subtract $\sum_{i=1}^{K} \binom{N-(c_i+1)+K-1}{K-1}$. - **Counterexample:** Let $K = 2, N = 4, c_1 = 1, c_2 = 1$. Obviously $x_1 + x_2 = 4$ has **0** solutions with $x_1 \leq 1, x_2 \leq 1$. - Unconstrained solutions to $x_1 + x_2 = 4$: $\binom{4+1}{1} = 5$, namely $(4,0), (3,1), (2,2), (1,3), (0,4)$. - Bad set $A_1$ ($x_1 \geq 2$): $(4,0), (3,1), (2,2)$ — size 3. - Bad set $A_2$ ($x_2 \geq 2$): $(2,2), (1,3), (0,4)$ — size 3. - Subtracting once gives $5 - 3 - 3 = -1 \neq 0$ because $(2,2)$ violated **both** bounds and was subtracted twice! Adding back $|A_1 \cap A_2| = 1$ repairs the count to $5 - 6 + 1 = 0$.

**Mistake 2: Using Precomputed Factorial Tables When $N + K - 1 \geq p$.** Here $N \leq 10^{12}$, which is larger than $p = 10^9 + 7$ (or too large for a $10^6$ array). Wait: since $K \leq 18$, in $\binom{M+K-1}{K-1}$ the bottom parameter is $K - 1 \leq 17$! We should compute $\binom{M+K-1}{K-1} = \frac{\prod_{j=1}^{K-1}((M+j) \bmod p)}{(K-1)!}$ directly in $O(K)$ steps! (Note: if the problem guarantees $N < p$, this product formula is exact; we assume $N + K < p$ or work in $\mathbb{Q}$ reduced $\bmod p$ for $K \leq 18$).

### Step 3 — The Turn: Discovering the Structure

For each vault $i \in \{1, \ldots, K\}$, define the **bad property** $A_i$ as:

$$A_i = \{\, (x_1, \ldots, x_K) : x_i \geq c_i + 1 \,\}$$

That is, vault $i$ overflows its capacity by receiving at least $c_i + 1$ bars! Why is the intersection $\bigcap_{i \in S} A_i$ so easy to count for any subset $S \subseteq \{1, \ldots, K\}$? Because requiring $x_i \geq c_i + 1$ for all $i \in S$ (and $x_i \geq 0$ for $i \notin S$) is a **pure lower-bound constraint**! We simply pre-place $c_i + 1$ bars into every vault $i \in S$, leaving:

$$N_S = N - \sum_{i \in S} (c_i + 1)$$

remaining bars to distribute freely among all $K$ vaults with no upper bounds at all! By Stars and Bars, if $N_S \geq 0$, there are $\binom{N_S+K-1}{K-1}$ ways; if $N_S < 0$, there are $0$ ways.

**Upper-Bound to Lower-Bound Subset PIE Formula:**

The number of non-negative integer solutions to $x_1 + \ldots + x_K = N$ subject to $0 \leq x_i \leq c_i$ is:

$$\text{Valid} = \sum_{S \subseteq \{1,\ldots,K\}} (-1)^{|S|} \binom{N - \sum_{i \in S}(c_i+1) + K - 1}{K-1}$$

where $\binom{M}{K-1} = 0$ whenever $M < K - 1$.

### Step 4 — Progressive Construction & Visual Trace

**Inclusion-Exclusion Trace for K = 2, N = 4, Capacities c = (1, 1)**

| S = ø (\|S\| = 0, Sign +) | S = {1} (\|S\| = 1, Sign -) | S = {2} (\|S\| = 1, Sign -) | S = {1,2} (\|S\| = 2, Sign +) |
|---|---|---|---|
| Pre-place: 0 bars | Force $x_1 \geq 2$ (pre-place 2) | Force $x_2 \geq 2$ (pre-place 2) | Force $x_1 \geq 2$ and $x_2 \geq 2$ |
| Remaining N_S = 4 | Remaining N_S = 4 - 2 = 2 | Remaining N_S = 4 - 2 = 2 | Remaining N_S = 4 - 4 = 0 |
| + C(4+1, 1) = +5 | - C(2+1, 1) = -3 | - C(2+1, 1) = -3 | + C(0+1, 1) = +1 |

**Alternating Sum: +5 - 3 - 3 + 1 = 0 valid allocations (Double-subtracted state (2,2) restored!)**

```cpp
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;

long long mod_pow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1;
    }
    return res;
}

// Computes C(n, r) mod MOD for huge n (assuming n < MOD) and small r <= 20
long long nCr_small_r(long long n, int r, long long inv_fact_r) {
    if (n < r || r < 0) return 0;
    long long num = 1;
    for (int i = 0; i < r; ++i) {
        num = (num * ((n - i) % MOD)) % MOD;
    }
    return (num * inv_fact_r) % MOD;
}

long long solve_vaults(long long N, const vector<long long>& c) {
    int K = c.size();
    long long fact_k_minus_1 = 1;
    for (int i = 1; i <= K - 1; ++i) fact_k_minus_1 = (fact_k_minus_1 * i) % MOD;
    long long inv_fact = mod_pow(fact_k_minus_1, MOD - 2);

    long long ans = 0;
    for (int mask = 0; mask < (1 << K); ++mask) {
        long long rem = N;
        int bits = 0;
        for (int i = 0; i < K; ++i) {
            if ((mask >> i) & 1) {
                rem -= (c[i] + 1);
                bits++;
            }
        }
        if (rem >= 0) {
            long long ways = nCr_small_r(rem + K - 1, K - 1, inv_fact);
            if (bits % 2 == 1) {
                ans = (ans - ways + MOD) % MOD;
            } else {
                ans = (ans + ways) % MOD;
            }
        }
    }
    return ans;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Exact Cancellation:** Consider any allocation $\mathbf{x} = (x_1, \ldots, x_K)$ with $\sum x_i = N$, and let $V(\mathbf{x}) = \{i : x_i \geq c_i + 1\}$ be the exact set of vaults whose upper bounds are violated by $\mathbf{x}$. In the PIE sum, $\mathbf{x}$ is counted in the term for subset $S$ if and only if $S \subseteq V(\mathbf{x})$. Thus the net weight contributed by $\mathbf{x}$ is $\sum_{S \subseteq V(\mathbf{x})} (-1)^{|S|} = (1-1)^{|V(\mathbf{x})|}$, which equals $1$ if $|V(\mathbf{x})| = 0$ (all capacities respected) and $0$ if $|V(\mathbf{x})| \geq 1$.
- **Small-Case Oracle:** Test $K \in [1, 5]$, $N \in [0, 15]$, and $c_i \in [0, 8]$ against a simple recursive brute-force search that tries all $0 \leq x_i \leq c_i$.

---

## Worked Problem 33.2: Exact-Stride Resonance Triplets

### Problem & Constraints

Given an array $A = [a_1, a_2, ..., a_N]$ of $N$ integers ($3 \le N \le 2 \times 10^5$, $1 \le a_i \le M = 10^6$), compute for **every** integer $d \in [1, M]$ the number of unordered index triplets $1 \le i < j < k \le N$ whose greatest common divisor is **exactly** $d$:

$$\gcd(a_i, a_j, a_k) = d$$

> **Pause & Discover:** Directly checking whether $\gcd(a_i, a_j, a_k) = d$ couples all three elements together. By contrast, when is $d$ a *common divisor* of $(a_i, a_j, a_k)$ (that is, $d \mid \gcd(a_i, a_j, a_k)$)? Once you know the count of triplets whose GCD is a *multiple* of $d$, how can you peel off $2d, 3d, 4d, ...$ from top to bottom?

### Step 1 — Exact Model & Bottleneck

Enumerating all $\binom{N}{3}$ triplets and computing `gcd(a[i], gcd(a[j], a[k]))` takes $O(N^3 \log M)$ time—roughly $1.3 \times 10^{15}$ operations for $N = 2 \times 10^5$. We must aggregate in value space $[1, M]$ instead of index space.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Subtracting** $\text{mult}[2d], \text{mult}[3d], ...$ **Instead of** $\text{exact}[2d], \text{exact}[3d], ...$ Let $\text{mult}[d]$ be the number of triplets whose GCD is a multiple of $d$. A classic bug is writing `exact[d] = mult[d] - mult[2*d] - mult[3*d] - ...`. - **Counterexample:** Suppose a triplet has exact GCD $6$, and we are computing $\text{exact}[1]$. That triplet is counted inside $\text{mult}[1]$, $\text{mult}[2]$, $\text{mult}[3]$, and $\text{mult}[6]$. If we subtract $\text{mult}[2], \text{mult}[3], \text{mult}[6]$ from $\text{mult}[1]$, the triplet with GCD $6$ gets subtracted **three times** instead of once! - **The Repair:** If we sweep $d$ **downward from** $M$ **to** $1$, by the time we are at $d$, we have *already* computed the disjoint buckets $\text{exact}[2d], \text{exact}[3d], ..., \text{exact}[kd]$! Since a triplet with $d \mid \gcd$ has a unique exact GCD equal to $k \cdot d$ for some $k \ge 1$, the buckets $\text{exact}[kd]$ are **mutually disjoint** and can be subtracted directly without Mobius weights!

### Step 3 — The Turn: Discovering the Structure

Let us build the two-pass Harmonic Sieve: 1. **Pass 1 (Count Multiples):** Let $\text{cnt}[v]$ be the frequency of value $v$ in $A$. For each $d \in [1, M]$, count how many elements of $A$ are divisible by $d$:

$$c(d) = \sum_{k=1}^{\lfloor M/d \rfloor} \text{cnt}[k \cdot d]$$

A triplet $(a_i, a_j, a_k)$ has $d \mid \gcd(a_i, a_j, a_k)$ if and only if $d$ divides all three elements. Thus the number of triplets whose GCD is a multiple of $d$ is simply:

$$\text{mult}[d] = \binom{c(d)}{3} = \frac{c(d)\,\big(\,c(d)-1\,\big)\,\big(\,c(d)-2\,\big)}{6}$$

2. **Pass 2 (Top-Down Divisor Inclusion-Exclusion):** Since $\text{mult}[d] = \sum_{k \ge 1} \text{exact}[k \cdot d]$, we isolate $\text{exact}[d]$ by iterating $d$ from $M$ **down to** $1$:

$$\text{exact}[d] = \text{mult}[d] - \sum_{k=2}^{\lfloor M/d \rfloor} \text{exact}[k \cdot d]$$

Both passes iterate over multiples $k \cdot d \le M$, so total time is $O(M \log M)$!

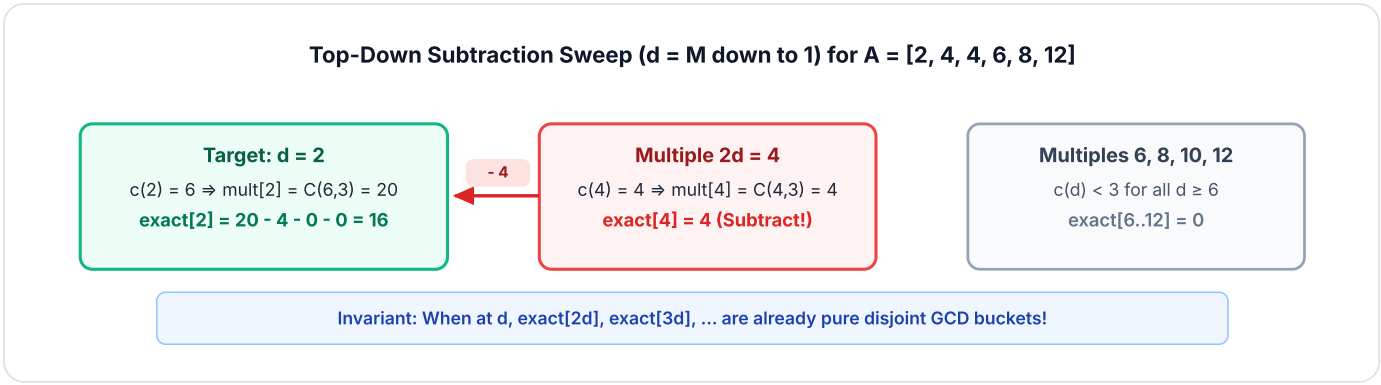
**Top-Down GCD Sieve Identity:**

For any divisor $d \in [1, M]$, let $c(d) = \sum_{k \ge 1} \text{cnt}[kd]$ be the number of array elements divisible by $d$. Then:

$$\text{mult}[d] = \binom{c(d)}{3}, \qquad \text{exact}[d] = \text{mult}[d] - \sum_{k=2}^{\lfloor M/d \rfloor} \text{exact}[k \cdot d] \quad \text{(evaluated for } d = M, M-1, ..., 1)$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace $A = [2, 4, 4, 6, 8, 12]$ ($M = 12$) at $d = 2$: - Multiples of 6: $\{6, 12\} \implies c(6) = 2 \implies \text{mult}[6] = \binom{2}{3} = 0 \implies \text{exact}[6] = 0$. - Multiples of 4: $\{4, 4, 8, 12\} \implies c(4) = 4 \implies \text{mult}[4] = \binom{4}{3} = 4$. Since $\text{exact}[8] = \text{exact}[12] = 0$, $\text{exact}[4] = 4$. - Multiples of 2: all 6 elements $\implies c(2) = 6 \implies \text{mult}[2] = \binom{6}{3} = 20$. - Subtracting higher multiples of 2:
$\text{exact}[2] = \text{mult}[2] - (\text{exact}[4] + \text{exact}[6] + \text{exact}[8] + \text{exact}[10] + \text{exact}[12]) = 20 - (4 + 0 + 0 + 0 + 0) = 16$.




```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

vector<long long> count_exact_gcd_triplets(const vector<int>& a) {
    int M = *max_element(a.begin(), a.end());
    vector<long long> cnt(M + 1, 0), exact(M + 1, 0);
    for (int x : a) cnt[x]++;

    // Sweep d from M down to 1
    for (int d = M; d >= 1; --d) {
        long long c = 0;
        for (int m = d; m <= M; m += d) {
            c += cnt[m];
        }
        // mult[d] = C(c, 3)
        long long mult_d = (c >= 3) ? c * (c - 1) * (c - 2) / 6 : 0;

        // Subtract disjoint buckets exact[2d], exact[3d], ...
        long long sub = 0;
        for (int m = 2 * d; m <= M; m += d) {
            sub += exact[m];
        }
        exact[d] = mult_d - sub;
    }
    return exact;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof by Partition of Multiples:** Fix $d \geq 1$. The set of triplets $T_{\text{mult}}(d) = \{(i, j, k) : d \mid \gcd(a_i, a_j, a_k)\}$ has size $\binom{c(d)}{3}$. Every triplet in $T_{\text{mult}}(d)$ has $\gcd(a_i, a_j, a_k) = m$ for a unique multiple $m \in \{d, 2d, 3d, \ldots\}$. Hence $T_{\text{mult}}(d) = \bigsqcup_{k \geq 1} T_{\text{exact}}(k \cdot d)$ is a disjoint union, proving $\text{exact}[d] = \text{mult}[d] - \sum_{k \geq 2} \text{exact}[k \cdot d]$.
- **Small-Case Oracle:** Generate random arrays of size $N = 25$ with $a_i \in [1, 30]$ and verify `exact[d]` for every $d \in [1, 30]$ against the $O(N^3)$ triplet loop.

---

# Section 34 — Probability and Expected Value

## The Discovery & Modeling Approach

Random processes intimidate many programmers because the sample space of trajectories is exponentially large (or infinite when self-loops and resets occur), and local events in a permutation or graph are almost always **strongly dependent**.

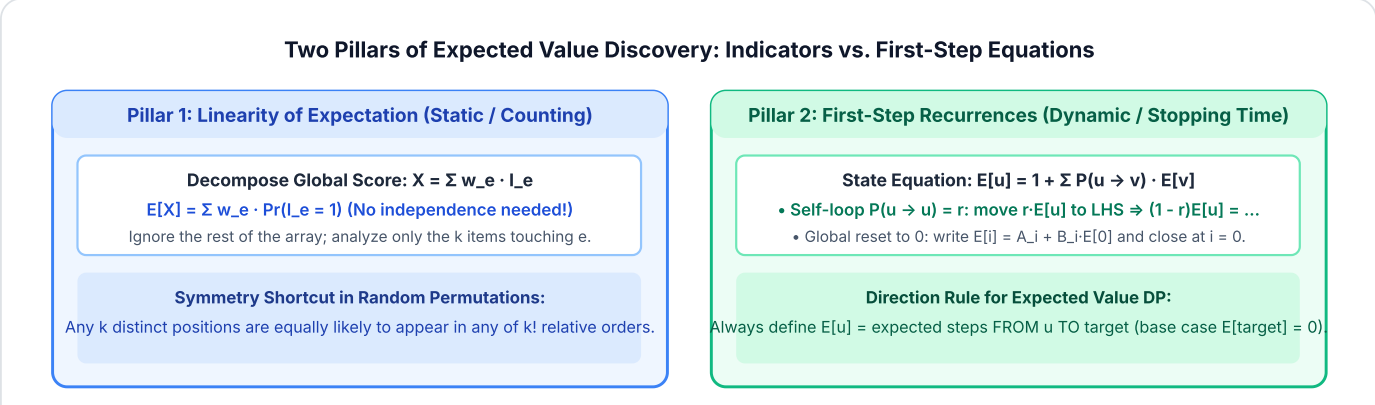
Section 34 rests on two liberating algebraic superpowers that bypass the sample space completely: 1. **Linearity of Expectation via Indicator Random Variables:** For *any* collection of random variables $X_1, \dots, X_m$ on the same probability space—**even when they are heavily dependent!**—the expectation of their sum is the sum of their expectations:

$$\mathbb{E}\left[\sum_{i=1}^{m} X_i\right] = \sum_{i=1}^{m} \mathbb{E}[X_i]$$

Whenever a problem asks for the expected count of objects (peaks, inversions, connected components, surviving elements), define an **indicator variable** $I_e \in \{0, 1\}$ for each candidate object $e$. Then $\mathbb{E}[\text{Count}] = \sum_e \Pr(I_e = 1)$, reducing a global random process to a tiny **local marginal probability** involving only the elements that touch $e$! 2. **First-Step Markov Analysis (Expected Value DP with Self-Loops & Resets):** When waiting for a random walk to reach a target state $N$, define $E[u]$ as the expected **remaining** steps from state $u$ to the target. Write $E[u] = 1 + \sum_v p_{u \to v} E[v]$. If $u$ can transition back to $u$ (a self-loop) or back to $0$ (a global reset), do not try to expand an infinite geometric series—**isolate** $E[u]$ **algebraically** or propagate linear coefficients $E[i] = A_i + B_i E[0]$!

### Decision Compass & Structural Lenses

| Problem Signal | Probabilistic Lens | Algebraic Reduction | Fatal Trap |
|---|---|---|---|
| Expected number of peaks / inversions / cycles / matches | **Indicator Linearity** ( $X = \sum I_e$) | $\mathbb{E}[X] = \sum_e \Pr(I_e = 1)$ using local symmetry on the small neighborhood of $e$ | Assuming independence ( $\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y]$ is false for dependent variables; only $\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ is unconditional!) |
| Expected value of $X^2$ where $X = \sum_{i=1}^{m} I_i$ | **Expanded Bilinear Indicators** | $X^2 = \sum_i I_i + 2\sum_{i<j} I_i I_j$, so $\mathbb{E}[X^2] = \sum_i \Pr(I_i = 1) + 2\sum_{i<j} \Pr(I_i = 1 \wedge I_j = 1)$ | Computing $(\mathbb{E}[X])^2$ instead of $\mathbb{E}[X^2]$ |
| State $u$ succeeds with prob $p$ or stays at $u$ with prob $1-p$ | **Self-Loop Isolation** | $E[u] = 1 + pE[\text{next}] + (1-p)E[u] \implies pE[u] = 1 + pE[\text{next}] \implies E[u] = \frac{1}{p} + E[\text{next}]$ | Leaving $E[u]$ on the right-hand side and running an unroll loop instead of dividing by $p$ |
| State $i$ advances to $i+1$ (prob $p_i$) or resets to $0$ (prob $1-p_i$) | **Symbolic Root Parameterization** | Express $E[i] = A_i + B_i E[0]$ from $i = N-1$ down to $0$; solve $E[0] = A_0 + B_0 E[0] \implies E[0] = \frac{A_0}{1-B_0}$ | Summing $\sum 1/p_i$ as if floors were independent checkpoints |



## Worked Problem 34.1: Local-Peak Sensor Cascade

### Problem & Constraints

You are given $N$ sensors with **distinct** calibration heights $h_1, h_2, ..., h_N$ ($1 \le N \le 5 \times 10^5$) and $N$ position weights $w_1, w_2, ..., w_N$ ($1 \le w_i \le 10^9$). The sensors are placed into the $N$ slots $1, 2, ..., N$ in a **uniformly random permutation** $\pi$ (all $N!$ orderings are equally likely).

After placement, slot $i$ ($1 \le i \le N$) triggers a **peak surge** if the sensor at slot $i$ is **strictly taller than all of its existing immediate neighbors**: - For an interior slot ($1 < i < N$), slot $i$ is a peak if $H[i] > H[i-1]$ and $H[i] > H[i+1]$. - For a boundary slot ($i = 1$ or $i = N$, when $N \ge 2$), slot $i$ is a peak if $H[1] > H[2]$ (resp. $H[N] > H[N-1]$). - If $N = 1$, slot 1 has no neighbors and is always a peak.

Each peak at slot $i$ adds $w_i$ to the total surge score. Compute the **expected total surge score** modulo $10^9 + 7$.

> **Pause & Discover:** Notice that if slot $i$ is a peak, slot $i+1$ *cannot* be a peak—so adjacent slots are strongly negatively correlated! Does that dependence stop you from summing the expected surge of each slot $i$? For a fixed interior slot $i$, look only at the 3 random sensors landing in slots $\{i-1, i, i+1\}$: what is the probability that the tallest of those 3 lands in the middle?

### Step 1 — Exact Model & Bottleneck

The exact definition averages the total surge over all $N!$ permutations of the sensors:

$$\mathbb{E}[\text{Score}] = \frac{1}{N!} \sum_{\pi \in S_N} \sum_{i=1}^{N} w_i \cdot \mathbf{1}[\text{slot } i \text{ is a peak in } \pi]$$

The bottleneck is $N!$ (or $O(2^N)$ bitmask DP): for $N = 5 \times 10^5$, enumerating permutations is impossible.

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Multiplying Independent Pairwise Win Probabilities ($\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$).** For an interior slot $i$ ($1 < i < N$), we need both $H[i] > H[i-1]$ (which happens with probability $1/2$) and $H[i] > H[i+1]$ (which also happens with probability $1/2$). It is terribly tempting to multiply them and claim $\Pr(\text{slot } i \text{ is a peak}) = \frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$. - **Counterexample:** Take $N = 3$ with heights $\{10, 20, 30\}$ and weights $w = [1, 1, 1]$. Examine slot $i = 2$ (the middle slot) across all $3! = 6$ permutations: - $[10, \mathbf{30}, 20]$ (Peak! $30 > 10$ and $30 > 20$) - $[20, \mathbf{30}, 10]$ (Peak! $30 > 20$ and $30 > 10$) - $[10, 20, 30]$, $[20, 10, 30]$, $[30, 10, 20]$, $[30, 20, 10]$ (Not a peak at $i = 2$) Slot 2 is a peak in **2 out of 6** permutations, so its true probability is $\frac{2}{6} = \frac{1}{3}$, **not** $\frac{1}{4}$! Why? Because the events $H[i] > H[i-1]$ and $H[i] > H[i+1]$ share the random variable $H[i]$: knowing $H[i] > H[i-1]$ makes $H[i]$ more likely to be large, increasing the conditional probability that $H[i] > H[i+1]$ from $1/2$ to $2/3$!

**Mistake 2: Worrying About Whether $i-1$ or $i+1$ Are Also Peaks.** Trying to track joint configurations of multiple peaks via DP is completely unnecessary because expectation is linear regardless of correlation.

### Step 3 — The Turn: Discovering the Structure

Define an indicator random variable $I_i \in \{0, 1\}$ for each slot $i \in \{1, ..., N\}$, equal to 1 if slot $i$ is a peak and 0 otherwise. Then the total surge score is $X = \sum_{i=1}^{N} w_i I_i$. By **Linearity of Expectation**:

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{N} w_i I_i\right] = \sum_{i=1}^{N} w_i \, \mathbb{E}[I_i] = \sum_{i=1}^{N} w_i \Pr(I_i = 1)$$

What is $\Pr(I_i = 1)$ when all $N$ heights are distinct? - **Case $N = 1$:** Slot 1 has no neighbors, so $\Pr(I_1 = 1) = 1$. - **Boundary Slots ($i = 1$ or $i = N$, for $N \ge 2$):** Look at the 2 sensors placed in $\{1, 2\}$. Whatever two distinct sensors land there, both relative orderings $(a, b)$ and $(b, a)$ are equally likely, and in **1 out of 2** orderings the taller one is at slot 1. Thus $\Pr(I_1 = 1) = \Pr(I_N = 1) = \frac{1}{2}$. - **Interior Slots ($1 < i < N$, for $N \ge 3$):** Look at the 3 distinct sensors placed in $\{i-1, i, i+1\}$. All $3! = 6$ relative orderings of those 3 sensors are equally likely, and slot $i$ is a peak if and only if the **unique maximum** of those 3 sensors lands in the middle slot $i$ (while the other two land in $i-1$ and $i+1$ in either of $2! = 2$ orders). Thus:

$$\Pr(I_i = 1) = \frac{2!}{3!} = \frac{1}{3}$$

Notice that the actual numerical values $h_1, ..., h_N$ don't even change the answer, as long as they are distinct!

**Local-Neighborhood Indicator Expectation Formula:**

For any uniformly random permutation of $N$ distinct heights:

$$\mathbb{E}[\text{Score}] = \sum_{i=1}^{N} w_i \Pr(I_i = 1) = \begin{cases} w_1 & \text{if } N = 1 \\ \frac{1}{2}(w_1 + w_N) + \frac{1}{3}\sum_{i=2}^{N-1} w_i \pmod p & \text{if } N \geq 2 \end{cases}$$

### Step 4 — Progressive Construction & Visual Trace

**Marginal Peak Probabilities via Local Neighborhood Symmetry (N ≥ 3)**

**Boundary Slot i = 1**

Window: {H[1], H[2]} (size 2)

2! = 2 equally likely relative orders:

• **[Max, Min] → Peak! (1 way)**

• [Min, Max] → Not peak

**$\Pr(I_1 = 1) = 1 / 2$**

**Interior Slot 1 < i < N**

Window: {H[i-1], H[i], H[i+1]} (size 3)

3! = 6 equally likely relative orders:

• **[Min, Max, Mid] & [Mid, Max, Min] (2 ways)**

• Other 4 orders: Max is on left or right

**$\Pr(I_i = 1) = 2 / 6 = 1 / 3$**

**Boundary Slot i = N**

Window: {H[N-1], H[N]} (size 2)

2! = 2 equally likely relative orders:

• **[Min, Max] → Peak! (1 way)**

• [Max, Min] → Not peak

**$\Pr(I_N = 1) = 1 / 2$**

```cpp
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;

long long mod_pow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1;
    }
    return res;
}

long long solve_expected_peaks(const vector<long long>& w) {
    int n = w.size();
    if (n == 1) return w[0] % MOD;

    long long inv2 = mod_pow(2, MOD - 2);
    long long inv3 = mod_pow(3, MOD - 2);

    long long ans = ((w[0] + w[n - 1]) % MOD) * inv2 % MOD;
    long long interior_sum = 0;
    for (int i = 1; i < n - 1; ++i) {
        interior_sum = (interior_sum + w[i]) % MOD;
    }
    ans = (ans + interior_sum * inv3) % MOD;
    return ans;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Marginal Symmetry:** Partition the $N!$ permutations of $S_N$ into equivalence classes of size $3! = 6$ that agree on the set of values at $\{i-1, i, i+1\}$ and on the exact positions of all other $N-3$ values. Within every equivalence class of 6 permutations, exactly 2 place the largest of the 3 values at index $i$. Summing over all $N!/6$ classes proves $\Pr(I_i = 1) = 2/6 = 1/3$.

- **Small-Case Oracle:** For $N \in [1, 8]$, iterate through all $N!$ permutations using `std::next_permutation`, sum the exact peak scores, divide by $N!$ modulo $10^9 + 7$, and verify exact agreement with `solve_expected_peaks`.

---

## Worked Problem 34.2: Slippery Elevator with Self-Loops & Reset Floor

### Problem & Constraints

An experimental elevator climbs from floor $0$ to floor $N$ ($1 \le N \le 2 \times 10^5$). At each floor $i$ ($0 \le i < N$), pressing the boost button costs $1$ second and produces one of **three** outcomes: - With probability $p_i > 0$, the elevator **advances** to floor $i+1$. - With probability $r_i \ge 0$, the elevator **slips in place** and stays at floor $i$ (a self-loop). - With probability $q_i = 1 - p_i - r_i \ge 0$, a safety trip **resets** the elevator all the way back to floor $0$.

Each probability $p_i, r_i, q_i$ is given as a rational fraction modulo $M = 10^9 + 7$ (with $p_i \not\equiv 0 \pmod M$). Compute the expected total number of button presses required to reach floor $N$ starting from floor $0$, modulo $10^9 + 7$.

> **Pause & Discover:** Write the one-step equation for $E[i]$, the expected remaining presses from floor $i$ to floor $N$. Notice that $E[i]$ appears on both sides (from the self-loop $r_i E[i]$) and $E[0]$ appears in every equation (from the reset $q_i E[0]$). How do you eliminate the self-loop first, and how can you express every $E[i]$ as a linear function $A_i + B_i E[0]$?

### Step 1 — Exact Model & Bottleneck

Let $E[i]$ be the expected number of button presses needed to reach floor $N$ starting from floor $i$. By conditioning on the outcome of the very first button press at floor $i$:

$$E[N] = 0$$
$$E[i] = 1 + p_i E[i+1] + r_i E[i] + q_i E[0] \qquad (0 \le i < N)$$

This is a coupled system of $N$ linear equations in $N$ unknowns $E[0], \ldots, E[N-1]$. Running generic Gaussian elimination on an $N \times N$ matrix takes $O(N^3)$ time, which is impossible for $N = 2 \times 10^5$. We must exploit the **sparse upper-Hessenberg / reset structure** of the transitions to solve the system in $O(N)$ time!

### Step 2 — Tempting Mistakes & Counterexample

**Mistake 1: Treating Floors as Independent Geometric Stages.** If there were no resets ($q_i = 0$, so $r_i = 1 - p_i$), each floor $i$ would simply take $1/p_i$ expected presses, and $E[0] = \sum_{i=0}^{N-1} 1/p_i$. When $q_i > 0$, a reset at floor $i$ destroys all progress from floors $0 \ldots i-1$ and forces the elevator to re-climb from $0$!

**Mistake 2: Trying to Compute $E[i]$ Numerically Before $E[0]$ Is Known.** Because $E[i]$ depends on both $E[i+1]$ and $E[0]$, neither a purely forward DP ($0 \to N$) nor a purely numerical backward DP ($N \to 0$) has all its inputs ready.

### Step 3 — The Turn: Discovering the Structure

We resolve both circularities in two clean algebraic steps:

1. **Step A — Eliminate the Local Self-Loop ($r_i E[i]$):** Subtract $r_i E[i]$ from both sides of the equation for $E[i]$:
   $$(1 - r_i) E[i] = 1 + p_i E[i+1] + q_i E[0]$$
   Divide by $1 - r_i = p_i + q_i$ (multiplying by $(1 - r_i)^{-1} \bmod M$):
   $$E[i] = \frac{1}{1 - r_i} + \frac{p_i}{1 - r_i} E[i+1] + \frac{q_i}{1 - r_i} E[0]$$
2. **Step B — Symbolic Backward Substitution ($E[i] = A_i + B_i E[0]$):** Treat $E[0]$ as a symbolic unknown! Every state $E[i]$ can be written as a linear function of $E[0]$:
   $$E[i] = A_i + B_i \cdot E[0]$$
   At the target floor $N$, $E[N] = 0 = 0 + 0 \cdot E[0]$, so the base case is $(A_N, B_N) = (0, 0)$. Sweeping **backward** from $i = N-1$ down to $0$, substitute $E[i+1] = A_{i+1} + B_{i+1} E[0]$ into the self-loop-free equation:
   $$E[i] = \underbrace{\frac{1 + p_i A_{i+1}}{1 - r_i}}_{A_i} + \underbrace{\frac{p_i B_{i+1} + q_i}{1 - r_i}}_{B_i} \cdot E[0]$$
   Finally, when the backward sweep reaches $i = 0$, we obtain the closed identity for $E[0]$ itself:
   $$E[0] = A_0 + B_0 \cdot E[0] \implies (1 - B_0) E[0] = A_0 \implies E[0] = \frac{A_0}{1 - B_0}$$

**Self-Loop & Global-Reset Linearization Formula:**

From $E[i] = 1 + p_i E[i+1] + r_i E[i] + q_i E[0]$ with base case $A_N = 0, B_N = 0$, sweep $i = N-1 \ldots 0$:

$$A_i = \frac{1 + p_i A_{i+1}}{1 - r_i}, \qquad B_i = \frac{p_i B_{i+1} + q_i}{1 - r_i}$$

At $i = 0$, closing the loop $E[0] = A_0 + B_0 E[0]$ yields the exact solution in $O(N)$ time:

$$E[0] = A_0 \cdot (1 - B_0)^{-1} \pmod{M}$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace a 2-floor elevator ($N = 2$) over the reals: - Floor 0: $p_0 = \frac{1}{2}$ (advance to 1), $r_0 = \frac{1}{4}$ (self-loop at 0), $q_0 = \frac{1}{4}$ (reset to 0). Note $1 - r_0 = \frac{3}{4}$. - Floor 1: $p_1 = \frac{1}{2}$ (advance to 2), $r_1 = \frac{1}{6}$ (self-loop at 1), $q_1 = \frac{1}{3}$ (reset to 0). Note $1 - r_1 = \frac{5}{6}$.

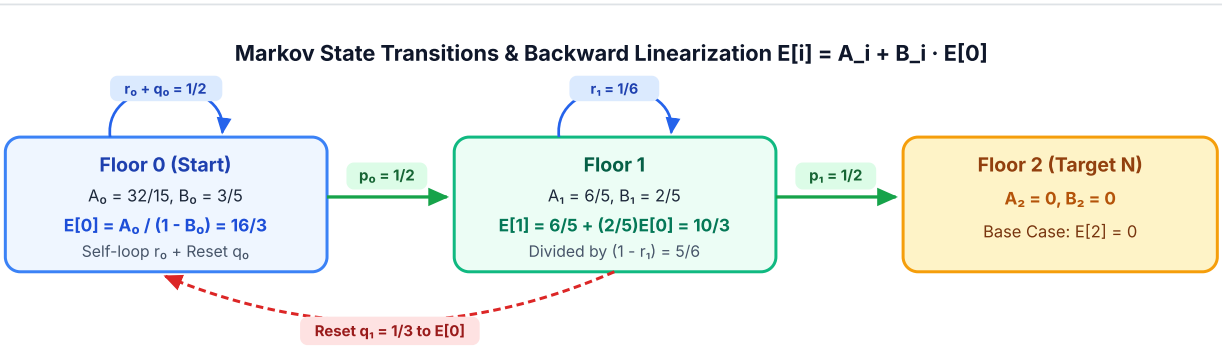
1. **Base case ($i = 2$):** $A_2 = 0, B_2 = 0$.
2. **At $i = 1$:**

$$A_1 = \frac{1 + \frac{1}{2}(0)}{5/6} = \frac{6}{5}, \qquad B_1 = \frac{\frac{1}{2}(0) + \frac{1}{3}}{5/6} = \frac{2}{5}$$

   So $E[1] = \frac{6}{5} + \frac{2}{5}E[0]$.
3. **At $i = 0$:**

$$A_0 = \frac{1 + \frac{1}{2}(6/5)}{3/4} = \frac{8/5}{3/4} = \frac{32}{15}, \qquad B_0 = \frac{\frac{1}{2}(2/5) + \frac{1}{4}}{3/4} = \frac{9/20}{3/4} = \frac{3}{5}$$

   So $E[0] = \frac{32}{15} + \frac{3}{5}E[0] \implies \frac{2}{5}E[0] = \frac{32}{15} \implies E[0] = \frac{16}{3}$.




```cpp
#include <iostream>
#include <vector>
using namespace std;

const long long MOD = 1000000007;

long long mod_pow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1;
    }
    return res;
}

long long mod_inv(long long x) {
    return mod_pow((x % MOD + MOD) % MOD, MOD - 2);
}

// p[i]: advance to i+1, r[i]: stay at i, q[i]: reset to 0 (all given mod MOD)
long long solve_elevator(int N, const vector<long long>& p,
                         const vector<long long>& r, const vector<long long>& q) {
    long long A = 0; // A_N = 0
```

```
    long long B = 0; // B_N = 0

    for (int i = N - 1; i >= 0; --i) {
        long long inv_denom = mod_inv(1 - r[i]);
        long long next_A = (1 + p[i] * A) % MOD * inv_denom % MOD;
        long long next_B = (p[i] * B + q[i]) % MOD * inv_denom % MOD;
        A = next_A;
        B = next_B;
    }
    // Now E[0] = A_0 + B_0 * E[0] => E[0] = A_0 * (1 - B_0)^(-1) mod MOD
    return A * mod_inv(1 - B) % MOD;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Uniqueness & Non-Zero Denominator:** Over the real probabilities with $p_i > 0$ and $p_i + r_i + q_i = 1$, notice by downward induction on $i$ that $B_N = 0 < 1$, and if $B_{i+1} < 1$, then $B_i = \frac{p_i B_{i+1} + q_i}{p_i + q_i} < \frac{p_i + q_i}{p_i + q_i} = 1$. Thus $1 - B_0 > 0$ strictly, proving that the Markov chain is absorbing at $N$ with finite expectation and a unique solution to the linear system.
- **Small-Case Oracle:** For $N = 3$ and small rational probabilities (e.g., multiples of $1/12$), solve the $3 \times 3$ linear system $(I - T)\mathbf{E} = \mathbf{1}$ using exact rational Gaussian elimination and verify that `solve_elevator` produces the identical rational answer reduced modulo $10^9 + 7$.

# Section 36 — Fenwick Trees

## The Discovery & Modeling Approach

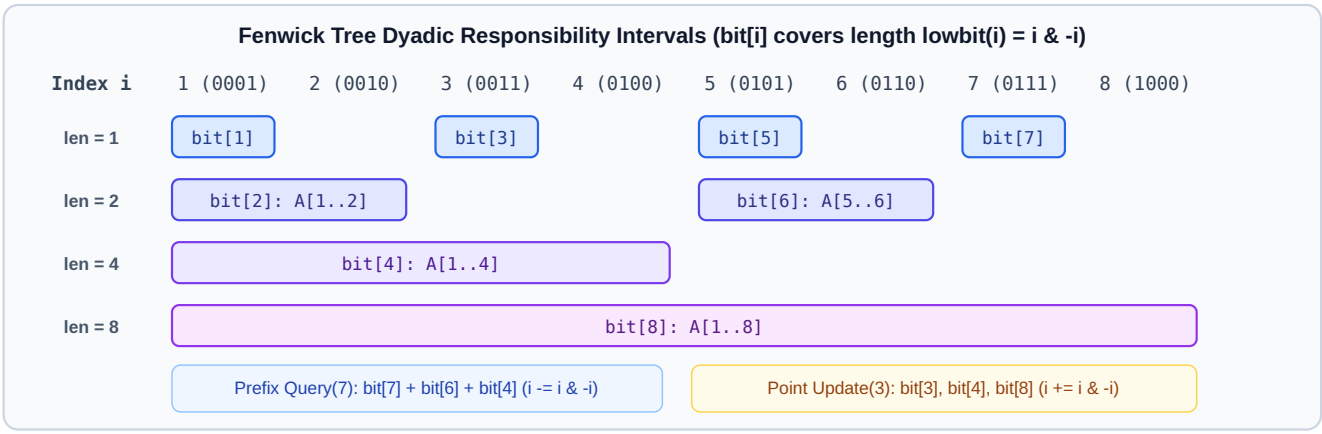
A **Fenwick Tree** (Binary Indexed Tree, or BIT) is not merely a shorter way to code a range-sum structure; it is the canonical representation of **dynamic prefix aggregation over an Abelian group**. Whenever a problem's bottleneck is repeatedly computing a prefix aggregate $\bigoplus_{i=1}^{r} A[i]$ while elements are dynamically updated, and the binary operation $\oplus$ possesses an **inverse** $\ominus$ (so that any range $[l, r]$ can be recovered by prefix subtraction $\text{pref}(r) \ominus \text{pref}(l-1)$), a Fenwick tree is the minimal, lowest-overhead compressor available.

To discover whether a problem reduces to a Fenwick tree, inspect the **domain** on which prefixes are taken and the **algebra** of the aggregate:

1. **Index-Space Prefix Dynamics**: Updates modify $A[i]$ at given array positions, and queries ask for sums, XORs, or counts over index intervals $[l, r]$.
2. **Value-Space Sweep (2D Dominance & Inversions)**: When a condition compares two dimensions simultaneously—such as $i < j$ and $A[i] < A[j]$—neither dimension is static. By sorting or sweeping along the first dimension (time or index $j$), the condition $i < j$ becomes an invariant ("only already-processed items are present in the structure"), reducing the second condition $A[i] < A[j]$ to a 1D prefix query in **rank-compressed value space**.
3. **Difference-Array Algebra (Range Update / Point or Range Query)**: A range addition $+v$ on $[l, r]$ is two point updates on the discrete derivative array $D[i] = A[i] - A[i-1]$. Integrating once recovers point values $A[x] = \sum_{i=1}^{x} D[i]$; integrating twice via a linear combination of two Fenwick trees recovers range sums in $O(\log N)$.
4. **Fenwick Binary Lifting (Order Statistics)**: Because node $idx$ in a Fenwick tree stores the exact aggregate of the interval $(idx - \text{lowbit}(idx), idx]$ of length $2^k$, the array of Fenwick nodes *already forms a binary search tree* over prefixes. Searching for the smallest index whose prefix sum reaches a threshold $K$ takes $O(\log N)$ by descending bit powers from $\lfloor \log_2 N \rfloor$ down to $0$, avoiding an $O(\log^2 N)$ outer binary search.

### Decision Compass & Structural Lenses

| Structural Lens | Target Signature in Problem | Fenwick Domain & State | Why Plain Prefix / Other Tools Fail |
|---|---|---|---|
| **Invertible Prefix Group** | Point updates $A[i] \leftarrow A[i] \oplus \Delta$ and range queries $[l, r]$ with $(G, \oplus, \ominus)$ | 1-indexed array `bit[1..N]` over index space | Static prefix sums require $O(N)$ rebuild per update; general Segment Trees use $2\times$–$4\times$ more memory and instructions. |
| **Value-Space Sweep** | Count pairs $(i, j)$ with $i < j$ and $f(A[i]) \le g(A[j])$ | `bit[1..M]` over coordinate-compressed values | 2D brute force is $O(N^2)$; sorting alone destroys the original index order unless paired with a dynamic frequency structure. |
| **Dual-Fenwick Integration** | Add $+v$ to all $A[i]$ in $[l, r]$ and query $\sum_{i=l}^{r} A[i]$ | Two trees: `B1` storing $D[i]$ and `B2` storing $D[i] \cdot (i-1)$ | Naive range update touches $O(N)$ cells; a single difference Fenwick only supports point queries, not range sums. |
| **Bitwise Tree Walk** | Find $k$-th active slot or first prefix where $\sum_{i=1}^{p} A[i] \ge K$ $(A[i] \ge 0)$ | Direct bit-jumping `pos += 1 << b` on `bit[]` | Binary searching over `query(mid)` costs $O(\log^2 N)$; std::set lacks fast random-access rank arithmetic in standard C++. |




---

## Worked Problem 36.1: Resonance Triplets in a Particle Drift

### Problem & Constraints

A linear detector records $N$ particles arriving in sequence $i = 1, 2, ..., N$. The $i$-th particle has energy $E[i]$ and charge weight $W[i]$ ( $1 \le E[i] \le 10^9, 1 \le W[i] \le 10^6$). A **resonance triplet** is an ordered index triple $(i, j, k)$ with $1 \le i < j < k \le N$ such that the energies strictly increase ($E[i] < E[j] < E[k]$), and its **resonance yield** is the product $W[i] \cdot W[j] \cdot W[k]$. Compute the sum of resonance yields over all valid triplets $(i, j, k)$ modulo $10^9 + 7$.

- **Constraints**: $3 \le N \le 2 \cdot 10^5$, time limit $1.5$ s, memory $256$ MB.

**Pause & Reflect Before Reading On:**

- Which of the three indices $(i, j, k)$ is the natural "pivot" that separates the other two into independent choices?
- If you fix the middle index $j$, what single summary of all valid $i < j$ do you need, and why does compressing $E[1..N]$ allow a Fenwick tree to maintain it?

### Step 1 — Exact Model & Bottleneck

The exact specification sums over all triples of indices:

$$\text{Total} = \sum_{1 \le i < j < k \le N} [E[i] < E[j] < E[k]] \cdot W[i]W[j]W[k] \pmod{10^9 + 7}$$

Directly enumerating $(i, j, k)$ takes $\Theta(N^3)$ operations. Even if we notice that for a fixed middle index $j$, the choices of $i$ and $k$ are independent—

$$\text{Total} = \sum_{j=1}^{N} W[j] \cdot \left( \sum_{i < j,\ E[i] < E[j]} W[i] \right) \cdot \left( \sum_{k > j,\ E[k] > E[j]} W[k] \right)$$

—scanning left for $i$ and right for $k$ for each $j$ still costs $\Theta(N^2)$ time ( $\approx 4 \cdot 10^{10}$ operations for $N = 2 \cdot 10^5$). The bottleneck is the repeated linear scan over $i < j$ (and $k > j$) filtered by the 2D condition $(i < j \text{ and } E[i] < E[j])$.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Sorting by energy first without preserving index order.** If you sort particles by $E$, the condition $E[i] < E[j] < E[k]$ becomes trivial, but you must now enforce $i < j < k$, which is the exact same 2D problem in disguise. Worse, ignoring index order altogether counts triplets that arrived out of sequence.
- **Mistake 2: Off-by-one rank queries on duplicate energies.** Suppose $E = [5, 5, 5]$ and $W = [2, 3, 4]$. The condition $E[i] < E[j] < E[k]$ is *strict*, so the true answer is $0$. If a Fenwick tree queries prefix rank $\le \text{rank}(E[j])$ instead of $\text{rank}(E[j]) - 1$, or if equal energies are assigned distinct ranks during coordinate compression, the algorithm falsely accumulates previous particles with $E[i] = E[j]$ and outputs $2 \cdot 3 \cdot 4 = 24$.

### Step 3 — The Turn: Discovering the Structure

Fix the middle index $j$ and sweep $j$ from $1$ to $N$. At the moment the sweep reaches $j$, *only* particles $i \in \{1, ..., j-1\}$ have been inserted into our active multiset. That means the index condition $i < j$ holds automatically for everything inside the data structure!

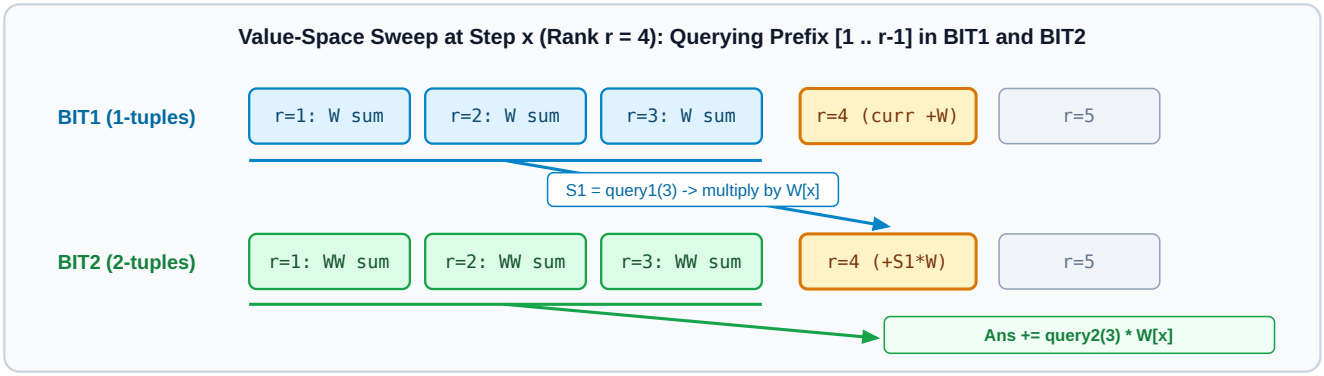
We only need to maintain an array indexed by **compressed energy rank** $r \in \{1, ..., M\}$ ($M \leq N$). In fact, instead of doing a separate backward pass for $k$, we can maintain a **two-stage Fenwick pipeline** in a single left-to-right pass: - $\mathrm{BIT}_1[r]$: sum of $W[i]$ for processed particles $i$ with $\mathrm{rank}(E[i]) = r$. - $\mathrm{BIT}_2[r]$: sum of $W[i] \cdot W[j]$ for processed pairs $(i < j)$ with $E[i] < E[j]$ and $\mathrm{rank}(E[j]) = r$.

**Two-Stage Fenwick Transition at Particle $x$ with Rank $r = \text{rank}(E[x])$:** $$\begin{aligned} S_1(x) &= \text{BIT}_1.\text{query}(r - 1) = \sum_{i < x,\, E[i] < E[x]} W[i] \\ S_2(x) &= \text{BIT}_2.\text{query}(r - 1) = \sum_{i < j < x,\, E[i] < E[j] < E[x]} W[i] W[j] \\ \text{Answer} &\gets \text{Answer} + S_2(x) \cdot W[x] \\ \text{BIT}_1.\text{add}(r,\; W[x]), &\quad \text{BIT}_2.\text{add}(r,\; S_1(x) \cdot W[x]) \end{aligned}$$

Why must we query $r-1$ *before* updating at $r$? Because even if we queried $r$, querying $r-1$ strictly excludes equal energies $E[i] = E[x]$ and prevents particle $x$ from pairing with itself in the same step.

### Step 4 — Progressive Construction & Visual Trace

1. **Coordinate Compression**: Collect all $E[i]$, sort, and remove duplicates via `std::unique`. Map each $E[i]$ to its 1-based rank $r \in [1, M]$.
2. **Single-Pass Sweep**: Initialize $\mathrm{BIT}_1$ and $\mathrm{BIT}_2$ of size $M$ with zeros. For each particle $x = 1...N$ with rank $r$ and weight $w$:
   - Query $s_1 = \mathrm{BIT}_1.\mathrm{query}(r-1)$ and $s_2 = \mathrm{BIT}_2.\mathrm{query}(r-1)$.
   - Add $s_2 \cdot w \pmod{10^9+7}$ to the total answer.
   - Update $\mathrm{BIT}_1.\mathrm{add}(r, w)$ and $\mathrm{BIT}_2.\mathrm{add}(r, s_1 \cdot w \bmod (10^9+7))$.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Necessity & Soundness (No Invalid Triplets Counted)**: Every term added to `Ans` at step $k$ has the form $W[k] \cdot (W[j] \cdot W[i])$ where $(i, j)$ was inserted into $\mathrm{BIT}_2$ strictly before step $k$ (so $j < k$) at rank $\mathrm{rank}(E[j]) \leq \mathrm{rank}(E[k]) - 1$ (so $E[j] < E[k]$). By induction on the insertion into $\mathrm{BIT}_2$ at step $j$, $i < j$ and $E[i] < E[j]$ also hold.
- **Sufficiency (No Valid Triplets Missed)**: Take any valid triplet $(i, j, k)$ with $i < j < k$ and $E[i] < E[j] < E[k]$. When the sweep is at $i$, $W[i]$ is added to $\mathrm{BIT}_1[\mathrm{rank}(E[i])]$. When the sweep reaches $j$, $\mathrm{rank}(E[i]) \leq \mathrm{rank}(E[j]) - 1$, so $W[i]$ is included in $S_1(j)$ and $W[i]W[j]$ is added to $\mathrm{BIT}_2[\mathrm{rank}(E[j])]$. When the sweep reaches $k$, $\mathrm{rank}(E[j]) \leq \mathrm{rank}(E[k]) - 1$, so $W[i]W[j]$ is included in $S_2(k)$ and multiplied by $W[k]$.
- **Small-Case Oracle**: Test on all $3^5 = 243$ arrays of length $N = 5$ with $E[i] \in \{1, 2, 3\}$ (forcing heavy duplicate collisions) against a 6-line $O(N^3)$ triple loop. Complexity: $O(N \log N)$ time and $O(N)$ memory.

---

## Worked Problem 36.2: Orbital Dock Allocation & Telemetry

### Problem & Constraints

A space station has $N$ sequential docking bays numbered $1$ to $N$. Initially, all bays are empty (occupancy $0$) and have radiation baseline $R[i] = 0$. Process $Q$ operations of three types:

1. `DOCK k`: Find the $k$-th currently **empty** bay from the left, mark it as occupied, and report its bay index $p$ (or $-1$ if fewer than $k$ empty bays exist).
2. `UNDOCK p`: Bay $p$ (currently occupied) becomes empty again.
3. `RADIATE l r v`: Add $+v$ to the radiation level $R[i]$ of every bay $i \in [l, r]$ (regardless of occupancy).
4. `SURVEY l r`: Report the total radiation $\sum_{i=l}^{r} R[i]$ across bays $l$ through $r$.

- **Constraints**: $1 \le N, Q \le 2 \cdot 10^5$, $|v| \le 10^6$, time limit $1.5$ s.

> **Pause & Reflect Before Reading On:**
> - In Operation 1, binary searching the answer with `query(mid) >= k` takes $O(\log^2 N)$. How do the dyadic block lengths of a Fenwick tree let you find $p$ in a single $O(\log N)$ top-down pass?
> - In Operations 3 and 4, if $D[i] = R[i] - R[i-1]$ is the difference array, write $\sum_{x=1}^{p} R[x]$ as a sum over $D[i]$ and isolate the coefficients of $D[i]$.

### Step 1 — Exact Model & Bottleneck

- For `DOCK k` / `UNDOCK p`: Maintain an indicator array $F[1..N] \in \{0, 1\}$ where $F[i] = 1$ means bay $i$ is empty. We need the smallest $p$ with $\sum_{i=1}^{p} F[i] = k$, followed by $F[p] \leftarrow 0$.
- For `RADIATE` / `SURVEY`: Maintain $R[1..N]$ under range addition and range sum.

Scanning $F$ or updating $R[l..r]$ element-by-element takes $O(N)$ per query, totaling $O(NQ)$. Even if we use a Fenwick tree on $F$, calling `query(mid)` inside a binary search performs $\log_2 N$ prefix queries of cost $\log_2 N$ each—$O(\log^2 N)$—when the tree structure itself already partitions prefixes into powers of two.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Lifting with `>= k` instead of `< k` during Fenwick binary lifting.** Suppose $F = [1, 1, 0, 1]$ and we search for the $k = 2$-nd empty bay (true answer: index $2$). If you jump whenever `sum + bit[pos + step] <= k`, then at `step = 2` you see `bit[2] = 2 <= 2` and jump to `pos = 2`; then at `step = 1`, `bit[3] = 0`, so `2 + 0 <= 2` and you jump again to `pos = 3` (an occupied bay!). We must jump only when `sum + bit[pos + step] < k`, landing at the longest prefix with sum $< k$, and then add $+1$.
- **Mistake 2: Using a single difference-array Fenwick tree for range sums.** A difference array $D[i]$ updated at $l$ and $r + 1$ gives the *point* value $R[x] = \sum_{i=1}^{x} D[i]$, not the prefix sum $\sum_{x=1}^{p} R[x]$.

### Step 3 — The Turn: Discovering the Structure

We combine two algebraic properties of Fenwick trees:

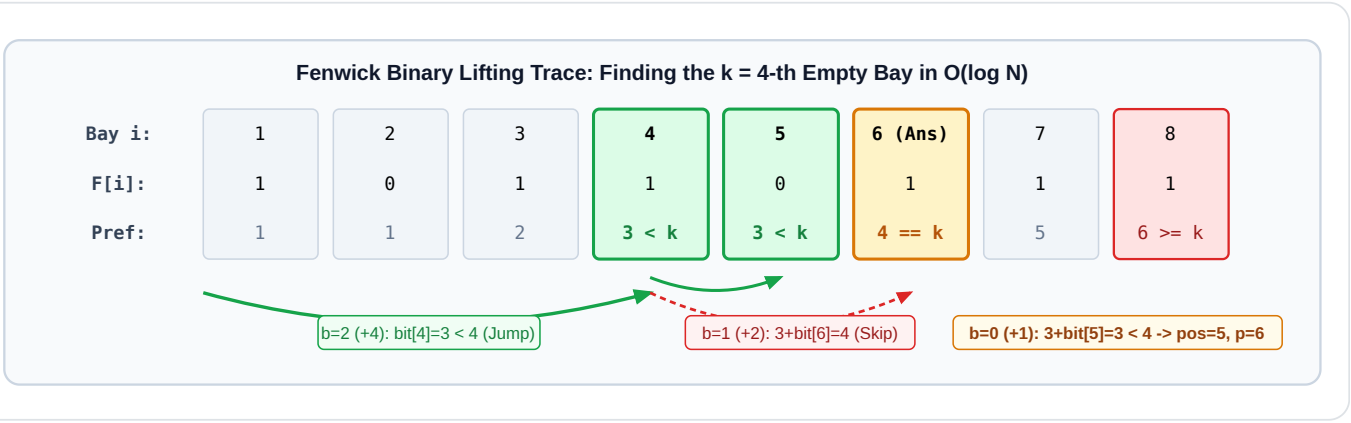
**1. Bitwise Tree Walk on $F[1..N]$**: Initialize `pos = 0` and `acc = 0`. Iterate bit powers $b = \lfloor \log_2 N \rfloor$ down to $0$. Consider extending `pos` by $2^b$: because `pos` is a multiple of $2^{b+1}$, the Fenwick entry `bitF[pos + 2^b]` covers *precisely* the interval $(\text{pos}, \text{pos} + 2^b]$! If `acc + bitF[pos + 2^b] < k`, that entire block of $2^b$ bays contains fewer than the remaining needed empty slots, so we safely advance `pos += 2^b` and `acc += bitF[pos]`. At the end, `pos + 1` is the exact first index reaching sum $k$.

**2. Dual-Fenwick Prefix Integration for Range Add / Range Sum**: Let $D[i] = R[i] - R[i-1]$ be the difference array, so a range add of $+v$ on $[l, r]$ only modifies $D[l] \mathrel{+}= v$ and $D[r+1] \mathrel{-}= v$. Now expand the prefix sum $\text{Pref}(p) = \sum_{x=1}^{p} R[x]$:

> **Dual-Fenwick Prefix Sum Algebra:** $$\text{Pref}(p) = \sum_{x=1}^{p} \sum_{i=1}^{x} D[i] = \sum_{i=1}^{p} D[i] \cdot (p - i + 1) = p \sum_{i=1}^{p} D[i] \;-\; \sum_{i=1}^{p} D[i]\cdot(i - 1)$$ Maintain $\text{BIT}_{D_1}[i] = D[i]$ and $\text{BIT}_{D_2}[i] = D[i] \cdot (i - 1)$. Both update at only two points ($l$ and $r+1$), and $\text{Pref}(p) = p \cdot \text{BIT}_{D_1}.\text{query}(p) - \text{BIT}_{D_2}.\text{query}(p)$.

### Step 4 — Progressive Construction & Visual Trace

1. **Initialization**: Set $F[1..N] = 1$. Build `bitF` in $O(N)$ by accumulating `bitF[i] += 1` and propagating `if (i + (i & -i) <= N) bitF[i + (i & -i)] += bitF[i]`.
2. **Operation** `DOCK k`: If `bitF.query(N) < k`, return `-1`. Otherwise, lift `pos` using bit steps $2^b$ where `acc + bitF[pos + (1 << b)] < k`. Set `p = pos + 1`, update `bitF.add(p, -1)`, and return `p`.
3. **Operation** `RADIATE l r v`:
   - $\text{BIT}_{D_1}.\text{add}(l, v), \text{BIT}_{D_1}.\text{add}(r+1, -v)$
   - $\text{BIT}_{D_2}.\text{add}(l, v \cdot (l-1)), \text{BIT}_{D_2}.\text{add}(r+1, -v \cdot r)$




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Lifting Invariant**: After inspecting bit $b$, `pos` is a multiple of $2^b$, `acc` equals $\sum_{i=1}^{\text{pos}} F[i] < k$, and $\sum_{i=1}^{\text{pos}+2^b} F[i] \geq k$. At $b = 0$, pos is the exact last index with prefix sum $< k$, and because $F[\text{pos}+1] \in \{0, 1\}$ brings the sum to $\geq k$, we must have $F[\text{pos}+1] = 1$ and $\sum_{i=1}^{\text{pos}+1} F[i] = k$.
- **Dual-Fenwick Identity**: A single update `RADIATE l r v` changes $R[x]$ by $0$ for $x < l$, by $v(x - l + 1)$ for $l \leq x \leq r$, and by $v(r - l + 1)$ for $x > r$. Evaluating $p \cdot \text{BIT}_{D_1}.\text{query}(p) - \text{BIT}_{D_2}.\text{query}(p)$ in all three regimes reproduces these exact piecewise formulas.
- **Oracle**: Compare against a brute-force vector of size $N = 50$ over $10^4$ random intermixed operations. Total runtime: $O((N+Q)\log N)$.

---

# Section 37 — Segment Trees

## The Discovery & Modeling Approach

Where a Fenwick tree relies on **prefix invertibility** ($[l, r] = [1, r] \ominus [1, l-1]$), a **Segment Tree** replaces subtraction with **explicit hierarchical decomposition**: any interval $[l, r]$ is partitioned into $O(\log N)$ disjoint canonical dyadic blocks, and their summaries are combined via an **associative monoid merge** $\mathrm{merge}(L, R)$.

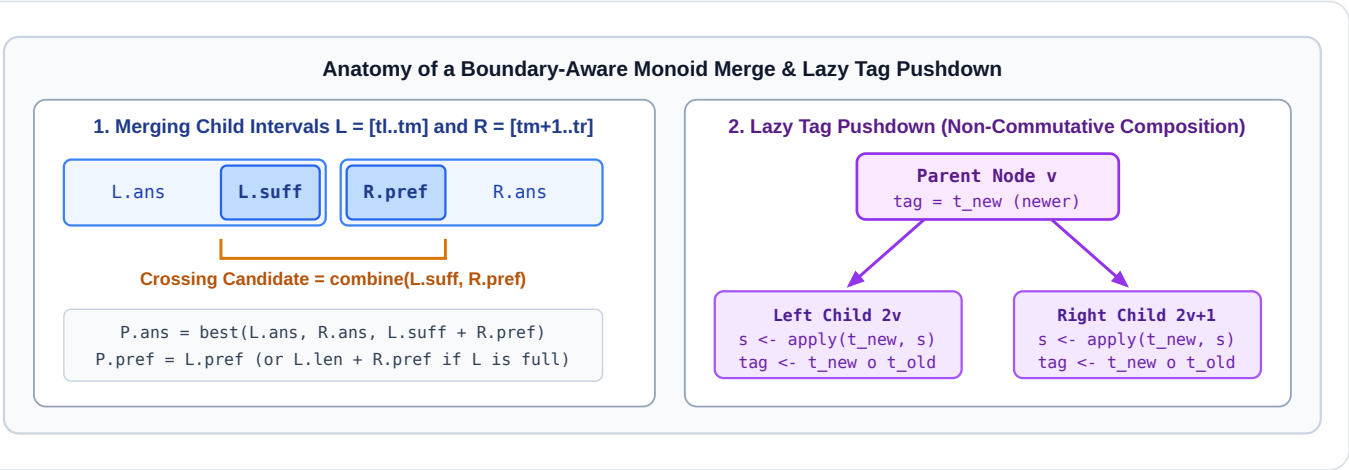
Consequently, whenever an operation lacks an inverse (such as $\min$, $\max$, $\gcd$, matrix multiplication, or maximum contiguous subarray sum), or whenever range updates must modify non-trivial node summaries via **Lazy Propagation**, the Segment Tree is the natural discovery target.

Designing a Segment Tree is a two-question algebraic exercise:

1. **The Node Monoid Contract**: *If I know the answer for left half $L$ and right half $R$, what extra fields must each node store so that $\mathrm{merge}(L, R)$ can be computed in $O(1)$ time without inspecting individual elements?* Whatever `merge(L, R)` reads on the right-hand side must be added to the node state and maintained recursively.
2. **The Lazy Endomorphism Contract**: *Given a pending range update tag $t$ on a node covering $[tl, tr]$, can I update the node's entire summary in $O(1)$ time using only $t$ and $(tr - tl + 1)$, and can I compose an incoming tag $t_{\text{new}}$ with an existing deferred tag $t_{\text{old}}$ into a single tag $t_{\text{new}} \circ t_{\text{old}}$?*

### Decision Compass & Structural Lenses

| Segment Tree Variant | Required Algebraic Property | Node & Tag Signature | Canonical Use Case |
|---|---|---|---|
| **Point Update, Monoid Range Query** | Associative operation $(S, \oplus)$ with identity $e$ (need not be commutative or invertible) | Node stores $s \in S$; no lazy tags | Range $\min/\max$, $2 \times 2$ matrix products, modular polynomial hashes. |
| **Boundary-Aware Non-Commutative Monoid** | Sub-patterns can cross the midpoint $m$ between $L$ and $R$ | Node stores `(ans, pref, suff, full_summary)` | Maximum subarray sum (GSS1), longest valid/monotonic/alternating subsegment. |
| **Lazy Propagation (Affine / Assignment)** | Tag set $T$ acts on $S$ ($t(a \oplus b) = t(a) \oplus t(b)$) and closes under composition $t_2 \circ t_1$ | Node stores $s \in S$ and deferred tag $t \in T$ | Range affine $x \leftarrow ax + b$ with range sum; range add with range $\min$ & count of minima. |
| **Segment Tree Walk** | Monotone predicate on prefixes/elements inside the tree | Descend left/right child in $O(\log N)$ using node summaries | First index $\geq l$ where $A[i] \geq X$; longest prefix with sum $\leq K$. |



## Worked Problem 37.1: Maximum Yield of a Ascending Pipeline

### Problem & Constraints

You manage a pipeline of $N$ compressor stations. Station $i$ has pressure rating $P[i]$ and energy yield $Y[i]$ ($1 \le P[i] \le 10^9$, $1 \le Y[i] \le 10^9$; note that yields are positive, while pressures may rise or fall). A contiguous subsegment of stations $[a, b]$ ($l \le a \le b \le r$) is **laminar** if its pressures are strictly increasing: $P[a] < P[a+1] < \ldots < P[b]$. Process $Q$ operations:

1. `SET i p y`: Update station $i$ to have pressure $P[i] = p$ and yield $Y[i] = y$.
2. `QUERY l r`: Within the sub-pipeline $[l, r]$, find the maximum total yield $\sum_{k=a}^{b} Y[k]$ of any laminar contiguous subsegment $[a, b] \subseteq [l, r]$.

- **Constraints**: $1 \le N, Q \le 2 \cdot 10^5$, time limit $1.5$ s.

> **Pause & Reflect Before Reading On:**
> - Suppose left child $L$ covers $[tl, tm]$ and right child $R$ covers $[tm+1, tr]$. Under what exact condition can a laminar subsegment cross from $L$ into $R$?
> - If a crossing segment is valid, what is its maximum yield, and when does $L$'s laminar prefix extend all the way into $R$?

### Step 1 — Exact Model & Bottleneck

In any query range $[l, r]$, the array breaks into maximal strictly increasing runs of $P$. Because all yields $Y[k] > 0$, the best laminar subsegment inside $[l, r]$ is simply the maximal strictly increasing run inside $[l, r]$ with the largest sum of $Y[k]$. Scanning $i$ from $l$ to $r$ takes $O(r - l + 1)$ per query, or $O(NQ)$ in the worst case. We need a representation of any interval $[tl, tr]$ that merges two adjacent intervals $L$ and $R$ in $O(1)$ time.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Storing only `(best_yield, first_P, last_P)` per node.** When $L.\text{last_P} < R.\text{first_P}$, the boundary merges, so we need the yield of the laminar suffix of $L$ and the laminar prefix of $R$—neither of which is necessarily equal to $L.\text{best}$ or $R.\text{best}$!
  - *Counterexample*: $L$ has $P = [10, 20, 5], Y = [100, 100, 10]$, and $R$ has $P = [8, 30, 40], Y = [20, 100, 100]$. Here $L.\text{last_P} = 5 < R.\text{first_P} = 8$, so the boundary merges to give $10 + 20 + 100 + 100 = 230$, which beats $L.\text{best} = 200$ and $R.\text{best} = 220$. Without storing `suff_yield` ($10$ for $L$) and `pref_yield` ($220$ for $R$), `merge(L, R)` cannot discover $230$.
- **Mistake 2: Assuming the merged prefix yield is always `L.pref_yield`.** If $L$ is *entirely* strictly increasing *and* $L.\text{last_P} < R.\text{first_P}$, then the laminar prefix of $L \cup R$ extends across the boundary into $R$'s laminar prefix! We must store a boolean flag `is_full` indicating whether the entire interval is strictly increasing.

### Step 3 — The Turn: Discovering the Structure

Write down what `merge(L, R)` needs on the right-hand side, and close the set of variables under self-dependency:

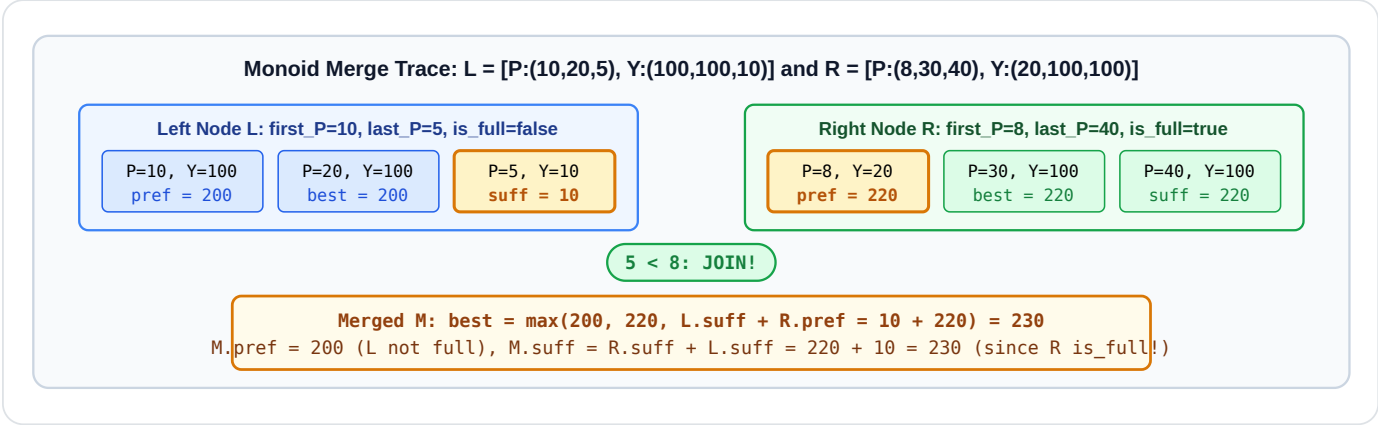
**Complete Monoid State for Interval $I = [tl, tr]$:** $$\text{Node} = (\text{best},\; \text{pref},\; \text{suff},\; \text{first_P},\; \text{last_P},\; \text{is_full})$$ Let $\text{can_join} = (L.\text{last_P} < R.\text{first_P})$. Then for $M = \text{merge}(L, R)$: $$\begin{aligned} M.\text{first_P} &= L.\text{first_P}, \qquad M.\text{last_P} = R.\text{last_P} \\ M.\text{is_full} &= L.\text{is_full} \land R.\text{is_full} \land \text{can_join} \\ M.\text{pref} &= L.\text{pref} + (\text{if } L.\text{is_full} \land \text{can_join} \text{ then } R.\text{pref} \text{ else } 0) \\ M.\text{suff} &= R.\text{suff} + (\text{if } R.\text{is_full} \land \text{can_join} \text{ then } L.\text{suff} \text{ else } 0) \\ M.\text{best} &= \max\!\big(L.\text{best},\; R.\text{best},\; \text{if } \text{can_join} \text{ then } L.\text{suff} + R.\text{pref} \text{ else } 0\big) \end{aligned}$$

Notice that no other fields are ever demanded: the state is **closed** under merging!

### Step 4 — Progressive Construction & Visual Trace

1. **Leaf Initialization**: For a single station $i$, `Node(Y[i], Y[i], Y[i], P[i], P[i], true)`.
2. **Point Update**: Descend to leaf $i$, replace its node, and recompute `tree[v] = merge(tree[2*v], tree[2*v+1])` on the $O(\log N)$ ancestors.

3. **Range Query** $[l, r]$: Do **not** return a single integer from recursive query calls; return the full `Node` struct for the intersection of the current subtree with $[l, r]$, and combine left and right query results using the exact same `merge(left_res, right_res)` function!




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Structural Induction**: Every laminar subsegment $[a, b] \subseteq L \cup R$ is either (1) wholly inside $L$ (bounded by $L.\text{best}$), (2) wholly inside $R$ (bounded by $R.\text{best}$), or (3) crosses the midpoint $tm \to tm + 1$. A crossing segment is laminar if and only if $[a, tm]$ is a laminar suffix of $L$, $[tm + 1, b]$ is a laminar prefix of $R$, and $P[tm] < P[tm + 1]$. Since $Y[k] > 0$, the maximal crossing laminar segment achieves the maximum yield $L.\text{suff} + R.\text{pref}$.
- **Complexity**: $O(N)$ build, $O(\log N)$ per update or query.

---

## Worked Problem 37.2: Reactor Coolant Valves (Lazy Tag Composition & Tree Walk)

### Problem & Constraints

An array of $N$ coolant valves has flow capacities $C[1..N] \geq 0$. Process $Q$ operations of three types:

1. `RESET l r v`: Set $C[i] \leftarrow v$ for all $i \in [l, r]$ ($v \geq 0$).
2. `BOOST l r d`: Add $+d$ to $C[i]$ for all $i \in [l, r]$ ($d \geq 0$).
3. `BREACH l X`: Find the smallest valve index $i \geq l$ such that $C[i] \geq X$, or report $-1$ if no such valve exists in $[l, N]$.

- **Constraints**: $1 \leq N, Q \leq 2 \cdot 10^5$, $0 \leq v, d \leq 10^9$, $1 \leq X \leq 10^{15}$, time limit $1.5$ s.

> **Pause & Reflect Before Reading On:**
> - If a node already has a pending `BOOST(+d1)` tag and receives a `RESET(v)` tag—or vice versa—do those two tags commute? How can one unified tag represent any sequence of `RESET` and `BOOST` operations?
> - To answer `BREACH l X`, what single number should each node store so we can walk down the segment tree in $O(\log N)$ time?

### Step 1 — Exact Model & Bottleneck

We maintain $C[1..N]$ under range assignment (`RESET`), range addition (`BOOST`), and first-element-at-least-$X$ queries on $[l, N]$. Naive range updates cost $O(N)$, and even with a lazy segment tree maintaining range maxima, binary searching the endpoint $r \in [l, N]$ such that $\max_{i \in [l, r]} C[i] \geq X$ calls range-max $O(\log N)$ times, costing $O(\log^2 N)$ per query. Walking the segment tree directly eliminates the extra $\log N$ factor.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Applying lazy `set` and `add` tags in a fixed order regardless of arrival time.**

- *Counterexample*: Valve 1 starts at $C[1] = 5$. We apply `BOOST(1, 1, +10)` (so $C[1] = 15$) and then `RESET(1, 1, 3)` (so $C[1]$ must become 3). If a child node stores separate `lazy_set` and `lazy_add` fields and pushes `lazy_set` *before* `lazy_add` without clearing `lazy_add` when `RESET` arrives, the child computes $3 + 10 = 13 \neq 3$!

- **Mistake 2: Exploring both subtrees during the tree walk.** If both children of a node have $\max \geq X$, you must *only* recurse into the left child (unless the left child lies completely before $l$, or returns $-1$ because its $\geq X$ elements all lie at indices $< l$).

### Step 3 — The Turn: Discovering the Structure

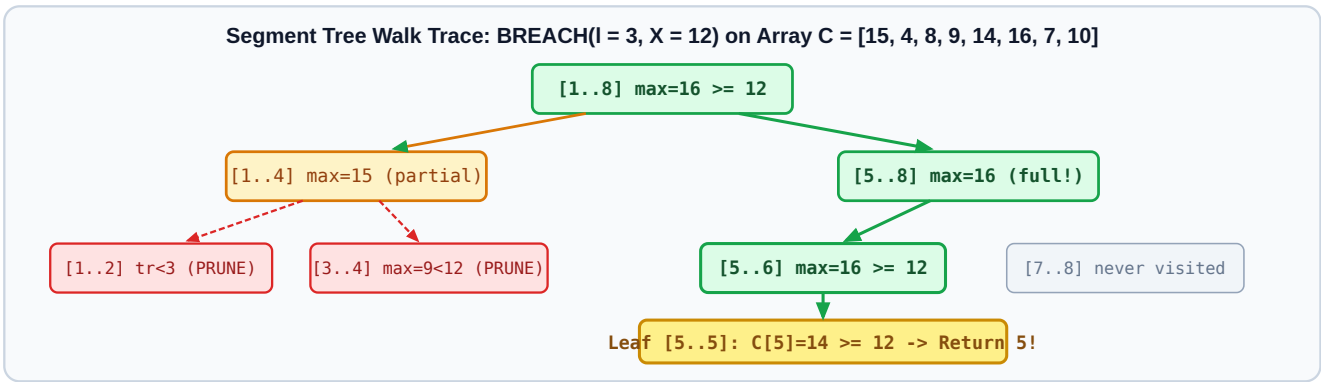
**1. Unified Lazy Tag (Optional Set + Add)**: Any arbitrary sequence of assignments $x \leftarrow v$ and additions $x \leftarrow x + d$ collapses into a two-field tag $(\text{has_set}, \text{set_val}, \text{add_val})$: - A `BOOST(+d)` never erases a previous `RESET`; it simply increments $\text{add_val} += d$. - A `RESET(v)` completely overwrites history: it sets $\text{has_set}=\text{true}$, $\text{set_val} = v$, and resets $\text{add_val} = 0$.

**Tag Application & Composition Algebra:** $$\text{apply}(\text{tag}, \text{node_max}) = \begin{cases} \text{tag.set_val} + \text{tag.add_val} & \text{if tag.has_set} \\ \text{node_max} + \text{tag.add_val} & \text{otherwise} \end{cases}$$ $$\text{compose}(\text{new}, \text{old}) = \begin{cases} (\text{true},\; \text{new.set_val},\; \text{new.add_val}) & \text{if new.has_set} \\ (\text{old.has_set},\; \text{old.set_val},\; \text{old.add_val} + \text{new.add_val}) & \text{otherwise} \end{cases}$$

**2. Segment Tree Walk (`find_first(v, tl, tr, l, X)`)**: - Base pruning: if $tr < l$ or $\text{tree}[v].\max < X$, no valid index exists in this subtree; return $-1$ in $O(1)$! - Leaf reached ($tl == tr$): return $tl$. - Otherwise, `push(v)`, try left child `res = find_first(2*v, tl, tm, l, X)`. If `res != -1`, return `res` immediately without touching the right child; otherwise return `find_first(2*v+1, tm+1, tr, l, X)`.

### Step 4 — Progressive Construction & Visual Trace

Why is `find_first` guaranteed to run in $O(\log N)$ even when it backtracks from a left child? - A node $v$ is either **partially overlapping** $[l, N]$ (meaning $tl < l \leq tr$, which happens for at most one node per tree level, i.e., $O(\log N)$ nodes total) or **fully contained** inside $[l, N]$ (meaning $tl \geq l$). - Once `find_first` enters a **fully contained** node $v$ with $\text{tree}[v].\max \geq X$, the answer is *guaranteed* to exist inside $v$'s subtree! From that moment on, it walks straight down to a leaf in at most $\log_2 N$ steps without ever backtracking.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Amortized & Worst-Case Bound**: In `find_first`, at most one node per depth has $tl < l \leq tr$. Every such boundary node spawns at most two children: one left child (which either has $tr < l$ and returns in $O(1)$, or is the new boundary node) and one right child (which has $tl \geq l$). The *first* right child with $tl \geq l$ and $\max \geq X$ descends straight to a leaf and terminates the entire recursion. Hence at most $4\log_2 N$ nodes are visited.
- **Oracle**: Verify on random sequences of `RESET`, `BOOST`, and `BREACH` with $N = 30$ against a direct array loop.

---

# Section 38 — Static Queries and Binary Lifting

## The Discovery & Modeling Approach

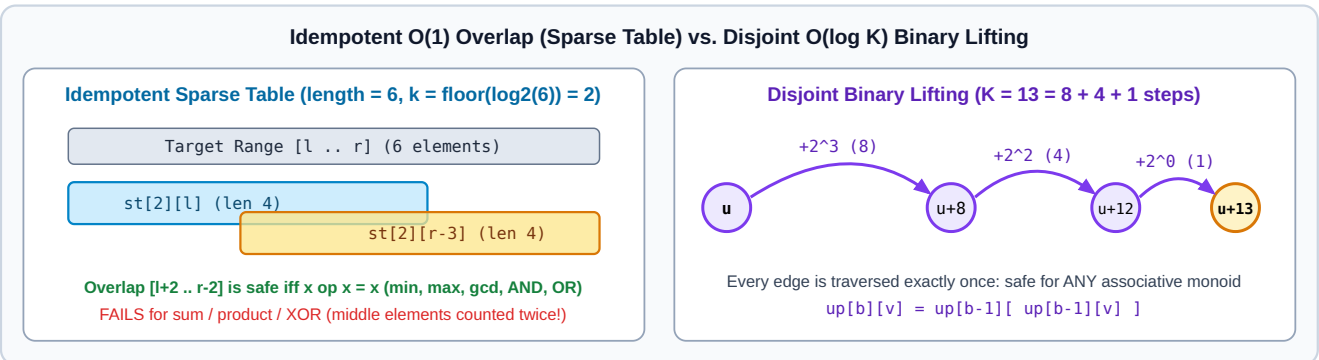
When the underlying array or transition graph is **static** (no interleaving updates), maintaining a dynamic tree is unnecessary overhead. Instead, we precompute answers over **power-of-two blocks** of length $2^0, 2^1, \ldots, 2^K$ ($K = \lfloor \log_2 N \rfloor$).

This single idea splits into two major paradigms depending on whether the query operation is **idempotent** ($x \oplus x = x$):

1. **Idempotent Sparse Table ($O(1)$ Range Queries)**: If $x \oplus x = x$ (true for $\min$, $\max$, $\gcd$, $\mathrm{lcm}$, bitwise $\mathrm{AND}$, and bitwise $\mathrm{OR}$), then two overlapping blocks of length $2^k$ covering $[l, l + 2^k - 1]$ and $[r - 2^k + 1, r]$ (where $k = \lfloor \log_2(r - l + 1) \rfloor$) have union $[l, r]$, and the double-counted middle overlap does not corrupt the answer! Thus any range query is answered in $O(1)$ **time** with a single table lookup on two intervals.
2. **Disjoint Binary Lifting ($O(\log K)$ Jumps & Path Aggregates)**: If the operation is **not idempotent** (such as range sum, matrix multiplication, or composing $K$ steps along a functional graph $u \to \mathrm{next}[u]$ where $K \le 10^{18}$), overlapping blocks would double-count elements. Instead, we decompose the length or step count $K$ into its **disjoint binary bits** $K = \sum_b 2^b$ and chain non-overlapping jumps of length $2^b$ in $O(\log K)$ time.

### Decision Compass & Structural Lenses

| Paradigm | Algebraic Requirement | Precomputation | Query Time & Mechanism |
|---|---|---|---|
| **Idempotent Sparse Table** | Associative + **Idempotent** ($a \oplus a = a$): $\min, \max, \gcd, \&, \|$ | $O(N \log N)$ time & space: `st[k][i]` covers $[i, i + 2^k - 1]$ | $O(1)$: `op(st[k][l], st[k][r - (1<<k) + 1])` with $k = \lfloor \log_2(r - l + 1) \rfloor$. |
| **Disjoint Sparse Table / Bit Decomposition** | Associative only (e.g., $+$, $\times$, matrix product) | $O(N \log N)$ table | $O(\log N)$ by summing `st[b][pos]` for set bits $b$ of $(r - l + 1)$. |
| **Functional Graph Binary Lifting** | Out-degree 1 transition $f: V \to V$+ path monoid $(S, \oplus)$ | $O(N \log K_{\max})$: `up[b][u]` and `agg[b][u]` over $2^b$ edges | $O(\log K)$: advance `u = up[b][u]` for set bits of $K$, or lift while a path predicate holds. |




---

## Worked Problem 38.1: Harmonic Lock Intervals (Idempotent Sparse Table + Monotone Search)

### Problem & Constraints

Given a static array of $N$ positive integers $A[1..N]$ ($1 \le A[i] \le 10^9$), a subsegment $[l, r]$ is **harmonically locked** if the greatest common divisor of its elements equals its minimum element:

$$\gcd(A[l], A[l+1], \ldots, A[r]) = \min(A[l], A[l+1], \ldots, A[r])$$

For each starting index $l \in \{1, \ldots, N\}$, find the **farthest** right endpoint $r \ge l$ such that $[l, r]$ is harmonically locked, and also answer $Q$ queries `(L, R)` asking whether the specific interval $[L, R]$ is harmonically locked.

- **Constraints**: $1 \le N \le 2 \cdot 10^5$, $1 \le Q \le 5 \cdot 10^5$, time limit $1.5$ s.

> **Pause & Reflect Before Reading On:**
> - Why are both $\gcd$ and $\min$ overlap-safe for an $O(1)$ Sparse Table query?
> - For a fixed left endpoint $l$, can $\gcd(A[l..r]) = \min(A[l..r])$ ever become false at $r$ and then become true again at some $r' > r$? Wait—test $A = [6, 4, 2]$ from $l=1$!

### Step 1 — Exact Model & Bottleneck

Checking a single interval $[L, R]$ requires comparing $\gcd(A[L..R])$ and $\min(A[L..R])$. Doing this naively takes $O(R - L + 1)$ per query. Moreover, finding the farthest $r$ for every $l$ by testing all $r \ge l$ takes $\Theta(N^2)$ time.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Assuming the predicate $\gcd(A[l..r]) = \ = \min(A[l..r])$ is monotone in $r$ for a fixed $l$.**
  - *Counterexample*: Let $A = [6, 4, 2]$ and $l = 1$.
    - For $r = 1$: $\gcd(6) = 6 = \min(6)$ (**True**).
    - For $r = 2$: $\gcd(6, 4) = 2 \ne \min(6, 4) = 4$ (**False**!).
    - For $r = 3$: $\gcd(6, 4, 2) = 2 = \min(6, 4, 2)$ (**True again**!). Binary searching directly on whether $\gcd(A[l..r]) = \ = \min(A[l..r])$ is completely invalid because a smaller multiple of the GCD can appear later and pull $\min$ down to match $\gcd$!
- **Mistake 2: Using `std::log2` floating-point calls inside $5 \cdot 10^5$ queries**, or using $\gcd$ inside a Segment Tree when $Q = 5 \cdot 10^5$ (where $O(\log N)$ per query multiplied by $O(\log(\max A))$ per GCD risks TLE).

### Step 3 — The Turn: Discovering the Structure

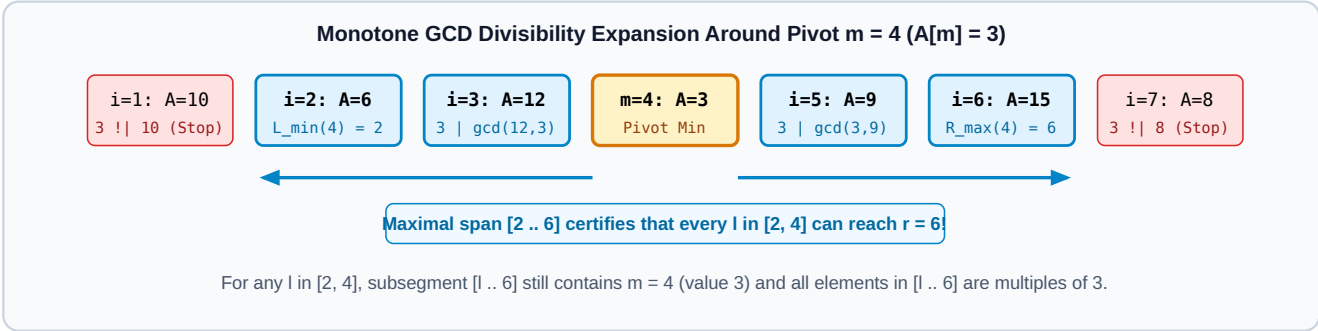
First, notice that $\gcd(A[L..R]) = \min(A[L..R])$ holds if and only if **every element in $A[L..R]$ is divisible by $A[L..R]$'s minimum**, which is equivalent to saying the minimum element *actually divides* $\gcd(A[L..R])$. Since $\gcd(S) \le \min(S)$ is always true for positive integers, equality holds iff $\gcd(A[L..R]) = \min(A[L..R])$. With two Sparse Tables (`st_gcd` and `st_min`), each explicit `[L, R]` query is answered in $O(1)$!

Second, why did monotonicity fail on $A = [6, 4, 2]$? Because the minimum was $A[3] = 2$, which wasn't even inside $[1, 2]$! Instead of fixing $l$ and guessing $r$, **fix the position $m$ of the minimum element $A[m]$**: - Any subsegment $[l, r]$ that achieves $\gcd(A[l..r]) = \min(A[l..r]) = A[m]$ must contain $m$ ($l \le m \le r$) and have **every element divisible by** $A[m]$, i.e., $A[m] \mid \gcd(A[l..r])$. - Now, as we expand $l$ leftward from $m$ and $r$ rightward from $m$, the condition "$A[m]$ divides $\gcd(A[l..m])$" (and "$A[m]$ divides $\gcd(A[m..r])$") **IS 100% monotone**! Adding more elements can only make the GCD stop being a multiple of $A[m]$; it can never become a multiple of $A[m]$ again once a non-multiple is included.

> **Idempotent Sparse Table & Monotone Expansion from Minimum $m$:** $$\begin{aligned} \text{query_gcd}(L, R) &= \gcd\!\big(\text{st_gcd}[k][L],\; \text{st_gcd}[k][R - 2^k + 1]\big), \quad k = \lfloor \log_2(R - L + 1) \rfloor \\ \text{For each } m \in [1, N]: &\quad L_{\min}(m) = \min\{l \le m : A[m] \mid \text{query_gcd}(l, m)\} \\ &\quad R_{\max}(m) = \max\{r \ge m : A[m] \mid \text{query_gcd}(m, r)\} \end{aligned}$$ Then $[L_{\min}(m), R_{\max}(m)]$ is the maximal valid interval with minimum $A[m]$, so for every $l \in [L_{\min}(m), m]$ we can reach up to $R_{\max}(m)$!

### Step 4 — Progressive Construction & Visual Trace

1. **Precompute** `lg[1..N]` (`lg[i] = lg[i/2] + 1`), `st_gcd[k][i]`, and `st_min[k][i]` in $O(N \log N)$.
2. **Binary Search Maximal Divisible Span Around Each $m$**: For each $m = 1 \ldots N$, binary search $L_{\min}(m) \in [1, m]$ and $R_{\max}(m) \in [m, N]$ using $O(1)$ `query_gcd` calls.
3. **Propagate Farthest Right Endpoint**: Each $m$ certifies that every starting index $l \in [L_{\min}(m), m]$ can reach at least $R_{\max}(m)$. Record `best_r[L_min(m)] = max(best_r[L_min(m)], R_max(m))` and sweep $l = 1 \ldots N$ with a running prefix maximum `ans[l] = max(ans[l-1], best_r[l])` (since if $[L_{\min}(m), R_{\max}(m)]$ is divisible by $A[m]$ and $l \le m$, the subsegment $[l, R_{\max}(m)]$ still contains $A[m]$ and is divisible by $A[m]$, so its GCD and minimum both equal $A[m]$!).




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Equivalence**: $[l, r]$ is harmonically locked with minimum $A[m]$ ($m \in [l, r]$) $\Leftrightarrow$ every element in $[l, r]$ is a multiple of $A[m]$ $\Leftrightarrow L_{\min}(m) \le l \le m \le r \le R_{\max}(m)$. Thus the maximum valid $r$ for a given $l$ is $\max\{R_{\max}(m) : L_{\min}(m) \le l \le m\}$.
- **Complexity**: $O(N \log N \log(\max A))$ preprocessing and $O(1)$ per `[L, R]` query.

---

## Worked Problem 38.2: Autonomous Relay Hops over $10^{18}$ Cycles

### Problem & Constraints

A network of $N$ orbital relays is arranged in a directed functional graph: in one cycle, a packet at relay $u$ is forwarded to a unique relay $\text{next}[u]$, consuming $\text{cost}[u]$ energy and experiencing a peak thermal spike of $\text{heat}[u]$. Answer $Q$ queries of two types:

1. `SIMULATE u K`: Starting at relay $u$, trace $K$ cycles ($1 \le K \le 10^{18}$) and report the final relay reached, the total energy consumed, and the maximum thermal spike along those $K$ transitions.
2. `BUDGET u E`: Starting at relay $u$ with energy budget $E$ ($1 \le E \le 10^{18}$, where $\text{cost}[v] \ge 1$ for all $v$), find the maximum number of complete hops $K$ the packet can make before total energy consumed exceeds $E$.

- **Constraints**: $1 \le N, Q \le 10^5$, $1 \le \text{cost}[u] \le 10^9$, $1 \le \text{heat}[u] \le 10^9$, time limit $1.5$ s.

> **Pause & Reflect Before Reading On:**
> - Since $K \le 10^{18} < 2^{60}$, how many binary lifting levels $b$ are needed?
> - When combining two jumps of length $2^{b-1}$ from $u$, why must you read $\text{cost}[b-1][\text{up}[b-1][u]]$ before updating $\text{up}[b][u]$?

### Step 1 — Exact Model & Bottleneck

Simulating $K$ hops one step at a time takes $O(K)$ operations, which is impossible for $K = 10^{18}$. Even cycle detection (tortoise and hare) is cumbersome when $Q$ queries start at arbitrary vertices inside trees feeding into different cycles and ask for path maxima and budget thresholds. Because out-degree is $1$ everywhere, the $2^b$-th successor of any vertex $u$ is uniquely determined!

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Updating `u = up[b][u]` before accumulating `sum_cost` and `max_heat` during a query.** If you write `u = up[b][u]; total += cost_up[b][u];`, you add the cost of the *next* $2^b$ block starting at the new $u$ instead of the block you just traversed!
- **Mistake 2: Overflowing `long long` when adding two $2^{59}$-length costs.** With $b = 60$ and $\text{cost}[v] = 10^9$, a jump of $2^{60}$ steps has cost $10^9 \cdot 2^{60} \approx 10^{27}$, which overflows 64-bit signed integers! For `BUDGET u E` where $E \le 10^{18}$ and $\text{cost}[v] \ge 1$, the answer $K$ can never exceed $E \le 10^{18}$, and we must cap stored costs at $\text{INF} = 2 \cdot 10^{18}$ to prevent 64-bit overflow when building the table.

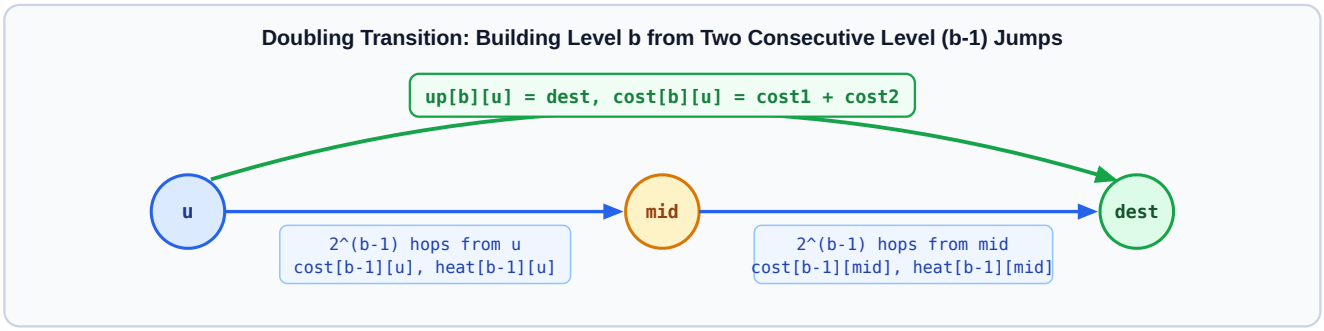
### Step 3 — The Turn: Discovering the Structure

For each level $b \in [0, 59]$ and each vertex $u \in [1, N]$, precompute: - $\text{up}[b][u]$: the destination after $2^b$ hops from $u$. - $\text{sum_cost}[b][u]$: the total energy over those $2^b$ hops (or capped at $\text{INF}$ for budget queries, or modulo/128-bit if exact sum is requested; since $K \leq 10^{18}$ and $\text{cost} \leq 10^9$, `__int128` holds exact sums up to $10^{27}$ effortlessly). - $\text{max_heat}[b][u]$: the maximum thermal spike over those $2^b$ hops.

**Binary Lifting Doubling Transition ($b = 1 \dots 59$):** $$\begin{aligned} \text{mid} &= \text{up}[b-1][u] \\ \text{up}[b][u] &= \text{up}[b-1][\text{mid}] \\ \text{sum_cost}[b][u] &= \text{sum_cost}[b-1][u] + \text{sum_cost}[b-1][\text{mid}] \\ \text{max_heat}[b][u] &= \max\!\big(\text{max_heat}[b-1][u],\; \text{max_heat}[b-1][\text{mid}]\big) \end{aligned}$$

For `BUDGET u E`, iterate $b = 59$ down to $0$: if $\text{sum_cost}[b][u] \leq E$, take the jump ($E -= \text{sum_cost}[b][u]$, $K += 2^b$, $u \leftarrow \text{up}[b][u]$).

### Step 4 — Progressive Construction & Visual Trace




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Unique Binary Representation**: Every integer $K \in [0, 2^{60} - 1]$ has a unique binary expansion $K = \sum_{b \in B} 2^b$, and the path of length $K$ from $u$ partitions into $|B|$ contiguous segments of lengths $2^b$. Because addition and $\max$ are associative, combining the precomputed summaries of those $|B|$ segments equals the step-by-step aggregate over all $K$ edges.
- **Monotonicity of Budget Lifting**: Since $\text{cost}[v] \geq 1 > 0$, total energy is strictly increasing in the number of hops $K$, so greedy binary lifting from $b = 59$ down to $0$ finds the exact largest $K$ with cost $\leq E$.
- **Complexity**: $O(N \log K_{\max})$ preprocessing time/memory ( $\approx 60 \times 10^5$ states) and $O(\log K_{\max})$ per query.

---

# Section 39 — Heaps and Priority Queues

## The Discovery & Modeling Approach

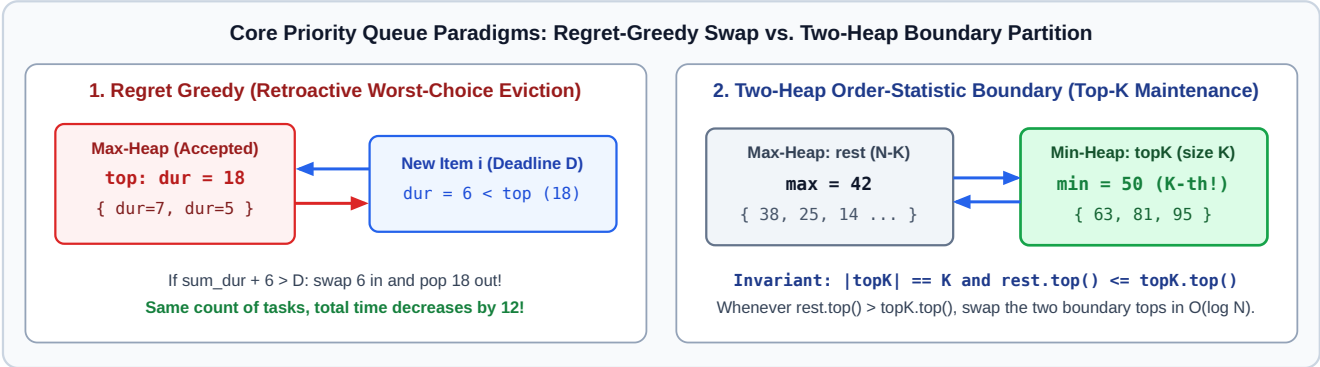
A **Priority Queue (Binary Heap)** is far more than a sorting replacement: it is a **dynamic extreme-value oracle** over an evolving set of candidates. In competitive programming discovery, a heap appears whenever time/index order conflicts with value/rank order, forcing an algorithm to maintain the *best available decision* or *worst revocable commitment* online.

Three structural patterns govern almost all non-trivial heap reductions:

1. **Regret Greedy (Provisional Commitment + Retroactive Swap)**: Many scheduling and resource problems look like dynamic programming ($O(N^2)$ or $O(N \cdot T)$), yet greedy commitment alone fails because an early choice may block a better later choice. **Regret Greedy** processes items in deadline/position order, provisionally accepts the current item if capacity allows, and—if capacity is violated—uses a heap of *already accepted items* to **swap out the single worst past commitment** (e.g., the accepted task with the largest duration or smallest profit), restoring feasibility while keeping the maximum number of items at minimum total cost!
2. **Two-Heap Rank / Quantile Partition**: A single heap exposes only the global minimum or maximum. To track the $K$**-th largest element** or **running median**, partition the active set into two heaps sharing a boundary: a min-heap `topK` holding the $K$ largest active values, and a max-heap `rest` holding all remaining values. Rebalancing their sizes and comparing `rest.top()` against `topK.top()` maintains the exact $K$-th order statistic in $O(\log N)$ per insertion or lazy deletion.
3. **Best-First State Space Expansion ($K$-th Smallest Configuration)**: When asked for the $K$-th smallest sum, subset, or tuple from an exponentially large space ($2^N$ or $N^M$) with $K \leq 2 \cdot 10^5$, design a **branching DAG** on states where every state generates at most $O(1)$ successor states in non-decreasing cost order, and pop the min-heap $K$ times in $O(K \log K)$.

### Decision Compass & Structural Lenses

| Heap Pattern | Bottleneck Replaced | Heap Content & Ordering | Invariant Maintained |
|---|---|---|---|
| **Regret Greedy** | $O(N \cdot C)$ knapsack-like DP when item count / values are unit or exchange-symmetric | Accepted items ordered by "most regretful" (e.g., max duration or min profit) | At every prefix $i$, heap holds a maximum-cardinality feasible subset with minimum resource consumption. |
| **Two-Heap Boundary** | $O(N)$ re-sorting or balanced BST overhead for dynamic $K$-th element / median | `L` (max-heap of smaller part) and `R` (min-heap of top $K$ part) | $\|R\| = K$ and $\max(L) \leq \min(R)$ at all times; sums of both halves maintained in $O(1)$. |
| **Best-First Successor DAG** | Generating all $N^2$ or $2^N$ configurations to sort and pick the first $K$ | Frontier states `(cost, state_descriptor)` in a min-heap | Every unvisited state is reachable from some state currently in the heap via $\geq 0$ cost edges; no state is pushed twice. |
| **Lazy Deletion Heap** | Arbitrary-element removal from `std::priority_queue` | Pair `(value, id)` + external `is_active[id]` or secondary `erased` heap | Top of heap is popped and discarded whenever `!is_active[top.id]` before reading `.top()`. |




---

## Worked Problem 39.1: Emergency Satellite Patch Scheduling (Regret Greedy)

### Problem & Constraints

A ground station must transmit software patches to a passing satellite. There are $N$ candidate patches; patch $i$ requires $T[i]$ seconds of uninterrupted transmission time and must be **completely finished** on or before deadline $D[i]$ ($1 \le T[i], D[i] \le 10^9$). The station can transmit at most one patch at a time starting from $t = 0$. Find the **maximum number of patches** that can be successfully transmitted before their respective deadlines, and among all maximum-size schedules, the minimum total transmission time required.

- **Constraints**: $1 \le N \le 2 \cdot 10^5$, time limit $1.0$ s.

> **Pause & Reflect Before Reading On:**
> - If you have selected a subset of patches that CAN all be completed before their deadlines, in what order should they always be executed?
> - When you consider the next patch in Earliest-Deadline-First order and adding it exceeds its deadline $D[i]$, why is evicting the single longest patch among $\{ \text{accepted} \cup \{i\} \}$ guaranteed to be both feasible and optimal?

### Step 1 — Exact Model & Bottleneck

First, establish the **canonical schedule order** (the Jackson / Earliest-Deadline-First lemma): if any subset $S$ of patches can be scheduled feasibly, then scheduling $S$ in non-decreasing order of deadlines $D[i_1] \le D[i_2] \le \ldots \le D[i_k]$ is also feasible (if two adjacent tasks in a feasible schedule are out of deadline order, swapping them only moves the tighter deadline earlier without changing the completion time of the pair).

After sorting patches by $D[1] \le D[2] \le \ldots \le D[N]$, the exact DP state is:

$$\mathrm{dp}[i][c] = \text{minimum total duration of } c \text{ feasible patches chosen from } 1..i$$

with transition $\mathrm{dp}[i][c] = \min(\mathrm{dp}[i-1][c],\ \mathrm{dp}[i-1][c-1] + T[i] \text{ if } \mathrm{dp}[i-1][c-1] + T[i] \le D[i])$. This DP takes $\Theta(N^2)$ time, which is too slow for $N = 2 \cdot 10^5$.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Greedily sorting by shortest duration $T[i]$ first.**
  - *Counterexample*: Patch A has $(T = 2, D = 10)$, Patch B has $(T = 3, D = 3)$. Shortest-first schedules A on $[0, 2]$, leaving B to finish at $2 + 3 = 5 > 3$ (only $1$ patch completed). Executing B on $[0, 3]$ and A on $[3, 5]$ completes *both*!
- **Mistake 2: Greedily sorting by deadline $D[i]$ and simply skipping any patch that doesn't fit.**
  - *Counterexample*: Patch 1 has $(T = 10, D = 10)$, Patch 2 has $(T = 5, D = 11)$, Patch 3 has $(T = 5, D = 12)$. Accepting Patch 1 consumes all $10$ seconds, so neither Patch 2 nor Patch 3 fits (score $1$). If we regret accepting Patch 1 when Patch 2 arrives, we drop Patch 1 $(T = 10)$ in favor of Patch 2 $(T = 5)$ and then also fit Patch 3 $(T = 5)$, achieving $2$ patches!

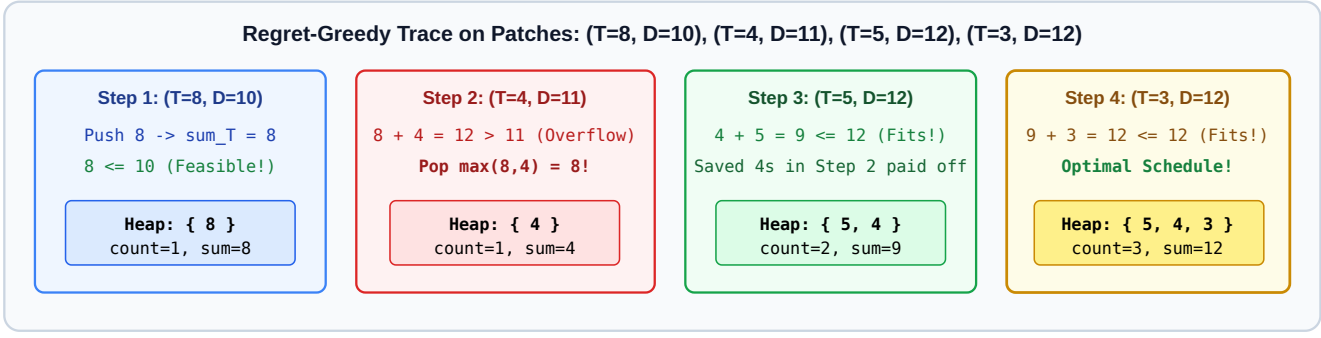
### Step 3 — The Turn: Discovering the Structure

In the DP $\mathrm{dp}[i][c]$, notice that at any prefix $i$, we only ever care about the **largest achievable count** $c^*$ and its **minimum total duration** $\mathrm{sum_T} = \mathrm{dp}[i][c^*]$! Why? Suppose processing patch $i$ allows us to increment $c^*$ by 1 ($\mathrm{sum_T} + T[i] \le D[i]$). We simply push $T[i]$ into our accepted set.

What if $\mathrm{sum_T} + T[i] > D[i]$? Then we cannot reach count $c^* + 1$ at step $i$. To give future patches $j > i$ (which all have $D[j] \ge D[i]$) the maximum slack, we want the subset of size $c^*$ with the **smallest possible sum of durations**. By inserting $T[i]$ into our max-heap of accepted durations and immediately popping the **maximum duration** $T_{\max} = \mathrm{pq.top}()$: 1. The size of the accepted set remains $c^*$. 2. Because $T_{\max} \ge T[i]$, the new total duration $\mathrm{sum_T}' = \mathrm{sum_T} + T[i] - T_{\max} \le \mathrm{sum_T}$ does not increase! 3. Every remaining accepted patch $x < i$ still finishes no later than it did before, and if $T[i]$ remains in the heap (meaning $T_{\max} > T[i]$ was evicted), patch $i$ finishes at $\mathrm{sum_T}' \le \mathrm{sum_T} \le D[\mathrm{prev}] \le D[i]$, so the new schedule is **100% feasible**!

**Regret-Greedy Transition at Patch $i$ (sorted by $D[1] \le \dots \le D[N]$):** $$\begin{aligned} &\text{Push } T[i] \text{ into max-heap } H; \quad \text{sum_T} \gets \text{sum_T} + T[i] \\ &\text{If } \text{sum_T} > D[i]: \quad \text{sum_T} \gets \text{sum_T} - H.\text{top}(); \quad H.\text{pop}() \end{aligned}$$ After processing $i = 1 \dots N$, $|H|$ is the maximum number of patches and $\text{sum_T}$ is the minimum total time.

### Step 4 — Progressive Construction & Visual Trace




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Exchange Argument by Induction**: Claim: After step $i$, the heap $H_i$ has the maximum possible cardinality $k = |H_i|$ among all feasible subsets of $\{1, \dots, i\}$, and among all feasible subsets of size $k$, $H_i$ has the minimum possible duration sum (in fact, $H_i$ majorizes any other feasible $k$-subset elementwise). If an optimal subset $O_i$ at step $i$ differs from $H_i$, the largest element evicted by our algorithm was strictly $\ge$ the largest element in $O_i \setminus H_i$ and had an earlier-or-equal deadline, so swapping it preserves feasibility while reducing or maintaining sum.
- **Complexity**: $O(N \log N)$ time and $O(N)$ space.

---

## Worked Problem 39.2: $K$-th Lowest-Energy Multi-Layer Quantum Configuration

### Problem & Constraints

A quantum system consists of $N$ independent layers ($1 \le N \le 10^5$). Layer $i$ has $M_i \ge 1$ discrete energy states sorted in non-decreasing order: $E_{i,1} \le E_{i,2} \le \dots \le E_{i,M_i}$ ($1 \le E_{i,j} \le 10^9$, $\sum M_i \le 2 \cdot 10^5$). Choosing exactly one state $j_i \in [1, M_i]$ from each layer $i$ produces a configuration $(j_1, j_2, \dots, j_N)$ with total energy $\sum_{i=1}^{N} E_{i,j_i}$. Output the total energies of the $K$ lowest-energy configurations in non-decreasing order ($1 \le K \le 2 \cdot 10^5$).

- **Constraints**: $1 \le N, K \le 2 \cdot 10^5$, $\sum M_i \le 2 \cdot 10^5$, time limit $1.5$ s.

**Pause & Reflect Before Reading On:**

- If you represent a configuration by its offset from the ground state $S_0 = \sum_{i=1}^N E_{i, 1}$, why must you discard layers with $M_i = 1$ and sort the remaining layers by their **first upgrade cost** $\Delta_i = E_{i, 2} - E_{i, 1}$?
- If a state in the priority queue could advance any of the $N$ layers, each pop would push $N$ children ($O(KN \log(KN))$—TLE!). How do you restrict each state to push at most **3** children while still visiting every configuration exactly once?

### Step 1 — Exact Model & Bottleneck

There are $\prod_{i=1}^N M_i$ total configurations, which can be as large as $2^{10^5}$. However, $K \le 2 \cdot 10^5$ is small! We only need to explore the $K$ smallest states in a **Best-First Search (Dijkstra on an implicit state tree)** where: 1. The root is the global minimum configuration $S_0 = \sum_{i=1}^N E_{i,1}$. 2. Every other valid configuration has a **unique parent** with $\le$ energy, so there are no duplicate states (no `visited` hash set needed!). 3. Every node in this implicit tree has **out-degree at most 3** (ideally 2 or 3), so popping $K$ states pushes at most $3K$ states into the min-heap!

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Pushing $N$ successors from each popped state (incrementing $j_i \to j_i + 1$ for every $i = 1...N$).** Not only does that push $O(NK)$ states (causing TLE and MLE for $N, K = 2 \cdot 10^5$), it also reaches the same configuration $(2, 2, 1, ...)$ along multiple paths ($(1, 1) \to (2, 1) \to (2, 2)$ and $(1, 1) \to (1, 2) \to (2, 2)$), requiring an expensive hash set of length-$N$ vectors!
- **Mistake 2: Forgetting to sort layers by their first jump $\Delta_i = E_{i,2} - E_{i,1}$.** If we transition from modifying layer $i$ to modifying layer $i + 1$, that transition is only valid in a min-heap if moving the *first step* from layer $i$ to layer $i + 1$ has non-negative cost $\Delta_{i+1} - \Delta_i \ge 0$!

### Step 3 — The Turn: Discovering the Structure

1. **Filter & Sort Layers**: Every layer starts at state $j_i = 1$ contributing $E_{i,1}$ to the base sum $S_0$. Any layer with $M_i = 1$ can never be upgraded, so discard it. For the remaining layers, define their first upgrade cost $\Delta_i = E_{i,2} - E_{i,1} \ge 0$, and **sort the layers in non-decreasing order of $\Delta_1 \le \Delta_2 \le ... \le \Delta_{N'}$.**
2. **Canonical Tree of Non-Ground Layers**: Any configuration is uniquely identified by the sequence of layers where $j_i > 1$, listed in increasing order of layer index $1 \le i_1 < i_2 < ... < i_m \le N'$. Thus, we only need to track `(cur_cost, i, j)`, where `i` is the **last modified layer** ($1 \le i \le N'$) and `j` is its current state index ($2 \le j \le M_i$).

From a state `(cost, i, j)`, what are its unique children in the decision tree?

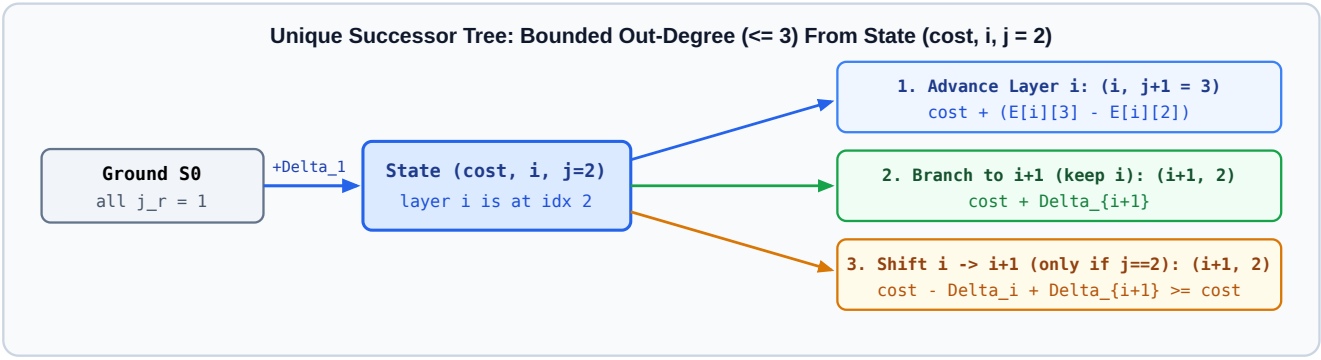
**Bounded-Degree Best-First Branching Rules from State $(\text{cost}, i, j)$:**

1. **Upgrade current layer $i$ further** (if $j + 1 \le M_i$): $$(\text{cost} + E_{i, j+1} - E_{i, j},\; i,\; j + 1)$$
2. **Keep layer $i$ at $j \ge 2$, and start upgrading layer $i+1$ to $2$** (if $i + 1 \le N'$): $$(\text{cost} + E_{i+1, 2} - E_{i+1, 1},\; i + 1,\; 2)$$
3. **Undo layer $i$ back to $1$, and instead upgrade layer $i+1$ to $2$** (ONLY if $j == 2$ and $i + 1 \le N'$): $$(\text{cost} - \Delta_i + \Delta_{i+1},\; i + 1,\; 2)$$

Every child has $\text{cost}_{\text{child}} \ge \text{cost}_{\text{parent}}$ (since $E_{i, j+1} \ge E_{i, j}$ and $\Delta_{i+1} \ge \Delta_i$), and every configuration is generated along a **unique** path!

### Step 4 — Progressive Construction & Visual Trace

1. Output $S_0 = \sum_{i=1}^N E_{i,1}$ as the 1-st configuration.
2. If $K = = 1$ or $N' = = 0$, stop. Otherwise push `(S_0 + \Delta_1, 1, 2)` into the min-heap.
3. Repeat $K - 1$ times: pop the minimum `(cost, i, j)`, output `cost`, and push up to 3 children according to the branching rules above.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Bijection with Valid Configurations**: Let $(i_1, j_{i_1}), ..., (i_m, j_{i_m})$ $(m \geq 1, i_1 < ... < i_m, j_{i_r} \geq 2)$ be the non-ground choices of any configuration. It is reached from $S_0$ by: shifting $(1, 2) \to (2, 2) \to ... \to (i_1, 2)$ via Rule 3; advancing $(i_1, 2) \to ... \to (i_1, j_{i_1})$ via Rule 1; branching to $(i_1 + 1, 2)$ via Rule 2 and shifting to $(i_2, 2)$ via Rule 3, and so on. Because Rule 3 is *only* permitted when $j = 2$, the decision at each layer $i$ between Rule 2 (keep layer $i$ at its final $j_i \geq 2$) and Rule 3 (leave layer $i$ at $1$) is mutually exclusive and irreversible!
- **Monotonicity**: Because $E_{i,j+1} - E_{i,j} \geq 0$ and $\Delta_{i+1} - \Delta_i \geq 0$, every edge in the implicit tree has non-negative weight, so Dijkstra's best-first extraction order outputs the exact $K$ smallest configurations in $O((N + K)\log(N + K))$ time.

---

# Section 41 — Euler Tour, LCA, and Binary Lifting

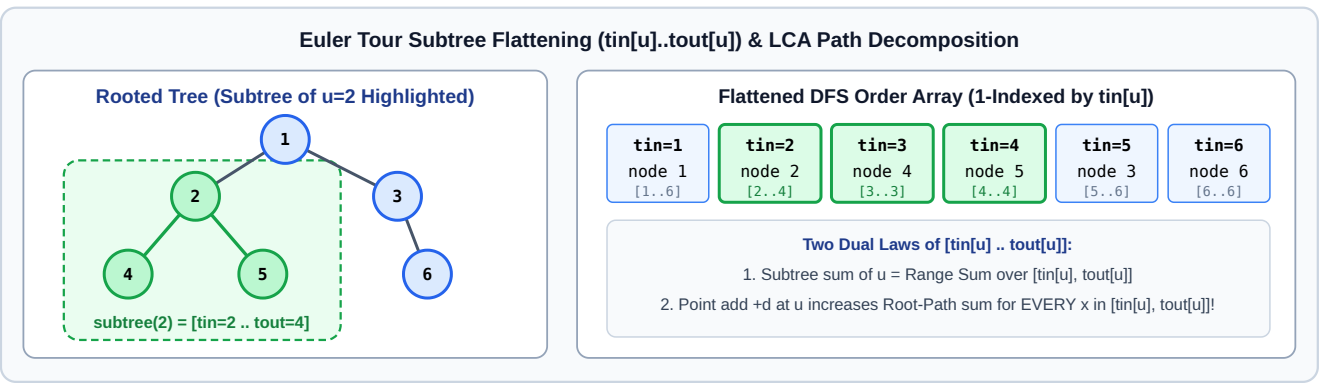
## The Discovery & Modeling Approach

A tree does not natively support array interval operations—until we **linearize** it. Almost every tree query problem reduces to one of two geometric decompositions of a rooted tree:

1. **Subtree Flattening via DFS Entry/Exit Timestamps ($\text{tin}[u]..\text{tout}[u]$)**: During a depth-first search from the root, record the counter `tin[u] = ++timer` upon entering $u$ and `tout[u] = timer` upon finishing $u$'s subtree. Because DFS visits all descendants of $u$ contiguously before backtracking out of $u$, **the entire subtree of $u$ maps bijectively to the contiguous 1D index interval $[\text{tin}[u], \text{tout}[u]]$**! Any subtree update or subtree aggregate query immediately becomes a standard 1D Fenwick Tree or Segment Tree operation.
2. **Tree Path Decomposition via Lowest Common Ancestor ($\text{LCA}(u, v)$)**: In any rooted tree, the unique simple path between $u$ and $v$ decomposes into two vertical ancestor-descendant chains meeting at $w = \text{LCA}(u, v)$:
$$\text{path}(u, v) = \text{chain}(u \to w) \cup \text{chain}(v \to w)$$
This yields two complementary tools:
   - **Binary Lifting on Tree Paths**: Precomputing $2^b$-th ancestors `up[b][u]` (and optional path monoid summaries `agg[b][u]`) finds $\text{LCA}(u, v)$, the $k$-th ancestor, or the $k$-th vertex along $u \to v$ in $O(\log N)$ time.
   - **Root-Path Difference / Prefix Cancellation** (for invertible operations like sum or XOR): If $\text{pref}[u]$ is the aggregate on the path from the root to $u$, then
$$\text{path_sum}(u, v) = \text{pref}[u] + \text{pref}[v] - 2\,\text{pref}[\text{LCA}(u, v)] + \text{val}[\text{LCA}(u, v)]$$
Furthermore, in an Euler Tour where entering $u$ adds $+val[u]$ at `in[u]` and exiting $u$ adds $-val[u]$ at `out[u]`, any exited subtree cancels out ($+x - x = 0$), so the prefix sum $[1..\text{in}[u]]$ equals the exact root-to-$u$ path sum under point updates!

### Decision Compass & Structural Lenses

| Tree Query / Update Pattern | Linearization / Lifting Tool | Underlying Data Structure | Why Alternative Fails |
|---|---|---|---|
| **Subtree Update & Subtree Query** | 1-Entry Euler Tour `tin[u]..tout[u]` (length $N$) | Fenwick Tree or Segment Tree on `[1..N]` | Ancestor-chain walks cannot touch all descendants in $o(N)$. |
| **Subtree Update & Root-to-Node Path Query** (dually: Node Update & Root-Path Query) | Node update $+v$ at $u$ adds $+v$ to $\text{pref}[x]$ for *all* $x \in \text{subtree}(u)$! | Range Add on `[tin[u], tout[u]]`, Point Query at `tin[x]` (Fenwick difference array) | 2-entry $+1/-1$ tour cannot simultaneously do subtree range updates. |
| **Static Path $\min/\max/\gcd$ on $u \leftrightarrow v$** | Tree Binary Lifting `up[b][u]` + `agg[b][u]` | $O(N \log N)$ static table | Non-invertible operations ($\min, \max, \gcd$) cannot cancel out $1 \to \text{LCA}(u, v)$ by subtraction! |
| **Tree Path Intersection / Steiner Subtree of $k$ Terminals** | Sort terminals by `tin[]`: $v_1, \ldots, v_k$ | Union of paths uses $\frac{1}{2}\sum_{i=1}^{k} \text{dist}(v_i, v_{(i \bmod k)+1})$ | Pairwise distances double-count shared trunk edges unless ordered around the Euler Tour perimeter. |



## Worked Problem 41.1: Power Grid Cascade & Path Diagnostics (Euler Tour Duality)

### Problem & Constraints

A hierarchical power grid is a rooted tree of $N$ substations with root $1$. Initially, each substation $u$ has load $A[u]$. Process $Q$ operations of three types:

1. `SET u x`: Change the load of substation $u$ to $x$.
2. `SUBTREE u`: Report the sum of loads of all substations in the subtree of $u$.
3. `PATH u v`: Report the sum of loads of all substations along the unique simple path between $u$ and $v$.

- **Constraints**: $1 \le N, Q \le 2 \cdot 10^5$, $|A[u]|, |x| \le 10^9$, time limit $1.5$ s.

> **Pause & Reflect Before Reading On:**
> - Why is a single 1D Fenwick tree over `tin[1..N]` enough for `SET` and `SUBTREE`, and how can a *second* 1D Fenwick tree over the exact same `tin[1..N]` indices answer `PATH u v` in $O(\log N)$ without Heavy-Light Decomposition?
> - When $A[u]$ changes by $\Delta = x - A[u]$, which nodes $w$ in the tree have $u$ on their root-to-$w$ path?

### Step 1 — Exact Model & Bottleneck

Walking the subtree of $u$ for `SUBTREE u` or walking the path $u \leftrightarrow v$ for `PATH u v` takes $O(N)$ per query in a degenerate line or star tree, giving $O(NQ)$ total time. Meanwhile, Heavy-Light Decomposition (HLD) takes $O(\log^2 N)$ and $120+$ lines of code, which is complete overkill when updates are **point updates** and path queries are **invertible sums**!

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Confusing the 1-entry Euler Tour (`tin[u]..tout[u]` of size $N$) with the 2-entry $+1/-1$ Euler Tour (`in[u]..out[u]` of size $2N$).** If you put $+A[u]$ at `in[u]` and $-A[u]$ at `out[u]` in a single array, prefix sums give root-to-node path sums, *but* the range sum over `[in[u], out[u]]` is identically $0$ because every node in the subtree enters and exits inside `[in[u], out[u]]`!
- **Mistake 2: Subtracting `2 * pref[lca]` in `PATH u v` without adding back `A[lca]` (or subtracting `pref[lca]` and `pref[parent[lca]]`).** The vertex $\mathrm{LCA}(u, v)$ lies on both the root-to-$u$ path and the root-to-$v$ path; subtracting $2 \cdot \mathrm{pref}[\mathrm{LCA}(u, v)]$ erases $\mathrm{LCA}(u, v)$ twice, leaving its load missing from the path sum!

### Step 3 — The Turn: Discovering the Structure

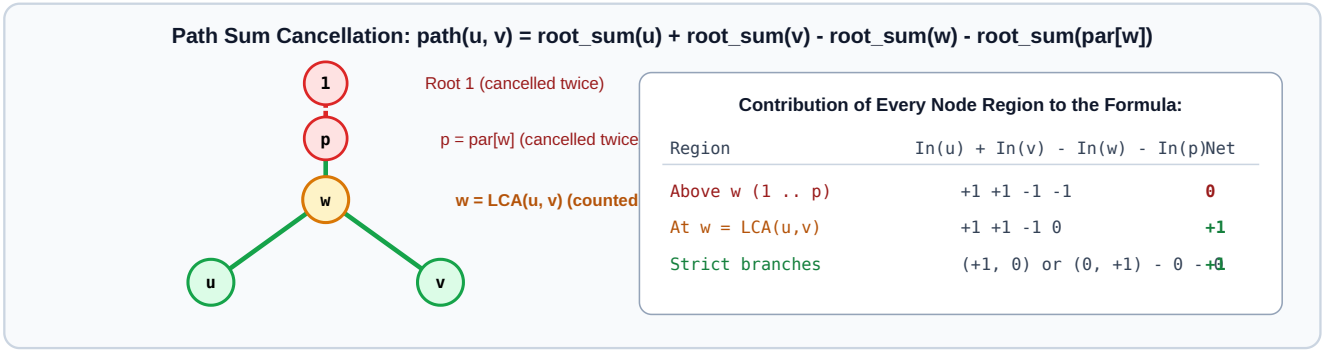
Ask the dual question: **When $A[u]$ increases by $\Delta$, whose root-to-node path sum $\mathrm{root_sum}(w)$ increases by $\Delta$?** A node $w$ has $u$ as an ancestor on the path $1 \to w$ **if and only if** $w$ **lies in the subtree of** $u$, which is equivalent to $\mathrm{tin}[w] \in [\mathrm{tin}[u], \mathrm{tout}[u]]$!

Therefore, on the single DFS ordering `tin[1..N]`, we maintain two simple Fenwick trees of size $N$: 1. `bit_sub` **(Point Update, Range Sum)**: Stores $A[u]$ at index $\mathrm{tin}[u]$. - `SET u x` ($\Delta = x - A[u]$): `bit_sub.add(tin[u], \Delta)`. - `SUBTREE u`: `bit_sub.query(tout[u]) - bit_sub.query(tin[u] - 1)`. 2. `bit_path` **(Range Update, Point Query via Difference Array)**: Stores the difference array of $\mathrm{root_sum}$ indexed by $\mathrm{tin}[w]$. - `SET u x` ($\Delta = x - A[u]$): add $+\Delta$ to the entire interval $[\mathrm{tin}[u], \mathrm{tout}[u]]$ via two point updates: `bit_path.add(tin[u], \Delta)` and `bit_path.add(tout[u] + 1, -\Delta)`. - Then for any vertex $w$, $\mathrm{root_sum}(w) = \mathrm{bit_path.query}(\mathrm{tin}[w])$!

> **Root-Path Inclusion-Exclusion with LCA:** $$\text{path_sum}(u, v) = \text{root_sum}(u) + \text{root_sum}(v) - \text{root_sum}(\text{LCA}(u, v)) - \text{root_sum}(\text{parent}[\text{LCA}(u, v)])$$ (where $\text{root_sum}(0) = 0$ for the parent of root $1$).

### Step 4 — Progressive Construction & Visual Trace

1. Run a single DFS from root $1$ to compute `tin[u]`, `tout[u]`, `depth[u]`, and the binary lifting table `up[b][u]` ($b = 0 \ldots 17$).
2. Initialize both Fenwick trees by calling the `add_delta(u, A[u])` helper for each $u \in [1, N]$.
3. Answer `SET`, `SUBTREE`, and `PATH` in $O(\log N)$ each!




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Subtree-Ancestry Equivalence**: In any DFS tree, $u$ is an ancestor of $w \Leftrightarrow \text{tin}[u] \le \text{tin}[w] \le \text{tout}[u]$. Thus $\text{root_sum}(w) = \sum_{u \text{ anc of } w} A[u] = \sum_{u:\, \text{tin}[w] \in [\text{tin}[u], \text{tout}[u]]} A[u]$, proving that `bit_path.query(tin[w])` is exact after arbitrary point updates.
- **Complexity**: $O(N \log N)$ preprocessing, $O(\log N)$ per operation, using two flat arrays of size $N + 2$.

---

## Worked Problem 41.2: Emergency Drone Rendezvous & Patrol Perimeter

### Problem & Constraints

A communication tree has $N$ stations connected by $N - 1$ bidirectional links, where link $e$ has positive length $W_e \ge 1$. Answer $Q$ independent queries of two types:

1. `RENDEZVOUS a b c`: Three drones start at stations $a, b, c$. Find the optimal rendezvous station $m^*$ that minimizes the weighted sum of distances $\text{dist}(a, m) + \text{dist}(b, m) + \text{dist}(c, m)$, and report both $m^*$ and the minimum total distance.
2. `PATROL k v_1 v_2 ... v_k`: Given $k$ stations ($2 \le k \le N$, $\sum k \le 2 \cdot 10^5$), find the minimum total edge weight of a connected subgraph (Steiner subtree) that contains all $k$ stations.

- **Constraints**: $1 \le N, Q \le 2 \cdot 10^5$, $\sum k \le 2 \cdot 10^5$, $1 \le W_e \le 10^9$, time limit $1.5$ s.

**Pause & Reflect Before Reading On:**

- In a tree, the three paths $a \leftrightarrow b$, $b \leftrightarrow c$, and $c \leftrightarrow a$ intersect at a single unique vertex $m^*$ (the tree median). Why is $m^*$ always equal to $\text{LCA}(a,b) \oplus \text{LCA}(b,c) \oplus \text{LCA}(c,a)$ (the deepest of the three pairwise LCAs)?
- If you sort $k$ vertices by their DFS entry time $\text{tin}[v_1] < \text{tin}[v_2] < \dots < \text{tin}[v_k]$ and walk the cycle $v_1 \to v_2 \to \dots \to v_k \to v_1$, how many times is each edge of their minimal spanning subtree traversed?

### Step 1 — Exact Model & Bottleneck

Testing every candidate vertex $m \in [1, N]$ for `RENDEZVOUS` or running a tree DFS for each `PATROL` query takes $\Theta(N)$ per query, or $\Theta(NQ)$ total time. We need formulas that depend *only* on the queried vertices (3 for `RENDEZVOUS`, $k$ for `PATROL`) using precomputed depths and LCAs.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1: Assuming the optimal rendezvous for $(a, b, c)$ is $\text{LCA}(a, \text{LCA}(b, c))$.**
  - *Counterexample*: Root the tree at $1$, with edge $1 - 2$ and children $3, 4$ of $2$. Let $a = 3, b = 4, c = 2$. All three vertices lie in the subtree of $2$, and $m^* = 2$. Yet if $a = 3, b = 4, c = 1$, $\text{LCA}(3, \text{LCA}(4, 1)) = 1$, whereas moving from $1$ to $2$ gets closer to *both* $3$ and $4$ while moving away from only $1$! The true median is $2 = \text{LCA}(3, 4)$.

- **Mistake 2: Using unweighted hop depth `depth[u]` instead of weighted root distance `dist_root[u]` in the distance formula, or conversely using `dist_root[u]` to test LCA depth.** Binary lifting jumps by *edge counts* (`depth[u]`), whereas path lengths sum *edge weights* (`dist_root[u]`). Keep both arrays!

### Step 3 — The Turn: Discovering the Structure

Let $D(u)$ be the weighted distance from root $1$ to $u$, so for any $x, y$:

$$\text{dist}(x, y) = D(x) + D(y) - 2\,D(\text{LCA}(x, y))$$

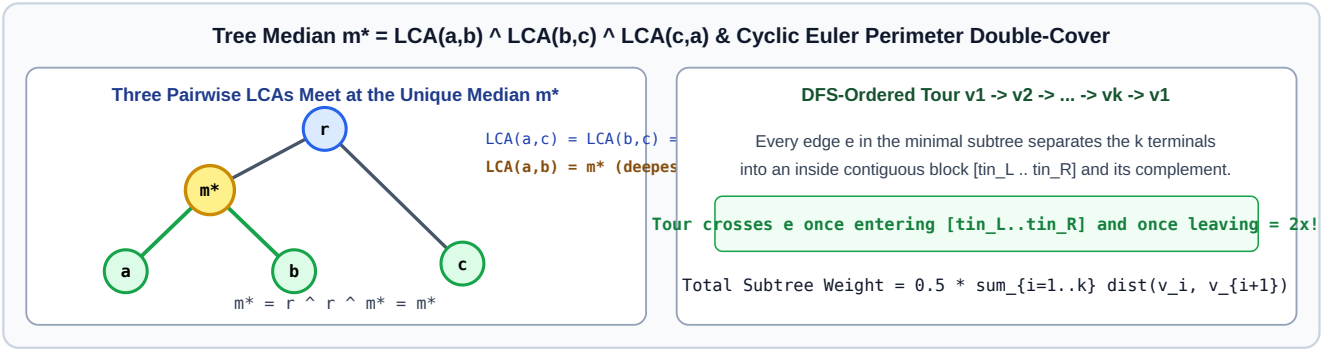
**1. The Tree Median of Three Vertices (`RENDEZVOUS a b c`)**: Compute the three pairwise LCAs: $x = \text{LCA}(a, b)$, $y = \text{LCA}(b, c)$, $z = \text{LCA}(c, a)$. In any rooted tree, at least two of $(x, y, z)$ are identical (equal to the overall lowest common ancestor of $\{a, b, c\}$), and the third is at least as deep (it lies inside the subtree where two of the three terminals branch apart). Therefore: - The unique median vertex is $m^* = x \oplus y \oplus z$ (or equivalently, the vertex among $\{x, y, z\}$ with the maximum `depth`). - Even simpler, every edge on the minimal subtree spanning $\{a, b, c\}$ is traversed by **exactly two** of the three pairwise paths $(a \leftrightarrow b)$, $(b \leftrightarrow c)$, $(c \leftrightarrow a)$, so:

$$\text{dist}(a, m^*) + \text{dist}(b, m^*) + \text{dist}(c, m^*) = \frac{\text{dist}(a, b) + \text{dist}(b, c) + \text{dist}(c, a)}{2}$$

**2. Generalization to $k$ Terminals (`PATROL k v_1 ... v_k`)**: Sort the $k$ distinct terminals by their DFS entry time: $\text{tin}[v_1] < \text{tin}[v_2] < \ldots < \text{tin}[v_k]$. Imagine walking the closed tour $v_1 \to v_2 \to \ldots \to v_k \to v_1$ along simple tree paths. Because the vertices are ordered by DFS discovery time, this tour traces the exact boundary of the minimal connected subgraph (Steiner tree) spanning $\{v_1, \ldots, v_k\}$, traversing every edge of that subtree **exactly twice** (once downward, once upward)!

**Euler-Sorted Steiner Subtree Weight Formula:** Sort $v_1, v_2, \dots, v_k$ so that $\text{tin}[v_1] < \text{tin}[v_2] < \dots < \text{tin}[v_k]$ (and set $v_{k+1} = v_1$). Then: $$\text{Weight}\!\big(\text{Steiner}(v_1, \dots, v_k)\big) = \frac{1}{2} \sum_{i=1}^{k} \text{dist}(v_i, v_{i+1})$$ Note that for $k = 3$, this reduces immediately to $\frac{1}{2}\big(\text{dist}(a,b) + \text{dist}(b,c) + \text{dist}(c,a)\big)$!

### Step 4 — Progressive Construction & Visual Trace




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Cut-Crossing Proof of the $\frac{1}{2}$ Perimeter Formula**: Remove any edge $e = (p, u)$ (where $p$ is the parent of $u$) in the tree. The subtree of $u$ corresponds to a single contiguous interval of DFS timestamps $I_u = [\text{tin}[u], \text{tout}[u]]$.
  - If all $k$ terminals lie inside $I_u$, or all $k$ terminals lie outside $I_u$, then edge $e$ does not belong to the minimal Steiner subtree of $\{v_1, \ldots, v_k\}$—and indeed, in the sorted cyclic sequence $\text{tin}[v_1] < \ldots < \text{tin}[v_k]$, the path $v_i \to v_{i+1}$ crosses $e$ zero times!
  - If some terminals lie inside $I_u$ and some lie outside $I_u$, then because $I_u$ is a single contiguous interval of `tin` values, the sorted sequence $v_1, \ldots, v_k$ enters $I_u$ **exactly once** and exits $I_u$ **exactly once** around the cycle $v_1 \to v_2 \to \ldots \to v_k \to v_1$. Thus the union of paths crosses $e$ **exactly twice**, contributing $2W_e$ to the cyclic distance sum!
- **Complexity**: $O(N \log N)$ preprocessing; $O(\log N)$ per `RENDEZVOUS` query and $O(k \log k + k \log N)$ per `PATROL` query.

# Section 43 — String Algorithms I: Prefix Function, KMP, and Z-Function

## The Discovery & Modeling Approach

Whenever a string problem asks you to align a pattern against a text, detect periodicity, find self-overlapping prefixes and suffixes, or maintain the state of a partial match as characters arrive one by one, naive character-by-character re-comparison creates an $O(N^2)$ bottleneck. Every time a mismatch occurs at index $i$ after matching $k$ characters, throwing away those $k$ matched characters discards structural information you have already paid to discover: **you already know the exact characters of the matched prefix!**

Linear-time string primitives ($\pi$-function / KMP and the $Z$-function) are discovered by asking a single question: *what self-similarity inside the pattern allows us to shift or resize a partial match in $O(1)$ amortized time without moving the text pointer backward?*

### Decision Compass: Three Structural Lenses for Exact String Matching

Although the prefix function $\pi$ and the $Z$-function carry closely related information, they organize string self-similarity along two different coordinates—**ending position** versus **starting position**—plus an automaton view for dynamic transitions:

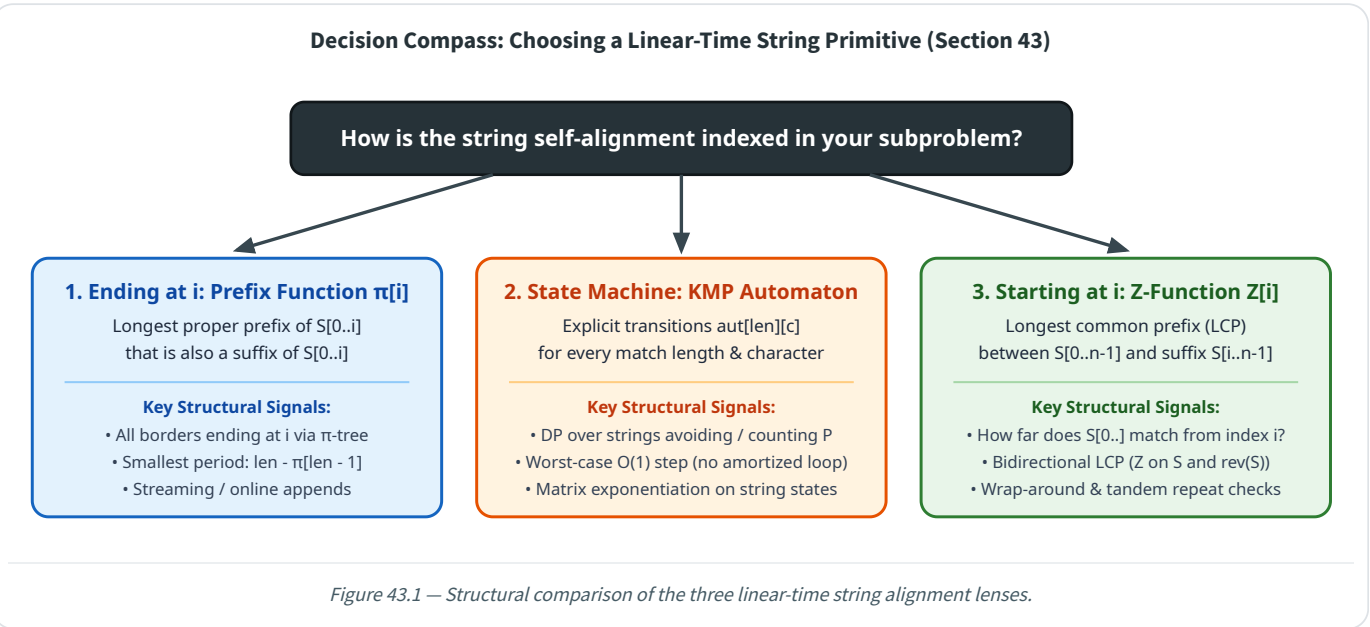



*Figure 43.1 — Structural comparison of the three linear-time string alignment lenses.*

| **Structural Feature** | **Prefix Function ($\pi[i]$) & Border Tree** | **KMP Prefix Automaton ($\mathrm{aut}[q][c]$)** | **Z-Function ($Z[i]$)** |
|---|---|---|---|
| **Core Definition** | Length of longest proper border of prefix $S[0..i]$ | Next match length when state $q$ reads char $c$ | $\mathrm{LCP}(S[0..n-1], S[i..n-1])$ starting at $i$ |
| **Anchor Coordinate** | Anchored at **right endpoint** $i$ | Anchored at **current matched prefix length** $q$ | Anchored at **left endpoint** $i$ |
| **Recursive Reuse** | Follow failure link $j \leftarrow \pi[j-1]$ | $O(1)$ lookup: $\mathrm{aut}[\pi[q-1]][c]$ | Reuse $Z[i-L]$ inside rightmost $Z$-box $[L, R]$ |
| **Best Suited For** | Streaming borders, periodicity, counting prefix frequencies via $\pi$-tree | DP over strings (counting strings with/without pattern $P$), branching walks | Prefix-suffix decomposition, 1-error/patch alignment, cyclic shifts |

## Worked Problem 43.1: Repeating Header Monitor (Border Tree & Prefix Function)

### Problem & Constraints

A network monitor inspects a calibration string $S$ of length $N$ ($1 \le N \le 5 \cdot 10^5$) over lowercase English letters. A substring $B$ is called a **verified header** of $S$ if:

1. $B$ is both a non-empty proper prefix and a proper suffix of $S$ (that is, $B$ is a **border** of $S$), and
2. $B$ also appears at least once as an **interior** substring of $S$ (starting at some index $p \ge 1$ and ending at some index $q \le N-2$), and
3. Among all such substrings, we also want to report, for every border of $S$, the **exact total number of times** it occurs anywhere in $S$.

Find the longest verified header of $S$ (or report that none exists), and compute the occurrence count of every border of $S$ in $O(N)$ time.

> **Pause & Reflect (Before the Turn):** What is the smallest obviously correct algorithm to enumerate all borders of $S$ and check whether each occurs strictly inside $S[1..N-2]$? Why does checking every candidate border independently take $O(N^2)$ time on a string like `aaaa...a`? How are all borders ending at position $i$ related to the single value $\pi[i]$?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** A border of $S$ of length $L \in \{1, \dots, N-1\}$ satisfies $S[0..L-1] = S[N-L..N-1]$. To verify whether it appears strictly inside $S$, we must check if there exists an ending index $i \in \{L, \dots, N-2\}$ such that the substring of length $L$ ending at $i$ equals $S[0..L-1]$.
2. **Bottleneck:** On a highly periodic string such as $S = \mathtt{a}^N$, every length $L \in \{1, \dots, N-1\}$ is a valid border, and comparing substrings or running a separate search for each candidate border takes $\Theta(N^2)$ operations—far exceeding the budget for $N = 5 \cdot 10^5$.

### Step 2 — Tempting Mistakes & Counterexample

A tempting shortcut is to check only whether the longest border $L_1 = \pi[N-1]$ appears directly as $\pi[i] = L_1$ for some $i \in [1, N-2]$, and if not, immediately give up or reset to $0$. Two subtle traps break naive attempts:

- **Trap 1 (Hidden shorter border at index $i$):** Suppose $S =$ `ababacababa`. Here $\pi[N-1] = 5$ (`ababa`), which only appears at the prefix and suffix. However, its shorter border $L_2 = \pi[5-1] = 3$ (`aba`) appears inside the prefix (`ababa` ends with `aba` at index $4$), even if no interior index $i$ has $\pi[i] = 3$ directly—because an interior position might have $\pi[i] = 4$ or $5$, which *implicitly* contains a border of length $3$!
- **Trap 2 (Repeated chain traversal):** Walking the failure chain $j \leftarrow \pi[j-1]$ from every index $i \in \{1, \dots, N-2\}$ to mark all interior prefix matches takes $O(N^2)$ time on $\mathtt{a}^N$.

### Step 3 — The Turn: Discovering the Border Tree ($\pi$-Tree)

For any position $i$, the longest prefix of $S$ that ends at $i$ (excluding the full prefix when $i = 0$) has length $\pi[i]$. What is the *second* longest prefix of $S$ ending at $i$? Any prefix of $S$ of length $k < \pi[i]$ ending at $i$ must match the suffix of $S[0..\pi[i]-1]$ of length $k$—which means $k$ is a proper border of $S[0..\pi[i]-1]$!

Therefore, the set of **all** prefix lengths of $S$ that occur ending at index $i$ is given by the ancestral chain of $\pi[i]$ in the **Border Tree** ($\pi$-tree), where the parent of each length $L \ge 1$ is $\pi[L-1]$ (and the root is $0$).

$$\text{Prefixes ending at } i \;=\; \{\, \pi[i],\ \pi[\pi[i]-1],\ \pi[\pi[\pi[i]-1]-1],\ \dots \,\} \setminus \{0\}$$
$$\text{Total occurrences of prefix } L \;=\; 1 + \sum_{u \,\in\, \text{Subtree}_{\pi}(L)} \text{freq}(\pi[i] = u)$$

Instead of walking up the $\pi$-chain from every index $i$, we accumulate the raw frequencies $\text{cnt}[\pi[i]]{+}{+}$ once for all $i \in \{0, \dots, N-1\}$ and push them **downward** (from larger lengths $L$ to their parent $\pi[L-1]$) in a single pass from $L = N$ down to $1$!

### Step 4 — Progressive Construction & Visual Trace

Let us trace this on $S = \texttt{abababcaababab}$ ($N = 14$), or even more compactly on $S = \texttt{abababaaba}$ ($N = 10$):

- Indices $0..9$: `a b a b a a b a b a`
- $\pi$-array: $\pi = [0, 0, 1, 2, 3, 1, 2, 3, 4, 5]$.
- The borders of the full string $S$ are the ancestors of $\pi[9] = 5$ in the $\pi$-tree:
  - $L_1 = 5$ (`ababa`)
  - $L_2 = \pi[5-1] = \pi[4] = 3$ (`aba`)
  - $L_3 = \pi[3-1] = \pi[2] = 1$ (`a`)

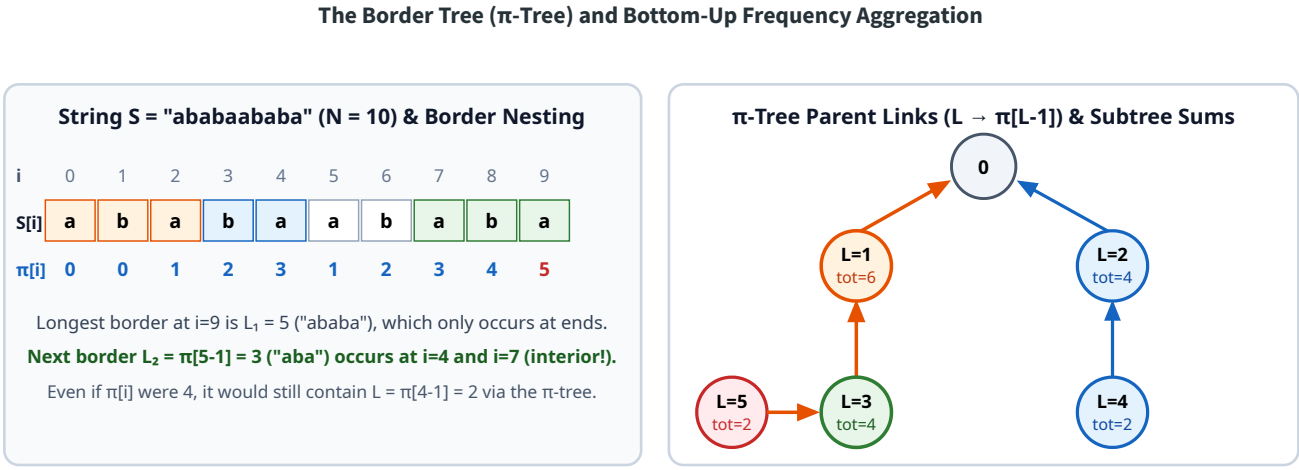



*Figure 43.2 — Every occurrence of prefix length L implicitly adds an occurrence to its parent π[L-1] in the Border Tree.*

The complete $O(N)$ algorithm is remarkably clean:

1. Compute $\pi[0..N-1]$ in $O(N)$ time.
2. Initialize $\text{cnt}[L] = 0$ for $1 \le L \le N$. For each $i \in \{0, \ldots, N-1\}$, increment $\text{cnt}[\pi[i]]{+}{+}$.
3. Propagate counts down the $\pi$-tree: for $L = N-1$ down to 1, if $\pi[L-1] > 0$, add $\text{cnt}[L]$ to $\text{cnt}[\pi[L-1]]$.
4. Finally, add 1 to every $\text{cnt}[L]$ (accounting for the prefix $S[0..L-1]$ itself).
5. Now, a border $L$ (an ancestor of $\pi[N-1]$) occurs at the prefix, at the suffix, and **at least once strictly inside** if and only if $\text{cnt}[L] \ge 3$! Since $\text{cnt}[L]$ is non-decreasing as we walk up the $\pi$-chain $L \leftarrow \pi[L-1]$, the first border on the chain from $\pi[N-1]$ with $\text{cnt}[L] \ge 3$ (equivalently, $\le \max_{1 \le i \le N-2} \pi[i]$) is the longest verified header.

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Prefix-to-Tree Equivalence:** Every occurrence of $S[0..L-1]$ ending at position $i \ge L$ means $S[0..L-1]$ is a proper border of $S[0..i]$, so $L$ belongs to the $\pi$-ancestor chain of $\pi[i]$. Conversely, every ancestor $L$ of $\pi[i]$ is a border of $S[0..\pi[i]-1]$ and therefore matches $S[i-L+1..i]$. Since the $\pi$-tree has a unique path from $\pi[i]$ to 0, each ending position $i$ contributes $+1$ to $L$ if and only if $\pi[i]$ lies in the subtree of $L$.
- **Amortized Complexity Proof:** In the KMP computation of $\pi[i]$, the candidate length $j$ increases at most once per step ($j{+}{+}$ at most $N$ times total), and each failure step $j \leftarrow \pi[j-1]$ strictly decreases $j \ge 0$. Thus the while-loop executes at most $N$ times across the entire string.
- **Small-Case Oracle:** Test all strings of length $N \le 8$ over $\{\texttt{a}, \texttt{b}\}$ against a brute-force oracle that uses `S.substr(0, L) == S.substr(i - L + 1, L)` to count occurrences and verify interior matches.

---

## Worked Problem 43.2: Cyclic Patch Alignment (Bidirectional Z-Function)

### Problem & Constraints

You are given a target string $T$ of length $N$ and a corrupted stream $S$ of the same length $N$ ($2 \le N \le 5 \cdot 10^5$). You want to find whether there exists a contiguous substring $S[l..r]$ of fixed length $K$ ($1 \le K < N$) such that if you **move** the block $S[l..r]$ to the very front of $S$—wait: more concretely, suppose $T$ was formed by taking a prefix of $S$ of length $p$, a suffix of $S$ of length $q$ (with $p + q = N - K$), and replacing the middle $K$ characters at one arbitrary mismatch position with a wildcard patch of length $K$.

Let us state the canonical alignment version cleanly: Given two strings $A$ and $B$ of length $N$ ($1 \le N \le 5 \cdot 10^5$), you may choose **one contiguous interval** $[L, R]$ of length $K$ in $A$ and replace $A[L..R]$ with any characters you wish, then **cyclically shift** $A$ by any offset $s \in \{0, ..., N-1\}$ so that the resulting string equals $B$. Determine all valid cyclic shift offsets $s$ (where shift $s$ aligns $A[(i+s) \bmod N]$ with $B[i]$) that require a patch of length at most $K$.

> **Pause & Reflect (Before the Turn):** For a fixed cyclic shift $s$, how can you tell in $O(1)$ time whether $A$ shifted by $s$ and $B$ differ only inside a contiguous segment of length at most $K$? What two quantities—one measured from the left of $B$ and one measured from the right of $B$—determine the uncorrupted prefix and suffix of the alignment?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** Double the string $A$ into $A' = A + A$ of length $2N$. A cyclic shift $s \in \{0, ..., N-1\}$ corresponds to the window $W_s = A'[s..s+N-1]$. Comparing $W_s$ against $B[0..N-1]$, let $p_s$ be the length of their **Longest Common Prefix (LCP)** and $q_s$ be the length of their **Longest Common Suffix (LCSuf)**. The mismatches between $W_s$ and $B$ fit inside a contiguous patch of length $K$ if and only if all characters outside that patch match—that is:
$$p_s + q_s \ge N - K$$
2. **Bottleneck:** Computing $p_s$ and $q_s$ by scanning $N$ characters for each of the $N$ shifts $s \in \{0, ..., N-1\}$ takes $O(N^2)$ time.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1 (Trying KMP directly):** KMP finds where the *entire* string $B$ occurs in $A'$, or finds the longest prefix of $B$ ending at each position of $A'$. Here, however, the match is broken in the **middle** of $B$: we need both the prefix match starting at $s$ and the suffix match ending at $s + N - 1$.
- **Mistake 2 (Hash binary search when $O(N)$ is required):** While rolling hashes with binary search can find $p_s$ and $q_s$ in $O(N \log N)$, tight time limits or anti-hash tests demand a deterministic $O(N)$ solution—and the $Z$-function computes *all* $p_s$ and $q_s$ directly in linear time!

### Step 3 — The Turn: Bidirectional Z-Function Alignment

Recall that the $Z$-function of a string $U$ stores $Z[i] = \mathrm{LCP}(U[0..], U[i..])$.

1. **Forward $Z$-array for $p_s$:** Build $U_{\text{fwd}} = B + \# + A + A$. For each shift $s \in \{0, ..., N-1\}$, the window $A'[s..s+N-1]$ starts at index $(N+1+s)$ in $U_{\text{fwd}}$. Thus its LCP with $B$ is simply:
$$p_s = Z_{\text{fwd}}[N+1+s]$$
2. **Reverse $Z$-array for $q_s$:** A common suffix of $W_s$ and $B$ is a common **prefix** of $\mathrm{rev}(W_s)$ and $\mathrm{rev}(B)$! Build $U_{\text{rev}} = \mathrm{rev}(B) + \# + \mathrm{rev}(A + A)$. The end of window $W_s$ is index $s + N - 1$ in $A + A$, which maps to starting index $2N - 1 - (s + N - 1) = N - s$ in $\mathrm{rev}(A + A)$. Thus:
$$q_s = Z_{\text{rev}}[N + 1 + (N - s)]$$

$$\text{Shift } s \text{ is valid} \Leftrightarrow \underbrace{Z_{\text{fwd}}[N+1+s]}_{\mathrm{LCP}(A'[s..],\, B)} + \underbrace{Z_{\text{rev}}[2N+1-s]}_{\mathrm{LCSuf}(A'[..s+N-1],\, B)} \ge N - K$$

### Step 4 — Progressive Construction & Visual Trace

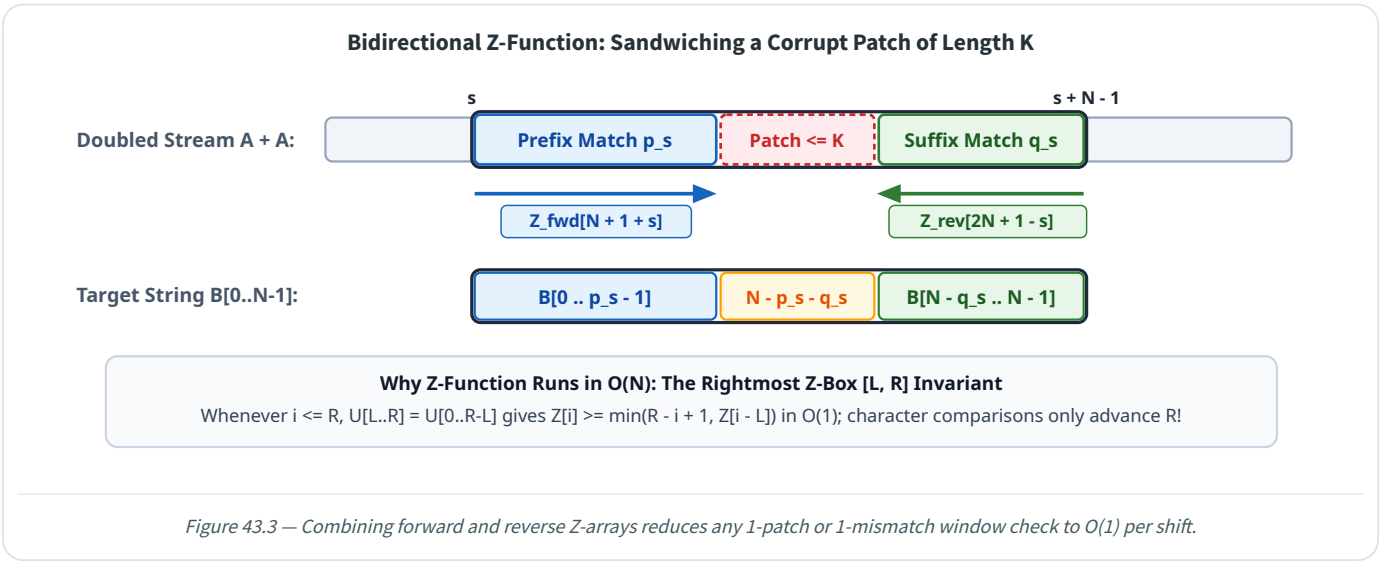



*Figure 43.3 — Combining forward and reverse Z-arrays reduces any 1-patch or 1-mismatch window check to O(1) per shift.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Equivalence:** Suppose shift $s$ can be turned into $B$ by modifying a contiguous subsegment $[l, r]$ of length $r - l + 1 \leq K$. Then $W_s[0..l-1] = B[0..l-1]$ (so $p_s \geq l$) and $W_s[r+1..N-1] = B[r+1..N-1]$ (so $q_s \geq N-1-r$). Adding the two inequalities yields $p_s + q_s \geq N - (r - l + 1) \geq N - K$. Conversely, if $p_s + q_s \geq N - K$, the prefix of length $\min(p_s, N)$ and the suffix of length $\min(q_s, N - \min(p_s, N))$ match $B$ identically, leaving at most $N - (p_s + q_s) \leq K$ middle characters to patch.
- **Z-Box Linear-Time Proof:** At step $i$, if $i \leq R$, we initialize $Z[i] = \min(R - i + 1, Z[i - L])$ without any character comparisons. The `while` loop only compares characters at positions $> R$, and each successful match increments $R$. Since $R < |U|$ is monotone non-decreasing, the total work is $O(N)$.
- **Small-Case Oracle:** Verify all $s \in \{0, ..., N-1\}$ for random strings of length $N \leq 12$ and $K \in \{0, 1, 3\}$ against a direct loop that finds the first mismatch $l$ and last mismatch $r$ between $W_s$ and $B$ and checks $r - l + 1 \leq K$.

---

# Section 44 — String Algorithms II: Rolling Hash, Tries, and Suffix Arrays

## The Discovery & Modeling Approach

When exact linear-time prefix/suffix alignment (Section 43) is no longer enough—for instance, when you need to compare **arbitrary substrings** $S[l_1..r_1]$ and $S[l_2..r_2]$ in $O(1)$ time, maintain a dynamic dictionary of **prefixes or bit-vectors**, or reason globally about **all** $\binom{N}{2}$ **substrings** of a text—you enter the territory of Section 44.

### Decision Compass: Three Structural Lenses for Substring & Prefix Collections

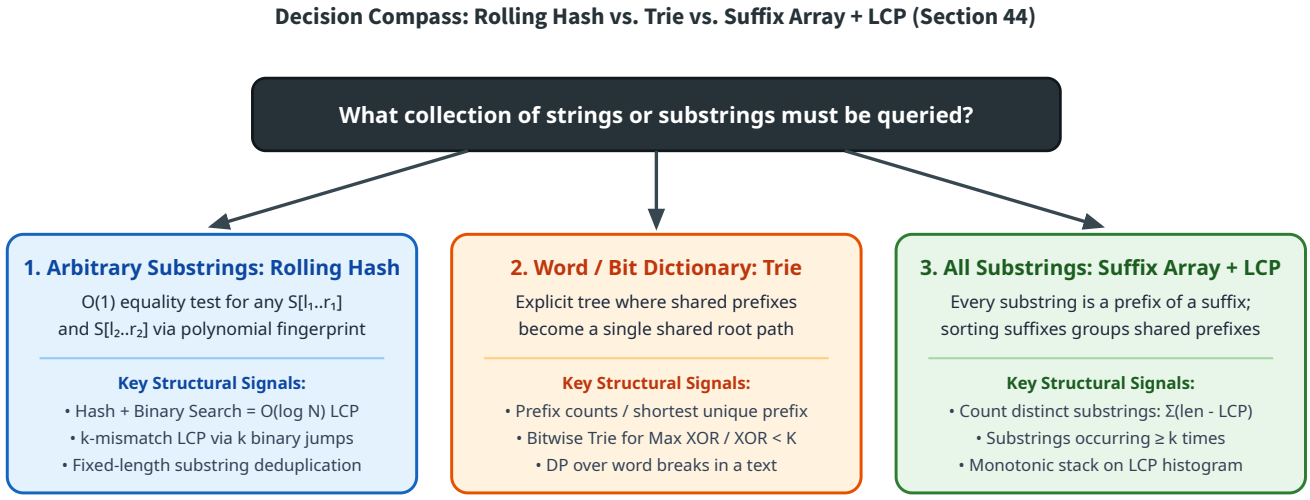



*Figure 44.1 — Choosing between Rolling Hashes, Tries, and Suffix Arrays based on substring structure.*

| Dimension | Polynomial Rolling Hash | Character / Bitwise Trie | Suffix Array ($\mathrm{SA}$) + $\mathrm{LCP}$ Array |
|---|---|---|---|
| **Core Principle** | Algebraic compression into $\mathbb{Z}_{M_1} \times \mathbb{Z}_{M_2}$ | Prefix-sharing tree over explicit dictionary | Lexicographical order turns prefix-sharing into contiguous intervals |
| **Preprocessing & Space** | $O(N)$ time, $O(N)$ memory | $O(\sum \lvert W_i \rvert)$ time, $O(\Sigma \cdot \text{nodes})$ space | $O(N \log N)$ time, $O(N)$ space |
| **Signature Superpower** | $O(1)$ equality of *any* two substrings; composable under concatenation | Dynamic insertions, exact subtree counts, greedy bit-by-bit XOR choices | Simultaneous view of all $\Theta(N^2)$ substrings in $O(N)$ space |

## Worked Problem 44.1: One-Fault Lexicographical Comparator (Rolling Hash + Binary Search)

### Problem & Constraints

A genome sequencer stores a string $S$ of length $N$ ($1 \le N \le 2 \cdot 10^5$) and answers $Q$ queries ($1 \le Q \le 10^5$). Each query specifies two substrings $A = S[l_1..r_1]$ and $B = S[l_2..r_2]$ of equal length $L$ and asks:

1. Do $A$ and $B$ differ in **at most one** position (Hamming distance $\le 1$)?
2. In standard exact lexicographical order, which of $A$ or $B$ is smaller?

**Pause & Reflect (Before the Turn):** Why can't you just compare the rolling hashes $\mathrm{Hash}(A)$ and $\mathrm{Hash}(B)$ to find which substring is lexicographically smaller? Why is "having equal prefixes of length $m$" a **monotone predicate** in $m$, and what happens immediately after the longest common prefix $m^*$?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** To compare $A = S[l_1..l_1 + L - 1]$ and $B = S[l_2..l_2 + L - 1]$ lexicographically, scan $k = 0, 1, ..., L - 1$ until the first index $k^*$ where $A[k^*] \neq B[k^*]$. The character comparison $A[k^*] \lessgtr B[k^*]$ decides the lexicographical order. To check if the Hamming distance is $\leq 1$, continue scanning $k = k^* + 1, ..., L - 1$ and verify that no second mismatch occurs.
2. **Bottleneck:** Scanning up to $L = O(N)$ characters per query costs $O(NQ)$ time in the worst case (e.g., when $A$ and $B$ share a long prefix).

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1 (Comparing numerical hash values for `<`):** $\mathrm{Hash}(A) < \mathrm{Hash}(B) \pmod M$ has **no relationship** with lexicographical order because modular reduction wraps around unpredictably!
- **Mistake 2 (Single 32-bit or `unsigned long long` hash on large comparison sets):** If you perform $Q = 10^5$ queries or sort $N$ substrings ($N \log N$ comparisons), Birthday Paradox collisions occur once roughly $\sqrt{M}$ distinct hashes are evaluated. Using a single modulus $M \approx 10^9 + 7$ fails with high probability when $\geq 3 \cdot 10^4$ distinct substrings exist. Use a **double hash** $(M_1, M_2)$ or a 61-bit Mersenne prime $M = 2^{61} - 1$ with randomized base $B$.

### Step 3 — The Turn: Monotonicity of Prefix Equality

Notice that the first mismatch position $k^*$ is simply the **Longest Common Prefix** $m^* = \mathrm{LCP}(A, B)$! For any candidate length $m \in \{0, ..., L\}$:

- If $m \leq m^*$, the prefixes $A[0..m-1]$ and $B[0..m-1]$ are identical, so $\mathrm{Hash}(l_1, l_1 + m - 1) = \mathrm{Hash}(l_2, l_2 + m - 1)$.
- If $m > m^*$, both prefixes contain the mismatch at $m^*$, so they are strictly different.

Because the predicate $P(m) \equiv [\mathrm{Hash}(l_1, l_1 + m - 1) = = \mathrm{Hash}(l_2, l_2 + m - 1)]$ is **monotone** (`true, true, ..., true, false, false`), we can binary-search for $m^*$ in $O(\log L)$ time using $O(1)$ range-hash queries!

Once we find the first mismatch at offset $m^*$:

1. **Lexicographical order** is decided immediately in $O(1)$ by comparing the single characters $S[l_1 + m^*]$ and $S[l_2 + m^*]$.
2. **1-Fault check** is decided in $O(1)$ **additional time** without a second binary search: just check whether the remaining suffixes after $m^*$ have equal hashes!

$$\mathrm{Hash}(l, r) = \left( \mathrm{pref}[r+1] - \mathrm{pref}[l] \cdot B^{r-l+1} \right) \bmod M$$
$$\mathrm{Hamming}(A, B) \leq 1 \iff m^* = L \text{ or } \mathrm{Hash}(l_1 + m^* + 1, r_1) = = \mathrm{Hash}(l_2 + m^* + 1, r_2)$$

### Step 4 — Progressive Construction & Visual Trace

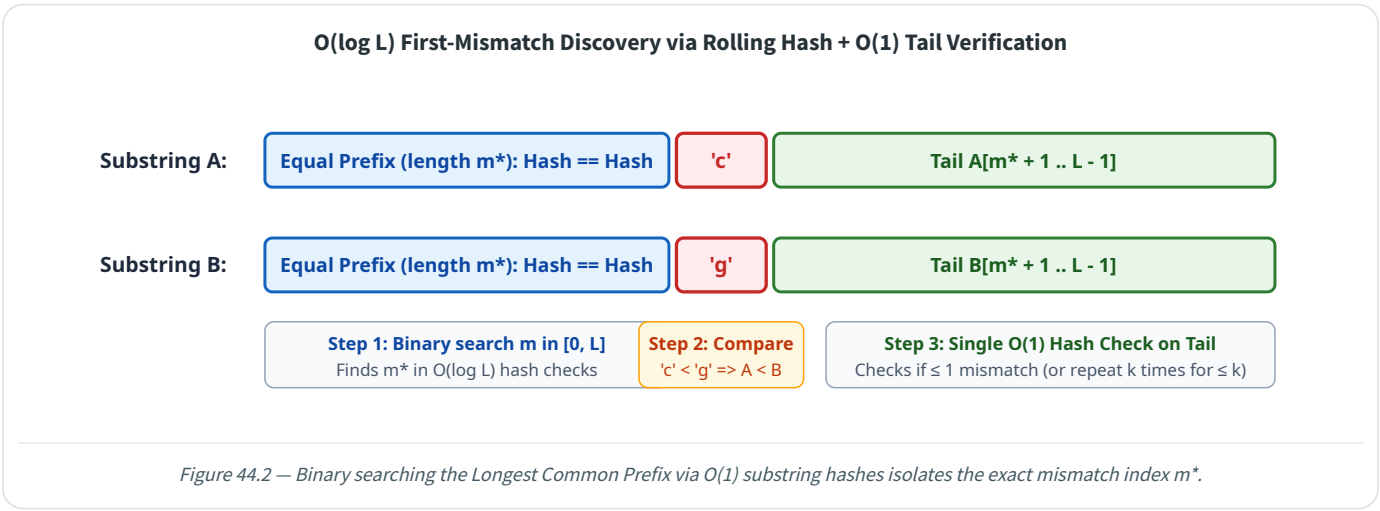



*Figure 44.2 — Binary searching the Longest Common Prefix via O(1) substring hashes isolates the exact mismatch index m*.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Correctness of Monotone Split:** If $A[0..m-1] = B[0..m-1]$, their polynomial hashes are identical under every modulus. Under a double modulus $(10^9+7, 10^9+9)$ with random base $B \in [257, 10^9]$, the probability that two distinct prefixes of length $\leq N$ collide in both moduli is at most $\frac{1}{M_1 M_2} \approx 10^{-18}$. Thus $m^*$ is the exact first index where $A[m^*] \neq B[m^*]$, which by definition determines both lexicographical order and whether the remaining suffixes $[m^*+1..L-1]$ match.
- **Small-Case Oracle:** Compare against a direct `S.substr(l1, L)` vs `S.substr(l2, L)` loop on random strings over a tiny alphabet $\{a, b\}$ where long shared prefixes with 1 or 2 mismatches occur frequently.

---

## Worked Problem 44.2: Shortest Distinguishing Command & Bitwise XOR Walk (Trie)

### Problem & Constraints

Consider two closely related prefix-tree problems that reveal both sides of the Trie lens:

- **Part A (Character Trie — Shortest Distinguishing Command):** Given $N$ command words ($1 \leq \sum |W_i| \leq 5 \cdot 10^5$), find for each word $W_i$ its shortest prefix that is **not** a prefix of any other word $W_j$ ($j \neq i$), or report `IMPOSSIBLE` if $W_i$ is a prefix of another word.
- **Part B (Bitwise Trie — Subarray Maximum XOR):** Given an array of $N$ integers $A[1..N]$ ($0 \leq A[i] < 2^{30}$), find the maximum XOR sum of a contiguous subarray $A[l..r]$.

> **Pause & Reflect (Before the Turn):** In Part A, if you walk down the path of word $W_i$ in a Trie, what property of a node tells you that no other word $W_j$ ($j \neq i$) shares the prefix up to that node? In Part B, once you convert a subarray XOR $A[l] \oplus ... \oplus A[r]$ into a pair of prefix XORs $P[r] \oplus P[l-1]$, why does a binary Trie from most significant bit ($b = 29$) to least significant bit ($b = 0$) find the optimal partner $P[l-1]$ greedily without backtracking?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:**
   - In Part A, for each word $W_i$ and each prefix length $k \in \{1, ..., |W_i|\}$, compare $W_i[0..k-1]$ against all other $N-1$ words.
   - In Part B, let $P[k] = A[1] \oplus ... \oplus A[k]$ (with $P[0] = 0$). Then $\bigoplus_{i=l}^{r} A[i] = P[r] \oplus P[l-1]$. Checking all pairs $(l-1, r)$ takes $O(N^2)$ time.
2. **Bottleneck:** Comparing against $N-1$ other words or $r$ previous prefix XORs adds an $O(N)$ factor per query.

### Step 2 — Tempting Mistakes & Counterexample

- **Part A Mistake (Comparing only with the previous sorted word):** If you sort the words and compare $W_i$ only with $W_{i-1}$, you miss cases where $W_i$ shares a longer prefix with $W_{i+1}$! For example, in `["a", "app", "apple"]`, comparing `"app"` only with `"a"` suggests `"ap"` is unique, whereas `"app"` is actually a prefix of `"apple"` (`IMPOSSIBLE`).
- **Part B Mistake (Trying to greedily extend a subarray without prefix XORs):** Unlike positive sums, XOR is not monotone with respect to subarray length; extending a subarray can turn a high bit from $1$ to $0$.

### Step 3 — The Turn: Shared Prefix Paths & Pass Counts

A **Trie** merges all identical prefixes into a single path from the root. If we maintain $\text{pass}[u]$ = the number of inserted strings whose path visits node $u$:

1. **In Part A:** As we trace $W_i$ from the root, any node $u$ with $\text{pass}[u] \geq 2$ represents a prefix shared by at least one other word. The **first node** $u$ **with** $\text{pass}[u] = \ = 1$ is the exact point where $W_i$ diverges from all other words! If even the terminal node of $W_i$ has $\text{pass}[u] \geq 2$, then $W_i$ is a prefix of another word (or duplicated), so the answer is `IMPOSSIBLE`.
2. **In Part B:** View every 30-bit prefix XOR $P[k]$ as a 30-character binary string from bit $29$ (MSB) down to bit $0$ (LSB). Since $2^b > \sum_{j=0}^{b-1} 2^j$, gaining a $1$ in bit $b$ of $P[r] \oplus P[l-1]$ beats *any* combination of lower bits! Thus, as we walk from bit $29$ down to $0$, we always step to the child with bit $\mathbf{1 - \text{bit}_b(P[r])}$ if that child exists in the Trie, and fall back to $\text{bit}_b(P[r])$ otherwise.

$$\text{Shortest Unique Prefix}(W_i) = \min\left\{\ d \geq 1 : \text{pass}(\text{node}(W_i[0..d-1])) = 1\ \right\}$$
$$\max_{0 \leq l < r} \left( P[r] \oplus P[l] \right) \ \rightarrow\ \text{at bit } b = 29 \ldots 0,\ \text{prefer child } c = 1 - \left( (P[r] \gg b)\ \&\ 1 \right)$$

### Step 4 — Progressive Construction & Visual Trace

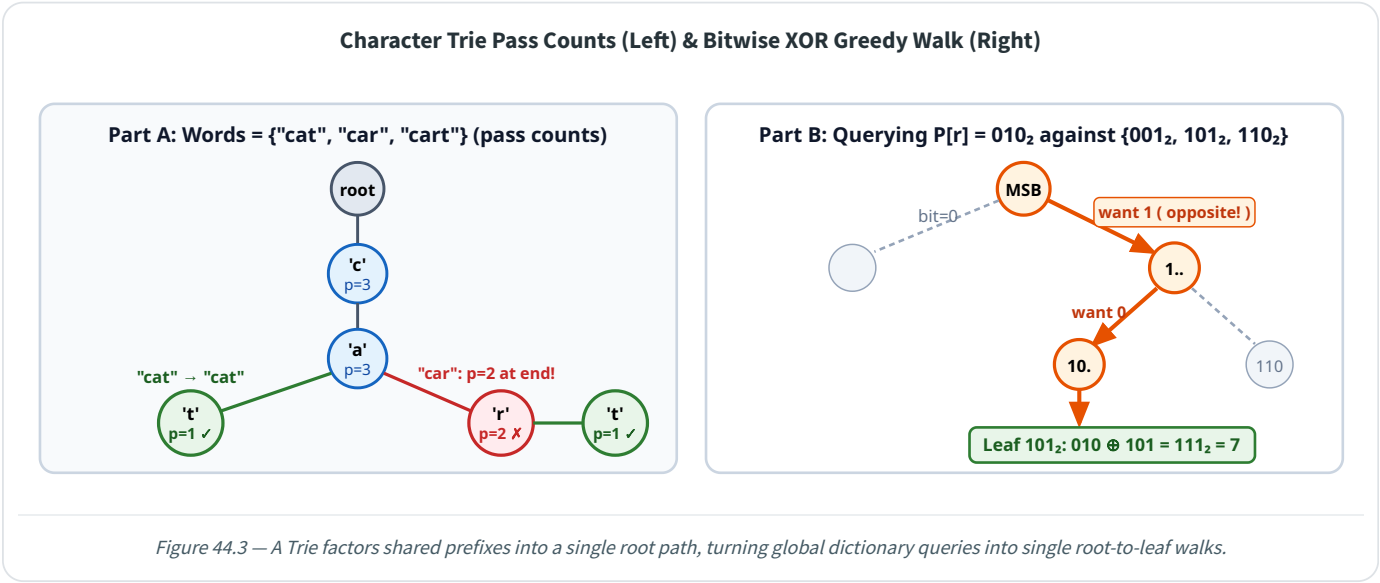



*Figure 44.3 — A Trie factors shared prefixes into a single root path, turning global dictionary queries into single root-to-leaf walks.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Pass Count Proof:** Node $u$ at depth $d$ along the path of $W_i$ represents the exact prefix $W_i[0..d-1]$. Since every word is inserted once, $\text{pass}[u] = 1$ if and only if $W_i$ is the unique word in the collection with prefix $W_i[0..d-1]$. Because $\text{pass}[\cdot]$ is non-increasing along any root-to-leaf path, the shallowest node with $\text{pass}[u] = 1$ gives the shortest distinguishing prefix.
- **Bitwise Dominance Proof:** At bit $b$, any candidate in the preferred branch achieves $(P[r] \oplus P[l]) \geq \text{prefix_mask} + 2^b$, whereas every candidate in the non-preferred branch has bit $b$ equal to $0$ in the XOR and therefore achieves at most $\text{prefix_mask} + (2^b - 1) < \text{prefix_mask} + 2^b$.

---

## Worked Problem 44.3: Repeating Motif Score (Suffix Array + LCP Histogram)

### Problem & Constraints

Given a string $S$ of length $N$ ($1 \le N \le 2 \cdot 10^5$), define the **motif score** of any non-empty substring $U$ of $S$ as:

$$\text{Score}(U) = |U| \times \text{occ}(U)$$

where $|U|$ is the length of $U$ and $\text{occ}(U)$ is the total number of times $U$ occurs as a substring in $S$. Find the maximum motif score over all substrings of $S$, and also compute the total number of **distinct** substrings of $S$.

> **Pause & Reflect (Before the Turn):** Every substring of $S$ is a **prefix of some suffix** of $S$. When you sort all $N$ suffixes of $S$ lexicographically to form the Suffix Array $\text{SA}[0..N-1]$, where do all occurrences of a fixed substring $U$ live in the sorted array? What determines the longest common prefix shared by a contiguous range of suffixes $\text{SA}[L..R]$?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** Enumerate every substring $U = S[i..j]$, count how many starting indices $p$ satisfy $S[p..p+|U|-1] = U$, and maximize $|U| \times \text{occ}(U)$.
2. **Bottleneck:** A string of length $N$ has $\binom{N+1}{2} = \Theta(N^2)$ substrings. We cannot even afford to list all candidate substrings!

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1 (Only checking single adjacent LCP values):** Looking only at pairs of adjacent suffixes $(k, k+1)$ tests occurrences of count $2$, missing substrings that occur $3, 4, \ldots$ times across a wider interval $[L, R]$ of the Suffix Array.
- **Mistake 2 (Binary-searching every length separately):** You might binary-search the length for a fixed minimum occurrence $c$, which only gives one slice of the trade-off between $|U|$ and $\text{occ}(U)$.

### Step 3 — The Turn: Suffix Array Intervals & the LCP Histogram

Sorting all $N$ suffixes lexicographically into $\text{SA}[0..N-1]$ and computing the adjacent Longest Common Prefix array $\text{LCP}[k] = \text{lcp}(S[\text{SA}[k]..], S[\text{SA}[k+1]..])$ for $0 \le k < N-1$ unlocks three fundamental structural theorems:

1. **Contiguity of Prefix Matches:** All suffixes of $S$ that begin with a given substring $U$ form a **single contiguous interval** $\text{SA}[L..R]$, and therefore $\text{occ}(U) = R - L + 1$ (when $[L, R]$ is maximal).
2. **Range Minimum LCP:** For any interval $L < R$ in the Suffix Array, the length of the longest common prefix shared by *all* suffixes in $\text{SA}[L..R]$ is:
$$\text{lcp}(\text{SA}[L..R]) = \min_{L \le k < R} \text{LCP}[k]$$
3. **Largest Rectangle in a Histogram:** Consider a maximal substring $U$ occurring at $\text{SA}[L..R]$ with length $H = \min_{L \le k < R} \text{LCP}[k]$ and occurrence count $R - L + 1$ (spanning $w = R - L$ adjacent LCP bars, so $\text{occ}(U) = w + 1$). Maximizing $|U| \times \text{occ}(U)$ for $\text{occ}(U) \ge 2$ is **literally finding the maximum area** $\text{LCP}[k] \times (r_k - l_k + 2)$ **in the LCP histogram**, solvable in $O(N)$ with a monotonic stack! (For $\text{occ}(U) = 1$, the best is the full string $S$ with score $N \times 1 = N$.)

And how many **distinct substrings** does $S$ have? Each suffix $\text{SA}[k]$ of length $N - \text{SA}[k]$ contributes $(N - \text{SA}[k])$ prefixes, of which $\text{LCP}[k-1]$ were already counted in $\text{SA}[k-1]$!

$$\text{Distinct Substrings}(S) = \frac{N(N+1)}{2} - \sum_{k=0}^{N-2} \text{LCP}[k]$$

$$\max_{|U| \ge 1} |U| \cdot \text{occ}(U) = \max\left( N, \max_{0 \le k < N-1} \text{LCP}[k] \times (\text{right}[k] - \text{left}[k] + 2) \right)$$

### Step 4 — Progressive Construction & Visual Trace

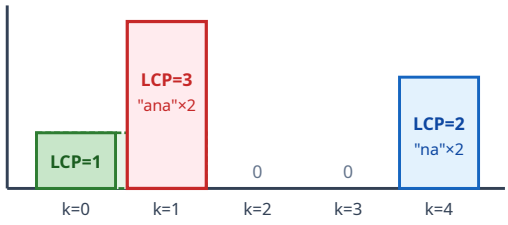
Let us trace $S = \texttt{banana}$ ($N = 6$):

**Suffix Array & LCP Histogram for S = "banana" (N = 6)**

**Sorted Suffixes SA[0..5] & Adjacent LCP[0..4]**

| Rank k | SA[k] | Suffix S[SA[k]..] | LCP[k] |
|---|---|---|---|
| 0 | 5 | a | 1 ("a") |
| 1 | 3 | ana | 3 ("ana") |
| 2 | 1 | anana | 0 |
| 3 | 0 | banana | 0 |
| 4 | 4 | na | 2 ("na") |
| 5 | 2 | nana | — |

**LCP Histogram & Contiguous Suffix Intervals**




**Distinct substrings = 21 - (1+3+0+0+2) = 15 | Max score = 3×2 = 6 ("ana" or "banar**

*Figure 44.4 — Every repeated substring corresponds to a horizontal rectangle in the LCP histogram of width w bars (meaning w + 1 suffix occurrences).*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Bijection for Distinct Substrings:** Every substring $U$ occurs at least once as a prefix of some suffix in $\mathrm{SA}$. Charge $U$ to the **first** suffix $\mathrm{SA}[k]$ in lexicographical order that starts with $U$. For a fixed $k$, a prefix of $S[\mathrm{SA}[k]..]$ of length $\ell$ appeared earlier in $\mathrm{SA}[0..k-1]$ if and only if it also appears in $\mathrm{SA}[k-1]$ (by lexicographical contiguity), which happens if and only if $\ell \le \mathrm{LCP}[k-1]$. Thus $\mathrm{SA}[k]$ introduces $(N - \mathrm{SA}[k]) - \mathrm{LCP}[k-1]$ brand-new substrings.
- **Histogram Completeness:** For any substring $U$ with $\mathrm{occ}(U) = c \ge 2$, let $[L, R]$ ($R - L + 1 = c$) be the maximal interval of suffixes starting with $U$. Extending $U$ to the full common prefix of $\mathrm{SA}[L..R]$ has length $H = \min_{L \le k < R} \mathrm{LCP}[k] \ge |U|$ without decreasing $\mathrm{occ}$ below $c$. That $H$ is achieved at some minimum bar $k^* \in [L, R-1]$ whose maximal span $[l_{k^*}, r_{k^*}]$ of bars $\ge H$ is precisely $[L, R-1]$, examined by our monotonic stack.
- **Small-Case Oracle:** Insert all substrings of $S$ ($N \le 40$) into a `std::map<string, int>` to compute both `map.size()` (distinct substrings) and $\max_U |U| \cdot \mathrm{count}[U]$, and compare with the Suffix Array + LCP formula.

---

# Section 45 — Flows and Matchings

## The Discovery & Modeling Approach

In introductory flow problems, the statement hands you the network on a silver platter: pipes, capacities, a source $s$, and a sink $t$. In genuine modeling problems, **no pipe, no fluid, and no source are mentioned at all**. Instead, the statement talks about volunteers and shifts, emergency evacuations through bottlenecks, or binary choices with logical implications and signed profits.

How do you recognize that a problem belongs to Section 45, and—crucially—**which of the three distinct modeling paradigms** should you build?

1. **Constraint Signal:** Look at the input bounds first. When $N, M \leq 200$ to $1000$—too large for exponential $O(2^N)$ or bitmask DP, yet suspiciously small compared to $10^5$—and greedy or tree DP fails because choices share resources or form DAG/cyclic dependencies, suspect a polynomial flow or matching reduction.
2. **Three Structural Lenses:** Although they all execute the same augmenting-path engine (such as Dinic's algorithm), Section 45 actually contains **three completely different ways of reading a problem**:

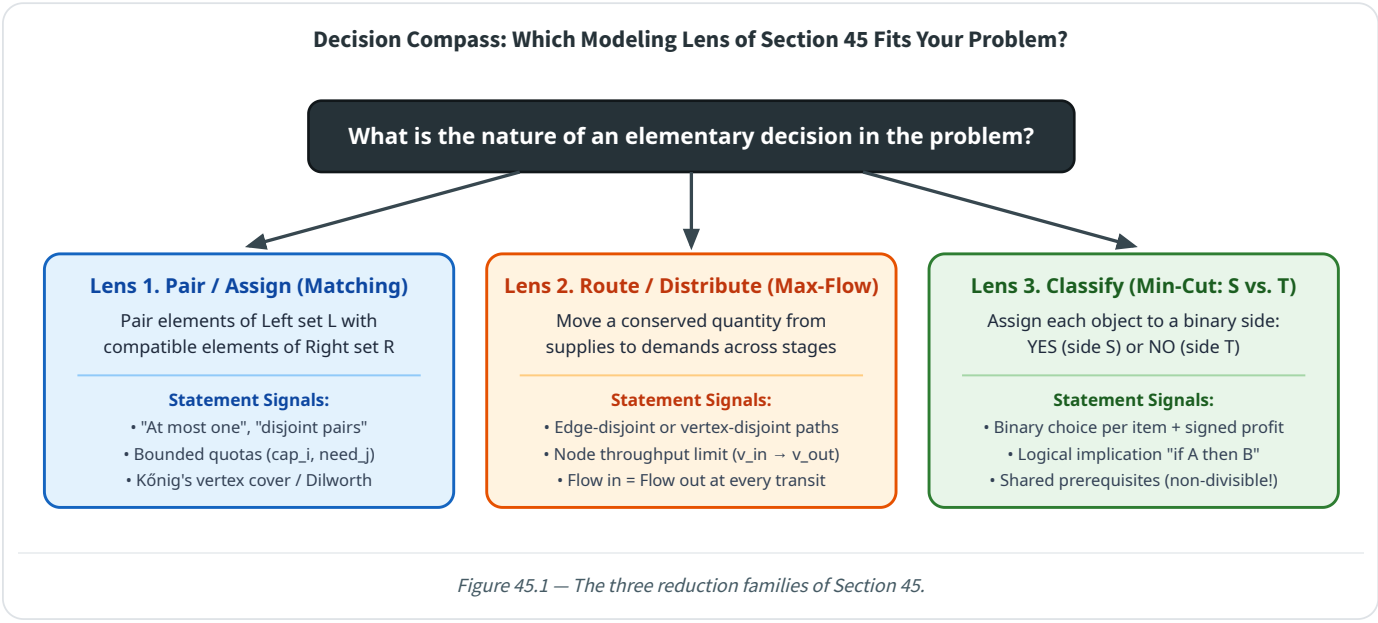



*Figure 45.1 — The three reduction families of Section 45.*

### How to Tell Flow Conservation Apart from Min-Cut Partitioning

Beginners frequently confuse Lens 2 (Flow Routing) and Lens 3 (Min-Cut Binary Partition). Use this diagnostic test:

| Diagnostic Question | Lens 1 & 2: Flow / Matching Model | Lens 3: Minimum Cut ($S$ vs. $T$) Model |
|---|---|---|
| **What does a vertex $v$ represent?** | A physical location, transit stage, or actor with a capacity quota | A **Boolean decision variable**: $v \in S$ (YES / True) or $v \in T$ (NO / False) |
| **If item $b$ supports two neighbors $a_1, a_2$...** | The quantity leaving $b$ must **split** between $a_1$ and $a_2$ (conservation: $f_1 + f_2 \leq \text{cap}$) | Selecting $b$ ($b \in S$) satisfies $a_1$ and $a_2$ **simultaneously without splitting**! |
| **What does capacity $c$ on $u \to v$ mean?** | Upper bound on how many units can travel from $u$ to $v$ | **Disagreement penalty** $c$ paid *if and only if* $u \in S$ (YES) and $v \in T$ (NO) |

---

## Worked Problem 45.1: Volunteer Shift & Quota Allocation (Lens 1: Bipartite Matching & Quota Flow)

### Problem & Constraints

An emergency relief center has $N$ volunteers ($1 \leq N \leq 250$) and $M$ operational shifts ($1 \leq M \leq 250$).

- Volunteer $i$ is qualified for a subset of shifts $E_i \subseteq \{1, \ldots, M\}$ and can work at most $\text{cap}_i$ shifts in total.
- Volunteer $i$ can be assigned to any single shift $j \in E_i$ **at most once**.
- Shift $j$ requires at least $\text{need}_j$ distinct volunteers to operate safely.

Determine whether all shift requirements can be met simultaneously, and if so, output an explicit assignment of volunteers to shifts.

> **Pause & Reflect (Before the Turn):** Why does a greedy strategy ("assign volunteers to the most constrained shift first") fail even when all $\text{cap}_i = 1$ and $\text{need}_j = 1$? How can you enforce both "volunteer $i$ works at most $\text{cap}_i$ shifts total" and "volunteer $i$ takes shift $j$ at most once" using edge capacities alone?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** Choose a subset of allowed pairs $X \subseteq \{(i, j) : j \in E_i\}$ such that every volunteer $i$ appears in at most $\text{cap}_i$ pairs of $X$ and every shift $j$ appears in exactly $\text{need}_j$ pairs of $X$.
2. **Bottleneck:** There are up to $NM$ candidate edges; testing subsets takes $O(2^{NM})$ time.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1 (Scarcity Greedy):** Suppose Volunteer $1$ can take shifts $\{A, B\}$, Volunteer $2$ can only take $\{A\}$, and each needs $1$ volunteer. If Volunteer $1$ is assigned to Shift $A$ first, Volunteer $2$ is stranded and Shift $B$ goes unfilled. We need a mechanism that **reassigns** Volunteer $1$ from $A$ to $B$ so Volunteer $2$ can take $A$. That rollback-and-reroute mechanism is precisely an **augmenting path in the residual graph**!
- **Mistake 2 (Omitting the middle capacity $1$):** If you connect $i \to j$ with capacity $\infty$, a single volunteer $i$ with $\text{cap}_i = 3$ could send $3$ units of flow to the *same* shift $j$, violating the rule that one volunteer cannot fill multiple slots of the same shift.

### Step 3 — The Turn: Three-Layer Quota Network

Each unit of integral flow from $s$ to $t$ will represent **one volunteer assigned to one shift**. We separate the three constraints onto three disjoint layers of edges:

1. **Source to Volunteers ($s \to i$):** capacity $\text{cap}_i$ (caps the total shifts worked by volunteer $i$).
2. **Volunteer to Allowed Shift ($i \to j$ for $j \in E_i$):** capacity $1$ (ensures volunteer $i$ works shift $j$ at most once).
3. **Shifts to Sink ($j \to t$):** capacity $\text{need}_j$ (caps how many useful assignments shift $j$ absorbs).

$$\text{All shifts filled} \iff \text{MaxFlow}(s, t) = \sum_{j=1}^{M} \text{need}_j$$
$$\text{Explicit assignment:} \quad \text{Volunteer } i \text{ works shift } j \iff f(i \to j) = 1$$

### Step 4 — Progressive Construction & Visual Trace

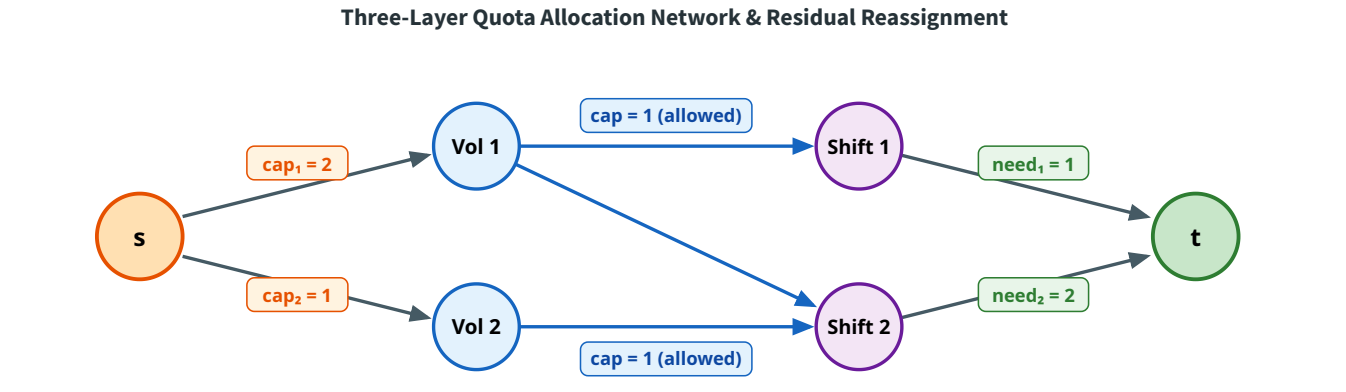



*Figure 45.2 — Separating volunteer quotas (left), pair uniqueness (middle), and shift demands (right).*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Allocation $\implies$ Flow:** Given any valid assignment $X$, set $f(i \to j) = 1$ for $(i, j) \in X$, $f(s \to i) = |\{j : (i, j) \in X\}| \le \text{cap}_i$, and $f(j \to t) = \text{need}_j$. Flow conservation holds at every node and total flow is $\sum_j \text{need}_j$.
- **Flow $\implies$ Allocation:** Because all capacities are integers, the **Integrality Theorem** guarantees an integer maximum flow. If $\text{MaxFlow}(s, t) = \sum_j \text{need}_j$, then every edge $j \to t$ is saturated ($f(j \to t) = \text{need}_j$). Since $f(i \to j) \in \{0, 1\}$, choosing $X = \{(i, j) : f(i \to j) = 1\}$ gives each shift $j$ exactly $\text{need}_j$ distinct volunteers and assigns each volunteer $i$ at most $f(s \to i) \le \text{cap}_i$ shifts.
- **Small-Case Oracle:** For $N, M \le 4$, enumerate all $2^{|E|}$ edge subsets and verify that a valid subset exists if and only if Dinic returns $\sum_j \text{need}_j$.

---

## Worked Problem 45.2: Emergency Relay Evacuation (Lens 2: Conserved Routing & Vertex Splitting)

### Problem & Constraints

A communication grid has $V$ relay stations ($1 \le V \le 200$) and $E$ directed fiber links ($1 \le E \le 1000$).

- A subset of stations $A \subseteq \{1, \ldots, V\}$ are **data generators**, each producing up to $\text{sup}_u$ packets per millisecond.
- A disjoint subset $B \subseteq \{1, \ldots, V\}$ are **archive bunkers**, each able to store up to $\text{dem}_v$ packets per millisecond.
- Every directed link $u \to v$ can transmit at most $c(u, v)$ packets per millisecond.
- **Crucial bottleneck:** Due to CPU heat limits, each station $v \in \{1, \ldots, V\}$ can process at most $K_v$ packets per millisecond in total (including packets generated at $v$, passing through $v$, or archived at $v$).

Find the maximum number of packets per millisecond that can be generated and safely archived.

> **Pause & Reflect (Before the Turn):** Standard flow networks only allow capacities on **edges**, never on **vertices**. How can you replace a single vertex $v$ with throughput limit $K_v$ by a small gadget of two vertices so that every unit of flow visiting $v$ is forced to pass through a single capacity-$K_v$ edge?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** We want a set of paths from generators $u \in A$ to bunkers $w \in B$ (with non-negative flow weights) respecting link capacities $c(x, y)$, generator limits $\text{sup}_u$, bunker limits $\text{dem}_w$, and node throughput limits $\sum_{\text{paths visiting } v} f \le K_v$.
2. **Bottleneck:** The number of simple paths between $A$ and $B$ can be exponential in $V$, and greedily sending packets along shortest paths can block a bottleneck node $v$ that two other paths needed to bypass each other.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake (Putting $K_v$ on all incoming edges of $v$):** If $v$ has three incoming edges and you give each capacity $K_v$, then up to $3K_v$ units of flow can enter $v$ simultaneously! A vertex capacity restricts the **sum** of all incoming (and outgoing) flows at $v$, not each individual edge.

### Step 3 — The Turn: Super-Source/Sink & Vertex Splitting ( `v_in -> v_out` )

Two universal graph transformations reduce this directly to standard Max-Flow:

1. **Super-Source $S^*$ and Super-Sink $T^*$:** Create a single source $S^*$ with edges $S^* \xrightarrow{\text{sup}_u} u_{\text{in}}$ for each $u \in A$, and a single sink $T^*$ with edges $w_{\text{out}} \xrightarrow{\text{dem}_w} T^*$ for each $w \in B$.
2. **Vertex Splitting ($v_{\text{in}} \to v_{\text{out}}$):** Split every station $v$ into an **entry port** $v_{\text{in}}$ and an **exit port** $v_{\text{out}}$ connected by a single internal bridge:

$$v_{\text{in}} \xrightarrow{K_v} v_{\text{out}}$$

   - Every **incoming** fiber link $x \to v$ is wired into the entry port: $x_{\text{out}} \xrightarrow{c(x,v)} v_{\text{in}}$.
   - Every **outgoing** fiber link $v \to y$ is wired out of the exit port: $v_{\text{out}} \xrightarrow{c(v,y)} y_{\text{in}}$.

$$\text{Flow through station } v \;=\; \sum_{x} f(x_{\text{out}} \to v_{\text{in}}) \;=\; f(v_{\text{in}} \to v_{\text{out}}) \;\le\; K_v$$

### Step 4 — Progressive Construction & Visual Trace

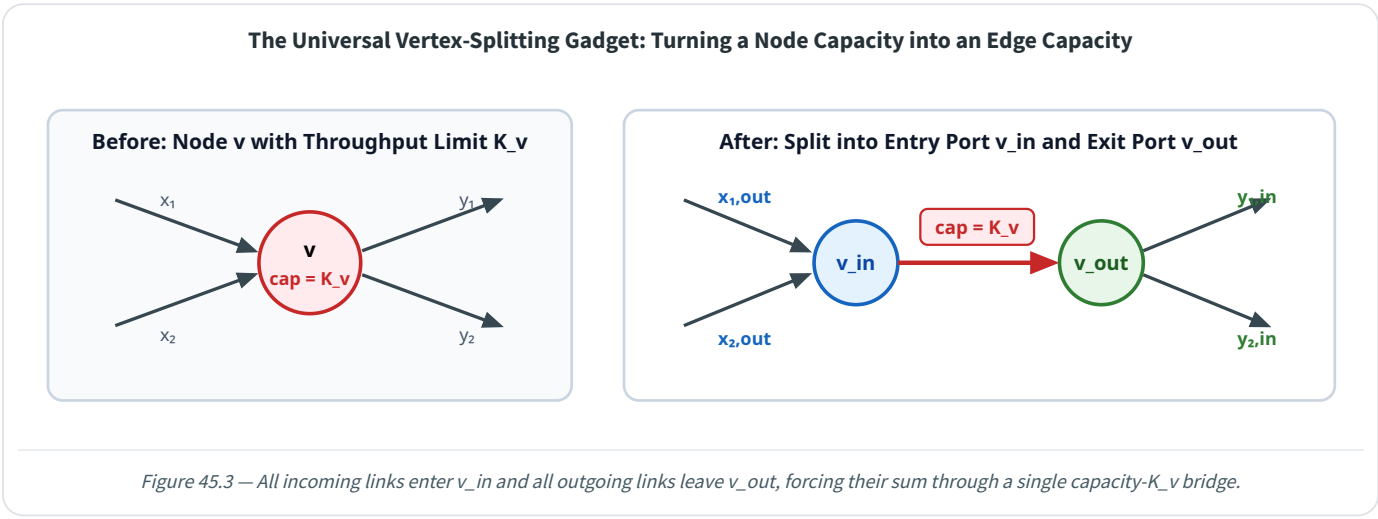



*Figure 45.3 — All incoming links enter v_in and all outgoing links leave v_out, forcing their sum through a single capacity-K_v bridge.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Conservation Bijection:** By flow conservation at $v_{\text{in}}$ (which has only one outgoing edge $v_{\text{in}} \to v_{\text{out}}$), the total flow entering station $v$ from $S^*$ and from all neighbors $x_{\text{out}}$ equals $f(v_{\text{in}} \to v_{\text{out}})$. By flow conservation at $v_{\text{out}}$ (which has only one incoming edge $v_{\text{in}} \to v_{\text{out}}$), this same quantity equals the total flow leaving station $v$ toward $T^*$ and all neighbors $y_{\text{in}}$. Thus $0 \le f(v_{\text{in}} \to v_{\text{out}}) \le K_v$ enforces the exact node throughput limit.
- **Vertex-Disjoint Paths Corollary:** Setting $K_v = 1$ for all intermediate vertices immediately computes the maximum number of **vertex-disjoint paths** from $s$ to $t$ (and by Menger's Theorem, the minimum number of intermediate vertices whose removal disconnects $s$ from $t$).

---

## Worked Problem 45.3: Satellite Calibration & Dependency Closure (Lens 3: Min-Cut as Binary Partition $S$ vs. $T$)

### Problem & Constraints

A deep-space observatory has $N$ scientific instruments ($1 \le N \le 200$).

- Activating instrument $i$ yields a net scientific value $w_i \in [-10^9, 10^9]$:
  - Some instruments ($w_i > 0$) are **observation sensors** that produce a positive reward $w_i$.
  - Other instruments ($w_j < 0$) are **cryogenic cooling or calibration modules** that incur an operating cost $|w_j| = -w_j$.
- There are $M$ **strict dependency rules** $(a \to b)$: if instrument $a$ is activated, instrument $b$ **must also be activated** ($a$ cannot function without $b$).
- Furthermore, there are $K$ **soft calibration penalties** $(u, v, c)$ with $c > 0$: if instrument $u$ is activated while instrument $v$ is left **inactive**, an extra interference penalty of $c$ must be paid.

Choose a subset of instruments to activate (possibly empty) that respects all strict dependency rules and **maximizes the total net score** (sum of $w_i$ of activated instruments minus any triggered soft penalties).

> **Pause & Reflect (Before the Turn):** Why can't we evaluate each sensor $a$ independently by summing $w_a$ and the costs of its required cooling modules, or run tree DP on the dependency graph? Draw a 3-instrument example where two sensors share the same expensive cooling module!

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** A valid configuration is a subset $X \subseteq \{1, \ldots, N\}$ such that for every strict rule $a \to b$, we never have $(a \in X \text{ and } b \notin X)$. Its score is:
$$\text{Score}(X) = \sum_{i \in X} w_i - \sum_{\substack{(u,v,c) \\ u \in X,\, v \notin X}} c$$
2. **Bottleneck:** Testing all $2^N$ subsets is impossible for $N = 200$.

### Step 2 — Tempting Mistakes & Counterexample

Consider a tiny instance with $3$ instruments and no soft penalties:

- Sensor $1$: reward $+6$, requires Cooling Module $3$.
- Sensor $2$: reward $+6$, also requires Cooling Module $3$.
- Cooling Module $3$: cost $-9$.

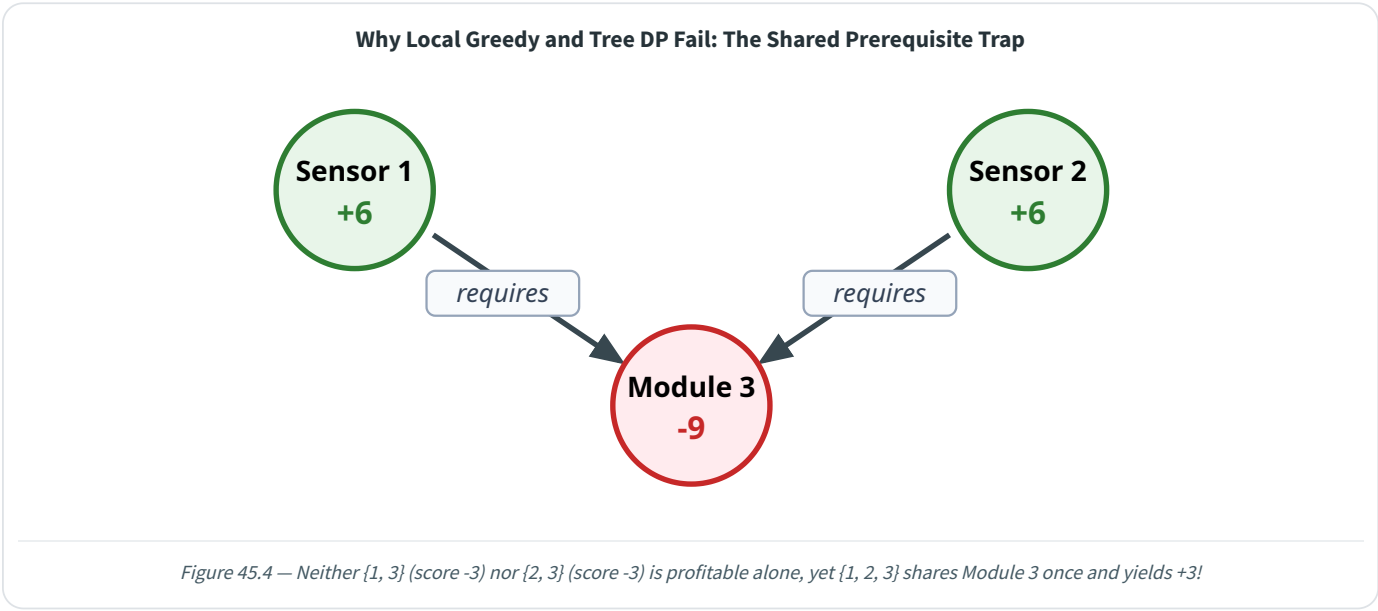



*Figure 45.4 — Neither {1, 3} (score -3) nor {2, 3} (score -3) is profitable alone, yet {1, 2, 3} shares Module 3 once and yields +3!*

- Evaluating Sensor $1$ alone with its prerequisite $\{1, 3\}$ gives $+6 - 9 = -3 < 0$ (unprofitable).
- Evaluating Sensor $2$ alone with $\{2, 3\}$ gives $+6 - 9 = -3 < 0$ (unprofitable).
- Yet activating $\{1, 2, 3\}$ together pays the $-9$ cost **only once** and achieves $+6 + 6 - 9 = +3 > 0$!
- Furthermore, why does **Flow Conservation (Lens 2)** not apply directly here? Because Module $3$ does not "split" $-9$ into two halves: once Module $3$ is paid for, it satisfies *all* sensors that depend on it simultaneously. And why does **Tree DP** fail? Because the dependency graph is a general directed graph (DAG with shared prerequisites, or even cycles $a \to b \to a$).

### Step 3 — The Turn: Min-Cut as a Binary Partition ($S$ vs. $T$) & "Ideal Gain Minus Regret"

Look back at the Decision Compass (Figure 45.1): each instrument $i$ requires a **binary decision**:

- **Activated (YES)** $\Leftrightarrow$ we place vertex $i$ on the source side: $i \in S$ (where $s \in S$).
- **Inactive (NO)** $\Leftrightarrow$ we place vertex $i$ on the sink side: $i \in T$ (where $t \in T$).

Recall the fundamental asymmetry of an $(S, T)$ cut in a directed graph:
$$\text{cap}(S, T) = \sum_{\substack{u \to v \\ u \in S,\, v \in T}} c(u \to v)$$
A directed edge $u \xrightarrow{c} v$ is **cut** (and adds $c$ to the bill) **if and only if** $u \in S$ **(YES) and** $v \in T$ **(NO)**!

## 1. Modeling Strict Dependencies ($a \to b$) and Soft Penalties ($u, v, c$)

Check the 4 possible states of $(a, b)$ across $(S, T)$:

| State of $a$ | State of $b$ | Meaning | Strict Rule $a \to b$ | Soft Rule $(u, v, c)$ |
|---|---|---|---|---|
| $a \in S$ | $b \in S$ | Both activated | Allowed $(0)$ | Cost $0$ |
| $a \in T$ | $b \in T$ | Neither activated | Allowed $(0)$ | Cost $0$ |
| $a \in T$ | $b \in S$ | $b$ activated without $a$ | Allowed $(0)$ | Cost $0$ |
| $a \in S$ | $b \in T$ | $a$ **activated,** $b$ **omitted!** | **FORBIDDEN** $(\infty)$ | **Penalty** $c$ |

Therefore:

- Every **soft penalty** $(u, v, c)$ becomes a directed edge $u \xrightarrow{c} v$.
- Every **strict dependency** $a \to b$ becomes a directed edge $a \xrightarrow{\infty} b$, making any cut with $a \in S, b \in T$ infinitely expensive!

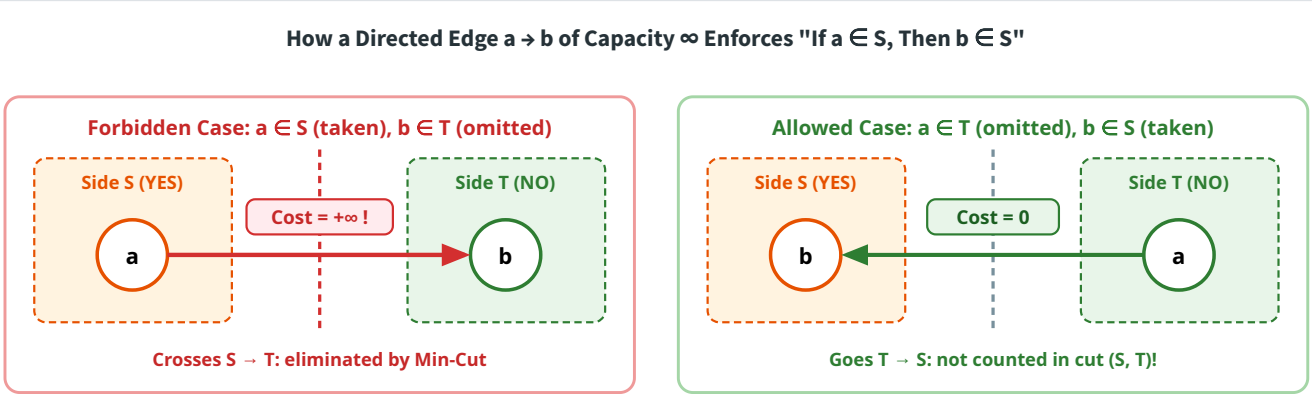



*Figure 45.5 — Directed cut asymmetry turns logical implications into capacity-∞ edges and soft penalties into capacity-c edges.*

## 2. The Accounting Turn: From "Maximizing Signed Profit" to "Minimizing Non-Negative Regret"

How do we turn **maximizing** a sum of signed values $w_i \in [-10^9, 10^9]$ into **minimizing** a cut with strictly **non-negative** capacities?

Imagine an optimistic banker pays us up front the **Ideal Gain** $P$= the sum of all positive rewards:

$$P = \sum_{w_i > 0} w_i$$

Compared to this utopian jackpot $P$, any actual valid selection $S$ loses money in only three ways (all $\geq 0$!):

1. **Foregone reward on a positive instrument ($w_i > 0$):** If we **leave** $i$ **inactive ($i \in T$)**, we must give back $w_i > 0$. We wire $s \xrightarrow{w_i} i$ so that the edge is cut ($s \in S, i \in T$) if and only if $i \in T$.
2. **Bill paid on a negative module ($w_j < 0$):** If we **activate** $j$ **($j \in S$)**, we must pay its cost $-w_j > 0$. We wire $j \xrightarrow{-w_j} t$ so that the edge is cut ($j \in S, t \in T$) if and only if $j \in S$.
3. **Soft penalty ($u \in S, v \in T$):** Paid by cutting $u \xrightarrow{c} v$.

$$\begin{aligned}\text{Score}(S) &= \textstyle\sum_{i \in S,\, w_i > 0} w_i + \sum_{j \in S,\, w_j < 0} w_j - \sum_{u \in S,\, v \in T} c(u, v) \\ &= \underbrace{\textstyle\sum_{w_i > 0} w_i}_{= P} - \underbrace{\left( \textstyle\sum_{i \in T,\, w_i > 0} w_i \;+\; \sum_{j \in S,\, w_j < 0} (-w_j) \;+\; \sum_{u \in S,\, v \in T} c(u, v) \right)}_{= \text{cap}(S, T) \;\geq\; 0} \\ \max_{S} \text{Score}(S) &= P - \min_{(S,T)} \text{cap}(S, T) \;=\; P - \text{MaxFlow}(s, t)\end{aligned}$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace a concrete 4-instrument instance:

- Weights: $w = [+12, -5, +8, -4]$ ($P = 12 + 8 = 20$).
- Strict dependencies: $1 \to 2$, $3 \to 2$, $3 \to 4$ (both sensors $1$ and $3$ share cooling module $2$, and sensor $3$ also requires module $4$).
- Soft penalty: $(1, 3, 2)$ (activating sensor $1$ without sensor $3$ incurs a penalty of $2$).

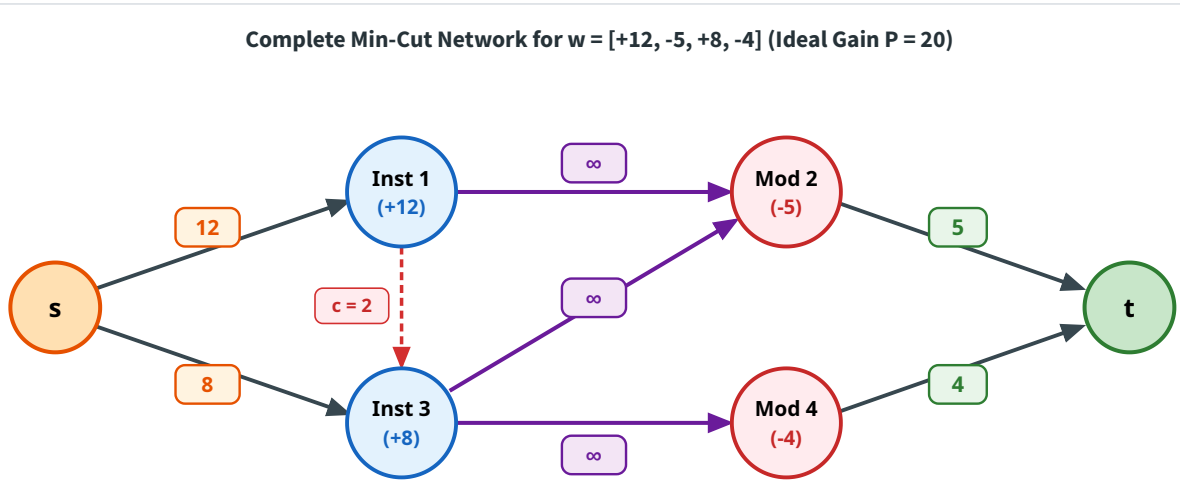



*Figure 45.6 — Full Min-Cut network: s → positive rewards, negative costs → t, strict dependencies = ∞, soft penalty = 2.*

Let us compare the three candidate valid cuts (none of which cuts an $\infty$ edge):

| **Chosen Set** $S \backslash \{s\}$ | **Cut Edges** ($S \to T$) | **Cut Capacity (Regret)** | **Net Score** $= P -$ Cut |
|---|---|---|---|
| $\emptyset$ $(S = \{s\})$ | $s \to 1$ (12) and $s \to 3$ (8) | $12 + 8 = 20$ | $20 - 20 = \mathbf{0}$ |
| $\{1, 2\}$ $(3, 4 \in T)$ | $s \to 3$ (8), $1 \to 3$ (2), $2 \to t$ (5) | $8 + 2 + 5 = 15$ | $20 - 15 = \mathbf{5}$ |
| $\{1, 2, 3, 4\}$ **(**$T = \{t\}$**)** | $2 \to t$ **(5) and** $4 \to t$ **(4)** | $5 + 4 = 9$ **(Min-Cut!)** | $20 - 9 = \mathbf{11}$ **(Optimal!)** |

### Step 5 — Two-Way Proof, Choosing `INF`, & Small-Case Oracle

- **How large must `INF` be in code?** Never use a careless 32-bit constant like `1e9` when $P$ can reach $200 \times 10^9 = 2 \cdot 10^{11}$! Notice the first row of our table: the trivial cut $S = \{s\}$ (activating nothing) **never cuts any internal edge** and always has capacity $P$. Therefore, the minimum cut is guaranteed to satisfy $\text{cap}(S^*, T^*) \leq P$. Setting:
$$\text{INF} = P + 1$$
in 64-bit `long long` guarantees that no minimum cut can ever cut even a single INF edge (since $P + 1 > P$) while staying completely safe from 64-bit integer overflow!
- **Two-Way Reduction Proof:**
  1. *Valid Selection* $\implies$ *Finite Cut:* Any subset $X$ satisfying all strict rules $a \to b$ defines $S = \{s\} \cup X, T = (\{1..N\} \backslash X) \cup \{t\}$ with no $a \in S, b \in T$ on any INF edge, and $\text{cap}(S, T) = P - \text{Score}(X)$.
  2. *Minimum Cut* $\implies$ *Optimal Selection:* Because $\text{cap}(\{s\}, V \backslash \{s\}) = P < \text{INF}$, any minimum cut $(S^*, T^*)$ cuts zero INF edges, so $X^* = S^* \backslash \{s\}$ obeys every strict rule $a \to b$ and achieves $\text{Score}(X^*) = P - \text{cap}(S^*, T^*) = P - \text{MaxFlow}(s, t)$.
  3. *Constructing the Optimal Subset* $X^*$*:* In the final residual graph after Dinic terminates, $S^*$ is the exact set of vertices reachable from $s$ via edges with strictly positive residual capacity ($\text{cap} - \text{flow} > 0$).

---

# Section 46 — Rerooting and Advanced Tree DP

## The Discovery & Modeling Approach

A standard Tree DP roots the tree arbitrarily at vertex $1$ and computes bottom-up aggregates $\mathrm{dp}[u]$ for the subtree of every vertex $u$ in $O(N)$ time. When a problem asks for the answer **if the entire tree were rooted at $r$, for every possible root $r \in \{1, \ldots, N\}$**, running $N$ separate Tree DPs takes $O(N^2)$ time.

The **Rerooting Technique** (Up-Down Tree DP) solves all $N$ roots in $O(N)$ total time via two DFS passes:

1. **Pass 1 (Bottom-Up `down[u]`):** Root at vertex $1$ and compute the contribution of $u$'s subtree.
2. **Pass 2 (Top-Down `up[v]` / Moving the Root $u \to v$):** When the root steps across a single tree edge from parent $u$ to child $v$, the tree partitions into two pieces: $v$'s subtree $\mathrm{Sub}(v)$ (which steps $1$ edge *closer* to the new root $v$) and its complement $V \setminus \mathrm{Sub}(v)$ (which steps $1$ edge *farther* from $v$).

### Decision Compass: Two Structural Lenses for Rerooting

How you compute the complement contribution when crossing $u \to v$ depends on a single algebraic question: **is the DP combining operator invertible?**

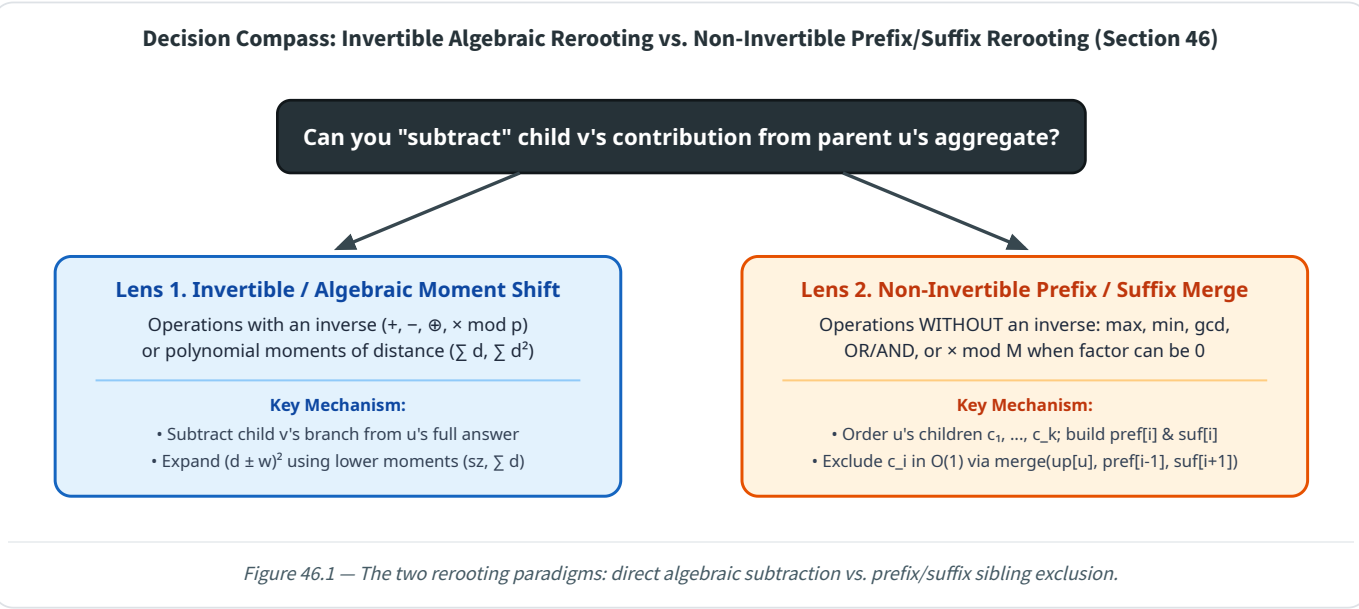



*Figure 46.1 — The two rerooting paradigms: direct algebraic subtraction vs. prefix/suffix sibling exclusion.*

| Property | Lens 1: Algebraic / Moment Rerooting | Lens 2: Prefix / Suffix Monoid Rerooting |
|---|---|---|
| **Typical Aggregations** | $\sum_x \mathrm{dist}(r,x)$, $\sum_x w_x \mathrm{dist}(r,x)^2$, subtree sizes, XOR | $\max_x \mathrm{dist}(r,x)$, $\min$, $\gcd$, DP with $\times \bmod M$ (zeros possible) |
| **Why Care Is Needed** | Nonlinear powers $(d \pm 1)^k$ couple $k$-th moments with $(0..k-1)$-th moments | You cannot do `max_all - child_v`; division by $0 \pmod M$ is undefined |
| **Transition Cost at $u \to c_i$** | $O(1)$ linear combination of moment tuple `(M0, M1, M2)` | $O(1)$ after precomputing prefix/suffix arrays over $u$'s children |

## Worked Problem 46.1: Squared-Distance Broadcast Cost (Algebraic Moment Rerooting)

### Problem & Constraints

A communication tree has $N$ servers ($1 \le N \le 2 \cdot 10^5$) connected by $N-1$ edges. Each server $x$ has a positive traffic weight $w_x \ge 1$. If we place the master controller at server $r$, the quadratic latency cost of the network is:

$$F_2(r) = \sum_{x=1}^{N} w_x \cdot \mathrm{dist}(r,x)^2$$

where $\mathrm{dist}(r,x)$ is the number of edges on the unique tree path between $r$ and $x$. Compute $F_2(r)$ for **every** server $r \in \{1,...,N\}$ in $O(N)$ total time.

> **Pause & Reflect (Before the Turn):** In the classic sum-of-distances problem $F_1(r) = \sum_x w_x \mathrm{dist}(r,x)$, moving the root across an edge $u \to v$ changes $\mathrm{dist}(\cdot,x)$ by $-1$ for $x \in \mathrm{Sub}(v)$ and by $+1$ for $x \notin \mathrm{Sub}(v)$, so you only need the 0th moment $W(v) = \sum_{x \in \mathrm{Sub}(v)} w_x$. What happens when you expand $(d-1)^2$ and $(d+1)^2$? Which additional moment must be carried in your state to make the transition closed?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** For each candidate root $r \in \{1,...,N\}$, run a BFS/DFS from $r$ to find $\mathrm{dist}(r,x)$ for all $x$ and accumulate $\sum_x w_x \mathrm{dist}(r,x)^2$.
2. **Bottleneck:** $N$ independent traversals take $\Theta(N^2)$ time, which times out for $N = 2 \cdot 10^5$.

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake (Applying a linear delta directly to $F_2$):** A common error is to write $F_2(v) = F_2(u) - W_{\text{in}} + W_{\text{out}}$. That fails even on a 3-node path $1-2-3$ with unit weights $w_x = 1$:
  - At $u = 1$: distances are $(0,1,2)$, so $F_2(1) = 0^2 + 1^2 + 2^2 = 5$.
  - Moving to $v = 2$: distances become $(1,0,1)$, so $F_2(2) = 1^2 + 0^2 + 1^2 = 2$.
  - Notice that vertex 3's squared distance dropped from $2^2 = 4$ to $1^2 = 1$ (a drop of 3, not 1!), because $d^2 - (d-1)^2 = 2d - 1$ depends on the **first-order distance** $d$!

### Step 3 — The Turn: Closing the Algebraic State with Lower Moments

When the current root moves across edge $u \to v$:

- For every vertex $x \in \mathrm{Sub}(v)$, its distance to the root changes from $d = \mathrm{dist}(u,x)$ to $d-1$:
$$(d-1)^2 = d^2 - 2d + 1$$
- For every vertex $y \notin \mathrm{Sub}(v)$, its distance to the root changes from $d = \mathrm{dist}(u,y)$ to $d+1$:
$$(d+1)^2 = d^2 + 2d + 1$$

Summing over both sets shows that to update the **2nd moment** $F_2$, we need the **1st moment** (sum of weighted distances) and the **0th moment** (sum of weights) of both $\mathrm{Sub}(v)$ and $V \backslash \mathrm{Sub}(v)$!

Let us define for each subtree $\mathrm{Sub}(v)$ (in Pass 1) and for the full tree at root $u$ (in Pass 2):

- **0th Moment:** $M_0(v) = \sum_{x \in \mathrm{Sub}(v)} w_x$, and total weight $W_{\text{tot}} = M_0(1)$.
- **1st Moment:** $M_1(v) = \sum_{x \in \mathrm{Sub}(v)} w_x \mathrm{dist}(v,x)$, and global $F_1(u) = \sum_{x \in V} w_x \mathrm{dist}(u,x)$.
- **2nd Moment:** $M_2(v) = \sum_{x \in \mathrm{Sub}(v)} w_x \mathrm{dist}(v,x)^2$, and global $F_2(u) = \sum_{x \in V} w_x \mathrm{dist}(u,x)^2$.

Note that for $x \in \mathrm{Sub}(v)$, $\mathrm{dist}(u,x) = \mathrm{dist}(v,x) + 1$, so the 1st moment of $V \backslash \mathrm{Sub}(v)$ measured from $u$ is:
$$F_1^{\text{out}}(u \backslash v) = F_1(u) - \big( M_1(v) + M_0(v) \big)$$

$$\begin{aligned} F_1(v) &= F_1(u) - M_0(v) + \big( W_{\text{tot}} - M_0(v) \big) \\ F_2(v) &= F_2(u) - \big( 2M_1(v) + M_0(v) \big) \\ &\quad + 2 \underbrace{\big( F_1(u) - M_1(v) - M_0(v) \big)}_{F_1^{\text{out}}(u \backslash v)} + \big( W_{\text{tot}} - M_0(v) \big) \end{aligned}$$

### Step 4 — Progressive Construction & Visual Trace

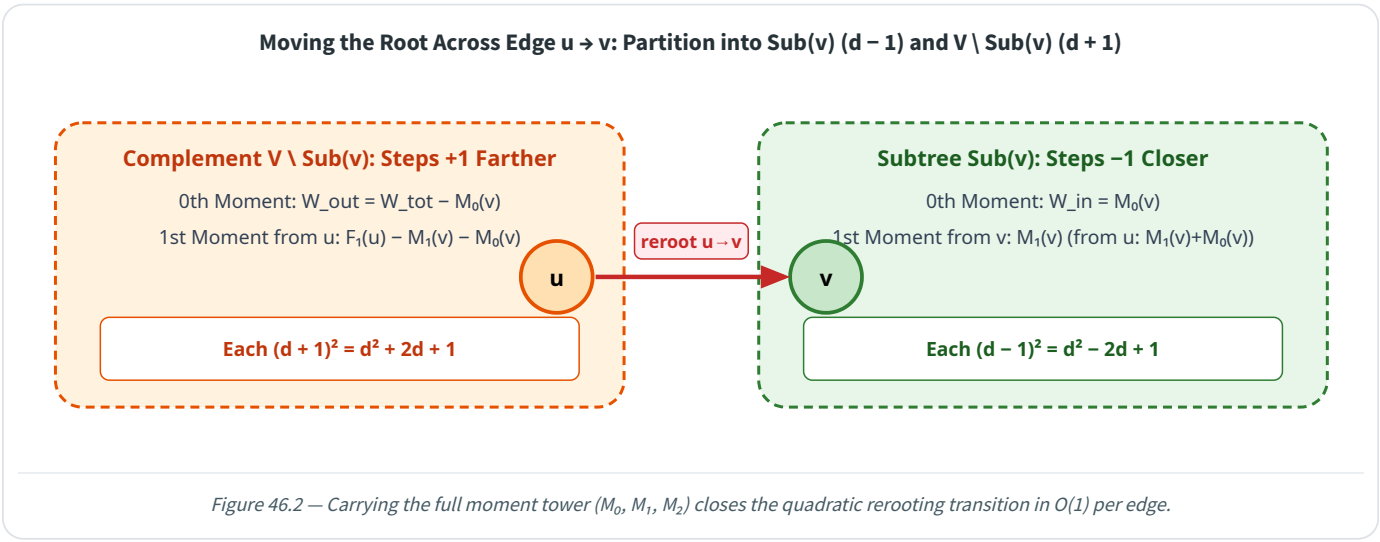



*Figure 46.2 — Carrying the full moment tower ($M_0$, $M_1$, $M_2$) closes the quadratic rerooting transition in O(1) per edge.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Algebraic Identity Proof:** For $x \in \mathrm{Sub}(v)$, let $d_v = \mathrm{dist}(v, x)$, so $\mathrm{dist}(u, x) = d_v + 1$. When moving from $u$ to $v$, the change in $x$'s squared distance is $d_v^2 - (d_v + 1)^2 = -2d_v - 1$. Summing $w_x(-2d_v - 1)$ over $x \in \mathrm{Sub}(v)$ gives exactly $-2M_1(v) - M_0(v)$. For $y \notin \mathrm{Sub}(v)$, let $d_u = \mathrm{dist}(u, y)$, so $\mathrm{dist}(v, y) = d_u + 1$. The change is $(d_u + 1)^2 - d_u^2 = 2d_u + 1$. Summing $w_y(2d_u + 1)$ over $y \notin \mathrm{Sub}(v)$ gives $2F_1^{\mathrm{out}}(u \backslash v) + (W_{\mathrm{tot}} - M_0(v))$. Since $V = \mathrm{Sub}(v) \sqcup (V \backslash \mathrm{Sub}(v))$, the formula is exact.
- **Small-Case Oracle:** Compare $F_2(1..N)$ against an $O(N^2)$ all-pairs DFS on random weighted trees of size $N \leq 15$.

---

## Worked Problem 46.2: All-Roots Longest Relay Path (Non-Invertible Prefix/Suffix Rerooting)

### Problem & Constraints

You are given a weighted tree with $N$ vertices ($1 \leq N \leq 2 \cdot 10^5$) where each edge $(u, v)$ has a positive length $\ell(u, v) > 0$. For **every** vertex $r \in \{1, \ldots, N\}$, compute the **eccentricity** of $r$:

$$E(r) = \max_{x \in V} \mathrm{dist}(r, x)$$

(and more generally, explain how to do this for *any* associative commutative DP transition that has **no inverse**, such as $\max$, $\min$, $\gcd$, or multiplication modulo a composite number $M$).

> **Pause & Reflect (Before the Turn):** In Pass 1, $\mathrm{down}[u] = \max_{c \in \mathrm{children}(u)} \big( \ell(u, c) + \mathrm{down}[c] \big)$ is easy to compute. In Pass 2, to compute $\mathrm{up}[c_i]$ (the longest path starting at $c_i$, stepping up to parent $u$, and continuing into $V \backslash \mathrm{Sub}(c_i)$), we need the maximum over all branches of $u$ **except** $c_i$ **itself**. Since $\max$ has no subtraction operator, how can we compute $\max_{j \neq i} B_j$ for all $k$ children of $u$ in $O(k)$ total time without ever dividing or subtracting?

### Step 1 — Exact Model & Bottleneck

1. **Exact model:** At parent $u$ with children $c_1, c_2, \ldots, c_k$, each child $c_j$ offers a branch length $B_j = \ell(u, c_j) + \mathrm{down}[c_j]$ into $\mathrm{Sub}(c_j)$. When computing $\mathrm{up}[c_i]$ (the longest path from $c_i$ that initially goes up to $u$), the path at $u$ can either continue further up into $u$'s parent (length $\mathrm{up}[u]$) or go down into a **sibling** $c_j$ ($j \neq i$, length $B_j$):
$$\mathrm{up}[c_i] = \ell(u, c_i) + \max \Big( \mathrm{up}[u], \max_{j \neq i} B_j \Big)$$
2. **Bottleneck:** If vertex $u$ is the center of a star graph with $k = N - 1$ children, scanning all $j \neq i$ separately for each child $c_i$ takes $O(k^2) = \Theta(N^2)$ time!

### Step 2 — Tempting Mistakes & Counterexample

- **Mistake 1 (Naive loop over siblings $j \neq i$):** Works in $O(N)$ on binary trees, but degrades to $\Theta(N^2)$ on star or broom graphs—a classic TLE on tree problems!

- **Mistake 2 (Relying on "top-2 maxima" when the DP operator is more complex):** While keeping the largest and second-largest values works specifically for $\max$, it does **not** generalize to $\gcd$, bitwise OR, DP polynomials, or modular products $\prod(1+\mathrm{dp}[c_j]) \bmod M$ where one factor might be $0 \pmod M$. The Prefix/Suffix method works universally for **every** associative operation!

### Step 3 — The Turn: Prefix & Suffix Aggregations Over $u$'s Children

For a fixed parent $u$ with $k$ children $c_1, c_2, \ldots, c_k$ and branch values $B_j = \ell(u, c_j) + \mathrm{down}[c_j]$, the set of all siblings $\{1, \ldots, k\} \setminus \{i\}$ is the disjoint union of the **prefix** $\{1, \ldots, i-1\}$ and the **suffix** $\{i+1, \ldots, k\}$!

Precompute two arrays of length $k$ in $O(k)$ time:

- $\mathrm{pref}[i] = \max_{1 \le j \le i} B_j$ (with $\mathrm{pref}[0] = 0$)
- $\mathrm{suf}[i] = \max_{i \le j \le k} B_j$ (with $\mathrm{suf}[k+1] = 0$)

Then for each child $c_i$ ($1 \le i \le k$), the maximum over all siblings $j \ne i$ is simply $\max(\mathrm{pref}[i-1], \mathrm{suf}[i+1])$ in $O(1)$ **time**!

$$\begin{aligned} \mathrm{ExcludeChild}(u, i) &= \mathrm{op}\left(\ \mathrm{up}[u],\ \mathrm{pref}[i-1],\ \mathrm{suf}[i+1]\ \right) \\ \mathrm{up}[c_i] &= \ell(u, c_i) + \max\left(\ \mathrm{up}[u],\ \mathrm{pref}[i-1],\ \mathrm{suf}[i+1]\ \right) \\ E(u) &= \max\left(\ \mathrm{down}[u],\ \mathrm{up}[u]\ \right) \end{aligned}$$

### Step 4 — Progressive Construction & Visual Trace

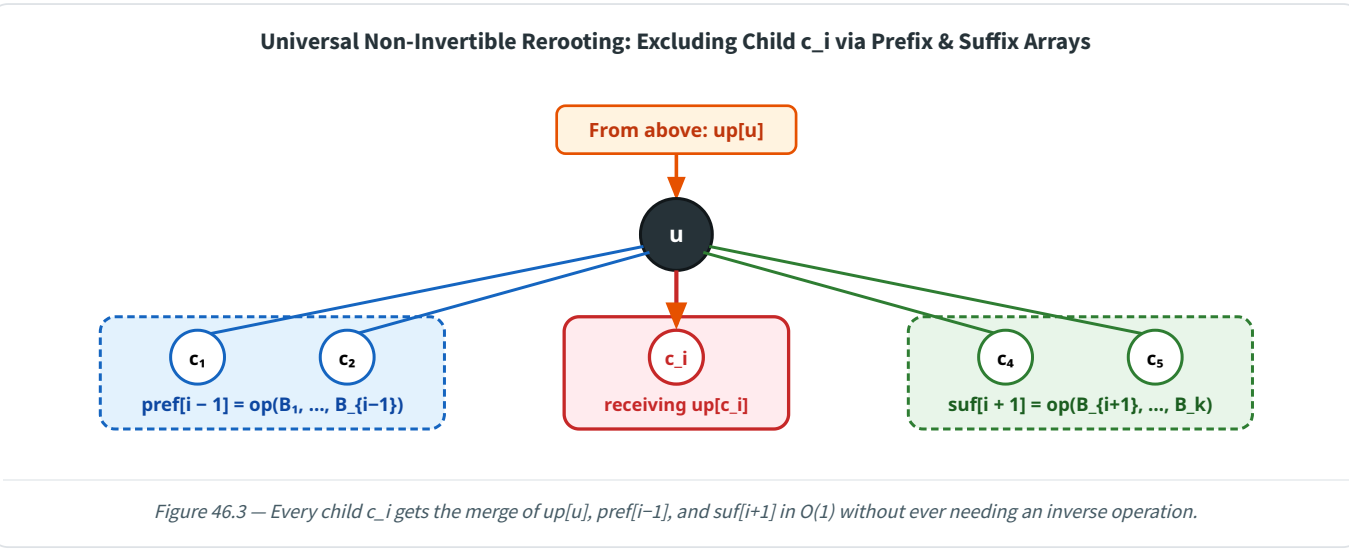



*Figure 46.3 — Every child c_i gets the merge of up[u], pref[i−1], and suf[i+1] in O(1) without ever needing an inverse operation.*

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Partition & Associativity Proof:** Every simple path starting at $u$ and staying inside $V \setminus \mathrm{Sub}(c_i)$ either takes its first step to $\mathrm{parent}(u)$ (captured by $\mathrm{up}[u]$), to a left sibling $c_j$ with $1 \le j < i$ (captured by $\mathrm{pref}[i-1]$), or to a right sibling $c_j$ with $i < j \le k$ (captured by $\mathrm{suf}[i+1]$). Since these three sets of paths partition all valid continuations from $u$ outside $\mathrm{Sub}(c_i)$, associativity of the DP operator guarantees that merging those three summaries yields the exact state of $u$ when rooted at $c_i$.
- **Linear Complexity Proof:** At each vertex $u$ with $k = \deg(u) - 1$ children, building $\mathrm{pref}[1..k]$ and $\mathrm{suf}[1..k]$ and computing $\mathrm{up}[c_1..c_k]$ takes $O(k)$ operations. Summing over all $u \in V$ gives $\sum_u O(\deg(u)) = O(N)$ total time and space.
- **Small-Case Oracle:** Test on star graphs, caterpillars, and random trees ($N \le 20$) against an $O(N^2)$ oracle that roots the tree freshly at each $r \in \{1, \ldots, N\}$ and runs only Pass 1 (`down[r]`).

# Section 47 — Meet in the Middle

## The Discovery & Modeling Approach

When a problem has a tiny constraint on the number of items or steps—typically $N \in [30, 46]$ for binary choices or $N \in [20, 28]$ for ternary choices—and its values or weights are large enough ($10^9$ to $10^{18}$) to rule out dynamic programming over the value domain, your immediate diagnostic should be **Meet in the Middle (MitM)**.

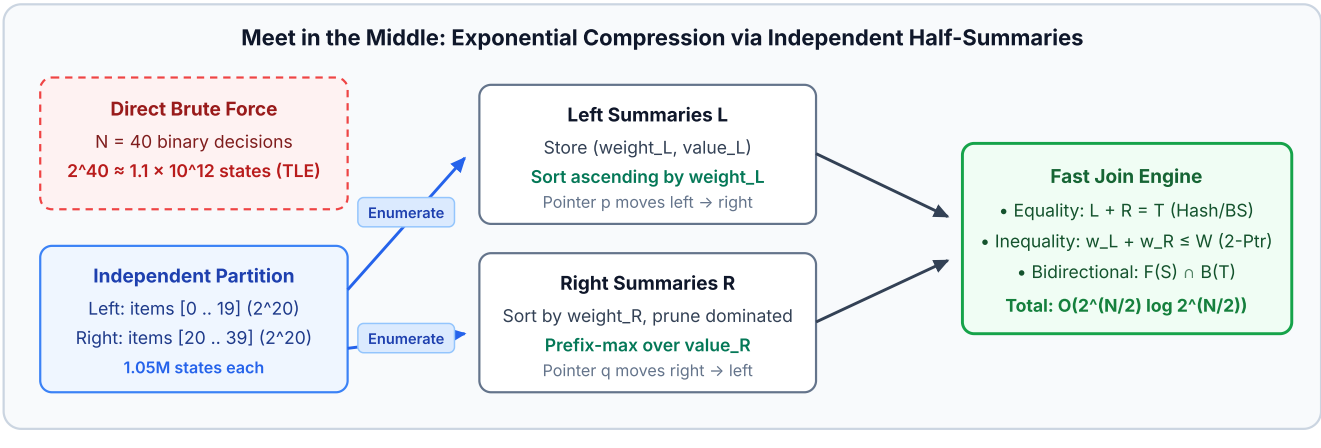
Why does $N = 40$ sit in a special regime? Because a direct exhaustive search over $2^{40} \approx 1.1 \times 10^{12}$ states exceeds time limits by four orders of magnitude, yet half of the decisions—$N/2 = 20$—produce only $2^{20} = 1,048,576$ states. One million partial configurations easily fit in memory and can be sorted or indexed in fraction of a second.

However, splitting the decision space in half is only useful if the two halves can be **recombined without iterating over all pairs** $(x, y) \in \mathcal{L} \times \mathcal{R}$ (which would bring us right back to $|\mathcal{L}| \cdot |\mathcal{R}| = 2^{N/2} \cdot 2^{N/2} = 2^N$). The discovery process for Meet in the Middle therefore revolves around one structural question:

> **The Separation & Join Question:** Can a full decision be decomposed into two independent partial decisions whose summaries combine via an equality test, an order inequality, or an algebraic projection?

### Decision Compass & Structural Lenses

| Sub-Pattern | Signal in Constraints & Statement | Left/Right State Summary | Recombination Mechanism | Time & Space Complexity |
|---|---|---|---|---|
| **Equality / Exact Target Join** | $N \leq 40$, exact target sum/XOR/hash $L + R = T$, large weights up to $10^{18}$ | Scalar summary $x \in \mathcal{L}$ and $y \in \mathcal{R}$ | Hash table lookup or sorted binary search for $y = T - x$ | $\mathcal{O}(2^{N/2} \cdot N)$ or $\mathcal{O}(2^{N/2} \log(2^{N/2}))$ time, $\mathcal{O}(2^{N/2})$ space |
| **Inequality / Optimization Join** | $N \leq 40$, constraint $L + R \leq W$, maximize value or count valid subsets | Pair $(w_L, v_L)$ sorted by weight $w_L$; prefix-max on $v_R$ | Two pointers (`left` increasing, `right` decreasing) or `upper_bound` | $\mathcal{O}(2^{N/2} \log(2^{N/2}))$ time, $\mathcal{O}(2^{N/2})$ space |
| **Ternary / Partition Split** | $N \leq 26$, choose two disjoint subsets with equal sum or balanced 3-way split | Choices $\{-1, 0, +1\}^k$ giving $3^{N/2}$ states; group by $\Delta = \text{sum}$ | Sort by $\Delta_L = -\Delta_R$, track maximum covered sum inside each bucket | $\mathcal{O}(3^{N/2} \log(3^{N/2}))$ time, $\mathcal{O}(3^{N/2})$ space |
| **Bidirectional State-Space Search** | Reversible transitions, huge implicit graph, fixed start $S$ and target $T$, depth $K \leq 20$ | Set of states reachable in $\lfloor K/2 \rfloor$ steps from $S$ and $\lceil K/2 \rceil$ inverse steps from $T$ | Hash-set intersection $\mathcal{F}_{\lfloor K/2 \rfloor}(S) \cap \mathcal{B}_{\lceil K/2 \rceil}(T)$ | $\mathcal{O}(b^{K/2})$ time and space, where $b$ is branching factor |



## Worked Problem 47.1: Payload Relay under Dual-Budget Thresholds

### Problem & Constraints

A deep-space probe carries $N$ scientific instruments ($1 \le N \le 40$). Instrument $i$ requires $w_i$ milliwatts of peak power ($1 \le w_i \le 10^{15}$) and transmits $v_i$ gigabits of telemetry ($1 \le v_i \le 10^{15}$). You must choose a subset of instruments to activate simultaneously such that: 1. Their total peak power does not exceed the bus limit $W$ ($1 \le W \le 10^{16}$), and 2. Their total telemetry is as large as possible.

In addition, report how many distinct subsets achieve a total peak power at most $W$ (regardless of telemetry).

> **Pause & Discover Before Reading On:**
> 1. Why does standard $0/1$ Knapsack dynamic programming fail immediately here?
> 2. If we split the $N$ instruments into two halves $A$ and $B$ of size at most $20$, each subset of $A$ gives a pair $(w_A, v_A)$ and each subset of $B$ gives $(w_B, v_B)$. How can we find the best partner in $B$ for every subset of $A$ without checking all $2^{20}$ candidates in $B$?
> 3. Why can some pairs $(w_B, v_B)$ be permanently discarded before we even look at $A$?

### Step 1 — Exact Model & Bottleneck

Every subset $S \subseteq \{0, 1, \ldots, N-1\}$ is characterized by its total weight $w(S) = \sum_{i \in S} w_i$ and total value $v(S) = \sum_{i \in S} v_i$. The exact mathematical problem is:

$$\max_{S \subseteq \{0,\ldots,N-1\}} \{v(S) \mid w(S) \le W\} \quad \text{and} \quad \left|\{S \subseteq \{0,\ldots,N-1\} \mid w(S) \le W\}\right|.$$

- **Bottleneck 1 (Value domain):** A DP table indexed by weight requires $O(W)$ states, and $W \le 10^{16}$ is astronomical. Even coordinate compression fails because there can be up to $2^{40}$ distinct subset sums.
- **Bottleneck 2 (Subset domain):** Exhaustively testing all $2^{40} \approx 1.1 \times 10^{12}$ subsets takes thousands of seconds.

Notice that the transition between instruments has **zero sequential dependency**: whether instrument $i$ is chosen does not constrain which other instruments are legal, except through the global additive sum $\sum_{i \in S} w_i \le W$.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Value-to-Weight Ratio Greedy):** Sorting instruments by $v_i/w_i$ descending and taking instruments while capacity remains fails on 0/1 indivisible items.
  - *Counterexample:* $W = 10$, items $(w_1, v_1) = (6, 7)$, $(w_2, v_2) = (5, 5)$, $(w_3, v_3) = (5, 5)$. Greedy picks item 1 for value 7 and gets stuck; taking items 2 and 3 achieves weight $10 \le 10$ and value $10 > 7$.
- **Tempting Mistake 2 (Binary searching on $B$ without dominance pruning):** Suppose we sort the $2^{20}$ subsets of $B$ by weight $w_B$. For a fixed $w_A$, the valid partners in $B$ form a prefix $\{(w_B, v_B) \mid w_B \le W - w_A\}$. If you only binary search for the rightmost valid $w_B$ and take its value $v_B$, you might pick a heavier subset of $B$ that has *smaller* telemetry than a lighter subset of $B$ earlier in the prefix!

### Step 3 — The Turn: Discovering the Structure

Partition the $N$ instruments into a left half $L = \{0, \ldots, m-1\}$ where $m = \lfloor N/2 \rfloor \le 20$, and a right half $R = \{m, \ldots, N-1\}$ of size $N - m \le 20$. Every subset $S$ splits uniquely as $S = S_L \cup S_R$ with $S_L \subseteq L$ and $S_R \subseteq R$, and:

$$w(S) = w(S_L) + w(S_R), \qquad v(S) = v(S_L) + v(S_R).$$

Generate the list of all $2^m$ pairs $\mathcal{L} = \{(w(S_L), v(S_L))\}$ and all $2^{N-m}$ pairs $\mathcal{R} = \{(w(S_R), v(S_R))\}$. Sort both $\mathcal{L}$ and $\mathcal{R}$ in nondecreasing order of weight $w$.

**Monotone Prefix-Envelope & Two-Pointer Join:**
For any fixed left choice $(w_L, v_L) \in \mathcal{L}$, the feasible right choices are precisely the prefix of $\mathcal{R}$ satisfying:

$$w_R \leq W - w_L.$$

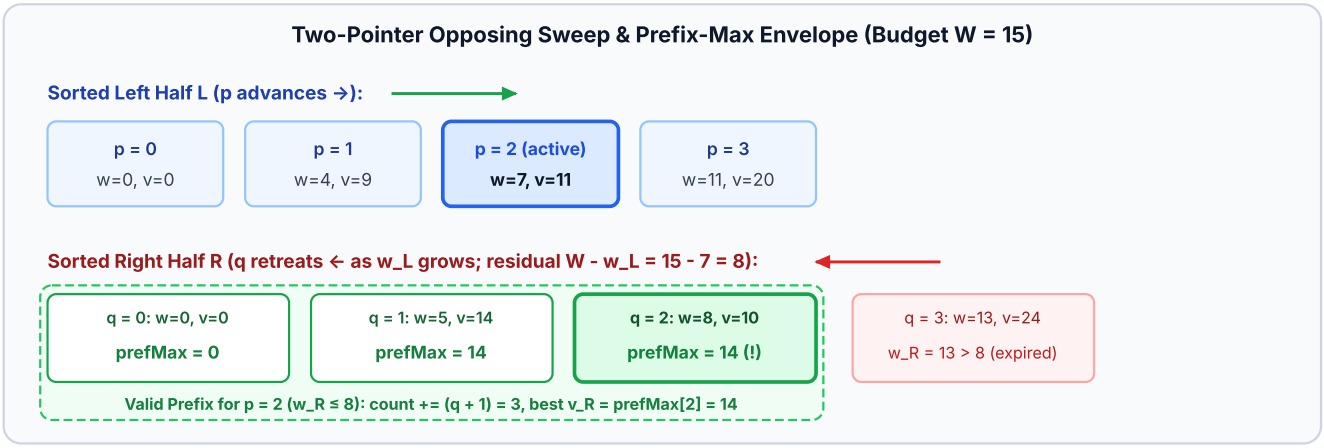
As $w_L$ increases in sorted order, the residual budget $W - w_L$ **monotonically decreases**. Therefore, the rightmost feasible index $q$ in sorted $\mathcal{R}$ only moves **leftward** ($q$ decreases from $|\mathcal{R}|-1$ down to $-1$).

During this single sweep of $p = 0 \ldots |\mathcal{L}|-1$ and $q = |\mathcal{R}|-1 \ldots 0$:

- **Count of valid subsets:** Add $(q+1)$ to the running count.
- **Maximum telemetry:** Combine $v_L$ with $\text{prefMax}_R[q] = \max_{0 \leq k \leq q} v_R[k]$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace a concrete micro-instance or structure of $\mathcal{L}$ and $\mathcal{R}$. Furthermore, observe an implementation optimization: we can generate $\mathcal{L}$ and $\mathcal{R}$ **already sorted by weight** in $O(2^{N/2})$ time (without the $\log(2^{N/2})$ sorting factor!) by iterating items $i$ one by one and merging the existing sorted list $V$ with the shifted list $V + (w_i, v_i)$ via `std::merge`.




Notice what happens at $q = 2$ in the diagram: the subset at $q = 2$ has weight $8$ and value $10$, whereas the lighter subset at $q = 1$ had weight $5$ and value $14$. Because we maintain $\text{prefMax}[q] = \max_{0 \leq k \leq q} v_R[k]$, pointer $q = 2$ immediately reports both the exact count of valid right partners ($q + 1 = 3$) and the maximum telemetry achievable in that prefix ($\text{prefMax}[2] = 14$), yielding $v_L + \text{prefMax}[2] = 11 + 14 = 25$.

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

struct State {
    long long w, v;
    bool operator<(const State& other) const {
        return w < other.w;
    }
};

// Generates all 2^k subset summaries already sorted by weight in O(2^k) time
vector<State> build_half(const vector<pair<long long, long long>>& items) {
    vector<State> cur = {{0, 0}};
    for (auto [w_i, v_i] : items) {
        vector<State> shifted;
        shifted.reserve(cur.size());
        for (const auto& s : cur) {
            shifted.push_back({s.w + w_i, s.v + v_i});
```

```
        }
        vector<State> merged;
        merged.reserve(cur.size() * 2);
        merge(cur.begin(), cur.end(), shifted.begin(), shifted.end(), back_inserter(merged));
        cur.swap(merged);
    }
    return cur;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long W;
    if (!(cin >> n >> W)) return 0;

    int mid = n / 2;
    vector<pair<long long, long long>> left_items(mid), right_items(n - mid);
    for (int i = 0; i < mid; ++i) cin >> left_items[i].first >> left_items[i].second;
    for (int i = mid; i < n; ++i) cin >> right_items[i - mid].first >> right_items[i - mid].second;

    vector<State> L = build_half(left_items);
    vector<State> R = build_half(right_items);

    // Precompute prefix maximums of value on R
    vector<long long> pref_max(R.size());
    long long run_max = 0;
    for (size_t i = 0; i < R.size(); ++i) {
        run_max = max(run_max, R[i].v);
        pref_max[i] = run_max;
    }

    long long valid_subsets = 0;
    long long best_telemetry = 0;
    int q = static_cast<int>(R.size()) - 1;

    for (const auto& l_state : L) {
        if (l_state.w > W) break;
        while (q >= 0 && l_state.w + R[q].w > W) {
            --q;
        }
        if (q >= 0) {
            valid_subsets += (q + 1);
            best_telemetry = max(best_telemetry, l_state.v + pref_max[q]);
        }
    }

    cout << best_telemetry << " " << valid_subsets << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Bijection of Subsets:** Every subset $S \subseteq \{0, \ldots, N-1\}$ corresponds to a unique pair $(S \cap L, S \cap R) \in 2^L \times 2^R$. Since `build_half` generates one entry per subset of $L$ and $R$, no valid subset is omitted or double-counted.
- **Monotonicity of Pointer $q$:** Since `L` is sorted with $w_L[p] \leq w_L[p+1]$, we have $W - w_L[p+1] \leq W - w_L[p]$. Any index $q'$ that violated $w_R[q'] \leq W - w_L[p]$ also violates $w_R[q'] \leq W - w_L[p+1]$. Thus $q$ never needs to move right, and the two-pointer pass takes $O(|\mathcal{L}|+|\mathcal{R}|) = O(2^{N/2})$ steps.
- **Small-Case Oracle:** Test against a $2^N$ bitmask loop for $N = 10$ with random weights and values in $[1, 50]$, explicitly including cases with duplicate weights and cases where a heavier subset has strictly smaller telemetry.

---

## Worked Problem 47.2: Reversible Permutation Cipher Breakdown

### Problem & Constraints

A cryptographic hardware module holds a register of $M = 12$ distinct bytes, initialized to the identity permutation $P_{\text{start}} = [0, 1, 2, ..., 11]$. In each clock cycle, the module can apply one of $d = 4$ fixed wiring permutations $\pi_1, \pi_2, \pi_3, \pi_4 \in S_{12}$: applying $\pi_r$ moves the byte at index $j$ to index $\pi_r(j)$.

Given a target permutation $P_{\text{target}}$ and a step limit $K = 14$, determine the minimum number of clock cycles $0 \le k \le K$ required to transform $P_{\text{start}}$ into $P_{\text{target}}$, or report $-1$ if it cannot be reached within $K$ cycles.

**Pause & Discover Before Reading On:**

1. How many states could a standard Breadth-First Search (BFS) from $P_{\text{start}}$ visit up to depth $K = 14$ with branching factor $4$?
2. Are the transitions $\pi_1, ..., \pi_4$ invertible? If we know the exact destination $P_{\text{target}}$, how can we search backward from $P_{\text{target}}$?
3. How do we guarantee we find the *minimum* total steps $k = k_F + k_B$ rather than just any collision?

### Step 1 — Exact Model & Bottleneck

Model the system as an unweighted directed graph where the vertices are the $12! = 479,001,600$ permutations of $\{0, ..., 11\}$, and each vertex $P$ has $4$ outgoing edges $P \to \pi_r \circ P$. We want the shortest path from $P_{\text{start}}$ to $P_{\text{target}}$ of length at most $K = 14$.

- **Bottleneck:** A one-directional BFS up to depth $14$ with branching factor $4$ explores up to $\sum_{i=0}^{14} 4^i \approx \frac{4^{15}}{3} \approx 3.57 \times 10^8$ paths (bounded by $12! \approx 4.79 \times 10^8$ states), exhausting both time and memory limits.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Applying the forward permutation $\pi_r$ during the backward search):** A permutation $\pi_r$ is generally not an involution ($\pi_r \circ \pi_r \neq \text{id}$). If a forward edge goes from $U$ to $V = \pi_r(U)$, then stepping backward from $V$ to $U$ requires applying the **inverse permutation** $\pi_r^{-1}$, not $\pi_r$!
  - *Counterexample:* Let $\pi_1$ be a cyclic left shift on $3$ elements: $[0, 1, 2] \xrightarrow{\pi_1} [1, 2, 0]$. If $P_{\text{target}} = [1, 2, 0]$ is 1 step away from $P_{\text{start}} = [0, 1, 2]$, applying $\pi_1$ backward to $P_{\text{target}}$ yields $[2, 0, 1] \neq P_{\text{start}}$, missing the 1-step solution completely.
- **Tempting Mistake 2 (Stopping at the first hash collision without layer synchronization):** If you expand all $\le 7$ steps backward first and then check forward states, a state $X$ reached at forward depth $d_F = 3$ might match both a backward state at $d_B = 7$ (total $10$) and another state $Y$ at $d_F = 4$ might match $d_B = 4$ (total $8$). You must either expand layer-by-layer or minimize $d_F(X) + d_B(X)$ across all colliding states $X$.

### Step 3 — The Turn: Discovering the Structure

Every permutation $\pi_r$ is a bijection on $\{0, ..., 11\}$, so its inverse $\pi_r^{-1}$ exists and is unique:

$$V = \pi_r \circ U \Leftrightarrow U = \pi_r^{-1} \circ V.$$

Consequently, a path of length $k \le 14$ from $P_{\text{start}}$ to $P_{\text{target}}$ exists if and only if there is a midpoint permutation $M$ at distance $d_F = \lfloor k/2 \rfloor \le 7$ forward from $P_{\text{start}}$ and distance $d_B = \lceil k/2 \rceil \le 7$ backward (via $\pi_r^{-1}$) from $P_{\text{target}}$.

At depth $K/2 = 7$, the number of states in each half is at most $\sum_{i=0}^{7} 4^i = \frac{4^8 - 1}{3} = 21,845$—reducing a $3.5 \times 10^8$-state search to twenty thousand states!

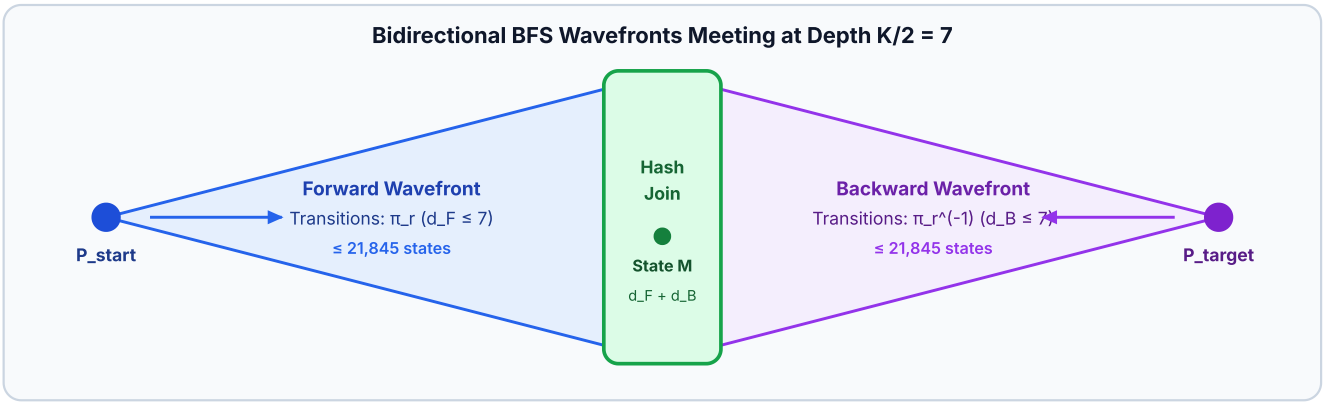
**Bidirectional Meet-in-the-Middle Criterion:**

Pack each 12-element permutation of $\{0, ..., 11\}$ into a single 64-bit integer (4 bits per position: $12 \times 4 = 48$ bits). Run a forward BFS from $P_{\text{start}}$ up to depth 7 storing $\text{dist}_F[X]$, and a backward BFS using $\pi_r^{-1}$ from $P_{\text{target}}$ up to depth 7 storing $\text{dist}_B[X]$. Then:

$$d_{\min} = \min_{X \,\in\, \text{dom}(\text{dist}_F) \,\cap\, \text{dom}(\text{dist}_B)} \left( \text{dist}_F[X] + \text{dist}_B[X] \right).$$

### Step 4 — Progressive Construction & Visual Trace

Let us visualize how the forward and backward wavefronts shrink the search radius from $K$ to $K/2$, and how 48-bit nibble packing makes each state a primitive `uint64_t`.




```cpp
#include <algorithm>
#include <array>
#include <cstdint>
#include <iostream>
#include <queue>
#include <unordered_map>
#include <vector>

using namespace std;

using Perm = array<int, 12>;

uint64_t pack_perm(const Perm& p) {
    uint64_t code = 0;
    for (int i = 0; i < 12; ++i) {
        code |= (static_cast<uint64_t>(p[i]) << (4 * i));
    }
    return code;
}

Perm unpack_perm(uint64_t code) {
    Perm p{};
    for (int i = 0; i < 12; ++i) {
        p[i] = static_cast<int>((code >> (4 * i)) & 0xFULL);
    }
    return p;
}

Perm apply_op(const Perm& state, const Perm& op) {
    Perm next_state{};
    for (int j = 0; j < 12; ++j) {
        next_state[op[j]] = state[j];
    }
    return next_state;
}

unordered_map<uint64_t, int> bfs_half(uint64_t start_code, const vector<Perm>& ops, int max_depth) {
    unordered_map<uint64_t, int> dist;
    queue<uint64_t> q;
    dist[start_code] = 0;
    q.push(start_code);

    while (!q.empty()) {
        uint64_t u_code = q.front();
        q.pop();
        int d = dist[u_code];
        if (d == max_depth) continue;
```

```
        Perm u = unpack_perm(u_code);
        for (const auto& op : ops) {
            uint64_t v_code = pack_perm(apply_op(u, op));
            if (dist.find(v_code) == dist.end()) {
                dist[v_code] = d + 1;
                q.push(v_code);
            }
        }
    }
    return dist;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int K = 14;
    vector<Perm> fwd_ops(4), bwd_ops(4);
    for (int r = 0; r < 4; ++r) {
        for (int j = 0; j < 12; ++j) {
            if (!(cin >> fwd_ops[r][j])) return 0;
            bwd_ops[r][fwd_ops[r][j]] = j; // Exact inverse permutation
        }
    }

    Perm start_p, target_p;
    for (int j = 0; j < 12; ++j) start_p[j] = j;
    for (int j = 0; j < 12; ++j) cin >> target_p[j];

    auto dist_f = bfs_half(pack_perm(start_p), fwd_ops, K / 2);
    auto dist_b = bfs_half(pack_perm(target_p), bwd_ops, (K + 1) / 2);

    int best = K + 1;
    for (const auto& [code, d_f] : dist_f) {
        auto it = dist_b.find(code);
        if (it != dist_b.end()) {
            best = min(best, d_f + it->second);
        }
    }

    cout << (best <= K ? best : -1) << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Necessity & Sufficiency:** Suppose a shortest path from $P_{\text{start}}$ to $P_{\text{target}}$ has length $k \le K$: $P_0 \to P_1 \to \dots \to P_k$. Let $m = \min(k, \lfloor K/2 \rfloor)$. Then $P_m$ is at distance $m \le \lfloor K/2 \rfloor$ from $P_{\text{start}}$ and distance $k - m \le \lceil K/2 \rceil$ backward from $P_{\text{target}}$. Because BFS computes shortest distances within radius $\lfloor K/2 \rfloor$ and $\lceil K/2 \rceil$, we have $\text{dist}_F[P_m] = m$ and $\text{dist}_B[P_m] = k - m$, so $P_m$ is guaranteed to be in the intersection with sum $k$. Conversely, any state $X$ in the intersection yields a valid walk of length $\text{dist}_F[X] + \text{dist}_B[X]$ by concatenating the forward path to $X$ and the inverted backward path from $X$.
- **Small-Case Oracle:** Compare `bfs_half` with $K = 14$ against a single forward BFS up to depth $K = 8$ on smaller random permutations.

---

# Section 48 — DP Optimization: Monotonic Queue and Divide & Conquer

## The Discovery & Modeling Approach

Once you have written a correct Dynamic Programming recurrence, how do you know whether its transition loop can be accelerated from $O(N)$ per state to $O(1)$ or $O(\log N)$ amortized?

Never guess a DP optimization from the story of the problem. Instead, write down the **exact algebraic recurrence** for $dp[i]$ (or $dp[k][i]$) and inspect how the query index $i$ and the predecessor index $j$ interact inside the $\min/\max$:

$$dp[i] = \min_{j \in \mathcal{J}(i)} \left\{ dp[j] + \text{cost}(j, i) \right\}.$$

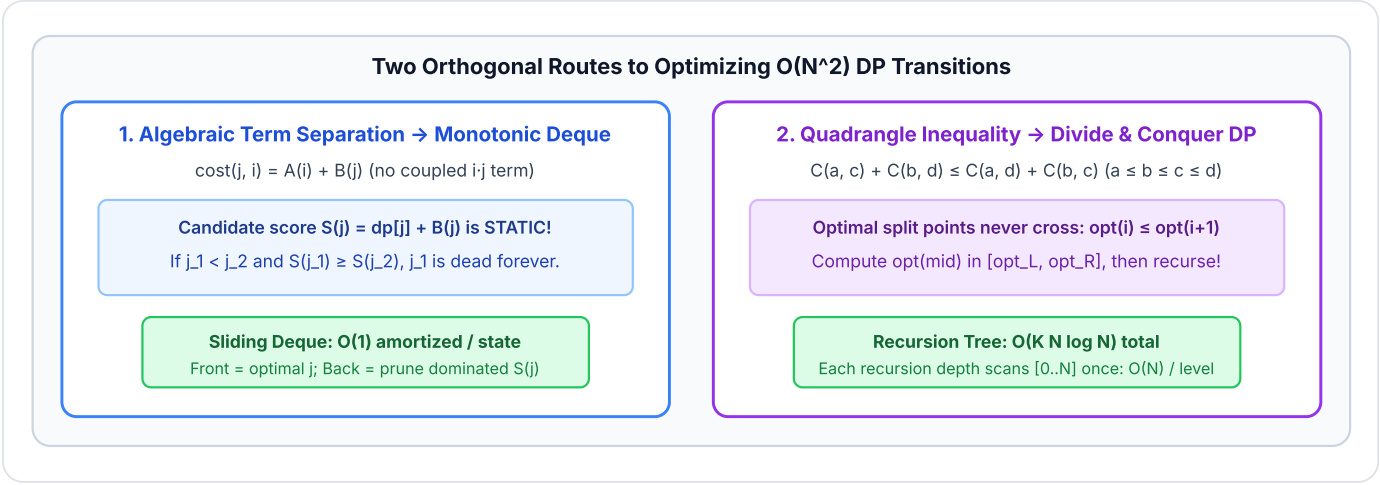
Look through two structural lenses:

1. **The Additive Separation Lens (Monotonic Queue / Sliding Window DP):** Can $\text{cost}(j, i)$ be algebraically split into $A(i) + B(j)$ where there is **no mixed product or nonlinear interaction** between $i$ and $j$, and the feasible window $\mathcal{J}(i) = [L(i), R(i)]$ has monotonically nondecreasing endpoints $L(i) \le L(i+1)$ and $R(i) \le R(i+1)$? If yes, pull $A(i)$ completely outside the $\min$:
$$dp[i] = A(i) + \min_{L(i) \le j \le R(i)} \left\{ dp[j] + B(j) \right\}.$$
Every candidate $j$ now has a **fixed scalar score** $S(j) = dp[j] + B(j)$ that never changes as $i$ advances! A monotonic deque maintains the minimum of $S(j)$ across the sliding window in $O(1)$ amortized time per state.

2. **The Quadrangle Inequality / Opt-Monotonicity Lens (Divide & Conquer DP):** Suppose we partition an array of length $N$ into $K$ layers:
$$dp[k][i] = \min_{0 \le j < i} \left\{ dp[k-1][j] + C(j+1, i) \right\},$$
where $C(a, b)$ cannot be separated into $A(b) + B(a)$. Ask instead: if we move the right endpoint from $i$ to $i+1$, can the first optimal split point $\text{opt}(k, i)$ ever jump *leftward*? Whenever the cost function satisfies the **Quadrangle Inequality** (for minimization):
$$C(a, c) + C(b, d) \le C(a, d) + C(b, c) \quad \text{for all } a \le b \le c \le d,$$
the optimal decision points are monotone: $\text{opt}(k, i) \le \text{opt}(k, i+1)$. Because every $i$ in a range $[L, R]$ has its optimal split trapped inside $[\text{opt}(k, L), \text{opt}(k, R)]$, resolving the midpoint $mid = \lfloor (L+R)/2 \rfloor$ splits the search interval in two, reducing a $O(KN^2)$ DP to $O(KN \log N)$.

### Decision Compass & Structural Lenses

| Optimization Technique | Recurrence Archetype | Required Structural Condition | Complexity Reduction | Diagnostic Red Flag (When It Fails!) |
|---|---|---|---|---|
| **Monotonic Queue (1D Sliding Window)** | $dp[i] = A(i) + \min_{L(i) \le j \le R(i)} B(j)$ | Pure additive separation $A(i) + B(j)$ and monotone window boundaries $L(i), R(i)$ | $O(N \cdot W) \to O(N)$ | Fails if $\text{cost}(j, i)$ has cross-terms like $x_i \cdot y_j$ or $\lfloor (i-j)/2 \rfloor$ |
| **Monotonic Queue (Bounded Knapsack Remainder Split)** | $dp[w] = \max_{0 \le c \le m} \{ dp_{\text{prev}}[w - c \cdot v] + c \cdot p \}$ | Group states by residue $r = w \bmod v$; write $w = q \cdot v + r$ to separate $q$ and $q - c$ | $O(N \cdot W \cdot m) \to O(N \cdot W)$ | Fails if you slide a window over consecutive weights $w, w+1$ instead of step $v$ |
| **Divide & Conquer DP** | $dp[k][i] = \min_{j < i} \{ dp[k-1][j] + C(j+1, i) \}$ | Quadrangle Inequality $C(a, c) + C(b, d) \le C(a, d) + C(b, c)$ $\implies \text{opt}(k, i) \le \text{opt}(k, i+1)$ | $O(KN^2) \to O(KN \log N)$ | Fails if $C(a, b)$ violates QI (e.g., non-convex fixed-charge or XOR costs) |




---

## Worked Problem 48.1: Cryogenic Relay Furnace Scheduling

### Problem & Constraints

A linear pipeline has $N$ relay stations ($1 \le N \le 5 \times 10^5$) at positions $1, 2, \ldots, N$, with source station $0$ at the start and sink station $N+1$ at the end. You must activate a sequence of relay stations $0 = j_0 < j_1 < \ldots < j_m = N+1$ such that the gap between consecutive active relays never exceeds $W$ ($1 \le j_r - j_{r-1} \le W$).

Activating station $i$ ($1 \le i \le N$) incurs an installation cost $a_i$ ($1 \le a_i \le 10^9$). In addition, every step from an active station $j$ to the next active station $i$ ($j < i$) incurs a thermal dissipation penalty proportional to the distance and the intervening ambient load:

$$\text{penalty}(j, i) = (i - j) \cdot \alpha + \sum_{t=j+1}^{i-1} h_t,$$

where $\alpha \ge 0$ is a constant and $h_t \ge 0$ is the ambient heat at position $t$. Find the minimum total cost to reach station $N+1$ from station $0$.

**Pause & Discover Before Reading On:**

1. Write the exact recurrence for $dp[i]$, the minimum cost to reach and activate station $i$ from $j \in [i-W, i-1]$.
2. At first glance, $\text{penalty}(j, i)$ depends on both $j$ and $i$. How can we rewrite $\text{penalty}(j, i)$ using prefix sums $H[k] = \sum_{t=1}^{k} h_t$ so that every term involves *either* $i$ alone *or* $j$ alone?

### Step 1 — Exact Model & Bottleneck

Let $H[k] = \sum_{t=1}^{k} h_t$ (with $H[0] = 0$) be the prefix sum of ambient heat, and set $a_0 = a_{N+1} = 0$. The exact DP state is $dp[i]$: the minimum total cost of a valid relay chain from $0$ ending at active relay $i$. For $1 \le i \le N+1$:

$$dp[i] = \min_{\max(0,\, i-W) \le j \le i-1} \Big\{ dp[j] + a_i + (i-j)\alpha + \big( H[i-1] - H[j] \big) \Big\}.$$

- **Bottleneck:** For $N = 5 \times 10^5$ and $W = 2.5 \times 10^5$, scanning all $W$ predecessors for each $i$ requires $O(NW) \approx 1.25 \times 10^{11}$ operations—far too slow.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake (Ordering the deque by raw $dp[j]$ alone):** A common reflex when seeing $[i-W, i-1]$ is to maintain the minimum of $dp[j]$ in the deque and add the transition cost $(i-j)\alpha + (H[i-1] - H[j])$ only when reading the front of the deque.
  - *Counterexample:* Suppose $W = 3, \alpha = 10$, and $h_t = 0$. At $j = 1$, $dp[1] = 10$; at $j = 2$, $dp[2] = 15$. If we order the deque by $dp[j]$ alone, $j = 1$ looks better than $j = 2$ ($10 < 15$). Yet for any future $i = 3$:
    - From $j = 1$: $dp[1] + (3-1) \cdot 10 = 10 + 20 = 30$.
    - From $j = 2$: $dp[2] + (3-2) \cdot 10 = 15 + 10 = 25 < 30$! Candidate $j = 2$ is strictly better than $j = 1$ for *every* future $i$ because it saves $10$ units of distance penalty while costing only $5$ more in $dp$!

### Step 3 — The Turn: Discovering the Structure

Expand the recurrence and separate every term that depends on $i$ from every term that depends on $j$:

$$dp[j] + a_i + i\alpha - j\alpha + H[i-1] - H[j] = \underbrace{\big(\, a_i + i\alpha + H[i-1] \,\big)}_{\text{depends ONLY on } i} + \underbrace{\big(\, dp[j] - j\alpha - H[j] \,\big)}_{\text{depends ONLY on } j}.$$

**Algebraic Term Separation & Static Candidate Potential:**

Define the **static potential** of predecessor $j$ as:

$$S(j) = dp[j] - j \cdot \alpha - H[j].$$

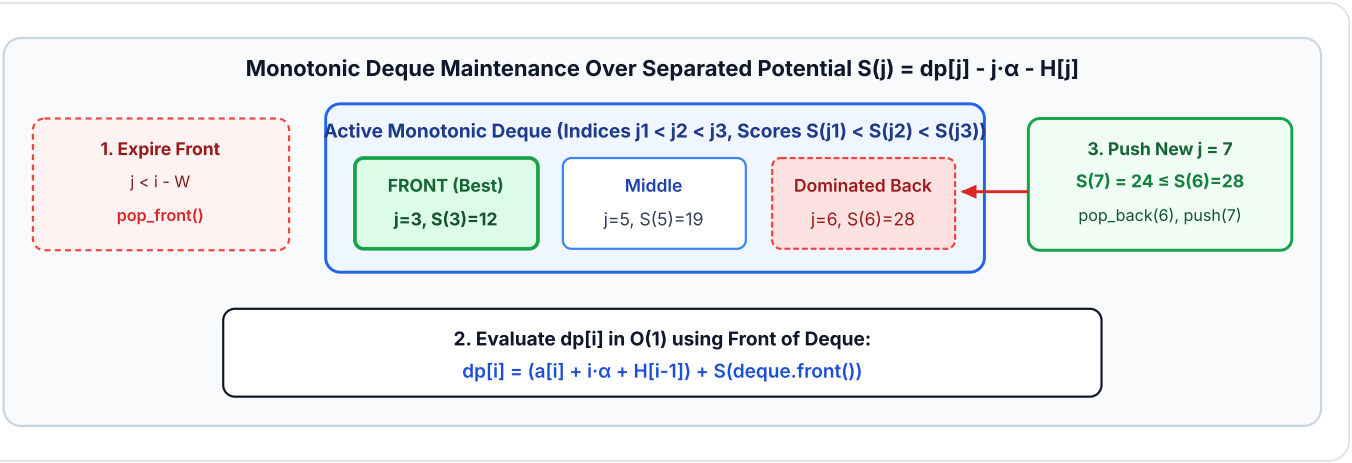
Then the exact DP transition simplifies to:

$$dp[i] = \big(\, a_i + i \cdot \alpha + H[i-1] \,\big) + \min_{\max(0,\, i-W) \le j \le i-1} S(j).$$

Because $S(j)$ is completely independent of $i$, if $j_1 < j_2$ and $S(j_1) \ge S(j_2)$, then $j_2$ is both **at least as cheap** and **expires later** than $j_1$. Thus $j_1$ can be permanently popped from the back of the monotonic deque.

### Step 4 — Progressive Construction & Visual Trace

Let us visualize the separation of $(j, i)$ into the external $i$-offset and the monotonic deque over $S(j)$.




```cpp
#include <deque>
#include <iostream>
#include <vector>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, W;
    long long alpha;
    if (!(cin >> n >> W >> alpha)) return 0;

    vector<long long> a(n + 2, 0), h(n + 2, 0), H(n + 2, 0);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    for (int i = 1; i <= n; ++i) {
        cin >> h[i];
        H[i] = H[i - 1] + h[i];
    }
    H[n + 1] = H[n];

    vector<long long> dp(n + 2, 0);
    auto S = [&](int j) -> long long {
        return dp[j] - j * alpha - H[j];
    };
```

```
    deque<int> dq;
    dq.push_back(0); // Base state j = 0 with dp[0] = 0, S(0) = 0

    for (int i = 1; i <= n + 1; ++i) {
        // 1. Remove expired indices outside [i - W, i - 1]
        while (!dq.empty() && dq.front() < i - W) {
            dq.pop_front();
        }

        // 2. Compute dp[i] from the optimal predecessor at the front
        int best_j = dq.front();
        dp[i] = (a[i] + i * alpha + H[i - 1]) + S(best_j);

        // 3. Maintain monotonicity of S(j) before inserting i
        long long s_i = S(i);
        while (!dq.empty() && S(dq.back()) >= s_i) {
            dq.pop_back();
        }
        dq.push_back(i);
    }

    cout << dp[n + 1] << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Deque Invariant Proof:** At the moment we query $dp[i]$, the deque holds a subset of indices $j_1 < j_2 < \ldots < j_r$ inside $[i-W, i-1]$ with strictly increasing scores $S(j_1) < S(j_2) < \ldots < S(j_r)$. Every index $j' \in [i-W, i-1]$ that is *not* in the deque was popped when some later index $j'' > j'$ with $S(j'') \le S(j')$ entered. By transitivity, some surviving index in the deque has $S(j_t) \le S(j')$, so $S(j_1) = \min_{i-W \le j \le i-1} S(j)$. Each index enters `dq` once and leaves at most once, giving $O(N)$ total time.
- **Small-Case Oracle:** Verify against the $O(NW)$ double loop for $N \le 100$ with random $\alpha, a_i, h_i$.

---

## Worked Problem 48.2:Segmented Sensor Consolidation (Quadrangle Inequality & D&C DP)

### Problem & Constraints

You are given $N$ sensors along a corridor ($1 \le N \le 4000$), where sensor $i$ generates $w_i \ge 0$ packets per second ($1 \le w_i \le 10^5$). You must partition the $N$ sensors into at most $K$ contiguous segments ($1 \le K \le \min(N, 800)$).

Whenever a contiguous segment $[a, b]$ ($1 \le a \le b \le N$) is consolidated into a single hub, every unordered pair of distinct sensors $u < v$ inside $[a, b]$ interferes with intensity $w_u \cdot w_v$, so the segment's interference cost is:

$$C(a,b) = \sum_{a \le u < v \le b} w_u w_v.$$

Find the minimum total interference cost across all $K$ segments.

> **Pause & Discover Before Reading On:**
> 1. First, how can we evaluate $C(a,b)$ in $O(1)$ time using prefix sums of $w_i$ and $w_i^2$?
> 2. Write down $C(a,d) + C(b,c) - C(a,c) - C(b,d)$ for $a \le b \le c \le d$. Which pairs $(u,v)$ survive after cancellation, and why is their sum guaranteed to be $\ge 0$?

### Step 1 — Exact Model & Bottleneck

Let $dp[k][i]$ be the minimum interference cost to partition the prefix $1 \ldots i$ into $k$ contiguous segments. For $k = 1 \ldots K$ and $i = 1 \ldots N$:

$$dp[k][i] = \min_{0 \le j < i} \left\{ dp[k-1][j] + C(j+1, i) \right\},$$

with base case $dp[0][0] = 0$ and $dp[0][i] = \infty$ for $i \ge 1$.

To evaluate $C(a,b)$ in $O(1)$ time, let $P_1[x] = \sum_{t=1}^{x} w_t$ and $P_2[x] = \sum_{t=1}^{x} w_t^2$. By the algebraic identity $\left(\sum w_t\right)^2 = \sum w_t^2 + 2\sum_{u<v} w_u w_v$:

$$C(a,b) = \frac{\left(P_1[b] - P_1[a-1]\right)^2 - \left(P_2[b] - P_2[a-1]\right)}{2}.$$

- **Bottleneck:** Computing $K$ layers of $N$ states by scanning $O(N)$ predecessors per state takes $O(KN^2) \approx 800 \times 1.6 \times 10^7 \approx 1.28 \times 10^{10}$ operations—too slow for a 1–2 second limit.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Trying to apply Monotonic Queue):** Here $C(j+1,i)$ contains the cross-product $-P_1[j]P_1[i]$, so $j$ and $i$ are multiplicatively coupled. A monotonic queue cannot be used.
- **Tempting Mistake 2 (Assuming $dp[k][i]$ is unimodal in $j$ and using ternary search):** Because $dp[k-1][j]$ depends on discrete prefix structures, the objective function $f(j) = dp[k-1][j] + C(j+1,i)$ can have flat plateaus or local minima where ternary search gets trapped.

### Step 3 — The Turn: Discovering the Structure

Let us check the **Quadrangle Inequality** for $a \le b \le c \le d$. In $C(x,y)$, we sum $w_u w_v$ over all pairs $x \le u < v \le y$. When we compute the cross difference:

$$\Delta = \Big[C(a,d) - C(a,c)\Big] - \Big[C(b,d) - C(b,c)\Big],$$

notice that: - $C(a,d) - C(a,c)$ is the sum of $w_u w_v$ where $v \in [c+1,d]$ and $u \in [a,v-1]$. - $C(b,d) - C(b,c)$ is the sum of $w_u w_v$ where $v \in [c+1,d]$ and $u \in [b,v-1]$. Subtracting the two leaves precisely the pairs with $u \in [a,b-1]$ and $v \in [c+1,d]$:

$$C(a,d) + C(b,c) - C(a,c) - C(b,d) = \sum_{u=a}^{b-1} \sum_{v=c+1}^{d} w_u w_v = \left(P_1[b-1] - P_1[a-1]\right)\left(P_1[d] - P_1[c]\right) \ge 0$$

because $w_t \ge 0$!

**Quadrangle Inequality & Divide-and-Conquer DP Theorem:**
Because $C(a,c) + C(b,d) \le C(a,d) + C(b,c)$ for all $a \le b \le c \le d$, the first optimal split point $\mathrm{opt}(k,i) = \mathrm{argmin}_{j<i}\{dp[k-1][j] + C(j+1,i)\}$ satisfies:

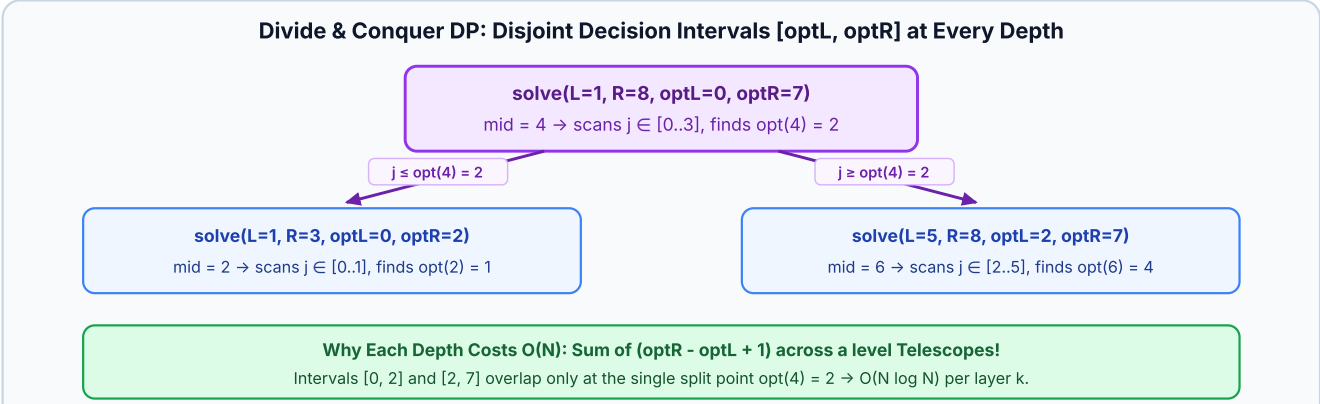
$$\mathrm{opt}(k,1) \le \mathrm{opt}(k,2) \le \ldots \le \mathrm{opt}(k,N).$$

We therefore compute layer $k$ using a recursive procedure $\mathrm{solve}(L,R,\mathrm{optL},\mathrm{optR})$:

1. Set $mid = \lfloor (L+R)/2 \rfloor$ and scan $j \in [\mathrm{optL}, \min(mid-1, \mathrm{optR})]$ to find both $dp[k][mid]$ and its optimal split $best = \mathrm{opt}(k,mid)$.
2. Recurse left: $\mathrm{solve}(L, mid-1, \mathrm{optL}, best)$.
3. Recurse right: $\mathrm{solve}(mid+1, R, best, \mathrm{optR})$.

### Step 4 — Progressive Construction & Visual Trace

Let us visualize the recursion tree of $\mathrm{solve}(L,R,\mathrm{optL},\mathrm{optR})$ and see why the total work across an entire depth level of the tree is $O(N)$.



```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

const long long INF = 4e18;

int n, K;
vector<long long> P1, P2;
vector<long long> dp_prev, dp_cur;

inline long long cost(int a, int b) {
    long long sum1 = P1[b] - P1[a - 1];
    long long sum2 = P2[b] - P2[a - 1];
    return (sum1 * sum1 - sum2) / 2;
}

void compute_dc(int L, int R, int optL, int optR) {
    if (L > R) return;
    int mid = (L + R) / 2;
    long long best_val = INF;
    int best_j = optL;

    int upper_j = min(mid - 1, optR);
    for (int j = optL; j <= upper_j; ++j) {
        long long cand = dp_prev[j] + cost(j + 1, mid);
        if (cand < best_val) {
            best_val = cand;
            best_j = j;
        }
    }

    dp_cur[mid] = best_val;
    compute_dc(L, mid - 1, optL, best_j);
    compute_dc(mid + 1, R, best_j, optR);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    if (!(cin >> n >> K)) return 0;
    P1.assign(n + 1, 0);
    P2.assign(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        long long w;
        cin >> w;
        P1[i] = P1[i - 1] + w;
        P2[i] = P2[i - 1] + w * w;
    }

    dp_prev.assign(n + 1, INF);
    dp_cur.assign(n + 1, INF);
    dp_prev[0] = 0;

    for (int k = 1; k <= K; ++k) {
        fill(dp_cur.begin(), dp_cur.end(), INF);
        compute_dc(1, n, 0, n - 1);
        dp_prev.swap(dp_cur);
    }

    cout << dp_prev[n] << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Proof of Opt-Monotonicity from Quadrangle Inequality:** Let $j_1 = \mathrm{opt}(k, i)$ be the *smallest* optimal split for $i$, and suppose for contradiction that for $i' > i$, some $j_0 < j_1$ is strictly better than $j_1$ for $i'$:
$$dp[k-1][j_0] + C(j_0 + 1, i') < dp[k-1][j_1] + C(j_1 + 1, i').$$
By optimality of $j_1$ for $i$, we also have:
$$dp[k-1][j_1] + C(j_1 + 1, i) \le dp[k-1][j_0] + C(j_0 + 1, i).$$
Adding the two inequalities cancels $dp[k-1][j_0]$ and $dp[k-1][j_1]$ and yields:
$$C(j_0 + 1, i') + C(j_1 + 1, i) < C(j_1 + 1, i') + C(j_0 + 1, i),$$
which directly contradicts the Quadrangle Inequality for $j_0 + 1 \le j_1 + 1 \le i \le i'$!
- **Complexity Accounting:** At recursion depth $d$, there are at most $2^d$ calls with intervals $[\mathrm{optL}_r, \mathrm{optR}_r]$ satisfying $\mathrm{optR}_r \le \mathrm{optL}_{r+1}$. Thus $\sum_r (\mathrm{optR}_r - \mathrm{optL}_r + 1) \le N + 2^d$, which sums over $d = 0 \ldots \lceil \log_2 N \rceil$ to $O(N \log N)$ per layer.
- **Small-Case Oracle:** Cross-check against the $O(KN^2)$ triple loop on $N = 40, K = 6$, including zero-weight sensors ($w_i = 0$) where ties in $\mathrm{argmin}$ occur.

---

# Section 49 — Convex Hull Trick and Li Chao Trees

## The Discovery & Modeling Approach

Suppose you have expanded your DP transition algebraically, isolated the pure $i$-terms outside the $\min/\max$, and grouped the pure $j$-terms together—yet one stubborn cross-term remains, and it has the **bilinear form** $f(i) \cdot g(j)$:

$$dp[i] = A(i) + \min_{j<i} \Big\{ \underbrace{m(j)}_{\text{slope } m_j} \cdot \underbrace{x(i)}_{\text{query } x_i} + \underbrace{b(j)}_{\text{intercept } b_j} \Big\}.$$

Once you view $x_i = x(i)$ as the horizontal coordinate of a query point and each completed predecessor $j$ as a straight line:

$$L_j(x) = m_j \cdot x + b_j,$$

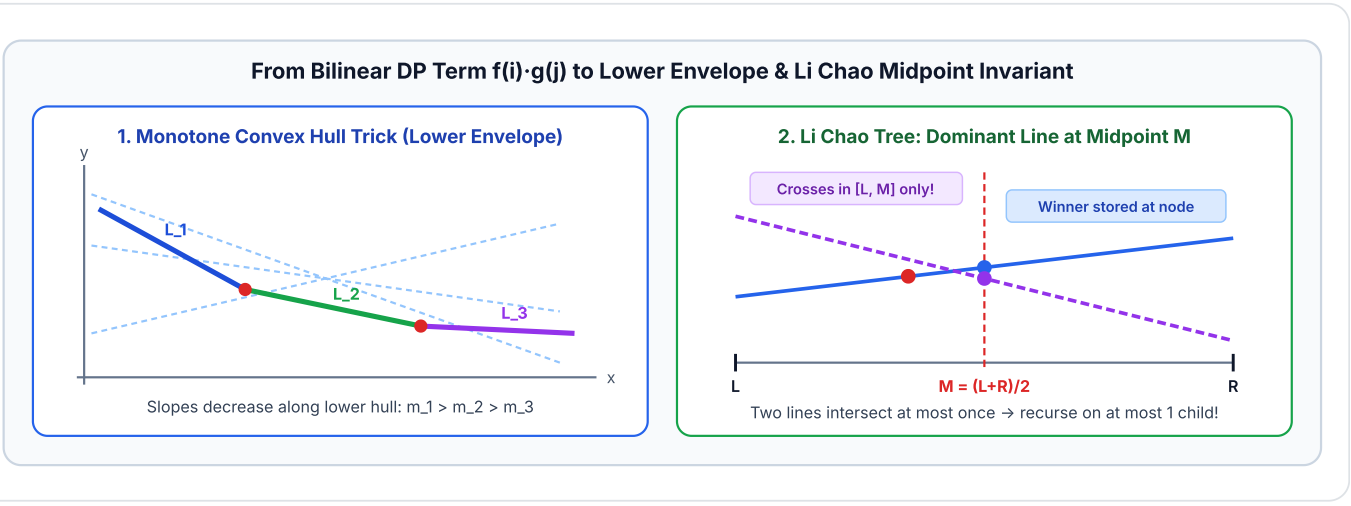
the entire inner transition loop asks a single geometric question:

> **The Envelope Query:** Given a collection of lines $\{L_j(x) = m_j x + b_j\}$, what is the minimum (lower envelope) or maximum (upper envelope) value at $x = x_i$?

How you maintain this set of lines depends entirely on whether the **slopes** $m_j$ and the **query coordinates** $x_i$ are monotonic:

### Decision Compass & Structural Lenses

| Variant | Slope Order $m_j$ | Query Order $x_i$ | Core Data Structure | Time per Insertion / Query | Key Implementation Mechanism |
|---|---|---|---|---|---|
| **Two-Pointer Deque CHT** | Monotone (sorted as $j$ grows) | Monotone (sorted as $i$ grows) | `std::deque<Line>` | $O(1)$ amortized insert, $O(1)$ amortized query | Pop redundant back lines via intersection $X(L_{k-1}, L_k) \geq X(L_k, L_{\text{new}})$; pop expired front lines |
| **Binary-Search Hull CHT** | Monotone | Arbitrary order | `std::vector<Line>` | $O(1)$ amortized insert, $O(\log N)$ query | Same back-pruning on insert; binary search the hull vertices for query $x_i$ |
| **Dynamic Li Chao Tree** | Arbitrary order | Arbitrary order ( $x_i \in [-X, X]$) | Segment tree over coordinate domain $[L, R]$ | $O(\log X)$ insert, $O(\log X)$ query | Each node stores the single line dominant at `mid`; push the loser into the half-interval where it can still win |
| **Segment-Insertion Li Chao Tree** | Arbitrary lines restricted to $[l_j, r_j]$ | Arbitrary order | Li Chao tree with range decomposition | $O(\log^2 X)$ segment insert, $O(\log X)$ query | Split segment $[l_j, r_j]$ into $O(\log X)$ canonical tree nodes, then run $O(\log X)$ Li Chao descent on each |



## Worked Problem 49.1: Optical Packet Batching along a Fiber Trunk

### Problem & Constraints

An optical fiber trunk has $N$ repeater buffers at strictly increasing distances $0 < d_1 < d_2 < \ldots < d_N \leq 10^6$, where buffer $i$ holds $c_i \geq 1$ data packets ($1 \leq N \leq 3 \times 10^5$, $1 \leq c_i \leq 10^6$). You must partition the $N$ buffers into contiguous batches $[j+1, i]$ and place an amplifier at the rightmost buffer $i$ of each batch (buffer $N$ must always receive an amplifier).

Placing an amplifier at buffer $i$ incurs hardware cost $f_i$ ($1 \leq f_i \leq 10^{12}$), plus a dispersion cost equal to $d_i - d_t$ for each packet at buffer $t \in [j+1, i]$:

$$\text{batch_cost}(j+1, i) = f_i + \sum_{t=j+1}^{i} c_t \cdot (d_i - d_t).$$

Find the minimum total cost to amplify all $N$ buffers.

**Pause & Discover Before Reading On:**

1. Introduce prefix sums $C[k] = \sum_{t=1}^{k} c_t$ and $CD[k] = \sum_{t=1}^{k} c_t d_t$ to write $\text{batch_cost}(j+1, i)$ in $O(1)$ form.
2. When you expand $(C[i] - C[j])d_i - (CD[i] - CD[j])$, which term couples $j$ and $i$? Identify the slope $m_j$, the query variable $x_i$, and the intercept $b_j$.
3. Are the slopes $m_j$ and queries $x_i$ monotonic as $j$ and $i$ increase?

### Step 1 — Exact Model & Bottleneck

Define prefix sums $C[k] = \sum_{t=1}^{k} c_t$ and $CD[k] = \sum_{t=1}^{k} c_t d_t$ (with $C[0] = CD[0] = 0$). Then:

$$\sum_{t=j+1}^{i} c_t(d_i - d_t) = d_i \sum_{t=j+1}^{i} c_t - \sum_{t=j+1}^{i} c_t d_t = d_i \Big( C[i] - C[j] \Big) - \Big( CD[i] - CD[j] \Big).$$

Let $dp[i]$ be the minimum cost to cover the prefix $1 \ldots i$ with an amplifier at $i$ ($dp[0] = 0$). For $1 \leq i \leq N$:

$$dp[i] = \min_{0 \leq j < i} \Big\{ dp[j] + f_i + d_i(C[i] - C[j]) - (CD[i] - CD[j]) \Big\}.$$

- **Bottleneck:** Evaluating all $j \in [0, i-1]$ directly takes $O(N^2) = 9 \times 10^{10}$ steps.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Floating-point division when checking line intersections):** Computing the intersection abscissa $x = (b_1 - b_2)/(m_2 - m_1)$ in `double` loses precision when $b_j \sim 10^{18}$ (`double` has only 53 bits of significand $\approx 9 \times 10^{15}$), causing adjacent lines on the hull to be wrongly pruned. Always cross-multiply using `__int128`!
- **Tempting Mistake 2 (Popping from the wrong end of the deque when slopes decrease):** Because $c_t \geq 1$, $C[j]$ is strictly increasing, so the slope $m_j = -C[j]$ is **strictly decreasing** as $j$ increases. For a lower-envelope minimization with decreasing slopes and increasing queries $x_i = d_i$, the steepest negative slopes become optimal for large $x_i$, so newly added lines belong at the **back** of the deque and queries advance from the **front**.

### Step 3 — The Turn: Discovering the Structure

Separate the $i$-only terms from the $j$-dependent terms in $dp[i]$:

$$dp[i] = \underbrace{\Big( f_i + d_i C[i] - CD[i] \Big)}_{A(i)} + \min_{0 \leq j < i} \Big\{ \underbrace{(-C[j])}_{m_j} \cdot \underbrace{d_i}_{x_i} + \underbrace{\Big( dp[j] + CD[j] \Big)}_{b_j} \Big\}.$$

**Bilinear Factorization into Monotone CHT:**

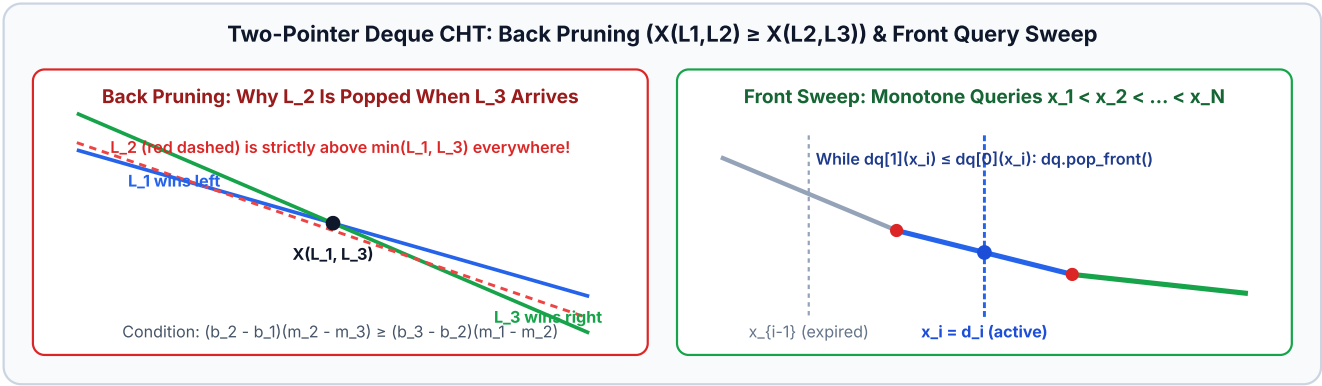
Each completed state $j \in [0, i-1]$ inserts the line $L_j(x) = m_j x + b_j$ with:

$$m_j = -C[j], \qquad b_j = dp[j] + CD[j], \qquad x_i = d_i.$$

- **Slope Monotonicity:** Since $c_j \geq 1$, $C[j]$ is strictly increasing, so $m_j = -C[j]$ is strictly decreasing.
- **Query Monotonicity:** Since $d_1 < d_2 < \ldots < d_N$, $x_i = d_i$ is strictly increasing.
- **Exact Redundancy Test (Cross-Multiplication):** Line $L_2$ between $L_1$ and $L_3$ ($m_1 > m_2 > m_3$) is redundant on the lower hull iff the intersection $X(L_1, L_2) \geq X(L_2, L_3)$, i.e.:
  $(b_2 - b_1)(m_2 - m_3) \geq (b_3 - b_2)(m_1 - m_2)$ (evaluated in `__int128`).

### Step 4 — Progressive Construction & Visual Trace

Let us trace why a middle line $L_2$ becomes completely hidden above $L_1$ and $L_3$ when $X(L_1, L_2) \geq X(L_2, L_3)$.




```cpp
#include <deque>
#include <iostream>
#include <vector>

using namespace std;

struct Line {
    long long m, b;
    long long eval(long long x) const {
        return m * x + b;
    }
};

// Returns true if l2 is made redundant on the lower hull by l1 and l3 (m1 > m2 > m3)
bool is_redundant(const Line& l1, const Line& l2, const Line& l3) {
    __int128 lhs = static_cast<__int128>(l2.b - l1.b) * (l2.m - l3.m);
    __int128 rhs = static_cast<__int128>(l3.b - l2.b) * (l1.m - l2.m);
    return lhs >= rhs;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> d(n + 1, 0), c(n + 1, 0), f(n + 1, 0);
    vector<long long> C(n + 1, 0), CD(n + 1, 0);

    for (int i = 1; i <= n; ++i) {
        cin >> d[i] >> c[i] >> f[i];
        C[i] = C[i - 1] + c[i];
        CD[i] = CD[i - 1] + c[i] * d[i];
    }

    vector<long long> dp(n + 1, 0);
    deque<Line> hull;
    hull.push_back({0, 0}); // Base state j = 0: m_0 = -C[0] = 0, b_0 = 0

    for (int i = 1; i <= n; ++i) {
        long long x = d[i];
        // Pop front lines overtaken at query coordinate x = d[i]
        while (hull.size() >= 2 && hull[1].eval(x) <= hull[0].eval(x)) {
            hull.pop_front();
```

```
        }

        dp[i] = (f[i] + d[i] * C[i] - CD[i]) + hull.front().eval(x);

        Line next_line = {-C[i], dp[i] + CD[i]};
        while (hull.size() >= 2 && is_redundant(hull[hull.size() - 2], hull.back(), next_line)) {
            hull.pop_back();
        }
        hull.push_back(next_line);
    }

    cout << dp[n] << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Amortized $O(N)$ Complexity & Exactness:** Because slopes $m_j = -C[j]$ are strictly decreasing, any two lines $L_{j_1}, L_{j_2}$ ($j_1 < j_2$) satisfy $m_{j_1} > m_{j_2}$, so $L_{j_2}(x) - L_{j_1}(x) = (m_{j_2} - m_{j_1})x + (b_{j_2} - b_{j_1})$ is strictly decreasing in $x$. Once $L_{j_2}(x_i) \le L_{j_1}(x_i)$ for some $x_i$, it remains $\le L_{j_1}(x)$ for all $x > x_i$. Each line is pushed once and popped at most once, giving $O(N)$ total time.
- **Small-Case Oracle:** Verify against the $O(N^2)$ DP for $N = 200$, including steep cost spikes where $N - 2$ consecutive lines get popped at once from the back of the hull.

---

## Worked Problem 49.2: Bidirectional Drone Courier Relay (Dynamic Li Chao Tree)

### Problem & Constraints

$N$ drone stations are ordered chronologically $1, 2, ..., N$ ($1 \le N \le 2 \times 10^5$). Station $i$ is located at signed integer coordinate $x_i \in [-10^9, 10^9]$ (note that $x_i$ is **not** sorted—stations jump back and forth across the city!), has dispatch rate $p_i \in [-10^6, 10^6]$ (which is also **not** monotonic), and incurs fixed landing fee $s_i \in [-10^{12}, 10^{12}]$.

Starting at station $1$ with initial cost $dp[1] = s_1$, a courier drone visits a subsequence of stations $1 = j_1 < j_2 < ... < j_m = N$. Flying directly from station $j$ to station $i$ ($j < i$) costs:

$$\text{flight_cost}(j, i) = p_j \cdot (x_i - x_j) + s_i.$$

Find the minimum total cost $dp[N]$ to reach station $N$.

> **Pause & Discover Before Reading On:**
> 1. Rewrite $dp[j] + p_j(x_i - x_j) + s_i$ as $s_i + (m_j x_i + b_j)$. What are $m_j$ and $b_j$?
> 2. Why does the deque-based Convex Hull Trick fail here?
> 3. In a Li Chao tree over $[-10^9, 10^9]$, if node $[L, R]$ has midpoint $M$ and two lines $L_{\text{cur}}, L_{\text{new}}$, why do we only ever need to recurse into **at most one** child interval?

### Step 1 — Exact Model & Bottleneck

For $2 \le i \le N$, the exact recurrence is:

$$dp[i] = s_i + \min_{1 \le j < i} \left\{ p_j \cdot x_i + (dp[j] - p_j x_j) \right\}.$$

Each station $j$ defines a line $L_j(x) = m_j x + b_j$ with slope $m_j = p_j$ and intercept $b_j = dp[j] - p_j x_j$, and station $i$ queries the lower envelope at $x = x_i$.

- **Bottleneck:** Both $m_j = p_j$ and $x_i$ arrive in completely arbitrary order, so deque CHT is invalid, and maintaining a dynamic balanced BST of hull segments requires delicate floating-point or 128-bit neighbor intersection maintenance.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Sorting the stations by $x_i$ or $p_j$):** You cannot sort the stations because a transition $j \to i$ is legal **only if $j < i$ in the original chronological order**! Sorting by $x_i$ would allow transitions from the future into the past.

- **Tempting Mistake 2 (Allocating a static array for $[-10^9, 10^9]$):** The coordinate range has width $2 \times 10^9$, so a static segment tree array would exceed memory limits. We must either **allocate tree nodes dynamically** on demand (at most $O(N \log(\text{range}))$ nodes) or coordinate-compress the $N$ distinct query points $x_1, \ldots, x_N$.

### Step 3 — The Turn: Discovering the Structure

A **Dynamic Li Chao Tree** maintains lines over the coordinate domain $[L, R] = [-10^9, 10^9]$ (or over the sorted compressed array of query coordinates) using a remarkably simple invariant.

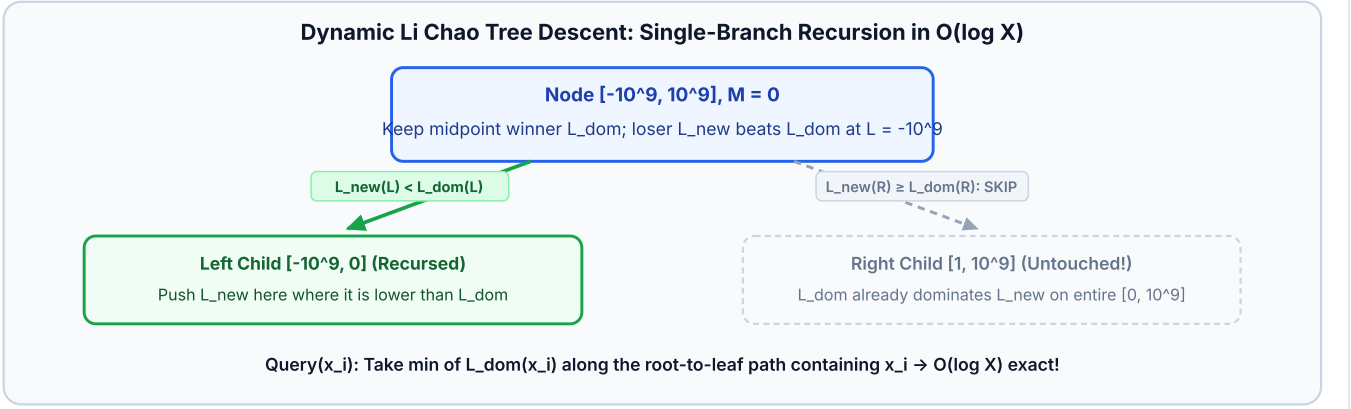
**The Li Chao Tree Midpoint Dominance Invariant:**
Each node covering coordinate interval $[L, R]$ with midpoint $M = \lfloor (L + R)/2 \rfloor$ stores a single line $L_{\text{dom}}$. When inserting a new line $L_{\text{new}}$ at node $[L, R]$:

1. Compare $L_{\text{new}}(M)$ and $L_{\text{dom}}(M)$. If $L_{\text{new}}(M) < L_{\text{dom}}(M)$, **swap** $L_{\text{new}}$ and $L_{\text{dom}}$ so that $L_{\text{dom}}$ is guaranteed to be the winner (smaller value) at $x = M$.
2. Now $L_{\text{new}}$ is the *loser* at $x = M$. Because two straight lines intersect at most once, $L_{\text{new}}$ cannot beat $L_{\text{dom}}$ on **both** $[L, M]$ and $[M+1, R]$!
   - If $L_{\text{new}}(L) < L_{\text{dom}}(L)$, their intersection lies in $[L, M] \implies$ recurse left into $[L, M]$ with $L_{\text{new}}$.
   - Else if $L_{\text{new}}(R) < L_{\text{dom}}(R)$, their intersection lies in $[M+1, R] \implies$ recurse right into $[M+1, R]$ with $L_{\text{new}}$.

To query $x_i$, walk the unique root-to-leaf path containing $x_i$ and take the minimum of $L_{\text{dom}}(x_i)$ across all $O(\log X)$ visited nodes.

### Step 4 — Progressive Construction & Visual Trace

Let us trace how `insert_line` swaps the midpoint winner and pushes the loser into the single half where the lines cross.




Below, we compress the distinct query coordinates $X_{\text{coords}} = \text{sorted_unique}(x_1, \ldots, x_N)$ so the Li Chao tree has exact size $M_{\text{uniq}} \leq N$, requiring only $4N$ nodes, zero pointer overhead, and $O(N \log N)$ time.

```
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

const long long INF = 4e18;

struct Line {
    long long m = 0, b = INF;
    long long eval(long long x) const {
        return m * x + b;
    }
};
```

```
class LiChaoTree {
    int sz;
    vector<long long> coords;
    vector<Line> tree;

    void insert(int node, int l, int r, Line nw) {
        int mid = (l + r) / 2;
        long long xl = coords[l], xm = coords[mid], xr = coords[r];

        if (nw.eval(xm) < tree[node].eval(xm)) {
            swap(tree[node], nw);
        }
        if (l == r) return;

        if (nw.eval(xl) < tree[node].eval(xl)) {
            insert(node * 2, l, mid, nw);
        } else if (nw.eval(xr) < tree[node].eval(xr)) {
            insert(node * 2 + 1, mid + 1, r, nw);
        }
    }

    long long query(int node, int l, int r, int idx) const {
        long long x = coords[idx];
        long long res = tree[node].eval(x);
        if (l == r) return res;
        int mid = (l + r) / 2;
        if (idx <= mid) {
            return min(res, query(node * 2, l, mid, idx));
        } else {
            return min(res, query(node * 2 + 1, mid + 1, r, idx));
        }
    }

public:
    explicit LiChaoTree(vector<long long> sorted_coords)
        : sz(static_cast<int>(sorted_coords.size())),
          coords(std::move(sorted_coords)),
          tree(4 * sz, Line{0, INF}) {}

    void add_line(long long m, long long b) {
        insert(1, 0, sz - 1, Line{m, b});
    }

    long long query_at_index(int compressed_idx) const {
        return query(1, 0, sz - 1, compressed_idx);
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;

    vector<long long> x(n + 1), p(n + 1), s(n + 1);
    vector<long long> coords;
    coords.reserve(n);
    for (int i = 1; i <= n; ++i) {
        cin >> x[i] >> p[i] >> s[i];
        coords.push_back(x[i]);
    }

    sort(coords.begin(), coords.end());
    coords.erase(unique(coords.begin(), coords.end()), coords.end());
```

```
    auto get_idx = [&](long long val) {
        return static_cast<int>(lower_bound(coords.begin(), coords.end(), val) - coords.begin());
    };

    LiChaoTree lichao(coords);
    vector<long long> dp(n + 1, 0);
    dp[1] = s[1];
    lichao.add_line(p[1], dp[1] - p[1] * x[1]);

    for (int i = 2; i <= n; ++i) {
        int q_idx = get_idx(x[i]);
        dp[i] = s[i] + lichao.query_at_index(q_idx);
        lichao.add_line(p[i], dp[i] - p[i] * x[i]);
    }

    cout << dp[n] << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Induction on Interval Dominance:** Claim: After inserting a set of lines $\mathcal{S}$, for every coordinate $x \in \text{coords}$ and every line $L \in \mathcal{S}$, there is at least one ancestor node on the root-to-leaf path of $x$ whose stored line $L_{\text{dom}}$ satisfies $L_{\text{dom}}(x) \leq L(x)$.
  - *Proof:* When $L_{\text{new}}$ is compared with $L_{\text{dom}}$ at node $[l, r]$ (after swapping so $L_{\text{dom}}(x_m) \leq L_{\text{new}}(x_m)$), affine linearity guarantees that on at least one of the two sub-intervals $[l, m]$ or $[m + 1, r]$, $L_{\text{dom}}(x) \leq L_{\text{new}}(x)$ holds for *every* $x$ in that sub-interval. On that sub-interval, the node's own $L_{\text{dom}}$ witnesses dominance over $L_{\text{new}}$; on the other sub-interval, $L_{\text{new}}$ is recursively inserted so induction applies.
- **Small-Case Oracle:** Compare against the $O(N^2)$ brute-force loop for $N = 300$ with negative and positive $x_i, p_i, s_i$.

---

# Section 51 — Invariants in Hard Problems

## The Discovery & Modeling Approach

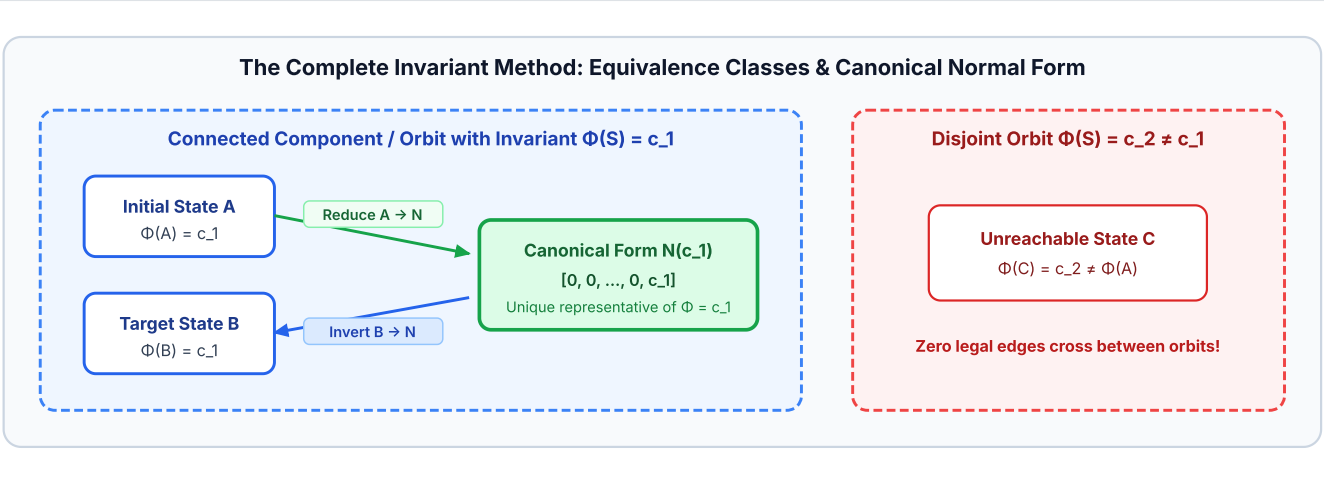
Suppose a problem gives you a combinatorial system—an array, a string, a token configuration on a graph, or a grid—and a set of **allowed local operations** (swaps, modular increments, block flips, token splits). It then asks one of three questions: 1. **Reachability:** Can configuration $A$ be transformed into configuration $B$? 2. **Canonical Extremum:** What is the lexicographically smallest or minimum-weight configuration reachable from $A$? 3. **Termination & Bound:** Does the process always terminate regardless of which valid moves are chosen, and what is the maximum number of moves?

Trying to run BFS or DP over the configuration space fails immediately when $N \sim 10^5$ and the number of reachable states is exponential. Instead, you must look for a **compressed quantity** $\Phi(\text{state})$ whose behavior under a single operation is completely rigid:

- **Invariant ($\Delta\Phi = 0$):** A function $\Phi(S)$ that remains **unchanged** under every legal move: $\Phi(S') = \Phi(S)$.
  - If $\Phi(A) \neq \Phi(B)$, then $B$ is **provably unreachable** from $A$ (Necessity).
  - If you can also show that every state $S$ can be reduced via legal moves to a unique **canonical normal form** $C(\Phi(S))$ determined solely by $\Phi(S)$ (and moves are invertible or connect to $C$), then $\Phi(A) = \Phi(B)$ is **both necessary and sufficient**!
- **Monovariant ($\Delta\Phi > 0$ or $\Delta\Phi < 0$):** A real or integer potential function $\Phi(S)$ that **strictly decreases** (or strictly increases) on every move. If $\Phi(S) \in \mathbb{Z}_{\geq 0}$ and $\Phi(S') \leq \Phi(S) - 1$, the game must terminate in at most $\Phi(S_{\text{init}})$ moves!

### Decision Compass & Structural Lenses

| Problem Question | Algebraic / Structural Clue in the Move Rule | Candidate Invariant / Monovariant $\Phi(S)$ | How to Prove Sufficiency / Tightness |
|---|---|---|---|
| **Reachability ($A \to B$?)** | Simultaneous change at adjacent indices $(i, i+1)$ by $(+\delta, +\delta)$ | Alternating sum $\sum_{i=1}^{N} (-1)^i a_i$ (mod $K$ or in $\mathbb{Z}$) | Reduce both $A$ and $B$ left-to-right to $[0, 0, \ldots, 0, \Phi]$ |
| **Reachability on Strings/Arrays** | Delete/insert $xx$ or replace $ab \leftrightarrow c$ | Homomorphism into a small non-abelian group (e.g., Quaternion $Q_8$, Klein-4 $V_4$) or $\mathbb{Z}_3$ weighted sum | Show every string reduces to a shortest canonical word of length $\leq 2$ |
| **Prefix/Difference Array Decoupling** | Operation $a_i \leftarrow a_{i-1} + a_{i+1} - a_i$ or range flip | Pass to prefix sums $P_i$ or adjacent differences $D_i = a_i - a_{i-1}$; operation becomes a **simple swap** $P_{i-1} \leftrightarrow P_i$! | Check multiset equality of $P$ (or $D$) and boundary elements |
| **Termination & Max Move Bound** | Chaotic splitting, carry propagation, or local sorting swaps | Exponential weight $\sum a_i B^i$, inversion count, or sum of squares $\sum a_i^2$ | Construct an explicit worst-case move sequence achieving the bound $\Delta\Phi = 1$ |



## Worked Problem 51.1: Tri-Phase Capacitor Ring Equalization

### Problem & Constraints

A linear bank of $N$ capacitors ($3 \le N \le 2 \times 10^5$) has initial integer charge levels $A = [a_1, a_2, ..., a_N]$ and target charge levels $B = [b_1, b_2, ..., b_N]$, all computed modulo a fixed integer $K$ ($2 \le K \le 10^9$, $0 \le a_i, b_i < K$).

In one operation, you may choose any interior index $i \in \{2, 3, ..., N-1\}$ and any integer shift $\delta \in \mathbb{Z}_K$, and simultaneously update the three consecutive capacitors $(i-1, i, i+1)$ as follows:

$$a_{i-1} \leftarrow (a_{i-1} + \delta) \bmod K, \qquad a_i \leftarrow (a_i - 2\delta) \bmod K, \qquad a_{i+1} \leftarrow (a_{i+1} + \delta) \bmod K.$$

Determine whether $A$ can be transformed into $B$ using a finite sequence of operations.

> **Pause & Discover Before Reading On:**
> 1. Look at the stencil $[+1, -2, +1]$ applied at indices $(i-1, i, i+1)$. What linear combination $\sum_{j=1}^{N} c_j a_j \bmod K$ is annihilated by $[+1, -2, +1]$ for *every* $i$? (Can you find two independent such combinations?)
> 2. Better yet, what happens to the **first difference array** $d_j = a_{j+1} - a_j \bmod K$ when you apply $(+\delta, -2\delta, +\delta)$ to $a$?

### Step 1 — Exact Model & Bottleneck

Each operation adds $\delta \cdot v^{(i)} \pmod K$ to the vector $A$, where $v^{(i)}$ has $+1$ at $i-1$, $-2$ at $i$, and $+1$ at $i+1$. Thus $B$ is reachable from $A$ if and only if the difference vector $\Delta = (B - A) \bmod K$ lies in the $\mathbb{Z}_K$-module spanned by $\{v^{(2)}, v^{(3)}, ..., v^{(N-1)}\}$.

- **Bottleneck:** Solving a modular linear system over $\mathbb{Z}_K$ (where $K$ is not necessarily prime) by naive Gaussian elimination is slow and error-prone if done without exploiting the tridiagonal structure.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Checking only the total sum $\sum a_j \bmod K$):** Since $1 - 2 + 1 = 0$, the total sum $\sum_{j=1}^{N} a_j \bmod K$ is indeed invariant. However, assuming that equal total sum is sufficient fails immediately!
  - *Counterexample:* Let $N = 3, K = 5, A = [0, 0, 0], B = [1, 3, 1]$. Both have sum $0 \bmod 5$. Yet any operation at $i = 2$ yields $[\delta, -2\delta, \delta] \bmod 5$. Setting $\delta = 1$ gives $[1, 3, 1]$, which works; what about $B' = [1, 4, 0]$? Its sum is $1 + 4 + 0 \equiv 0 \pmod 5$, yet no $\delta$ gives $a_1 \equiv 1$ and $a_3 \equiv 0$!
- **Tempting Mistake 2 (Dividing by $N$ or $i$ modulo $K$ when $\gcd(i, K) > 1$):** You cannot blindly invert coefficients modulo composite $K$. Instead, use the triangular canonical reduction!

### Step 3 — The Turn: Discovering the Structure

There are two ways to see the structure—algebraic invariants and left-to-right canonical elimination—and they fit together like lock and key:

1. **Two Linear Invariants:** For any operation at $i$ with shift $\delta$:
   - **Total Sum ($0$-th moment):** $\Delta\left(\sum_{j=1}^{N} a_j\right) = \delta - 2\delta + \delta = 0 \pmod K$.
   - **Weighted Position Sum ($1$-st moment):** $\Delta\left(\sum_{j=1}^{N} j \cdot a_j\right) = (i-1)\delta - 2i\delta + (i+1)\delta = 0 \pmod K$.
2. **Triangular Left-to-Right Canonical Form:** Notice that the stencil $v^{(i)}$ has its **leftmost nonzero entry** at $i-1$ with coefficient $+1$ (which is a unit in $\mathbb{Z}_K$ for *every* $K$!). Therefore, starting from $i = 2$, we can uniquely choose $\delta_2 = -a_1 \bmod K$ to force $a_1 \leftarrow 0$. Next, for $i = 3$, choose $\delta_3 = -a_2 \bmod K$ (which only touches indices $2, 3, 4$, leaving $a_1 = 0$ untouched!) to force $a_2 \leftarrow 0$. Proceeding for $i = 2, 3, ..., N-1$, **every** array $A$ is uniquely and reversibly reduced to a canonical array of the form:
$$\mathrm{Canon}(A) = [0, 0, ..., 0, r_{N-1}, r_N].$$

**Two-Moment Invariant Theorem & Canonical Sufficiency:**

Since every legal operation preserves both:

$$\Phi_0(A) = \sum_{j=1}^{N} a_j \bmod K \qquad \text{and} \qquad \Phi_1(A) = \sum_{j=1}^{N} j \cdot a_j \bmod K,$$

and every state $A$ is reversibly equivalent to $[0, \ldots, 0, r_{N-1}, r_N]$, those last two coordinates satisfy:

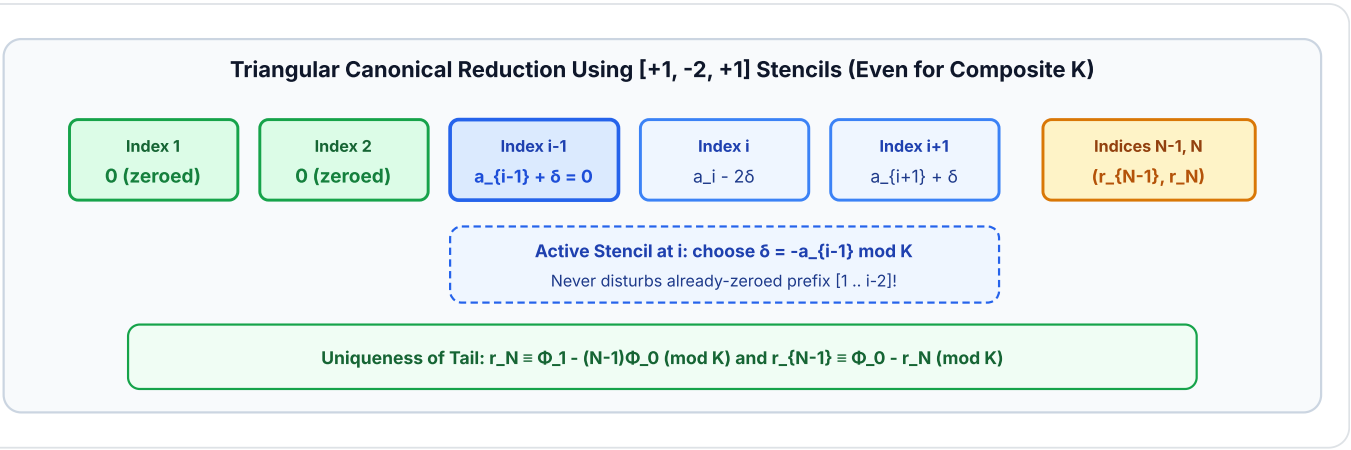
$$r_{N-1} + r_N \equiv \Phi_0(A) \pmod K, \qquad (N-1)r_{N-1} + N r_N \equiv \Phi_1(A) \pmod K.$$

Subtracting $(N-1)\Phi_0(A)$ from $\Phi_1(A)$ uniquely determines $r_N \equiv \Phi_1(A) - (N-1)\Phi_0(A) \pmod K$ and $r_{N-1} \equiv \Phi_0(A) - r_N \pmod K$ without any division! Therefore:

$$A \xrightarrow{*} B \Leftrightarrow \Phi_0(A) = \Phi_0(B) \ \text{ and } \ \Phi_1(A) = \Phi_1(B).$$

### Step 4 — Progressive Construction & Visual Trace

Let us visualize the left-to-right sweep that zeroes out $a_1, a_2, \ldots, a_{N-2}$ and concentrates the entire state into $(r_{N-1}, r_N)$.




```cpp
#include <iostream>
#include <vector>

using namespace std;

pair<long long, long long> compute_invariants(const vector<long long>& arr, long long K) {
    long long phi0 = 0;
    long long phi1 = 0;
    for (size_t i = 0; i < arr.size(); ++i) {
        long long val = ((arr[i] % K) + K) % K;
        long long pos = static_cast<long long>(i + 1) % K;
        phi0 = (phi0 + val) % K;
        phi1 = (phi1 + (__int128)pos * val) % K;
    }
    return {phi0, phi1};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    long long K;
    if (!(cin >> n >> K)) return 0;

    vector<long long> A(n), B(n);
    for (int i = 0; i < n; ++i) cin >> A[i];
    for (int i = 0; i < n; ++i) cin >> B[i];

    auto invA = compute_invariants(A, K);
```

```
    auto invB = compute_invariants(B, K);

    if (invA == invB) {
        cout << "YES\n";
    } else {
        cout << "NO\n";
    }
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Necessity ( $\implies$ ):** Every operation adds $\delta(1-2+1) = 0 \pmod K$ to $\Phi_0$ and $\delta((i-1) - 2i + (i+1)) = 0 \pmod K$ to $\Phi_1$. Thus any reachable $B$ must satisfy $\Phi_0(B) = \Phi_0(A)$ and $\Phi_1(B) = \Phi_1(A)$.
- **Sufficiency ( $\impliedby$ ):** If $\Phi_0(A) = \Phi_0(B)$ and $\Phi_1(A) = \Phi_1(B)$, then reducing both $A$ and $B$ via the left-to-right algorithm produces $[0, ..., 0, r_{N-1}, r_N]$ with identical $(r_{N-1}, r_N)$. Because every operation $(i, \delta)$ has the exact inverse $(i, -\delta \bmod K)$, concatenating the reduction of $A$ with the reversed reduction of $B$ transforms $A$ into $B$.
- **Small-Case Oracle:** Run BFS on all $K^N$ states for $N = 5, K = 4$ (composite!) and verify that every connected component of the state graph has size $K^{N-2} = 16$ and is uniquely indexed by $(\Phi_0, \Phi_1) \in \mathbb{Z}_4 \times \mathbb{Z}_4$.

---

## Worked Problem 51.2: Cascading Quantum Packet Splitter (Monovariant & Exact Bound)

### Problem & Constraints

A linear buffer consists of $N$ cells $0, 1, ..., N-1$ ($1 \le N \le 25$), where cell $i$ initially contains $a_i \ge 0$ tokens ($0 \le a_i \le 10^9$). In one operation, you may choose any cell $i \in \{0, 1, ..., N-2\}$ that currently holds **at least** $1$ **token**, remove $1$ token from cell $i$, and place **2 tokens** into cell $i+1$:

$$(a_i, a_{i+1}) \leftarrow (a_i - 1,\ a_{i+1} + 2).$$

Additionally, at cell $N-1$ (the sink), you may remove $1$ token at a time: $a_{N-1} \leftarrow a_{N-1} - 1$.

1. Prove that no matter what order of valid operations is chosen, the process **must terminate** in finitely many steps.
2. Compute the **exact number of operations** until the buffer becomes completely empty (or prove that the number of operations is independent of the choice of moves!), modulo $10^9 + 7$.

**Pause & Discover Before Reading On:**

1. At first sight, the total number of tokens $\sum a_i$ *increases* whenever we move from $i$ to $i+1$ ($-1 + 2 = +1$). Why doesn't the process run forever?
2. Assign a weight $w_i$ to each token in cell $i$ so that replacing $1$ token at $i$ with $2$ tokens at $i+1$ leaves the weighted sum $\sum w_i a_i$ **unchanged** (or decreases it by a fixed constant)!

### Step 1 — Exact Model & Bottleneck

Simulating the process step-by-step fails because each token at cell $0$ doubles at every step $0 \to 1 \to 2 \to ... \to N-1$, generating $2^{N-1}$ tokens at the sink and requiring $\sim a_i \cdot 2^{N-i}$ operations.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake (Thinking the move order changes the total number of operations):** Suppose you have two choices: split a token at cell $0$, or remove a token at cell $N-1$. Does the choice affect the total step count? No! Every operation acts linearly and independently on each token's descendants.

### Step 3 — The Turn: Discovering the Structure

Let us search for a **linear monovariant** $\Phi(A) = \sum_{i=0}^{N-1} w_i a_i$ that decreases by **exactly** $1$ on every single operation, whether it is an internal split $i \to i+1$ or a sink removal at $N-1$: 1. Removing $1$ token from cell $N-1$ changes $\Phi$ by $-w_{N-1}$. For $\Delta\Phi = -1$, set:

$$w_{N-1} = 1.$$

2. Splitting $1$ token at cell $i$ into $2$ tokens at cell $i+1$ changes $\Phi$ by $-w_i + 2w_{i+1}$. For $\Delta\Phi = -1$, we need:

$$-w_i + 2w_{i+1} = -1 \Leftrightarrow w_i = 2w_{i+1} + 1.$$

Solving this recurrence backward from $w_{N-1} = 1$: - $w_{N-1} = 1 = 2^1 - 1$, - $w_{N-2} = 2(1) + 1 = 3 = 2^2 - 1$, - $w_i = 2^{N-i} - 1$ for all $0 \le i \le N-1$!

**Exact Potential Function & Strong Confluence (Newman's Lemma / Monovariant):**

Define the potential function:

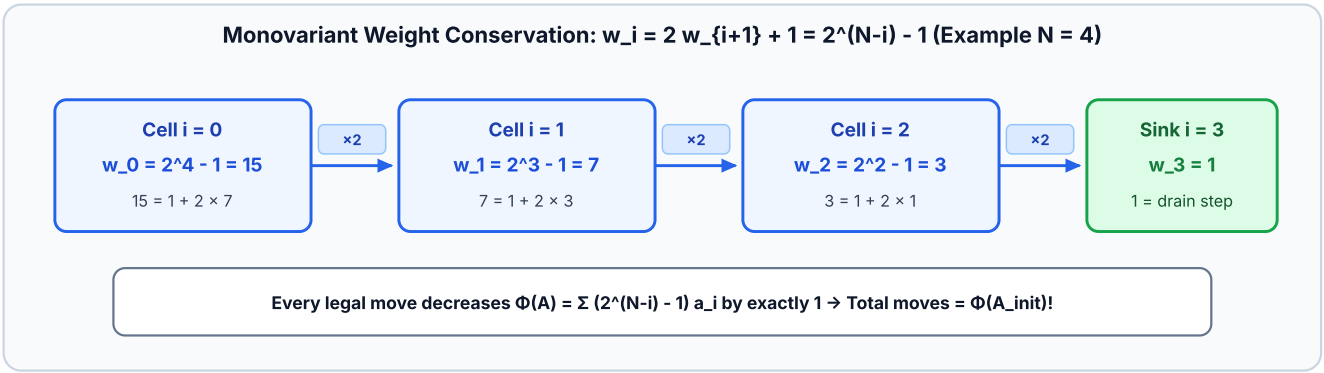
$$\Phi(A) = \sum_{i=0}^{N-1} \left(2^{N-i} - 1\right) a_i.$$

- In the initial state $A$, $\Phi(A) \ge 0$ is a finite integer.
- Every valid operation—regardless of which cell $i$ is chosen—satisfies $\Phi(A_{\text{next}}) = \Phi(A) - 1$.
- The process terminates only when $a_0 = a_1 = \ldots = a_{N-1} = 0$, where $\Phi(\mathbf{0}) = 0$.

Therefore, **every** maximal sequence of valid operations terminates in **exact step count** $\Phi(A_{\text{init}}) = \sum_{i=0}^{N-1} (2^{N-i} - 1) a_i$!

### Step 4 — Progressive Construction & Visual Trace

Let us visualize how the weight $w_i = 2^{N-i} - 1$ absorbs the binary branching tree of a token at cell $i$: one token at cell $i$ is the root of a complete binary tree of height $N - i$ whose total node count is $1 + 2 + 4 + \ldots + 2^{N-1-i} = 2^{N-i} - 1$.




```cpp
#include <iostream>
#include <vector>

using namespace std;

const long long MOD = 1000000007LL;

long long mod_pow(long long base, long long exp) {
    long long res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp & 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp >>= 1;
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    if (!(cin >> n)) return 0;
```

```
    long long total_ops = 0;
    for (int i = 0; i < n; ++i) {
        long long a_i;
        cin >> a_i;
        long long w_i = (mod_pow(2, n - i) - 1 + MOD) % MOD;
        total_ops = (total_ops + (a_i % MOD) * w_i) % MOD;
    }

    cout << total_ops << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Termination & Exact Step Count Proof:** Because $a_i \geq 0$ and $w_i = 2^{N-i} - 1 \geq 1$, $\Phi(A) = \sum_{i=0}^{N-1} w_i a_i$ is a nonnegative integer that equals $0$ if and only if $a_i = 0$ for all $i$. Every available move reduces $\Phi(A)$ by $1$, and as long as $\Phi(A) > 0$, at least one $a_i > 0$ so a legal move exists. Thus every play sequence has length $\Phi(A_{\text{init}})$.
- **Small-Case Oracle:** Simulate randomized move selection on $N = 6$ with small $a_i \in [0, 3]$ across $100$ random seeds and assert that every single run takes the exact same number of steps $\sum_{i=0}^{N-1} (2^{N-i} - 1)a_i$.

---

# Section 52 — Game Theory

## The Discovery & Modeling Approach

Combinatorial game problems in competitive programming look deceptively diverse—players remove stones from piles, cut grids, move tokens along directed graphs, or split heaps—yet they all reduce to classifying positions under optimal play. Never guess a closed-form rule from the story of a game! Instead, start by checking the **structural axioms** of the game:

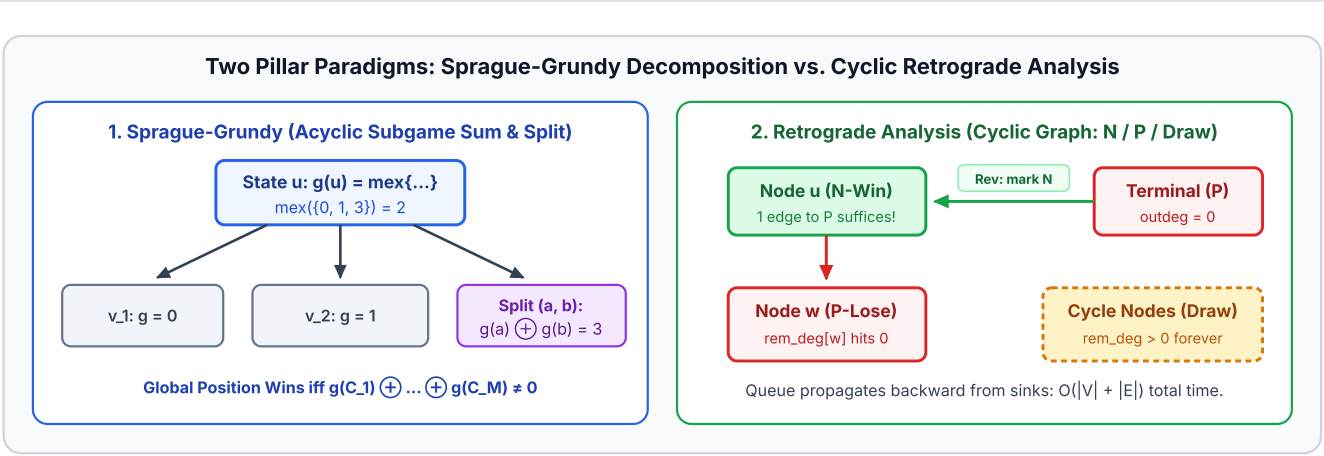
1. **Normal-Play Impartial DAG Game (W/L States & Sprague-Grundy):**
   - **Impartial:** Both players have the exact same legal moves from any given state.
   - **Normal Play:** The first player unable to move **loses** (equivalently, the player who makes the last move wins).
   - **Acyclic (Finite DAG):** Every play sequence terminates in finitely many moves. Under these three conditions, every state $u$ is uniquely either a **Losing ($\mathcal{P}$-position, Previous player wins)** or **Winning ($\mathcal{N}$-position, Next player wins)** state:
   - $u \in \mathcal{P} \Leftrightarrow$ **all** transitions $u \to v$ lead to $v \in \mathcal{N}$ (including terminal states with no moves).
   - $u \in \mathcal{N} \Leftrightarrow$ **at least one** transition $u \to v$ leads to $v \in \mathcal{P}$.

2. **Disjunctive Sum of Independent Subgames (`mex` & XOR Sum):** What if a position consists of $M$ independent components $(u_1, u_2, ..., u_M)$, and a turn consists of choosing **one** component $u_k$ and making a legal move in it (or splitting one component into two independent components $u' + u''$)? By the **Sprague-Grundy Theorem**, every impartial normal-play DAG game state $u$ is equivalent to a single Nim pile of size $g(u)$ (its **Grundy value** or **nim-value**), defined recursively by:
$$g(u) = \operatorname{mex}\left\{ g(v) \mid u \to v \right\}, \qquad g(u_1 + u_2) = g(u_1) \oplus g(u_2),$$
where $\operatorname{mex}(S) = \min(\mathbb{Z}_{\geq 0} \backslash S)$ is the minimum excluded nonnegative integer. The composite game $(u_1, ..., u_M)$ is winning ($\mathcal{N}$) if and only if $g(u_1) \oplus g(u_2) \oplus ... \oplus g(u_M) \neq 0$.

3. **Directed Graph Games with Cycles (Retrograde Analysis):** What if the state graph has **directed cycles**, so play can continue forever as a **Draw ($\mathcal{D}$)**? Standard top-down memoization enters an infinite loop! You must instead run **Retrograde Analysis** (a reverse BFS queue starting from known terminal $\mathcal{P}$-states):
   - Any predecessor $u$ of a known $\mathcal{P}$-state is immediately marked $\mathcal{N}$ (it needs only **one** winning move).
   - Any predecessor $u$ of a known $\mathcal{N}$-state decrements its `remaining_out_degree[u]`; only when `remaining_out_degree[u] == 0` (meaning **every** outgoing move leads to $\mathcal{N}$) does $u$ become $\mathcal{P}$!
   - All states still unresolved when the queue empties are **Draw ($\mathcal{D}$)** states under optimal play.

### Decision Compass & Structural Lenses

| Game Class | Defining Structural Signature | Core Algorithmic Engine | Complexity | Classic Pitfall |
|---|---|---|---|---|
| **Single Impartial Game (Small State Space)** | One game board, acyclic transitions, $\lvert V\rvert+\lvert E\rvert \leq 10^6$ | Memoized W/L DP on the DAG | $O(\lvert V\rvert+\lvert E\rvert)$ | Guessing parity without computing small states first |
| **Composite / Splitting Impartial Game** | $M$ independent subgames, or one move splits state $n \to (a, b)$ | Sprague-Grundy: $g(u) = \operatorname{mex}\{g(v)\}$, $g(a+b) = g(a) \oplus g(b)$ | $O(\sum \operatorname{outdeg}(u))$ to build $g$, $O(M)$ to XOR | Using boolean W/L instead of integer Grundy values when subgames sum! |
| **Huge-Parameter Nim / Periodic Grundy** | Pile sizes up to $10^{18}$ with subtraction/division rules | Compute $g(0...500)$ via brute-force oracle, detect preperiod + period or base-2 pattern, then prove inductively | $O(1)$ or $O(\log N)$ per pile | Assuming periodicity without checking for a preperiod! |
| **Symmetric / Strategy-Stealing Game** | Center-symmetric board or pass-like option available from start | Construct an explicit **mirror pairing** $m \leftrightarrow \bar{m}$ or non-constructive **strategy-stealing** proof | $O(1)$ analytical classification | Forgetting that odd vs. even board center cells break mirror disjointness |
| **Cyclic Graph Game (Win / Lose / Draw)** | Directed graph $G = (V, E)$ with cycles; normal play at sinks | **Reverse BFS Retrograde Analysis** tracking `rem_deg[u]` | $O(\lvert V\rvert+\lvert E\rvert)$ | Running DFS with a `visited` flag (confuses cycles with losing states) |




---

## Worked Problem 52.1: Silicon Wafer Cleaving (Subgame Splitting & Sprague-Grundy)

### Problem & Constraints

A semiconductor foundry starts with $M$ independent linear silicon bars of lengths $L_1, L_2, ..., L_M$ ($1 \le M \le 10^5, 1 \le L_i \le 2000$). Two engineers, Alice and Bob, alternate turns (Alice moves first) under normal play (last player to move wins).

In a single turn, a player chooses **one** bar of length $n \ge 2$ and performs one of two operations: 1. **Trim:** Shave off $1$ or $2$ units from one end, leaving a single bar of length $n-1$ or $n-2$ (provided the remaining length is $\ge 1$, or $0$ if the bar is completely consumed when $n \in \{2\}$ and $2$ units are removed). 2. **Fracture:** Choose any interior unit segment at position $k \in \{2, 3, ..., n-1\}$ and vaporize that 1-unit segment, splitting the bar of length $n$ into **two independent bars** of positive lengths $k-1$ and $n-k$ (so their lengths sum to $n-1$).

Determine whether Alice has a winning strategy from the initial multiset of bars $(L_1, ..., L_M)$, and if so, count how many distinct winning first moves Alice can make across all $M$ bars.

> **Pause & Discover Before Reading On:**
> 1. Why can't we simply mark each length $n$ as Winning ($W$) or Losing ($L$) and count how many bars are $W$?
> 2. When a bar of length $n$ is fractured into two independent bars of lengths $a = k-1$ and $b = n-k$ ($a+b = n-1$), what is the Grundy value of the resulting pair $(a, b)$?
> 3. Given the total XOR sum $X = g(L_1) \oplus ... \oplus g(L_M)$, what single condition must a transition $L_i \to S'$ inside bar $i$ satisfy to be a winning first move?

### Step 1 — Exact Model & Bottleneck

This is an impartial, finite, normal-play game that is the disjunctive sum of $M$ independent single-bar games. A move inside one bar of length $n$ either transitions to a single bar $n' \in \{n-1, n-2\}$ or splits the bar into two independent subgames $(a, b)$ with $a, b \ge 1$ and $a + b = n - 1$.

- **Bottleneck:** Tracking the full multiset of bar lengths as a DP state is impossible because the number of partitions is exponential.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Treating a split $n \to (a, b)$ as $g(a) + g(b)$ instead of $g(a) \oplus g(b)$):** Once a bar fractures into two pieces of lengths $a$ and $b$, future moves act on piece $a$ or piece $b$ independently. By the Sprague-Grundy theorem, the Grundy value of the composite state $(a, b)$ is the **bitwise XOR** $g(a) \oplus g(b)$, never their arithmetic sum!
- **Tempting Mistake 2 (Forgetting the empty state $n = 0$):** A bar of length $0$ has no legal moves, so $g(0) = 0$. A bar of length $1$ cannot be trimmed (since $n \ge 2$ is required) and has no interior segment $k \in \{2, ..., n-1\}$, so $g(1) = 0$ as well!

### Step 3 — The Turn: Discovering the Structure

For each single-bar length $n \in [0, 2000]$, let $g(n)$ be its Grundy value. - Base cases: $g(0) = 0$ and $g(1) = 0$. - For $n \geq 2$, the set of reachable successor Grundy values from a single bar of length $n$ is:

$$\mathcal{T}(n) = \Big\{ g(n-1),\ g(n-2) \Big\} \ \cup \ \Big\{ g(a) \oplus g(n-1-a) \ \Big|\ 1 \leq a \leq n-2 \Big\},$$

and therefore:

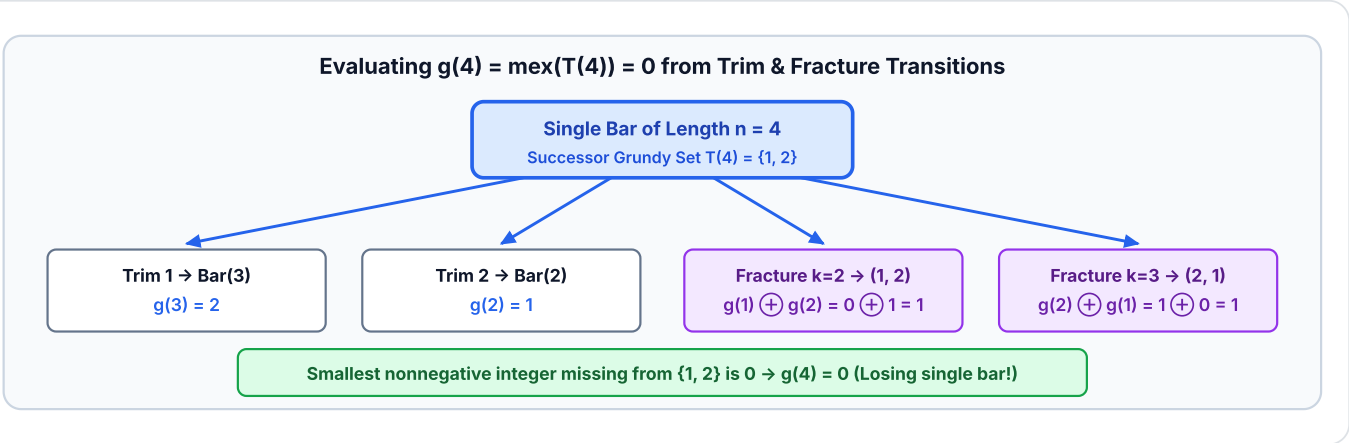
$$g(n) = \operatorname{mex}\Big( \mathcal{T}(n) \Big).$$

**Sprague-Grundy Split Recurrence & Winning First-Move Characterization:**
Let $X = g(L_1) \oplus g(L_2) \oplus \ldots \oplus g(L_M)$ be the global XOR sum.

- Alice has a winning strategy if and only if $X \neq 0$.
- If $X \neq 0$, a move on the $i$-th bar (of length $n = L_i$) to a successor state $u'$ (where $u'$ is either a single bar $n-1$, a single bar $n-2$, or a fractured pair $(k-1, n-k)$) leaves the new global XOR sum:
$$X_{\text{new}} = X \oplus g(L_i) \oplus g(u').$$
Thus the move is a **winning first move** (reaching a global $\mathcal{P}$-position $X_{\text{new}} = 0$) if and only if:
$$g(u') = X \oplus g(L_i).$$

### Step 4 — Progressive Construction & Visual Trace

Let us compute the first few values of $g(n)$ by hand and visualize the transitions: - $g(0) = 0, g(1) = 0$. - $n = 2$: Trim gives $n' = 1$ ( $g(1) = 0$) or $n' = 0$ ($g(0) = 0$); no interior fracture since $\{2 \ldots 1\} = \emptyset$. Thus $\mathcal{T}(2) = \{0\} \implies g(2) = \operatorname{mex}(\{0\}) = 1$. - $n = 3$: Trim gives $g(2) = 1, g(1) = 0$; Fracture at $k = 2$ gives $(1, 1)$ with $g(1) \oplus g(1) = 0 \oplus 0 = 0$. Thus $\mathcal{T}(3) = \{0, 1\} \implies g(3) = 2$. - $n = 4$: Trim gives $g(3) = 2, g(2) = 1$; Fracture at $k = 2, 3$ gives $(1, 2)$ and $(2, 1)$ with $g(1) \oplus g(2) = 0 \oplus 1 = 1$. Thus $\mathcal{T}(4) = \{1, 2\} \implies g(4) = \operatorname{mex}(\{1, 2\}) = 0$!




```cpp
#include <algorithm>
#include <iostream>
#include <vector>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int m;
    if (!(cin >> m)) return 0;

    vector<int> L(m);
    int max_L = 0;
    for (int i = 0; i < m; ++i) {
        cin >> L[i];
        max_L = max(max_L, L[i]);
    }
```

```
    vector<int> g(max_L + 1, 0);
    vector<int> seen_stamp(max_L + 5, 0);
    int stamp = 0;

    for (int n = 2; n <= max_L; ++n) {
        ++stamp;
        // 1. Trim operations: n -> n - 1 and n -> n - 2
        seen_stamp[g[n - 1]] = stamp;
        seen_stamp[g[n - 2]] = stamp;

        // 2. Fracture operations: k in [2, n - 1] splits into (k - 1, n - k)
        for (int a = 1; a <= n - 2; ++a) {
            int b = n - 1 - a;
            int xor_val = g[a] ^ g[b];
            if (xor_val < static_cast<int>(seen_stamp.size())) {
                seen_stamp[xor_val] = stamp;
            }
        }

        int mex = 0;
        while (seen_stamp[mex] == stamp) {
            ++mex;
        }
        g[n] = mex;
    }

    int total_xor = 0;
    for (int len : L) {
        total_xor ^= g[len];
    }

    if (total_xor == 0) {
        cout << "SECOND 0\n";
        return 0;
    }

    // Count winning first moves across all M bars
    // Precompute for each length n how many moves from n achieve a target Grundy value t
    long long winning_moves = 0;
    for (int len : L) {
        if (len < 2) continue;
        int target_g = total_xor ^ g[len];
        if (g[len - 1] == target_g) ++winning_moves;
        if (g[len - 2] == target_g) ++winning_moves;
        for (int a = 1; a <= len - 2; ++a) {
            int b = len - 1 - a;
            if ((g[a] ^ g[b]) == target_g) {
                ++winning_moves;
            }
        }
    }

    cout << "FIRST " << winning_moves << "\n";
    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Sprague-Grundy Equivalence Proof:** By induction on total length $\sum L_i$:
  - If $X = \bigoplus_{i=1}^{M} g(L_i) = 0$, any legal move in bar $i$ transitions $L_i$ to a state $u'$ whose Grundy value $g(u')$ belongs to $\mathcal{T}(L_i)$. By definition of $g(L_i) = \operatorname{mex}(\mathcal{T}(L_i))$, we have $g(u') \neq g(L_i)$, so the new XOR sum $X \oplus g(L_i) \oplus g(u') = g(L_i) \oplus g(u') \neq 0$. Thus every move from $X = 0$ leads to $X' \neq 0$.

- If $X \neq 0$, let $d$ be the highest set bit of $X$, and choose a bar $i$ whose $g(L_i)$ has bit $d$ set. Then $t = g(L_i) \oplus X < g(L_i)$. Since $g(L_i) = \text{mex}(\mathcal{T}(L_i))$, every integer $0 \leq t < g(L_i)$ must appear in $\mathcal{T}(L_i)$, so there exists a legal move in bar $i$ to $u'$ with $g(u') = t$, making $X' = X \oplus g(L_i) \oplus t = 0$.

- **Small-Case Oracle:** Compare the Grundy solver against a direct memoized recursive W/L search over sorted state tuples $(L_1, ..., L_m)$ for $\sum L_i \leq 12$.

---

## Worked Problem 52.2: Orbital Interceptor Pursuit (Retrograde Analysis on Cyclic Graphs)

### Problem & Constraints

A token is placed on a vertex of a directed graph $G = (V, E)$ with $N$ vertices and $M$ directed edges ($1 \leq N, M \leq 2 \times 10^5$). The graph **may contain directed cycles** and self-loops. Two players alternate moving the token along a directed edge $u \to v$ from its current vertex $u$.

- A vertex with **out-degree** $0$ is a **terminal sink**: a player whose turn starts at a sink has no legal move and **loses immediately**.
- In addition, a subset of vertices $T_{\text{trap}} \subseteq V$ are marked as **override traps**: any player who moves the token *into* a vertex $v \in T_{\text{trap}}$ **wins immediately** (meaning the player whose turn *starts* at $v \in T_{\text{trap}}$ has already lost before moving!).
- Both players play optimally: a player prefers **Winning ($\mathcal{N}$)** over **Drawing ($\mathcal{D}$, infinite play)**, and prefers **Drawing ($\mathcal{D}$)** over **Losing ($\mathcal{P}$)**.

Classify every starting vertex $u \in \{1, ..., N\}$ as `WIN` ($\mathcal{N}$), `LOSE` ($\mathcal{P}$), or `DRAW` ($\mathcal{D}$).

**Pause & Discover Before Reading On:**

1. Why does a standard top-down DFS with "visiting = DRAW" fail on cyclic graphs? (Hint: a vertex might have both an edge into a cycle AND an edge into a losing state!)
2. If we start from the known losing vertices ($\mathcal{P}$) and propagate backward along reversed edges $v \to u$, what is the difference between propagating a $\mathcal{P}$-state to its predecessors versus propagating an $\mathcal{N}$-state to its predecessors?

### Step 1 — Exact Model & Bottleneck

Every vertex $u \in V$ belongs to one of three disjoint classes $(\mathcal{P}, \mathcal{N}, \mathcal{D})$ satisfying the fixed-point game equations: - Base losing states: Every $u \in T_{\text{trap}}$ is $\mathcal{P}$ (any incoming move wins, so starting at $u$ is immediate loss; its outgoing edges are irrelevant and should be ignored). Every non-trap sink with $\text{outdeg}(u) = 0$ is also $\mathcal{P}$. - For any non-trap vertex $u$: - $u \in \mathcal{N} \Leftrightarrow \exists (u \to v) \in E$ such that $v \in \mathcal{P}$. - $u \in \mathcal{P} \Leftrightarrow \forall (u \to v) \in E$, we have $v \in \mathcal{N}$. - $u \in \mathcal{D} \Leftrightarrow$ $u$ has no edge to $\mathcal{P}$, and at least one edge to $\mathcal{D}$ (so a player at $u$ cannot force a win, but can always step to another $\mathcal{D}$-vertex to avoid stepping into $\mathcal{N}$).

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake (DFS Cycle Detection = Draw):** Suppose you run DFS from $u$, and when you encounter a vertex $w$ currently on the recursion stack, you conclude $w$ (or $u$) is a Draw.
  - *Counterexample:* Consider edges $1 \to 2$, $2 \to 1$ (a cycle between $1$ and $2$), and $2 \to 3$, where $3$ is a terminal sink ($\mathcal{P}$). A naive DFS $1 \to 2 \to 1$ sees the cycle $1 \leftrightarrow 2$ before exploring $2 \to 3$ and wrongly labels $2$ (or $1$) as Draw! In reality, from vertex $2$ the current player moves $2 \to 3$ and **wins immediately** ($2 \in \mathcal{N}$), and from vertex $1$ the only move is $1 \to 2 \in \mathcal{N}$, so vertex $1$ **loses** ($1 \in \mathcal{P}$)! Neither vertex is a draw!

### Step 3 — The Turn: Discovering the Structure

Instead of searching forward from unresolved states, propagate **backward** from resolved states using the reverse graph $G^{\text{rev}}$ and a degree counter `rem_deg[u]`:

**Retrograde Analysis Invariant & Queue Propagation:**

Initialize $\text{state}[u] = \text{UNKNOWN}$ for all $u$, and set $\text{rem_deg}[u] = \text{outdeg}(u)$ (for $u \in T_{\text{trap}}$, ignore outgoing edges and set $\text{rem_deg}[u] = 0$). Push all initial $\mathcal{P}$-vertices ($\text{rem_deg}[u] = 0$ or $u \in T_{\text{trap}}$) into a queue with $\text{state}[u] = \mathcal{P}$.

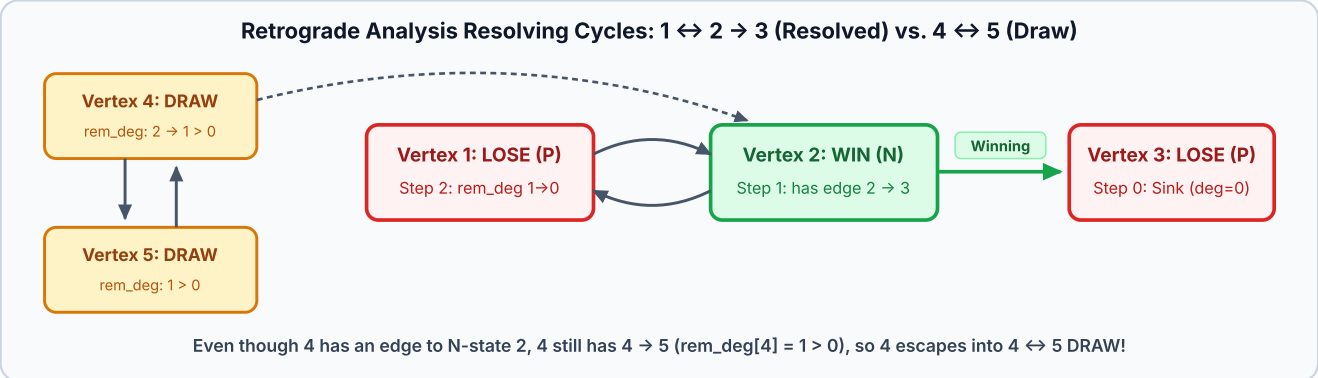
When popping a resolved vertex $v$ from the queue, inspect every reverse edge $u \leftarrow v$ (i.e., forward edge $u \to v$) where $\text{state}[u] = \text{UNKNOWN}$:

1. **If** $\text{state}[v] = \mathcal{P}$ **(Losing):** Then $u$ has a winning move $u \to v$! Immediately set $\text{state}[u] \leftarrow \mathcal{N}$ and push $u$ into the queue.
2. **If** $\text{state}[v] = \mathcal{N}$ **(Winning):** Moving $u \to v$ hands a win to the opponent, so $u$ will avoid this edge if possible. Decrement $\text{rem_deg}[u]$. If $\text{rem_deg}[u]$ reaches $0$, then **every** outgoing edge from $u$ leads to $\mathcal{N}$; set $\text{state}[u] \leftarrow \mathcal{P}$ and push $u$ into the queue!

When the queue becomes empty, every vertex still marked $\text{UNKNOWN}$ has $\text{rem_deg}[u] \geq 1$ (all pointing to other $\text{UNKNOWN}$ vertices) and zero edges to $\mathcal{P}$, so they are precisely the **Draw ($\mathcal{D}$)** states!

### Step 4 — Progressive Construction & Visual Trace

Let us trace our earlier counterexample ($1 \leftrightarrow 2$, $2 \to 3$, plus a separate cycle $4 \leftrightarrow 5$ with $4 \to 2$) under Retrograde Analysis.




```cpp
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

enum State { UNKNOWN = 0, WIN = 1, LOSE = 2 };

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    if (!(cin >> n >> m)) return 0;

    vector<int> is_trap(n + 1, 0);
    for (int i = 1; i <= n; ++i) {
        cin >> is_trap[i]; // 1 if override trap (immediate loss for player starting at i), 0 otherwise
    }

    vector<vector<int>> rev_adj(n + 1);
    vector<int> rem_deg(n + 1, 0);

    for (int i = 0; i < m; ++i) {
        int u, v;
        cin >> u >> v;
        // Outgoing edges from a trap vertex are never used because the game already ended at u
        if (!is_trap[u]) {
```

```
            rev_adj[v].push_back(u);
            rem_deg[u]++;
        }
    }

    vector<int> state(n + 1, UNKNOWN);
    queue<int> q;

    for (int i = 1; i <= n; ++i) {
        if (is_trap[i] || rem_deg[i] == 0) {
            state[i] = LOSE;
            q.push(i);
        }
    }

    while (!q.empty()) {
        int v = q.front();
        q.pop();

        for (int u : rev_adj[v]) {
            if (state[u] != UNKNOWN) continue;

            if (state[v] == LOSE) {
                // Found a move u -> v into a losing state!
                state[u] = WIN;
                q.push(u);
            } else if (state[v] == WIN) {
                // Edge u -> v leads to an opponent win; eliminate this option
                rem_deg[u]--;
                if (rem_deg[u] == 0) {
                    state[u] = LOSE;
                    q.push(u);
                }
            }
        }
    }

    for (int i = 1; i <= n; ++i) {
        if (state[i] == WIN) cout << "WIN\n";
        else if (state[i] == LOSE) cout << "LOSE\n";
        else cout << "DRAW\n";
    }

    return 0;
}
```

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Soundness of Resolved States (Induction on Queue Insertion Time):**
  - Base step: Vertices pushed initially are either in $T_{\text{trap}}$ or have out-degree $0$; they lose in $0$ moves.
  - Inductive step: If $u$ is marked `WIN` at step $t$, it has an edge to a vertex $v$ marked `LOSE` at step $t' < t$, so by induction the current player can force a win in at most $t' + 1$ moves. If $u$ is marked `LOSE` at step $t$, all of its outgoing edges go to vertices marked `WIN` at steps $< t$, so no matter which move is chosen, the opponent can force a win.
- **Completeness (Why `UNKNOWN` States Are Guaranteed Draws):** Let $U = \{u \in V \mid \text{state}[u] = \text{UNKNOWN}\}$ after the queue terminates.
  1. No vertex $u \in U$ has an edge to any `LOSE` vertex (otherwise $u$ would have been marked `WIN` when that `LOSE` vertex was popped). Thus neither player can ever force a transition from $U$ into `LOSE`.
  2. Every vertex $u \in U$ has $\text{rem_deg}[u] \geq 1$. Since all edges from $u$ to `WIN` vertices decremented $\text{rem_deg}[u]$ when those `WIN` vertices were popped, $u$ must have **at least one outgoing edge to another vertex** $w \in U$! Therefore, from any $u \in U$, both players can indefinitely stay inside $U$ by always choosing an edge into $U$, and neither player will ever voluntarily step into a `WIN` vertex (which would hand a forced win to the opponent). Hence every $u \in U$ is a Draw under optimal play.
- **Small-Case Oracle:** Compare against a bounded-horizon minimax solver of depth $2N$ on all directed graphs with $N = 5$ vertices.

# Section 53 — Advanced Combinatorics

## The Discovery & Modeling Approach

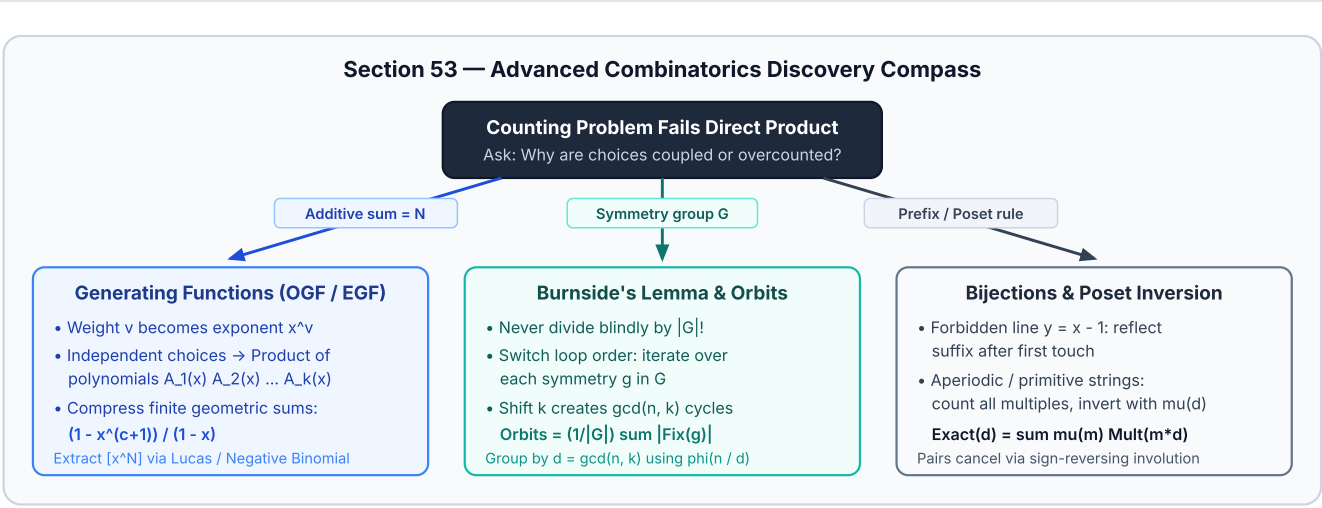
Elementary combinatorics counts sequences where choices are either independent (Multiplication Principle) or partitioned into a small number of disjoint cases (Addition Principle). You know you have crossed into **Advanced Combinatorics** when the problem statement introduces one of three structural obstacles that destroy simple product formulas:

1. **Constrained Additive Coupling with Bounded Multiplicity**: You must choose $K$ items whose weights sum to $N$, where each item $i$ has its own individual upper bound $c_i$, stride $d_i$, or parity rule. A dynamic programming state $(i, \text{current_sum})$ is too slow when $N \leq 10^{12}$ or when many queries ask for different $N$.
2. **Global Symmetry and Unlabeled Equivalence**: Two configurations are declared identical if one can be rotated, reflected, or permuted into the other. Dividing the total number of labeled configurations by the group size $|G|$ fails because highly symmetric configurations have smaller orbits and are under-counted or produce non-integer quotients.
3. **Poset / Lattice / Involution Cancellations**: Paths or sequences must never cross a boundary (Catalan / reflection principle), or an alternating sum of intersecting constraints must be collapsed into a closed form via a sign-reversing involution or Möbius inversion on a divisor/subset poset.

### Decision Compass & Structural Lenses

When a counting problem resists direct DP, translate the combinatorial operation into its exact algebraic or group-theoretic counterpart:

| Combinatorial Obstacle | Diagnostic Signal in Statement | Algebraic / Structural Lens | Core Reduction Formula | Complexity Target |
|---|---|---|---|---|
| **Bounded / Strided Sum** | Choose $x_1 + \ldots + x_k = N$ with $0 \leq x_i \leq c_i$ or $x_i \in S_i$; $k$ is small ($k \leq 20$) or polynomials are uniform, $N \leq 10^{18}$. | **Ordinary Generating Functions (OGF)** | $[x^N] \prod_{i=1}^{k} \frac{1-x^{c_i+1}}{1-x}$ | $O(2^k \log M)$ or $O(k^2)$ coeff extraction |
| **Labeled Structures & Tuples** | Order of positions matters; each item type $i$ uses a set of indices of size $x_i \in S_i$, total length $N$. | **Exponential Generating Functions (EGF)** | $N![x^N] \prod_{i=1}^{k} \left( \sum_{m \in S_i} \frac{x^m}{m!} \right)$ | $O(N \log N)$ via NTT or closed roots |
| **Equivalence under Rotation / Reflection** | "Up to cyclic shift", "necklaces", "bracelets", "grid coloring up to $90^\circ$ rotation". | **Burnside's Lemma & Pólya Enumeration** | $\frac{1}{\lvert G \rvert} \sum_{g \in G} \lvert \mathrm{Fix}(g) \rvert$ | $O(d(N) \cdot T_{\text{cycle}})$ where $d(N)$ is divisor count |
| **Boundary-Avoiding Lattice Walks** | Prefix sums never drop below $0$ (or never touch $y = x + c$). | **Reflection Principle (André's Bijection)** | $\lvert \text{All Paths} \rvert - \lvert \text{Reflected Bad Paths} \rvert$ | $O(1)$ per query after $O(N)$ factorials |
| **Exact-GCD or Exact-Period Counting** | Count objects whose minimal period or GCD is *exactly* $d$, while "divisible by $d$" is easy to count. | **Dirichlet / Divisor Möbius Inversion** | $f(d) = \sum_{m \geq 1} \mu(m) g(m \cdot d)$ | $O(N \log N)$ or $O(\sqrt{N})$ harmonic blocks |




## Worked Problem 53.1: Quantum Vault Calibration

### Problem & Constraints

A quantum vault has $K$ stabilization registers ($1 \le K \le 18$) and requires a total energy of exactly $N$ units ($0 \le N \le 10^{12}$). Each register $i$ operates at a fixed step size $s_i \ge 1$ and capacity limit $c_i \ge 0$: it can hold any energy of the form $m_i \cdot s_i$ where $0 \le m_i \le c_i$. In this hardware architecture, all registers share the **same** base step size $s_i = 1$, except that each register $i$ has its own distinct upper bound $c_i$ ($0 \le c_i \le 10^{12}$).

Count the number of tuples $(m_1, m_2, ..., m_K)$ modulo $10^9 + 7$ such that:

$$m_1 + m_2 + ... + m_K = N \quad \text{and} \quad 0 \le m_i \le c_i \text{ for all } 1 \le i \le K.$$

**Pause & Discover:** Before reading further, write down the brute-force recurrence and identify why $N \le 10^{12}$ breaks standard knapsack DP. If you replace each register's choice $m_i \in \{0, 1, ..., c_i\}$ by a polynomial, what closed-form algebra compresses a sum of $c_i + 1$ terms into just 2 terms?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: We want the size of the set

$$\mathcal{S} = \left\{ (m_1, ..., m_K) \in \mathbb{Z}^K \;\middle|\; 0 \le m_i \le c_i \text{ and } \sum_{i=1}^{K} m_i = N \right\}.$$

  A standard knapsack DP maintains `dp[i][w]` = number of ways to reach sum $w$ using the first $i$ registers, with transition $\mathrm{dp}[i][w] = \sum_{m=0}^{c_i} \mathrm{dp}[i-1][w-m]$.
- **Bottleneck**: Even with prefix sums optimizing each transition to $O(1)$, the state space has size $K \times N$. Since $N \le 10^{12}$, any algorithm proportional to $N$ will exceed both time and memory limits by six orders of magnitude. On the other hand, $K \le 18$ is remarkably small! The complexity must depend on $2^K$ or $K$, not $N$.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Greedy capping or clamping)**: Computing the unconstrained stars-and-bars count $\binom{N+K-1}{K-1}$ and subtracting $\sum_{i=1}^{K} \binom{N-(c_i+1)+K-1}{K-1}$ (subtracting configurations where one register overflows).
- **Counterexample**: Let $K = 2$, $c_1 = 1$, $c_2 = 1$, and $N = 2$.
  - Only $(1, 1)$ is valid, so the true answer is $1$.
  - Unconstrained ways for $m_1 + m_2 = 2$: $(2, 0), (1, 1), (0, 2) \implies \binom{2+2-1}{1} = 3$.
  - For $K = 3, c = (1, 1, 1), N = 4$: the true answer is $0$ (maximum sum is 3). Unconstrained: $\binom{4+2}{2} = 15$. Subtracting single violations: $3 \times \binom{4-2+2}{2} = 3 \times 6 = 18$. That yields $15 - 18 = -3 \neq 0$, because tuples like $(2, 2, 0)$ violate *two* bounds simultaneously and were subtracted twice!

### Step 3 — The Turn: Discovering the Structure

Encode the choices of register $i$ as an **Ordinary Generating Function (OGF)**: choosing $m_i$ units corresponds to picking the term $x^{m_i}$ from the polynomial

$$P_i(x) = 1 + x + x^2 + \ldots + x^{c_i}.$$

When we multiply $P_1(x)P_2(x)\cdots P_K(x)$, exponents add when terms multiply: $x^{m_1} \cdot x^{m_2} \cdots x^{m_K} = x^{m_1 + \ldots + m_K}$. Therefore, the number of valid tuples summing to $N$ is the coefficient of $x^N$ in the product:

$$\text{Ans} = [x^N] \prod_{i=1}^{K} \left(1 + x + x^2 + \ldots + x^{c_i}\right).$$

Now apply the finite geometric series identity to compress each $c_i$-degree polynomial into a quotient of two binomials:

$$1 + x + \ldots + x^{c_i} = \frac{1 - x^{c_i+1}}{1 - x}.$$

**Generating Function Rational Compression:**

$$\text{Ans} = [x^N] \left( \prod_{i=1}^{K} \left(1 - x^{c_i+1}\right) \right) \cdot \frac{1}{(1-x)^K}$$

where the numerator has at most $2^K$ terms:

$$\prod_{i=1}^{K} \left(1 - x^{c_i+1}\right) = \sum_{S \subseteq \{1,\ldots,K\}} (-1)^{|S|} x^{\sum_{i \in S} (c_i+1)}$$

and the denominator expands via the negative binomial series:

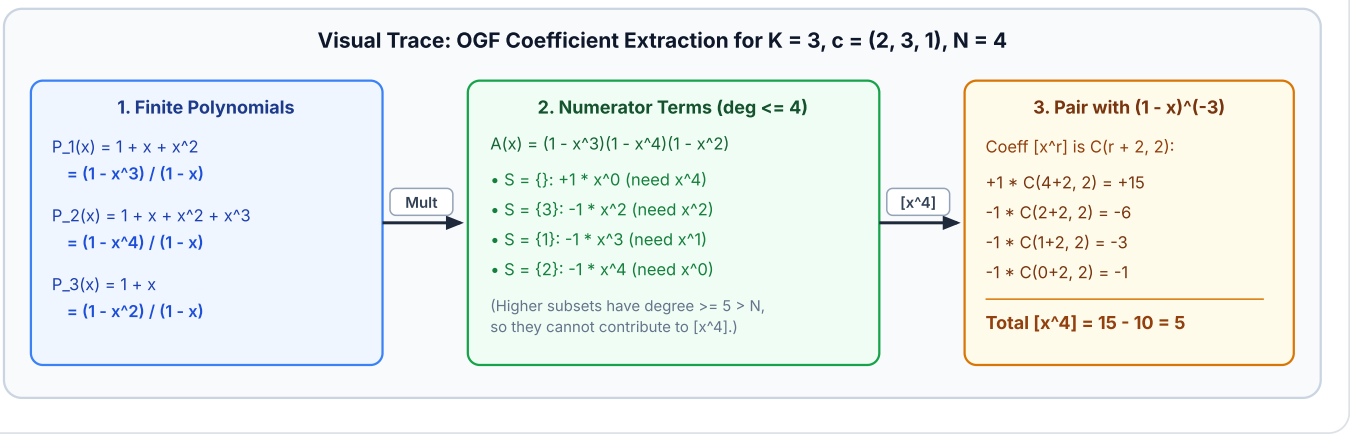
$$\frac{1}{(1-x)^K} = \sum_{r=0}^{\infty} \binom{r+K-1}{K-1} x^r.$$

To extract $[x^N]$ from the product $A(x) \cdot B(x)$, each subset $S \subseteq \{1, \ldots, K\}$ contributes $(-1)^{|S|} x^{W(S)}$ from the numerator, where $W(S) = \sum_{i \in S} (c_i + 1)$. To reach total exponent $N$, we must pair it with $x^{N - W(S)}$ from $(1-x)^{-K}$, which has coefficient $\binom{N - W(S) + K - 1}{K - 1}$ whenever $N \geq W(S)$ (and $0$ otherwise).

### Step 4 — Progressive Construction & Visual Trace

Let us trace how the OGF multiplication turns an $O(N)$ convolution into an $O(2^K \cdot K)$ coefficient extraction:

1. **Numerator Expansion**: Iterate over all bitmasks $\text{mask} \in [0, 2^K - 1]$.
2. **Subset Exponent**: Compute $W(\text{mask}) = \sum_{i \in \text{mask}} (c_i + 1)$ in 64-bit integer arithmetic (stopping early if $W(\text{mask}) > N$).
3. **Complementary Degree**: If $R = N - W(\text{mask}) \geq 0$, the required term from $(1-x)^{-K}$ has coefficient
$$\binom{R+K-1}{K-1} = \frac{(R+1)(R+2)\cdots(R+K-1)}{(K-1)!} \pmod{10^9+7},$$
which takes $O(K)$ operations to evaluate directly even when $R \approx 10^{12}$!
4. **Sign Application**: Add or subtract the term according to the parity of `popcount(mask)`.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Algebraic & Combinatorial Two-Way Proof**:

1. *Algebraic View*: In the ring of formal power series $\mathbb{Z}[[x]]$, $(1-x^{c_i+1}) = (1-x)(1+x+\ldots+x^{c_i})$ is an exact polynomial identity. Multiplying across $i = 1, \ldots, K$ and expanding $(1-x)^{-K} = \sum_{r \geq 0} \binom{r+K-1}{K-1} x^r$ preserves every coefficient up to $x^N$.
2. *Combinatorial Inclusion-Exclusion Dual*: Notice that the numerator expansion $\sum_S (-1)^{|S|} \binom{N - \sum_{i \in S}(c_i+1) + K - 1}{K-1}$ is **literally** the Principle of Inclusion-Exclusion where property $P_i$ is the violation $m_i \geq c_i + 1$! Subtracting $c_i + 1$ from $N$ pre-allocates $c_i + 1$ units to register $i$, guaranteeing it overflows, and distributes the remaining $N - W(S)$ units freely among all $K$ registers. Thus OGF algebra automatically discovers the exact Inclusion-Exclusion formula without manual casework.

- **Small-Case Oracle**: For $K = 3, c = (2, 3, 1), N = 4$, a 3-nested loop checks all $(2+1)(3+1)(1+1) = 24$ tuples $(m_1, m_2, m_3)$ and finds the $5$ valid solutions: $(2,2,0), (2,1,1), (1,3,0), (1,2,1), (0,3,1)$, matching our trace $15 - 6 - 3 - 1 = 5$. Total runtime is $O(2^K \cdot K)$, which takes $< 5 \times 10^6$ operations for $K = 18$.

---

## Worked Problem 53.2: Balanced Rotary Collars

### Problem & Constraints

A rotary encoder collar is a circular ring of $N$ slots ($3 \leq N \leq 10^9$), each painted with one of $C$ colors ($2 \leq C \leq 10^9$). Color $0$ is a special **calibration marker**; colors $1, 2, \ldots, C-1$ are standard signal colors. A collar is valid if **no two adjacent slots on the circle both contain the calibration marker (color $0$)** — including the circular wrap-around between slot $N-1$ and slot $0$. Two valid collars are considered **identical** if one can be obtained from the other by a cyclic rotation of the ring by $k \in \{0, 1, \ldots, N-1\}$ steps (reflections are *not* allowed).

Compute the number of distinct valid collars modulo $10^9 + 7$.

> **Pause & Discover:** Why can't we simply count valid linear strings of length $N$ with circular adjacency and divide by $N$? If we instead fix a rotation shift $k \in \{0, \ldots, N-1\}$, what must be true about a coloring so that shifting it by $k$ leaves every slot's color unchanged?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: Let $X$ be the set of all valid circular colorings of length $N$ (where slot $i$ and slot $(i+1) \bmod N$ are never both color $0$). The cyclic group $G = \mathbb{Z}_N = \{r_0, r_1, \ldots, r_{N-1}\}$ acts on $X$ by rotating indices: $(r_k \cdot x)_i = x_{(i+k) \bmod N}$. Two colorings $x, y \in X$ are equivalent iff $y = r_k \cdot x$ for some $k$. We want the number of equivalence classes (orbits) $|X/G|$.
- **Bottleneck**: First, $|X|$ grows as $\approx C^N$, and $N \leq 10^9$, so enumerating colorings or even iterating over all $N$ rotations $k = 0, \ldots, N-1$ is too slow ($O(N)$ is TLE for $N = 10^9$). We need a reduction that compresses both the counting of fixed colorings and the sum over $N$ rotations to $O(\sqrt{N} \log N)$ via the divisors of $N$.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Divide $|X|$ by $N$)**: Computing the number of valid circular colorings $|X|$ and multiplying by $N^{-1} \pmod{10^9+7}$.
- **Counterexample**: Let $N = 4, C = 2$ (color $0$ = Black, color $1$ = White). Valid circular colorings of length $4$ with no two adjacent $0$s:
  1. `1111` (orbit size $1$: rotating `1111` gives only `1111`).
  2. `0111`, `1011`, `1101`, `1110` (orbit size $4$: one necklace).
  3. `0101`, `1010` (orbit size $2$: one necklace, because period is $2$). Total valid colorings $|X| = 1 + 4 + 2 = 7$. Notice that $7$ is **not even divisible by** $N = 4$! The true number of distinct necklaces is $3$ (`1111`, `0111`, `0101`), whereas $7/4 = 1.75$.
- **Tempting Mistake 2 (Assuming each cycle of $r_k$ can be colored independently without constraint)**: Under rotation $r_k$, all elements in the same cycle of the permutation must share the same color. However, adjacent slots on the ring belong to *adjacent cycles*, so choosing color $0$ for one cycle forbids color $0$ for its neighboring cycle!

### Step 3 — The Turn: Discovering the Structure

By **Burnside's Lemma**, the number of orbits equals the average number of elements fixed by the group actions:

$$|X/G| = \frac{1}{N} \sum_{k=0}^{N-1} |\mathrm{Fix}(r_k)|,$$

where $\mathrm{Fix}(r_k) = \{x \in X \mid r_k \cdot x = x\}$.

What does $r_k \cdot x = x$ mean for a circular coloring $x$? It means $x_i = x_{(i+k) \bmod N}$ for all $i$. By Bézout's identity, repeatedly adding $k \pmod N$ generates all multiples of $d = \gcd(N, k)$. Thus, $x$ is invariant under shift $k$ if and only if $x$ is **periodic with period** $d = \gcd(N, k)$:

$$x_i = x_{i \bmod d} \quad \text{for all } 0 \le i < N.$$

Since $d$ divides $N$, wrapping around at $N$ is identical to wrapping around at $d$: adjacent slots in the full ring of length $N$ are simply adjacent slots in the repeating prefix block $x_0, x_1, \ldots, x_{d-1}$ of length $d$ (with circular adjacency between $x_{d-1}$ and $x_0$)!

Let $F(d)$ be the number of valid circular colorings of length $d$. Each slot is either color $0$ ($1$ way) or a non-zero color ($C-1$ ways). By standard $2 \times 2$ transition matrix $M = \begin{pmatrix} 0 & C-1 \\ 1 & C-1 \end{pmatrix}$ (where state $0$ is color $0$ and state $1$ is non-zero), the number of closed circular walks of length $d$ on the transition graph is the **matrix trace**:

$$F(d) = \operatorname{Tr}\left(M^d\right) \quad \text{for } d \ge 2, \qquad \text{and} \qquad F(1) = C - 1$$

(Wait: for $d = 1$, slot $0$ is adjacent to itself around the circle of length $N \ge 3$! If the period is $d = 1$, all $N$ slots have the same color, so choosing color $0$ makes adjacent slots both $0$, which is forbidden. And indeed $\operatorname{Tr}(M^1) = 0 + (C-1) = C - 1$, so $F(d) = \operatorname{Tr}(M^d)$ holds for **all** $d \ge 1$, including $d = 1$!).

Finally, how many shifts $k \in \{0, \ldots, N-1\}$ share the same $\gcd(N, k) = d$? Setting $k = d \cdot j$ with $0 \le j < N/d$, we have $\gcd(N, d \cdot j) = d \Leftrightarrow \gcd(N/d, j) = 1$. By definition, there are $\varphi(N/d)$ such values of $j$!

**Burnside + Euler Totient Divisor Reduction:**

$$|X/G| = \frac{1}{N} \sum_{d \mid N} \varphi\left(\frac{N}{d}\right) \operatorname{Tr}\left( \begin{pmatrix} 0 & C-1 \\ 1 & C-1 \end{pmatrix}^d \right) \pmod{10^9 + 7}$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace our earlier counterexample $N = 4, C = 2$, where $M = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$: - $d = 1$: $\varphi(4/1) = \varphi(4) = 2$ (shifts $k = 1, 3$). $\operatorname{Tr}(M^1) = 0 + 1 = 1$ (`1111`). - $d = 2$: $\varphi(4/2) = \varphi(2) = 1$ (shift $k = 2$). $M^2 = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix} \Longrightarrow \operatorname{Tr}(M^2) = 3$ (`1111`, `0101`, `1010`). - $d = 4$: $\varphi(4/4) = \varphi(1) = 1$ (shift $k = 0$). $M^4 = \begin{pmatrix} 2 & 3 \\ 3 & 5 \end{pmatrix} \Longrightarrow \operatorname{Tr}(M^4) = 7$. - Sum over divisors: $2 \cdot 1 + 1 \cdot 3 + 1 \cdot 7 = 2 + 3 + 7 = 12$. - Divide by $N = 4$: $12/4 = 3$ distinct collars!

**Burnside Table: Counting Pairs (Coloring x, Rotation r_k) where r_k(x) = x for N = 4, C = 2**

| Shift k | d = gcd(4, k) | Fixed Valid Colorings Fix(r_k) | \|Fix(r_k)\| |
|---|---|---|---|
| k = 0 | d = 4 | 1111, 0101, 1010, 0111, 1011, 1101, 1110 | 7 |
| k = 1 | d = 1 | 1111 (0000 invalid: adjacent 0s) | 1 |
| k = 2 | d = 2 | 1111, 0101, 1010 | 3 |
| k = 3 | d = 1 | 1111 (same cycle structure as k=1) | 1 |

**Why Each Orbit Contributes 4**

- Orbit {1111} (size 1): Fixed by k = 0, 1, 2, 3 → 1 x 4 = 4
- Orbit {0101, 1010} (size 2): Each fixed by k = 0, 2 → 2 x 2 = 4
- Orbit {0111, ...} (size 4): Each fixed by k = 0 → 4 x 1 = 4

**Sum = 12 ⇒ 12 / 4 = 3 Orbits**

### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Double Counting Proof (Orbit-Stabilizer)**: Count the bipartite incidence set $\mathcal{I} = \{(x, g) \in X \times G \mid g \cdot x = x\}$ in two ways:
  1. *By group element* $g \in G$: $\sum_{g \in G} |\operatorname{Fix}(g)|$.
  2. *By configuration* $x \in X$: $\sum_{x \in X} |\operatorname{Stab}(x)|$. By the Orbit-Stabilizer Theorem, $|\operatorname{Orb}(x)| \cdot |\operatorname{Stab}(x)| = |G|$. Grouping elements $x$ into their disjoint orbits $\mathcal{O} \in X/G$, each orbit contributes $\sum_{x \in \mathcal{O}} \frac{|G|}{|\mathcal{O}|} = |G|$. Hence $\sum_{g \in G} |\operatorname{Fix}(g)| = |G| \cdot |X/G|$, proving that every orbit contributes exactly $1$ after dividing by $|G|$, regardless of its internal symmetry!

- **Small-Case Oracle**: For $N \leq 10, C \leq 3$, generate all $C^N$ strings, filter those with no circular adjacent `0`s, canonicalize each string to its lexicographically smallest rotation, and insert into a `std::set`. Compare the set size against the $O(\sqrt{N}\log N)$ divisor-totient formula.

---

# Section 54 — Geometry

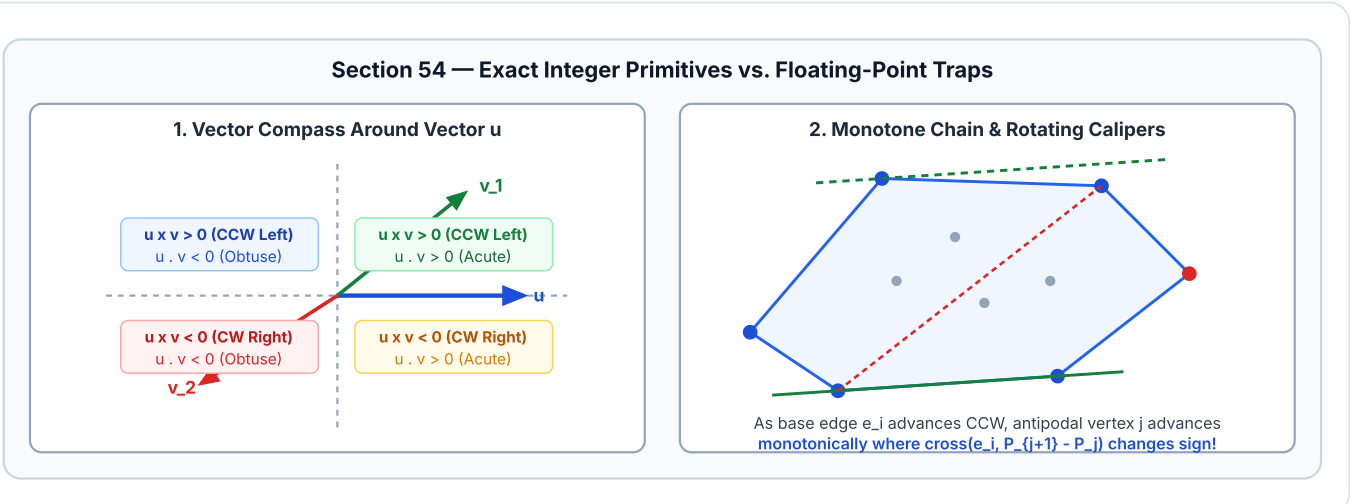
## The Discovery & Modeling Approach

Computational geometry problems become bug-prone only when solved with ad-hoc trigonometry (`acos`, `atan2`, slope $m = \Delta y / \Delta x$, line equations $y = mx + b$). Every slope division risks division by zero on vertical lines, and every `double` comparison risks catastrophic cancellation when coordinates reach $10^9$.

**The Core Principle of Competitive Geometry**: Replace angles, slopes, and distances with **exact integer vector algebra** (Dot Product and 2D Cross Product) and reduce 2D spatial queries to **1D monotonic sweeps** (by $x$-coordinate or by polar angle).

### Decision Compass & Structural Lenses

| Geometric Task | Coordinate / Input Signal | Exact Algebraic Primitive | Key Structural Reduction | Complexity Target |
|---|---|---|---|---|
| **Orientation, Side-of-Line, & Segment Crossing** | Integer coordinates $\|x_i\|, \|y_i\| \le 10^9$; classify left/right turn, collinearity, or proper intersection. | **2D Cross Product** $\vec{u} \times \vec{v} = u_x v_y - u_y v_x$ in `int64_t` / `__int128` | $\text{sgn}((B-A) \times (C-A)) \in \{-1, 0, +1\}$ | $O(1)$ exact integer predicate |
| **Angles, Projections, & Acute/Obtuse Tests** | Check whether $\angle ABC < 90^\circ$, $= 90^\circ$, or $> 90^\circ$; find closest point on segment. | **Dot Product** $\vec{u} \cdot \vec{v} = u_x v_x + u_y v_y$ | $\text{sgn}(\vec{u} \cdot \vec{v})$ gives acute/right/obtuse without `acos` | $O(1)$ exact integer predicate |
| **Polygon Area & Lattice Points** | Simple polygon with integer vertices $(x_i, y_i)$; ask for area or interior integer points. | **Shoelace Formula & Pick's Theorem** | $2A = \left\| \sum (p_i \times p_{i+1}) \right\|$; $2A = 2I + B - 2$ where $B = \sum \gcd(\|\Delta x\|, \|\Delta y\|)$ | $O(N)$ exact `int64_t` |
| **Extreme Linear Combinations / Farthest Pair** | Maximize $\alpha x + \beta y$, find minimum bounding box, or maximum distance among $N$ points. | **Convex Hull (Monotone Chain) + Rotating Calipers** | Any linear functional attains its maximum at a hull vertex; unimodal around hull | $O(N \log N)$ sort + $O(N)$ two-pointer sweep |
| **Circular / Ray Queries Around a Pivot** | Count points in a half-plane or wedge through pivot $P$; sort rays by angle without `atan2`. | **Half-Plane + Cross-Product Polar Sort** | Partition into upper ($y > 0 \vee (y = 0 \wedge x > 0)$) and lower half-planes, then sort by $\vec{u} \times \vec{v} > 0$ | $O(N \log N)$ sort + $O(N)$ cyclic two-pointer |



## Worked Problem 54.1: Geothermal Lattice Survey

### Problem & Constraints

A geothermal reserve is bounded by a simple $N$-gon ($3 \le N \le 2 \times 10^5$) whose vertices $P_0, P_1, ..., P_{N-1}$ are given in counterclockwise order with integer coordinates $|x_i|, |y_i| \le 10^9$. A sensor can be placed at any integer lattice point $(x, y) \in \mathbb{Z}^2$ that lies **strictly inside** the polygon (points on the boundary edges or vertices cannot host sensors due to pipeline pipes). Additionally, you are given $Q$ candidate maintenance stations ($1 \le Q \le 2 \times 10^5$) along the first boundary edge $P_0P_1$: for each station $S_q$, report whether the triangle $\triangle P_0P_1S_q$ turns strictly counterclockwise (left of $\overrightarrow{P_0P_1}$), strictly clockwise (right of $\overrightarrow{P_0P_1}$), or lies directly on the segment $P_0P_1$, and compute the total number of valid strictly-interior sensor lattice points of the reserve.

> **Pause & Discover:** Coordinates reach $10^9$, so the bounding box has $4 \times 10^{18}$ lattice points. How can you count the exact number of integer points strictly inside a simple lattice polygon in $O(N)$ time without iterating over the grid, and without using a single floating-point operation?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: We need the cardinality of $\mathcal{I} = \{(x, y) \in \mathbb{Z}^2 \mid (x, y) \in \text{int}(\mathcal{P})\}$, plus for each query $S_q$, the exact relation between $S_q$ and segment $P_0P_1$.
- **Bottleneck**: A scanline or point-in-polygon check over all $(x, y)$ in the bounding box takes $O((\max X - \min X)(\max Y - \min Y)) = O(10^{18})$, which is impossible. Even slicing by $y$-coordinates takes $O(10^9)$. We need an identity that relates discrete lattice point counts directly to vertex coordinates.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Using `double` for area or slope tests)**: For $P_0 = (0, 0)$, $P_1 = (10^9, 10^9 - 1)$, and $S_q = (10^9 - 1, 10^9 - 2)$, the slopes of $P_0P_1$ and $P_0S_q$ are $\frac{10^9-1}{10^9} = 0.999999999$ and $\frac{10^9-2}{10^9-1} = 0.999999998999999999...$. Their difference is $\approx 10^{-18}$, which is smaller than the 53-bit mantissa precision of `double` ($\epsilon \approx 2.2 \times 10^{-16}$)! In `double`, both slopes evaluate to the same floating-point number, falsely classifying a non-collinear point as collinear.
- **Tempting Mistake 2 (Dividing the Shoelace sum by 2 before Pick's Theorem)**: The area $A$ of a lattice polygon can be a half-integer (e.g., a right triangle with legs $1$ and $1$ has $A = 0.5$). Dividing by $2$ in integer arithmetic truncates $0.5 \to 0$, corrupting the interior point count by $1$.

### Step 3 — The Turn: Discovering the Structure

Every required quantity can be expressed in **doubled integer units** ($2A$) and **exact integer GCDs**:

1. **Exact Segment Classification**: For $\vec{u} = P_1 - P_0$ and $\vec{v} = S_q - P_0$, the 2D cross product
$$\vec{u} \times \vec{v} = (x_1 - x_0)(y_q - y_0) - (y_1 - y_0)(x_q - x_0)$$
has magnitude at most $2 \cdot (2 \times 10^9)^2 = 8 \times 10^{18}$, which fits in a signed 64-bit integer (`int64_t` max is $\approx 9.22 \times 10^{18}$, or `__int128` with zero overflow risk).
   - If $\vec{u} \times \vec{v} > 0$: $S_q$ is strictly to the **left** (CCW).
   - If $\vec{u} \times \vec{v} < 0$: $S_q$ is strictly to the **right** (CW).
   - If $\vec{u} \times \vec{v} = 0$: $S_q$ is collinear with line $P_0P_1$. It lies on the **segment** $P_0P_1$ if and only if $(P_0 - S_q) \cdot (P_1 - S_q) \le 0$ (or equivalently $x_q \in [\min(x_0, x_1), \max(x_0, x_1)]$ and $y_q \in [\min(y_0, y_1), \max(y_0, y_1)]$).
2. **Doubled Shoelace Area & Boundary Lattice Points**:
   - By the Shoelace theorem, the **doubled area** $2A = \left| \sum_{i=0}^{N-1} (x_i y_{i+1} - y_i x_{i+1}) \right|$ is always an exact integer.
   - Along each directed edge $\Delta x_i = |x_{i+1} - x_i|$, $\Delta y_i = |y_{i+1} - y_i|$, a lattice point occurs every time the step fraction is an integer, which happens $\gcd(\Delta x_i, \Delta y_i)$ times (counting the start vertex and excluding the end vertex). Summing over all edges gives the exact number of boundary lattice points $B$.
   - By **Pick's Theorem**, $A = I + \frac{B}{2} - 1$. Multiplying by $2$ eliminates all fractions:

**Doubled Integer Pick's & Shoelace Identity:**

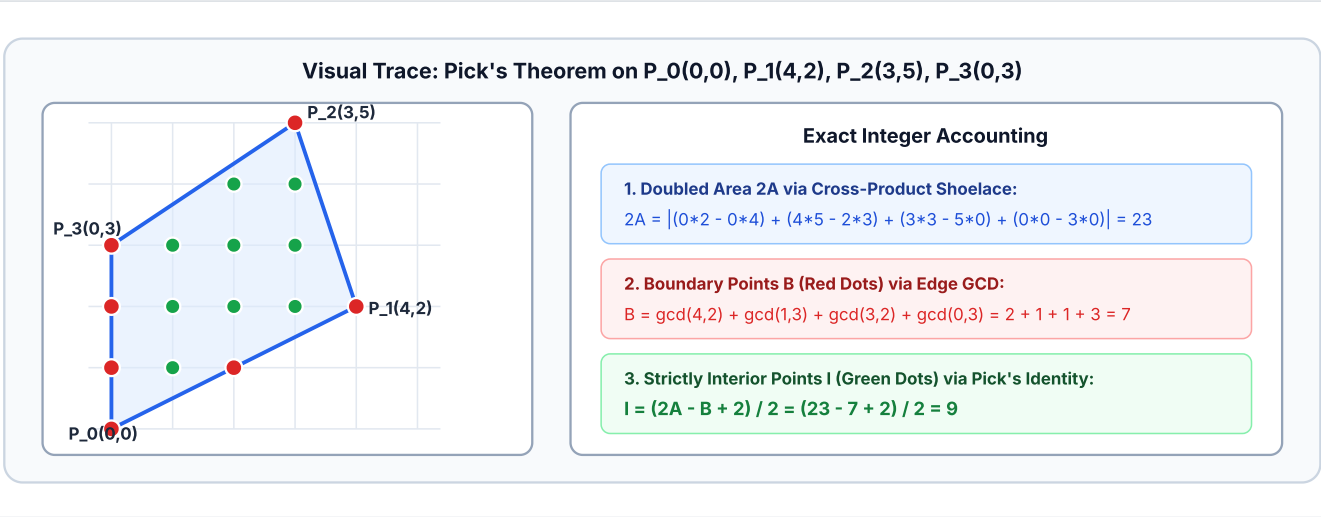
$$2A = \left| \sum_{i=0}^{N-1} \left( x_i y_{(i+1) \bmod N} - y_i x_{(i+1) \bmod N} \right) \right|, \qquad B = \sum_{i=0}^{N-1} \gcd\left( |x_{i+1} - x_i|, |y_{i+1} - y_i| \right)$$

$$I = \frac{2A - B + 2}{2}$$

### Step 4 — Progressive Construction & Visual Trace

Let us trace a concrete lattice quadrilateral with vertices $P_0 = (0,0)$, $P_1 = (4,2)$, $P_2 = (3,5)$, $P_3 = (0,3)$: 1. **Cross-Product Shoelace Sum**: - $P_0 \times P_1 = 0 \cdot 2 - 0 \cdot 4 = 0$ - $P_1 \times P_2 = 4 \cdot 5 - 2 \cdot 3 = 14$ - $P_2 \times P_3 = 3 \cdot 3 - 5 \cdot 0 = 9$ - $P_3 \times P_0 = 0 \cdot 0 - 3 \cdot 0 = 0$ - Total $2A = 0 + 14 + 9 + 0 = 23$ (so $A = 11.5$). 2. **Boundary Lattice Count** $B$: - Edge $P_0P_1$: $\gcd(4,2) = 2$ (points $(0,0), (2,1)$) - Edge $P_1P_2$: $\gcd(1,3) = 1$ (point $(4,2)$) - Edge $P_2P_3$: $\gcd(3,2) = 1$ (point $(3,5)$) - Edge $P_3P_0$: $\gcd(0,3) = 3$ (points $(0,3), (0,2), (0,1)$) - Total $B = 2 + 1 + 1 + 3 = 7$. 3. **Interior Lattice Count** $I$:

$$I = \frac{23 - 7 + 2}{2} = \frac{18}{2} = 9.$$




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof**:
  1. *Why* $B_e = \gcd(\Delta x, \Delta y)$: A point $(x_0 + t\Delta x, y_0 + t\Delta y)$ for $t \in [0,1)$ has integer coordinates iff $t = k/\gcd(\Delta x, \Delta y)$ for $k \in \{0, 1, \ldots, \gcd(\Delta x, \Delta y) - 1\}$.
  2. *Why Pick's Theorem is Additive*: Every simple lattice polygon can be triangulated into primitive lattice triangles (triangles with $I = 0, B = 3$). Each primitive triangle has $2A = 1$, satisfying $2I + B - 2 = 0 + 3 - 2 = 1 = 2A$, and gluing two polygons along an edge of $k$ lattice points adds their areas while turning $k - 2$ boundary points into interior points, preserving $2I + B - 2$.
- **Small-Case Oracle**: For polygons with coordinates in $[-20, 20]$, iterate over every integer grid point $(x, y)$ in the bounding box and test point-in-polygon by ray casting or winding number, verifying the exact match with $(2A - B + 2)/2$.

---

## Worked Problem 54.2: Orbital Relay Baseline

### Problem & Constraints

You are given $N$ relay towers ($3 \le N \le 2 \times 10^5$) on a flat plain with integer coordinates $|x_i|, |y_i| \le 10^9$. Not all towers are collinear. You must select two towers $A, B$ to maximize the **squared Euclidean distance** $\| A - B \|^2 = (x_A - x_B)^2 + (y_A - y_B)^2$, and simultaneously select a third tower $C$ that maximizes the **doubled area** $|(B - A) \times (C - A)|$ of the triangle $\triangle ABC$ formed with a given base edge $AB$ of the outer perimeter.

**Pause & Discover:** Checking all $\binom{N}{2}$ pairs takes $O(N^2)$ time. First, why can no strictly interior point ever belong to the farthest pair? Second, once you extract the Convex Hull of $M$ vertices in counterclockwise order, why does the vertex $P_j$ that is farthest from edge $P_iP_{i+1}$ move only forward (counterclockwise) as $i$ advances?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: Compute $\max_{1 \le i < j \le N} \| P_i - P_j \|^2$.
- **Bottleneck**: $\binom{N}{2} \approx 2 \times 10^{10}$ pairs for $N = 2 \times 10^5$, which is far beyond the 1-second budget. Even after reducing the point set to its Convex Hull, all $N$ points might lie on a circle, so the hull itself can still have $M = N$ vertices! Therefore, an $O(M^2)$ pair check on the hull is still TLE in the worst case; we need an $O(M)$ hull sweep after $O(N \log N)$ sorting.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Binary searching or two-pointer sweeping by Euclidean distance $\| P_i - P_j \|^2$ around the hull)**: It is very tempting to advance pointer $j$ as long as $\| P_i - P_{j+1} \|^2 > \| P_i - P_j \|^2$.
- **Counterexample**: Consider a long, thin kite or isosceles triangle with subtle side bulges. The Euclidean distance $\| P_i - P_j \|^2$ as $j$ walks around the convex hull is **not necessarily unimodal** — it can have two local maxima! If a two-pointer sweep stops at the first local decrease of $\| P_i - P_j \|^2$, it gets stuck at a false local maximum and misses the global diameter.
- **Tempting Mistake 2 (Keeping collinear points on hull edges during Rotating Calipers)**: If you use `< 0` instead of `<= 0` when popping points in Andrew's Monotone Chain, intermediate collinear points remain on the hull edges. Then consecutive hull edges have cross product $0$, which can stall or misalign antipodal pointer advancement.

### Step 3 — The Turn: Discovering the Structure

Why did Euclidean distance fail unimodality, and what quantity **is** guaranteed to be unimodal on a convex polygon? **Perpendicular distance to a directed hull edge $\vec{e}_i = P_{i+1} - P_i$!**

Fix a directed counterclockwise hull edge $\vec{e}_i = P_{i+1} - P_i$. For any hull vertex $P_j$, the doubled area of $\triangle P_iP_{i+1}P_j$ is

$$\text{Area}_2(i, j) = \vec{e}_i \times (P_j - P_i).$$

As $j$ advances counterclockwise around a strictly convex hull, the edge vector $\vec{e}_j = P_{j+1} - P_j$ rotates monotonically by $360^\circ$. Notice that:

$$\text{Area}_2(i, j+1) - \text{Area}_2(i, j) = \vec{e}_i \times (P_{j+1} - P_j) = \vec{e}_i \times \vec{e}_j.$$

Because the hull is convex, the angle from $\vec{e}_i$ to $\vec{e}_j$ increases monotonically from $0^\circ$ to $360^\circ$: - While the angle is in $(0^\circ, 180^\circ)$, $\vec{e}_i \times \vec{e}_j > 0$, so $\text{Area}_2(i, j)$ **strictly increases**. - Once the angle passes $180^\circ$, $\vec{e}_i \times \vec{e}_j < 0$, so $\text{Area}_2(i, j)$ **strictly decreases**.

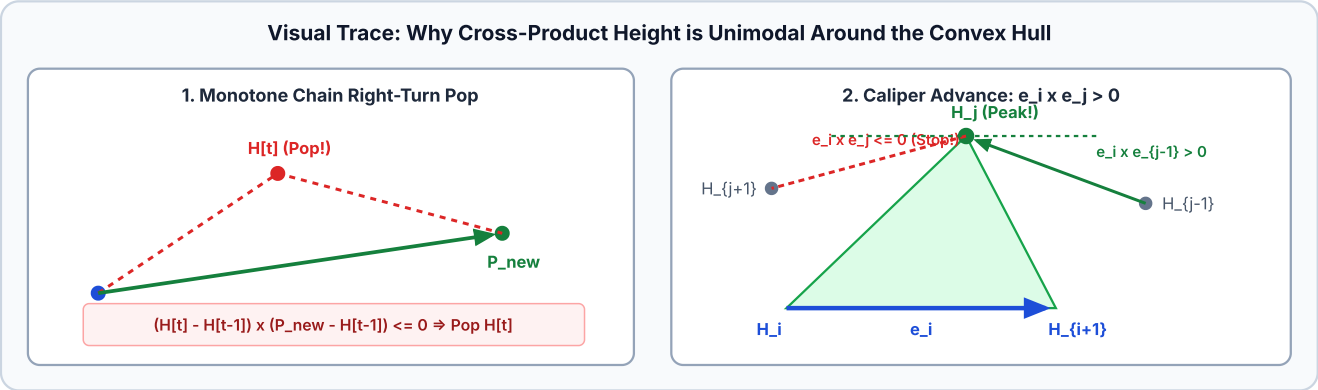
Therefore, $\text{Area}_2(i, j)$ is **strictly unimodal**! Moreover, when $i$ increments to $i + 1$, the base edge $\vec{e}_i$ rotates counterclockwise, so the antipodal vertex $P_j$ where the tangent turns past $180^\circ$ can **only move counterclockwise**. And by a classic theorem of Shamos (1978), the diameter of a convex polygon is always realized by a pair of antipodal vertices visited during this Rotating Calipers sweep!

**Monotone Chain + Rotating Calipers Invariant:**

1. **Strict Convex Hull (Andrew's Monotone Chain)**: Sort points by $(x, y)$. Pop the penultimate hull point $H[\text{top} - 1]$ whenever
$$(H[\text{top}] - H[\text{top} - 1]) \times (P_{\text{new}} - H[\text{top} - 1]) \le 0.$$
2. **Antipodal Two-Pointer Sweep**: For each hull edge $\vec{e}_i = H_{i+1} - H_i$, advance $j \leftarrow (j + 1) \bmod M$ while
$$\vec{e}_i \times (H_{j+1} - H_i) > \vec{e}_i \times (H_j - H_i) \Leftrightarrow \vec{e}_i \times (H_{j+1} - H_j) > 0,$$
and update the diameter candidate with $\| H_i - H_j \|^2$ and $\| H_{i+1} - H_j \|^2$.

### Step 4 — Progressive Construction & Visual Trace

1. **Step 4a (Build Lower and Upper Hulls)**: Sort $N$ points lexicographically by $(x, y)$ and deduplicate. Sweep left-to-right to build the lower hull, and right-to-left to build the upper hull, popping whenever the cross product is $\le 0$. If the resulting hull has $M = 2$ vertices, the answer is simply $\| H_0 - H_1 \|^2$.
2. **Step 4b (Rotating Calipers Sweep)**: Initialize $j = 1$. Loop $i$ from $0$ to $M - 1$. Advance $j$ cyclicly while $\vec{e}_i \times \vec{e}_j > 0$. At the peak $j$, record $\max(\| H_i - H_j \|^2, \| H_{i+1} - H_j \|^2)$ for the diameter and $\vec{e}_i \times (H_j - H_i)$ for the maximum triangle area on base $H_iH_{i+1}$.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Correctness Proof**:
  1. *Hull Sufficiency*: For any point $Q$, the function $f_Q(P) = \| P - Q \|^2$ is convex, and the maximum of a convex function over a compact convex polygon is always attained at an extreme point (vertex of the Convex Hull).
  2. *Antipodal Coverage*: Let $(H_a, H_b)$ be a diameter pair. Draw lines through $H_a$ and $H_b$ perpendicular to segment $H_aH_b$. No point of the hull can lie strictly outside this strip, or its distance to the opposite endpoint would exceed $\| H_a - H_b \|$. Rotating both parallel supporting lines until one coincides with a hull edge $H_iH_{i+1}$ shows that the other supporting line still passes through the opposite endpoint $H_j$, which is precisely the vertex maximizing $\vec{e}_i \times (H_j - H_i)$. Since pointer $j$ only increments ($O(M)$ increments total across all $M$ edges), the sweep runs in $O(M)$ time.
- **Small-Case Oracle**: Generate $1000$ random test cases with $N = 30$, including degenerate clusters with many collinear points on bounding box edges and thin triangles. Compare the Monotone Chain + Rotating Calipers diameter against the exact $O(N^2)$ all-pairs loop.

---

# Section 55 — Randomization and Hashing

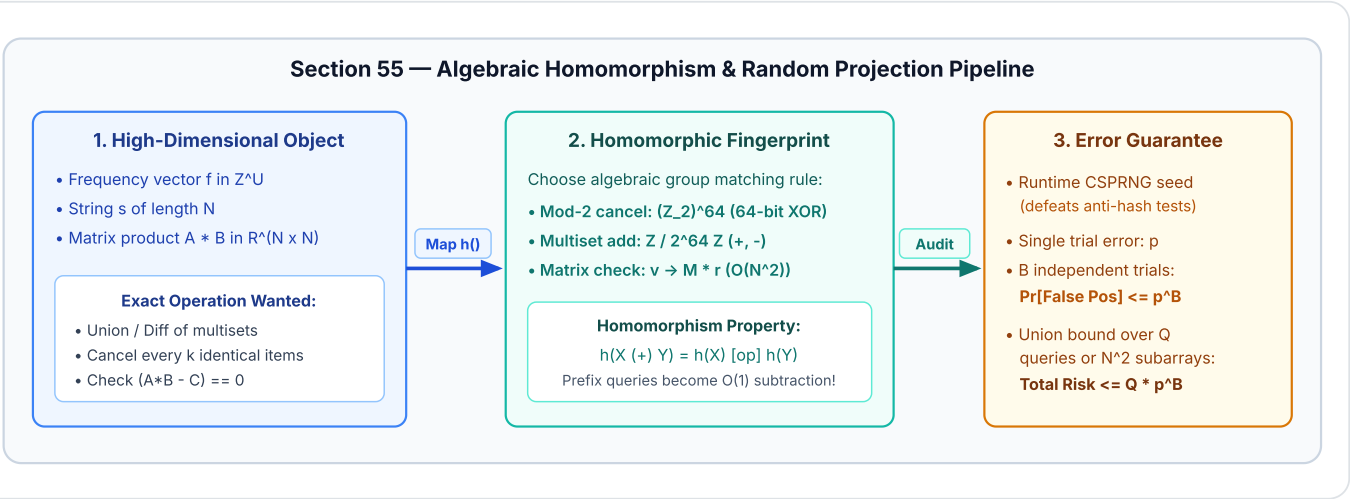
## The Discovery & Modeling Approach

Deterministic algorithms pay a worst-case price to maintain exact high-dimensional structures: comparing two multisets of $N$ elements takes $O(N)$, checking whether every element frequency in a range is divisible by $k$ takes $O(\text{alphabet})$, and multiplying two $N \times N$ matrices takes $O(N^3)$.

**Randomization and Hashing** compress a high-dimensional equality check $\overrightarrow{A} \stackrel{?}{=} \overrightarrow{B}$ into a **1D random algebraic projection** $h(\overrightarrow{A}) \stackrel{?}{=} h(\overrightarrow{B})$. If $\overrightarrow{A} = \overrightarrow{B}$, the projections are *always* equal. If $\overrightarrow{A} \neq \overrightarrow{B}$, their difference $\overrightarrow{A} - \overrightarrow{B} \neq \vec{0}$ is a non-zero vector or polynomial, and by the **Schwartz-Zippel Lemma**, a random evaluation point hits a root of $\overrightarrow{A} - \overrightarrow{B}$ with tiny probability ( $\leq d/|\mathbb{F}|$ or $\leq 2^{-B}$).

### Decision Compass & Structural Lenses

| Structural Equality Target | Deterministic Bottleneck | Algebraic Fingerprint / Projection | Collision Bound (Per Check) | Union Bound Budget ($Q$ checks) |
|---|---|---|---|---|
| **Multiset / Set Equality** ( $S_1 \stackrel{?}{=} S_2$) | Sorting or frequency maps cost $O(N)$ per comparison | **Zobrist Sum / Polynomial Value**: $h(S) = \sum_{x \in S} r(x) \bmod 2^{64}$ | $\leq 2^{-64}$ for random 64-bit `uint64_t` $r(x)$ | Safe up to $Q \approx 10^7$ ($\frac{Q^2}{2^{64}}$ or $\frac{Q}{2^{64}}$) |
| **Parity / Modulo-$k$ Frequency** (every count $\equiv 0 \pmod k$) | Tracking $U$ counters in prefix sums costs $O(N \cdot U)$ | **Group-Homomorphic Zobrist**: $k = 2$: 64-bit XOR; general $k$: base-$k$ vector or random sum | $2^{-64}$ ($k = 2$) or $k^{-B}$ ($B$ independent coords) | Prefix hashes match: $\text{pref}[R] = \text{pref}[L-1]$ |
| **Substring / Sequence Isomorphism** | $O(L)$ character-by-character scan per query | **Polynomial Rolling Hash**: $\sum s_i B^{L-1-i} \bmod M$ with random base $B \in [256, M-2]$ | $\leq \frac{L}{M}$ by Schwartz-Zippel | Double modulus ( $M_1, M_2 \approx 10^9$) $\implies \leq \frac{L}{10^{18}}$ |
| **Matrix / Linear Operator Identity** ($A \cdot B \stackrel{?}{=} C$) | $O(N^3)$ matrix-matrix multiplication | **Freivalds' Random Projection**: Pick $\vec{r} \in \{0,1\}^N$, check $A(B\vec{r}) \stackrel{?}{=} C\vec{r}$ in $O(N^2)$ | $\leq \frac{1}{2}$ per trial over $\{0,1\}^N$ (or $\frac{1}{p}$ over $\mathbb{F}_p$) | $T = 40$ trials $\implies 2^{-40} < 10^{-12}$ failure |
| **Dense Majority / Heavy Hitters** ( $\geq N/k$ frequency) | Full scan or sorting when only few queries allowed | **Monte Carlo Sampling**: Sample $S$ random indices and test candidates exactly | $(1-1/k)^S \leq e^{-S/k}$ | $S = 60k \implies e^{-60} < 10^{-26}$ |



## Worked Problem 55.1: Triadic Packet Audit

### Problem & Constraints

A network router logs a stream of $N$ packet IDs $A_1, A_2, ..., A_N$ ($1 \le N \le 3 \times 10^5$, $1 \le A_i \le 10^6$). Under the **Triadic Redundancy Protocol**, a contiguous subarray $A[L..R]$ is valid if and only if **every packet ID that appears in $A[L..R]$ appears a number of times divisible by** $3$ (i.e., $0, 3, 6, 9, ...$ times). In addition, you are given $Q$ queries ($1 \le Q \le 3 \times 10^5$) asking: 1. Whether subarray $A[L..R]$ is a valid Triadic block, and 2. Count the total number of valid Triadic subarrays $(L, R)$ across the entire stream $1 \le L \le R \le N$.

> **Pause & Discover:** If the protocol required every frequency to be divisible by $2$ (even), we could assign each packet ID a random 64-bit integer and XOR them: two equal values cancel because $v \oplus v = 0$. How do you design a 64-bit random hash where three identical values automatically cancel to $0$, so that any valid subarray has hash $0$ and equal prefix hashes identify valid subarrays?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: For each item $x \in [1, U]$ ($U = 10^6$), let $\mathrm{cnt}_x(r)$ be the frequency of $x$ in the prefix $A[1..r]$ modulo $3$. A subarray $A[L..R]$ has all frequencies divisible by $3$ if and only if the frequency vector modulo $3$ satisfies:
$$\overrightarrow{\mathrm{cnt}}(R) \equiv \overrightarrow{\mathrm{cnt}}(L-1) \pmod 3.$$
- **Bottleneck**: The vector $\overrightarrow{\mathrm{cnt}}(r) \in \mathbb{Z}_3^U$ has $U = 10^6$ coordinates! Storing or hashing a $10^6$-dimensional vector at each of the $N = 3 \times 10^5$ prefixes takes $O(N \cdot U) = 3 \times 10^{11}$ operations. We must compress the $U$-dimensional vector $\overrightarrow{\mathrm{cnt}}(r) \in \mathbb{Z}_3^U$ into $O(1)$ 64-bit integers.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (XORing random 64-bit integers)**: Standard Zobrist XOR hashing satisfies $r(x) \oplus r(x) = 0$, which works Modulo $2$, not Modulo $3$! Under XOR, $r(x) \oplus r(x) \oplus r(x) = r(x) \neq 0$.
- **Tempting Mistake 2 (Adding random integers $r(x)$ and checking if the sum is divisible by $3$)**: If $A[L..R] = [x, y, z]$ with distinct IDs whose random values happen to sum to a multiple of $3$ (which happens with probability $1/3$!), a single modulo-3 test gives a $33\%$ false-positive rate. Even worse, you cannot count pairs $(L-1, R)$ with $\mathrm{pref}[R] = \mathrm{pref}[L-1]$ if the hash does not reset to $0$ after $3$ occurrences of $x$.
- **Tempting Mistake 3 (Simply adding $+r(x)$ on the 1st and 2nd occurrence of $x$ in the global prefix, and $-2r(x)$ on the 3rd occurrence)**: Wait — why does this work?! Let's check carefully! Suppose globally $x$ appears at indices $i_1, i_2, i_3, i_4, ...$. If we set the step delta at occurrence $m$ of $x$ to be:
$$\Delta(x, m) = \begin{cases} +r(x) & \text{if } m \not\equiv 0 \pmod 3, \\ -2r(x) & \text{if } m \equiv 0 \pmod 3, \end{cases}$$
then the prefix sum contribution of $x$ after $m$ occurrences is $(m \bmod 3) \cdot r(x) \pmod{2^{64}}$! Let's verify: does $\mathrm{pref}[R] = \mathrm{pref}[L-1]$ iff every element's count in $[L, R]$ is $0 \bmod 3$? In prefix $R$, element $x$ contributes $(c_x(R) \bmod 3) \cdot r(x)$. In prefix $L-1$, element $x$ contributes $(c_x(L-1) \bmod 3) \cdot r(x)$. They are equal if and only if $c_x(R) \equiv c_x(L-1) \pmod 3 \Leftrightarrow c_x(R) - c_x(L-1) \equiv 0 \pmod 3$ !

### Step 3 — The Turn: Discovering the Structure

We have two equally clean, isomorphic ways to build a Modulo-$k$ Zobrist Hash:

1. **Method A (Remainder-Weighted Zobrist Sum in $\mathbb{Z}/2^{64}\mathbb{Z}$)**: Assign each distinct value $x$ two independent 64-bit random numbers $R_1(x), R_2(x)$ (or a single 128-bit random integer $R(x)$). At prefix $r$, let $rem_x(r) = \mathrm{cnt}_x(r) \bmod 3 \in \{0, 1, 2\}$. Maintain the prefix fingerprint:
$$H(r) = \sum_{x=1}^{U} (\ \mathrm{cnt}_x(r) \bmod 3\ ) \cdot R(x) \pmod{2^{64}}.$$
When $A_r = x$ arrives, $\mathrm{cnt}_x$ increments by $1$. If $\mathrm{cnt}_x(r) \bmod 3 \neq 0$, $H$ increases by $+R(x)$; if $\mathrm{cnt}_x(r) \bmod 3 = 0$, $H$ decreases by $-2R(x)$ (returning $x$'s contribution to $0$!).

2. **Method B (Bitwise Trinary XOR $\oplus_3$ on pairs of 64-bit words)**: Represent a 64-digit base-3 vector as two `uint64_t` bitmasks $(\mathrm{lo}, \mathrm{hi})$ where bit $b$ stores $(0,0)$ for $0$, $(1,0)$ for $1$, and $(0,1)$ for $2$. Then componentwise addition modulo $3$ of $64$ independent coordinates in $\mathbb{Z}_3^{64}$ runs in $6$ bitwise operations!

**Modulo-$k$ Zobrist Prefix Hash Invariant:**

Assign each label $x$ a uniform 64-bit random integer $R(x) \in [0, 2^{64}-1]$ (or a pair $(R_1(x), R_2(x))$ for 128-bit security). Define:

$$H(r) = \sum_x \left( \operatorname{cnt}_x(r) \bmod k \right) \cdot R(x) \pmod{2^{64}}.$$

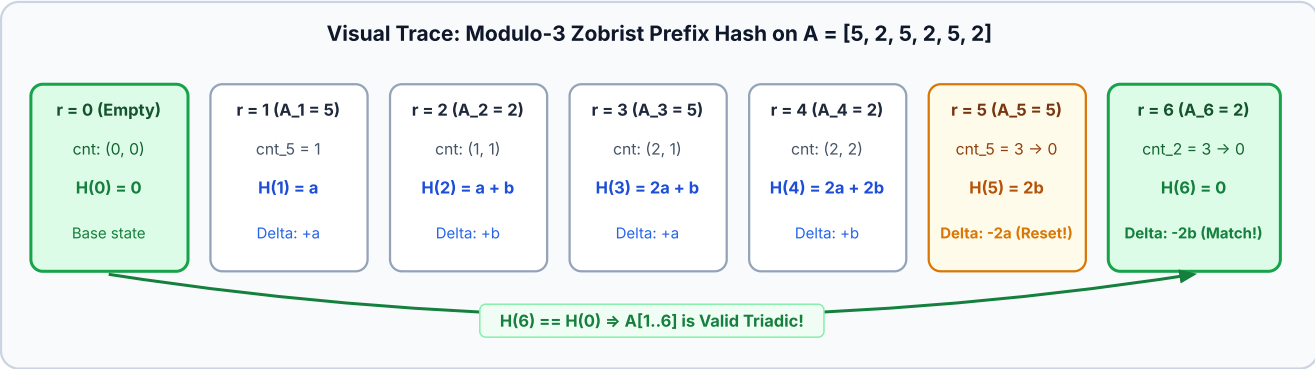
Incremental update at index $r$ with value $x = A_r$:

$$H(r) = H(r-1) + \begin{cases} R(x) & \text{if } \operatorname{cnt}_x(r) \not\equiv 0 \pmod k \\ -(k-1)R(x) & \text{if } \operatorname{cnt}_x(r) \equiv 0 \pmod k \end{cases} \pmod{2^{64}}.$$

Then $A[L..R]$ has all frequencies divisible by $k \Leftrightarrow H(R) = H(L-1)$ (up to collision probability $\leq 2^{-63}$ per word).

### Step 4 — Progressive Construction & Visual Trace

Let us trace $k = 3$ on the array $A = [5, 2, 5, 2, 5, 2]$ ($N = 6$), where $R(5) = \alpha$ and $R(2) = \beta$: - $r = 0$: $H(0) = 0$. - $r = 1$ ($A_1 = 5$, $\operatorname{cnt}_5 = 1$): $H(1) = \alpha$. - $r = 2$ ($A_2 = 2$, $\operatorname{cnt}_2 = 1$): $H(2) = \alpha + \beta$. - $r = 3$ ($A_3 = 5$, $\operatorname{cnt}_5 = 2$): $H(3) = 2\alpha + \beta$. - $r = 4$ ($A_4 = 2$, $\operatorname{cnt}_2 = 2$): $H(4) = 2\alpha + 2\beta$. - $r = 5$ ($A_5 = 5$, $\operatorname{cnt}_5 = 3 \equiv 0$): subtract $2\alpha \Longrightarrow H(5) = 2\beta$. - $r = 6$ ($A_6 = 2$, $\operatorname{cnt}_2 = 3 \equiv 0$): subtract $2\beta \Longrightarrow H(6) = 0$. Notice $H(6) = H(0) = 0$, immediately detecting that $A[1..6]$ has three 5s and three 2s! To count all valid subarrays across $A$, we simply insert $H(0), H(1), \ldots, H(N)$ into a hash map and sum $\binom{\text{freq}(h)}{2}$.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Collision Probability Proof**: Suppose $A[L..R]$ is *not* valid. Then there is at least one value $x^*$ whose count in $A[L..R]$ satisfies $d_{x^*} = (\operatorname{cnt}_{x^*}(R) \bmod 3) - (\operatorname{cnt}_{x^*}(L-1) \bmod 3) \in \{\pm 1, \pm 2\}$. Condition on all random assignments $R(y)$ for $y \neq x^*$. The false-positive equation is:

  $$d_{x^*} \cdot R(x^*) \equiv -\sum_{y \neq x^*} d_y \cdot R(y) \pmod{2^{64}}.$$

  Since $\gcd(|d_{x^*}|, 2^{64}) \in \{1, 2\}$, at most 2 values of $R(x^*) \in [0, 2^{64}-1]$ can satisfy this congruence! Thus any invalid pair $(L-1, R)$ collides with probability at most $2/2^{64} = 2^{-63}$. Over all $\binom{N+1}{2} \approx 4.5 \times 10^{10}$ pairs, the expected number of collisions is $\leq 4.5 \times 10^{10}/2^{63} < 5 \times 10^{-9}$ (or $< 10^{-27}$ if we use a pair of 64-bit hashes).
- **Small-Case Oracle**: For $N \leq 200$, brute-force all $\binom{N}{2}$ subarrays by maintaining an exact count array and counter of non-zero mod-3 frequencies, and assert equality with the Zobrist prefix hash solver.

---

## Worked Problem 55.2: Optical Tensor Verification

### Problem & Constraints

An optical co-processor claims to have computed the matrix product $C = A \cdot B \pmod{10^9+7}$ for two $N \times N$ matrices $A$ and $B$ ($1 \leq N \leq 1200$). However, single-event cosmic ray upsets may have corrupted one or more entries of $C$. Given $A, B,$ and $C$, determine whether $A \cdot B \equiv C \pmod{10^9+7}$, and if not, identify the exact $(r, c)$ coordinates of the corrupted entry under the promise that when corrupted, at most $K \leq 5$ entries in $C$ differ from $A \cdot B$ (first decide whether $A \cdot B = C$ globally within a 0.5-second limit).

**Pause & Discover:** Multiplying $A \cdot B$ takes $N^3 \approx 1.7 \times 10^9$ operations, which exceeds the time limit. Multiplication of an $N \times N$ matrix by an $N \times 1$ column vector $\vec{v}$ takes only $O(N^2)$ operations! How can you rewrite $(A \cdot B)\vec{v}$ so that it requires only matrix-vector products and never forms $A \cdot B$?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: Compute the difference matrix $D = A \cdot B - C \pmod P$ ($P = 10^9 + 7$) and check whether every entry $D_{i,j} = 0$.
- **Bottleneck**: Computing $A \cdot B$ explicitly takes $N^3 = 1.728 \times 10^9$ multiplications. At $N = 1200$, even cache-friendly blocked matrix multiplication with modular reductions struggles to finish under $0.5$ seconds. Yet reading the input already takes $O(N^2)$ operations ($1.44 \times 10^6$ integers). Can we verify $A \cdot B = C$ in $O(N^2)$ time?

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Checking the all-ones vector $\vec{v} = (1, 1, ..., 1)^T$ or row/column sums)**: Checking $A(B\vec{1}) = C\vec{1}$ only verifies that each row of $C$ has the correct sum!
- **Counterexample**: Suppose $C$ was corrupted by adding $+1$ to $C_{1,1}$ and $-1$ to $C_{1,2}$. Then the difference matrix $D = AB - C$ has row 1 equal to $(-1, +1, 0, ..., 0)$, and $D \cdot \vec{1} = (-1)(1) + (+1)(1) = 0$. Any fixed deterministic vector $\vec{v}$ has an $(N-1)$-dimensional nullspace of invisible corruptions!
- **Tempting Mistake 2 (Sampling $O(N^2)$ random individual entries $C_{i,j}$)**: Each entry $C_{i,j} = \sum_{k=1}^{N} A_{i,k}B_{k,j}$ takes $O(N)$ to verify. If only **one** entry of $C$ is corrupted, a random entry hits it with probability only $1/N^2$. To find it with high probability by entry sampling would require $\Omega(N^2)$ samples $\times O(N)$ work $= \Omega(N^3)$ time!

### Step 3 — The Turn: Discovering the Structure

Instead of testing one entry at a time, test a **random linear combination of all columns simultaneously** (**Freivalds' Algorithm**): 1. Pick a random column vector $\vec{r} \in \mathbb{F}_P^N$, where each coordinate $r_j$ is chosen independently and uniformly from $\{0, 1, ..., P-1\}$ (or even $\{0, 1\}$). 2. By associativity of matrix multiplication:

$$(A \cdot B)\vec{r} = A \cdot (B\vec{r}).$$

Compute $\vec{u} = B\vec{r}$ in $O(N^2)$ time, then $\vec{x} = A\vec{u}$ in $O(N^2)$ time, and compare against $\vec{y} = C\vec{r}$ in $O(N^2)$ time!

What if $D = AB - C \neq 0$? Suppose row $i$ of $D$ has at least one non-zero entry $D_{i,j^*} \neq 0 \pmod P$. Then the $i$-th component of $D\vec{r}$ is:

$$(D\vec{r})_i = D_{i,j^*}r_{j^*} + \sum_{j \neq j^*} D_{i,j}r_j \equiv 0 \pmod P \Leftrightarrow r_{j^*} \equiv -D_{i,j^*}^{-1} \sum_{j \neq j^*} D_{i,j}r_j \pmod P.$$

Conditioned on all $r_j$ ($j \neq j^*$), there is **at most** $1$ **value** of $r_{j^*} \in \mathbb{F}_P$ that makes $(D\vec{r})_i \equiv 0 \pmod P$!

**Freivalds' Projection & Monte Carlo Amplification:**

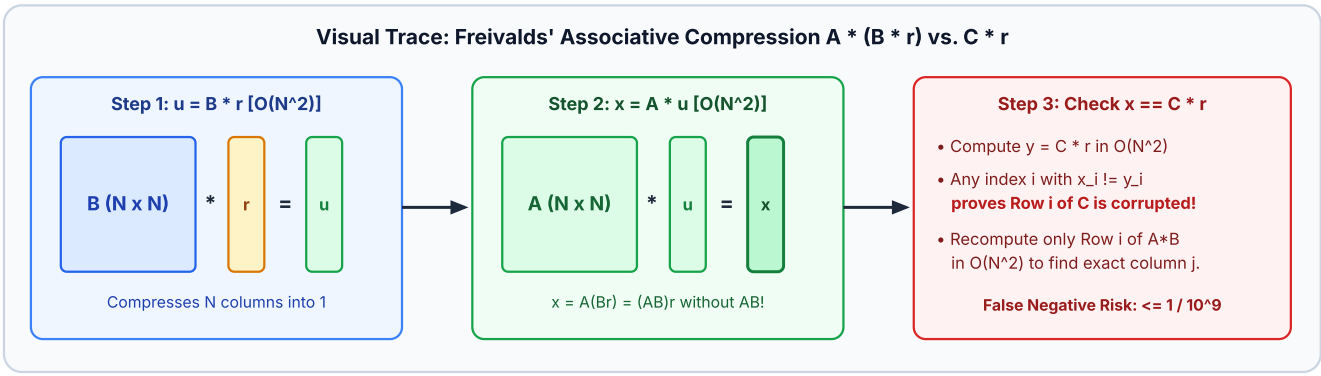
$$\Pr_{\vec{r} \sim \mathbb{F}_P^N}\left[A(B\vec{r}) \equiv C\vec{r} \pmod P \;\middle|\; AB \not\equiv C \pmod P\right] \leq \frac{1}{P} < 10^{-9}.$$

Repeating with $T = 2$ independent random vectors over $\mathbb{F}_P$ (or $T = 40$ binary vectors over $\{0, 1\}^N$) reduces the false-positive probability to $\leq P^{-2} < 10^{-18}$ (or $2^{-40}$) in $O(T \cdot N^2)$ time.

Better yet: if $D_{i,j^*} \neq 0$, then $(A(B\vec{r}) - C\vec{r})_i \neq 0$ immediately tells us **which exact rows $i$ of $C$ are corrupted**! For each corrupted row $i$ (there are at most $K \leq 5$), we can compute the entire $i$-th row of $A \cdot B$ in $O(N^2)$ time to pinpoint the exact corrupted columns $(i, j)$!

### Step 4 — Progressive Construction & Visual Trace

Let us trace how associativity collapses the $N \times N \times N$ cube into three $N \times N \times 1$ slabs:




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof**:
  1. *Completeness (One-Sided Error)*: If $AB \equiv C \pmod P$, then by associativity of matrix multiplication over the field $\mathbb{F}_P$, $A(B\vec{r}) = (AB)\vec{r} = C\vec{r}$ holds for **every** vector $\vec{r}$ with probability $1$.
  2. *Soundness*: If $AB \not\equiv C \pmod P$, each corrupted row $i$ is a non-zero linear form $L_i(\vec{r}) = \sum_{j=1}^{N} (AB - C)_{i,j} r_j$ over $\mathbb{F}_P$, whose kernel is an $(N-1)$-dimensional hyperplane of size $P^{N-1}$ inside $\mathbb{F}_P^N$ (fraction $1/P$).
- **Small-Case Oracle**: For $N = 20$, compute the exact $O(N^3)$ product $AB \pmod P$, inject $1$ to $5$ adversarial zero-sum row corruptions (such as $+v$ at $(i, j_1)$ and $-v$ at $(i, j_2)$), and verify that Freivalds' algorithm catches every corrupted row across $10^4$ trials.

---

# Section 56 — Hard Greedy and Matroids Intro

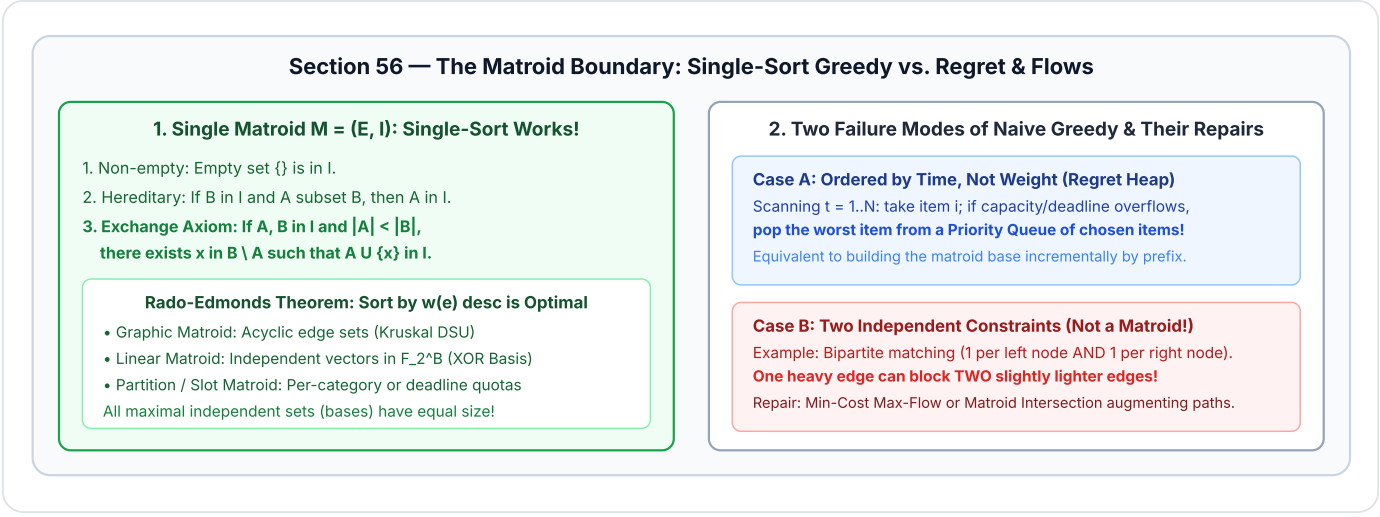
## The Discovery & Modeling Approach

Simple greedy algorithms sort items by weight and take every compatible item. In elementary problems (like Interval Scheduling or Minimum Spanning Tree), a quick 2-element swap proves correctness. In **Hard Greedy** problems, however, you face one of two deeper questions:

1. **When is "Sort Descending and Take Whenever Feasible" Guaranteed to Work?** Why does Kruskal's algorithm work for forests, and why does greedy Gaussian elimination work for maximum-weight XOR basis, whereas the *exact same* greedy algorithm fails on maximum-weight bipartite matching or directed spanning trees? The universal dividing line is the **Matroid Exchange Property** (Rado–Edmonds Theorem).
2. **When Simple One-Pass Greedy Fails on Sequential Capacity/Time Constraints, How Do We Repair It?** When decisions arrive chronologically (e.g., deadlines, running budgets, battery capacities) and taking an item now may block a better configuration later, we replace "blind greedy commitment" with **Greedy with Regret (Priority-Queue Augmentation)**: tentatively accept items or greedily commit, and when a capacity constraint is violated, retroactively evict the worst previously chosen item in $O(\log N)$ time.

### Decision Compass & Structural Lenses

| Constraint Family | Ground Set $E$ & Independent Sets $\mathcal{I}$ | Is $(E, \mathcal{I})$ a Single Matroid? | Optimal Algorithm | Complexity |
|---|---|---|---|---|
| **Graphic Matroid** | \$E = \$ undirected edges; $I \in \mathcal{I} \Leftrightarrow I$ contains no cycle (is a forest). | **Yes** | Sort by weight, add edge if `dsu.unite(u, v)` succeeds (Kruskal). | $O(M \log M)$ |
| **Linear / XOR-Basis Matroid** | \$E = \$ bitvectors in $\mathbb{F}_2^B$; $I \in \mathcal{I} \Leftrightarrow$ vectors in $I$ are linearly independent under XOR. | **Yes** | Sort by weight descending, insert into $B$-bit XOR basis if non-zero after reduction. | $O(N \log N + N \cdot B)$ |
| **Slot / Transversal / Matching Matroid** | \$E = \$ jobs with deadline $d_i$; $I \in \mathcal{I} \Leftrightarrow$ jobs in $I$ can all finish before their deadlines. | **Yes** | Sort by profit descending + DSU slot assignment, **or** sort by deadline + min-heap regret eviction. | $O(N \log N)$ |
| **Intersection of Two Matroids** | e.g., Bipartite Matching (degree $\leq 1$ in $L$ **and** degree $\leq 1$ in $R$), or Arborescence / Rainbow Forest. | **No** (Intersection of $2$ matroids; rank quotient $1/2$ ) | Single-pass greedy **fails**! Use Augmenting Paths (Matching / Min-Cost Flow / Matroid Intersection). | Polynomial, not single-sort |
| **Sequential Budget with Retroactive Swap** | Prefix sum $\sum_{j \leq i} x_j \geq 0$ at every step $i$, maximize count or reward. | **Yes** (Dual of slot matroid / discrete concave) | **Exchange-with-Regret**: Maintain max-heap of chosen costs; if budget $< 0$ , swap out largest cost. | $O(N \log N)$ |




## Worked Problem 56.1: Orthogonal Telemetry Basis

### Problem & Constraints

A deep-space probe has $N$ candidate telemetry channels ($1 \le N \le 2 \times 10^5$). Channel $i$ has a binary frequency signature $s_i \in [1, 2^{60} - 1]$ (a 60-bit integer) and a bandwidth reward $w_i \ge 1$ ($1 \le w_i \le 10^9$). A subset of channels $S$ is **interference-free** if no non-empty subset of signatures in $S$ has bitwise XOR sum equal to $0$. Find the maximum possible total bandwidth reward $\sum_{i \in S} w_i$ of an interference-free subset $S$.

> **Pause & Discover:** There are $2^N$ subsets, and $60$ bits is far too wide for a bitmask DP of size $2^{60}$. Why is the condition "no non-empty subset XORs to $0$" identical to linear independence over the finite field $\mathbb{F}_2^{60}$? And why does that guarantee that sorting channels by $w_i$ descending and greedily inserting them into a 60-row Gaussian XOR basis is $100\%$ optimal?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: View each signature $s_i$ as a 60-dimensional vector over $\mathbb{F}_2 = \{0, 1\}$, where vector addition is bitwise XOR ($\oplus$). A subset $S$ has no non-empty XOR sum equal to $0$ if and only if the vectors $\{s_i \mid i \in S\}$ are **linearly independent** over $\mathbb{F}_2$. We want a linearly independent subset $S$ maximizing $\sum_{i \in S} w_i$.
- **Bottleneck**: Any subset of $\mathbb{F}_2^{60}$ of size $> 60$ is linearly dependent, so $|S| \le 60$. However, checking all $\binom{N}{60}$ subsets is impossible, and DP over the subspace spanned so far would have as many states as there are subspaces of $\mathbb{F}_2^{60}$ ($> 2^{900}$). We need a proof that **no backtracking is ever required**.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Believing that taking a heavy vector $s_1 = a \oplus b$ might unfairly block *both* $a$ and $b$ simultaneously)**: In bipartite matching, picking one edge of weight $10$ can block two edges of weight $9$ ($10 < 9 + 9$). Why can't picking one vector $s_1 = a \oplus b$ of weight $10$ block *both* $a$ (weight $9$) and $b$ (weight $9$)?
- **Resolution of the Paradox**: Suppose we first pick $s_1 = a \oplus b$ (weight $10$). When we next consider $a$ (weight $9$), is $a$ blocked by $s_1$? **No!** The set $\{a \oplus b, a\}$ has size $2$ and spans the exact same 2D subspace $\text{span}(a, b)$ as $\{a, b\}$! Only the *third* vector $b$ becomes redundant because $b = (a \oplus b) \oplus a$, so we get $10 + 9 = 19 > 9 + 9 = 18$! One vector can increase the dimension of our span by at most $1$, so it can never reduce the number of vectors we can take from another independent set by more than $1$.

### Step 3 — The Turn: Discovering the Structure

Let $E = \{1, \ldots, N\}$ and $\mathcal{I} = \{S \subseteq E \mid \{s_i\}_{i \in S} \text{ is linearly independent over } \mathbb{F}_2\}$. Then $(E, \mathcal{I})$ is a **Linear Matroid**: 1. $\emptyset \in \mathcal{I}$. 2. Every subset of a linearly independent set is linearly independent (Hereditary). 3. **Steinitz Exchange Lemma**: If $A, B \in \mathcal{I}$ with $|A| < |B|$, then $\dim(\text{span}(A)) = |A| < |B| = \dim(\text{span}(B))$, so $B$ cannot be contained in $\text{span}(A)$. Thus there exists $x \in B \setminus A$ such that $x \notin \text{span}(A)$, meaning $A \cup \{x\} \in \mathcal{I}$!

By the **Rado–Edmonds Theorem**, the single-pass greedy algorithm (sort by $w_i$ descending, add $s_i$ if $s_i \notin \text{span}(S)$) is guaranteed to find the maximum-weight independent set! To test $s_i \notin \text{span}(S)$ in $O(B)$ time ($B = 60$), maintain an **upper-triangular XOR basis** `basis[0..59]`, where `basis[b]` holds a basis vector whose highest set bit (MSB) is $b$.

**Greedy XOR-Basis Matroid Algorithm:**

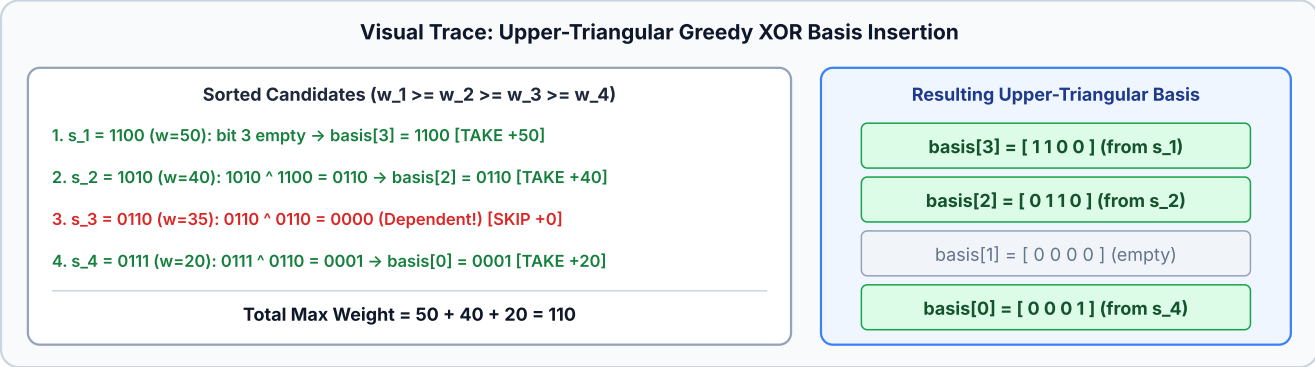
Sort channels so $w_1 \geq w_2 \geq \ldots \geq w_N$. For each $x = s_i$, iterate $b = 59$ down to $0$:

$$\text{if } (x \gg b)\,\&\,1\text{:} \quad \begin{cases} \text{if basis}[b] = 0\text{:} \;\; \text{basis}[b] \leftarrow x, \;\; \text{Ans} \mathrel{+}= w_i, \;\; \textbf{break} \\ \text{else:} \;\; x \leftarrow x \oplus \text{basis}[b] \end{cases}$$

If $x$ reduces to $0$, then $s_i \in \text{span}(S)$ and is skipped; otherwise $s_i$ is linearly independent and extends the basis.

### Step 4 — Progressive Construction & Visual Trace

Let us trace a 4-bit instance with 5 channels sorted by weight descending: 1. $c_1$: $s_1 = 1100_2(12)$, $w_1 = 50 \Longrightarrow \text{MSB} = 3$, `basis[3] = 1100`, **Taken (+50)**. 2. $c_2$: $s_2 = 1010_2(10)$, $w_2 = 40 \Longrightarrow b = 3$: $1010 \oplus 1100 = 0110_2 \Longrightarrow \text{MSB} = 2$, `basis[2] = 0110`, **Taken (+40)**. 3. $c_3$: $s_3 = 0110_2(6)$, $w_3 = 35 \Longrightarrow b = 2$: $0110 \oplus 0110 = 0000_2 \Longrightarrow$ reduces to $0$ ($s_3 = s_1 \oplus s_2$), **Rejected**. 4. $c_4$: $s_4 = 0111_2(7)$, $w_4 = 20 \Longrightarrow b = 2$: $0111 \oplus 0110 = 0001_2 \Longrightarrow \text{MSB} = 0$, `basis[0] = 0001`, **Taken (+20)**.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Matroid Exchange Proof of Greedy Optimality**: Let $G = \{g_1, g_2, \ldots, g_r\}$ be the greedy independent set indexed in the order added ($w(g_1) \geq w(g_2) \geq \ldots \geq w(g_r)$), and let $O = \{o_1, o_2, \ldots, o_k\}$ be any optimal independent set sorted so $w(o_1) \geq w(o_2) \geq \ldots \geq w(o_k)$. Since $G$ is a maximal independent set (basis), $|O| \leq |G|$. Suppose for contradiction that $w(g_j) < w(o_j)$ for some first index $j$. Consider the two independent sets $A = \{g_1, \ldots, g_{j-1}\}$ (size $j-1$) and $B = \{o_1, \ldots, o_j\}$ (size $j$). By the Matroid Exchange Axiom, there exists $x \in B \backslash A$ such that $A \cup \{x\} \in \mathcal{I}$. Since $x \in B$, $w(x) \geq w(o_j) > w(g_j)$. Thus $x$ was considered by the greedy algorithm *before* $g_j$, and since $A \cup \{x\} \in \mathcal{I}$, $x$ was independent of the current greedy prefix at that moment — contradicting the fact that greedy rejected $x$! Hence $w(g_j) \geq w(o_j)$ for every $j$, so $w(G) \geq w(O)$.
- **Small-Case Oracle**: For $N \leq 18$, test all $2^N$ subsets, check linear independence by rank, and compare the maximum weight against the $O(N \log N + 60N)$ greedy XOR basis algorithm.

---

## Worked Problem 56.2: Chronological Reactor Cooling (Exchange with Regret)

### Problem & Constraints

A reactor runs for $N$ hours ($1 \leq N \leq 2 \times 10^5$). At hour $i$ ($1 \leq i \leq N$), cooling rod $i$ becomes available, which has a thermal absorption value $v_i > 0$ ($1 \leq v_i \leq 10^9$) and an expiration deadline $d_i \in [1, N]$: if used, rod $i$ must be activated during some integer hour slot $t \in \{1, 2, \ldots, d_i\}$, and at most one rod can be activated per hour slot. Additionally, contrast this with a **Dual-Grid Coupling** variant where each rod $i$ simultaneously occupies row $r_i$ and column $c_i$ of a $K \times K$ grid and at most one rod may be placed in each row **and** each column. 1. Solve the deadline scheduling problem in $O(N \log N)$ time using chronological **Exchange with Regret**. 2. Prove by explicit counterexample why the Dual-Grid Coupling variant is the intersection of **two** matroids where single-pass weight greedy fails.

**Pause & Discover:** If you sort rods by deadline $d_i$ ascending and scan $i = 1...N$, you know that any subset of rods you keep from the prefix $1...i$ can have size at most $d_i$. If adding rod $i$ pushes your chosen set size to $d_i + 1$, which single rod should you discard so that the remaining $d_i$ rods are both valid and have maximum possible value?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: A subset of rods $S \subseteq \{1, ..., N\}$ can be scheduled without missing any deadline if and only if, when sorted by deadline $d_{i_1} \leq d_{i_2} \leq ... \leq d_{i_k}$, the $r$-th rod satisfies $r \leq d_{i_r}$ for all $1 \leq r \leq k$ (Hall's condition on prefix intervals $[1, d]$).
- **Bottleneck**: A DP over $(i, \text{number of rods chosen so far})$ takes $O(N^2)$ time and memory, which fails for $N = 2 \times 10^5$.

### Step 2 — Tempting Mistakes & Counterexample (Single Matroid vs. Matroid Intersection)

- **Tempting Mistake 1 (Sorting by deadline ascending and taking the first $d_i$ rods)**: Taking early rods blindly fills up slots with small values and blocks huge values that share the same deadline.
- **Tempting Mistake 2 (Applying single-pass weight-descending greedy to the Dual-Grid / Bipartite Matching variant)**: Suppose we have a $2 \times 2$ grid (at most $1$ rod per row and $1$ per column) with 3 rods:
  - Rod 1 at $(r_1, c_1)$ with value $10$.
  - Rod 2 at $(r_1, c_2)$ with value $9$.
  - Rod 3 at $(r_2, c_1)$ with value $9$. Weight-descending greedy picks Rod 1 (value $10$), which occupies **both** $r_1$ and $c_1$, blocking *both* Rod 2 and Rod 3! Greedy gets $10$, whereas $\{\text{Rod } 2, \text{Rod } 3\}$ is valid and gets $9 + 9 = 18$. *Why did the Matroid Exchange Axiom fail here?* Because $A = \{\text{Rod } 1\}$ (size $1$) and $B = \{\text{Rod } 2, \text{Rod } 3\}$ (size $2$) are both valid matchings, yet neither Rod 2 nor Rod 3 can be added to $A$! "At most 1 per row" is one partition matroid $\mathcal{M}_1$, and "at most 1 per column" is a *second* partition matroid $\mathcal{M}_2$; their intersection $\mathcal{M}_1 \cap \mathcal{M}_2$ is **not** a matroid.

### Step 3 — The Turn: Discovering the Structure

In the deadline scheduling problem, there is only **one** family of nested prefix constraints: for every threshold $T$, at most $T$ chosen rods can have $d_i \leq T$. This defines a **Transversal / Slot Matroid**! Instead of using a Disjoint Set Union to find empty time slots backward from $d_i$, we can process rods sorted by **deadline ascending** $d_1 \leq d_2 \leq ... \leq d_N$ and maintain a **Min-Heap** $Q$ of the values of currently chosen rods:

**Chronological Exchange-with-Regret Invariant:**

Sort rods by deadline ascending ($d_1 \leq d_2 \leq ... \leq d_N$). Maintain a min-heap $Q$ of selected values:

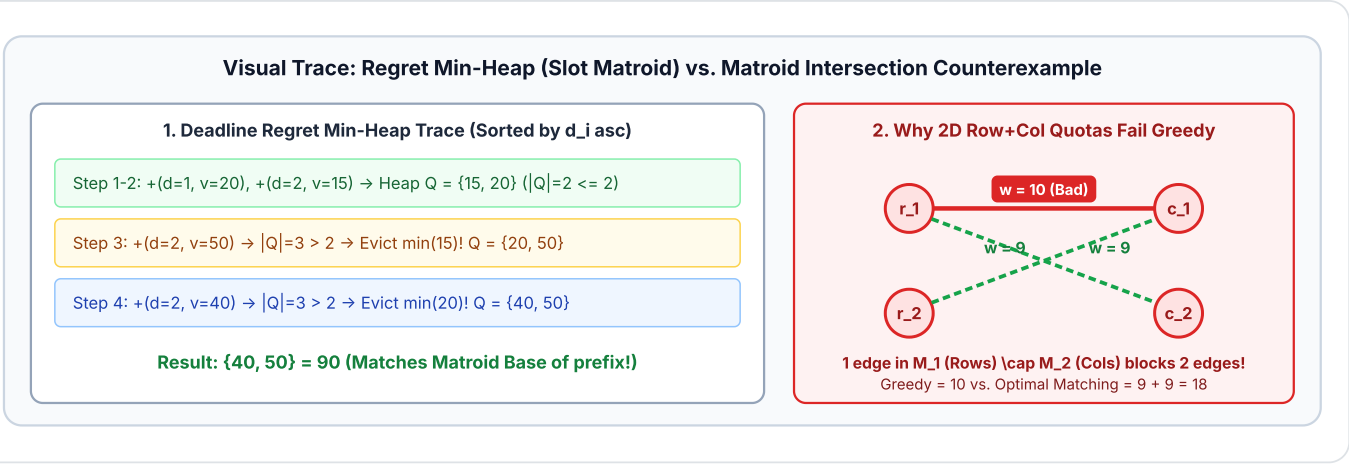
1. Push $v_i$ into $Q$ (tentatively schedule rod $i$ at time $|Q|$).
2. If $|Q| > d_i$ (deadline capacity violated), **evict the minimum element**: `Q.pop()`.

$$\text{Optimal Total Value} = \sum_{v \in Q} v.$$

Why does this regret step never break earlier deadlines? Because every rod currently in $Q$ has deadline $\leq d_i$, and removing the smallest-value rod from $Q$ shifts all subsequent rods in the schedule $1$ slot *earlier*, which only makes their deadlines even easier to satisfy!

### Step 4 — Progressive Construction & Visual Trace

Let us trace 4 rods sorted by deadline: $(d = 1, v = 20)$, $(d = 2, v = 15)$, $(d = 2, v = 50)$, $(d = 2, v = 40)$: 1. Rod 1 $(d = 1, v = 20)$: push $20 \implies Q = \{20\}$, $|Q| = 1 \leq 1$. 2. Rod 2 $(d = 2, v = 15)$: push $15 \implies Q = \{15, 20\}$, $|Q| = 2 \leq 2$. 3. Rod 3 $(d = 2, v = 50)$: push $50 \implies Q = \{15, 20, 50\}$, $|Q| = 3 > d_3 = 2 \implies$ **Regret!** Pop $\min(Q) = 15 \implies Q = \{20, 50\}$. 4. Rod 4 $(d = 2, v = 40)$: push $40 \implies Q = \{20, 40, 50\}$, $|Q| = 3 > d_4 = 2 \implies$ **Regret!** Pop $\min(Q) = 20 \implies Q = \{40, 50\}$. Final optimal schedule: $\{40, 50\}$ with total value $90$.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof of Regret Heap Equivalence to Matroid Greedy**:
  1. *Feasibility*: After step $i$, $|Q| \leq d_i$, and every earlier step $j \leq i$ enforced $|Q_j| \leq d_j$. Since elements are only popped (never added to earlier prefixes), for every threshold $T$, the number of surviving elements in $Q$ with deadline $\leq T$ is at most $T$. By Hall's Marriage Theorem (or earliest-deadline-first scheduling), the elements in $Q$ can always be scheduled without conflict.
  2. *Optimality*: Consider the Transversal Matroid restricted to prefix $\{1, \ldots, i\}$. When element $i$ is added to the optimal base $Q_{i-1}$ of $\{1, \ldots, i-1\}$, if $Q_{i-1} \cup \{i\}$ is dependent, it contains a unique minimal circuit $C$, which consists of all elements in $Q_{i-1} \cup \{i\}$ up to the tight deadline $d_i$ (here, the entire set $Q_{i-1} \cup \{i\}$ since $|Q_{i-1}| = d_i$). Removing the minimum-weight element from the circuit $C$ yields the maximum-weight base of $\{1, \ldots, i\}$!
- **Small-Case Oracle**: For $N \leq 10$, test all $2^N$ subsets, check feasibility by sorting each subset by $d_i$ and verifying $r \leq d_{i_r}$, and compare the maximum subset sum with the min-heap regret algorithm.

---

# Section 57 — Reductions and Modeling

## The Discovery & Modeling Approach

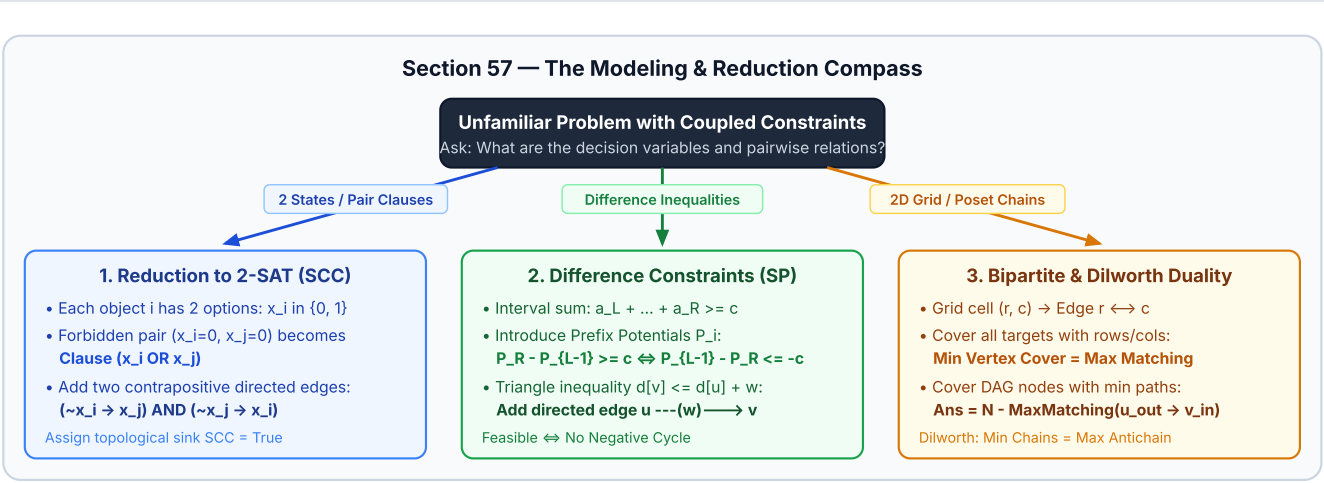
In advanced competitive programming, the hardest problems do not ask for a brand-new algorithm; they disguise a classic polynomial-time structure (2-SAT, Shortest Paths, Bipartite Matching, Min-Cut, or DAG Path Cover) inside an unfamiliar story about grids, schedules, intervals, or logic rules.

**What is a Reduction?** A reduction is a **two-way dictionary** between: 1. **Domain Objects & Rules** (e.g., "every interval $[L, R]$ contains at least $c$ sensors", or "each node picks one of $2$ states and no edge has two forbidden endpoints", or "each cell $(r, c)$ is covered by either its row laser or its column laser"), and 2. **Canonical Graph / Algebraic Primitives** (potentials $P_r - P_{l-1} \geq c$, implication edges $\neg u \to v$, or bipartite vertex cover $r \leftrightarrow c$).

A reduction is valid **if and only if** you prove **both directions** of the equivalence: - **Forward Soundness**: Every valid solution to the original problem maps to a valid structure (satisfying assignment, feasible potential, vertex cover, cut) of the same cost. - **Backward Reconstruction**: Every optimal/valid structure found by the black-box solver can be decoded back into a legal configuration of the original problem.

### Decision Compass & Structural Lenses

| Surface Story / Constraint Shape | Hidden Mathematical Structure | Target Canonical Solver | Core Translation Rule | Complexity |
|---|---|---|---|---|
| **Exactly 2 choices per item + pairwise forbidden pairs** | Boolean CNF where every clause has $\leq 2$ literals $(a \vee b)$ | **2-SAT (Tarjan / Kosaraju SCC)** | Clause $(a \vee b) \Leftrightarrow$ directed edges $(\neg a \to b)$ and $(\neg b \to a)$; satisfiable $\Leftrightarrow \mathrm{scc}(x) \neq \mathrm{scc}(\neg x)$ | $O(N + M)$ |
| **Linear inequalities on variables or interval sums** ($x_v - x_u \leq w$, $\sum_{i=l}^{r} a_i \geq c$) | **System of Difference Constraints** (Dual of Shortest Paths) | **Bellman-Ford / SPFA / 0-1 BFS / Dijkstra** (on prefix potentials) | Inequality $x_v - x_u \leq w \Leftrightarrow$ directed edge $u \xrightarrow{w} v$; feasible $\Leftrightarrow$ no negative cycle | $O(N \cdot M)$ or $O(M \log N)$ |
| **2D Grid cells $(r, c)$ acting on rows & columns** | Incidence between Two Orthogonal Families ($R$ and $C$) | **Bipartite Graph Matching / Kőnig's Vertex Cover** | Row $r \in L$, Col $c \in R$, Cell $(r, c) \Leftrightarrow$ Edge $(r, c)$; Min Cover $=$ Max Matching | $O(E\sqrt{V})$ (Dinic / Hopcroft-Karp) |
| **Partition a Poset / DAG into minimum number of chains** | **Dilworth's Theorem** (Chains vs. Antichains; DAG Path Cover) | **Bipartite Matching on Split Nodes** ($u_{\text{out}} \to v_{\text{in}}$) | Min Chains $= N - \mathrm{MaxMatching}(G_{\text{split}})$ $=$ Max Antichain | $O(E\sqrt{V})$ |
| **Binary partition with item rewards & pairwise conflict penalties** | **Project Selection / Min $s$-$t$ Cut** (Submodular 2-variable energy) | **Max-Flow / Min-Cut (Dinic)** | Source $s = \mathrm{True}$, Sink $t = \mathrm{False}$; penalty for $(u \in S, v \in T)$ is edge $u \xrightarrow{w} v$ | $O(V^2 E)$ or $O(E\sqrt{V})$ |




---

## Worked Problem 57.1: Dual-Polarization Transceiver Array (Reduction to 2-SAT)

### Problem & Constraints

A radio telescope array has $N$ transceivers ($1 \le N \le 2 \times 10^5$). Each transceiver $i$ can be configured in **Horizontal ($H$)** or **Vertical ($V$)** polarization, except that for each transceiver $i$, one of the three modes $\{H, V, X\}$ is hardware-locked out (so each transceiver $i$ actually has **only** $2$ **allowed modes** from $\{H, V, X\}$; in our simplified binary formulation, each transceiver $i$ chooses between two specific frequencies $f_{i,0}$ and $f_{i,1}$). You are given $M$ interference rules ($1 \le M \le 2 \times 10^5$) of three types: 1. **Conflict**: Transceiver $i$ using $f_{i,a}$ and transceiver $j$ using $f_{j,b}$ cannot be active simultaneously. 2. **Coupling**: If transceiver $i$ uses $f_{i,a}$, then transceiver $j$ *must* use $f_{j,b}$. 3. **Mandate**: At least one of (transceiver $i$ uses $f_{i,a}$) or (transceiver $j$ uses $f_{j,b}$) must hold.

Determine in $O(N+M)$ time whether a valid frequency assignment exists for all $N$ transceivers, and if so, output one valid assignment.

> **Pause & Discover:** Every transceiver has a binary choice $x_i \in \{\text{False}, \text{True}\}$. Why does backtracking or greedy assignment fail when a choice has two valid options? How can every pairwise rule between two binary variables be rewritten as a pair of directed implication edges $(p \to q)$ and $(\neg q \to \neg p)$ so that reachability in a directed graph completely characterizes satisfiability?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: Introduce a Boolean variable $x_i \in \{0, 1\}$ for each transceiver $i$, where $x_i = 0$ selects $f_{i,0}$ and $x_i = 1$ selects $f_{i,1}$. Let literal $\ell_{i,1}$ denote $x_i$ and $\ell_{i,0}$ denote $\neg x_i$. Every rule is a disjunction (OR clause) of two literals $(\alpha \vee \beta)$:
  1. Conflict $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$.
  2. Coupling $(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$.
  3. Mandate $(\alpha \vee \beta)$.
- **Bottleneck**: Testing all $2^N$ assignments is exponential. Naive backtracking (`dfs`) that tries $x_i = 0$ and then $x_i = 1$ can explore $O(N)$ vertices before discovering a contradiction, and if it backtracks at multiple branching points without recording global reachability, it degrades to $O(2^N)$ or $O(N(N+M))$.

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Adding only one directed edge per implication, forgetting the contrapositive)**: Suppose a student models the Mandate clause $(x_1 \vee x_2)$ by adding $\neg x_1 \to x_2$ (if $x_1$ is false, $x_2$ must be true) and stops there. Then starting from $\neg x_2$, the graph has no outgoing edge to $x_1$! The implication graph **must** be skew-symmetric: every clause $(\alpha \vee \beta)$ requires **both** $\neg\alpha \to \beta$ and $\neg\beta \to \alpha$.
- **Tempting Mistake 2 (Checking only direct 2-cycles $x_i \leftrightarrow \neg x_i$ or bipartite 2-coloring)**: 2-SAT is **not** undirected 2-coloring! If $x_1 \to x_2$ and $\neg x_1 \to x_2$, both choices of $x_1$ force $x_2 = \text{True}$, which is completely legal (whereas an odd cycle in an undirected graph is illegal). Contradiction occurs if and only if $x_i$ can reach $\neg x_i$ **and** $\neg x_i$ can reach $x_i$ along directed paths!

### Step 3 — The Turn: Discovering the Structure

Construct the **Implication Graph** $G_{\text{imp}}$ with $2N$ vertices: for each $i \in \{1, ..., N\}$, create vertex $2i$ (representing $x_i = \text{True}$) and vertex $2i + 1$ (representing $\neg x_i$, i.e., $x_i = \text{False}$), with negation toggled by bitwise XOR `u ^ 1`.

**2-SAT Implication Graph & Topological Assignment Theorem:**

1. **Clause Translation**: For every clause $(\alpha \vee \beta)$, add two directed edges:

$$\neg\alpha \to \beta \qquad \text{and} \qquad \neg\beta \to \alpha.$$

2. **Satisfiability Criterion**: The formula is satisfiable if and only if for every $i \in \{1, ..., N\}$,

$$\text{scc}(x_i) \neq \text{scc}(\neg x_i).$$

3. **Constructive Assignment**: In the DAG of Strongly Connected Components (ordered topologically from sources to sinks), set:

$$x_i = \text{True} \Leftrightarrow \text{topological_order}(\text{scc}(x_i)) > \text{topological_order}(\text{scc}(\neg x_i))$$

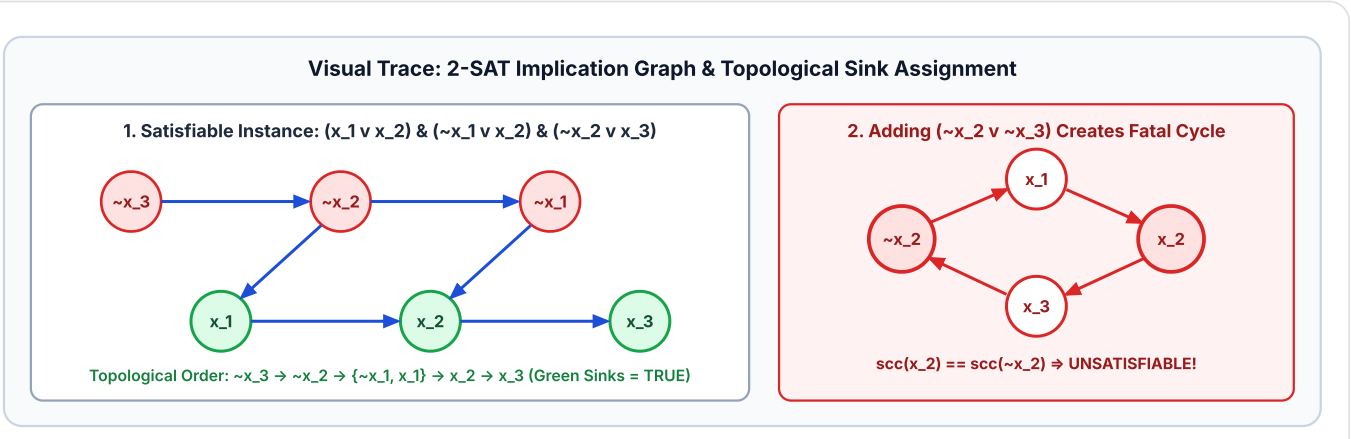
(Note: Tarjan's and Kosaraju's algorithms assign SCC IDs in reverse or forward topological order; in Tarjan's algorithm, smaller SCC ID means closer to a sink, so $x_i = \text{True} \Leftrightarrow \text{id}(x_i) < \text{id}(\neg x_i)$.)

Why must we pick the literal that appears **later** in topological order (closer to the sink)? Because in an implication $P \to Q$, if we set the earlier node $P = \text{True}$, we are forced to set $Q = \text{True}$ (and if $Q = \neg P$, we get $\text{True} \to \text{False}$, a contradiction!). By always choosing the literal that appears **after** its negation in topological order, we never set a literal to $\text{True}$ when it can reach its own negation $\text{False}$!

### Step 4 — Progressive Construction & Visual Trace

Consider $N = 3$ variables with 4 clauses: 1. $(x_1 \vee x_2) \implies (\neg x_1 \to x_2),\ (\neg x_2 \to x_1)$ 2. $(\neg x_1 \vee x_2) \implies (x_1 \to x_2),\ (\neg x_2 \to \neg x_1)$ 3. $(\neg x_2 \vee x_3) \implies (x_2 \to x_3),\ (\neg x_3 \to \neg x_2)$ 4. $(\neg x_2 \vee \neg x_3) \implies (x_2 \to \neg x_3),\ (x_3 \to \neg x_2)$

Let us trace the implication paths in $G_{\text{imp}}$: - From $\neg x_1$: $\neg x_1 \to x_2$. And from $x_1$: $x_1 \to x_2$. So $x_2$ is forced $\text{True}$! - Wait: from $x_2$, clause 3 gives $x_2 \to x_3$, and clause 4 gives $x_2 \to \neg x_3$! And by contrapositive of clause 3, $x_3 \to \neg x_2$, so $x_2 \to x_3 \to \neg x_2$! - Meanwhile, from $\neg x_2$: clause 1 gives $\neg x_2 \to x_1 \to x_2$! - Thus $x_2 \to x_3 \to \neg x_2 \to x_1 \to x_2$ forms a cycle containing both $x_2$ and $\neg x_2$ in the **same SCC** $\implies$ **UNSATISFIABLE**! If instead clause 4 were removed, the topological order would place $\neg x_2 \to x_1 \to x_2 \to x_3$, so $x_2$ and $x_3$ lie in later SCCs than $\neg x_2$ and $\neg x_3$, giving the valid assignment $(x_1, x_2, x_3) = (\text{True}, \text{True}, \text{True})$.




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof**:
  1. *Forward (Satisfiable $\implies \text{scc}(x_i) \neq \text{scc}(\neg x_i)$)*: Every edge $u \to v$ in $G_{\text{imp}}$ is logically equivalent to the material implication $u \Rightarrow v$. By transitivity of $\Rightarrow$, a directed path from $x_i$ to $\neg x_i$ forces $x_i = \text{False}$, and a path from $\neg x_i$ to $x_i$ forces $x_i = \text{True}$. Both cannot hold simultaneously.

2. *Backward ($\text{scc}(x_i) \neq \text{scc}(\neg x_i) \implies$ Topological Rule Satisfies All Clauses)*: By skew-symmetry of $G_{\text{imp}}$, if $A \to B$ is an edge between SCCs, then $\neg B \to \neg A$ is also an edge, so $\text{topo}(x_i) \neq \text{topo}(\neg x_i)$ and every variable gets a well-defined truth value. Suppose for contradiction some edge $u \to v$ has $u = \text{True}$ and $v = \text{False}$. Then $\text{topo}(u) > \text{topo}(\neg u)$ and $\text{topo}(v) < \text{topo}(\neg v)$. But $u \to v$ implies $\text{topo}(u) \leq \text{topo}(v)$ and $\neg v \to \neg u$ implies $\text{topo}(\neg v) \leq \text{topo}(\neg u)$, giving $\text{topo}(u) \leq \text{topo}(v) < \text{topo}(\neg v) \leq \text{topo}(\neg u) < \text{topo}(u)$, a contradiction!

- **Small-Case Oracle**: For $N \leq 15$, test all $2^N$ bitmasks against the $M$ clauses. Verify that the 2-SAT solver reports satisfiable on the exact same instances and that its returned assignment satisfies all $M$ clauses.

---

## Worked Problem 57.2: Linear Accelerator Timing Calibration (System of Difference Constraints)

### Problem & Constraints

A linear particle accelerator has $N$ sequential cavities ($1 \leq N \leq 3000$). Each cavity $i \in \{1, ..., N\}$ is assigned an integer beam boost $a_i \in \{0, 1\}$. You are given $M$ timing calibration specifications ($1 \leq M \leq 5000$) of two kinds on contiguous cavity intervals $[L_k, R_k]$: - **Minimum Output**: The sum of boosts in $[L_k, R_k]$ must be at least $c_k$: $\sum_{i=L_k}^{R_k} a_i \geq c_k$. - **Maximum Output**: The sum of boosts in $[L_k, R_k]$ must be at most $d_k$: $\sum_{i=L_k}^{R_k} a_i \leq d_k$.

Determine whether there exists a sequence $a_1, ..., a_N \in \{0, 1\}$ satisfying all $M$ specifications, and if so, find the **minimum possible total boost** $\sum_{i=1}^{N} a_i$ (or maximum possible total boost).

> **Pause & Discover:** An inequality on a range sum $\sum_{i=L}^{R} a_i \leq d$ couples $R - L + 1$ variables at once. What standard substitution turns every range sum $\sum_{i=L}^{R} a_i$ into a difference of just **two** variables? Once every constraint has the form $P_v - P_u \leq w$, why is that inequality identical to the shortest-path triangle inequality $\text{dist}[v] \leq \text{dist}[u] + w$?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**: Define the **Prefix Sum Variables** $P_0, P_1, ..., P_N$ by $P_0 = 0$ and $P_i = a_1 + a_2 + ... + a_i$. Then every domain rule and every specification becomes a **two-variable difference constraint**:
  1. $a_i \geq 0 \Leftrightarrow P_i - P_{i-1} \geq 0 \Leftrightarrow P_{i-1} - P_i \leq 0$.
  2. $a_i \leq 1 \Leftrightarrow P_i - P_{i-1} \leq 1$.
  3. $\sum_{i=L}^{R} a_i \geq c \Leftrightarrow P_R - P_{L-1} \geq c \Leftrightarrow P_{L-1} - P_R \leq -c$.
  4. $\sum_{i=L}^{R} a_i \leq d \Leftrightarrow P_R - P_{L-1} \leq d$.
- **Bottleneck**: Solving a general Integer Linear Program (ILP) is NP-hard. However, here every inequality has coefficient $+1$ on one variable, $-1$ on one variable, and $0$ everywhere else ($P_v - P_u \leq w$, whose constraint matrix is **totally unimodular**)!

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Forgetting the implicit unit constraints $0 \leq P_i - P_{i-1} \leq 1$)**: If you only add edges for the $M$ interval specifications and omit $P_i - P_{i-1} \leq 1$ and $P_{i-1} - P_i \leq 0$, the solver assigns values where $a_i = P_i - P_{i-1}$ can be $-5$ or $+10$, violating $a_i \in \{0, 1\}$!
- **Tempting Mistake 2 (Confusing Maximum vs. Minimum Potential in Shortest Paths)**: Shortest-path distances $\text{dist}[v]$ satisfy $\text{dist}[v] \leq \text{dist}[u] + w(u, v)$, which gives the **largest** (maximum) possible value of $P_v - P_0$ subject to upper-bound chains $P_v - P_u \leq w$! If instead you want the **minimum** possible value of $P_N - P_0$, you can either:
  - Flip to longest-path form $P_v - P_u \geq w'$ (minimizing upper bounds), or
  - Use the identity $\min(P_N - P_0) = -\max(P_0 - P_N)$, which is simply $-\text{dist}(N \to 0)$ in the standard $\leq$ graph!

### Step 3 — The Turn: Discovering the Structure

In any shortest-path tree from a source $s$, the shortest distances $d[\cdot]$ satisfy the **Bellman-Ford Triangle Inequality** on every directed edge $u \xrightarrow{w} v$:

$$d[v] \leq d[u] + w \Leftrightarrow d[v] - d[u] \leq w.$$

This is a **1-to-1 algebraic isomorphism** between difference constraints and directed weighted edges!

**System of Difference Constraints $\leftrightarrow$ Shortest Path Duality:**
Every linear inequality of the form

$$x_v - x_u \le w \qquad (\text{or equivalently } x_u - x_v \ge -w)$$

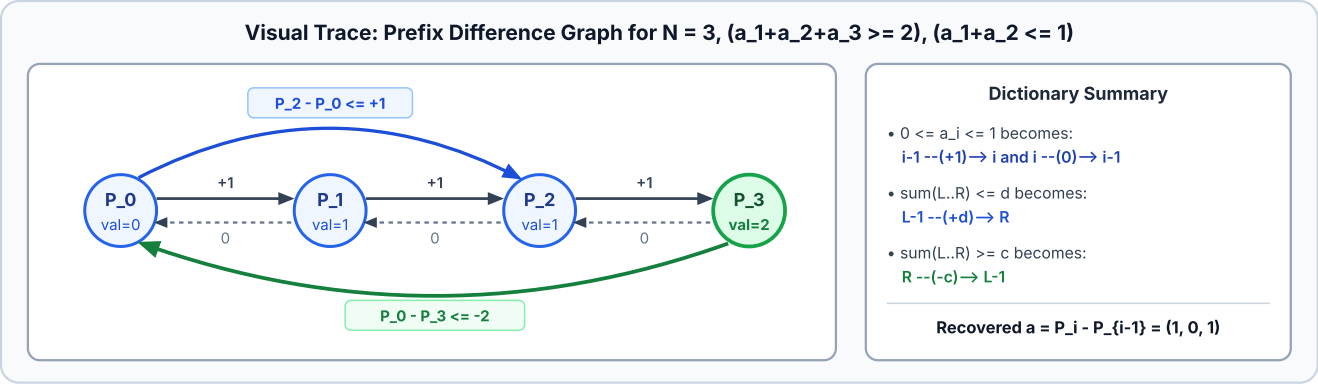
translates to a **directed edge** $u \xrightarrow{w} v$.

- **Feasibility**: A valid integer assignment $x_0, \ldots, x_N$ exists if and only if the directed graph contains **no negative-weight cycle**.
- **Extremal Bounds**: Fixing $x_s = 0$, the shortest path distance $\mathrm{dist}(s \to v)$ equals the **maximum** feasible value of $x_v - x_s$, and $-\mathrm{dist}(v \to s)$ equals the **minimum** feasible value of $x_v - x_s$.
- **Reconstruction**: Setting $x_i = \mathrm{dist}(\text{super_source} \to i)$ (or $x_i = -\mathrm{dist}(i \to N)$) yields an exact integer solution $a_i = x_i - x_{i-1}$.

### Step 4 — Progressive Construction & Visual Trace

Let us trace $N = 3$ cavities with $a_i \in \{0, 1\}$ and two interval rules: - Rule 1: $a_1 + a_2 + a_3 \ge 2 \Leftrightarrow P_0 - P_3 \le -2$ (Edge $3 \xrightarrow{-2} 0$). - Rule 2: $a_1 + a_2 \le 1 \Leftrightarrow P_2 - P_0 \le 1$ (Edge $0 \xrightarrow{+1} 2$). - Implicit unit bounds for $i = 1, 2, 3$: - $P_i - P_{i-1} \le 1 \implies$ Edge $i - 1 \xrightarrow{+1} i$. - $P_{i-1} - P_i \le 0 \implies$ Edge $i \xrightarrow{0} i - 1$.

Let us find the **minimum** possible total boost $P_3 - P_0$: By our duality formula, $\min(P_3 - P_0) = -\mathrm{dist}(3 \to 0)$. In the graph from source $3$: - Edge $3 \xrightarrow{-2} 0$ gives $\mathrm{dist}(3 \to 0) \le -2$. - Can we get an even smaller path from $3$ to $0$? No, all other backward edges $i \to i - 1$ have weight $0$. - Shortest distances from $3$: $d[3] = 0$, $d[0] = -2$, $d[2] = d[0] + 1 = -1$, $d[1] = d[2] + 0 = -1$. - Negating to recover the minimum feasible prefix potentials $P_i = -d[i] - (-d[0])$: $P = (0, 1, 1, 2) \implies a = (1, 0, 1)$, which has $a_1 + a_2 = 1 \le 1$ and $a_1 + a_2 + a_3 = 2 \ge 2$!




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof**:
  1. *Forward (Feasible Sequence $\implies$ No Negative Cycle)*: Suppose a valid sequence $a_1, \ldots, a_N$ exists with prefix sums $P_0, \ldots, P_N$. Along any directed cycle $v_0 \to v_1 \to \ldots \to v_k = v_0$, summing the inequalities $P_{v_j} - P_{v_{j-1}} \le w(v_{j-1}, v_j)$ telescopes the left-hand side to $0$:

$$0 = \sum_{j=1}^{k} \left( P_{v_j} - P_{v_{j-1}} \right) \le \sum_{j=1}^{k} w(v_{j-1}, v_j).$$

     Thus every cycle has weight $\ge 0$.
  2. *Backward (No Negative Cycle $\implies$ Shortest Paths Satisfy All Constraints)*: When no negative cycle exists, shortest-path distances $d[v]$ from a super-source (connected to all vertices with weight $0$) are finite integers and satisfy $d[v] \le d[u] + w(u, v) \Leftrightarrow d[v] - d[u] \le w(u, v)$ for every edge!
- **Small-Case Oracle**: For $N \le 12$, iterate over all $2^N$ binary arrays $a \in \{0, 1\}^N$, check the $M$ interval sums directly, and verify that both feasibility and extremal sum match the Bellman-Ford difference-constraints solver.

---

## Worked Problem 57.3: Wafer Defect Laser Scrubbing & Drone Fleet Routing (Grid-to-Bipartite &

## DAG Dilworth Reductions)

### Problem & Constraints

A semiconductor fabrication plant faces two closely related optimization tasks: 1. **Part A (2D Grid Orthogonal Cover)**: An $R \times C$ silicon wafer ($1 \leq R, C \leq 1000$) has $K$ point defects ($1 \leq K \leq 10^4$) at coordinates $(r_i, c_i)$. Firing a laser along row $r$ scrubs all defects in row $r$; firing along column $c$ scrubs all defects in column $c$. Find the **minimum total number of laser firings** (rows + columns) needed to scrub all $K$ defects. 2. **Part B (Poset / DAG Minimum Fleet Cover)**: Suppose instead that only $N \leq 500$ defects exist, and a single cleaning drone can move from defect $i$ to defect $j$ ($i \neq j$) if and only if $r_i \leq r_j$ and $c_i \leq c_j$ (moving only South and East). What is the **minimum number of drones** needed to visit all $N$ defects, and how does it relate to the maximum number of defects that are pairwise unreachable (strictly North-East of each other)?

> **Pause & Discover:** In Part A, why does turning each row $r$ into a left vertex, each column $c$ into a right vertex, and each defect $(r, c)$ into an edge between them immediately reduce laser scrubbing to Minimum Vertex Cover in a Bipartite Graph? In Part B, why does each drone path that links $k$ defects reduce the number of required drones from $N$ down to $N - (\text{number of links})$, and how do we maximize the number of valid links when paths cannot share internal vertices?

### Step 1 — Exact Model & Bottleneck

- **Exact Model**:
  - *Part A*: Select a subset of rows $S_R \subseteq \{1, ..., R\}$ and columns $S_C \subseteq \{1, ..., C\}$ minimizing $|S_R| + |S_C|$ such that for every defect $(r_i, c_i)$, either $r_i \in S_R$ or $c_i \in S_C$.
  - *Part B*: Partition the $N$ defects in the transitively closed DAG $G_{\text{DAG}}$ (where $i \to j \Leftrightarrow (r_i \leq r_j \wedge c_i \leq c_j \wedge i \neq j)$) into the minimum number of vertex-disjoint directed paths (chains).
- **Bottleneck**: Both problems are NP-hard on general graphs (Minimum Vertex Cover is NP-hard; Partition into Paths / Hamiltonian Path is NP-hard). Greedy selection of the row/column with the most defects fails on simple $3 \times 3$ grids, and greedily taking the longest monotone path in Part B can maroon two points on opposite sides that could have been covered by two balanced paths!

### Step 2 — Tempting Mistakes & Counterexample

- **Tempting Mistake 1 (Greedy Row/Column with Most Defects in Part A)**: Consider defects at $(1, 1), (1, 2), (2, 1), (3, 2), (4, 1)$. Row $1$ has $2$ defects, while rows $2, 3, 4$ have $1$ each. If a tie-breaking greedy picks Row $1$ first, the remaining defects at $(2, 1), (3, 2), (4, 1)$ still span both columns $1$ and $2$, requiring $1 + 2 = 3$ lasers, whereas simply picking **Column 1 and Column 2** scrubs all $5$ defects in $2$ **lasers**!
- **Tempting Mistake 2 (Greedy Longest Path via LIS DP in Part B)**: Consider 4 defects at $(1, 2), (2, 2), (2, 3), (3, 1)$:
  - The longest South-East path has length $3$: $(1, 2) \to (2, 2) \to (2, 3)$, leaving $(3, 1)$ isolated (2 drones total).
  - What if we have 6 points where one diagonal path of length 4 cuts across two optimal paths of length 3 each? Taking the single longest path leaves 2 points that cannot reach each other, requiring $1 + 2 = 3$ drones instead of $2$ drones!

### Step 3 — The Turn: Discovering the Structure

Both problems reduce directly to **Maximum Bipartite Matching**:

1. **Part A (Grid Incidence $\to$ Kőnig's Theorem)**: Build a bipartite graph $G = (L \cup R, E)$ where $L = \{1, ..., R\}$ (rows), $R = \{1, ..., C\}$ (columns), and each defect $(r_i, c_i)$ is an **edge** $(r_i, c_i) \in E$. Choosing a row $r$ or column $c$ to cover every defect edge $(r, c)$ is **by definition** a **Vertex Cover** of $G$! By **Kőnig's Theorem** (1931), in any bipartite graph:
$$\text{Minimum Vertex Cover} = \text{Maximum Bipartite Matching}.$$
2. **Part B (DAG Minimum Disjoint Path Cover & Dilworth's Theorem)**: Start with $N$ trivial paths of $1$ vertex each ($N$ drones). Every time we link the end of one drone's path $u$ to the start of another drone's path $v$ along a DAG edge $u \to v$, the number of paths drops by $1$! A set of directed edges forms a valid collection of vertex-disjoint paths if and only if every vertex $u$ has **out-degree** $\leq 1$ (at most one successor) and **in-degree** $\leq 1$ (at most one predecessor). Split each DAG vertex $u$ into two bipartite nodes: $u_{\text{out}} \in L$ and $u_{\text{in}} \in R$. For each DAG edge $u \to v$, add the bipartite edge $(u_{\text{out}}, v_{\text{in}})$.

**Grid-to-Bipartite (Kőnig) & DAG Path Cover (Dilworth) Reductions:**

1. **Grid Row/Column Cover (Kőnig's Theorem)**:

$$\text{Min Rows + Cols to Cover } E = \text{MaxMatching}(L_{\text{rows}}, R_{\text{cols}}, E_{\text{cells}}).$$

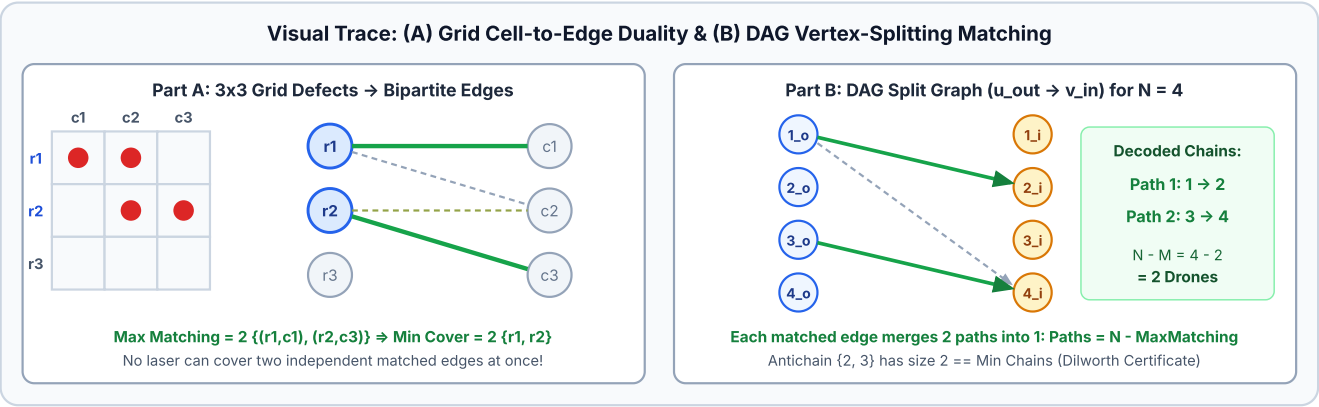
2. **DAG Minimum Vertex-Disjoint Path Cover & Dilworth's Chain Duality**:

$$\text{Min Disjoint Paths} = N - \text{MaxMatching}\left( L_{\text{out}}, R_{\text{in}}, \{(u_{\text{out}}, v_{\text{in}}) \mid u \to v \in E_{\text{DAG}}\} \right).$$

When $E_{\text{DAG}}$ is transitively closed (a Poset), **Dilworth's Theorem** states that the minimum number of chains equals the maximum size of an **antichain** (pairwise unreachable elements).

### Step 4 — Progressive Construction & Visual Trace

Let us trace both reductions side by side:




### Step 5 — Two-Way Proof & Small-Case Oracle

- **Two-Way Proof**:
  1. *Part A (Weak & Strong Duality)*: Let $M$ be any matching and $C$ be any vertex cover. Since every edge in $M$ has at least one endpoint in $C$ and no two edges in $M$ share an endpoint, $|C| \geq |M|$ always holds. Kőnig's theorem constructs a vertex cover of size $|M|$ from the alternating reachability tree of the residual network (taking $L \setminus Z$ and $R \cap Z$ where $Z$ is the set of vertices reachable from unmatched vertices in $L$ via alternating paths).
  2. *Part B (Path-Matching Bijection)*: In a DAG $G$, let $F \subseteq E_{\text{DAG}}$ be a subgraph. $F$ is a union of vertex-disjoint directed paths if and only if $\deg_F^+(u) \leq 1$ and $\deg_F^-(u) \leq 1$ for every vertex $u$ (acyclicity is inherited from the DAG!). In the split bipartite graph, $\deg_F^+(u) \leq 1$ means $u_{\text{out}}$ has degree $\leq 1$, and $\deg_F^-(u) \leq 1$ means $u_{\text{in}}$ has degree $\leq 1$. Thus path covers $F$ of size $k$ paths are in **exact 1-to-1 bijection** with bipartite matchings of size $|F| = N - k$!
- **Small-Case Oracle**:
  - For Part A ($R, C \leq 10$): test all $2^R$ row masks, union the columns of uncovered defects, and verify that $\min(\text{popcount}(\text{mask}) + |\text{uncovered_cols}|)$ equals Hopcroft-Karp's maximum matching size.
  - For Part B ($N \leq 10$): find the maximum independent set in the comparability graph (largest antichain) by brute force over $2^N$ masks and verify that it equals $N - \text{MaxMatching}(G_{\text{split}})$.

---

# Synthesis — A Universal Modeling Checklist

When facing a brand-new problem under contest conditions, run through these five diagnostic questions in order:

| Step | Diagnostic Question | What It Reveals |
|---|---|---|
| **1. Scale & Budget** | What do the constraints ($N \leq 20$, $N \leq 200$, $N \leq 2000$, $N \leq 2 \cdot 10^5$, $N \leq 10^{18}$) permit? | Distinguishes $O(2^N/3^N)$ bitmask/MITM ($N \leq 40$), $O(N^3)$ flows/interval DP ($N \leq 400$), $O(N^2)$ DP ($N \leq 5000$), $O(N \log N)$ sweeps/trees ($N \leq 2 \cdot 10^5$), and $O(\log N)$ matrix/number theory ($N \leq 10^{18}$). |
| **2. Decision Anatomy** | Are we ordering items, choosing a subset, pairing two sides, routing a conserved quantity, or partitioning into binary camps? | Separates exchange-argument sorting, DP/knapsack, bipartite matching, max-flow, and min-cut / 2-SAT reductions. |
| **3. Future Equivalence** | If two partial prefixes reach step $i$, what exact numbers must agree so every suffix continuation behaves identically? | Defines the minimal DP or state-space graph tuple and prevents both state explosion (keeping too much) and Markov bugs (forgetting required state). |
| **4. Algebraic Decoupling** | Can a coupled two-index condition $F(i, j)$ be rewritten by grouping all $i$-terms on one side and all $j$-terms on the other? | Exposes frequency-map lookups ($g(i) = h(j)$), Fenwick/Segment tree dominance ($g(i) \leq h(j)$), Monotonic Queue DP (additive separation), and Convex Hull Trick ($f(i) \cdot g(j)$ bilinear lines). |
| **5. Two-Way Audit** | Can I break my proposed rule with $\leq 4$ elements, and does every optimal model output decode to a legal problem solution? | Catches false greedy heuristics before coding and verifies that $\infty$-edges, virtual hubs, or complement counts cannot leak invalid states into the optimum. |